

算法刷题与 C++ 面试教材

数据结构、算法设计、复杂度证明与工程化题解

从暴力枚举到优化推理，从容器原理到面试表达

面向长期能力建设的完整学习稿

前言

这本书的目标不是堆题解，也不是把学习压缩进某个固定周期，而是把数据结构与算法训练成一套可以长期迁移的工程能力。面试题看似零散，背后其实围绕两条主线：一条是数据结构（data structure），也就是信息怎样存、怎样查、怎样更新；另一条是算法（algorithm），也就是在约束下怎样组织计算、怎样减少重复工作、怎样证明结果正确。数据结构和算法不能分开学。只会说“这题用滑动窗口”，但不知道窗口状态应该放在数组、哈希表还是队列里，代码一定会不稳；只会背 `unordered_map`，但不知道它解决的是历史查询问题，也很难在新题里想到它。

本书按“知识完整性”组织，而不是按倒计时组织。前面的章节先建立复杂度、暴力法、复盘方法和 C++ 代码标准；中间把数组、字符串、链表、栈、队列、哈希表、堆、树、图和并查集讲成数据结构主线；后面再进入双指针、滑动窗口、前缀和、二分、搜索、动态规划、回溯、贪心和综合题。训练周期可以按短期集中训练、学期式推进或长期滚动复习来调整，但正文必须把原理、背景、案例、代码、证明、边界和复盘方法讲完整。

每一类算法都必须从暴力方法开始。暴力方法不是低级答案，而是理解问题空间的方式。两数之和先枚举下标对，才能看出慢点是反复查找补数；最短子数组先枚举所有区间，才能看出正数数组存在窗口单调性；岛屿数量先把每个格子当成节点，才能把题目翻译成图的连通块。优化不是凭感觉，而是来自重复工作、单调性、可维护状态、候选淘汰、最优子结构和数据结构查询能力。

代码使用 C++20。示例代码优先使用 `const` 引用、明确边界、清楚的变量名和可测试函数。书里会解释 `vector`、`string`、`deque`、`stack`、`queue`、`priority_queue`、`unordered_map`、`set`、树、图和并查集的用法与底层复杂度，也会说明迭代器失效、哈希退化、扩容、拷贝、溢出和递归栈这些真实面试和线上评测里会遇到的问题。

读法

读这本书时，不要只看最终代码。每道题至少走四遍：第一遍只读题并写暴力思路；第二遍找慢点和可优化性质；第三遍写 C++ 优化代码；第四遍复盘边界、复杂度和面试表达。

每天训练时，把题目记录成固定格式：题型、输入规模、暴力枚举对象、优化来源、不变量、使用的数据结构、复杂度、错因和一周后是否能独立重写。只写“没想到”没有意义，必须写成具体原因：没有发现单调性、哈希表插入顺序错、窗口收缩条件错、二分边界没有排除 `mid`、`int` 溢出、递归深度过大、容器选择导致复杂度不对。

本书目录只显示部和章，小节不进入目录。章内标题用于串联正文，不把内容切碎成大量编号。你应该把每章当成一条推理链来读，而不是把每个小标题当成孤立知识点。

目录

第一部分	基础方法：从题目到复杂度预算	
第 1 章	刷题方法：从题目到可复用能力	2
第二部分	数据结构：算法能力的骨架	
第 2 章	线性数据结构：数组、字符串、链表、栈和队列	11
第 3 章	非线性数据结构：哈希表、堆、树、图和并查集	33
第三部分	核心算法：从暴力到优化	
第 4 章	数组、双指针和滑动窗口	56
第 5 章	哈希表和前缀和	84
第 6 章	二分查找和答案空间	114
第 7 章	树、图、BFS 和 DFS	122
第 8 章	回溯、枚举和搜索空间剪枝	164
第 9 章	动态规划：状态、转移和重复子问题	176
第四部分	面试专项：从题型到工程表达	
第 10 章	线性结构专项：链表、栈、队列和单调结构	227
第 11 章	排序、选择和区间问题	241
第 12 章	堆、Top K 和贪心证明	254
第 13 章	图论进阶：并查集、最短路和状态图	267
第 14 章	动态规划进阶、容器原理和源码阅读	285
附录 A	力扣刷题手册	307

第零部分

基础方法：从题目到复杂度预算

第 1 章

刷题方法：从题目到可复用能力

刷题和面试训练的核心不是“见过多少题”，而是能否把陌生题拆成熟悉结构。题目只是外壳，真正要识别的是数据怎么组织、状态怎么变化、答案依赖哪些历史信息、是否存在单调性、是否能淘汰候选、是否有重叠子问题。

面对一道题，第一步不是想模板，而是读约束。输入规模决定复杂度上限，值域决定能不能用计数数组或位图，有序性决定能不能双指针或二分，是否允许重复决定哈希表和去重逻辑，是否有负数决定滑动窗口和前缀和的选择。很多错误题解不是算法错，而是约束没看清。

第二步写暴力方法。暴力方法要明确枚举对象：枚举元素、下标对、子数组、子序列、路径、排列、状态，还是答案值。枚举对象一旦明确，复杂度就能估出来。比如两数之和枚举下标对是 $O(n^2)$ ；所有子数组是 $O(n^2)$ ；所有排列是 $O(n!)$ ；网格 BFS 扫所有格子是 $O(mn)$ 。

第三步找重复工作。重复工作有几种典型形态：反复查询某个历史值，适合哈希表；反复求区间和，适合前缀和；反复维护窗口内统计，适合滑动窗口；反复找当前最大或最小，适合堆或单调队列；反复搜索同一状态，适合记忆化或动态规划。

面试表达原则

不要直接说“这题用哈希表”。更好的表达是：暴力会枚举所有下标对，时间是 $O(n^2)$ ；慢点在于每个数都要反复查找另一个数是否存在；我从左到右扫描，用哈希表保存已经看到的值到下标的映射；处理当前值时查 `target - nums[i]` 是否出现过；这样每个元素只插入和查询一次，平均时间 $O(n)$ ，空间 $O(n)$ 。

第四步写不变量。不变量是优化算法正确性的核心。滑动窗口的不变量可能是“窗口内没有重复字符”；单调栈的不变量可能是“栈内下标对应的值单调递减”；BFS 的不变量是“当前层所有节点距离源点相同”；二分的不变量是“答案一定在当前左右边界之间”。没有不变量，代码只是碰巧通过样例。

第五步落到 C++。C++ 代码要讲容器选择。`vector` 适合连续存储和下标访问；`unordered_map` 适合平均常数查询历史信息；`deque` 适合两端弹入弹出；`priority_queue` 适合动态极值但不能删除任意元素；`set` 适合有序前驱后继；图的邻接表通常用 `vector<vector<int>>`，稀疏图比邻接矩阵更节省空间。

仍然以两数之和为例。题目问的是两个不同下标的数能否凑成目标值，最直接的想法不是哈希表，而是枚举下标对。先固定左端 `left`，再枚举它右边的 `right`，检查二者之和是否等于 `target`。这段暴力代码覆盖了全部候选，因为任意合法答案都是某一对 `left < right` 的下标；它也天然避免同一个下标被用两次。

图示：两数之和从暴力到哈希

暴力枚举	固定 <code>left</code> ，扫描 <code>right</code> ，逐对检查 <code>nums[left] + nums[right]</code> 。
重复工作	每个当前位置都在历史元素里重新寻找补数，历史查询没有被保存。
哈希状态	<code>value_to_index</code> 保存已经见过的“值 -> 下标”，只表示当前下标之前的历史。
循环顺序	先查 <code>target - nums[index]</code> ，再插入 <code>nums[index]</code> ，保证不会使用同一个下标两次。

Listing 1.1 两数之和：先写完整暴力基线

```
1 std::vector<int> two_sum_bruteforce(const std::vector<int>& nums, int target)
```

```

2 {
3     for (int left = 0; left < static_cast<int>(nums.size()); ++left) {
4         for (int right = left + 1; right < static_cast<int>(nums.size()); ++right) {
5             const long long sum =
6                 static_cast<long long>(nums[left]) + nums[right];
7             if (sum == target) {
8                 return {left, right};
9             }
10        }
11    }
12    return {};
13 }

```

暴力版本慢在“查找补数”这一步。固定当前数 `nums[right]` 时，我们真正需要知道的是此前是否出现过 `target - nums[right]`，不需要把所有历史下标重新扫一遍。于是优化不是凭空说“用哈希表”，而是把历史值组织成值到下标的映射：扫描到当前下标时，先查询补数是否已经在历史表中；如果在，答案就是历史下标和当前下标；如果不在，再把当前值放进历史表，供后面的元素查询。

Listing 1.2 两数之和：哈希表把补数查找变成历史查询

```

1 std::vector<int> two_sum_hash_table(const std::vector<int>& nums, int target)
2 {
3     std::unordered_map<int, int> value_to_index;
4     value_to_index.reserve(nums.size());
5
6     for (int index = 0; index < static_cast<int>(nums.size()); ++index) {
7         const int need = target - nums[index];
8         const auto found = value_to_index.find(need);
9         if (found != value_to_index.end()) {
10            return {found->second, index};
11        }
12        value_to_index.emplace(nums[index], index);
13    }
14    return {};
15 }

```

这段优化代码的不变量是：处理下标 `index` 前，哈希表只保存 `[0, index)` 中已经出现过的值。先查再插入，正是为了保持“历史”这个语义，避免当前元素和自己配对。参数用 `const std::vector<int>&`，避免复制输入数组。`reserve` 不是必须，但能减少扩容和 rehash。返回空数组表示没有答案，真实力扣题如果保证有答案，也可以直接返回。这样讲完以后，读者能看到完整链条：枚举下标对可以保证正确，瓶颈是重复扫描历史，哈希表只负责把历史补数查询降到平均常数。

训练路线：从基础题到综合能力

学习路线不是倒计时。有人一个月集中准备，有人一边工作一边长期训练，也有人先补 C++ 和数据结构基础，再进入高强度题目练习。因此本章只给出能力阶段和训练节奏，不把整本书绑定到固定月份。你可以把每个阶段压缩成一周，也可以展开成一个月；真正不能省略的是每个阶段要掌握的知识、题型、代码能力和复盘方法。

第一阶段建立基础语言和复杂度意识。目标是能读懂输入规模，估计可接受复杂度，写出暴力解，并说清楚暴力解慢在哪里。这个阶段学习数组、字符串、`vector`、`string`、排序、二分库函数、下标边界、引用传参和整数溢出。推荐题包括两数之和、移动零、删除有序数组重复项、合并两个

有序数组、最大子数组和、反转字符串和有效回文。做题时不要急着背模板，先把枚举对象写清楚：枚举元素、下标对、区间，还是答案边界。

第二阶段学习线性结构和窗口类算法。双指针、滑动窗口、前缀和、单调栈和单调队列看起来像技巧，本质都是“线性序列上的状态维护”。窗口题要写出左右边界的含义，说明什么时候扩张、什么时候收缩；单调结构题要说明哪些候选被淘汰，为什么淘汰后不会丢答案。推荐题包括盛最多水的容器、三数之和、长度最小的子数组、无重复字符的最长子串、找到字符串中所有字母异位词、最小覆盖子串、每日温度、柱状图最大矩形和滑动窗口最大值。

第三阶段把哈希、堆、树和图串起来。哈希表负责历史查询，堆负责动态极值，树负责层次和有序关系，图负责任意状态转移。这个阶段要形成建模习惯：节点是什么，边是什么，状态保存在哪里，访问标记何时更新，答案在入队、出队、入栈、出栈还是回溯时产生。推荐题包括字母异位词分组、最长连续序列、和为 K 的子数组、前 K 个高频元素、数组中的第 K 个最大元素、二叉树层序遍历、验证二叉搜索树、最近公共祖先、岛屿数量、腐烂的橘子、课程表和课程表 II。

第四阶段进入搜索、回溯和动态规划。回溯题要写清选择列表、路径状态、终止条件和撤销动作；动态规划题要写清状态定义、转移来源、初始化、遍历顺序和空间优化条件。不要先背方程，先问一个状态是否足够描述未来。推荐题包括全排列、子集、组合总和、括号生成、单词搜索、爬楼梯、打家劫舍、零钱兑换、最长递增子序列、最长公共子序列、编辑距离、分割等和子集和目标。

第五阶段处理贪心、并查集和综合题。贪心题的关键是证明局部选择为什么能推出全局最优；并查集题的关键是把动态连通关系建模成集合合并；综合题则常常把哈希、排序、堆、图和 DP 混在一起。推荐题包括跳跃游戏、加油站、合并区间、冗余连接、账户合并、实现 Trie、网络连接、单词接龙和数据流中位数。这个阶段不要只追求 AC，要训练面试表达：暴力解是什么，瓶颈在哪里，优化依赖哪条性质，代码如何保证边界正确。

如果需要集中训练，可以把这些阶段压成十二周：基础复杂度与数组字符串，双指针与窗口，哈希与前缀和，栈队列堆与单调结构，二分查找，链表与树，图和拓扑排序，回溯，一维动态规划，二维动态规划和背包，贪心与并查集，综合模拟与查漏补缺。这只是节奏表，不是本书的边界。书的目标是把知识完整讲清楚，训练周期由你的基础、时间和目标岗位决定。

阶段交付

每个阶段至少留下四类材料：一组代表题的暴力解和优化解，一份数据结构选择说明，一份 C++ 容器和边界错误清单，一份一周后能独立重写的回访题表。刷题报告不是形式，它是把题解转化成长期能力的工具。

学习方法和题解复盘的深层要求

刷题教材不能只给最终代码。读者真正需要的是从题目约束到算法选择的推理链：输入规模允许什么复杂度，数据是否有序，值域能否压缩，状态是否会重复，局部选择能否证明，容器操作成本是否符合题目热路径。每道题都要先写暴力解，因为暴力解暴露了最朴素的状态空间；然后再指出哪个查询、哪个枚举、哪个重复子问题最贵。优化不是把模板贴上去，而是把最贵的那一步替换为更合适的数据结构或更强的不变量。

高质量复盘至少包含三层。第一层是题面复述，用自己的话说明输入、输出和约束，尤其要把隐藏条件翻译成算法语言，例如正数数组带来窗口单调性，排序数组带来二分和双指针，树节点唯一性决定能否用值建索引。第二层是复杂度预算，把 n 、 m 、值域、字符串长度、图边数分开，不要只写一个 n 。第三层是正确性说明，用不变量、交换论证、归纳或最短路层次证明解释为什么不会漏答案。

题解里的代码要服务于解释，而不是炫技。变量名应该表达状态含义，例如 `left` 表示窗口左边界，`farthest` 表示当前可达最远位置，`indegree` 表示尚未满足的依赖数量。复杂代码要先拆出检查函数、松弛函数或合并函数。这样做不是为了形式，而是让读者能在面试中逐句讲清楚代码对应的数学状态。代码写完以后，至少用一个最小用例、一个边界用例、一个反例用例和一个规模用例手算。

三个月计划只能作为节奏，不应该成为内容边界。真正的学习路径是循环式的：第一轮建立题型地图，第二轮补齐证明和边界，第三轮把相似题横向比较，第四轮把代码改成工程上也能接受的版本。每一轮都要记录失败原因。失败原因不是笼统的不会，而是要拆成读错题、复杂度预算错误、状态定义不完整、边界初始化错误、容器 API 误用、证明缺失或代码实现失误。

面试表达要像讲工程方案。先说暴力方案，说明它为什么慢；再说关键观察，说明它让哪一步

变便宜；然后给出数据结构职责，说明保存什么、查询什么、删除什么；最后给出不变量和复杂度。这个顺序能防止答案变成模板堆砌。面试官追问时，优先回到约束和不变量，而不是机械背另一段代码。

实验是把知识落地的关键。数组题打印指针区间，窗口题打印计数表和 formed，二分题打印 left、mid、right 和谓词结果，图题打印队列层次，DP 题打印小表格，堆题打印堆顶和过期状态。只要一次实验能解释一个常见错误，它就不是额外负担，而是把题解变成长期能力的工具。

专题复盘要求

读完本节后，不要只记结论。请把每个结论还原成一个可验证的问题：它解决了哪一步重复工作，依赖哪个题目条件，使用哪个容器或状态，边界条件如何破坏错误写法。

复杂度、暴力法和优化来源

复杂度是算法设计的预算。输入规模如果是 $n \leq 100$ ， $O(n^3)$ 可能可以接受；如果是 $n \leq 2 \times 10^5$ ，通常要接近 $O(n \log n)$ 或 $O(n)$ ；如果网格规模是 $m * n \leq 10^5$ ，BFS 或 DFS 扫一遍通常合理；如果状态空间是指数级，就要考虑剪枝、记忆化或动态规划。

暴力法的作用是把问题完整展开。两数之和暴力枚举所有下标对，最长无重复子串暴力枚举所有起点和终点，岛屿数量暴力从每个未访问陆地扩展连通块。暴力方法慢，但它通常是正确性的起点。

Listing 1.3 两数之和暴力解

```
1 std::vector<int> two_sum_bruteforce(const std::vector<int>& nums, int target)
2 {
3     for (int left = 0; left < static_cast<int>(nums.size()); ++left) {
4         for (int right = left + 1; right < static_cast<int>(nums.size()); ++right) {
5             if (nums[left] + nums[right] == target) {
6                 return {left, right};
7             }
8         }
9     }
10    return {};
11 }
```

这段代码枚举的是下标对，数量约为 $n * (n - 1) / 2$ ，时间复杂度是 $O(n^2)$ 。它慢在每个数都要重新扫描后面的数。优化思路不是凭空出现，而是把“扫描后面所有数”改成“快速查询某个补数是否出现过”。这就是哈希表。

常见优化来源有六类。第一类是缓存重复计算，例如前缀和、记忆化搜索和动态规划。第二类是单调性，例如二分、双指针和某些滑动窗口。第三类是局部状态可维护，例如字符计数、窗口和、不同元素个数。第四类是候选淘汰，例如单调栈和单调队列。第五类是数据结构查询能力，例如哈希表、堆、平衡树、并查集。第六类是改变建模方式，例如把网格问题建成图，把区间问题排序，把字符串问题转为频次。

写完暴力法后，应该固定问几个问题：我枚举的对象是什么？有没有重复计算同一段信息？是否只需要历史信息的一小部分？是否存在有序或单调？是否可以把区间查询变成前缀差？是否可以用哈希把线性查询变成平均常数？是否可以把指数搜索合并成状态 DP？

面试时要把这些问题讲出来。一个好答案通常不是从“我用某某模板”开始，而是从暴力法、瓶颈和优化依据开始。

复杂度分析首先要分清“输入长度”和“输入数值”。数组长度为 n 时，扫描一遍是 $O(n)$ ；背包容量为 $target$ 时， $O(n \cdot target)$ 并不等于严格意义上的多项式时间，因为 $target$ 是数值大小，不是它在输入中占用的二进制位数。刷题中我们通常接受这种伪多项式复杂度，因为题目约束直接给出了容量上限；但面试表达里要知道它和 $O(n^2)$ 这类只依赖元素个数的复杂度不同。分割等和子集、零钱兑换、目标和都属于这种情况：如果数值总和很大，即使数组长度不大，DP 表也可能开不下。

复杂度还要看隐藏常数和内存访问。两个算法同为 $O(n)$ ，一个顺序扫描 `vector`，另一个沿链表随机跳转，实际速度可能差很多。原因在于 CPU 缓存和内存局部性。顺序数组能让硬件预取连续内存，链表每个节点可能分散在堆上，访问下一个节点前必须等当前节点指针加载完成。这不改变大 O 记号，但会影响工程判断。刷题时大 O 是第一道门槛；写可维护的 C++ 程序时，还要知道容器布局、分配次数和临时对象成本。

深入理解

大 O 记号描述的是增长趋势，不是精确运行时间。它会忽略常数、低阶项和机器细节，所以不能用它解释所有性能差异。比如 `std::sort` 的 $O(n \log n)$ 在中等规模输入上可能比一个写得很差的 $O(n)$ 哈希方案更快，因为排序是连续内存扫描，哈希表可能频繁分配、冲突和 rehash。反过来，当输入规模足够大，数量级差距会压倒常数。高质量答案要同时具备两层意识：先用复杂度判断能不能过，再用实现成本判断写法是否稳。

暴力法不是低级答案，而是问题建模的显微镜。写暴力时，你会被迫明确“枚举对象”是什么：枚举子数组、枚举切分点、枚举路径、枚举排列、枚举状态，还是枚举答案。不同枚举对象决定了后续优化方向。枚举所有子数组时，优化往往来自前缀和或滑动窗口；枚举所有路径时，优化可能来自 BFS、Dijkstra、记忆化或剪枝；枚举所有排列时，优化可能来自排序去重、位掩码状态压缩或贪心证明；枚举答案时，若可行性单调，就可能二分答案。

以“和为 K 的子数组”为例，最朴素暴力枚举左右端点后再求和，三层成本是 $O(n^3)$ 。第一步优化是把求和缓存成前缀和，左右端点枚举仍然存在，但区间和查询变成 $O(1)$ ，总复杂度降到 $O(n^2)$ 。第二步优化是换一个枚举对象：不再枚举所有左端点，而是在扫描右端点时，查询历史中有多少前缀和等于当前前缀减 k 。这一步把“寻找左端点”交给哈希表，复杂度降到平均 $O(n)$ 。

Listing 1.4 和为 K 的子数组：从前缀和到哈希计数

```
1 int subarray_sum_equals_k(const std::vector<int>& nums, int k)
2 {
3     std::unordered_map<int, int> prefix_count;
4     prefix_count.reserve(nums.size() * 2);
5     prefix_count[0] = 1;
6
7     int prefix = 0;
8     int answer = 0;
9     for (const int x : nums) {
10         prefix += x;
11         const auto found = prefix_count.find(prefix - k);
12         if (found != prefix_count.end()) {
13             answer += found->second;
14         }
15         ++prefix_count[prefix];
16     }
17     return answer;
18 }
```

这段代码背后的数学关系是：若 `prefix[right] - prefix[left] == k`，那么 `prefix[left] == prefix[right] - k`。扫描到当前右端时，所有可能的左端都在历史前缀里，所以查询历史计数即可。必须先查再更新当前前缀，否则当 `k == 0` 时会把长度为 0 的区间错误计入。这个例子说明，优化不是把代码写短，而是把问题从“枚举区间”改写成“查询历史状态”。

再看“长度最小的子数组”。如果数组全是正数，滑动窗口能成立，因为右端右移只会让窗口和增加，左端右移只会让窗口和减少。这个单调性让每个元素最多进入窗口一次、离开窗口一次，总复杂度 $O(n)$ 。如果数组允许负数，单调性消失：右端加入负数可能让和变小，左端删除负数可能让和变大。此时同样的窗口代码会漏解。一个成熟的答案必须说明算法成立的输入条件，而不是看到

“子数组”就写窗口。

常见误区

很多错误来自把“题型标签”当作证明。看到连续子数组就写滑动窗口，看到最短就写 BFS，看到最优就写 DP，看到局部最大就写贪心，都是危险的。真正的问题是：窗口是否有单调性？边权是否相等或非负？状态是否无后效？局部选择是否有交换论证？如果这些条件不存在，模板就会变成错误答案。

摊还复杂度也是刷题必须掌握的工具。单调栈和单调队列里，某一次循环可能弹出很多元素，看起来像嵌套循环，似乎是 $O(n^2)$ 。但每个元素一生只会入栈一次、出栈一次，所以所有弹出操作加起来最多 n 次，总复杂度是 $O(n)$ 。摊还分析不是平均随机情况，而是把一组操作的总成本平摊到每个元素上。`vector::push_back` 也常用摊还分析：多数插入是常数时间，偶尔扩容搬移很多元素；如果容量按倍数增长，总搬移次数仍然是线性级别。

Listing 1.5 单调栈摊还分析的代码形态

```
1 std::vector<int> next_greater_elements(const std::vector<int>& nums)
2 {
3     std::vector<int> answer(nums.size(), -1);
4     std::vector<int> stack;
5     stack.reserve(nums.size());
6
7     for (int i = 0; i < static_cast<int>(nums.size()); ++i) {
8         while (!stack.empty() && nums[i] > nums[stack.back()]) {
9             const int index = stack.back();
10            stack.pop_back();
11            answer[index] = nums[i];
12        }
13        stack.push_back(i);
14    }
15
16    return answer;
17 }
```

内层 `while` 不是复杂度灾难，因为被弹出的下标不会再次入栈。面试时可以直接说：每个下标至多被压入一次、弹出一次，因此所有 `while` 循环的总迭代次数是 $O(n)$ 。这种解释比简单写“单调栈 $O(n)$ ”更有说服力。

递归搜索的复杂度要看搜索树。全排列的叶子数是 $n!$ ，子集的叶子数是 2^n ，组合总和的搜索树取决于目标值和候选数。剪枝能减少实际访问节点，但通常不改变最坏复杂度，除非剪枝来自强约束或状态合并。记忆化搜索的关键是把搜索树中的重复节点合并成状态图。动态规划表的大小，就是不同状态的数量；每个状态的转移成本乘上状态数，就是总复杂度。

以爬楼梯为例，暴力递归 $f(n) = f(n-1) + f(n-2)$ 会重复计算大量子问题。 $f(5)$ 需要 $f(4)$ 和 $f(3)$ ，而 $f(4)$ 又需要 $f(3)$ ，同一个 $f(3)$ 被反复计算。记忆化把每个 $f(i)$ 只算一次，复杂度从指数级降到 $O(n)$ 。表格 DP 只是把“递归加缓存”改成按顺序填表。二者思想相同，表达形式不同。

Listing 1.6 爬楼梯：从重复递归到滚动状态

```
1 int climb_stairs(int n)
2 {
3     if (n <= 2) {
4         return n;
5     }
6 }
```

```

7   int previous_two = 1;
8   int previous_one = 2;
9   for (int step = 3; step <= n; ++step) {
10      const int current = previous_one + previous_two;
11      previous_two = previous_one;
12      previous_one = current;
13  }
14  return previous_one;
15 }

```

这里的状态依赖只有前两个值，所以不需要保存整个数组。空间优化的前提是你已经知道未来不会再访问更早状态。背包 DP 的一维滚动数组也是这个道理，但它更容易错，因为遍历方向会决定当前物品是否被重复使用。空间优化不是机械删除一维，而是重新检查转移读取的是上一阶段还是当前阶段。

二分答案的复杂度由“答案范围”和“检查函数”共同决定。爱吃香蕉的速度范围是 $[1, \text{maxPile}]$ ，每次检查扫描所有堆，因此复杂度是 $O(n \log M)$ 。分割数组最大值的答案范围是 $[\text{maxElement}, \text{sum}]$ ，检查函数贪心扫描数组，因此复杂度是 $O(n \log S)$ 。如果检查函数本身是 $O(n \log n)$ ，总复杂度就会多一个因子。分析二分题时，不要只写 $O(\log n)$ ，因为很多二分不是在数组长度上二分。

Listing 1.7 答案二分：容量越大越容易可行

```

1  int ship_within_days(const std::vector<int>& weights, int days)
2  {
3      int left = *std::max_element(weights.begin(), weights.end());
4      int right = std::accumulate(weights.begin(), weights.end(), 0);
5
6      auto can_ship = [&](int capacity) {
7          int used_days = 1;
8          int current_load = 0;
9          for (const int weight : weights) {
10             if (current_load + weight > capacity) {
11                 ++used_days;
12                 current_load = 0;
13             }
14             current_load += weight;
15         }
16         return used_days <= days;
17     };
18
19     while (left < right) {
20         const int mid = left + (right - left) / 2;
21         if (can_ship(mid)) {
22             right = mid;
23         } else {
24             left = mid + 1;
25         }
26     }
27     return left;
28 }

```

这个题的检查函数也需要证明。给定容量上限时，按顺序尽量把包裹装到当天，装不下再开新一天，是最少天数策略。因为提前开新一天不会让当前天装得更多，只会浪费容量。容量越大，所需

天数不会增加，所以可行性具有单调性。二分答案必须同时具备这两个条件：检查函数正确，答案可行性单调。

空间复杂度也不能只看“额外数组”。递归调用栈是空间，哈希表桶和节点是空间，输出答案是否计入空间要按题目约定说明。比如三数之和，如果不计输出，排序后的双指针额外空间可以说是 $O(1)$ 或排序栈空间；如果计输出，最坏可能有 $O(n^2)$ 个三元组。BFS 的队列空间不是 $O(1)$ ，它可能达到图的最大宽度；二叉树层序遍历的队列空间是最大层宽。

算法优化还要避免过度优化。对于输入 $n \leq 100$ 的题，清晰的 $O(n^3)$ DP 可能比复杂的优化更适合面试；对于 $n \leq 2 \times 10^5$ 的题， $O(n^2)$ 通常不可接受，必须寻找排序、堆、哈希、线段树或单调结构。判断一段代码能否通过，先做粗略预算：一秒级评测里，一千万到一亿级简单操作可能可接受，十亿级通常危险。这个数字不是绝对标准，但能帮助你快速排除明显不合适的方案。

复杂度实验：第一，把两数之和的暴力版和哈希版分别跑在一万、十万规模随机数组上，记录时间差。第二，把和为 K 的子数组写成三层、前缀和两层、哈希一层三个版本，观察复杂度如何影响增长曲线。第三，用 `vector` 和 `list` 分别顺序遍历一百万个整数，比较实际时间，理解缓存局部性。第四，用单调栈统计每个元素入栈出栈次数，验证摊还 $O(n)$ 。

复杂度推理的面试表达

先给暴力法，让问题完整展开；再指出慢点来自重复计算、重复扫描、候选过多还是状态爆炸；然后说明优化依据是缓存、单调性、候选淘汰、数据结构查询能力还是重新建模；最后给出时间、空间、成立条件和反例。能完整说出这条链，比直接背最优解更有价值。

第零部分

数据结构：算法能力的骨架

第 2 章

线性数据结构：数组、字符串、链表、栈和队列

数据结构不是算法之前的一张 API 表，而是算法能够成立的物理基础。刷题时遇到一个问题，第一步不是急着套模板，而是判断答案依赖什么信息：需要按位置访问、按最近关系回退、按先进先出扩展，还是只在两端维护候选。数组、字符串、链表、栈、队列和双端队列构成了这条主线。它们的抽象接口很简单，但背后的存储方式、缓存局部性、迭代器失效规则和边界条件，决定了暴力方法能不能被优化成面试可接受的复杂度。

线性结构的共同特点是元素被组织成一条序列。区别在于这条序列是否连续存储，是否支持下标随机访问，插入删除发生在什么位置，算法状态如何随着左右边界移动而更新。真正掌握这一章，意味着看到题目时可以回答四个问题：序列是否固定，窗口是否连续，历史状态能否增量维护，删除或回退是否只发生在某一端。

从能力边界看，`vector` 负责连续存储、随机访问和顺序扫描，是 DP 表、排序数组、邻接表和双指针题的默认选择；`string` 负责字符序列和子串窗口，但要警惕 `substr` 的复制成本；链表负责节点重连，适合反转、合并、环检测和快慢指针；`stack` 负责最近未解决状态，常用于括号、表达式、路径和单调栈；`queue` 负责按层扩展，是 BFS 的自然结构；`deque` 负责两端维护候选，尤其适合滑动窗口极值。把这些结构先放在同一张能力地图里，后面每个算法就不会变成孤立模板。

数组和 `std::vector` 是刷题默认容器。连续存储带来两个关键优势：下标访问是 $O(1)$ ，相邻元素通常落在相近内存位置，CPU 缓存命中率高。很多初学者只记住第一个优势，忽略第二个优势。实际程序中，同样是线性扫描，`vector` 往往比链表快得多，并不是因为复杂度写法不同，而是因为连续内存更容易被硬件预取。代价也来自连续存储：中间插入和删除必须移动后续元素；容量不足时扩容会重新申请一段更大的空间，把旧元素移动或复制过去，原来的指针、引用和迭代器都可能失效。

Listing 2.1 `vector` 的容量、大小和失效规则

```
1 std::vector<int> nums;
2 nums.reserve(n);           // 只预留容量，不构造元素，size 仍然是 0。
3 nums.push_back(10);
4 nums.emplace_back(20);
5
6 std::vector<int> dp(n + 1, 0); // 直接创建 n + 1 个元素。
7
8 const int* first = nums.data();
9 nums.push_back(30);         // 如果触发扩容，first 立刻失效。
```

`reserve` 和 `resize` 必须分清。`reserve` 只改变容量，不构造元素，适合已经知道将要追加多少元素的场景；`resize` 改变大小，会构造或销毁元素，适合直接创建 DP 数组、计数数组或矩阵行。写 `vector<int> dp(n + 1)` 后，`dp[i]` 是合法的；只写 `reserve(n + 1)` 后访问 `dp[i]` 是越界。面试里这类错误很隐蔽，因为代码可能在小样例上运行，到了评测环境才崩溃。

数组题的暴力方法通常是枚举一个区间、一个配对或一个分割点。优化的关键，是让“下一步答案”复用“上一步状态”。例如区间和暴力做法是每次重新累加 `nums[left..right]`，复杂度 $O(n^3)$ 或 $O(n^2)$ ；前缀和把任意区间和转成两个端点差值，使查询变成 $O(1)$ 。滑动窗口则在区间右端加入一个元素、左端删除一个元素，用增量更新维护窗口状态。它们都依赖数组按下标稳定访问。

Listing 2.2 从重新求和到前缀和查询

```
1 std::vector<long long> build_prefix_sum(const std::vector<int>& nums)
2 {
```



```

3     std::vector<long long> prefix(nums.size() + 1, 0);
4     for (std::size_t i = 0; i < nums.size(); ++i) {
5         prefix[i + 1] = prefix[i] + nums[i];
6     }
7     return prefix;
8 }
9
10 long long range_sum(const std::vector<long long>& prefix,
11                     std::size_t left,
12                     std::size_t right)
13 {
14     return prefix[right + 1] - prefix[left];
15 }

```

这段代码里 `prefix[i]` 表示前 `i` 个元素之和，因此区间 `[left, right]` 的和是 `prefix[right + 1] - prefix[left]`。多出来的第 0 位看似绕，但它消除了 `left == 0` 的特殊判断。很多高质量题解都会主动设计这种哨兵位，因为它能让边界条件服从同一条公式。

字符串 `std::string` 可以看成字符数组，但不能只按字符数组理解。第一，`substr` 会创建新字符串，频繁调用会引入大量复制；第二，`size()` 返回字节数，不等于所有自然语言里的“字符数”；第三，题目如果限定小写英文，计数结构可以用 `std::array<int, 26>`，如果字符集不固定，就应使用哈希表。刷题常见输入大多是 ASCII 或小写英文，面试时仍然要主动说明自己的假设。

Listing 2.3 异位词窗口：避免反复 `substr`

```

1 std::vector<int> find_anagrams(const std::string& s, const std::string& p)
2 {
3     constexpr int ALPHABET_SIZE = 26;
4     std::vector<int> answer;
5     if (s.size() < p.size()) {
6         return answer;
7     }
8
9     std::array<int, ALPHABET_SIZE> need{};
10    std::array<int, ALPHABET_SIZE> window{};
11    for (const char ch : p) {
12        ++need[ch - 'a'];
13    }
14
15    for (std::size_t right = 0; right < s.size(); ++right) {
16        ++window[s[right] - 'a'];
17        if (right >= p.size()) {
18            --window[s[right - p.size()] - 'a'];
19        }
20        if (window == need) {
21            answer.push_back(static_cast<int>(right + 1 - p.size()));
22        }
23    }
24    return answer;
25 }

```

暴力版本会枚举每个长度为 `p.size()` 的子串，复制出来再排序或统计。复制子串本身就是 $O(k)$ ，排序还要 $O(k \log k)$ 。窗口版本只维护 26 个计数，右端进入一个字符，左端离开一个字符，每步常数更新。优化成立的原因不是“用了滑动窗口”这个名词，而是题目只关心固定长度窗口内的

字符频次，窗口相邻状态只差两个字符。

链表是线性结构里最容易被误解的一个。教材常说链表插入删除是 $O(1)$ ，但这句话有前提：你已经拿到了要操作位置的前驱节点或当前节点。如果还需要从头查找位置，查找本身就是 $O(n)$ 。链表的价值不在于比数组更快，而在于节点位置稳定、局部重连便宜、结构可以自然表达“下一个”。面试链表题考的不是容器 API，而是指针不丢、边界不漏、连接顺序清楚。

Listing 2.4 迭代反转链表：先保存后继，再改变指向

```

1 struct ListNode {
2     int value = 0;
3     ListNode* next = nullptr;
4 };
5
6 ListNode* reverse_list(ListNode* head)
7 {
8     ListNode* previous = nullptr;
9     ListNode* current = head;
10
11     while (current != nullptr) {
12         ListNode* next = current->next;
13         current->next = previous;
14         previous = current;
15         current = next;
16     }
17
18     return previous;
19 }

```

这段代码只有四行核心操作，顺序却不能乱。先保存 `next`，因为一旦执行 `current->next = previous`，原来的后继就找不到了；再把当前节点接到已反转部分前面；最后整体向后推进。写链表题时，建议在纸上画三个指针：已经处理好的部分、当前节点、尚未处理的部分。只要这三段关系清楚，反转、两两交换、K 个一组反转都可以从同一套指针动作推出来。

虚拟头节点是链表题的另一个重要技巧。删除倒数第 N 个节点、合并两个链表、分隔链表时，真实头节点可能被删除或替换。如果每次都单独判断头节点，代码会被边界分支切碎。虚拟头节点把“修改头节点”和“修改普通节点”统一成“修改某个前驱节点的 `next`”。

Listing 2.5 虚拟头节点统一删除逻辑

```

1 ListNode* remove_nth_from_end(ListNode* head, int n)
2 {
3     ListNode dummy;
4     dummy.next = head;
5
6     ListNode* fast = &dummy;
7     ListNode* slow = &dummy;
8
9     for (int step = 0; step < n; ++step) {
10         fast = fast->next;
11     }
12     while (fast->next != nullptr) {
13         fast = fast->next;
14         slow = slow->next;
15     }
16 }

```

```

17     ListNode* removed = slow->next;
18     slow->next = removed->next;
19     return dummy.next;
20 }

```

快慢指针的本质是让两个指针之间保持固定距离或固定速度差。删除倒数第 N 个节点时，快指针先走 N 步，之后快慢一起走；当快指针到尾部，慢指针正好停在待删节点前驱。环检测时，快指针每次走两步，慢指针每次走一步；如果存在环，速度差会让它们在环内相遇。不要把快慢指针背成模板，应该说清楚“两个指针之间的不变量是什么”。

栈是后进先出结构，适合处理“最近出现、最先解决”的关系。括号匹配中，最近的左括号必须先被右括号匹配；路径简化中，最近进入的目录会被 `..` 回退；表达式求值中，最近的运算符和操作数构成局部归约。栈的强大之处在于把嵌套结构压平成一条处理过程。

Listing 2.6 有效括号：最近未匹配结构

```

1 bool is_valid_parentheses(const std::string& s)
2 {
3     std::vector<char> stack;
4     stack.reserve(s.size());
5
6     for (const char ch : s) {
7         if (ch == '(' || ch == '[' || ch == '{') {
8             stack.push_back(ch);
9             continue;
10        }
11        if (stack.empty()) {
12            return false;
13        }
14
15        const char left = stack.back();
16        stack.pop_back();
17        const bool matched = (left == '(' && ch == ')') ||
18                             (left == '[' && ch == ']') ||
19                             (left == '{' && ch == '}');
20        if (!matched) {
21            return false;
22        }
23    }
24
25    return stack.empty();
26 }

```

这里直接使用 `std::vector<char>` 作为栈，比 `std::stack<char>` 更方便查看和预留容量。`std::stack` 是容器适配器，只暴露 `push`、`pop`、`top`、`empty` 等接口；`pop()` 不返回元素，因此必须先 `top()` 再 `pop()`。面试写代码时两者都可以，但要明白适配器牺牲了一部分访问能力，换来更明确的抽象边界。

单调栈是栈在刷题中的高频变体。它维护的不是时间顺序本身，而是一组“尚未找到答案的候选”。每日温度中，栈保存还没有找到更高温度的下标，并且这些下标对应的温度从栈底到栈顶单调不减。新温度到来时，只要它高于栈顶下标的温度，就可以确定栈顶答案；因为这是栈顶下标右侧第一个更高温度的下标。

Listing 2.7 每日温度：单调栈保存下标

```

1 std::vector<int> daily_temperatures(const std::vector<int>& temperatures)

```

```

2 {
3     std::vector<int> answer(temperatures.size(), 0);
4     std::vector<int> stack;
5     stack.reserve(temperatures.size());
6
7     for (int day = 0; day < static_cast<int>(temperatures.size()); ++day) {
8         while (!stack.empty() && temperatures[day] > temperatures[stack.back()]) {
9             const int previous_day = stack.back();
10            stack.pop_back();
11            answer[previous_day] = day - previous_day;
12        }
13        stack.push_back(day);
14    }
15
16    return answer;
17 }

```

单调栈常保存下标而不是值，因为题目经常需要距离、宽度，或者需要回到原数组访问更多信息。它能把暴力枚举“右侧第一个更大元素”的 $O(n^2)$ 降到 $O(n)$ ，原因是每个下标最多入栈一次、出栈一次。这个摊还分析要会说：虽然某一次循环里可能弹出很多元素，但全局弹出次数不超过 n 。

队列是先进先出结构，适合按层扩展。BFS 的正确性来自队列顺序：先入队的是距离更短或时间更早的状态，因此第一次到达一个未访问状态时，就已经得到了最短步数。二叉树层序遍历、腐烂橘子、多源最短扩散、无权图最短路，都依赖这个性质。与栈不同，队列不会深挖某一条路径，而是把同一层的状态先处理完。

Listing 2.8 层序遍历：队列保存当前边界

```

1 std::vector<std::vector<int>> level_order(TreeNode* root)
2 {
3     std::vector<std::vector<int>> levels;
4     if (root == nullptr) {
5         return levels;
6     }
7
8     std::queue<TreeNode*> queue;
9     queue.push(root);
10
11    while (!queue.empty()) {
12        const int level_size = static_cast<int>(queue.size());
13        std::vector<int> level;
14        level.reserve(level_size);
15
16        for (int i = 0; i < level_size; ++i) {
17            TreeNode* node = queue.front();
18            queue.pop();
19            level.push_back(node->value);
20
21            if (node->left != nullptr) {
22                queue.push(node->left);
23            }
24            if (node->right != nullptr) {
25                queue.push(node->right);
26            }
27        }
28    }
29    return levels;
30 }

```

```

27     }
28     levels.push_back(std::move(level));
29 }
30
31 return levels;
32 }

```

这段代码里的 `level_size` 是关键。没有它，也可以 BFS，但很难知道哪些节点属于同一层。层序遍历、最短时间、多源扩散都需要把“当前层的节点数量”固定下来，然后处理完这一层再增加距离或分钟数。

`std::deque` 支持两端 $O(1)$ 插入删除，是 `std::queue` 的默认底层容器，也是单调队列的常用实现。滑动窗口最大值的暴力方法是每个窗口扫描一遍最大值，复杂度 $O(nk)$ 。如果用堆，需要处理过期下标。单调队列更贴合题意：队头永远是当前窗口最大值下标，队尾负责淘汰不可能再成为最大值的候选。

Listing 2.9 滑动窗口最大值：单调队列

```

1 std::vector<int> max_sliding_window(const std::vector<int>& nums, int k)
2 {
3     std::vector<int> answer;
4     if (nums.empty() || k <= 0) {
5         return answer;
6     }
7     answer.reserve(nums.size());
8
9     std::deque<int> deque;
10    for (int right = 0; right < static_cast<int>(nums.size()); ++right) {
11        while (!deque.empty() && nums[deque.back()] <= nums[right]) {
12            deque.pop_back();
13        }
14        deque.push_back(right);
15
16        const int left = right - k + 1;
17        if (deque.front() < left) {
18            deque.pop_front();
19        }
20        if (left >= 0) {
21            answer.push_back(nums[deque.front()]);
22        }
23    }
24
25    return answer;
26 }

```

单调队列的优化理由和单调栈相似。一个新元素如果大于等于队尾元素，那么队尾元素在未来任何包含新元素的窗口里都不可能成为最大值，因为它更小且更早过期。于是可以安全删除。每个下标最多进队一次、出队一次，总复杂度 $O(n)$ 。

理解 `vector` 的底层模型，可以解释很多看似奇怪的 C++ 行为。典型实现内部并不是保存“一个数组对象”，而是保存三个位置：起始位置、当前结束位置、容量结束位置。`size()` 等于当前结束减起始，`capacity()` 等于容量结束减起始。`push_back` 时，如果当前结束还没碰到容量结束，就在当前位置构造新元素并移动结束指针；如果容量满了，就申请更大的连续内存，把旧元素移动或复制过去，再释放旧内存。正因为扩容会换一整段内存，旧的指针、引用和迭代器才会失效。

Listing 2.10 观察 vector 扩容：地址变化意味着旧指针失效

```

1 std::vector<std::size_t> collect_capacity_changes(int count)
2 {
3     std::vector<int> values;
4     std::vector<std::size_t> capacities;
5     capacities.reserve(static_cast<std::size_t>(count));
6
7     std::size_t last_capacity = values.capacity();
8     for (int i = 0; i < count; ++i) {
9         values.push_back(i);
10        if (values.capacity() != last_capacity) {
11            capacities.push_back(values.capacity());
12            last_capacity = values.capacity();
13        }
14    }
15
16    return capacities;
17 }

```

这段实验代码不是算法题答案，而是学习容器的办法。你可以打印每次容量变化，观察容量通常按倍数增长。标准并不要求具体增长倍数，不同标准库实现可能不同，但“容量不足时整体搬迁”这个语义是理解失效规则的关键。刷题时，如果你要保存 `vector` 中元素的地址，然后继续 `push_back`，就必须先 `reserve` 到足够容量，或者改用不会整体搬迁节点的结构。反过来，如果你只保存下标，扩容不会让下标语义失效，这也是很多算法保存下标而不是指针的原因。

`vector` 的摊还 $O(1)$ 插入也值得讲清楚。一次扩容可能移动很多元素，看起来很贵；但如果容量按倍数增长，元素一生被搬迁的次数是对数级，总搬迁成本在一串 `push_back` 操作中可以摊成线性。因此连续追加 n 个元素的总成本仍然是 $O(n)$ 。这并不意味着每一次 `push_back` 都是严格常数，也不意味着扩容时没有延迟尖峰。面试算法一般关心摊还复杂度，工程系统如果关心实时延迟，就要提前预留容量或使用其他结构。

常见误区

`reserve` 不是 `resize`。前者只改变容量，不创建可访问元素；后者改变大小，会构造元素。写 `values.reserve(n)` 后访问 `values[i]` 是越界；写 `values.resize(n)` 或构造 `vector<int> values(n)` 后才可以按下标写。DP 表、计数数组、距离数组通常需要 `resize` 或直接构造；收集答案、邻接表追加边、构造结果列表通常适合 `reserve` 后 `push_back`。

`string` 的底层也不只是“字符数组”。现代标准库实现通常会使用小字符串优化：短字符串直接存在 `string` 对象内部，长字符串才分配堆内存。标准不规定小字符串容量，因此不能依赖某个具体长度，但这个优化解释了为什么短字符串拷贝有时比想象中便宜。即便如此，热循环中的 `substr` 仍然要谨慎，因为它会构造一个新 `string`，可能复制字符、分配内存，并参与哈希计算。单词拆分、回文切分、字符串匹配中，如果在双层循环里频繁 `substr`，实际成本可能比复杂度公式更高。

Listing 2.11 用 `string_view` 表达非拥有子串视图

```

1 bool starts_with_at(std::string_view text,
2                    int position,
3                    std::string_view pattern)
4 {
5     if (position < 0 ||
6         position + static_cast<int>(pattern.size()) >
7             static_cast<int>(text.size())) {
8         return false;

```



```

9     }
10
11     for (int i = 0; i < static_cast<int>(pattern.size()); ++i) {
12         if (text[static_cast<std::size_t>(position + i)] !=
13             pattern[static_cast<std::size_t>(i)]) {
14             return false;
15         }
16     }
17     return true;
18 }

```

`string_view` 不拥有字符，它只是指向已有字符串的一段视图，所以创建便宜，也不会复制内容。代价是生命周期风险：原字符串销毁或修改导致重分配后，视图可能悬空。刷题中，如果只是函数内部临时比较，`string_view` 很合适；如果要把视图存进容器并在原字符串之后继续使用，就必须非常小心。教材里保留一些 `substr` 写法，是为了让主算法清晰；源码阅读和工程优化时，再讨论如何减少复制。

`deque` 经常被误以为是链表。实际上，`deque` 通常是分段连续存储：有一个中控表保存若干固定大小缓冲区的地址，每个缓冲区内部连续。它支持两端高效插入删除，因为在头尾缓冲区还有空间时，只移动段内指针；空间不够时，分配新缓冲区并挂到中控表。它也支持随机访问，但要先定位缓冲区再定位段内偏移，局部性通常不如 `vector`。这解释了为什么滑动窗口最大值适合 `deque`，而普通 DP 表不应该随便用 `deque` 替代 `vector`。

`list` 是双向链表，节点地址稳定，已知迭代器时插入删除是常数时间。它的缺点也明显：每个节点单独分配，额外保存前后指针，缓存局部性差，不支持下标随机访问。绝大多数刷题场景中，`list` 不是首选。LRU 缓存是少数非常适合 `list` 的例子，因为我们需要在已知迭代器的情况下，把任意节点移动到表头，并在容量满时删除表尾。这个需求正好匹配 `list::splice` 的节点重连能力。

Listing 2.12 `list::splice`：节点移动而不是复制元素

```

1 void move_key_to_front(std::list<int>& keys,
2                       std::list<int>::iterator position)
3 {
4     keys.splice(keys.begin(), keys, position);
5 }

```

`splice` 不复制元素，也不重新分配节点，只改变链表链接关系。LRU 中哈希表保存 key 到链表迭代器，链表维护新旧顺序；访问一个 key 时，用迭代器把节点移动到表头。若用 `vector` 保存顺序，移动到表头会搬移大量元素，且保存的迭代器会失效；若只用哈希表，又无法知道谁最久未使用。这个案例说明，容器不是按“哪个复杂度低”孤立选择，而是按“多个操作是否同时满足约束”来选择。

栈和队列在 C++ 中通常是容器适配器。`std::stack` 默认底层是 `deque`，只暴露栈需要的接口；`std::queue` 默认底层也是 `deque`，只暴露队列接口。适配器让代码表达意图更明确，但也限制访问能力。例如 `std::stack` 不能遍历，也不能 `reserve`。刷题中如果需要遍历、预留容量、查看任意位置，直接用 `vector` 或 `deque` 模拟栈/队列更方便；如果只需要标准栈队列操作，适配器能让语义更干净。

迭代器失效规则是容器原理的考试点。`vector` 扩容会让所有迭代器、指针和引用失效；未扩容时，尾部插入通常只影响尾后迭代器，中间插入删除会影响插入点之后的位置。`deque` 两端插入删除的失效规则比 `vector` 复杂，不应长期保存它的迭代器做复杂操作。`list` 删除某个节点只让该节点迭代器失效，其他节点仍稳定。`unordered_map` rehash 会让迭代器失效，但指向元素的引用在很多实现中可能仍稳定；标准语义和具体实现细节要区分，不要在可移植代码里依赖实现偶然行为。

深入理解

源码阅读建议从行为反推结构。读 `vector`，先找内部表示大小和容量的成员，再跟 `push_back` 的容量足够路径和容量不足路径；读 `string`，先找短字符串和长字符串的分支；读 `deque`，先找段表和段大小；读 `list`，先找节点结构和链接操作；读 `stack` 和 `queue`，先确认它们只是包装底层容器。不要一开始陷进 `allocator` 和模板特化细节，先抓住数据如何存、操作如何移动、什么时候失效。

线性容器源码实验：第一，连续 `push_back` 并打印 `vector` 的 `capacity()` 和 `data()`，观察扩容时地址变化。第二，把一段单词拆分代码中的 `substr` 次数统计出来，再改成下标比较，比较运行时间。第三，用 `deque` 和 `vector` 分别实现滑动窗口最大值，解释为什么 `vector` 不适合从头部弹出。第四，实现一个最小 LRU，只支持 `get` 和 `put`，强制自己说明哈希表和链表各自负责什么。

线性结构选型

需要稳定下标和顺序扫描，优先 `vector`；需要字符窗口，使用 `string` 加计数结构，不要频繁 `substr`；需要局部重连，考虑链表并画清楚前驱、当前、后继；需要最近未解决状态，用栈；需要按层或最短步数扩展，用队列；需要窗口两端同时维护候选，用双端队列。选择结构时同时说出它让哪一步查询、删除或更新变快。

本章建议配套练习：数组与前缀和做 303、304、560；字符串窗口做 3、438、567、76；链表做 19、21、24、141、142、206、92、25；栈做 20、71、150、155、739、84；队列和双端队列做 102、199、239、994。复盘时不要只写最终代码，要写三行说明：暴力方法慢在哪里，数据结构保存了什么状态，为什么每个元素只被处理有限次。

线性结构的教材级展开

线性数据结构看起来简单，实际最容易被学薄。数组、字符串、链表、栈、队列和双端队列不是几组 API 名称，而是不同访问模式的承诺：数组承诺按下标常数访问，链表承诺已知节点附近的局部重连，栈承诺只处理最近未解决对象，队列承诺按进入顺序扩展，双端队列承诺两端候选都能快速进出。算法题里的“优化”常常就是把原本昂贵的访问模式换成某个结构天然支持的访问模式。经典教材会反复强调这一点，因为如果只会背容器名字，遇到变体题时就无法判断模板为什么失效。

学习线性结构时可以用三条线索串起来。第一条是物理存储：元素是否连续，是否会整体搬迁，节点地址是否稳定，是否能被 CPU 预取。第二条是抽象接口：能不能随机访问，能不能从中间删除，能不能只看一端，能不能同时维护两端。第三条是算法不变量：慢指针左侧是什么，栈内保存什么未解决对象，队列当前层表示什么距离，窗口左端为什么可以丢弃。真正掌握数据结构，就是能把这三条线索同时讲清楚。

访问模式先于容器名称

选择容器前先问访问模式：按下标频繁访问，优先连续数组；只在尾部追加且要排序或 DP，优先 `vector`；需要头尾都弹入弹出，考虑 `deque`；需要已知节点处常数移动，考虑链表；需要最近未匹配结构，用栈；需要按层扩展，用队列。容器不是越复杂越好，而是越贴合访问模式越好。

数组和动态数组：连续内存、下标语义和原地维护

数组的核心能力是“位置稳定且可计算”。第 i 个元素的位置可以由起始地址加上 i 乘以元素大小得到，所以按下标访问是常数时间。这一条性质让数组天然适合二分、前缀和、差分数组、排序、双指针、动态规划表和邻接表。数组的弱点也来自连续性：在中间插入或删除一个元素，后面的元素必须整体搬移；容量不足时，动态数组要申请新内存并移动已有元素。

`std::vector` 的概念模型可以理解成三个指针：起始位置、当前结束位置、容量结束位置。`size()` 来自当前结束位置减起始位置，`capacity()` 来自容量结束位置减起始位置。尾部追加时如果容量够，只在尾部构造一个元素；容量不够时，申请更大内存、移动旧元素、释放旧内存。这个

过程解释了两个常见问题：为什么 `push_back` 的均摊复杂度是常数，为什么扩容后旧指针、引用、迭代器会失效。

数组算法要特别重视下标区间语义。左闭右闭区间 `[left, right]` 适合描述题目原始边界，左闭右开区间 `[left, right)` 更适合循环和切片，因为长度正好是 `right - left`，空区间表示为 `left == right`。前缀和通常使用左闭右开语义：`prefix[i]` 表示前 `i` 个元素之和，区间 `[left, right)` 的和就是 `prefix[right] - prefix[left]`。这种设计把下标零的特殊情况变成统一公式，是教材里非常重要的哨兵思想。

原地维护类题日本质是在数组上划分语义区间。删除有序数组重复项中，慢指针左侧是已经写好的唯一值前缀；颜色分类中，数组被划成零区、一号区、未知区和二区；移动零中，慢指针左侧是已经保留相对顺序的非零元素；下一个排列中，后缀是最长非递增区，枢轴和后缀重排决定字典序的下一步。题目不同，但证明方式相同：每一步循环只扩大已经正确的区域，未知区域严格缩小，被排除或交换出去的元素不会再破坏不变量。

先看颜色分类。题目只有三种颜色，最直接的暴力办法是排序，或者先扫描三遍分别统计 0、1、2 的数量，再按数量覆盖回数组。排序当然能得到结果，但它没有利用值域只有三类这个条件；计数覆盖虽然是线性的，却把题目变成“先统计再重写”，没有训练原地分区。进一步观察，扫描过程中真正需要维护的不是完整频次，而是四段区间：左边已经确定为 0，中间已经确定为 1，右边已经确定为 2，剩下的是未知区。于是优化自然变成荷兰国旗式的三指针维护，而不是直接背一段交换代码。

图示：颜色分类的四段区间

区间	语义	变化方式
<code>[0, next_red)</code>	已经确定为 0	只向右扩大
<code>[next_red, current)</code>	已经确定为 1	只向右扩大
<code>[current, next_blue]</code>	尚未检查	每轮缩小
<code>(next_blue, n)</code>	已经确定为 2	只向左扩大

Listing 2.13 颜色分类：四段区间不变量

```

1 void sort_colors(std::vector<int>& nums)
2 {
3     constexpr int RED = 0;
4     constexpr int WHITE = 1;
5     constexpr int BLUE = 2;
6
7     int next_red = 0;
8     int current = 0;
9     int next_blue = static_cast<int>(nums.size()) - 1;
10
11     while (current <= next_blue) {
12         if (nums[current] == RED) {
13             std::swap(nums[next_red], nums[current]);
14             ++next_red;
15             ++current;
16         } else if (nums[current] == BLUE) {
17             std::swap(nums[current], nums[next_blue]);
18             --next_blue;
19         } else {
20             ++current;
21         }
22     }
23 }
```

这段代码的关键不是三指针本身，而是区间解释： $[0, \text{next_red})$ 全是 0， $[\text{next_red}, \text{current})$ 全是 1， $[\text{current}, \text{next_blue}]$ 未知， $(\text{next_blue}, n)$ 全是 2。遇到 2 时不能推进 `current`，因为从右侧换回来的元素还没有检查；遇到 0 时可以推进，因为换回来的位置属于已经扫描过的一号区或就是当前。这个细节正是很多错误实现的来源。

数组变种还包括稀疏数组、位图、环形缓冲区和压缩坐标数组。位图把布尔集合压到二进制位，适合值域有限且只需要存在性的问题；环形缓冲区用固定数组和头尾下标模拟队列，适合实时系统和滑动窗口；离散化把大值域映射到连续小编号，适合树状数组、线段树和频次统计。刷题时常见的“坐标压缩”不是数学技巧，而是把无法直接开数组的值域转成数组能支持的下标访问。

字符串：字符序列、编码假设和子串成本

字符串是数组的特例，但题目里的字符串又比数组多几个坑。第一，字符集决定计数结构：小写英文可以用 26 长度数组，ASCII 可以用 128 或 256 长度数组，Unicode 则不能简单按字节计数。第二，`substr` 会构造新字符串，热循环中频繁使用会引入复制、分配和哈希成本。第三，字符串算法常常需要区分子串和子序列：子串必须连续，适合窗口、哈希、KMP 和中心扩展；子序列不要求连续，常用动态规划或贪心扫描。

固定长度窗口题的本质是相邻窗口只差一个进入字符和一个离开字符。找到所有字母异位词、排列包含、固定长度最大元音数都属于这一类。变长窗口题则依赖合法性可以通过移动左端恢复，例如最小覆盖子串中右端扩展增加覆盖能力，满足后移动左端尝试变短。若窗口状态不会随左右端单调变化，例如数组含负数且目标是区间和，就不能硬套滑动窗口。

字符串匹配类题目有多条路线。暴力匹配每个起点再逐字符比较，最坏可能 $O(nm)$ ；KMP 通过前缀函数避免主串指针回退；滚动哈希把子串比较转成哈希值比较，但要处理冲突；Trie 把多个模式串的公共前缀合并，适合单词搜索 II、前缀统计和自动补全。面试时不一定要写出所有高级算法，但必须能说明它们解决的是哪种重复工作。

排列包含是固定长度窗口的典型例子。题目问 `text` 中是否存在某个长度等于 `pattern.size()` 的子串，字符频次和 `pattern` 完全相同。暴力做法是枚举每个起点，截出长度为 m 的子串，再排序比较或重新统计频次。相邻两个候选子串只差左边移出的一个字符和右边进入的一个字符，如果每次从零统计，就把 m 个字符的绝大部分重复计算了。优化点因此很明确：窗口长度固定，右端每前进一步，只增量加入一个字符，并在窗口超过长度时移出一个字符；窗口计数等于需求计数时返回真。

图示：固定长度窗口的增量更新

旧窗口	<code>text[left ... right]</code> 中已经维护好 26 个字符计数。
右端进入	新字符 <code>text[right + 1]</code> 的计数加一。
左端离开	若窗口长度超过 <code>pattern.size()</code> ，旧字符 <code>text[left]</code> 的计数减一。
判断答案	窗口长度固定后，比较 <code>window == need</code> ，不用重新统计整段。

Listing 2.14 字符串窗口：用数组计数表达字符集假设

```

1 bool contains_permutation(const std::string& text, const std::string& pattern)
2 {
3     constexpr int ALPHABET_SIZE = 26;
4     if (text.size() < pattern.size()) {
5         return false;
6     }
7
8     std::array<int, ALPHABET_SIZE> need{};
9     std::array<int, ALPHABET_SIZE> window{};
10    for (const char ch : pattern) {
11        ++need[ch - 'a'];
12    }

```



```

13
14     for (std::size_t right = 0; right < text.size(); ++right) {
15         ++window[text[right] - 'a'];
16         if (right >= pattern.size()) {
17             --window[text[right - pattern.size()] - 'a'];
18         }
19         if (window == need) {
20             return true;
21         }
22     }
23     return false;
24 }

```

这段代码必须附带条件：输入只含小写英文字母。如果题目没有这个条件，数组下标 `ch - 'a'` 就没有意义。经典教材会反复强调“前提条件”，因为算法复杂度和正确性都依赖前提。把字符集换成 Unicode 后，代码结构仍然可以是窗口，但计数结构需要换成哈希表或更明确的编码处理。

链表：节点身份、局部重连和哨兵节点

链表的价值不在于“删除一定比数组快”，而在于节点身份稳定和局部重连便宜。已知一个节点或它的前驱时，插入删除只需要改少数指针；但如果要寻找第 k 个节点，仍然必须从头走 k 步。链表题考的是结构不变量：哪些节点已经接好，哪些节点还没有处理，当前节点的后继是否已经保存，操作后是否仍然能到达未处理部分。

单链表、双链表、循环链表和侵入式链表各有不同用途。单链表空间少，适合反转、合并、快慢指针；双链表能从节点向前向后移动，适合 LRU、删除已知节点和双端队列；循环链表适合轮转和约瑟夫类问题；侵入式链表把链接字段放进业务对象本身，Linux 内核大量使用这种方式来减少额外分配并精确控制对象生命周期。刷题通常使用平台给出的 `ListNode`，但理解这些变体能帮助你解释为什么 LRU 需要哈希表加双向链表。

哨兵节点是链表题的统一边界工具。删除头节点、合并空链表、分隔链表、反转前 k 个节点时，如果直接操作真实头节点，经常要写一堆特殊分支；引入虚拟头节点后，所有操作都变成“修改某个前驱的 `next`”。这跟前缀和多开一个 `prefix[0]` 是同一种思想：用一个额外状态把边界并入普通逻辑。

两两交换节点如果从数组视角出发，暴力办法是把节点值复制到数组里，交换相邻值后再写回链表。这样能得到相同值序列，却没有真正交换节点，也无法迁移到要求保持节点身份的题。若坚持在链表上做，问题可以缩小到一个局部：前驱节点 `previous` 后面有两个待交换节点 `first` 和 `second`，再后面是下一段入口 `after`。优化不是减少时间复杂度，因为暴力重建也是线性；优化的是结构质量和可迁移性：只改三条边，保持已经处理好的前缀不动，未处理后缀仍然可达。

图示：两两交换的局部重连

交换前	<code>previous -> first -> second -> after</code>
第一条边	<code>previous -> second</code> ，让前缀接到交换后的头。
第二条边	<code>second -> first</code> ，完成这一对内部交换。
第三条边	<code>first -> after</code> ，把未处理后缀接回链表。
推进位置	<code>previous = first</code> ，因为 <code>first</code> 已成为已处理前缀的尾部。

Listing 2.15 两两交换链表节点：哨兵和局部三指针

```

1 ListNode* swap_pairs(ListNode* head)
2 {
3     ListNode dummy;
4     dummy.next = head;
5
6     ListNode* previous = &dummy;

```

```

7   while (previous->next != nullptr && previous->next->next != nullptr) {
8       ListNode* first = previous->next;
9       ListNode* second = first->next;
10      ListNode* after = second->next;
11
12      previous->next = second;
13      second->next = first;
14      first->next = after;
15
16      previous = first;
17  }
18  return dummy.next;
19 }

```

这段代码的局部不变量是：`previous` 之前的链表已经交换完成，`previous->next` 是当前待交换对的第一个节点，`after` 保存下一段入口。先保存 `after`，再改三条边，最后把 `previous` 移到这一对交换后的尾部。K 个一组反转也是同一模式，只是局部段更长，需要先检查是否有 k 个节点，再在段内反转并接回。

链表常见变种包括环检测、环入口、相交链表、回文链表、链表排序、复制带随机指针链表和 LRU 缓存。环检测用快慢指针，核心是速度差；相交链表比较的是节点地址，不是节点值；回文链表需要找到中点、反转后半段、比较并最好恢复结构；链表排序适合自底向上归并，避免递归栈；随机指针复制需要原节点到新节点的映射，或者把复制节点插入原链表再拆分。每个变种都不是新模板，而是围绕“节点身份和链接关系”展开。

栈和单调栈：最近关系、表达式和候选淘汰

栈保存的是最近尚未解决的结构。括号匹配中，最近的左括号必须先匹配；路径简化中，最近进入的目录可能被 `..` 取消；表达式求值中，最近的运算符和操作数形成局部归约；DFS 中，栈保存待展开路径。普通栈的正确性来自后进先出，单调栈的正确性来自候选淘汰。

单调栈不是为了“保持栈有序”而有序，而是为了让新元素到来时能结算一批旧元素。每日温度、下一个更大元素、柱状图最大矩形、接雨水都可以用这个模型解释。栈里保存尚未找到右边界的下标；新元素若破坏单调性，就说明某些栈顶元素的右边界已经确定。被弹出的元素不会再回来，因此总复杂度是摊还线性。

表达式类题目则体现栈的另一面：保存上下文。逆波兰表达式已经把优先级编码到顺序里，只需要数字栈；基本计算器 II 需要遇到乘除立即结合、加减延迟求和；带括号的计算器需要在进入括号时保存外层结果和符号。写表达式题时，先定义 token 类型、当前数字、当前运算符、栈中保存内容，再动手编码，错误会少很多。

逆波兰表达式可以先按题意暴力模拟：从左到右读 token，遇到数字暂存，遇到运算符就回头找离它最近的两个尚未参与归约的数字，计算后再把结果放回序列。这个做法的瓶颈是“最近尚未归约对象”的查找和删除很别扭，因为序列中间会不断被替换。观察运算符的定义后会发现，真正需要的永远是最近的两个操作数，且先出现的操作数在左，后出现的操作数在右。这正是栈的职责：数字入栈，运算符弹出右操作数和左操作数，归约结果再入栈。

图示：逆波兰表达式的栈归约

读到数字	压入栈，表示一个尚未被运算符消费的操作数。
读到运算符	弹出 <code>rhs</code> ，再弹出 <code>lhs</code> ，顺序不能反。
完成归约	计算 <code>lhs op rhs</code> ，把结果重新压栈。
最终状态	所有 token 扫完后，栈顶就是整个表达式的值。

Listing 2.16 逆波兰表达式：栈中保存待归约操作数

```

1 int eval_rpn(const std::vector<std::string>& tokens)

```



```

2 {
3     std::vector<int> stack;
4     stack.reserve(tokens.size());
5
6     for (const std::string& token : tokens) {
7         if (token == "+" || token == "-" || token == "*" || token == "/") {
8             const int rhs = stack.back();
9             stack.pop_back();
10            const int lhs = stack.back();
11            stack.pop_back();
12
13            if (token == "+") {
14                stack.push_back(lhs + rhs);
15            } else if (token == "-") {
16                stack.push_back(lhs - rhs);
17            } else if (token == "*") {
18                stack.push_back(lhs * rhs);
19            } else {
20                stack.push_back(lhs / rhs);
21            }
22        } else {
23            stack.push_back(std::stoi(token));
24        }
25    }
26    return stack.back();
27 }

```

这里弹出的第一个数是右操作数，第二个数是左操作数。减法和除法不能交换顺序，这是表达式栈题最常见的细节。工程实现中，如果 token 很多且转换成本重要，可以先写一个更明确的解析函数；但刷题训练重点是栈状态语义。

队列、双端队列和环形缓冲区

队列保存的是时间顺序或层次顺序。BFS 第一次到达某个状态即为最短距离，前提是所有边权相等且访问标记在入队时设置。多源 BFS 只是把多个起点同时作为第零层，腐烂橘子、地图中离最近零的距离、从边界反向标记安全区域都可以这样处理。队列不负责“选最优候选”，它只负责“按层推进”；如果边权不同，就需要堆或其他最短路结构。

双端队列比队列多了从两端进出的能力，因此能维护窗口候选。单调队列保存的不是窗口内所有元素，而是仍可能成为答案的候选下标。队头是当前答案，队尾负责删除被新元素支配的旧候选。滑动窗口最大值、和至少为 K 的最短子数组、受限子序列和都依赖类似的支配关系。

环形缓冲区是队列的底层变体。固定数组配合头尾下标，可以避免频繁分配；当尾部走到数组末尾时回到开头。它适合实现固定容量队列、日志缓冲、生产者消费者队列和某些实时系统。刷题中不常要求手写环形队列，但理解它能帮助你明白为什么 `deque` 和 `queue` 的接口强调两端操作，而不是随机插入。

Listing 2.17 固定容量环形队列：头尾下标表达状态

```

1 class CircularQueue {
2 public:
3     explicit CircularQueue(int capacity)
4         : values_(static_cast<std::size_t>(capacity)), capacity_(capacity)
5     {
6     }
7 }

```

```

8     bool push(int value)
9     {
10         if (this->size_ == this->capacity_) {
11             return false;
12         }
13         this->values_[static_cast<std::size_t>(this->tail_)] = value;
14         this->tail_ = (this->tail_ + 1) % this->capacity_;
15         ++this->size_;
16         return true;
17     }
18
19     std::optional<int> pop()
20     {
21         if (this->size_ == 0) {
22             return std::nullopt;
23         }
24         const int value = this->values_[static_cast<std::size_t>(this->head_)];
25         this->head_ = (this->head_ + 1) % this->capacity_;
26         --this->size_;
27         return value;
28     }
29
30 private:
31     std::vector<int> values_;
32     int capacity_ = 0;
33     int head_ = 0;
34     int tail_ = 0;
35     int size_ = 0;
36 };

```

这段代码保留 `size_`，是为了区分空队列和满队列。如果只用 `head_ == tail_`，空和满都会满足这个条件，需要浪费一个槽位或额外标记。真实工程中还要考虑并发、异常安全和容量为零的输入约束；刷题版本则要把状态不变量讲清楚：`head_` 指向下一个弹出位置，`tail_` 指向下一个写入位置，`size_` 表示当前元素数。

线性结构的实际应用和变种题谱

数组的实际应用包括排序缓冲、DP 表、图邻接表、前缀统计、差分更新、坐标压缩后的频次数组。字符串应用包括日志解析、词典匹配、搜索建议、DNA 序列重复检测、表达式解析。链表应用包括 LRU、内核对象链、空闲块管理、任务队列和需要稳定节点地址的缓存。栈应用包括函数调用、解析器、撤销操作、表达式求值、单调候选。队列应用包括 BFS、任务调度、消息缓冲、多源扩散。双端队列应用包括滑动窗口极值、0-1 BFS、双端搜索和单调 DP 优化。

变种题复盘可以按结构归类。数组原地维护：26、27、75、88、189、283；前缀与差分：303、304、560、1109、1094；字符串窗口：3、76、438、567；链表重连：19、21、24、25、92、206；链表身份：141、142、160、138；普通栈：20、71、150、224、227、394；单调栈：42、84、496、503、739；队列 BFS：102、199、200、994、1091；单调队列：239、862、1438、1425。每组题都要问同一个问题：结构保存什么，什么时候加入，什么时候移除，移除是否会丢答案。

线性结构实验：第一，用 `vector`、`deque`、`list` 分别顺序扫描一百万个整数，比较实际时间并解释缓存局部性。第二，把字符串窗口题写成 `substr` 加排序和计数窗口两版，记录输入长度增长时的差异。第三，把链表反转的每一步打印为节点地址，确认比较的是节点身份。第四，把单调栈中的每个下标入栈出栈次数记录下来，验证摊还复杂度。第五，手写环形队列并测试空、满、绕回和连续弹入弹出。

线性结构变种题谱

数组变种可以分成七类。第一类是原地过滤，例如删除重复项、移除元素、移动零、稳定划分。这类题的共同点是输出仍放在原数组前缀，慢指针表示下一个写入位置，快指针负责扫描未知区。第二类是双向收缩，例如盛水容器、两数之和 II、回文判断。这类题依赖左右边界比较能排除一侧。第三类是前后缀贡献，例如除自身以外数组乘积、接雨水的前后缀解法、最短无序连续子数组。第四类是矩阵边界模拟，例如螺旋矩阵、旋转图像、矩阵置零。第五类是差分和扫描线，例如航班预订、拼车、区间加法。第六类是坐标压缩后用数组维护频次或排名。第七类是环形数组，例如轮转数组、循环队列、下一个更大元素 II。

这些变种应该统一回到区间不变量。原地过滤要说明写入区不会被未来读操作破坏；双向收缩要说明被移走的一侧不可能产生更优答案；前后缀贡献要说明当前位置答案由左右两边已经缓存的信息合成；矩阵模拟要说明四条边界何时收缩，剩一行或一列时不会重复访问；差分数组要说明每个区间更新如何转化为两个端点事件。题目名称不同，但证明方式不是散的。

字符串变种可以按“连续、非连续、多模式”分。连续子串题包括无重复最长子串、最小覆盖子串、异位词窗口、回文子串、重复 DNA 序列。非连续子序列题包括判断子序列、最长公共子序列、不同的子序列。多模式题包括单词搜索 II、替换单词、搜索建议系统。连续子串常用窗口、哈希、中心扩展和 KMP；子序列常用双指针或 DP；多模式常用 Trie。复盘时先判断目标是否要求连续，能直接避免很多错误方向。

链表变种可以按“位置、结构、身份、排序、缓存”分。位置类题包括删除倒数第 N 个、旋转链表、分隔链表，核心是快慢指针和前驱节点。结构类题包括反转链表、两两交换、K 个一组反转、重排链表，核心是保存后继和局部接回。身份类题包括环形链表、环入口、相交链表，核心是比较节点地址。排序类题包括合并两个有序链表、合并 K 个链表、排序链表，核心是节点重连和归并。缓存类题包括 LRU 和 LFU，核心是哈希定位和链表维护顺序。

栈变种可以按“匹配、归约、单调、上下文”分。匹配类题包括有效括号、删除无效括号、最长有效括号；归约类题包括逆波兰表达式、基本计算器、字符串解码；单调类题包括每日温度、下一个更大元素、柱状图最大矩形、接雨水；上下文类题包括路径简化、文件系统解析、嵌套列表解析。判断一个栈题属于哪一类，关键看栈里保存的是符号、操作数、下标、局部字符串，还是上一层上下文。

队列变种可以按“层次、双端、单调、优先级”分。普通队列服务 BFS 层次，双端队列服务两端扩展和 0-1 BFS，单调队列服务窗口极值和 DP 转移最大值，优先级队列服务动态极值。不要把这些队列混成一个概念。普通 BFS 队列不按权重选点，所以不能处理不同边权；单调队列不是为了先进先出，而是为了删除被支配候选；优先级队列不是按时间顺序，而是按优先级顺序。

线性结构复盘表

每做一道线性结构题，都在题解末尾补一张小表：输入是否连续，是否允许修改，是否需要保持相对顺序，是否要比较节点身份，窗口是否合法依赖什么状态，候选什么时候过期，元素最多进入和离开结构几次。能填完这张表，说明你掌握的是结构，不是题号。

线性结构的底层成本模型

经典教材不会只给出一张复杂度表，因为复杂度表把太多真实代价藏起来了。线性结构的底层成本至少有五个维度：随机访问、顺序扫描、局部插入删除、内存分配、缓存局部性。数组和 **vector** 在随机访问和顺序扫描上很强，因为第 i 个元素的位置可以直接计算；链表在已知节点附近重连很强，但每走一步都要先加载下一个节点地址；**deque** 介于两者之间，分段连续让两端扩张便宜，却让纯顺序扫描的局部性通常弱于 **vector**。

这五个维度会直接改变刷题方案。最长递增子序列的 $O(n \log n)$ 解法需要频繁在一个有序尾数组上二分，尾数组必须支持下标访问，所以 **vector** 合适；LRU 缓存需要把任意已知节点移到表头并删除表尾，**list** 合适；滑动窗口最大值需要从两端删除候选，**deque** 合适；环形日志缓冲不需要无限扩容，只需要固定容量覆盖旧值，数组加头尾下标更合适。容器不是按照“高级程度”选择，而是按照操作组合选择。

再看插入删除的说法。说链表删除是 $O(1)$ ，必须补上前提：已经有待删节点的前驱或迭代器。如果你只有一个值，还要从头找这个值，查找仍然是 $O(n)$ 。说 **vector** 尾部插入是均摊 $O(1)$ ，也必须补上前提：它可能偶尔扩容，扩容时旧地址全部失效。说 **deque** 两端插入是 $O(1)$ ，也不等于

它适合所有随机访问密集任务。高质量答案要把这些前提说出来。

深入理解

CPU 缓存解释了为什么很多教材和工程代码偏爱连续数组。处理器从内存读取一个整数时，通常会把相邻的一小段字节一起载入缓存；如果下一次访问正好落在这段缓存行里，就不必再等主内存。数组顺序扫描天然吃到这个收益。链表节点可能散落在堆上，访问下一个节点前必须先读出当前节点里的指针，硬件很难提前知道下一步在哪里。两段代码同为 $O(n)$ ，实际速度可能差出很多，这不是大 O 错了，而是大 O 的抽象层次不同。

数组模型：从区间到事件

数组题要先判断自己维护的是“区间状态”还是“位置事件”。区间状态题关心某个连续范围内的和、最大值、字符频次、不同元素数；位置事件题关心某个下标开始或结束了一段影响。前缀和把区间查询转成两个端点的差，差分数组把区间更新转成两个端点事件。二者是同一思想的两个方向：前缀和适合多次查询，差分适合多次更新后统一还原。

差分数组特别适合讲清“事件化”的思维。航班预订、拼车、区间加法这类题，如果对每个区间逐个位置累加，复杂度会被区间长度拖垮。差分数组只在左端点加一个事件，在右端点后一位减一个事件；最后扫描一次，把事件累积成每个位置的真实值。它把“影响一整段”改写成“从这里开始多一个影响，从那里之后少一个影响”。

Listing 2.18 差分数组：区间更新变成端点事件

```
1 std::vector<int> apply_range_additions(
2     int length,
3     const std::vector<std::array<int, 3>>& operations)
4 {
5     std::vector<int> difference(static_cast<std::size_t>(length + 1), 0);
6
7     for (const auto& operation : operations) {
8         const int left = operation[0];
9         const int right = operation[1];
10        const int delta = operation[2];
11        difference[static_cast<std::size_t>(left)] += delta;
12        if (right + 1 < length) {
13            difference[static_cast<std::size_t>(right + 1)] -= delta;
14        }
15    }
16
17    std::vector<int> values(static_cast<std::size_t>(length), 0);
18    int current = 0;
19    for (int i = 0; i < length; ++i) {
20        current += difference[static_cast<std::size_t>(i)];
21        values[static_cast<std::size_t>(i)] = current;
22    }
23    return values;
24 }
```

这段代码的关键是端点语义。若操作作用在闭区间 $[left, right]$ ，就在 $left$ 处增加影响，在 $right + 1$ 处撤销影响。若题目使用半开区间 $[left, right)$ ，撤销位置就是 $right$ 。很多差分题错在闭区间和半开区间混用。写题解时先定义区间语义，再写公式，代码才不会靠样例猜。

数组还能表达“排名”和“值域”。计数排序、桶排序、位图、坐标压缩都利用这一点。若值域很小，直接用值作下标，查询和统计都可以很快；若值域很大但实际出现的值不多，就把出现过的值排序后映射到 $0..m-1$ ，这就是离散化。树状数组统计逆序对、区间和个数、动态排名时，经常先

离散化，因为原始数值可能达到十亿，不能直接开数组。

字符串模型：窗口、自动机和字典

字符串题的第一分叉是连续还是不连续。连续子串题可以用窗口、哈希、KMP、Manacher、后缀结构；不连续子序列题更多走双指针和动态规划；多个模式串同时匹配时，要考虑 Trie 或 AC 自动机。只要这个分叉判断错，后面的模板就容易全错。比如最小覆盖子串必须连续，适合滑动窗口；最长公共子序列不要求连续，窗口没有意义；单词搜索 II 同时查很多单词，逐个单词 DFS 会重复大量前缀，Trie 才能合并重复工作。

滑动窗口的本质是“窗口合法性可以增量维护”。无重复最长子串中，合法性由每个字符出现次数是否超过一决定；最小覆盖子串中，合法性由所有必需字符是否都达到需求决定；替换后的最长重复字符中，合法性由窗口长度减最高频字符数是否不超过替换次数决定。这些题的状态不同，但都能随着左右端移动做常数或近似常数更新。

窗口题的三句证明

第一，右端扩张时，窗口状态如何更新；第二，窗口不合法时，左端如何移动并恢复合法；第三，左端移动不会错过答案的原因是什么。正数数组求最短和至少为 K 的子数组能用窗口，是因为和随右端扩张不减；数组允许负数时，这条单调性消失，就要换成前缀和加单调队列。

字符串工程应用还要重视编码和生命周期。刷题题面常说只含小写字母，此时 `array<int, 26>` 既快又清晰；日志、路径、URL、自然语言文本则不能这样写。若函数只临时观察一段字符串，`string_view` 能减少复制；若要把子串作为长期 key 存进哈希表，就必须拥有一份真正的 `string`。算法题为了清晰常省略这些工程前提，但教材应该把它们说清楚。

链表模型：缓存、空闲链和节点生命周期

链表题表面是指针题，底层是节点生命周期题。数组的元素位置由下标决定，链表的元素位置由指针关系决定；数组移动元素会改变位置，链表重连节点不会改变节点身份。这个差异让链表适合缓存、内存分配器、内核对象队列和需要稳定节点地址的系统结构。Linux 内核常用侵入式链表：链表节点字段嵌入业务对象中，通过节点地址反推出外层对象。这样做能减少额外分配，也让对象生命周期由系统自己精确控制。

刷题里最能体现工程味的是 LRU。它需要两个操作同时快：按 key 找到缓存项，按新旧顺序删除最久未使用项。哈希表能快速定位 key，却不知道谁最旧；链表能维护顺序，却不能按 key 快速定位。把二者组合起来，哈希表保存 key 到链表迭代器，链表保存从新到旧的缓存项，访问时用 `splice` 把节点移到表头。

如果直接写暴力 LRU，可以用一个数组或链表保存缓存项。每次 `get` 时线性查找 key，找到后把它移到最前；每次 `put` 时也先线性查找，容量满了删除最后一个。这个版本容易理解，但每个操作最坏都是 $O(n)$ ，慢点集中在“按 key 定位”这一步。换成单独哈希表可以把定位降到常数，却失去新旧顺序；换成单独链表能维护顺序，却仍然无法快速定位 key。于是组合结构是从两个瓶颈推出来的：哈希表负责定位，双向链表负责顺序，二者通过迭代器连接。

图示：LRU 的两个索引

哈希表	<code>key -> list iterator</code> ，负责常数时间定位缓存项。
双向链表	<code>newest <-> ... <-> oldest</code> ，负责维护访问新旧顺序。
<code>get</code>	哈希定位节点，再把节点移动到链表头部。
<code>put</code>	新 key 插入头部；容量满时删除链表尾部，并同步删除哈希表 key。

Listing 2.19 LRU 缓存：哈希定位和链表维护顺序

```
1 class LruCache {
2 public:
```



```

3   explicit LruCache(int capacity) : capacity_(capacity) {}
4
5   int get(int key)
6   {
7       const auto found = this->position_by_key_.find(key);
8       if (found == this->position_by_key_.end()) {
9           return -1;
10      }
11      this->items_.splice(this->items_.begin(), this->items_, found->second);
12      return found->second->value;
13  }
14
15  void put(int key, int value)
16  {
17      if (this->capacity_ == 0) {
18          return;
19      }
20
21      const auto found = this->position_by_key_.find(key);
22      if (found != this->position_by_key_.end()) {
23          found->second->value = value;
24          this->items_.splice(this->items_.begin(), this->items_, found->second);
25          return;
26      }
27
28      if (static_cast<int>(this->items_.size()) == this->capacity_) {
29          const int expired_key = this->items_.back().key;
30          this->position_by_key_.erase(expired_key);
31          this->items_.pop_back();
32      }
33
34      this->items_.push_front(Entry{key, value});
35      this->position_by_key_[key] = this->items_.begin();
36  }
37
38 private:
39  struct Entry {
40      int key = 0;
41      int value = 0;
42  };
43
44  int capacity_ = 0;
45  std::list<Entry> items_;
46  std::unordered_map<int, std::list<Entry>::iterator> position_by_key_;
47 };

```

这个实现的核心不在代码长度，而在职责分离。`position_by_key_` 回答“这个 key 在哪里”，`items_` 回答“谁最新，谁最旧”。`splice` 只改链表链接，不复制节点，所以哈希表里保存的迭代器仍然指向同一个节点。如果用 `vector` 存缓存顺序，把元素移到开头会导致大量搬移，也会破坏保存的位置。这个题是数据结构组合的典型教材案例。

链表变体题也要从节点生命周期解释。复制带随机指针链表时，原节点和新节点之间必须建立映射，否则随机边无法接回；回文链表如果反转后半段，比较后最好恢复原结构，避免修改输入的副作用；排序链表适合自底向上归并，因为链表不支持随机访问，快速排序的分区优势不明显；K 个

一组反转需要先确认剩余节点够 K 个，再局部反转，否则尾部不足一组的节点不能被破坏。这些细节都来自“节点关系是结构本身”。

栈、队列和调度模型

栈和队列不只是简单容器，它们分别表达两类调度策略。栈表达最近上下文优先处理，适合解析嵌套结构、撤销操作、显式 DFS 和单调候选；队列表达先到状态优先处理，适合 BFS、任务缓冲和层次传播；双端队列表达两端都有特殊含义，适合单调窗口、0-1 BFS 和双向搜索。

0-1 BFS 是双端队列的代表。边权只有 0 和 1 时，普通 Dijkstra 用堆当然能做，但双端队列更贴合结构：走 0 权边不增加距离，应该放到队头尽快处理；走 1 权边距离加一，放到队尾。队列里状态的距离仍然近似按非降顺序展开，因此可以得到最短路。

从暴力角度看，带权最短路可以枚举路径，但路径数量会指数级增长；普通 BFS 会把每条边都当成同样代价，遇到 0 权边和 1 权边混在一起时，层数不再等于距离。Dijkstra 的堆版本是通用优化，但在 0/1 边权下，堆顶排序只需要区分“不增加距离”和“距离加一”两类。于是双端队列成为更轻的结构：0 权转移放队头，1 权转移放队尾，保证更小距离的状态先被继续扩展。

图示：0-1 BFS 的双端队列

队头方向	0 权边不增加距离，新状态放到队头，尽快继续扩展。
队尾方向	1 权边让距离加一，新状态放到队尾，排在当前距离层之后。
核心不变量	队列中的候选距离近似非降，先处理的状态不会被更小距离状态越过。
失效条件	边权不是 0 或 1 时，双端队列不再足够，应改用 Dijkstra。

Listing 2.20 0-1 BFS：双端队列表达两类边权

```

1 std::vector<int> zero_one_bfs(
2     const std::vector<std::vector<std::pair<int, int>>>& graph,
3     int source)
4 {
5     constexpr int INF = 1'000'000'000;
6     std::vector<int> distance(graph.size(), INF);
7     std::deque<int> deque;
8
9     distance[static_cast<std::size_t>(source)] = 0;
10    deque.push_front(source);
11
12    while (!deque.empty()) {
13        const int node = deque.front();
14        deque.pop_front();
15
16        for (const auto& [next, weight] : graph[static_cast<std::size_t>(node)]) {
17            const int candidate = distance[static_cast<std::size_t>(node)] + weight;
18            if (candidate >= distance[static_cast<std::size_t>(next)]) {
19                continue;
20            }
21            distance[static_cast<std::size_t>(next)] = candidate;
22            if (weight == 0) {
23                deque.push_front(next);
24            } else {
25                deque.push_back(next);
26            }
27        }
28    }

```

```

29     return distance;
30 }

```

这段代码比普通 BFS 多了一个判断：边权为 0 的状态放前面，边权为 1 的状态放后面。它也比堆版 Dijkstra 少了对数因子。成立条件非常窄：边权必须只有 0 和 1；如果出现权重 2、5 或任意非负数，就该回到 Dijkstra。教材式学习要把这种边界讲清楚，否则学生会把一个漂亮技巧误用到不满足条件的图上。

线性结构题目的讲解标准

一题讲清楚，至少要包含五层。第一层是暴力方法：枚举什么，复杂度为什么高。第二层是瓶颈：重复扫描、重复计算、候选过多、边界分支太多，还是查询能力不足。第三层是结构选择：为什么数组、链表、栈、队列或双端队列能降低瓶颈。第四层是不变量：哪些区间、节点、候选或层次已经正确。第五层是边界条件：空输入、单元素、重复值、负数、环、容量为零、字符集假设和是否允许修改输入。

以滑动窗口最大值为例，暴力方法是每个窗口扫描 K 个元素；瓶颈是相邻窗口大量重叠，却重新求最大；结构选择是双端队列，因为窗口左端会过期，右端会加入新候选；不变量是队列下标从前到后递增，值从前到后单调不增，队头始终在窗口内；边界条件是 K 非法、数组为空、重复元素是否用小于还是小于等于淘汰。只有这些都讲清楚，题解才像教材，而不是模板注释。

以反转链表为例，暴力方法并不是重点，因为它本身已经是线性；重点在结构不变量。循环开始时，`previous` 指向已经反转好的前缀头，`current` 指向尚未处理部分的第一个节点，`current` 之后的链仍保持原方向。每次先保存 `next`，再把当前节点接到已反转前缀前，最后推进两个指针。边界条件是空链表和单节点链表。能用这套语言解释，两两交换和 K 个一组反转就不再是新题。

最后要把线性结构放进复盘表里。数组题复盘区间语义和是否原地修改；字符串题复盘字符集、连续性和子串复制成本；链表题复盘节点身份、前驱和后继保存顺序；栈题复盘保存的是符号、下标、操作数还是上下文；队列题复盘层次、距离和访问标记时机；双端队列题复盘候选淘汰和过期规则。这样复盘一段时间后，你看到新题会先想到访问模式，而不是先搜索记忆里的题号。

数据结构源码视角和容器选择

数据结构学习要同时看抽象接口和存储模型。`vector` 的接口像可变长数组，底层却有大小、容量和连续内存三件事；`deque` 的接口像两端都能高效进出的序列，底层通常是分段连续块；`list` 的接口保证节点稳定和常数移动，但牺牲随机访问和缓存局部性；`unordered_map` 的接口给平均常数查询，底层却受哈希函数、桶数量、负载因子和 rehash 影响。理解这些模型后，刷题时选择容器才不是背名字。

`vector` 的扩容是最值得做的实验。连续 `push_back` 时，`size` 每次加一，`capacity` 只在容量不足时跳变，`data` 地址也可能改变。这个现象解释了为什么保存 `vector` 元素指针或迭代器后继续插入很危险，也解释了 `reserve` 和 `resize` 的区别。`reserve` 改容量，不构造新元素；`resize` 改大小，会构造或销毁元素。很多性能问题不是算法复杂度错，而是无意中触发了大量重新分配和拷贝。

`string` 的成本不能只看语法。`substr` 往往会构造新字符串，动态规划中两层循环里反复 `substr` 可能把复杂度悄悄提高一阶。`string_view` 能表达非拥有视图，但它要求底层字符串生命周期足够长。刷题中为了清晰可以先用 `substr`，复盘时必须知道它的复制成本；工程代码中，如果热路径上大量切片，应该考虑下标范围、视图或 `Trie`。

`unordered_map` 的平均常数时间不是无条件承诺。哈希冲突、负载因子过高、频繁 rehash 都会让性能波动。刷题时调用 `reserve` 可以减少 rehash，使用 `find` 可以避免 `operator[]` 的默认插入副作用。当前缀和计数题里，哈希表语义是历史前缀频次，如果用 `operator[]` 随意读取不存在键，就可能把调试输出和真实状态混在一起。

`map` 和 `set` 基于有序结构，常用于前驱、后继、区间查询和动态有序集合。它们的查询是对数复杂度，不如哈希表平均常数快，但能回答哈希表回答不了的顺序问题。区间覆盖、日程安排、滑动窗口有序统计、离线扫描线，都可能需要有序容器。选择 `map` 不是因为它高级，而是因为题目需要顺序语义。

`priority_queue` 是容器适配器，不是完整排序容器。它只承诺堆顶极值，不承诺遍历顺序。需要频繁删除任意元素时，普通 `priority_queue` 不支持直接删除，常见办法是懒删除：另一个哈希表记

录应该删除的元素，真正取堆顶前不断弹出过期项。理解这个限制，才能写好滑动窗口中位数、任务调度和带过期时间的事件模拟。

Linux 内核的数据结构给了另一种视角。内核常见侵入式链表把 `list_head` 嵌入业务对象，链表节点不拥有对象，只负责把对象串起来。红黑树节点也常嵌入结构体，调度器、内存管理和定时器等模块按自己的键组织对象。与 STL 容器相比，这种设计更强调对象生命周期、内存分配控制和并发保护。刷题不需要手写侵入式链表，但理解它能帮助你解释 LRU、调度队列和缓存淘汰为什么常把索引结构与业务对象分离。

读源码时不要从模板细节硬啃。先读接口语义和复杂度承诺，再找核心成员，最后追一条热路径。vector 看 `push_back` 和扩容，deque 看段表如何定位下标，unordered_map 看查找和 rehash，map 看 `lower_bound` 如何沿树下降，priority_queue 看 `push_heap` 和 `pop_heap`。每次阅读只回答一个问题：这个 API 为什么是这个复杂度，什么时候会让迭代器失效，题目里用它是否合适。

专题复盘要求

读完本节后，不要只记结论。请把每个结论还原成一个可验证的问题：它解决了哪一步重复工作，依赖哪个题目条件，使用哪个容器或状态，边界条件如何破坏错误写法。

第 3 章

非线性数据结构：哈希表、堆、树、图和并查集

线性结构擅长处理顺序，非线性结构擅长处理关系和查询。哈希表回答“这个状态以前出现过吗”；堆回答“当前最极端的候选是谁”；树表达层次、有序性和递归子问题；图表达任意状态之间的连接；并查集维护“这些元素现在是否属于同一组”。这些结构不是刷题技巧的附属品，而是算法优化的来源。很多题从 $O(n^2)$ 变成 $O(n)$ ，并不是因为循环少写了一层，而是因为某个昂贵查询被换成了合适的数据结构查询。

哈希表是保存历史信息的核心结构。`std::unordered_map` 和 `std::unordered_set` 的平均查询、插入、删除复杂度是 $O(1)$ ，适合把“回头查找”变成“即时查询”。两数之和的暴力方法枚举两个下标，慢在每个数都要向后找补数；哈希表版本边扫描边记录已经见过的值，当前数只需要查询一次补数。这个优化成立的前提是题目只要求找到一对下标，且补数是否出现过可以由历史集合回答。

Listing 3.1 两数之和：哈希表把补数查询变成平均常数时间

```
1 std::vector<int> two_sum(const std::vector<int>& nums, int target)
2 {
3     std::unordered_map<int, int> index_by_value;
4     index_by_value.reserve(nums.size());
5
6     for (int i = 0; i < static_cast<int>(nums.size()); ++i) {
7         const int complement = target - nums[i];
8         const auto found = index_by_value.find(complement);
9         if (found != index_by_value.end()) {
10             return {found->second, i};
11         }
12         index_by_value.emplace(nums[i], i);
13     }
14
15     return {};
16 }
```

代码里先查再插入，是为了避免同一个元素被用两次。例如目标是 6，当前数是 3，如果先插入再查，可能错误地返回同一个下标。这里用 `find` 而不是 `operator[]`，因为 `operator[]` 在 key 不存在时会插入默认值。只想判断存在性时，C++20 可以用 `contains`；需要同时取值时，用 `find`，这样只查一次。刷题中还应该习惯 `reserve`，它能减少 rehash 次数。rehash 会重新组织桶，导致迭代器失效；如果保存了哈希表元素迭代器，扩容后不能继续使用。

哈希表不仅能存值到下标，也能存频次、前缀和、状态编码和访问标记。前缀和加哈希表很重要的一类优化。和为 K 的子数组如果暴力枚举左右端点，需要 $O(n^2)$ ；如果知道当前位置前缀和是 `current`，那么以当前位置结尾、和为 k 的子数组数量，等于以前出现过多少次 `current - k`。哈希表保存的是“某个前缀和出现次数”，查询的是“能和当前前缀配成目标的历史前缀”。

Listing 3.2 和为 K 的子数组：前缀和频次

```
1 int subarray_sum_equals_k(const std::vector<int>& nums, int k)
2 {
3     std::unordered_map<int, int> prefix_count;
4     prefix_count.reserve(nums.size() + 1);
5     prefix_count.emplace(0, 1);
```



```

6
7     int prefix = 0;
8     int answer = 0;
9     for (const int x : nums) {
10         prefix += x;
11         const auto found = prefix_count.find(prefix - k);
12         if (found != prefix_count.end()) {
13             answer += found->second;
14         }
15         ++prefix_count[prefix];
16     }
17
18     return answer;
19 }

```

这个算法容易错在初始化。空前缀和 0 必须先出现一次，否则从下标 0 开始的合法子数组会漏掉。它也解释了为什么有负数时不能随使用滑动窗口：窗口和不再随右端单调增加，左端移动也不保证更接近目标；前缀和加哈希表不依赖正数单调性，所以适用范围更广。

哈希表的限制同样要清楚。第一，它不维护顺序，如果题目要求按排序顺序取最小、最大或区间，就该考虑 `std::map`、`std::set`、堆、树状数组或线段树。第二，平均 $O(1)$ 不等于绝对 $O(1)$ ，糟糕哈希或恶意数据可能退化。第三，自定义 key 需要提供相等比较和哈希函数。面试里一般不会刻意卡哈希攻击，但你要知道 `unordered_map<pair<int,int>, int>` 不能直接在所有标准库环境下工作，需要自定义哈希。

Listing 3.3 为 pair 坐标提供哈希函数

```

1 struct PairHash {
2     std::size_t operator()(const std::pair<int, int>& point) const noexcept
3     {
4         constexpr unsigned COORDINATE_BITS = 32U;
5         const auto first = static_cast<std::uint64_t>(
6             static_cast<std::uint32_t>(point.first));
7         const auto second = static_cast<std::uint64_t>(
8             static_cast<std::uint32_t>(point.second));
9         return std::hash<std::uint64_t>{}((first << COORDINATE_BITS) ^ second);
10    }
11 };
12
13 std::unordered_set<std::pair<int, int>, PairHash> visited;

```

堆解决的是动态极值问题。C++ 里通常使用 `std::priority_queue`，默认是大顶堆；如果要小顶堆，需要显式给出容器和比较器。堆的插入和弹出堆顶都是 $O(\log n)$ ，查看堆顶是 $O(1)$ 。它不适合频繁删除任意元素，也不适合遍历出有序结果而不破坏结构。换句话说，堆只承诺“我能很快告诉你当前最大或最小是谁”。

Listing 3.4 `priority_queue` 的大顶堆和小顶堆

```

1 std::priority_queue<int> max_heap;
2
3 std::priority_queue<int,
4                     std::vector<int>,
5                     std::greater<int>> min_heap;
6
7 max_heap.push(3);

```

```

8 max_heap.push(10);
9 const int largest = max_heap.top();
10 max_heap.pop();

```

Top K 是堆最经典的用法。求数组中第 K 大元素，直觉做法是排序，复杂度 $O(n \log n)$ 。如果只关心前 K 大，不需要完整排序，可以维护一个大小为 K 的小顶堆。堆顶是当前前 K 大中最小的元素；新元素如果不超过堆顶，就不可能进入前 K；如果超过堆顶，就弹出堆顶并插入新元素。最终堆顶就是第 K 大。

Listing 3.5 第 K 大元素：大小为 K 的小顶堆

```

1 int find_kth_largest(const std::vector<int>& nums, int k)
2 {
3     std::priority_queue<int,
4                         std::vector<int>,
5                         std::greater<int>>> heap;
6
7     for (const int x : nums) {
8         if (static_cast<int>(heap.size()) < k) {
9             heap.push(x);
10        } else if (x > heap.top()) {
11            heap.pop();
12            heap.push(x);
13        }
14    }
15
16    return heap.top();
17 }

```

这个方法的复杂度是 $O(n \log k)$ ，适合 k 远小于 n 的场景。还有一种平均线性复杂度的快速选择算法，后文会讲。面试表达时要说明选择堆的理由：不需要完整有序，只需要维护 K 个候选的边界。类似地，合并 K 个升序链表用小顶堆保存每条链表当前头节点；任务调度用堆保存当前最早结束或最高优先级任务；数据流中位数用一个大顶堆和一个小顶堆分别维护较小一半和较大一半。

树把数据组织成层次关系。二叉树题看似在考递归，实际在考状态如何沿边传播。状态可以从父节点传给子节点，例如验证二叉搜索树时传递取值上下界；也可以从子节点汇总到父节点，例如求树的高度、直径、最大路径和；还可以按层从左到右传播，例如层序遍历。只背前序、中序、后序三个词是不够的，必须知道遍历顺序服务于哪类状态。

Listing 3.6 二叉搜索树验证：显式栈中序遍历

```

1 bool is_valid_bst(TreeNode* root)
2 {
3     std::vector<TreeNode*> stack;
4     TreeNode* current = root;
5     std::optional<long long> previous;
6
7     while (current != nullptr || !stack.empty()) {
8         while (current != nullptr) {
9             stack.push_back(current);
10            current = current->left;
11        }
12
13        current = stack.back();
14        stack.pop_back();

```

```

15
16     if (previous.has_value() && current->value <= previous.value()) {
17         return false;
18     }
19     previous = current->value;
20     current = current->right;
21 }
22
23 return true;
24 }

```

二叉搜索树的关键性质是中序遍历严格递增。验证 BST 的暴力想法可能是对每个节点扫描整棵左子树和右子树，判断左边都小、右边都大，这会重复访问大量节点。中序遍历把全局约束转成相邻访问值的递增关系，每个节点只处理一次。这里用显式栈而不是递归，是为了避免极端深树导致 C++ 调用栈过深，同时也让“遍历状态”更清楚。

树题经常需要区分“节点为空的返回值”和“答案的初始值”。求最大深度时，空节点深度是 0；求最大路径和时，答案不能初始化为 0，因为所有节点可能都是负数；求最近公共祖先时，返回值表示“当前子树是否找到了目标节点或祖先”。这些不是小细节，而是树形 DP 的语义。每个返回值都应该能用一句话解释：它代表当前子树对父节点贡献什么信息。

图是最通用的关系结构。只要有状态和一步转移，就可以建模成图。图题最重要的不是 DFS 或 BFS 写法，而是建模四问：节点是什么，边是什么，访问状态是什么，队列或栈里保存什么。岛屿数量里，节点是陆地格子，边是上下左右相邻；课程表里，节点是课程，有向边表示先修依赖；单词接龙里，节点是单词，边表示一次字符变化；开锁问题里，节点是四位密码状态，边是转动一位数字。

图的存储方式要跟输入规模匹配。稠密图可以用邻接矩阵，判断两点是否有边是 $O(1)$ ，但空间是 $O(n^2)$ 。大多数面试题是稀疏图，更适合邻接表，空间是 $O(n + m)$ 。如果节点编号连续，用 `vector<vector<int>>`；如果节点是字符串或稀疏编号，用哈希表映射到编号，或者直接用 `unordered_map<string, vector<string>>`。

Listing 3.7 课程表：邻接表和入度数组

```

1 bool can_finish(int num_courses,
2                 const std::vector<std::vector<int>>& prerequisites)
3 {
4     std::vector<std::vector<int>> graph(num_courses);
5     std::vector<int> indegree(num_courses, 0);
6
7     for (const auto& edge : prerequisites) {
8         const int course = edge[0];
9         const int prerequisite = edge[1];
10        graph[prerequisite].push_back(course);
11        ++indegree[course];
12    }
13
14    std::queue<int> queue;
15    for (int course = 0; course < num_courses; ++course) {
16        if (indegree[course] == 0) {
17            queue.push(course);
18        }
19    }
20
21    int learned = 0;
22    while (!queue.empty()) {
23        const int course = queue.front();

```

```

24     queue.pop();
25     ++learned;
26
27     for (const int next : graph[course]) {
28         --indegree[next];
29         if (indegree[next] == 0) {
30             queue.push(next);
31         }
32     }
33 }
34
35 return learned == num_courses;
36 }

```

拓扑排序的暴力直觉是反复扫描所有课程，找当前没有前置依赖的课。这样每学一门课都可能重新扫描所有边。入度数组把“还有多少依赖没被满足”变成可增量更新的状态；队列保存当前入度为 0 的课程。每条边只在它的起点被移除时处理一次，复杂度 $O(n+m)$ 。如果最后学完的课程数少于总数，说明剩余节点互相依赖，图中有环。

DFS 和 BFS 都可以遍历图，但语义不同。BFS 适合无权最短步数和层次扩散，因为它按距离递增访问状态；DFS 适合连通块标记、路径搜索、拓扑状态检测和回溯枚举，因为它沿一条路径深入。C++ 里递归 DFS 简洁，但深度过大可能栈溢出。本书讲递归思想，但实现上会经常给出显式栈版本，让遍历过程、访问标记和状态回退更加可控。

Listing 3.8 岛屿数量：显式栈标记连通块

```

1 int num_islands(std::vector<std::vector<char>>& grid)
2 {
3     if (grid.empty() || grid[0].empty()) {
4         return 0;
5     }
6
7     constexpr char LAND = '1';
8     constexpr char WATER = '0';
9     constexpr std::array<std::pair<int, int>, 4> DIRECTIONS{
10         std::pair{1, 0}, std::pair{-1, 0},
11         std::pair{0, 1}, std::pair{0, -1}
12     };
13
14     const int rows = static_cast<int>(grid.size());
15     const int cols = static_cast<int>(grid[0].size());
16     int islands = 0;
17
18     for (int row = 0; row < rows; ++row) {
19         for (int col = 0; col < cols; ++col) {
20             if (grid[row][col] != LAND) {
21                 continue;
22             }
23
24             ++islands;
25             std::vector<std::pair<int, int>> stack{{row, col}};
26             grid[row][col] = WATER;
27
28             while (!stack.empty()) {

```

```

29         const auto [r, c] = stack.back();
30         stack.pop_back();
31
32         for (const auto [dr, dc] : DIRECTIONS) {
33             const int nr = r + dr;
34             const int nc = c + dc;
35             if (nr < 0 || nr >= rows || nc < 0 || nc >= cols ||
36                 grid[nr][nc] != LAND) {
37                 continue;
38             }
39             grid[nr][nc] = WATER;
40             stack.emplace_back(nr, nc);
41         }
42     }
43 }
44 }
45
46 return islands;
47 }

```

这段代码把访问过的陆地直接改成水，省掉单独的 `visited` 数组。这样做是否合适取决于题目是否允许修改输入。如果不允许，就创建同样大小的布尔矩阵。图题里访问标记必须在入队或入栈时就设置，而不是弹出时才设置；否则同一个节点可能被多个邻居重复加入，轻则浪费时间，重则导致状态爆炸。

并查集维护动态连通性。它有两个核心操作：`find` 找到一个元素所在集合的代表，`unite` 合并两个集合。路径压缩让查找时顺手把沿途节点直接挂到根上，按大小或按秩合并让树保持较浅。两者结合后，单次操作的均摊复杂度接近常数。它适合回答“是否在同一组”“合并后还有几个连通块”，不适合回答两点之间的具体路径或最短距离。

Listing 3.9 并查集：路径压缩与按大小合并

```

1 class DisjointSetUnion {
2 public:
3     explicit DisjointSetUnion(int n)
4         : parent_(n), size_(n, 1)
5     {
6         std::iota(this->parent_.begin(), this->parent_.end(), 0);
7     }
8
9     int find(int x)
10    {
11        int root = x;
12        while (this->parent_[root] != root) {
13            root = this->parent_[root];
14        }
15        while (this->parent_[x] != x) {
16            const int next = this->parent_[x];
17            this->parent_[x] = root;
18            x = next;
19        }
20        return root;
21    }
22 }

```



```

23     bool unite(int a, int b)
24     {
25         int root_a = find(a);
26         int root_b = find(b);
27         if (root_a == root_b) {
28             return false;
29         }
30         if (this->size_[root_a] < this->size_[root_b]) {
31             std::swap(root_a, root_b);
32         }
33         this->parent_[root_b] = root_a;
34         this->size_[root_a] += this->size_[root_b];
35         return true;
36     }
37
38 private:
39     std::vector<int> parent_;
40     std::vector<int> size_;
41 };

```

冗余连接是并查集的标准例题。给一棵树额外加一条边，依次扫描边。如果一条边连接的两个点已经在同一集合里，那么再加入它就会形成环，这条边就是冗余边。暴力方法可以每次加边后做一次 DFS 检查连通性，复杂度很高；并查集把“两个点是否已经连通”维护成近乎常数时间查询。

账户合并也能体现并查集的建模能力。邮箱是连接账户的证据：两个账户只要共享一个邮箱，就属于同一个人。可以把账户编号作为并查集元素，用哈希表记录邮箱第一次出现在哪个账户；再次看到同一邮箱时，把两个账户合并。最后按根节点收集邮箱并排序。这里哈希表负责从字符串邮箱找到账户编号，并查集负责维护账户之间的连通分量。一个题常常需要多个数据结构协作，这也是“数据结构和算法不分家”的真实含义。

数据结构和算法不分家

优化通常来自查询模型的改变。哈希表把历史查找变快，堆把动态极值变快，树把层次状态和有序性质显式化，图把状态转移统一成边，并查集把动态连通性维护变快。刷题复盘时必须同时写出算法动作和数据结构职责：保存什么，支持什么查询，删除什么过期状态，为什么不会漏答案。

本章建议配套练习：哈希表做 1、49、128、560、146；堆做 215、347、23、295、621；树做 94、98、102、104、105、124、236、230；图做 200、207、210、417、994、127、752；并查集做 547、684、721、990、1319。每道题至少复盘一次建模：如果不用这个结构，暴力瓶颈在哪里；用了这个结构，哪一步查询或合并被降下来了；结构本身有什么边界和失效条件。

非线性结构的教材级展开

非线性结构解决的是“关系查询”而不只是“顺序扫描”。哈希表把对象映射到桶，堆把候选组织成局部有序的完全二叉树，平衡树维护全局有序关系，Trie 把字符串前缀共享起来，树状数组和线段树把区间信息按层聚合，图把任意状态连接成网络，并查集把动态连通关系压缩成集合代表。经典教材会把这些结构讲得很细，因为它们不仅出现在面试题里，也出现在编译器符号表、数据库索引、操作系统调度、网络路由、文件系统目录和缓存系统中。

学习非线性结构要警惕一个误区：只记复杂度表。复杂度表只告诉你某个操作大概多快，却没有告诉你结构为什么能这么快、在哪些操作上不擅长、变体题需要保存哪些额外信息。比如哈希表平均查询常数，但无法找前驱后继；堆能快速取极值，但不能快速删除任意元素；平衡树能维护有序，但常数比数组扫描大；并查集能判断同组，却不能告诉你最短路径。高质量题解必须把能力边界讲清楚。

哈希表：桶、冲突、负载因子和历史状态

哈希表的核心思想是把 key 通过哈希函数映射到桶。理想情况下，key 均匀分布到桶中，每个桶里元素很少，查询接近常数时间。实际实现需要处理冲突：常见方式有链地址法和开放寻址。C++ 的 `std::unordered_map` 抽象上承诺平均常数时间，具体实现通常围绕桶数组、节点和负载因子展开。当元素数量相对桶数量过高时，容器会 rehash，重新分配桶并把元素放到新桶里。

刷题中，哈希表最常见的职责是保存历史状态。两数之和保存历史值到下标，前缀和题保存历史前缀出现次数，最长连续序列保存所有值的存在性，快乐数保存已经出现过的状态，克隆图保存原节点到新节点的映射。虽然都叫哈希表，保存内容完全不同：有时保存布尔存在，有时保存次数，有时保存最早位置，有时保存对象指针。题目目标决定哈希表的 value 类型。

同一题型的变体常常只改哈希表语义。和为 K 的子数组问数量，哈希表保存前缀和频次；问最长长度，保存前缀和最早出现位置；问是否存在长度至少二的倍数子数组，保存余数最早出现位置；问最短且允许负数，则普通哈希不够，需要前缀和加单调队列。复盘时不能只写“用哈希表”，必须写“哈希表保存什么历史信息”。

Listing 3.10 最长和为 K 的子数组：保存最早前缀位置

```
1 int longest_subarray_sum_k(const std::vector<int>& nums, int target)
2 {
3     std::unordered_map<long long, int> first_index;
4     first_index.reserve(nums.size() + 1);
5     first_index.emplace(0LL, -1);
6
7     long long prefix = 0;
8     int answer = 0;
9     for (int i = 0; i < static_cast<int>(nums.size()); ++i) {
10         prefix += nums[i];
11         const auto found = first_index.find(prefix - target);
12         if (found != first_index.end()) {
13             answer = std::max(answer, i - found->second);
14         }
15         first_index.emplace(prefix, i);
16     }
17     return answer;
18 }
```

这里使用 `emplace` 而不是直接赋值，是为了保留某个前缀和第一次出现的位置。要最长区间，左端越早越好；如果每次更新为最新位置，就会把潜在更长答案破坏掉。这说明同样是前缀和哈希，计数题、最长题、最短题的 value 语义完全不同。

哈希表的实际应用远超刷题。编译器符号表用哈希表从名字找到声明信息；解释器运行时环境用哈希表表示对象属性；缓存系统用哈希表定位缓存项；数据库执行层用哈希表做 hash join；操作系统内核用哈希表或类似结构快速定位对象。应用越工程化，越要关心哈希函数、内存分配、并发访问和迭代器失效。刷题可以先掌握模型，进阶时必须知道平均常数背后的条件。

堆和优先队列：动态极值、懒删除和多路归并

二叉堆通常用数组表示完全二叉树。下标 i 的左右孩子在 $2*i+1$ 和 $2*i+2$ ，父节点在 $(i-1)/2$ 。大顶堆要求父节点不小于孩子，小顶堆要求父节点不大于孩子。堆不是完全有序结构，只保证堆顶是全局极值。插入时向上调整，弹出堆顶时把末尾元素放到根再向下调整，复杂度都是 $O(\log n)$ 。

优先队列适合“候选集合动态变化，但每次只需要当前最优候选”的问题。Top K、合并 K 个有序链表、数据流中位数、任务调度、Dijkstra、最小区间覆盖 K 个列表、IPO 项目选择都属于这一类。堆不适合的场景是频繁查找或删除任意元素，因为普通 `priority_queue` 只能访问堆顶。

懒删除是堆的重要变体。滑动窗口中位数需要删除窗口左端过期元素，但堆无法直接删除任意元素。常见做法是用哈希表记录某个值应该被删除的次数，真正访问堆顶前，不断弹出已标记删除

的堆顶。这种策略把任意删除延迟到元素成为堆顶时执行，保持堆接口简单。Dijkstra 中的过期距离也是同样思想：不更新堆中旧节点，而是压入新距离，弹出时检查是否过期。

Listing 3.11 堆中旧状态懒跳过：Dijkstra 的核心形态

```

1 std::vector<long long> dijkstra(
2     const std::vector<std::vector<std::pair<int, int>>>& graph,
3     int source)
4 {
5     constexpr long long INF = 4'000'000'000'000'000'000LL;
6     using State = std::pair<long long, int>;
7
8     std::vector<long long> distance(graph.size(), INF);
9     std::priority_queue<State, std::vector<State>, std::greater<State>> heap;
10    distance[static_cast<std::size_t>(source)] = 0;
11    heap.emplace(0, source);
12
13    while (!heap.empty()) {
14        const auto [current_distance, node] = heap.top();
15        heap.pop();
16        if (current_distance != distance[static_cast<std::size_t>(node)]) {
17            continue;
18        }
19
20        for (const auto& [next, weight] : graph[static_cast<std::size_t>(node)]) {
21            const long long candidate = current_distance + weight;
22            if (candidate >= distance[static_cast<std::size_t>(next)]) {
23                continue;
24            }
25            distance[static_cast<std::size_t>(next)] = candidate;
26            heap.emplace(candidate, next);
27        }
28    }
29    return distance;
30 }

```

堆的变种包括二叉堆、d 叉堆、可并堆、斐波那契堆、索引堆和双堆。面试中通常只需要二叉堆和双堆。双堆维护中位数时，一个大顶堆保存较小一半，一个小顶堆保存较大一半；不变量是大顶堆所有值不大于小顶堆所有值，两个堆大小差不超过一。很多错误实现只维护大小不变量，忘了顺序不变量，最终中位数会错。

有序结构：平衡树、前驱后继和区间维护

哈希表不能回答顺序问题。若题目需要找小于等于某值的最大元素、大于等于某值的最小元素、动态维护区间、按时间顺序查询，通常要考虑有序结构。C++ 的 `std::map` 和 `std::set` 通常由红黑树实现，插入、删除、查询和 `lower_bound` 都是 $O(\log n)$ 。它们常数比哈希表大，但能提供顺序语义。

有序结构典型应用包括日程安排、区间合并、滑动窗口内有序统计、在线前驱后继、时间戳键值存储、离线扫描线。比如 My Calendar 类问题中，要插入一个新区间，需要找到第一个起点不小于新区间起点的区间，再检查它和前一个区间是否冲突。哈希表无法回答“相邻区间是谁”，有序 map 正好能回答。

Listing 3.12 有序 map 维护互不重叠区间

```

1 class IntervalSet {

```

```

2 public:
3     bool add(int left, int right)
4     {
5         auto next = this->ranges_.lower_bound(left);
6         if (next != this->ranges_.end() && next->first < right) {
7             return false;
8         }
9         if (next != this->ranges_.begin()) {
10             auto previous = std::prev(next);
11             if (previous->second > left) {
12                 return false;
13             }
14         }
15         this->ranges_.emplace(left, right);
16         return true;
17     }
18
19 private:
20     std::map<int, int> ranges_;
21 };

```

这段代码体现了有序结构的独特能力：通过 `lower_bound` 直接定位可能冲突的相邻区间，而不是扫描所有区间。变体包括闭区间和半开区间的端点差异、允许重叠次数最多为二、插入后合并相邻区间、查询某点被多少区间覆盖。每个变体都要先定义端点语义，再决定比较条件里的等号。

Trie：前缀共享和字符串集合

Trie 把字符串集合组织成字符边组成的树。根到某节点的路径表示一个前缀，节点上的标记表示是否有单词在这里结束。它适合前缀查询、多模式搜索、自动补全、字典树剪枝和按位 Trie 最大异或。相比哈希表，Trie 的优势不是单个完整字符串查询更快，而是前缀共享和前缀剪枝。

实现 Trie 时要根据字符集选择孩子结构。小写英文可以用 `array<int, 26>` 或指针数组；字符集大时可以用哈希表保存孩子；若内存敏感，可以用压缩 Trie 或把节点存进 `vector` 里用整数下标引用。刷题中，单词搜索 II 是 Trie 的代表：如果对每个单词单独 DFS，会重复遍历棋盘；Trie 合并所有单词前缀后，DFS 一旦发现当前前缀不存在，就可以立即剪枝。

Listing 3.13 Trie 节点用数组孩子表达小写字母前提

```

1 class Trie {
2 public:
3     void insert(const std::string& word)
4     {
5         int node = 0;
6         for (const char ch : word) {
7             const int index = ch - 'a';
8             if (this->nodes_[static_cast<std::size_t>(node)].next[index] == -1) {
9                 this->nodes_[static_cast<std::size_t>(node)].next[index] =
10                     static_cast<int>(this->nodes_.size());
11                 this->nodes_.push_back(Node{});
12             }
13             node = this->nodes_[static_cast<std::size_t>(node)].next[index];
14         }
15         this->nodes_[static_cast<std::size_t>(node)].terminal = true;
16     }
17 };

```

```

18 bool starts_with(const std::string& prefix) const
19 {
20     int node = 0;
21     for (const char ch : prefix) {
22         const int index = ch - 'a';
23         node = this->nodes_[static_cast<std::size_t>(node)].next[index];
24         if (node == -1) {
25             return false;
26         }
27     }
28     return true;
29 }
30
31 private:
32 struct Node {
33     std::array<int, 26> next{};
34     bool terminal = false;
35
36     Node()
37     {
38         this->next.fill(-1);
39     }
40 };
41
42 std::vector<Node> nodes_{Node{}};
43 };

```

这里使用整数下标而不是裸指针，可以避免手动释放节点，也让节点存储更紧凑。代价是 `vector` 扩容时如果外部保存了节点引用会失效，所以代码只保存下标。字符集假设同样必须说明：输入如果不是小写英文，`ch - 'a'` 就不安全。

树状数组和线段树：区间聚合和动态更新

前缀和适合静态数组区间求和，但如果数组中间会更新，普通前缀和每次更新都要影响后面所有位置。树状数组和线段树解决的是“动态更新 + 区间查询”。树状数组适合可逆前缀聚合，代码短、常数小；线段树更通用，可以维护区间最大、最小、和、懒标记、复杂合并信息。

树状数组把每个位置负责的一段长度设置为最低位一的大小。更新一个点时，沿着 `index += lowbit(index)` 更新所有包含它的树节点；查询前缀时，沿着 `index -= lowbit(index)` 汇总若干不重叠块。它常用于逆序对、动态前缀和、离散化后的频次排名。

Listing 3.14 树状数组：点更新和前缀查询

```

1 class FenwickTree {
2 public:
3     explicit FenwickTree(int n) : tree_(static_cast<std::size_t>(n + 1), 0) {}
4
5     void add(int index, long long delta)
6     {
7         for (int i = index + 1; i < static_cast<int>(this->tree_.size());
8             i += i & -i) {
9             this->tree_[static_cast<std::size_t>(i)] += delta;
10        }
11    }
12

```



```

13     long long prefix_sum(int count) const
14     {
15         long long result = 0;
16         for (int i = count; i > 0; i -= i & -i) {
17             result += this->tree_[static_cast<std::size_t>(i)];
18         }
19         return result;
20     }
21
22 private:
23     std::vector<long long> tree_;
24 };

```

这段代码使用外部零基下标，内部一基下标。外部 `add(index, delta)` 更新第 `index` 个元素，`prefix_sum(count)` 查询前 `count` 个元素之和。把接口设计成这种语义，可以减少 `+1` 和 `-1` 在业务代码里到处出现。

线段树把数组按区间递归划分成二叉树。递归概念容易理解，但 C++ 实现可以用迭代线段树避免调用栈。叶子放在底层，父节点保存两个孩子的合并结果。点更新时从叶子改起一路向上重算；区间查询时左右边界向上跳，收集覆盖查询区间的若干节点。它适合动态区间最值、区间和、扫描线、最长连续段、括号匹配信息等。

图表示：邻接矩阵、邻接表、边表和状态图

图结构的核心是节点和边。邻接矩阵适合点数小且边很多的图，判断两点是否相邻是常数，但空间是 $O(n^2)$ 。邻接表适合稀疏图，空间 $O(n+m)$ ，遍历某点邻居高效。边表适合 Kruskal、Bellman-Ford 和离线处理，因为算法按边扫描而不是按点找邻居。网格图可以不显式建图，用方向数组现场生成邻居。

建图题最容易错在边方向和状态维度。课程表中，如果边从课程指向先修课，拓扑输出顺序就和学习顺序相反；单词接龙中，如果每次找邻居都扫描所有单词，BFS 外层正确也会超时；障碍消除最短路中，状态不是位置，而是位置加剩余消除次数；访问所有节点的最短路径中，状态是当前位置加已访问集合。状态图的节点可能不是输入中的一个对象，而是多个变量的组合。

图结构在实际系统中非常常见。编译器构建控制流图和调用图，包管理器维护依赖图，数据库查询优化器处理算子图，网络路由维护拓扑图，操作系统调度和资源等待可以形成等待图。刷题中的 BFS、拓扑排序和最短路，是这些系统问题的简化模型。

并查集：等价关系、扩展权值和回滚

并查集维护的是等价关系。每个集合有一个代表元，`find` 返回代表，`unite` 把两个代表连接起来。路径压缩降低未来查找成本，按大小合并避免树退化。它适合动态合并、判断同组、统计连通块、检测无向环和 Kruskal 最小生成树。

并查集的变体很多。带权并查集维护节点到父节点的权值关系，可用于除法求值、相对距离和势能差；奇偶并查集维护节点到根的奇偶关系，可用于二分图约束和敌友关系；带大小并查集维护集合规模；回滚并查集支持撤销合并，常用于离线动态连通性。每个变体都遵循同一原则：合并代表时，额外信息必须同步更新到新的父子关系上。

Listing 3.15 并查集统计连通块数量

```

1 class ComponentSet {
2 public:
3     explicit ComponentSet(int n)
4         : parent_(static_cast<std::size_t>(n)),
5           size_(static_cast<std::size_t>(n), 1),
6           components_(n)
7     {

```

```

8      std::iota(this->parent_.begin(), this->parent_.end(), 0);
9  }
10
11  bool unite(int lhs, int rhs)
12  {
13      int root_lhs = this->find(lhs);
14      int root_rhs = this->find(rhs);
15      if (root_lhs == root_rhs) {
16          return false;
17      }
18      if (this->size_[static_cast<std::size_t>(root_lhs)] <
19          this->size_[static_cast<std::size_t>(root_rhs)]) {
20          std::swap(root_lhs, root_rhs);
21      }
22      this->parent_[static_cast<std::size_t>(root_rhs)] = root_lhs;
23      this->size_[static_cast<std::size_t>(root_lhs)] +=
24          this->size_[static_cast<std::size_t>(root_rhs)];
25      --this->components_;
26      return true;
27  }
28
29  int components() const { return this->components_; }
30
31 private:
32  int find(int x)
33  {
34      int root = x;
35      while (this->parent_[static_cast<std::size_t>(root)] != root) {
36          root = this->parent_[static_cast<std::size_t>(root)];
37      }
38      while (this->parent_[static_cast<std::size_t>(x)] != x) {
39          const int next = this->parent_[static_cast<std::size_t>(x)];
40          this->parent_[static_cast<std::size_t>(x)] = root;
41          x = next;
42      }
43      return root;
44  }
45
46  std::vector<int> parent_;
47  std::vector<int> size_;
48  int components_ = 0;
49 };

```

这段代码把连通块数量作为并查集状态维护。每次成功合并两个不同集合，连通块数减一；重复合并不改变数量。省份数量、网络连通操作次数、岛屿动态添加都可以使用这个模式。若题目需要知道集合内最大值、边数或权值差，只需要在根节点上维护额外数组，并在合并时更新。

非线性结构的应用谱和实验

哈希表应用：两数之和、字母异位词分组、前缀和计数、克隆图、LRU 索引、字符串到编号映射。堆应用：Top K、动态中位数、Dijkstra、多路归并、任务调度、最小区间。平衡树应用：日程安排、前驱后继、动态区间、滑动窗口有序统计。Trie 应用：前缀搜索、单词搜索 II、最大异或、敏感词过滤。树状数组和线段树应用：逆序对、区间和、扫描线、动态排名、范围更新。图应用：依赖分析、最短路、连通块、拓扑排序。并查集应用：动态连通、冗余连接、账户合并、等式约束、最小生成树。

非线性结构实验：第一，给 `unordered_map` 设置不同 `reserve`，统计 rehash 次数和运行时间。第二，用完整排序、大小为 K 的堆、快速选择分别求第 K 大，比较不同 K 下的表现。第三，用 `map` 实现日程安排，再用无序表尝试，解释为什么无序表无法找相邻区间。第四，用树状数组统计逆序对，并手算离散化后的更新和查询。第五，把并查集的路径压缩关掉，构造链式合并，观察查找路径长度变化。

非线性结构选型

需要历史精确查询，用哈希表；需要当前极值，用堆；需要前驱后继或动态有序，用平衡树；需要大量前缀共享，用 Trie；需要动态前缀或区间聚合，用树状数组或线段树；需要状态转移和可达性，用图；只需要合并集合和判断同组，用并查集。每次选型都要同时说明它不能做什么，避免把结构能力无限放大。

非线性结构变种题谱

哈希表变种可以按 value 语义分类。保存下标：两数之和、同构字符串、最长无重复子串的上次出现位置。保存次数：异位词分组、前缀和计数、优美子数组、频率栈。保存最早位置：最长和为 K 、连续子数组和、最长等量零一子数组。保存集合代表：并查集邮箱映射、字符串编号、图克隆映射。保存对象迭代器：LRU 缓存、全 $O(1)$ 数据结构。题解里必须写清 value 语义，否则变体一来就会把次数、位置和布尔存在混用。

堆变种可以按“极值来源”分类。固定大小堆维护 Top K ，双堆维护中位数，多路堆维护 K 个有序流的当前头，带过期堆处理滑动窗口和 Dijkstra 旧状态，贪心堆在任务调度和 IPO 中选择当前可执行最优项目。每类堆都要补一个过期或比较器问题：Top K 要说明堆顶代表边界；双堆要说明大小和平衡；多路堆要说明堆元素里保存来自哪一路；带过期堆要说明过期检查发生在使用堆顶之前。

平衡树和有序结构变种按查询目的分类。前驱后继类：存在重复元素 III、时间键值存储、最近座位。区间维护类：我的日程安排、插入区间、合并数据流区间。排名统计类：逆序对、区间和个数、滑动窗口中位数。扫描线类：会议室、天际线、矩形面积。若只需要存在性，哈希表更简单；一旦需要“最近的更大/更小”“第 k 个”“区间相邻关系”，就要转向有序结构、树状数组或线段树。

Trie 变种按边的含义分类。字符 Trie 用于单词前缀和字典匹配；反向 Trie 用于后缀查询和流式字符匹配；压缩 Trie 用于减少长单链空间；二进制 Trie 用于最大异或、按位贪心和网络前缀匹配；AC 自动机把 Trie 和失败指针结合，用于多模式匹配。刷题中最常见的是字符 Trie 和二进制 Trie。二进制 Trie 的边是 0/1 位，不是字符；每次为了最大异或，优先走当前位相反的分支。

树状数组和线段树变种按更新和查询分类。点更新加区间查询是最基础形态；区间更新加点查询可以用差分思想；区间更新加区间查询需要懒标记或两个树状数组；区间最大值、最小值、最大子段和、最长连续一都需要自定义合并信息。线段树难点不在模板长，而在节点信息能否合并。只要父节点信息可以由左右孩子合成，就可能使用线段树；如果合并依赖更远上下文，就需要换建模。

图变种按边权和状态分类。无权图最短路用 BFS，零一权图用 0-1 BFS，非负权图用 Dijkstra，有负权边但无负环可用 Bellman-Ford 或 SPFA 思想，有向无环图可用拓扑 DP。状态图题要明确状态维度：开锁是四位字符串，访问所有节点是节点加集合，障碍消除是位置加剩余次数，带钥匙迷宫是位置加钥匙掩码。状态维度决定 visited 数组维度，也决定复杂度。

并查集变种按附加信息分类。普通并查集维护连通块，带大小并查集维护集合规模，带权并查集维护相对比值或距离，奇偶并查集维护二分关系，网格并查集把二维坐标映射成一维编号，回滚并查集支持撤销。每个变种都要回答：路径压缩是否影响附加信息，合并两个根时附加信息如何更新，查询需要的是根信息还是节点到根的信息。

工程应用：从刷题结构到系统结构

哈希表和链表组合构成缓存系统的基本骨架。LRU 用哈希表定位节点，用双向链表维护新旧顺序；LFU 在 LRU 基础上增加频次维度，用频次到链表的映射维护同频内部顺序。这个结构也解释了为什么单独一个哈希表或单独一个链表都不够：哈希表不知道谁最旧，链表不能常数定位 key。

堆和有序树常用于调度。操作系统调度器、定时器、任务队列、网络包优先级，都需要从动态集合中取出某种最优对象。简单定时任务可以用小顶堆按到期时间排序；需要频繁删除和修改优先级

时，可能要用有序树或索引堆；需要公平性时，键可能不是绝对时间，而是虚拟运行时间。刷题中的任务调度器和会议室问题，是这些系统问题的简化模型。

Trie 和哈希索引用于检索系统。搜索建议、自动补全、敏感词过滤、路由前缀匹配，都依赖前缀共享或快速定位。普通哈希表适合完整 key 查询；Trie 适合前缀查询；倒排索引适合从词到文档集合；布隆过滤器适合快速判断“肯定不存在或可能存在”。刷题不一定直接考布隆过滤器，但理解这些结构能扩展你对“查询模型”的认识。

图结构用于依赖和流动。包管理依赖、编译依赖、课程安排、任务 DAG、网络路由、社交关系和知识图谱都可以建成图。拓扑排序回答依赖顺序，最短路回答最小代价，连通块回答分组，可达性回答影响范围。真正做工程时，图的节点和边往往来自业务对象，需要先把业务关系抽象成图，再选算法。

变种题迁移标准

看到变种题时，不要先问“它像哪道力扣题”，先问四个问题：查询对象是什么，更新对象是什么，是否需要顺序，是否需要合并历史状态。答案分别会指向哈希、堆、有序结构、Trie、区间树、图或并查集。能从访问模式推出结构，才算真正掌握数据结构。

哈希结构的深层语义：键、值和状态等价

哈希表题真正难的地方不是调用 `find`，而是定义 key 和 value。key 必须表示“哪些状态在题目里应当被看作同一类”，value 必须表示“遇到同一类状态时还需要保留什么信息”。两数之和的 key 是数值，value 是下标；字母异位词分组的 key 是字符频次签名，value 是字符串列表；同构字符串的 key 可以是字符映射，value 是另一侧字符；子数组和的 key 是前缀和，value 可能是次数、最早位置或最近位置。key 定义错，哈希表越快越会快地得到错误答案。

状态等价尤其重要。字母异位词里，“eat”和“tea”的原字符串不同，但字符频次相同，所以它们在分组问题中等价；快乐数里，一个整数经过平方和转移后如果再次出现，就说明进入循环，所以 key 是当前数值状态；克隆图里，原节点地址决定唯一身份，节点值可能重复，所以 key 应该是原节点指针或稳定编号，而不是节点值。经典教材讲哈希时会强调抽象等价关系，因为这比容器 API 更接近算法本质。

字母异位词分组先从暴力说起。最直接的做法是拿每个单词和已有分组逐个比较：若两个单词排序后相同，或逐字符计数后相同，就放进同一组。这个版本的瓶颈有两层：第一，单词之间会反复两两比较；第二，同一个单词的规范表示可能被重复计算。优化的关键是把“是否属于同一组”改写成一个状态等价 key：异位词共享同一份字符频次签名。每个单词只计算一次签名，然后用哈希表或有序 map 直接进入对应桶。这样，比较单词对的问题就变成了计算 key 并追加到 value 列表。

图示：异位词分组的状态等价

原始单词	eat、tea、ate 是不同字符串。
规范签名	三者字符频次都为 a:1, e:1, t:1。
哈希键	频次签名作为 key，表示“属于同一组”的等价状态。
哈希值	value 保存这一组所有原始字符串，输出时直接取桶。

Listing 3.16 异位词分组：频次签名定义状态等价

```

1 std::vector<std::vector<std::string>> group_anagrams(
2     const std::vector<std::string>& words)
3 {
4     constexpr int ALPHABET_SIZE = 26;
5     std::map<std::array<int, ALPHABET_SIZE>, std::vector<std::string>> groups;
6
7     for (const std::string& word : words) {
8         std::array<int, ALPHABET_SIZE> signature{};
9         for (const char ch : word) {
10             ++signature[ch - 'a'];

```



```

11     }
12     groups[signature].push_back(word);
13 }
14
15 std::vector<std::vector<std::string>> answer;
16 answer.reserve(groups.size());
17 for (auto& entry : groups) {
18     auto& group = entry.second;
19     answer.push_back(std::move(group));
20 }
21 return answer;
22 }

```

这里用 `std::map` 是为了避免给 `array` 写自定义哈希，同时让输出顺序稳定；若追求平均性能，可以换成 `unordered_map` 并提供哈希函数。这个例子还说明 `value` 的类型由题目输出决定：如果只问有多少组，`value` 可以是计数；如果要返回每组字符串，`value` 必须保存列表。不要把“哈希表”当成固定模板，它只是实现 `key` 到 `value` 的映射。

哈希表的工程问题也不能省略。第一，扩容会 `rehash`，迭代器失效，复杂对象的移动和分配成本会上升。第二，自定义 `key` 要保证相等对象具有相同哈希值，否则容器行为不符合预期。第三，浮点数不适合作为普通哈希 `key`，精度误差会让数学上相等的状态变成不同 `key`。第四，如果输入可能被恶意构造，哈希退化会影响性能。刷题环境通常不要求处理所有工程问题，但写书应该把边界讲出来。

堆结构的边界：局部有序不是全局有序

堆的最重要边界是：它只保证堆顶最优，不保证内部整体有序。很多学生误以为从 `priority_queue` 里遍历出来就是有序序列，这是错的。堆适合反复取当前极值，不适合查找任意元素，不适合直接删除任意元素，不适合回答前驱后继。如果题目需要“窗口内第 k 小”“动态排名”“相邻区间”，堆往往不够，需要有序结构、双堆加懒删除，或树状数组和线段树。

双堆维护中位数是堆边界的经典扩展。一个大顶堆保存较小一半，一个小顶堆保存较大一半。中位数来自两个堆顶，而不是某一个全局有序数组。不变量有两个：大小不变量和顺序不变量。大小不变量保证两个堆元素数量差不超过一；顺序不变量保证左堆每个元素都不大于右堆每个元素。只维护大小会出错，因为堆顶可能交叉。

数据流中位数的暴力版本很自然：每来一个数就插入数组，查询中位数时排序或线性选择。若每次完整排序，旧元素之间已经确定的相对关系被反复计算；若维护一个有序数组，插入位置可以二分找到，但中间插入仍要搬移大量元素。中位数只需要中间边界，而不是完整有序序列。于是把数据切成两半：较小一半只关心最大值，较大一半只关心最小值。左半用大顶堆，右半用小顶堆，插入后重平衡，就把“维护全序”降成“维护两侧边界”。

图示：数据流中位数的双堆边界

左半部分	大顶堆 <code>lower</code> 保存较小一半，堆顶是左半最大值。
右半部分	小顶堆 <code>upper</code> 保存较大一半，堆顶是右半最小值。
顺序不变量	<code>lower.top() <= upper.top()</code> ，左半所有元素不大于右半。
大小不变量	两堆大小差不超过一；本书示例让左堆可以多一个。
中位数	奇数取左堆顶，偶数取两个堆顶平均值。

Listing 3.17 数据流中位数：双堆维护大小和顺序

```

1 class MedianFinder {
2 public:
3     void add_num(int value)
4     {
5         if (this->lower_.empty() || value <= this->lower_.top()) {

```



```

6         this->lower_.push(value);
7     } else {
8         this->upper_.push(value);
9     }
10    this->rebalance();
11 }
12
13 double find_median() const
14 {
15     if (this->lower_.size() == this->upper_.size()) {
16         return (static_cast<double>(this->lower_.top()) +
17                 static_cast<double>(this->upper_.top())) /
18                 2.0;
19     }
20     return static_cast<double>(this->lower_.top());
21 }
22
23 private:
24     void rebalance()
25     {
26         if (this->lower_.size() < this->upper_.size()) {
27             this->lower_.push(this->upper_.top());
28             this->upper_.pop();
29         } else if (this->lower_.size() > this->upper_.size() + 1) {
30             this->upper_.push(this->lower_.top());
31             this->lower_.pop();
32         }
33     }
34
35     std::priority_queue<int> lower_;
36     std::priority_queue<int, std::vector<int>, std::greater<int>> upper_;
37 };

```

这段代码规定左堆元素数可以比右堆多一个，因此奇数个元素时中位数在左堆顶。插入时先按值进入对应堆，再平衡大小。若题目要求删除窗口外元素，就要引入懒删除表，并在访问堆顶前清理过期元素。双堆题的难点不是写堆，而是维护不变量并在删除时保持两个堆的有效大小。

堆在工程里常用于调度和归并。定时器系统可以把任务按到期时间放入小顶堆；日志合并可以把多个有序文件的当前行放入小顶堆；Dijkstra 把候选节点按当前距离放入小顶堆；操作系统或线程池可以用优先级队列选择下一项任务。所有这些场景都有同一结构：候选集合动态变化，但每次只需要当前最优候选。若还需要修改任意候选的优先级，普通堆就不够，要用索引堆、有序树或重新插入加过期检查。

有序结构和区间结构：从相邻关系到聚合信息

有序结构回答的是“在顺序上的邻居是谁”。`lower_bound` 找到第一个不小于目标的元素，配合前一个迭代器，就能检查插入区间是否和相邻区间冲突。会议室、日程安排、数据流区间、最近座位、存在重复元素 III，都依赖相邻关系。哈希表再快，也不能告诉你一个值在有序集合中的前驱和后继。

区间结构回答的是“一个范围内的聚合值是什么”。树状数组擅长前缀可逆聚合，线段树擅长更通用的区间合并。选择它们时先问两个问题：更新是点还是区间，查询是点还是区间。点更新加区间查询，树状数组简单；区间更新加区间查询，线段树懒标记更直接；如果只做静态查询，前缀和或稀疏表可能更合适。不要一看到区间就写线段树，很多题只需要排序加扫描线或前缀和。

区间最大值如果数组不变，暴力每次扫描区间就能回答，只是多次查询会重复访问同一段；前缀和只能处理可逆的和，不能直接由两个前缀最大值相减得到区间最大。若又有点更新，静态稀疏

表也不合适。线段树的推导来自“区间答案能由左右子区间合并”：一个区间的最大值等于左右半区最大值的较大者。把这个合并关系按层缓存起来，点更新只影响从叶子到根的一条路径，区间查询只需收集若干完整覆盖的小区间。

图示：线段树的区间合并

叶子节点	保存单个数组位置的值。
父节点	保存左右孩子区间的合并结果，例如 <code>max(left, right)</code> 。
点更新	修改一个叶子，只沿父指针向上重算一条路径。
区间查询	把查询区间拆成若干不相交的完整节点区间，再合并这些节点。
成立前提	父区间答案必须能由左右子区间答案合成。

Listing 3.18 迭代线段树：点更新和区间最大值

```

1 class SegmentTree {
2 public:
3     explicit SegmentTree(const std::vector<int>& values)
4         : size_(static_cast<int>(values.size())),
5           tree_(static_cast<std::size_t>(values.size() * 2), 0)
6     {
7         for (int i = 0; i < this->size_; ++i) {
8             this->tree_[static_cast<std::size_t>(this->size_ + i)] = values[i];
9         }
10        for (int i = this->size_ - 1; i > 0; --i) {
11            this->tree_[static_cast<std::size_t>(i)] =
12                std::max(this->tree_[static_cast<std::size_t>(i * 2)],
13                        this->tree_[static_cast<std::size_t>(i * 2 + 1)]);
14        }
15    }
16
17    void update(int index, int value)
18    {
19        int position = index + this->size_;
20        this->tree_[static_cast<std::size_t>(position)] = value;
21        for (position /= 2; position > 0; position /= 2) {
22            this->tree_[static_cast<std::size_t>(position)] =
23                std::max(this->tree_[static_cast<std::size_t>(position * 2)],
24                        this->tree_[static_cast<std::size_t>(position * 2 + 1)]);
25        }
26    }
27
28    int range_max(int left, int right) const
29    {
30        constexpr int NEGATIVE_INF = std::numeric_limits<int>::min();
31        int answer = NEGATIVE_INF;
32        int l = left + this->size_;
33        int r = right + this->size_;
34        while (l < r) {
35            if ((l & 1) != 0) {
36                answer = std::max(answer, this->tree_[static_cast<std::size_t>(l)]);
37                ++l;
38            }
39            if ((r & 1) != 0) {
40                --r;

```

```

41         answer = std::max(answer, this->tree_[static_cast<std::size_t>(r)]);
42     }
43     l /= 2;
44     r /= 2;
45 }
46 return answer;
47 }
48
49 private:
50     int size_ = 0;
51     std::vector<int> tree_;
52 };

```

这个实现使用半开区间 `[left, right)`。查询时，若左边界是右孩子，就把它纳入答案并右移；若右边界是右孩子的后一位，就先左移再纳入答案。迭代线段树比递归版本短，也避免 C++ 调用栈问题，但它要求你非常清楚半开区间语义。若题目需要区间加法和区间最大值，就要增加懒标记；若节点信息不是一个整数，而是最大子段和、前缀最大、后缀最大和总和，就要定义可合并的结构体。

树结构：从遍历顺序到信息流

树题不应该只按前序、中序、后序背模板，而要按照信息流分类。自顶向下题把父节点信息传给子节点，例如路径和、验证 BST 上下界、二叉树所有路径；自底向上题把子树信息汇总给父节点，例如高度、直径、平衡二叉树、最大路径和；层次题按距离或深度组织节点，例如层序遍历、右视图、最小深度；结构改造题改变节点链接，例如展开二叉树、反转、删除 BST 节点。遍历顺序服务于信息流，而不是目的本身。

二叉搜索树的中序递增只是它的一种表象。更本质的性质是：每个节点把值域划分成左右两个区间。验证 BST 可以中序，也可以传上下界；第 K 小可以中序计数；删除节点时，如果左右孩子都存在，可以用后继节点替换当前值，再删除后继节点；最近公共祖先在 BST 中可以利用值域方向，而普通二叉树必须在左右子树中寻找目标。把值域约束讲清楚，很多 BST 题就能串起来。

树形 DP 的核心是定义返回值。最大路径和中，返回给父节点的不是当前子树内部最大路径，而是“从当前节点向下延伸到某一侧的最大贡献”；全局答案才记录可能经过当前节点同时连接左右两侧的路径。打家劫舍 III 中，一个节点的状态至少要区分偷和不偷；若只返回一个最大值，会丢失父子不能同时选的约束。树题最常见的错误就是返回值语义太含糊。

图结构：状态空间、边权和访问时机

图题的第一步是定义状态，而不是选择 DFS 或 BFS。状态可能是一个节点，也可能是多个变量的组合。普通岛屿题的状态是坐标；障碍消除最短路的状况是坐标加剩余消除次数；访问所有节点的最短路径是当前节点加访问集合；带钥匙迷宫是坐标加钥匙掩码；单词接龙是一个字符串或字符串编号。状态定义少了维度，会把不同情况错误合并；状态定义多了无用维度，会让复杂度爆炸。

访问标记的时机也很关键。普通 BFS 中，通常在入队时标记已访问，而不是出队时标记。因为一个节点可能被多个邻居发现，如果等出队才标记，它会被重复入队。Dijkstra 则不同，一个节点可以用更短距离再次进入堆，弹出时检查当前距离是否过期。并查集又不同，它没有访问队列，而是用集合代表压缩连通关系。不同结构有不同“状态已经确定”的时刻，不能混用。

障碍消除最短路不能直接把状态写成坐标。暴力搜索会枚举从起点出发的所有路径，并在路径里记录已经消除过多少障碍；普通 BFS 若只用 `visited[row][col]`，会把“到达同一格子但剩余消除次数不同”的两种情况错误合并。瓶颈不是 BFS 本身，而是状态定义不完整导致要么漏解，要么保留太多等价路径。优化后的状态至少包含位置和剩余资源；进一步观察，若两条路径到达同一格子，距离按 BFS 层已经不差，而剩余消除次数更多的状态支配剩余更少的状态。因此每个格子只保存见过的最大剩余次数，就能剪掉未来不可能更好的状态。

图示：状态图 BFS 的支配关系

错误状态	只保存 (row, col), 会把资源不同的路径合并。
完整状态	保存 (row, col, remaining), 位置和剩余消除次数共同决定未来能力。
支配剪枝	同一格子上, 剩余次数更少且距离不更短的状态没有保留价值。
压缩记录	best_remaining[row][col] 保存到达该格子时见过的最大剩余资源。
BFS 语义	队列按步数扩展, 第一次到达终点时步数最短。

Listing 3.19 状态图 BFS: 位置加剩余资源才是完整状态

```

1  int shortest_path_with_elimination(
2      const std::vector<std::vector<int>>& grid,
3      int eliminations)
4  {
5      if (grid.empty() || grid[0].empty()) {
6          return -1;
7      }
8
9      constexpr std::array<std::pair<int, int>, 4> DIRECTIONS{
10         std::pair{1, 0}, std::pair{-1, 0},
11         std::pair{0, 1}, std::pair{0, -1}
12     };
13
14     const int rows = static_cast<int>(grid.size());
15     const int cols = static_cast<int>(grid[0].size());
16     std::vector<std::vector<int>> best_remaining(
17         static_cast<std::size_t>(rows),
18         std::vector<int>(static_cast<std::size_t>(cols), -1));
19
20     std::queue<std::array<int, 4>> queue;
21     queue.push({0, 0, eliminations, 0});
22     best_remaining[0][0] = eliminations;
23
24     while (!queue.empty()) {
25         const auto [row, col, remaining, distance] = queue.front();
26         queue.pop();
27         if (row == rows - 1 && col == cols - 1) {
28             return distance;
29         }
30
31         for (const auto& [dr, dc] : DIRECTIONS) {
32             const int nr = row + dr;
33             const int nc = col + dc;
34             if (nr < 0 || nr >= rows || nc < 0 || nc >= cols) {
35                 continue;
36             }
37             const int next_remaining =
38                 remaining - grid[static_cast<std::size_t>(nr)]
39                     [static_cast<std::size_t>(nc)];
40             if (next_remaining < 0 ||
41                 next_remaining <= best_remaining[static_cast<std::size_t>(nr)]

```

```

42                                     [static_cast<std::size_t>(nc))] {
43                                     continue;
44                                 }
45                                 best_remaining[static_cast<std::size_t>(nr)]
46                                     [static_cast<std::size_t>(nc)] = next_remaining;
47                                 queue.push({nr, nc, next_remaining, distance + 1});
48                             }
49                         }
50                     return -1;
51 }

```

这里没有用三维 `visited[row][col][remaining]`，而是为每个格子保存到达它时剩余消除次数的最好值。若再次到达同一格子但剩余次数更少或相等，未来不会比之前更好，可以剪掉。这个剪枝来自状态支配关系：位置相同、距离不更短、资源更少的状态没有保留价值。状态图题经常有这种优化，但前提是你能证明一个状态支配另一个状态。

边权决定最短路算法。无权边用 BFS；0/1 权边用 0-1 BFS；非负权边用 Dijkstra；存在负权边时，Dijkstra 的贪心确定性失效，需要 Bellman-Ford 或其他算法；DAG 最短路可以按拓扑序 DP。题目如果写的是“每一步代价相同”，才是普通 BFS；如果每条边代价不同，即使看起来也是从一个点走到另一个点，也不能直接用普通队列。

并查集：等价类、势能和离线思维

并查集维护的是等价类。它能快速回答两个元素是否已经连通，却不能告诉你连通路径是什么，也不能处理删除边后的在线连通性。这个能力边界很重要。省份数量、冗余连接、账户合并、等式可满足性，都只需要合并和查询同组；最短路径、路径条数、删除某条边后是否仍连通，就不是普通并查集擅长的范围。

带权并查集把“到父节点的关系”也保存下来。除法求值中，若 $a / b = 2$ ，可以把 a 和 b 放到同一集合，并维护每个节点到根的比例。查询 x / y 时，如果两者根不同，答案不存在；根相同，则用 x/root 除以 y/root 。奇偶并查集也是同理，只是权值从比例变成奇偶关系。难点在合并两个根时，要根据已知约束推导新父子关系上的权值。

回滚并查集和离线动态连通则体现了并查集的高级用法。普通路径压缩会破坏历史，不适合回滚；如果要支持撤销，常用按大小合并并记录被修改的父亲和大小，撤销时恢复。区间时间上的边添加和删除可以用线段树分治把边分配到生效时间段，再配合回滚并查集处理查询。面试不常要求手写完整实现，但理解这个方向能帮助你知普通并查集为什么不能删除边，以及离线算法如何绕开在线限制。

数据结构组合：一道题往往不只一种结构

很多高质量面试题不是考单个结构，而是考组合。LRU 是哈希表加双向链表；LFU 是哈希表加频次表加链表；Twitter 设计题是哈希表加链表或数组加堆归并；单词搜索 II 是 Trie 加 DFS 回溯；账户合并是哈希表加并查集；滑动窗口中位数是双堆加延迟删除哈希表；最小覆盖区间是排序加堆或多指针；天际线是扫描线加堆或有序表。组合题要求你给每个结构分配明确职责。

设计题可以按数据流拆解。先列出操作：插入、删除、查询、移动、合并、取极值、取前驱后继、按前缀查找。再为每个操作找结构能力。若两个操作互相冲突，就用两个结构互相索引，并维护一致性。LRU 中，哈希表和链表必须同时更新；Twitter 中，用户到推文列表的映射和关注关系必须分开；滑动窗口中位数中，堆里有旧值，延迟删除表必须在取堆顶前清理。组合题的 bug 往往来自一个结构更新了，另一个结构没同步。

组合结构的写法

先写每个结构的职责，再写同步规则。职责包括：谁负责定位，谁负责顺序，谁负责极值，谁负责连通性，谁负责前缀，谁负责区间。同步规则包括：插入时更新哪些结构，删除时更新哪些结构，移动时是否保持迭代器有效，查询前是否要清理过期状态。没有同步规则的组合结构很容易在边界输入上失效。

非线性结构题目的讲解标准

非线性结构题讲解至少要回答六个问题。第一，状态或对象是什么。第二，结构里的 key、节点、边、堆元素或集合代表分别表示什么。第三，暴力方法慢在哪里，是查找历史、找极值、找相邻、查前缀、查区间，还是判断连通。第四，所选结构把哪一步降到了什么复杂度。第五，结构的限制是什么，比如哈希无序、堆不能任意删除、并查集不能给路径。第六，变体来临时如何调整 value、比较器、状态维度或附加信息。

以课程表为例，对象是课程，边是先修关系，暴力慢在反复找没有依赖的课程；邻接表保存从一门课指向依赖它的后续课程，入度数组保存未满足依赖数量，队列保存当前可学课程。每弹出一门课，就减少后续课程入度。若最终弹出数量不足，说明有环。变体课程表 II 需要输出顺序，只需记录弹出序列；若课程有持续时间并要求最短完成时间，就要在拓扑序上做 DP；若依赖会动态增删，普通拓扑排序就不够。

以账户合并为例，对象是账户和邮箱。哈希表把邮箱映射到第一次出现的账户，并查集合并共享邮箱的账户，最后按根收集邮箱。暴力方法可能两两比较账户邮箱集合，复杂度很高；哈希表让共享邮箱检测变成平均常数，并查集让账户连通分量合并接近常数。变体如果要求按用户名区分，key 就必须包含用户名或先按用户名分组；如果邮箱大小写不敏感，就要先标准化邮箱。题解必须把这些建模前提说清楚。

以单词搜索 II 为例，对象是棋盘路径和单词集合。暴力做法是对每个单词在棋盘上搜索一次，重复遍历大量相同前缀。Trie 把所有单词的公共前缀合并，DFS 当前路径同时对应 Trie 中一个节点；如果当前字符没有对应边，就立即剪枝。这里 Trie 负责前缀查询，DFS 负责空间路径，访问标记负责同一单元格不能重复使用。变体如果允许八方向移动，只改邻居生成；如果允许重复使用格子，访问标记规则要改；如果字符集很大，Trie 孩子结构也要改。

把这些标准坚持下来，非线性结构就不会变成孤立模板。哈希、堆、树、图、并查集只是不同查询模型的工具；教材的任务是让读者知道每个工具解决什么瓶颈、付出什么代价、在什么条件下失效，以及如何和其他结构组合。

第零部分

核心算法：从暴力到优化

第 4 章

数组、双指针和滑动窗口

数组题是刷题的地基。双指针和滑动窗口不是孤立技巧，而是从暴力枚举子结构中优化出来的方法。双指针常用于有序数组、两端收缩和左右边界协作；滑动窗口常用于连续子数组或子串，并且窗口状态能够增量维护。

盛最多水的容器暴力枚举左右边界。面积由宽度和较短边决定，所以每个下标对都可能成为候选。

Listing 4.1 盛水容器暴力解

```
1 int max_area_bruteforce(const std::vector<int>& height)
2 {
3     int best = 0;
4     for (int left = 0; left < static_cast<int>(height.size()); ++left) {
5         for (int right = left + 1; right < static_cast<int>(height.size()); ++right) ←
        ↪ {
6             const int width = right - left;
7             const int bounded_height = std::min(height[left], height[right]);
8             best = std::max(best, width * bounded_height);
9         }
10    }
11    return best;
12 }
```

暴力法是 $O(n^2)$ 。优化来自候选淘汰。初始取最宽的左右边界，如果移动较高的一边，宽度变小，而较短边没有变高，面积不可能变大。因此只能移动较短边，才有机会提高瓶颈高度。

Listing 4.2 盛水容器双指针解

```
1 int max_area_two_pointers(const std::vector<int>& height)
2 {
3     int left = 0;
4     int right = static_cast<int>(height.size()) - 1;
5     int best = 0;
6
7     while (left < right) {
8         const int width = right - left;
9         const int bounded_height = std::min(height[left], height[right]);
10        best = std::max(best, width * bounded_height);
11
12        if (height[left] <= height[right]) {
13            ++left;
14        } else {
15            --right;
16        }
17    }
18    return best;
19 }
```

不变量是：每一步排除的短边，不会和更内侧的任何边形成更优答案，因为宽度只会变小，短边高度仍然是瓶颈。这个证明比“左右指针往中间走”重要得多。

长度最小的子数组展示滑动窗口。暴力法枚举所有起点和终点，遇到和达到目标时更新答案。它慢在每个起点都重新累加。

Listing 4.3 长度最小子数组滑动窗口

```
1 int min_subarray_len_sliding_window(int target, const std::vector<int>& nums)
2 {
3     int best = std::numeric_limits<int>::max();
4     int left = 0;
5     int sum = 0;
6
7     for (int right = 0; right < static_cast<int>(nums.size()); ++right) {
8         sum += nums[right];
9         while (sum >= target) {
10             best = std::min(best, right - left + 1);
11             sum -= nums[left];
12             ++left;
13         }
14     }
15
16     return best == std::numeric_limits<int>::max() ? 0 : best;
17 }
```

这个优化依赖正整数。右边界右移时窗口和只会增加，左边界右移时窗口和只会减少。如果数组包含负数，这个单调性消失，滑动窗口可能失效，需要考虑前缀和、单调队列或其他结构。

无重复字符的最长子串也是窗口题，但窗口状态不是和，而是字符最后出现位置。窗口不变量是 `[left, right]` 内没有重复字符。

Listing 4.4 无重复字符最长子串

```
1 int longest_substring_without_repeat_window(const std::string& s)
2 {
3     constexpr int ASCII_RANGE = 256;
4     std::array<int, ASCII_RANGE> last_seen{};
5     last_seen.fill(-1);
6
7     int left = 0;
8     int best = 0;
9
10    for (int right = 0; right < static_cast<int>(s.size()); ++right) {
11        const auto ch = static_cast<unsigned char>(s[right]);
12        if (last_seen[ch] >= left) {
13            left = last_seen[ch] + 1;
14        }
15        last_seen[ch] = right;
16        best = std::max(best, right - left + 1);
17    }
18
19    return best;
20 }
```

这里用 `std::array<int, 256>` 是因为按 ASCII 字符处理。如果题目要求完整 Unicode 字

符，需要先明确字符单元，或使用哈希表保存字符到位置。

三数之和是双指针的第二个标准模型。题目要求找出所有不重复三元组，使得三个数之和为 0。暴力方法枚举三个下标，时间复杂度 $O(n^3)$ ，还要处理去重；如果用哈希表固定两数再找第三数，可以降到 $O(n^2)$ ，但去重和重复值处理仍然复杂。真正稳定的做法是先排序，再固定第一个数，把剩下两个数变成有序数组上的两数之和。

排序后的关键性质是：固定 `first` 后，如果 `nums[left] + nums[right]` 太小，说明左端值不够大，只能右移 `left`；如果和太大，说明右端值太大，只能左移 `right`。这就是有序性带来的候选淘汰。去重也依赖排序：同一层循环里，如果当前值和前一个值相同，就跳过；找到一个答案后，左右指针都要跳过连续重复值。

Listing 4.5 三数之和：排序加双指针

```

1 std::vector<std::vector<int>> three_sum(std::vector<int> nums)
2 {
3     std::sort(nums.begin(), nums.end());
4     std::vector<std::vector<int>> answer;
5
6     for (int first = 0; first < static_cast<int>(nums.size()); ++first) {
7         if (first > 0 && nums[first] == nums[first - 1]) {
8             continue;
9         }
10        if (nums[first] > 0) {
11            break;
12        }
13
14        int left = first + 1;
15        int right = static_cast<int>(nums.size()) - 1;
16        while (left < right) {
17            const int sum = nums[first] + nums[left] + nums[right];
18            if (sum == 0) {
19                answer.push_back({nums[first], nums[left], nums[right]});
20                const int left_value = nums[left];
21                const int right_value = nums[right];
22                while (left < right && nums[left] == left_value) {
23                    ++left;
24                }
25                while (left < right && nums[right] == right_value) {
26                    --right;
27                }
28            } else if (sum < 0) {
29                ++left;
30            } else {
31                --right;
32            }
33        }
34    }
35
36    return answer;
37 }

```

这里参数按值传入，是因为函数内部要排序。如果不想修改调用者的数组，按值传参是合理的；如果题目允许原地修改，也可以传引用。复杂度是排序 $O(n \log n)$ 加双指针枚举 $O(n^2)$ ，总时间 $O(n^2)$ ，额外空间不计答案为 $O(1)$ 或排序栈空间。面试表达要强调：去重不是最后把答案放进 `set`，而是在

生成答案时就避免重复，这样更可控。

找到字符串中所有字母异位词是固定长度窗口。题目给定字符串 `s` 和模式串 `p`，要求找出 `s` 中所有长度等于 `p` 且字符频次完全相同的起点。暴力方法枚举每个长度为 `p.size()` 的子串，然后排序比较或统计比较。如果每次排序，复杂度是 $O(nk \log k)$ ；如果每次重新统计，复杂度是 $O(nk)$ 。慢点在于相邻窗口高度重叠，却重复计算了大部分字符频次。

优化方法是维护固定长度窗口的频次数组。窗口右端每进入一个字符，就把它的计数加一；当窗口长度超过模式串长度，左端字符离开，计数减一。由于题目通常限定小写英文，可以用 `std::array<int, 26>`，比较两个数组是常数时间。窗口不变量是：处理到 `right` 时，`window` 正好记录当前长度不超过模式串长度的后缀窗口字符频次。

Listing 4.6 找到所有字母异位词：固定长度窗口

```
1 std::vector<int> find_anagrams_window(const std::string& s,
2                                     const std::string& p)
3 {
4     constexpr int ALPHABET_SIZE = 26;
5     std::vector<int> answer;
6     if (s.size() < p.size()) {
7         return answer;
8     }
9
10    std::array<int, ALPHABET_SIZE> need{};
11    std::array<int, ALPHABET_SIZE> window{};
12    for (const char ch : p) {
13        ++need[ch - 'a'];
14    }
15
16    for (std::size_t right = 0; right < s.size(); ++right) {
17        ++window[s[right] - 'a'];
18        if (right >= p.size()) {
19            --window[s[right - p.size()] - 'a'];
20        }
21        if (window == need) {
22            answer.push_back(static_cast<int>(right + 1 - p.size()));
23        }
24    }
25
26    return answer;
27 }
```

这题容易错在下标。窗口第一次达到目标长度是在 `right + 1 == p.size()`，起点是 `right + 1 - p.size()`。如果用 `int` 写下标，要小心 `s.size()` 是无符号类型；本书示例这里使用 `std::size_t` 控制循环，最终起点再转成 `int`，因为力扣返回 `vector<int>`。

最小覆盖子串是变长窗口里最重要的一题。题目要求在 `s` 中找最短子串，覆盖 `t` 中所有字符及其次数。暴力方法枚举所有子串，再检查是否覆盖，复杂度至少 $O(n^2|\Sigma|)$ 。优化的核心是：右边界扩张直到窗口满足覆盖条件，然后尽可能移动左边界缩短窗口；一旦左移导致窗口不满足，再继续右移。这个过程成立，因为右移只会增加字符，左移只会删除字符，覆盖关系可以被增量维护。

Listing 4.7 最小覆盖子串：变长窗口

```
1 std::string min_window_substring(const std::string& s, const std::string& t)
2 {
3     constexpr int ASCII_RANGE = 128;
4     std::array<int, ASCII_RANGE> need{};
```

```

5     std::array<int, ASCII_RANGE> window{};
6     int required = 0;
7
8     for (const char ch : t) {
9         const auto index = static_cast<unsigned char>(ch);
10        if (need[index] == 0) {
11            ++required;
12        }
13        ++need[index];
14    }
15
16    int formed = 0;
17    int left = 0;
18    int best_left = 0;
19    int best_length = std::numeric_limits<int>::max();
20
21    for (int right = 0; right < static_cast<int>(s.size()); ++right) {
22        const auto in = static_cast<unsigned char>(s[right]);
23        ++window[in];
24        if (need[in] > 0 && window[in] == need[in]) {
25            ++formed;
26        }
27
28        while (formed == required) {
29            const int length = right - left + 1;
30            if (length < best_length) {
31                best_length = length;
32                best_left = left;
33            }
34
35            const auto out = static_cast<unsigned char>(s[left]);
36            --window[out];
37            if (need[out] > 0 && window[out] < need[out]) {
38                --formed;
39            }
40            ++left;
41        }
42    }
43
44    if (best_length == std::numeric_limits<int>::max()) {
45        return "";
46    }
47    return s.substr(best_left, best_length);
48 }

```

required 表示需要满足的不同字符种类数，**formed** 表示当前窗口里已经满足需求的字符种类数。不能只用窗口总长度判断，因为 **t = "AABC"** 时，两个 **A** 都必须覆盖。复杂度是 $O(n + m + |\Sigma|)$ ，在 ASCII 数组固定时可视为 $O(n + m)$ 。面试表达要说清楚：这题不是固定窗口，因为答案长度未知；它是“先满足，再收缩”的最短覆盖模型。

本章题目复盘模板

数组和窗口题复盘时至少写五句话：暴力枚举什么；慢点是否来自重复扫描、重复统计或候选过多；优化使用了有序性、窗口增量状态还是候选淘汰；左右指针移动条件是什么；每个元素为什么只会被处理常数次。只写“用双指针”或“用滑动窗口”不算会。

本章力扣主线：11 盛最多水的容器、15 三数之和、209 长度最小的子数组、3 无重复字符的最长子串、438 找到字符串中所有字母异位词、76 最小覆盖子串。复盘时必须写出窗口不变量和指针移动条件。

数组题的第一层能力是下标语义。很多错误不是算法不会，而是区间闭开混乱。推荐统一使用半开区间描述窗口： $[left, right)$ 表示包含 $left$ ，不包含 $right$ ；窗口长度是 $right - left$ ；右端扩张时先处理 $s[right]$ 再让 $right$ 前进，左端收缩时处理 $s[left]$ 再让 $left$ 前进。力扣代码里常写闭区间 $[left, right]$ ，也没问题，但全篇必须保持一致。区间语义一乱，最小覆盖子串、二分边界、前缀和都会出错。

前缀和是数组题最基础的缓存。它适合静态数组的区间求和、区间计数、差值判断。若数组会频繁更新，普通前缀和就不够，因为一次修改会影响后面所有前缀；这时要考虑树状数组或线段树。二维前缀和则把矩形区域求和变成四个角的组合。它的本质仍然是容斥：目标矩形等于大矩形减去上方和左方，再加回被减了两次左上角。

Listing 4.8 二维前缀和：矩形查询

```
1 class MatrixPrefixSum {
2 public:
3     explicit MatrixPrefixSum(const std::vector<std::vector<int>>& matrix)
4     {
5         const int rows = static_cast<int>(matrix.size());
6         const int cols = rows == 0 ? 0 : static_cast<int>(matrix[0].size());
7         this->prefix_.assign(rows + 1, std::vector<int>(cols + 1, 0));
8
9         for (int row = 0; row < rows; ++row) {
10             for (int col = 0; col < cols; ++col) {
11                 this->prefix_[row + 1][col + 1] =
12                     this->prefix_[row][col + 1] +
13                     this->prefix_[row + 1][col] -
14                     this->prefix_[row][col] +
15                     matrix[row][col];
16             }
17         }
18     }
19
20     int query(int top, int left, int bottom, int right) const
21     {
22         return this->prefix_[bottom + 1][right + 1] -
23             this->prefix_[top][right + 1] -
24             this->prefix_[bottom + 1][left] +
25             this->prefix_[top][left];
26     }
27
28 private:
29     std::vector<std::vector<int>> prefix_;
30 };
```

二维前缀和的边界最容易错，所以建议多开一行一列哨兵。 $prefix[row + 1][col + 1]$ 表示从原矩阵左上角到 (row, col) 的闭矩形和。查询时， $bottom$ 和 $right$ 是原矩阵闭区间端点，

因此访问前缀数组要加一。这个模式和一维前缀和完全一致：多出来的第 0 行、第 0 列负责消除边界分支。

差分数组是前缀和的反向思维。前缀和把多次区间查询变快；差分把多次区间修改变快。若对区间 `[left, right]` 加 `delta`，只需要让变化从 `left` 开始，在 `right + 1` 结束。最后对差分数组做一次前缀累加，得到所有位置的最终值。航班预订、拼车、区间加法都属于这个模型。

Listing 4.9 差分数组：区间更新后一次还原

```
1 std::vector<int> apply_range_additions(
2     int length,
3     const std::vector<std::tuple<int, int, int>>& updates)
4 {
5     std::vector<int> diff(length + 1, 0);
6     for (const auto [left, right, delta] : updates) {
7         diff[left] += delta;
8         if (right + 1 < length) {
9             diff[right + 1] -= delta;
10        }
11    }
12
13    std::vector<int> values(length, 0);
14    int current = 0;
15    for (int i = 0; i < length; ++i) {
16        current += diff[i];
17        values[i] = current;
18    }
19    return values;
20 }
```

差分数组要求坐标能开数组。如果坐标很大但更新次数不多，就要做坐标压缩或扫描线事件排序。它也不适合频繁在线查询：如果每次更新后立刻问某个点或区间，普通差分每次都还原会很慢。这时树状数组可以看成“动态前缀和”，线段树可以支持更复杂的区间操作。

双指针并不只是一左一右。它至少有三类模型。第一类是两端收缩，如盛水容器和回文判断；第二类是同向快慢，如删除数组重复项、移动零、链表快慢指针；第三类是排序后的配对搜索，如两数之和 II、三数之和、四数之和。每类指针移动依据不同。两端收缩靠候选淘汰，同向快慢靠覆盖写入或距离不变量，配对搜索靠有序数组的和单调。

Listing 4.10 两数之和 II：有序数组配对搜索

```
1 std::vector<int> two_sum_sorted(const std::vector<int>& numbers, int target)
2 {
3     int left = 0;
4     int right = static_cast<int>(numbers.size()) - 1;
5     while (left < right) {
6         const int sum = numbers[left] + numbers[right];
7         if (sum == target) {
8             return {left + 1, right + 1};
9         }
10        if (sum < target) {
11            ++left;
12        } else {
13            --right;
14        }
15    }
```

```

16     return {};
17 }

```

这题成立的原因是数组有序。若当前和太小，固定右端时，左端再往左只会更小，所以左端必须右移；若当前和太大，固定左端时，右端再往右只会更大，所以右端必须左移。这个推理不能套到无序数组上。无序数组的两数之和通常用哈希表，因为没有单调性可供淘汰。

四数之和是三数之和的自然扩展。暴力四层循环是 $O(n^4)$ 。排序后固定前两个数，后两个数用双指针，复杂度降到 $O(n^3)$ 。去重必须发生在每一层：第一个数重复跳过，第二个数重复跳过，找到答案后左右指针跳过重复。还要注意整数溢出，四个 `int` 相加可能超过 `int` 范围，应该用 `long long` 保存和。

Listing 4.11 四数之和：排序、分层去重和长整型求和

```

1  std::vector<std::vector<int>> four_sum(std::vector<int> nums, int target)
2  {
3      std::sort(nums.begin(), nums.end());
4      std::vector<std::vector<int>> answer;
5      const int n = static_cast<int>(nums.size());
6
7      for (int first = 0; first < n; ++first) {
8          if (first > 0 && nums[first] == nums[first - 1]) {
9              continue;
10         }
11         for (int second = first + 1; second < n; ++second) {
12             if (second > first + 1 && nums[second] == nums[second - 1]) {
13                 continue;
14             }
15
16             int left = second + 1;
17             int right = n - 1;
18             while (left < right) {
19                 const long long sum =
20                     static_cast<long long>(nums[first]) + nums[second] +
21                     nums[left] + nums[right];
22                 if (sum == target) {
23                     answer.push_back(
24                         {nums[first], nums[second], nums[left], nums[right]});
25                     const int left_value = nums[left];
26                     const int right_value = nums[right];
27                     while (left < right && nums[left] == left_value) {
28                         ++left;
29                     }
30                     while (left < right && nums[right] == right_value) {
31                         --right;
32                     }
33                 } else if (sum < target) {
34                     ++left;
35                 } else {
36                     --right;
37                 }
38             }
39         }
40     }

```



```

41
42     return answer;
43 }

```

四数之和还可以加剪枝，例如当前最小可能和已经大于目标就 break，当前最大可能和仍小于目标就 continue。但剪枝要谨慎处理溢出和重复值，不要为了少几步让代码变得难以验证。面试中先写稳定版本，再说明可选剪枝，通常更好。

滑动窗口要分固定长度和可变长度。固定长度窗口的长度由题目直接给出，例如找到所有异位词、长度为 K 的最大平均值、滑动窗口最大值。可变长度窗口的长度由约束决定，例如无重复字符、最小覆盖子串、乘积小于 K 的子数组。固定窗口通常每步右进左出；可变窗口通常右端扩展到满足或破坏条件，再移动左端恢复条件或压缩答案。

乘积小于 K 的子数组是可变窗口的另一个典型题。数组元素为正，窗口乘积随着右端扩展会变大，随着左端收缩会变小，因此具有单调性。对每个右端，当窗口乘积小于 K 时，以右端结尾的合法子数组数量是 `right - left + 1`，因为从 `left` 到 `right` 的任意起点都能得到合法乘积。若 `k <= 1`，正整数乘积不可能小于 K，直接返回 0。

Listing 4.12 乘积小于 K 的子数组：窗口计数

```

1 int num_subarray_product_less_than_k(const std::vector<int>& nums, int k)
2 {
3     if (k <= 1) {
4         return 0;
5     }
6
7     int left = 0;
8     int product = 1;
9     int answer = 0;
10    for (int right = 0; right < static_cast<int>(nums.size()); ++right) {
11        product *= nums[right];
12        while (product >= k) {
13            product /= nums[left];
14            ++left;
15        }
16        answer += right - left + 1;
17    }
18    return answer;
19 }

```

这个题经常被错写成每次只更新最长长度，但题目问的是子数组数量。窗口恢复合法后，所有以 `right` 结尾、起点在 `[left, right]` 的子数组都合法，因此一次加上窗口长度。计数问题要特别注意“每个答案由哪个时刻统计”。这里每个合法子数组按右端点唯一统计一次，所以不会重复。

有些窗口题不能用普通滑动窗口，但可以用前缀和加单调队列。例如最短子数组和至少为 K，数组允许负数。由于负数存在，窗口和不再随右端扩展单调增加。我们改用前缀和：需要找 `prefix[right] - prefix[left] >= K` 且 `right - left` 最小。扫描右端时，用单调队列维护可能的左端前缀下标，并让前缀和递增。若当前前缀减队首前缀已经达到 K，就更新答案并弹出队首，因为更靠后的右端只会让这个左端形成更长区间。

Listing 4.13 允许负数的最短子数组：前缀和加单调队列

```

1 int shortest_subarray_at_least_k(const std::vector<int>& nums, int k)
2 {
3     const int n = static_cast<int>(nums.size());
4     std::vector<long long> prefix(n + 1, 0);

```

```

5   for (int i = 0; i < n; ++i) {
6       prefix[i + 1] = prefix[i] + nums[i];
7   }
8
9   std::deque<int> deque;
10  int answer = n + 1;
11  for (int right = 0; right <= n; ++right) {
12      while (!deque.empty() &&
13             prefix[right] - prefix[deque.front()] >= k) {
14          answer = std::min(answer, right - deque.front());
15          deque.pop_front();
16      }
17      while (!deque.empty() && prefix[deque.back()] >= prefix[right]) {
18          deque.pop_back();
19      }
20      deque.push_back(right);
21  }
22
23  return answer == n + 1 ? -1 : answer;
24 }

```

为什么队尾前缀和更大时可以删除? 若有两个候选左端 $i < j$ 且 $\text{prefix}[i] \geq \text{prefix}[j]$, 那么对于任何未来右端 r , $\text{prefix}[r] - \text{prefix}[j]$ 不小于 $\text{prefix}[r] - \text{prefix}[i]$, 并且 j 更靠右, 区间更短。因此 i 被 j 完全支配, 可以删除。这是单调队列的本质: 删除永远不可能成为最优的候选。

数组题还要重视“原地”要求。移动零、删除重复、颜色分类、轮转数组都要求在原数组上改。原地算法通常用覆盖写入或交换。覆盖写入适合保留相对顺序, 例如删除重复和移动零; 交换适合不要求稳定顺序或多区域划分, 例如颜色分类。不要为了原地而牺牲可读性: 如果题目没有要求原地, 新建结果数组往往更清楚; 如果题目明确要求 $O(1)$ 额外空间, 就必须说明如何避免额外数组。

Listing 4.14 轮转数组: 三次反转

```

1 void rotate_array(std::vector<int>& nums, int k)
2 {
3     if (nums.empty()) {
4         return;
5     }
6
7     k %= static_cast<int>(nums.size());
8     std::reverse(nums.begin(), nums.end());
9     std::reverse(nums.begin(), nums.begin() + k);
10    std::reverse(nums.begin() + k, nums.end());
11 }

```

三次反转的原理是把末尾 K 个元素整体搬到前面, 并保持它们内部顺序。整体反转后, 原末尾 K 个元素到了前面但内部顺序反了, 原前面部分也顺序反了; 分别反转两段后, 两个部分内部顺序恢复。若 k 大于数组长度, 必须先取模。环状替换也能做到原地, 但需要处理循环个数和最大公约数, 面试里三次反转更稳。

数组题的另一个深层主题是“输出规模”。子集、三数之和、区间交集这类题, 算法即使很高效, 输出本身也可能很大。复杂度分析时要区分计算成本和输出成本。三数之和不计输出是 $O(n^2)$, 但答案数量最坏也可能达到 $O(n^2)$ 。如果题目要求输出所有结果, 就不能期望低于输出规模的时间。面试中说出这一点, 能避免不现实的优化承诺。

窗口题的调试方法是打印四个量: `left`、`right`、窗口状态、答案。无重复字符打印字符最后

出现位置；最小覆盖打印 **formed** 和关键字符计数；乘积小于 K 打印 **product**；单调队列打印队列中的下标和值。很多窗口错误都是左边界多移或少移一步，日志能直接暴露不变量何时被破坏。

常见误区

滑动窗口有三个常见误用。第一，数组含负数却用窗口维护单调性。第二，题目要求非连续子序列，却套连续窗口。第三，窗口状态不能常数时间增删，却假设移动边界很便宜。使用窗口前必须确认：答案是连续区间，边界移动后状态能增量维护，且移动策略不会漏掉可能答案。

数组窗口实验：第一，把长度最小子数组中的正数条件去掉，构造包含负数的反例，观察窗口为什么漏解。第二，比较最小覆盖子串的暴力枚举、窗口版本在长字符串上的时间。第三，用随机数组验证四数之和的去重逻辑，把结果排序后检查是否有重复四元组。第四，打印最短子数组和至少 K 的单调队列，观察哪些前缀下标被支配删除。

array-window 深度题卡

数组与原地维护

下面的题卡都按同一条链路展开：从问题出发，先写暴力，再定位重复工作，然后选择数据结构或状态不变量，最后落到 C++ 实现、边界和扩展。格式统一是为了训练面试表达，而每张卡的关键观察和失效条件都要单独复盘。

图示：数组与原地维护推导链

暴力基线	枚举候选、完整模拟或逐个检查，先保证覆盖全部可能答案。
瓶颈定位	慢点在于重复搬移和重复求同一段信息。只要一个位置的状态已经由前缀、后缀、慢指针或边界指针确定，再回头扫描就是浪费。
结构选择	优化要把数组切成若干语义明确的区间：已经处理好的前缀、未知区、已经确定的后缀，或者左侧贡献和右侧贡献。双指针、前后缀数组、原地交换和稳定写入，本质都是让每个元素只进入少数几次状态转移。
不变量	不变量必须能落到下标上：慢指针左侧保存什么，右指针右侧保存什么，当前下标之前是否已经满足题目要求。只要这个叙述含糊，代码中的加一减一就会变成猜。
代码落地	C++ 实现优先使用 <code>std::vector<int></code> 和明确的整型下标。涉及乘积、面积、总和时要主动判断是否需要 <code>long long</code> 。如果题目要求原地修改，要说明返回值表示新长度还是数组本身，避免把输出语义和容器容量混在一起。

5 最长回文子串

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。回文围绕中心对称，中心可以是字符也可以是两个字符之间。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“数组与原地维护”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力做法通常枚举每个位置的最终去向，或者为每个位置重新扫描一段区间。它容易写出正确答案，却没有利用数组的连续存储、下标可随机访问、前后缀可复用这些条件。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。慢点在于重复搬移和重复求同一段信息。只要一个位置的状态已经由前缀、后缀、慢指针或边界指针确定，再回头扫描就是浪费。放到本题里，要重点观察“中心扩展避免枚举子串后再逐个检查；每次扩展只比较新增左右边界。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。优化要把数组切成若干语义明确的区间：已经处理好的前缀、未知区、已经确定的后缀，或者左侧贡献和右侧贡献。双指针、前后缀数组、原地交换和稳定写入，本质都是让每个元素只进入少数几次状态转移。本题的优化应当围绕“中心扩展避免枚举子串后再逐个检查；每次扩展只比较新增左右边界。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。不变量必须能落到下标上：慢指针左侧保存什么，右指针右侧保存什么，当前下标之前是否已经满足题目要求。只要这个叙述含糊，代码中的加一减一就会变成猜。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。正确性通常用区间不变量证明。每次循环只扩大已经正确的区域，未知区严格缩小，且被移出的元素已经放到符合语义的位置。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。C++ 实现优先使用 `std::vector<int>` 和明确的整型下标。涉及乘积、面积、总和时要主动判断是否需要 `long long`。如果题目要求原地修改，要说明返回值表示新长度还是数组本身，避免把输出语义和容器容量混在一起。本题还要特别注意：偶数长度回文、单字符、全相同字符、多个同长答案。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：偶数长度回文、单字符、全相同字符、多个同长答案。若要求回文子序列，应转成区间 DP，不能用中心扩展。实验时准备空数组、单元素、全相同、严格递增、严格递减、包含负数和极值的用例。把每轮指针位置打印出来，比只看最终答案更容易发现边界错误。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

26 删除有序数组重复项

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。有序数组让重复元素相邻，慢指针表示下一个唯一值写入位置。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“数组与原地维护”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力做法通常枚举每个位置的最终去向，或者为每个位置重新扫描一段区间。它容易写出正确答案，却没有利用数组的连续存储、下标可随机访问、前后缀可复用这些条件。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。慢点在于重复搬移和重复求同一段信息。只要一个位置的状态已经由前缀、后缀、慢指针或边界指针确定，再回头扫描就是浪费。放到本题里，要重点观察“快指针扫描，只有发现新值才写入慢指针并推进。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。优化要把数组切成若干语义明确的区间：已经处理好的前缀、未知区、已经确定的后缀，或者左侧贡献和右侧贡献。双指针、前后缀数组、原地交换和稳定写入，本质都是让每个元素只进入少数几次状态转移。本题的优化应当围绕“快指针扫描，只有发现新值才写入慢指针并推进。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。不变量必须能落到下标上：慢指针左侧保存什么，右指针右侧保存什么，当前下标之前是否已经满足题目要求。只要这个叙述含糊，代码中的加一减一就会变成猜。用这道题复

盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。正确性通常用区间不变量证明。每次循环只扩大已经正确的区域，未知区严格缩小，且被移出的元素已经放到符合语义的位置。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。C++ 实现优先使用 `std::vector<int>` 和明确的整型下标。涉及乘积、面积、总和时要主动判断是否需要 `long long`。如果题目要求原地修改，要说明返回值表示新长度还是数组本身，避免把输出语义和容器容量混在一起。本题还要特别注意：空数组、全重复、无重复、返回长度不等于物理容量。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：空数组、全重复、无重复、返回长度不等于物理容量。若允许每个元素最多保留两次，只需改变写入条件为和前两个比较。实验时准备空数组、单元素、全相同、严格递增、严格递减、包含负数和极值的用例。把每轮指针位置打印出来，比只看最终答案更容易发现边界错误。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

27 移除元素

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。原地过滤，慢指针左侧始终是不等于目标值的稳定前缀。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“数组与原地维护”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力做法通常枚举每个位置的最终去向，或者为每个位置重新扫描一段区间。它容易写出正确答案，却没有利用数组的连续存储、下标可随机访问、前后缀可复用这些条件。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。慢点在于重复搬移和重复求同一段信息。只要一个位置的状态已经由前缀、后缀、慢指针或边界指针确定，再回头扫描就是浪费。放到本题里，要重点观察“保持顺序用快慢指针；不保持顺序可用左右交换减少写入。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。优化要把数组切成若干语义明确的区间：已经处理好的前缀、未知区、已经确定的后缀，或者左侧贡献和右侧贡献。双指针、前后缀数组、原地交换和稳定写入，本质都是让每个元素只进入少数几次状态转移。本题的优化应当围绕“保持顺序用快慢指针；不保持顺序可用左右交换减少写入。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。不变量必须能落到下标上：慢指针左侧保存什么，右指针右侧保存什么，当前下标之前是否已经满足题目要求。只要这个叙述含糊，代码中的加一减一就会变成猜。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。正确性通常用区间不变量证明。每次循环只扩大已经正确的区域，未知区严格缩小，且被移出的元素已经放到符合语义的位置。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。C++ 实现优先使用 `std::vector<int>` 和明确的整型下标。涉及乘积、面积、总和时要主动判断是否需要 `long long`。如果题目要求原地修改，要说明返回值表示新长度还是数组本身，避免把输出语义和容器容量混在一起。本题还要特别注意：所有元素都被移除、没有元素被移除、目标值在尾部密集。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是

写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：所有元素都被移除、没有元素被移除、目标值在尾部密集。工程里要先确认是否要求稳定顺序，不同要求对应不同写法。实验时准备空数组、单元素、全相同、严格递增、严格递减、包含负数和极值的用例。把每轮指针位置打印出来，比只看最终答案更容易发现边界错误。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

31 下一个排列

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。字典序结构由最长非递增后缀决定，枢轴是后缀前一个位置。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“数组与原地维护”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力做法通常枚举每个位置的最终去向，或者为每个位置重新扫描一段区间。它容易写出正确答案，却没有利用数组的连续存储、下标可随机访问、前后缀可复用这些条件。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。慢点在于重复搬移和重复求同一段信息。只要一个位置的状态已经由前缀、后缀、慢指针或边界指针确定，再回头扫描就是浪费。放到本题里，要重点观察“从右找第一个上升点，再从右找刚好更大的元素交换，最后反转后缀。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。优化要把数组切成若干语义明确的区间：已经处理好的前缀、未知区、已经确定的后缀，或者左侧贡献和右侧贡献。双指针、前后缀数组、原地交换和稳定写入，本质都是让每个元素只进入少数几次状态转移。本题的优化应当围绕“从右找第一个上升点，再从右找刚好更大的元素交换，最后反转后缀。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。不变量必须能落到下标上：慢指针左侧保存什么，右指针右侧保存什么，当前下标之前是否已经满足题目要求。只要这个叙述含糊，代码中的加一减一就会变成猜。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。正确性通常用区间不变量证明。每次循环只扩大已经正确的区域，未知区严格缩小，且被移出的元素已经放到符合语义的位置。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。C++ 实现优先使用 `std::vector<int>` 和明确的整型下标。涉及乘积、面积、总和时要主动判断是否需要 `long long`。如果题目要求原地修改，要说明返回值表示新长度还是数组本身，避免把输出语义和容器容量混在一起。本题还要特别注意：完全降序、重复数字、只有一个元素、交换后后缀必须最小化。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：完全降序、重复数字、只有一个元素、交换后后缀必须最小化。若要求上一个排列，比较方向和后缀处理对称变化。实验时准备空数组、单元素、全相同、严格递增、严格递减、包含负数和极值的用例。把每轮指针位置打印出来，比只看最终答案更容易发现边界错误。实验报告建议记录三件事：

暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

54 螺旋矩阵

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。四个边界收缩模拟一圈一圈访问。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“数组与原地维护”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力做法通常枚举每个位置的最终去向，或者为每个位置重新扫描一段区间。它容易写出正确答案，却没有利用数组的连续存储、下标可随机访问、前后缀可复用这些条件。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。慢点在于重复搬移和重复求同一段信息。只要一个位置的状态已经由前缀、后缀、慢指针或边界指针确定，再回头扫描就是浪费。放到本题里，要重点观察“每走完一条边就收缩对应边界，并在走反方向前检查是否仍合法。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。优化要把数组切成若干语义明确的区间：已经处理好的前缀、未知区、已经确定的后缀，或者左侧贡献和右侧贡献。双指针、前后缀数组、原地交换和稳定写入，本质都是让每个元素只进入少数几次状态转移。本题的优化应当围绕“每走完一条边就收缩对应边界，并在走反方向前检查是否仍合法。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。不变量必须能落到下标上：慢指针左侧保存什么，右指针右侧保存什么，当前下标之前是否已经满足题目要求。只要这个叙述含糊，代码中的加一减一就会变成猜。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。正确性通常用区间不变量证明。每次循环只扩大已经正确的区域，未知区严格缩小，且被移出的元素已经放到符合语义的位置。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。C++ 实现优先使用 `std::vector<int>` 和明确的整型下标。涉及乘积、面积、总和时要主动判断是否需要 `long long`。如果题目要求原地修改，要说明返回值表示新长度还是数组本身，避免把输出语义和容器容量混在一起。本题还要特别注意：单行、单列、空矩阵、最后一圈只剩一个元素。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：单行、单列、空矩阵、最后一圈只剩一个元素。若改成生成矩阵，边界逻辑相同，只是读变成写。实验时准备空数组、单元素、全相同、严格递增、严格递减、包含负数和极值的用例。把每轮指针位置打印出来，比只看最终答案更容易发现边界错误。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

75 颜色分类

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。三指针维护零区、一号区、未知区和二区。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“数组与原地维护”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力做法通常枚举每个位置的最终去向，或者为每个位置重新扫描

一段区间。它容易写出正确答案，却没有利用数组的连续存储、下标可随机访问、前后缀可复用这些条件。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。慢点在于重复搬移和重复求同一段信息。只要一个位置的状态已经由前缀、后缀、慢指针或边界指针确定，再回头扫描就是浪费。放到本题里，要重点观察“遇到二时只移动右边界，不移动扫描指针，因为换回来的元素未知。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。优化要把数组切成若干语义明确的区间：已经处理好的前缀、未知区、已经确定的后缀，或者左侧贡献和右侧贡献。双指针、前后缀数组、原地交换和稳定写入，本质都是让每个元素只进入少数几次状态转移。本题的优化应当围绕“遇到二时只移动右边界，不移动扫描指针，因为换回来的元素未知。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。不变量必须能落到下标上：慢指针左侧保存什么，右指针右侧保存什么，当前下标之前是否已经满足题目要求。只要这个叙述含糊，代码中的加一减一就会变成猜。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。正确性通常用区间不变量证明。每次循环只扩大已经正确的区域，未知区严格缩小，且被移出的元素已经放到符合语义的位置。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。C++ 实现优先使用 `std::vector<int>` 和明确的整型下标。涉及乘积、面积、总和时要主动判断是否需要 `long long`。如果题目要求原地修改，要说明返回值表示新长度还是数组本身，避免把输出语义和容器容量混在一起。本题还要特别注意：全零、全二、已经有序、逆序、交换后漏检查。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：全零、全二、已经有序、逆序、交换后漏检查。这是荷兰国旗问题，能推广到三类分区的原地维护。实验时准备空数组、单元素、全相同、严格递增、严格递减、包含负数和极值的用例。把每轮指针位置打印出来，比只看最终答案更容易发现边界错误。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

88 合并两个有序数组

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。第一个数组尾部有空位，适合从后往前写入最终最大值。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“数组与原地维护”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力做法通常枚举每个位置的最终去向，或者为每个位置重新扫描一段区间。它容易写出正确答案，却没有利用数组的连续存储、下标可随机访问、前后缀可复用这些条件。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。慢点在于重复搬移和重复求同一段信息。只要一个位置的状态已经由前缀、后缀、慢指针或边界指针确定，再回头扫描就是浪费。放到本题里，要重点观察“逆向填充避免覆盖尚未读取的 `nums1` 元素。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。优化要把数组切成若干语义明确

的区间：已经处理好的前缀、未知区、已经确定的后缀，或者左侧贡献和右侧贡献。双指针、前后缀数组、原地交换和稳定写入，本质都是让每个元素只进入少数几次状态转移。本题的优化应当围绕“逆向填充避免覆盖尚未读取的 `nums1` 元素。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。不变量必须能落到下标上：慢指针左侧保存什么，右指针右侧保存什么，当前下标之前是否已经满足题目要求。只要这个叙述含糊，代码中的加一减一就会变成猜。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。正确性通常用区间不变量证明。每次循环只扩大已经正确的区域，未知区严格缩小，且被移出的元素已经放到符合语义的位置。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。C++ 实现优先使用 `std::vector<int>` 和明确的整型下标。涉及乘积、面积、总和时要主动判断是否需要 `long long`。如果题目要求原地修改，要说明返回值表示新长度还是数组本身，避免把输出语义和容器容量混在一起。本题还要特别注意：第二个数组为空、第一个有效元素为空、重复值、尾部剩余。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：第二个数组为空、第一个有效元素为空、重复值、尾部剩余。若没有额外空位，只能新开数组或使用更复杂原地归并。实验时准备空数组、单元素、全相同、严格递增、严格递减、包含负数和极值的用例。把每轮指针位置打印出来，比只看最终答案更容易发现边界错误。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

189 轮转数组

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。右移 k 位等价于把数组切成两段后换序。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“数组与原地维护”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力做法通常枚举每个位置的最终去向，或者为每个位置重新扫描一段区间。它容易写出正确答案，却没有利用数组的连续存储、下标可随机访问、前后缀可复用这些条件。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。慢点在于重复搬移和重复求同一段信息。只要一个位置的状态已经由前缀、后缀、慢指针或边界指针确定，再回头扫描就是浪费。放到本题里，要重点观察“三次反转能原地完成：整体、前 k 、后 n 减 k 。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。优化要把数组切成若干语义明确的区间：已经处理好的前缀、未知区、已经确定的后缀，或者左侧贡献和右侧贡献。双指针、前后缀数组、原地交换和稳定写入，本质都是让每个元素只进入少数几次状态转移。本题的优化应当围绕“三次反转能原地完成：整体、前 k 、后 n 减 k 。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。不变量必须能落到下标上：慢指针左侧保存什么，右指针右侧保存什么，当前下标之前是否已经满足题目要求。只要这个叙述含糊，代码中的加一减一就会变成猜。用这道题复

盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。正确性通常用区间不变量证明。每次循环只扩大已经正确的区域，未知区严格缩小，且被移出的元素已经放到符合语义的位置。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。C++ 实现优先使用 `std::vector<int>` 和明确的整型下标。涉及乘积、面积、总和时要主动判断是否需要 `long long`。如果题目要求原地修改，要说明返回值表示新长度还是数组本身，避免把输出语义和容器容量混在一起。本题还要特别注意：k 为零、k 大于 n、空数组、单元素。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：k 为零、k 大于 n、空数组、单元素。环状替换也可做，但要处理最大公约数个循环。实验时准备空数组、单元素、全相同、严格递增、严格递减、包含负数和极值的用例。把每轮指针位置打印出来，比只看最终答案更容易发现边界错误。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

238 除自身以外数组的乘积

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。不能用除法时，答案由左侧乘积和右侧乘积组成。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“数组与原地维护”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力做法通常枚举每个位置的最终去向，或者为每个位置重新扫描一段区间。它容易写出正确答案，却没有利用数组的连续存储、下标可随机访问、前后缀可复用这些条件。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。慢点在于重复搬移和重复求同一段信息。只要一个位置的状态已经由前缀、后缀、慢指针或边界指针确定，再回头扫描就是浪费。放到本题里，要重点观察“先写入左侧前缀积，再从右向左乘右侧后缀积。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。优化要把数组切成若干语义明确的区间：已经处理好的前缀、未知区、已经确定的后缀，或者左侧贡献和右侧贡献。双指针、前后缀数组、原地交换和稳定写入，本质都是让每个元素只进入少数几次状态转移。本题的优化应当围绕“先写入左侧前缀积，再从右向左乘右侧后缀积。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。不变量必须能落到下标上：慢指针左侧保存什么，右指针右侧保存什么，当前下标之前是否已经满足题目要求。只要这个叙述含糊，代码中的加一减一就会变成猜。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。正确性通常用区间不变量证明。每次循环只扩大已经正确的区域，未知区严格缩小，且被移出的元素已经放到符合语义的位置。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。C++ 实现优先使用 `std::vector<int>` 和明确的整型下标。涉及乘积、面积、总和时要主动判断是否需要 `long long`。如果题目要求原地修改，要说明返回值表示新长度还是数组本身，避免把输出语义和容器容量混在一起。本题还要特别注意：一个零、多个零、负数、乘积溢出。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要

历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：一个零、多个零、负数、乘积溢出。这个题展示输出数组可作为状态存储的一部分。实验时准备空数组、单元素、全相同、严格递增、严格递减、包含负数和极值的用例。把每轮指针位置打印出来，比只看最终答案更容易发现边界错误。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

283 移动零

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。稳定保留非零元素顺序，把零集中到尾部。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“数组与原地维护”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力做法通常枚举每个位置的最终去向，或者为每个位置重新扫描一段区间。它容易写出正确答案，却没有利用数组的连续存储、下标可随机访问、前后缀可复用这些条件。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。慢点在于重复搬移和重复求同一段信息。只要一个位置的状态已经由前缀、后缀、慢指针或边界指针确定，再回头扫描就是浪费。放到本题里，要重点观察“慢指针写入下一个非零位置，最后补零或用交换。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。优化要把数组切成若干语义明确的区间：已经处理好的前缀、未知区、已经确定的后缀，或者左侧贡献和右侧贡献。双指针、前后缀数组、原地交换和稳定写入，本质都是让每个元素只进入少数几次状态转移。本题的优化应当围绕“慢指针写入下一个非零位置，最后补零或用交换。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。不变量必须能落到下标上：慢指针左侧保存什么，右指针右侧保存什么，当前下标之前是否已经满足题目要求。只要这个叙述含糊，代码中的加一减一就会变成猜。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。正确性通常用区间不变量证明。每次循环只扩大已经正确的区域，未知区严格缩小，且被移出的元素已经放到符合语义的位置。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。C++ 实现优先使用 `std::vector<int>` 和明确的整型下标。涉及乘积、面积、总和时要主动判断是否需要 `long long`。如果题目要求原地修改，要说明返回值表示新长度还是数组本身，避免把输出语义和容器容量混在一起。本题还要特别注意：全零、无零、零在开头结尾、稳定顺序要求。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：全零、无零、零在开头结尾、稳定顺序要求。若目标是移动指定值，模型与移除元素相同。实验时准备空数组、单元素、全相同、严格递增、严格递减、包含负数和极值的用例。把每轮指针位置打印出来，比只看最终答案更容易发现边界错误。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

581 最短无序连续子数组

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。要找破坏全局有序性的最左和最右位置。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“数组与原地维护”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力做法通常枚举每个位置的最终去向，或者为每个位置重新扫描一段区间。它容易写出正确答案，却没有利用数组的连续存储、下标可随机访问、前后缀可复用这些条件。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。慢点在于重复搬移和重复求同一段信息。只要一个位置的状态已经由前缀、后缀、慢指针或边界指针确定，再回头扫描就是浪费。放到本题里，要重点观察“从左维护历史最大值确定右边界，从右维护历史最小值确定左边界。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。优化要把数组切成若干语义明确的区间：已经处理好的前缀、未知区、已经确定的后缀，或者左侧贡献和右侧贡献。双指针、前后缀数组、原地交换和稳定写入，本质都是让每个元素只进入少数几次状态转移。本题的优化应当围绕“从左维护历史最大值确定右边界，从右维护历史最小值确定左边界。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。不变量必须能落到下标上：慢指针左侧保存什么，右指针右侧保存什么，当前下标之前是否已经满足题目要求。只要这个叙述含糊，代码中的加一减一就会变成猜。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。正确性通常用区间不变量证明。每次循环只扩大已经正确的区域，未知区严格缩小，且被移出的元素已经放到符合语义的位置。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。C++ 实现优先使用 `std::vector<int>` 和明确的整型下标。涉及乘积、面积、总和时要主动判断是否需要 `long long`。如果题目要求原地修改，要说明返回值表示新长度还是数组本身，避免把输出语义和容器容量混在一起。本题还要特别注意：已经有序、逆序、重复值、局部乱序。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：已经有序、逆序、重复值、局部乱序。排序比较法更直观但复杂度更高。实验时准备空数组、单元素、全相同、严格递增、严格递减、包含负数和极值的用例。把每轮指针位置打印出来，比只看最终答案更容易发现边界错误。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

647 回文子串

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。每个回文都有唯一中心或中心间隙。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“数组与原地维护”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力做法通常枚举每个位置的最终去向，或者为每个位置重新扫描一段区间。它容易写出正确答案，却没有利用数组的连续存储、下标可随机访问、前后缀可复用这些条件。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，

哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。慢点在于重复搬移和重复求同一段信息。只要一个位置的状态已经由前缀、后缀、慢指针或边界指针确定，再回头扫描就是浪费。放到本题里，要重点观察“对每个中心向两边扩展，扩展成功一次就计数一次。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。优化要把数组切成若干语义明确的区间：已经处理好的前缀、未知区、已经确定的后缀，或者左侧贡献和右侧贡献。双指针、前后缀数组、原地交换和稳定写入，本质都是让每个元素只进入少数几次状态转移。本题的优化应当围绕“对每个中心向两边扩展，扩展成功一次就计数一次。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。不变量必须能落到下标上：慢指针左侧保存什么，右指针右侧保存什么，当前下标之前是否已经满足题目要求。只要这个叙述含糊，代码中的加一减一就会变成猜。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。正确性通常用区间不变量证明。每次循环只扩大已经正确的区域，未知区严格缩小，且被移出的元素已经放到符合语义的位置。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。C++ 实现优先使用 `std::vector<int>` 和明确的整型下标。涉及乘积、面积、总和时要主动判断是否需要 `long long`。如果题目要求原地修改，要说明返回值表示新长度还是数组本身，避免把输出语义和容器容量混在一起。本题还要特别注意：空串、全相同字符、偶数回文、奇数回文。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：空串、全相同字符、偶数回文、奇数回文。Manacher 算法可线性解决，但中心扩展更适合面试基础。实验时准备空数组、单元素、全相同、严格递增、严格递减、包含负数和极值的用例。把每轮指针位置打印出来，比只看最终答案更容易发现边界错误。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

滑动窗口与双指针

下面的题卡都按同一条链路展开：从问题出发，先写暴力，再定位重复工作，然后选择数据结构或状态不变量，最后落到 C++ 实现、边界和扩展。格式统一是为了训练面试表达，而每张卡的关键观察和失效条件都要单独复盘。

图示：滑动窗口与双指针推导链

暴力基线	枚举候选、完整模拟或逐个检查，先保证覆盖全部可能答案。
瓶颈定位	真正的瓶颈不是两层循环本身，而是窗口状态没有被增量维护。右端移动带来的变化只有一个新元素，左端移动移走的也只有一个旧元素，若每次重新统计，就丢掉了题目给的连续性。
结构选择	滑动窗口成立必须有单调性或固定长度边界。右端扩展让某个条件单调变好或变坏，左端收缩可以恢复合法性或缩短答案。若数组含负数、状态会来回震荡，就不能硬套窗口。
不变量	窗口不变量要写成当前窗口满足什么条件，或者当前窗口是第一个可能满足条件的最短前缀。每次移动指针之前先更新状态，移动之后状态要和区间边界一致。
代码落地	C++ 中建议把窗口写成左闭右开或左闭右闭中的一种，并在全题坚持。字符计数用 <code>std::array<int, 128></code> 或 26 长度数组时要说明字符集假设；泛化字符集再换哈希表。

3 无重复字符的最长子串

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。窗口保存当前无重复子串，右端每次加入一个字符。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“滑动窗口与双指针”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力做法枚举所有左端和右端，再检查窗口是否合法。对于字符串和正数数组，这种写法会把相邻窗口中大量相同内容反复计算。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。真正的瓶颈不是两层循环本身，而是窗口状态没有被增量维护。右端移动带来的变化只有一个新元素，左端移动移走的也只有一个旧元素，若每次重新统计，就丢掉了题目给的连续性。放到本题里，要重点观察“用上次出现位置直接跳过无效左端，左边界只能向右不能回退。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。滑动窗口成立必须有单调性或固定长度边界。右端扩展让某个条件单调变好或变坏，左端收缩可以恢复合法性或缩短答案。若数组含负数、状态会来回震荡，就不能硬套窗口。本题的优化应当围绕“用上次出现位置直接跳过无效左端，左边界只能向右不能回退。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。窗口不变量要写成当前窗口满足什么条件，或者当前窗口是第一个可能满足条件的最短前缀。每次移动指针之前先更新状态，移动之后状态要和区间边界一致。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。证明重点是排除。某个左端被移过之后，所有以它为左端的更长窗口都不可能比当前答案更优，或者已经被当前状态完整统计。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。C++ 中建议把窗口写成左闭右开或左闭右闭中的一种，并在全题坚持。字符计数用 `std::array<int, 128>` 或 26 长度数组时要说明字符集假设；泛化字符集再换哈希表。本题还要特别注意：空串、全相同字符、重复字符出现在窗口左侧之外、扩展字符集。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域

大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：空串、全相同字符、重复字符出现在窗口左侧之外、扩展字符集。若要求恰好包含 k 种字符，窗口条件从无重复变成种类计数。实验应包含重复字符集中、目标字符缺失、窗口刚好覆盖、窗口必须连续收缩、多次同字符需求等样例。若把负数放进正数窗口题，应能解释为什么算法不再适用。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

11 盛最多水的容器

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。左右指针代表当前选取的两条边，面积由短板和宽度共同决定。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“滑动窗口与双指针”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力做法枚举所有左端和右端，再检查窗口是否合法。对于字符串和正数数组，这种写法会把相邻窗口中大量相同内容反复计算。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。真正的瓶颈不是两层循环本身，而是窗口状态没有被增量维护。右端移动带来的变化只有一个新元素，左端移动移走的也只有一个旧元素，若每次重新统计，就丢掉了题目给的连续性。放到本题里，要重点观察“每次移动短板，因为移动长板只会缩小宽度且不会提高短板。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。滑动窗口成立必须有单调性或固定长度边界。右端扩展让某个条件单调变好或变坏，左端收缩可以恢复合法性或缩短答案。若数组含负数、状态会来回震荡，就不能硬套窗口。本题的优化应当围绕“每次移动短板，因为移动长板只会缩小宽度且不会提高短板。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。窗口不变量要写成当前窗口满足什么条件，或者当前窗口是第一个可能满足条件的最短前缀。每次移动指针之前先更新状态，移动之后状态要和区间边界一致。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。证明重点是排除。某个左端被移过之后，所有以它为左端的更长窗口都不可能比当前答案更优，或者已经被当前状态完整统计。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。C++ 中建议把窗口写成左闭右开或左闭右闭中的一种，并在全题坚持。字符计数用 `std::array<int, 128>` 或 26 长度数组时要说明字符集假设；泛化字符集再换哈希表。本题还要特别注意：两端高度相等、数组长度为二、面积乘法可能溢出。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：两端高度相等、数组长度为二、面积乘法可能溢出。接雨水计算每个位置贡献，不能把本题双指针证明直接搬过去。实验应包含重复字符集中、目标字符缺失、窗口刚好覆盖、窗口必须连续收缩、多次同字符需求等样例。若把负数放进正数窗口题，应能解释为什么算法不再适用。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

15 三数之和

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。排序后固定第一个数，剩余两数转成有序双指针。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“滑动窗口与双指针”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力做法枚举所有左端和右端，再检查窗口是否合法。对于字符串和正数数组，这种写法会把相邻窗口中大量相同内容反复计算。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。真正的瓶颈不是两层循环本身，而是窗口状态没有被增量维护。右端移动带来的变化只有一个新元素，左端移动移走的也只有一个旧元素，若每次重新统计，就丢掉了题目给的连续性。放到本题里，要重点观察“和太小移动左指针，和太大移动右指针；固定值和左右值都要跳过重复。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。滑动窗口成立必须有单调性或固定长度边界。右端扩展让某个条件单调变好或变坏，左端收缩可以恢复合法性或缩短答案。若数组含负数、状态会来回震荡，就不能硬套窗口。本题的优化应当围绕“和太小移动左指针，和太大移动右指针；固定值和左右值都要跳过重复。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。窗口不变量要写成当前窗口满足什么条件，或者当前窗口是第一个可能满足条件的最短前缀。每次移动指针之前先更新状态，移动之后状态要和区间边界一致。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。证明重点是排除。某个左端被移过之后，所有以它为左端的更长窗口都不可能比当前答案更优，或者已经被当前状态完整统计。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。C++ 中建议把窗口写成左闭右开或左闭右闭中的一种，并在全题坚持。字符计数用 `std::array<int, 128>` 或 26 长度数组时要说明字符集假设；泛化字符集再换哈希表。本题还要特别注意：全零、重复元素很多、没有答案、结果顺序不要求但组合不能重复。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：全零、重复元素很多、没有答案、结果顺序不要求但组合不能重复。四数之和再外层固定一个数，并用 `long long` 防止目标和溢出。实验应包含重复字符集中、目标字符缺失、窗口刚好覆盖、窗口必须连续收缩、多次同字符需求等样例。若把负数放进正数窗口题，应能解释为什么算法不再适用。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

16 最接近的三数之和

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。排序后固定一数，双指针寻找最接近目标的两数和。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“滑动窗口与双指针”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力做法枚举所有左端和右端，再检查窗口是否合法。对于字符串

和正数数组，这种写法会把相邻窗口中大量相同内容反复计算。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。真正的瓶颈不是两层循环本身，而是窗口状态没有被增量维护。右端移动带来的变化只有一个新元素，左端移动移走的也只有一个旧元素，若每次重新统计，就丢掉了题目给的连续性。放到本题里，要重点观察“每次根据当前和与目标的大小移动指针，同时更新最小绝对差。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。滑动窗口成立必须有单调性或固定长度边界。右端扩展让某个条件单调变好或变坏，左端收缩可以恢复合法性或缩短答案。若数组含负数、状态会来回震荡，就不能硬套窗口。本题的优化应当围绕“每次根据当前和与目标的大小移动指针，同时更新最小绝对差。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。窗口不变量要写成当前窗口满足什么条件，或者当前窗口是第一个可能满足条件的最短前缀。每次移动指针之前先更新状态，移动之后状态要和区间边界一致。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。证明重点是排除。某个左端被移过之后，所有以它为左端的更长窗口都不可能比当前答案更优，或者已经被当前状态完整统计。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。C++ 中建议把窗口写成左闭右开或左闭右闭中的一种，并在全题坚持。字符计数用 `std::array<int, 128>` 或 26 长度数组时要说明字符集假设；泛化字符集再换哈希表。本题还要特别注意：差值相等、负数、目标很大、三数和溢出。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：差值相等、负数、目标很大、三数和溢出。若要求返回所有最接近组合，需要记录同差值并去重。实验应包含重复字符集中、目标字符缺失、窗口刚好覆盖、窗口必须连续收缩、多次同字符需求等样例。若把负数放进正数窗口题，应能解释为什么算法不再适用。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

18 四数之和

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。两层固定加一层双指针，关键是剪枝和去重。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“滑动窗口与双指针”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力做法枚举所有左端和右端，再检查窗口是否合法。对于字符串和正数数组，这种写法会把相邻窗口中大量相同内容反复计算。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。真正的瓶颈不是两层循环本身，而是窗口状态没有被增量维护。右端移动带来的变化只有一个新元素，左端移动移走的也只有一个旧元素，若每次重新统计，就丢掉了题目给的连续性。放到本题里，要重点观察“排序提供单调性，外层可用最小可能和和最大可能和提前跳过。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。滑动窗口成立必须有单调性或固定长度边界。右端扩展让某个条件单调变好或变坏，左端收缩可以恢复合法性或缩短答案。若数组含负数、状态会来回震荡，就不能硬套窗口。本题的优化应当围绕“排序提供单调性，外层可用最小可能和和最大可能和提前跳过。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。窗口不变量要写成当前窗口满足什么条件，或者当前窗口是第一个可能满足条件的最短前缀。每次移动指针之前先更新状态，移动之后状态要和区间边界一致。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。证明重点是排除。某个左端被移过之后，所有以它为左端的更长窗口都不可能比当前答案更优，或者已经被当前状态完整统计。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。C++ 中建议把窗口写成左闭右开或左闭右闭中的一种，并在全题坚持。字符计数用 `std::array<int, 128>` 或 26 长度数组时要说明字符集假设；泛化字符集再换哈希表。本题还要特别注意：重复元素、目标超出 `int`、答案很多导致输出空间主导复杂度。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：重复元素、目标超出 `int`、答案很多导致输出空间主导复杂度。 k 数之和可以递归降维，但工程实现要控制重复和边界。实验应包含重复字符集中、目标字符缺失、窗口刚好覆盖、窗口必须连续收缩、多次同字符需求等样例。若把负数放进正数窗口题，应能解释为什么算法不再适用。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

76 最小覆盖子串

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。窗口内字符计数必须覆盖目标计数，满足后尽量收缩左端。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“滑动窗口与双指针”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力做法枚举所有左端和右端，再检查窗口是否合法。对于字符串和正数数组，这种写法会把相邻窗口中大量相同内容反复计算。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。真正的瓶颈不是两层循环本身，而是窗口状态没有被增量维护。右端移动带来的变化只有一个新元素，左端移动移走的也只有一个旧元素，若每次重新统计，就丢掉了题目给的连续性。放到本题里，要重点观察“用 `formed` 记录满足需求的字符种类，避免每次全量比较。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。滑动窗口成立必须有单调性或固定长度边界。右端扩展让某个条件单调变好或变坏，左端收缩可以恢复合法性或缩短答案。若数组含负数、状态会来回震荡，就不能硬套窗口。本题的优化应当围绕“用 `formed` 记录满足需求的字符种类，避免每次全量比较。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。窗口不变量要写成当前窗口满足什么条件，或者当前窗口是第一个可能满足条

件的最短前缀。每次移动指针之前先更新状态，移动之后状态要和区间边界一致。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。证明重点是排除。某个左端被移过之后，所有以它为左端的更长窗口都不可能比当前答案更优，或者已经被当前状态完整统计。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。C++ 中建议把窗口写成左闭右开或左闭右闭中的一种，并在全题坚持。字符计数用 `std::array<int, 128>` 或 26 长度数组时要说明字符集假设；泛化字符集再换哈希表。本题还要特别注意：目标有重复字符、源串缺字符、最小窗口在开头或结尾、大小写。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：目标有重复字符、源串缺字符、最小窗口在开头或结尾、大小写。若字符集固定，数组计数更快；若是 Unicode，哈希表更通用。实验应包含重复字符集中、目标字符缺失、窗口刚好覆盖、窗口必须连续收缩、多次同字符需求等样例。若把负数放进正数窗口题，应能解释为什么算法不再适用。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

209 长度最小的子数组

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。正数数组让窗口和随右端增加、随左端减少。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“滑动窗口与双指针”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力做法枚举所有左端和右端，再检查窗口是否合法。对于字符串和正数数组，这种写法会把相邻窗口中大量相同内容反复计算。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。真正的瓶颈不是两层循环本身，而是窗口状态没有被增量维护。右端移动带来的变化只有一个新元素，左端移动移走的也只有一个旧元素，若每次重新统计，就丢掉了题目给的连续性。放到本题里，要重点观察“右端扩展到满足目标后，不断左移缩短并更新答案。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。滑动窗口成立必须有单调性或固定长度边界。右端扩展让某个条件单调变好或变坏，左端收缩可以恢复合法性或缩短答案。若数组含负数、状态会来回震荡，就不能硬套窗口。本题的优化应当围绕“右端扩展到满足目标后，不断左移缩短并更新答案。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。窗口不变量要写成当前窗口满足什么条件，或者当前窗口是第一个可能满足条件的最短前缀。每次移动指针之前先更新状态，移动之后状态要和区间边界一致。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。证明重点是排除。某个左端被移过之后，所有以它为左端的更长窗口都不可能比当前答案更优，或者已经被当前状态完整统计。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。C++ 中建议把窗口写成左闭右开或左闭右闭中的一种，并在全题坚持。字符计数用 `std::array<int, 128>` 或 26 长度数组时要说明字符集假设；泛化字符集再换哈希表。本题还要特别注意：不存在答案、单元素达到目标、全正是必要条件。写代码前先把边界条件放进函数

入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：不存在答案、单元素达到目标、全正是必要条件。若含负数，需要前缀和加单调队列。实验应包含重复字符集中、目标字符缺失、窗口刚好覆盖、窗口必须连续收缩、多次同字符需求等样例。若把负数放进正数窗口题，应能解释为什么算法不再适用。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

438 找到字符串中所有字母异位词

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。固定长度窗口内字符频次等于目标频次即为答案。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“滑动窗口与双指针”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力做法枚举所有左端和右端，再检查窗口是否合法。对于字符串和正数数组，这种写法会把相邻窗口中大量相同内容反复计算。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。真正的瓶颈不是两层循环本身，而是窗口状态没有被增量维护。右端移动带来的变化只有一个新元素，左端移动移走的也只有一个旧元素，若每次重新统计，就丢掉了题目给的连续性。放到本题里，要重点观察“右进左出维护计数，窗口长度达到目标长度后检查差异。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。滑动窗口成立必须有单调性或固定长度边界。右端扩展让某个条件单调变好或变坏，左端收缩可以恢复合法性或缩短答案。若数组含负数、状态会来回震荡，就不能硬套窗口。本题的优化应当围绕“右进左出维护计数，窗口长度达到目标长度后检查差异。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。窗口不变量要写成当前窗口满足什么条件，或者当前窗口是第一个可能满足条件的最短前缀。每次移动指针之前先更新状态，移动之后状态要和区间边界一致。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。证明重点是排除。某个左端被移过之后，所有以它为左端的更长窗口都不可能比当前答案更优，或者已经被当前状态完整统计。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。C++ 中建议把窗口写成左闭右开或左闭右闭中的一种，并在全题坚持。字符计数用 `std::array<int, 128>` 或 26 长度数组时要说明字符集假设；泛化字符集再换哈希表。本题还要特别注意：目标比源串长、重复字符、字符集固定、答案重叠。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：目标比源串长、重复字符、字符集固定、答案重叠。维护差异计数可以避免每次比较整个数组。实验应包含重复字符集中、目标字符缺失、窗口刚好覆盖、窗口必须连续收缩、多次同字符需求等样例。若把负数放进正数窗口题，应能解释为什么算法不再适用。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

第 5 章

哈希表和前缀和

哈希表解决“快速查历史信息”，前缀和解决“快速计算区间信息”。两者结合后，可以把很多子数组问题从 $O(n^2)$ 降到 $O(n)$ 。

两数之和是哈希表入门，但真正要记住的是模型：当前元素需要查询历史中是否存在某个补数。只要查询对象可以作为 key，哈希表就能把线性查找降为平均常数查找。暴力方法枚举 i 和 j ，判断 $\text{nums}[i] + \text{nums}[j] == \text{target}$ ，时间复杂度 $O(n^2)$ 。慢点不在加法，而在“为了每个数反复查找另一个数”。哈希表版本从左到右扫描，用 `value_to_index` 保存已经出现过的值及其下标；处理当前值时，只查 $\text{target} - \text{nums}[i]$ 是否在历史中出现过。

Listing 5.1 两数之和：保存历史值到下标

```
1 std::vector<int> two_sum_hash(const std::vector<int>& nums, int target)
2 {
3     std::unordered_map<int, int> value_to_index;
4     value_to_index.reserve(nums.size());
5
6     for (int index = 0; index < static_cast<int>(nums.size()); ++index) {
7         const int need = target - nums[index];
8         const auto found = value_to_index.find(need);
9         if (found != value_to_index.end()) {
10             return {found->second, index};
11         }
12         value_to_index.emplace(nums[index], index);
13     }
14
15     return {};
16 }
```

这段代码必须先查再插入，避免同一个元素和自己配对。如果数组里有重复值，`emplace` 会保留第一次出现的下标；对力扣 1 这种保证唯一答案的题足够。如果题目要求所有答案，就不能这样提前返回，而要继续扫描并处理去重。面试时可以这样表达：哈希表保存历史值到下标的映射，查询 key 是当前值的补数，平均时间 $O(n)$ ，空间 $O(n)$ 。

前缀和的定义是：`prefix[i]` 表示前 i 个元素的和。那么子数组 `[left, right]` 的和是 `prefix[right + 1] - prefix[left]`。如果题目问和为 k 的子数组个数，就等价于对每个当前位置查找之前有多少个前缀和等于 `current_prefix - k`。

Listing 5.2 和为 K 的子数组

```
1 int subarray_sum(const std::vector<int>& nums, int k)
2 {
3     std::unordered_map<int, int> prefix_count;
4     prefix_count.reserve(nums.size() + 1);
5     prefix_count[0] = 1;
6
7     int prefix = 0;
8     int answer = 0;
9     for (int x : nums) {
10         prefix += x;
```

```

11     const int need = prefix - k;
12     if (const auto found = prefix_count.find(need); found != prefix_count.end()) ←
    ↪ {
13         answer += found->second;
14     }
15     ++prefix_count[prefix];
16 }
17 return answer;
18 }

```

为什么先查再插入？因为当前前缀只能和之前的前缀组成非空子数组。如果先插入，在 $k == 0$ 时可能把当前位置和自己配对。为什么保存次数而不是下标？因为题目问个数，一个当前前缀可以和多个历史前缀组成答案。

最长连续序列展示 `unordered_set` 的另一种用法。把所有数放进集合后，只从没有前驱 $x - 1$ 的数开始扩展。这样每条连续链只从头部扫描一次，每个数最多被访问常数次，平均时间 $O(n)$ 。

哈希题常见错误包括：用 `operator[]` 做纯查询导致插入多余 key；忘记处理重复值；依赖 `unordered_map` 遍历顺序；没有考虑 `long long` 前缀和溢出；把滑动窗口误用到含负数数组。

最长连续序列要求在未排序数组中找最长的连续整数链。排序可以做，复杂度 $O(n \log n)$ ；暴力方法从每个数开始向后找 $x + 1$ 、 $x + 2$ ，如果每次都线性查找，会退化到 $O(n^2)$ 。哈希集合优化了“某个数是否存在”的查询，但如果从每个数都向后扩展，连续链中间的数会被重复扫描。真正的关键是只从链头开始扩展：如果 $x - 1$ 存在，说明 x 不是开头，跳过。

Listing 5.3 最长连续序列：只从链头扩展

```

1 int longest_consecutive(const std::vector<int>& nums)
2 {
3     std::unordered_set<int> values;
4     values.reserve(nums.size());
5     for (const int x : nums) {
6         values.insert(x);
7     }
8
9     int best = 0;
10    for (const int x : values) {
11        if (values.contains(x - 1)) {
12            continue;
13        }
14
15        int current = x;
16        int length = 1;
17        while (values.contains(current + 1)) {
18            ++current;
19            ++length;
20        }
21        best = std::max(best, length);
22    }
23
24    return best;
25 }

```

这题的摊还分析要讲清楚：每条连续链只会从最小值开始扫描一次，链中每个元素只在这次扫描里被访问，因此平均时间是 $O(n)$ 。不能在遍历 `values` 时删除元素，除非你非常清楚迭代器失效规则；本书这里选择不删除，代码更直接。

四数相加 II 展示“拆半”的哈希思想。题目给四个数组，统计 $a + b + c + d == 0$ 的元组数量。暴力枚举四个下标是 $O(n^4)$ 。如果固定前三个再查第四个，仍然是 $O(n^3)$ 。更好的做法是把四个数组拆成两组：先统计所有 $a + b$ 的和出现次数，再枚举 $c + d$ ，查询 $-(c + d)$ 出现过多少次。

Listing 5.4 四数相加 II：两两拆分

```

1 int four_sum_count(const std::vector<int>& nums1,
2                   const std::vector<int>& nums2,
3                   const std::vector<int>& nums3,
4                   const std::vector<int>& nums4)
5 {
6     std::unordered_map<int, int> pair_sum_count;
7     pair_sum_count.reserve(nums1.size() * nums2.size());
8
9     for (const int a : nums1) {
10         for (const int b : nums2) {
11             ++pair_sum_count[a + b];
12         }
13     }
14
15     int answer = 0;
16     for (const int c : nums3) {
17         for (const int d : nums4) {
18             const auto found = pair_sum_count.find(-(c + d));
19             if (found != pair_sum_count.end()) {
20                 answer += found->second;
21             }
22         }
23     }
24
25     return answer;
26 }

```

拆半的本质是把 n^4 的组合空间拆成两个 n^2 的集合，再用哈希表连接它们。空间复杂度是 $O(n^2)$ 。这类题的面试表达要说明为什么保存次数：同一个两数和可能由多个下标对产生，每一个都能和后半部分组成不同元组。

哈希表的底层原理也要理解。哈希函数把 key 映射到桶编号；如果多个 key 映射到同一个桶，就产生冲突。标准库实现细节不完全相同，但复杂度承诺通常是平均 $O(1)$ ，最坏可能退化。为了让平均复杂度可靠，容器会在负载因子过高时 rehash，增加桶数量并重新分布元素。刷题中常用 `reserve` 预留容量，减少 rehash 次数；工程中还要考虑自定义 key 的哈希质量、相等比较是否正确、是否可能遭遇恶意输入。

Listing 5.5 自定义 pair 哈希：把结构化 key 放进 unordered_map

```

1 struct PairHash {
2     std::size_t operator()(const std::pair<int, int>& value) const noexcept
3     {
4         constexpr int HASH_SHIFT = 32;
5         const auto high = static_cast<std::uint64_t>(
6             static_cast<std::uint32_t>(value.first));
7         const auto low = static_cast<std::uint64_t>(
8             static_cast<std::uint32_t>(value.second));
9         return std::hash<std::uint64_t>{}((high << HASH_SHIFT) ^ low);

```



```

10     }
11 };
12
13 using PointCount = std::unordered_map<std::pair<int, int>, int, PairHash>;

```

这段代码说明 key 不一定只能是整数或字符串。只要能定义“相等”和“哈希”，结构化对象也可以作为 key。要注意哈希相等的必要条件：如果两个 key 相等，它们的哈希值必须相同；如果两个 key 哈希值相同，它们不一定相等，容器还会用相等比较区分。很多工程 bug 来自只写了哈希函数，却忘记相等语义，或者哈希组合方式让大量不同 key 落入相同桶。

字母异位词分组是 key 设计题。排序后的字符串可以作为 key，简单直接；字符计数也可以作为 key，避免每个单词排序。若只含小写字母，26 维计数就是一个规范表示。难点在于把计数数组变成无歧义 key。如果直接拼接数字，**1,11** 和 **11,1** 可能混淆；加入分隔符或使用数组作为自定义 key 更安全。

Listing 5.6 字母异位词分组：计数 key 带分隔符

```

1 std::vector<std::vector<std::string>> group_anagrams(
2     const std::vector<std::string>& words)
3 {
4     std::unordered_map<std::string, std::vector<std::string>> groups;
5     groups.reserve(words.size());
6
7     for (const std::string& word : words) {
8         std::array<int, 26> count{};
9         for (const char ch : word) {
10             ++count[ch - 'a'];
11         }
12
13         std::string key;
14         for (const int value : count) {
15             key += '#';
16             key += std::to_string(value);
17         }
18         groups[key].push_back(word);
19     }
20
21     std::vector<std::vector<std::string>> answer;
22     answer.reserve(groups.size());
23     for (auto& [key, group] : groups) {
24         (void)key;
25         answer.push_back(std::move(group));
26     }
27     return answer;
28 }

```

这个题的优化取舍要会说。排序 key 每个单词成本是 $O(L \log L)$ ，计数 key 是 $O(L + |\Sigma|)$ 。当字符集固定且较小，计数更好；当字符集复杂或实现时间紧，排序 key 更稳。面试中并不是所有题都追求理论最优，清晰、正确、符合约束的方案更重要。

连续的子数组和使用同余前缀。若两个前缀和对 k 的余数相同，它们之间的子数组和就是 k 的倍数。题目要求长度至少为 2，所以哈希表保存每个余数最早出现的位置。为什么保存最早位置？因为越早越容易满足长度条件；后面相同余数不应该覆盖它。初始化余数 0 的位置为 -1，表示从数组开头到当前位置的前缀本身就能被 k 整除。

Listing 5.7 连续的子数组和：余数最早位置

```

1 bool check_subarray_sum(const std::vector<int>& nums, int k)
2 {
3     std::unordered_map<int, int> first_index;
4     first_index.reserve(nums.size() + 1);
5     first_index[0] = -1;
6
7     int prefix = 0;
8     for (int i = 0; i < static_cast<int>(nums.size()); ++i) {
9         prefix += nums[i];
10        const int remainder = prefix % k;
11        const auto found = first_index.find(remainder);
12        if (found != first_index.end()) {
13            if (i - found->second >= 2) {
14                return true;
15            }
16        } else {
17            first_index[remainder] = i;
18        }
19    }
20    return false;
21 }

```

这题的关键不是取模代码，而是“同余差值可整除”的数学关系。若题目允许 $k == 0$ ，取模会出错，需要单独处理连续至少两个 0 或直接用前缀和相等判断；力扣当前约束通常给出非零 k ，但面试中要先确认。前缀和如果可能很大，要用 `long long`；取模后余数范围有限，才适合放进哈希表。

连续数组把 0 和 1 数量相等转成前缀和相等。把 0 看成 -1，1 看成 +1，一个区间内 0 和 1 数量相等，就等价于区间转换后的和为 0。扫描前缀和时，若某个前缀和第二次出现，两次出现之间的区间和为 0。因为要求最长长度，所以保存每个前缀和第一次出现的位置。

Listing 5.8 连续数组：0 转成 -1 后找相同前缀和

```

1 int find_max_length_equal_zero_one(const std::vector<int>& nums)
2 {
3     std::unordered_map<int, int> first_index;
4     first_index.reserve(nums.size() * 2);
5     first_index[0] = -1;
6
7     int prefix = 0;
8     int answer = 0;
9     for (int i = 0; i < static_cast<int>(nums.size()); ++i) {
10        prefix += nums[i] == 0 ? -1 : 1;
11        const auto found = first_index.find(prefix);
12        if (found != first_index.end()) {
13            answer = std::max(answer, i - found->second);
14        } else {
15            first_index[prefix] = i;
16        }
17    }
18    return answer;
19 }

```

这个转换技巧可以推广。两个类别数量相等，可以把一个类别记为 +1，另一个记为 -1；多个类别的平衡问题，则可能需要向量差作为 key。比如同时要求 A 和 B、A 和 C 数量差保持一致，可以保存二维差值。哈希表的 key 设计会更复杂，但核心仍然是：把区间约束转成两个前缀状态相等。

快乐数和环检测展示哈希集合的“历史状态去重”。每次把数字替换为各位平方和，如果出现 1 就成功；如果某个中间数字再次出现，就进入循环，不可能再到 1。哈希集合保存见过的数字，快慢指针也能做，因为转换函数把每个数字映射到唯一后继。哈希写法更容易解释，快慢指针空间更低。

Listing 5.9 快乐数：哈希集合检测循环

```

1 int digit_square_sum(int value)
2 {
3     int sum = 0;
4     while (value > 0) {
5         const int digit = value % 10;
6         sum += digit * digit;
7         value /= 10;
8     }
9     return sum;
10 }
11
12 bool is_happy(int n)
13 {
14     std::unordered_set<int> seen;
15     while (n != 1 && !seen.contains(n)) {
16         seen.insert(n);
17         n = digit_square_sum(n);
18     }
19     return n == 1;
20 }

```

这个题让我们看到哈希不只用于数组下标，也用于状态空间去重。图搜索中的 `visited` 集合、字符串 BFS 中的字典删除、回溯中的去重集合，都是同一思想。只要状态能被规范编码成 key，就可以避免重复访问。

哈希和前缀和也有边界。第一，哈希表不能提供有序关系；需要最小大于、最大不超过、排名、区间统计时，要考虑平衡树、树状数组、线段树。第二，前缀和适合可逆的区间聚合，例如加法；最大值、最小值不能用两个前缀值相减得到，通常要用稀疏表、线段树或单调队列。第三，哈希表平均常数不等于稳定常数，key 很大、字符串很长、自定义哈希很差时，实际成本会升高。

深入理解

从源码视角看，`unordered_map` 的一个元素通常不只是 key 和 value，还包含节点指针、哈希桶链接等额外信息。大量小对象会带来分配成本和缓存不友好。频次统计中，如果 key 是小范围整数，用数组计数通常比哈希表更快；如果 key 是稀疏大整数或字符串，哈希表才体现优势。刷题答案可以写 `unordered_map`，但工程判断要先看 key 的值域、数量、生命周期和访问模式。

哈希实验：第一，把两数之和分别用排序双指针和哈希表实现，比较是否需要保留原下标。第二，给字母异位词分组构造长单词，比较排序 key 和计数 key 的耗时。第三，给连续数组打印前缀和第一次出现位置，观察为什么不能覆盖旧下标。第四，用数组计数和 `unordered_map` 计数分别统计小写字母频次，比较常数差异。

哈希题复盘模板

哈希题不要只写“用 map”。复盘时必须写：key 是什么，value 是什么；查询发生在插入前还是插入后；重复 key 怎么处理；是否需要次数、最早位置、最新位置还是集合存在性；平均复杂度和最坏情况分别是什么。

本章力扣主线：1 两数之和、49 字母异位词分组、128 最长连续序列、560 和为 K 的子数组、523 连续的子数组和、525 连续数组、454 四数相加 II。复盘时必须说明哈希表保存的是值、下标、次数还是最早位置。

hash-binary-stack 深度题卡

前缀和与哈希表

下面的题卡都按同一条链路展开：从问题出发，先写暴力，再定位重复工作，然后选择数据结构或状态不变量，最后落到 C++ 实现、边界和扩展。格式统一是为了训练面试表达，而每张卡的关键观察和失效条件都要单独复盘。

图示：前缀和与哈希表推导链

暴力基线	枚举候选、完整模拟或逐个检查，先保证覆盖全部可能答案。
瓶颈定位	瓶颈在于当前右端需要寻找很多历史左端。若这些历史状态能被同一个键表示，就不该逐个扫描，而应把历史状态压进哈希表。
结构选择	前缀和把区间问题改写成两个前缀状态的差；哈希表把寻找匹配前缀改成查表。频次表、最早位置表、最近位置表对应不同目标：计数、最长区间、最短区间不能混用。
不变量	哈希表的不变量通常是当前下标之前的历史前缀信息。先查询还是先更新不是风格问题，而是决定是否允许空区间和是否重复使用当前前缀。
代码落地	实现时用 <code>find</code> 判断是否存在，用 <code>operator[]</code> 只在确认要插入或累加时使用。总和可能溢出时使用 <code>long long</code> ；模运算要处理负数余数归一化。

1 两数之和

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。当前元素只需要寻找唯一补数，历史值到下标的映射就是最小状态。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“前缀和与哈希表”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力做法枚举所有区间，直接求区间和或重新比较频次。即使用前缀和把求和降到常数，仍可能保留枚举左右端点的平方复杂度。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。瓶颈在于当前右端需要寻找很多历史左端。若这些历史状态能被同一个键表示，就不该逐个扫描，而应把历史状态压进哈希表。放到本题里，要重点观察“先查再插入，避免同一个下标被用两次；若数组有序才考虑双指针。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。前缀和把区间问题改写成两个前缀状态的差；哈希表把寻找匹配前缀改成查表。频次表、最早位置表、最近位置表对应不同目标：计数、最长区间、最短区间不能混用。本题的优化应当围绕“先查再插入，避免同一个下标被用两次；若数组有序才考虑双指针。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生

改变,就必须重新验证这个结构是否仍然成立。

不变量和证明。哈希表的不变量通常是当前下标之前的历史前缀信息。先查询还是先更新不是风格问题,而是决定是否允许空区间和是否重复使用当前前缀。用这道题复盘时,把不变量写成三句话:已经处理的部分满足什么,尚未处理的部分还保留什么可能性,当前操作为什么不会丢掉最优答案或合法答案。正确性来自代数等价。若两个前缀满足差值、同余或计数差要求,则它们之间的区间正好满足题意;反过来任意合法区间也对应这样一对前缀。证明不需要很形式化,但必须能排除反例;如果只能说“这样写好像可以”,说明你还没有把优化从暴力中推导出来。

代码落地。实现时用 `find` 判断是否存在,用 `operator[]` 只在确认要插入或累加时使用。总和可能溢出时使用 `long long`;模运算要处理负数余数归一化。本题还要特别注意:重复数字、目标为偶数且补数等于当前值、没有答案时返回空结果。写代码前先把边界条件放进函数入口、循环条件或状态定义里,而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致:需要顺序就别用无序表,需要极值就别完整排序,需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明,不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖:最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题,边界复盘重点是:重复数字、目标为偶数且补数等于当前值、没有答案时返回空结果。若改成返回所有不重复数对,要先排序或用频次表处理重复。实验要覆盖从下标零开始的答案、目标为零、负数、重复前缀、余数相同但长度不足、哈希表保留第一次出现还是累加次数的差异。实验报告建议记录三件事:暴力版本在什么规模开始变慢,优化版本减少了哪类重复工作,哪一个边界用例最容易打破错误实现。

49 字母异位词分组

从问题出发。先不要问“这题套什么模板”,而要把题面翻译成对象、关系、约束和输出目标。异位词共享同一个规范表示,可以是排序字符串或字符计数。对象决定我们要保存数组下标、节点、区间、字符还是图状态;关系决定候选之间能不能比较、合并或淘汰;输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后,才进入“前缀和与哈希表”这条主线。

先把暴力写清楚。按上面的模型,最直接的办法是完整枚举可能答案或完整模拟题目过程,再对每个候选重新检查是否合法。暴力做法枚举所有区间,直接求区间和或重新比较频次。即使用前缀和把求和降到常数,仍可能保留枚举左右端点的平方复杂度。写暴力不是为了提交它,而是为了暴露状态空间:哪些候选一定会出现,哪些判断被反复做,哪些中间量其实只和少量历史状态有关。只有这一步说清楚,后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢,通常不是因为代码写得不够巧,而是因为同一类信息被一次次重新取得。瓶颈在于当前右端需要寻找很多历史左端。若这些历史状态能被同一个键表示,就不该逐个扫描,而应把历史状态压进哈希表。放到本题里,要重点观察“哈希表按规范键分桶,避免两两比较字符串。”这条线索:它说明有一部分信息可以从已经处理过的输入中继承下来,而不需要回头重算。这里的关键观察不是结论,而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位,下一步才是选择结构。前缀和把区间问题改写成两个前缀状态的差;哈希表把寻找匹配前缀改成查表。频次表、最早位置表、最近位置表对应不同目标:计数、最长区间、最短区间不能混用。本题的优化应当围绕“哈希表按规范键分桶,避免两两比较字符串。”建立一个可维护状态:新增输入只更新一次,过期候选及时删除,已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入,而应只处理这一步真正变化的那一小块状态。若题目条件发生变化,例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变,就必须重新验证这个结构是否仍然成立。

不变量和证明。哈希表的不变量通常是当前下标之前的历史前缀信息。先查询还是先更新不是风格问题,而是决定是否允许空区间和是否重复使用当前前缀。用这道题复盘时,把不变量写成三句话:已经处理的部分满足什么,尚未处理的部分还保留什么可能性,当前操作为什么不会丢掉最优答案或合法答案。正确性来自代数等价。若两个前缀满足差值、同余或计数差要求,则它们之间的区间正好满足题意;反过来任意合法区间也对应这样一对前缀。证明不需要很形式化,但必须能排除反例;如果只能说“这样写好像可以”,说明你还没有把优化从暴力中推导出来。

代码落地。实现时用 `find` 判断是否存在,用 `operator[]` 只在确认要插入或累加时使用。总和可能溢出时使用 `long long`;模运算要处理负数余数归一化。本题还要特别注意:空字符串、重复单词、字符集不止小写字母、计数键必须无歧义。写代码前先把边界条件放进函数入口、循环条

件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：空字符串、重复单词、字符集不止小写字母、计数键必须无歧义。若字符串很长且字符集固定，计数键通常比排序键更稳定。实验要覆盖从下标零开始的答案、目标为零、负数、重复前缀、余数相同但长度不足、哈希表保留第一次出现还是累加次数的差异。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

128 最长连续序列

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。连续段只需要从没有前驱的数开始扩展。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“前缀和与哈希表”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力做法枚举所有区间，直接求区间和或重新比较频次。即使用前缀和把求和降到常数，仍可能保留枚举左右端点的平方复杂度。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。瓶颈在于当前右端需要寻找很多历史左端。若这些历史状态能被同一个键表示，就不该逐个扫描，而应把历史状态压进哈希表。放到本题里，要重点观察“哈希集合判断 x 减一是否存在，跳过非起点避免重复扫描。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。前缀和把区间问题改写成两个前缀状态的差；哈希表把寻找匹配前缀改成查表。频次表、最早位置表、最近位置表对应不同目标：计数、最长区间、最短区间不能混用。本题的优化应当围绕“哈希集合判断 x 减一是否存在，跳过非起点避免重复扫描。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。哈希表的不变量通常是当前下标之前的历史前缀信息。先查询还是先更新不是风格问题，而是决定是否允许空区间和是否重复使用当前前缀。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。正确性来自代数等价。若两个前缀满足差值、同余或计数差要求，则它们之间的区间正好满足题意；反过来任意合法区间也对应这样一对前缀。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。实现时用 `find` 判断是否存在，用 `operator[]` 只在确认要插入或累加时使用。总和可能溢出时使用 `long long`；模运算要处理负数余数归一化。本题还要特别注意：重复元素、负数、空数组、整数边界。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：重复元素、负数、空数组、整数边界。若数据流动态插入，可以用并查集或区间合并维护长度。实验要覆盖从下标零开始的答案、目标为零、负数、重复前缀、余数相同但长度不足、哈希表保留第一次出现还是累加次数的差异。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

202 快乐数

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。反复变换数字会进入一或循环，状态空间最终有限。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“前缀和与哈希表”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力做法枚举所有区间，直接求区间和或重新比较频次。即使用前缀和把求和降到常数，仍可能保留枚举左右端点的平方复杂度。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。瓶颈在于当前右端需要寻找很多历史左端。若这些历史状态能被同一个键表示，就不该逐个扫描，而应把历史状态压进哈希表。放到本题里，要重点观察“用集合检测重复状态，或用快慢指针看成函数图。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。前缀和把区间问题改写成两个前缀状态的差；哈希表把寻找匹配前缀改成查表。频次表、最早位置表、最近位置表对应不同目标：计数、最长区间、最短区间不能混用。本题的优化应当围绕“用集合检测重复状态，或用快慢指针看成函数图。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。哈希表的不变量通常是当前下标之前的历史前缀信息。先查询还是先更新不是风格问题，而是决定是否允许空区间和是否重复使用当前前缀。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。正确性来自代数等价。若两个前缀满足差值、同余或计数差要求，则它们之间的区间正好满足题意；反过来任意合法区间也对应这样一对前缀。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。实现时用 `find` 判断是否存在，用 `operator[]` 只在确认要插入或累加时使用。总和可能溢出时使用 `long long`；模运算要处理负数余数归一化。本题还要特别注意：输入为一、进入循环、位平方和函数、负数不在题意内。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：输入为一、进入循环、位平方和函数、负数不在题意内。快慢指针版本能训练把数值迭代看成链表。实验要覆盖从下标零开始的答案、目标为零、负数、重复前缀、余数相同但长度不足、哈希表保留第一次出现还是累加次数的差异。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

217 存在重复元素

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。判断是否出现过是哈希集合最直接的职责。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“前缀和与哈希表”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力做法枚举所有区间，直接求区间和或重新比较频次。即使用前缀和把求和降到常数，仍可能保留枚举左右端点的平方复杂度。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。瓶颈在于当前右端需要寻找很多历史左端。若这些历史状态能被同一个键表示，就不该逐个扫描，而应把历史状态压进哈希表。放到本题里，要重点观察“扫描时若插入前已经存在则立即返回真。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。前缀和把区间问题改写成两个前缀状态的差；哈希表把寻找匹配前缀改成查表。频次表、最早位置表、最近位置表对应不同目标：计数、最长区间、最短区间不能混用。本题的优化应当围绕“扫描时若插入前已经存在则立即返回真。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。哈希表的不变量通常是当前下标之前的历史前缀信息。先查询还是先更新不是风格问题，而是决定是否允许空区间和是否重复使用当前前缀。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。正确性来自代数等价。若两个前缀满足差值、同余或计数差要求，则它们之间的区间正好满足题意；反过来任意合法区间也对应这样一对前缀。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。实现时用 `find` 判断是否存在，用 `operator[]` 只在确认要插入或累加时使用。总和可能溢出时使用 `long long`；模运算要处理负数余数归一化。本题还要特别注意：空数组、全唯一、重复在末尾、值域很小。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：空数组、全唯一、重复在末尾、值域很小。若允许修改输入，排序后看相邻可以降低额外空间。实验要覆盖从下标零开始的答案、目标为零、负数、重复前缀、余数相同但长度不足、哈希表保留第一次出现还是累加次数的差异。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

560 和为 K 的子数组

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。当前前缀和减去目标值，就是需要匹配的历史前缀。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“前缀和与哈希表”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力做法枚举所有区间，直接求区间和或重新比较频次。即使用前缀和把求和降到常数，仍可能保留枚举左右端点的平方复杂度。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。瓶颈在于当前右端需要寻找很多历史左端。若这些历史状态能被同一个键表示，就不该逐个扫描，而应把历史状态压进哈希表。放到本题里，要重点观察“哈希表保存前缀和出现次数，先查询再更新。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。前缀和把区间问题改写成两个前缀状态的差；哈希表把寻找匹配前缀改成查表。频次表、最早位置表、最近位置表对应不同目标：计数、最长区间、最短区间不能混用。本题的优化应当围绕“哈希表保存前缀和出现次数，先查询再更新。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条

件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。哈希表的不变量通常是当前下标之前的历史前缀信息。先查询还是先更新不是风格问题，而是决定是否允许空区间和是否重复使用当前前缀。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。正确性来自代数等价。若两个前缀满足差值、同余或计数差要求，则它们之间的区间正好满足题意；反过来任意合法区间也对应这样一对前缀。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。实现时用 `find` 判断是否存在，用 `operator[]` 只在确认要插入或累加时使用。总和可能溢出时使用 `long long`；模运算要处理负数余数归一化。本题还要特别注意：负数、目标为零、从下标零开始的区间、重复前缀。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：负数、目标为零、从下标零开始的区间、重复前缀。若要求最长区间，哈希表应保存最早位置而不是次数。实验要覆盖从下标零开始的答案、目标为零、负数、重复前缀、余数相同但长度不足、哈希表保留第一次出现还是累加次数的差异。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

二分查找与答案空间

下面的题卡都按同一条链路展开：从问题出发，先写暴力，再定位重复工作，然后选择数据结构或状态不变量，最后落到 C++ 实现、边界和扩展。格式统一是为了训练面试表达，而每张卡的关键观察和失效条件都要单独复盘。

图示：二分查找与答案空间推导链

暴力基线	枚举候选、完整模拟或逐个检查，先保证覆盖全部可能答案。
瓶颈定位	慢点在于每次只排除一个候选。二分能成立，是因为一次比较可以排除一半下标，或者一次可行性检查可以排除一半答案范围。
结构选择	二分前必须先定义谓词：某个位置是否已经满足边界条件，或者某个答案是否可行。数组二分找的是边界，答案二分找的是最小可行值或最大可行值。
不变量	建议固定左闭右开区间。循环中始终维护答案在 <code>[left, right)</code> 内，左侧一定不可行或小于目标，右侧外部已经被排除。
代码落地	中点写成 <code>left + (right - left) / 2</code> 。判断平方、乘法、总量时避免溢出。用 <code>std::lower_bound</code> 能表达清楚时优先用它，但面试要能手写同等语义。

33 搜索旋转排序数组

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。旋转数组每次二分至少有一半保持有序。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“二分查找与答案空间”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力做法通常线性扫描数组或线性枚举答案。它的问题不是不会找到答案，而是没有利用有序性或可行性的单调边界。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。慢点在于每次只排除一个候选。二分能成立，是因为一次比较可以排除一半下标，或者一次

可行性检查可以排除一半答案范围。放到本题里，要重点观察“判断哪一半有序，再看目标是否落在有序半边范围内。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。二分前必须先定义谓词：某个位置是否已经满足边界条件，或者某个答案是否可行。数组二分找的是边界，答案二分找的是最小可行值或最大可行值。本题的优化应当围绕“判断哪一半有序，再看目标是否落在有序半边范围内。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。建议固定左闭右开区间。循环中始终维护答案在 $[left, right)$ 内，左侧一定不可行或小于目标，右侧外部已经被排除。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。证明二分不是证明循环会停，而是证明被排除的一半确实不含答案。答案空间二分还要证明可行性谓词单调。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。中点写成 $left + (right - left) / 2$ 。判断平方、乘法、总量时避免溢出。用 `std::lower_bound` 能表达清楚时优先用它，但面试要能手写同等语义。本题还要特别注意：未旋转、长度一、目标不存在、边界等号、存在重复元素时退化。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：未旋转、长度一、目标不存在、边界等号、存在重复元素时退化。若重复元素很多，需要在无法判断有序半边时收缩边界。实验准备长度为零、一、二的数组，目标小于全部元素、大于全部元素、重复元素边界、旋转数组、不可行答案和刚好可行答案。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

34 排序数组中查找首尾位置

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。首尾位置可以拆成两个边界：第一个大于等于和第一个大于。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“二分查找与答案空间”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力做法通常线性扫描数组或线性枚举答案。它的问题不是不会找到答案，而是没有利用有序性或可行性的单调边界。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。慢点在于每次只排除一个候选。二分能成立，是因为一次比较可以排除一半下标，或者一次可行性检查可以排除一半答案范围。放到本题里，要重点观察“不要找到一个目标后向两边线性扩展，否则重复元素很多时退化。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。二分前必须先定义谓词：某个位置是否已经满足边界条件，或者某个答案是否可行。数组二分找的是边界，答案二分找的是最小可行值或最大可行值。本题的优化应当围绕“不要找到一个目标后向两边线性扩展，否则重复元素很多时退化。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。建议固定左闭右开区间。循环中始终维护答案在 `[left, right)` 内，左侧一定不可行或小于目标，右侧外部已经被排除。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。证明二分不是证明循环会停，而是证明被排除的一半确实不含答案。答案空间二分还要证明可行性谓词单调。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。中点写成 `left + (right - left) / 2`。判断平方、乘法、总量时避免溢出。用 `std::lower_bound` 能表达清楚时优先用它，但面试要能手写同等语义。本题还要特别注意：目标不存在、目标在开头或结尾、数组为空、全是目标。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：目标不存在、目标在开头或结尾、数组为空、全是目标。这个模型能迁移到计数某值出现次数和插入区间边界。实验准备长度为零、一、二的数组，目标小于全部元素、大于全部元素、重复元素边界、旋转数组、不可行答案和刚好可行答案。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

35 搜索插入位置

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。题目等价于寻找第一个大于等于目标的位置。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“二分查找与答案空间”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力做法通常线性扫描数组或线性枚举答案。它的问题不是不会找到答案，而是没有利用有序性或可行性的单调边界。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。慢点在于每次只排除一个候选。二分能成立，是因为一次比较可以排除一半下标，或者一次可行性检查可以排除一半答案范围。放到本题里，要重点观察“统一用 `lower bound` 语义，不在相等时提前返回也能保持边界一致。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。二分前必须先定义谓词：某个位置是否已经满足边界条件，或者某个答案是否可行。数组二分找的是边界，答案二分找的是最小可行值或最大可行值。本题的优化应当围绕“统一用 `lower bound` 语义，不在相等时提前返回也能保持边界一致。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。建议固定左闭右开区间。循环中始终维护答案在 `[left, right)` 内，左侧一定不可行或小于目标，右侧外部已经被排除。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。证明二分不是证明循环会停，而是证明被排除的一半确实不含答案。答案空间二分还要证明可行性谓词单调。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。中点写成 `left + (right - left) / 2`。判断平方、乘法、总量时避免溢出。用 `std::lower_bound` 能表达清楚时优先用它，但面试要能手写同等语义。本题还要特别注意：目标小于所有元素、目标大于所有元素、重复值、空数组。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺

序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：目标小于所有元素、目标大于所有元素、重复值、空数组。手写二分时用左闭右开可直接返回 `left`。实验准备长度为零、一、二的数组，目标小于全部元素、大于全部元素、重复元素边界、旋转数组、不可行答案和刚好可行答案。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

153 寻找旋转排序数组中的最小值

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。最小值是旋转断点，右端点可帮助判断中点在哪一段。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“二分查找与答案空间”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力做法通常线性扫描数组或线性枚举答案。它的问题不是不会找到答案，而是没有利用有序性或可行性的单调边界。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。慢点在于每次只排除一个候选。二分能成立，是因为一次比较可以排除一半下标，或者一次可行性检查可以排除一半答案范围。放到本题里，要重点观察“若 `mid` 大于 `right`，最小值在右侧；否则在左侧含 `mid`。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。二分前必须先定义谓词：某个位置是否已经满足边界条件，或者某个答案是否可行。数组二分找的是边界，答案二分找的是最小可行值或最大可行值。本题的优化应当围绕“若 `mid` 大于 `right`，最小值在右侧；否则在左侧含 `mid`。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。建议固定左闭右开区间。循环中始终维护答案在 `[left, right)` 内，左侧一定不可行或小于目标，右侧外部已经被排除。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。证明二分不是证明循环会停，而是证明被排除的一半确实不含答案。答案空间二分还要证明可行性谓词单调。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。中点写成 `left + (right - left) / 2`。判断平方、乘法、总量时避免溢出。用 `std::lower_bound` 能表达清楚时优先用它，但面试要能手写同等语义。本题还要特别注意：未旋转、长度一、旋转一位、无重复条件。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：未旋转、长度一、旋转一位、无重复条件。有重复时遇到相等只能收缩 `right`，最坏退化。实验准备长度为零、一、二的数组，目标小于全部元素、大于全部元素、重复元素边界、旋转数组、不可行答案和刚好可行答案。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

154 寻找旋转排序数组中的最小值 II

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。重复元素会让 `mid` 和 `right` 相等时无法判断最小值方向。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“二分查找与答案空间”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力做法通常线性扫描数组或线性枚举答案。它的问题不是不会找到答案，而是没有利用有序性或可行性的单调边界。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。慢点在于每次只排除一个候选。二分能成立，是因为一次比较可以排除一半下标，或者一次可行性检查可以排除一半答案范围。放到本题里，要重点观察“相等时只能安全地丢掉一个 `right`，因为它不是唯一必要候选。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。二分前必须先定义谓词：某个位置是否已经满足边界条件，或者某个答案是否可行。数组二分找的是边界，答案二分找的是最小可行值或最大可行值。本题的优化应当围绕“相等时只能安全地丢掉一个 `right`，因为它不是唯一必要候选。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。建议固定左闭右开区间。循环中始终维护答案在 `[left, right)` 内，左侧一定不可行或小于目标，右侧外部已经被排除。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。证明二分不是证明循环会停，而是证明被排除的一半确实不含答案。答案空间二分还要证明可行性谓词单调。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。中点写成 `left + (right - left) / 2`。判断平方、乘法、总量时避免溢出。用 `std::lower_bound` 能表达清楚时优先用它，但面试要能手写同等语义。本题还要特别注意：全相等、重复值跨断点、最小值出现多次。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 `n`。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：全相等、重复值跨断点、最小值出现多次。这题说明二分依赖的信息不足时会退化，不是所有旋转题都严格对数。实验准备长度为零、一、二的数组，目标小于全部元素、大于全部元素、重复元素边界、旋转数组、不可行答案和刚好可行答案。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

162 寻找峰值

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。比较 `mid` 和 `mid` 加一的斜率能判断某侧一定存在峰值。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“二分查找与答案空间”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力做法通常线性扫描数组或线性枚举答案。它的问题不是不会找到答案，而是没有利用有序性或可行性的单调边界。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。慢点在于每次只排除一个候选。二分能成立，是因为一次比较可以排除一半下标，或者一次可行性检查可以排除一半答案范围。放到本题里，要重点观察“上升则右侧有峰，下降则左侧含 mid 有峰。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。二分前必须先定义谓词：某个位置是否已经满足边界条件，或者某个答案是否可行。数组二分找的是边界，答案二分找的是最小可行值或最大可行值。本题的优化应当围绕“上升则右侧有峰，下降则左侧含 mid 有峰。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。建议固定左闭右开区间。循环中始终维护答案在 $[left, right)$ 内，左侧一定不可行或小于目标，右侧外部已经被排除。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。证明二分不是证明循环会停，而是证明被排除的一半确实不含答案。答案空间二分还要证明可行性谓词单调。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。中点写成 $left + (right - left) / 2$ 。判断平方、乘法、总量时避免溢出。用 `std::lower_bound` 能表达清楚时优先用它，但面试要能手写同等语义。本题还要特别注意：长度一、峰在边界、多个峰、相邻不相等假设。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：长度一、峰在边界、多个峰、相邻不相等假设。二维峰值需要按行列最大值进行更复杂二分。实验准备长度为零、一、二的数组，目标小于全部元素、大于全部元素、重复元素边界、旋转数组、不可行答案和刚好可行答案。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

240 搜索二维矩阵 II

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。右上角同时具有向左变小、向下变大的排除能力。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“二分查找与答案空间”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力做法通常线性扫描数组或线性枚举答案。它的问题不是不会找到答案，而是没有利用有序性或可行性的单调边界。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。慢点在于每次只排除一个候选。二分能成立，是因为一次比较可以排除一半下标，或者一次可行性检查可以排除一半答案范围。放到本题里，要重点观察“当前值大于目标就左移，小于目标就下移。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。二分前必须先定义谓词：某个位置是否已经满足边界条件，或者某个答案是否可行。数组二分找的是边界，答案二分找的是最小可行值或最大可行值。本题的优化应当围绕“当前值大于目标就左移，小于目标就下移。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例

如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。建议固定左闭右开区间。循环中始终维护答案在 $[left, right)$ 内，左侧一定不可行或小于目标，右侧外部已经被排除。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。证明二分不是证明循环会停，而是证明被排除的一半确实不含答案。答案空间二分还要证明可行性谓词单调。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。中点写成 $left + (right - left) / 2$ 。判断平方、乘法、总量时避免溢出。用 `std::lower_bound` 能表达清楚时优先用它，但面试要能手写同等语义。本题还要特别注意：空矩阵、单行、单列、目标在角落、重复值。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：空矩阵、单行、单列、目标在角落、重复值。从左上角无法一次排除一行或一列。实验准备长度为零、一、二的数组，目标小于全部元素、大于全部元素、重复元素边界、旋转数组、不可行答案和刚好可行答案。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

704 二分查找

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。基础题的重点是固定区间语义。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“二分查找与答案空间”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力做法通常线性扫描数组或线性枚举答案。它的问题不是不会找到答案，而是没有利用有序性或可行性的单调边界。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。慢点在于每次只排除一个候选。二分能成立，是因为一次比较可以排除一半下标，或者一次可行性检查可以排除一半答案范围。放到本题里，要重点观察“左闭右开写法能统一 `lower bound` 和存在性检查。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。二分前必须先定义谓词：某个位置是否已经满足边界条件，或者某个答案是否可行。数组二分找的是边界，答案二分找的是最小可行值或最大可行值。本题的优化应当围绕“左闭右开写法能统一 `lower bound` 和存在性检查。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。建议固定左闭右开区间。循环中始终维护答案在 $[left, right)$ 内，左侧一定不可行或小于目标，右侧外部已经被排除。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。证明二分不是证明循环会停，而是证明被排除的一半确实不含答案。答案空间二分还要证明可行性谓词单调。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。中点写成 $left + (right - left) / 2$ 。判断平方、乘法、总量时避免溢出。用 `std::lower_bound` 能表达清楚时优先用它，但面试要能手写同等语义。本题还要特别注意：空数

组、长度一、目标在边界、目标不存在。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：空数组、长度一、目标在边界、目标不存在。所有复杂二分都应该能退回到这道题的边界不变量。实验准备长度为零、一、二的数组，目标小于全部元素、大于全部元素、重复元素边界、旋转数组、不可行答案和刚好可行答案。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

875 爱吃香蕉的珂珂

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。速度越大，总耗时越小，答案具有单调可行性。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“二分查找与答案空间”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力做法通常线性扫描数组或线性枚举答案。它的问题不是不会找到答案，而是没有利用有序性或可行性的单调边界。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。慢点在于每次只排除一个候选。二分能成立，是因为一次比较可以排除一半下标，或者一次可行性检查可以排除一半答案范围。放到本题里，要重点观察“二分速度，检查函数用向上取整累加小时数。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。二分前必须先定义谓词：某个位置是否已经满足边界条件，或者某个答案是否可行。数组二分找的是边界，答案二分找的是最小可行值或最大可行值。本题的优化应当围绕“二分速度，检查函数用向上取整累加小时数。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。建议固定左闭右开区间。循环中始终维护答案在 $[left, right)$ 内，左侧一定不可行或小于目标，右侧外部已经被排除。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。证明二分不是证明循环会停，而是证明被排除的一半确实不含答案。答案空间二分还要证明可行性谓词单调。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。中点写成 $left + (right - left) / 2$ 。判断平方、乘法、总量时避免溢出。用 `std::lower_bound` 能表达清楚时优先用它，但面试要能手写同等语义。本题还要特别注意：速度下界、堆很大、 h 等于堆数、乘法加法溢出。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：速度下界、堆很大、 h 等于堆数、乘法加法溢出。船运包裹和分割数组都是同类最小可行上限。实验准备长度为零、一、二的数组，目标小于全部元素、大于全部元素、重复元素边界、旋转数组、不可行答案和刚好可行答案。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

1011 在 D 天内送达包裹

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。容量越大，需要天数越少，存在最小可行容量。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“二分查找与答案空间”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力做法通常线性扫描数组或线性枚举答案。它的问题不是不会找到答案，而是没有利用有序性或可行性的单调边界。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。慢点在于每次只排除一个候选。二分能成立，是因为一次比较可以排除一半下标，或者一次可行性检查可以排除一半答案范围。放到本题里，要重点观察“下界是最大包裹，上界是总重量，检查函数按顺序贪心装船。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。二分前必须先定义谓词：某个位置是否已经满足边界条件，或者某个答案是否可行。数组二分找的是边界，答案二分找的是最小可行值或最大可行值。本题的优化应当围绕“下界是最大包裹，上界是总重量，检查函数按顺序贪心装船。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。建议固定左闭右开区间。循环中始终维护答案在 $[left, right)$ 内，左侧一定不可行或小于目标，右侧外部已经被排除。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。证明二分不是证明循环会停，而是证明被排除的一半确实不含答案。答案空间二分还要证明可行性谓词单调。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。中点写成 $left + (right - left) / 2$ 。判断平方、乘法、总量时避免溢出。用 `std::lower_bound` 能表达清楚时优先用它，但面试要能手写同等语义。本题还要特别注意：D 为一、D 等于包裹数、单个包裹超过猜测容量。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：D 为一、D 等于包裹数、单个包裹超过猜测容量。它和分割数组最大值都是连续分段最大和模型。实验准备长度为零、一、二的数组，目标小于全部元素、大于全部元素、重复元素边界、旋转数组、不可行答案和刚好可行答案。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

栈、单调栈与表达式

下面的题卡都按同一条链路展开：从问题出发，先写暴力，再定位重复工作，然后选择数据结构或状态不变量，最后落到 C++ 实现、边界和扩展。格式统一是为了训练面试表达，而每张卡的关键观察和失效条件都要单独复盘。

图示：栈、单调栈与表达式推导链

暴力基线	枚举候选、完整模拟或逐个检查，先保证覆盖全部可能答案。
瓶颈定位	瓶颈是未解决元素没有被组织起来。单调栈保存的是仍在等待某个未来元素解决的候选；普通栈保存的是最近打开但尚未关闭的结构。
结构选择	当新元素到来时，它会一次性解决栈顶一串被它支配的旧元素。被弹出的元素以后再也不用参与比较，因此总体线性。
不变量	栈内下标对应的值要保持单调，或者栈顶必须是最近未匹配对象。弹出时要说清楚当前元素充当右边界、下一个栈顶充当左边界，还是当前运算符触发计算。
代码落地	栈里通常存下标而不是值，因为要计算距离、宽度和过期。可以用 <code>std::vector<int></code> 当栈，便于查看顶部和减少适配器限制。

42 接雨水

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。每个位置的水由左侧最高和右侧最高的较小者决定。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“栈、单调栈与表达式”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力做法对每个位置向左或向右寻找第一个满足条件的元素，或者反复删除已经匹配的局部结构。这会让同一个元素被比较很多次。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。瓶颈是未解决元素没有被组织起来。单调栈保存的是仍在等待某个未来元素解决的候选；普通栈保存的是最近打开但尚未关闭的结构。放到本题里，要重点观察“前后缀、双指针、单调栈都是不同状态维护方式；要能说明各自结算对象。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。当新元素到来时，它会一次性解决栈顶一串被它支配的旧元素。被弹出的元素以后再也不用参与比较，因此总体线性。本题的优化应当围绕“前后缀、双指针、单调栈都是不同状态维护方式；要能说明各自结算对象。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。栈内下标对应的值要保持单调，或者栈顶必须是最近未匹配对象。弹出时要说清楚当前元素充当右边界、下一个栈顶充当左边界，还是当前运算符触发计算。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。每个元素最多入栈一次、出栈一次，所以时间线性。正确性来自弹出时机：一旦遇到破坏单调性的元素，栈顶元素的答案已经确定。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。栈里通常存下标而不是值，因为要计算距离、宽度和过期。可以用 `std::vector<int>` 当栈，便于查看顶部和减少适配器限制。本题还要特别注意：单调递增、单调递减、平台高度、多个凹槽、数组长度不足三。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点

是：单调递增、单调递减、平台高度、多个凹槽、数组长度不足三。若柱子变成二维网格，需要最小堆从边界向内扩展。实验应包含严格递增、严格递减、相等元素、哨兵边界、空栈弹出、减法除法操作数顺序等情况。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

84 柱状图中最大的矩形

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。每根柱子作为最矮高度时，需要知道左右第一个更矮位置。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“栈、单调栈与表达式”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力做法对每个位置向左或向右寻找第一个满足条件的元素，或者反复删除已经匹配的局部结构。这会让同一个元素被比较很多次。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。瓶颈是未解决元素没有被组织起来。单调栈保存的是仍在等待某个未来元素解决的候选；普通栈保存的是最近打开但尚未关闭的结构。放到本题里，要重点观察“单调递增栈在遇到更矮柱时结算被弹出柱子的最大宽度。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。当新元素到来时，它会一次性解决栈顶一串被它支配的旧元素。被弹出的元素以后再也不会参与比较，因此总体线性。本题的优化应当围绕“单调递增栈在遇到更矮柱时结算被弹出柱子的最大宽度。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。栈内下标对应的值要保持单调，或者栈顶必须是最近未匹配对象。弹出时要说清楚当前元素充当右边界、下一个栈顶充当左边界，还是当前运算符触发计算。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。每个元素最多入栈一次、出栈一次，所以时间线性。正确性来自弹出时机：一旦遇到破坏单调性的元素，栈顶元素的答案已经确定。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。栈里通常存下标而不是值，因为要计算距离、宽度和过期。可以用 `std::vector<int>` 当栈，便于查看顶部和减少适配器限制。本题还要特别注意：递增、递减、相等高度、哨兵零、宽度计算。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：递增、递减、相等高度、哨兵零、宽度计算。最大矩形可以作为二维矩阵最大矩形的行压缩子问题。实验应包含严格递增、严格递减、相等元素、哨兵边界、空栈弹出、减法除法操作数顺序等情况。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

150 逆波兰表达式求值

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。后缀表达式把优先级和括号都编码进 token 顺序。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“栈、单调栈与表达式”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力做法对每个位置向左或向右寻找第一个满足条件的元素，或者

反复删除已经匹配的局部结构。这会让同一个元素被比较很多次。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。瓶颈是未解决元素没有被组织起来。单调栈保存的是仍在等待某个未来元素解决的候选；普通栈保存的是最近打开但尚未关闭的结构。放到本题里，要重点观察“遇到数字入栈，遇到运算符弹出右操作数和左操作数计算。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。当新元素到来时，它会一次性解决栈顶一串被它支配的旧元素。被弹出的元素以后再也不用参与比较，因此总体线性。本题的优化应当围绕“遇到数字入栈，遇到运算符弹出右操作数和左操作数计算。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。栈内下标对应的值要保持单调，或者栈顶必须是最近未匹配对象。弹出时要说清楚当前元素充当右边界、下一个栈顶充当左边界，还是当前运算符触发计算。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。每个元素最多入栈一次、出栈一次，所以时间线性。正确性来自弹出时机：一旦遇到破坏单调性的元素，栈顶元素的答案已经确定。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。栈里通常存下标而不是值，因为要计算距离、宽度和过期。可以用 `std::vector<int>` 当栈，便于查看顶部和减少适配器限制。本题还要特别注意：负数 token、多位数、除法向零截断、减法顺序。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：负数 token、多位数、除法向零截断、减法顺序。中缀表达式求值需要另一个运算符栈处理优先级。实验应包含严格递增、严格递减、相等元素、哨兵边界、空栈弹出、减法除法操作数顺序等情况。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

155 最小栈

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。普通栈只能看到栈顶，不能常数时间知道历史最小值。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“栈、单调栈与表达式”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力做法对每个位置向左或向右寻找第一个满足条件的元素，或者反复删除已经匹配的局部结构。这会让同一个元素被比较很多次。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。瓶颈是未解决元素没有被组织起来。单调栈保存的是仍在等待某个未来元素解决的候选；普通栈保存的是最近打开但尚未关闭的结构。放到本题里，要重点观察“辅助栈同步保存每个深度对应的最小值。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。当新元素到来时，它会一次性解决栈顶一串被它支配的旧元素。被弹出的元素以后再也不用参与比较，因此总体线性。本题的优化应当围绕“辅助栈同步保存每个深度对应的最小值。”建立一个可维护状态：新增输入只更新一次，

过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。栈内下标对应的值要保持单调，或者栈顶必须是最近未匹配对象。弹出时要说清楚当前元素充当右边界、下一个栈顶充当左边界，还是当前运算符触发计算。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。每个元素最多入栈一次、出栈一次，所以时间线性。正确性来自弹出时机：一旦遇到破坏单调性的元素，栈顶元素的答案已经确定。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。栈里通常存下标而不是值，因为要计算距离、宽度和过期。可以用 `std::vector<int>` 当栈，便于查看顶部和减少适配器限制。本题还要特别注意：重复最小值、弹出最小值、空栈操作约定。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：重复最小值、弹出最小值、空栈操作约定。也可保存差值压缩空间，但可读性和溢出处理更复杂。实验应包含严格递增、严格递减、相等元素、哨兵边界、空栈弹出、减法除法操作数顺序等情况。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

232 用栈实现队列

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。两个栈分别负责输入顺序和输出顺序。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“栈、单调栈与表达式”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力做法对每个位置向左或向右寻找第一个满足条件的元素，或者反复删除已经匹配的局部结构。这会让同一个元素被比较很多次。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。瓶颈是未解决元素没有被组织起来。单调栈保存的是仍在等待某个未来元素解决的候选；普通栈保存的是最近打开但尚未关闭的结构。放到本题里，要重点观察“只有输出栈为空时，才把输入栈整体倒过去，保证均摊常数。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。当新元素到来时，它会一次性解决栈顶一串被它支配的旧元素。被弹出的元素以后再也不会参与比较，因此总体线性。本题的优化应当围绕“只有输出栈为空时，才把输入栈整体倒过去，保证均摊常数。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。栈内下标对应的值要保持单调，或者栈顶必须是最近未匹配对象。弹出时要说清楚当前元素充当右边界、下一个栈顶充当左边界，还是当前运算符触发计算。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。每个元素最多入栈一次、出栈一次，所以时间线性。正确性来自弹出时机：一旦遇到破坏单调性的元素，栈顶元素的答案已经确定。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。栈里通常存下标而不是值，因为要计算距离、宽度和过期。可以用 `std::vector<int>` 当栈，便于查看顶部和减少适配器限制。本题还要特别注意：连续 `peek`、

空队列、交替 push pop。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：连续 peek、空队列、交替 push pop。用队列实现栈则要通过轮转保持最近元素在队首。实验应包含严格递增、严格递减、相等元素、哨兵边界、空栈弹出、减法除法操作数顺序等情况。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

239 滑动窗口最大值

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。每个窗口最大值来自仍未过期且没有被新元素支配的候选。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“栈、单调栈与表达式”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力做法对每个位置向左或向右寻找第一个满足条件的元素，或者反复删除已经匹配的局部结构。这会让同一个元素被比较很多次。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。瓶颈是未解决元素没有被组织起来。单调栈保存的是仍在等待某个未来元素解决的候选；普通栈保存的是最近打开但尚未关闭的结构。放到本题里，要重点观察“单调队列保存下标，队尾小于等于新值的候选被弹出。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。当新元素到来时，它会一次性解决栈顶一串被它支配的旧元素。被弹出的元素以后再也不会参与比较，因此总体线性。本题的优化应当围绕“单调队列保存下标，队尾小于等于新值的候选被弹出。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。栈内下标对应的值要保持单调，或者栈顶必须是最近未匹配对象。弹出时要说清楚当前元素充当右边界、下一个栈顶充当左边界，还是当前运算符触发计算。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。每个元素最多入栈一次、出栈一次，所以时间线性。正确性来自弹出时机：一旦遇到破坏单调性的元素，栈顶元素的答案已经确定。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。栈里通常存下标而不是值，因为要计算距离、宽度和过期。可以用 `std::vector<int>` 当栈，便于查看顶部和减少适配器限制。本题还要特别注意： k 为一、 k 等于数组长度、重复最大值、队首过期。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是： k 为一、 k 等于数组长度、重复最大值、队首过期。受限子序列和使用同样队列维护最近 k 个 DP 最大值。实验应包含严格递增、严格递减、相等元素、哨兵边界、空栈弹出、减法除法操作数顺序等情况。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

394 字符串解码

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。嵌套结构需要保存上一层字符串和重复次数。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“栈、单调栈与表达式”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力做法对每个位置向左或向右寻找第一个满足条件的元素，或者反复删除已经匹配的局部结构。这会让同一个元素被比较很多次。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。瓶颈是未解决元素没有被组织起来。单调栈保存的是仍在等待某个未来元素解决的候选；普通栈保存的是最近打开但尚未关闭的结构。放到本题里，要重点观察“遇到左括号入栈当前状态，右括号弹出并拼接展开。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。当新元素到来时，它会一次性解决栈顶一串被它支配的旧元素。被弹出的元素以后再也不会参与比较，因此总体线性。本题的优化应当围绕“遇到左括号入栈当前状态，右括号弹出并拼接展开。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。栈内下标对应的值要保持单调，或者栈顶必须是最近未匹配对象。弹出时要说清楚当前元素充当右边界、下一个栈顶充当左边界，还是当前运算符触发计算。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。每个元素最多入栈一次、出栈一次，所以时间线性。正确性来自弹出时机：一旦遇到破坏单调性的元素，栈顶元素的答案已经确定。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。栈里通常存下标而不是值，因为要计算距离、宽度和过期。可以用 `std::vector<int>` 当栈，便于查看顶部和减少适配器限制。本题还要特别注意：多位数字、嵌套、空片段、数字后必须跟括号。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：多位数字、嵌套、空片段、数字后必须跟括号。递归下降解析更通用，但迭代栈更符合工程栈控制。实验应包含严格递增、严格递减、相等元素、哨兵边界、空栈弹出、减法除法操作数顺序等情况。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

496 下一个更大元素 I

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。第二个数组提供每个值右侧第一个更大值的全局关系。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“栈、单调栈与表达式”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力做法对每个位置向左或向右寻找第一个满足条件的元素，或者反复删除已经匹配的局部结构。这会让同一个元素被比较很多次。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新

取得。瓶颈是未解决元素没有被组织起来。单调栈保存的是仍在等待某个未来元素解决的候选；普通栈保存的是最近打开但尚未关闭的结构。放到本题里，要重点观察“单调栈预处理值到下一个更大值映射，再回答第一个数组查询。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。当新元素到来时，它会一次性解决栈顶一串被它支配的旧元素。被弹出的元素以后再也不用参与比较，因此总体线性。本题的优化应当围绕“单调栈预处理值到下一个更大值映射，再回答第一个数组查询。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。栈内下标对应的值要保持单调，或者栈顶必须是最近未匹配对象。弹出时要说清楚当前元素充当右边界、下一个栈顶充当左边界，还是当前运算符触发计算。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。每个元素最多入栈一次、出栈一次，所以时间线性。正确性来自弹出时机：一旦遇到破坏单调性的元素，栈顶元素的答案已经确定。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。栈里通常存下标而不是值，因为要计算距离、宽度和过期。可以用 `std::vector<int>` 当栈，便于查看顶部和减少适配器限制。本题还要特别注意：不存在更大值、递减数组、值唯一假设。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：不存在更大值、递减数组、值唯一假设。若值不唯一，应按下标而不是值建映射。实验应包含严格递增、严格递减、相等元素、哨兵边界、空栈弹出、减法除法操作数顺序等情况。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

503 下一个更大元素 II

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。环形数组可以通过遍历两倍下标模拟绕回。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“栈、单调栈与表达式”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力做法对每个位置向左或向右寻找第一个满足条件的元素，或者反复删除已经匹配的局部结构。这会让同一个元素被比较很多次。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。瓶颈是未解决元素没有被组织起来。单调栈保存的是仍在等待某个未来元素解决的候选；普通栈保存的是最近打开但尚未关闭的结构。放到本题里，要重点观察“第一遍入栈，第二遍只帮助未解决下标找到答案。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。当新元素到来时，它会一次性解决栈顶一串被它支配的旧元素。被弹出的元素以后再也不用参与比较，因此总体线性。本题的优化应当围绕“第一遍入栈，第二遍只帮助未解决下标找到答案。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。栈内下标对应的值要保持单调，或者栈顶必须是最近未匹配对象。弹出时要说

清楚当前元素充当右边界、下一个栈顶充当左边界，还是当前运算符触发计算。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。每个元素最多入栈一次、出栈一次，所以时间线性。正确性来自弹出时机：一旦遇到破坏单调性的元素，栈顶元素的答案已经确定。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。栈里通常存下标而不是值，因为要计算距离、宽度和过期。可以用 `std::vector<int>` 当栈，便于查看顶部和减少适配器限制。本题还要特别注意：全递减、全相等、长度一、取模下标。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：全递减、全相等、长度一、取模下标。不要在第二遍重复入栈，否则可能死循环或重复计算。实验应包含严格递增、严格递减、相等元素、哨兵边界、空栈弹出、减法除法操作数顺序等情况。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

739 每日温度

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。每一天等待右侧第一个更高温度。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“栈、单调栈与表达式”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力做法对每个位置向左或向右寻找第一个满足条件的元素，或者反复删除已经匹配的局部结构。这会让同一个元素被比较很多次。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。瓶颈是未解决元素没有被组织起来。单调栈保存的是仍在等待某个未来元素解决的候选；普通栈保存的是最近打开但尚未关闭的结构。放到本题里，要重点观察“单调递减栈保存尚未找到答案的下标。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。当新元素到来时，它会一次性解决栈顶一串被它支配的旧元素。被弹出的元素以后再也不用参与比较，因此总体线性。本题的优化应当围绕“单调递减栈保存尚未找到答案的下标。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。栈内下标对应的值要保持单调，或者栈顶必须是最近未匹配对象。弹出时要说清楚当前元素充当右边界、下一个栈顶充当左边界，还是当前运算符触发计算。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。每个元素最多入栈一次、出栈一次，所以时间线性。正确性来自弹出时机：一旦遇到破坏单调性的元素，栈顶元素的答案已经确定。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。栈里通常存下标而不是值，因为要计算距离、宽度和过期。可以用 `std::vector<int>` 当栈，便于查看顶部和减少适配器限制。本题还要特别注意：没有更高温、相等温度、递增、递减。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域

大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：没有更高温、相等温度、递增、递减。下一个更大元素与它同型，只是输出值或距离不同。实验应包含严格递增、严格递减、相等元素、哨兵边界、空栈弹出、减法除法操作数顺序等情况。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

位运算与状态压缩

下面的题卡都按同一条链路展开：从问题出发，先写暴力，再定位重复工作，然后选择数据结构或状态不变量，最后落到 C++ 实现、边界和扩展。格式统一是为了训练面试表达，而每张卡的关键观察和失效条件都要单独复盘。

图示：位运算与状态压缩推导链

暴力基线	枚举候选、完整模拟或逐个检查，先保证覆盖全部可能答案。
瓶颈定位	慢点在于状态复制和集合比较。位操作让包含、加入、删除、枚举子集变成常数或接近常数的机器指令。
结构选择	异或解决成对抵消，按位计数处理出现三次，掩码表示访问集合或可用集合，低位提取用于枚举候选。
不变量	掩码的每一位必须有固定含义。状态去重时，位置相同但掩码不同不能合并，因为未来能力不同。
代码落地	使用无符号整数能减少移位歧义。位数超过 31 时用 <code>long long</code> 或 <code>std::bitset</code> 。表达式加括号，避免位运算优先级误读。

136 只出现一次的数字

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。成对元素可以用异或抵消，剩下单个元素。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“位运算与状态压缩”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力做法用集合、数组或递归保存状态，空间和比较成本较高。若状态本质是若干布尔选择，位掩码能把它压进整数。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。慢点在于状态复制和集合比较。位操作让包含、加入、删除、枚举子集变成常数或接近常数的机器指令。放到本题里，要重点观察“异或满足交换结合，扫描顺序不影响答案。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。异或解决成对抵消，按位计数处理出现三次，掩码表示访问集合或可用集合，低位提取用于枚举候选。本题的优化应当围绕“异或满足交换结合，扫描顺序不影响答案。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。掩码的每一位必须有固定含义。状态去重时，位置相同但掩码不同不能合并，因为未来能力不同。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。正确性来自二进制位独立性。异或按位满足交换结合和自反抵消；掩码 DP 则是对子集大小做归纳。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。使用无符号整数能减少移位歧义。位数超过 31 时用 `long long` 或 `std::bitset`。表达式加括号，避免位运算优先级误读。本题还要特别注意：零、负数、唯一值在开头或结尾、所有

其他值恰好两次。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：零、负数、唯一值在开头或结尾、所有其他值恰好两次。若其他值出现三次，要改为按位计数取模。实验包含零、负数、最高位、重复数、全集掩码、空集合、只差一位的状态和资源耗尽后的不可达状态。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

287 寻找重复数

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。数组值域使下标到值形成函数图，重复数是环入口。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“位运算与状态压缩”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力做法用集合、数组或递归保存状态，空间和比较成本较高。若状态本质是若干布尔选择，位掩码能把它压进整数。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。慢点在于状态复制和集合比较。位操作让包含、加入、删除、枚举子集变成常数或接近常数的机器指令。放到本题里，要重点观察“快慢指针不修改数组且常数空间；值域二分用抽屉原理。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。异或解决成对抵消，按位计数处理出现三次，掩码表示访问集合或可用集合，低位提取用于枚举候选。本题的优化应当围绕“快慢指针不修改数组且常数空间；值域二分用抽屉原理。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。掩码的每一位必须有固定含义。状态去重时，位置相同但掩码不同不能合并，因为未来能力不同。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。正确性来自二进制位独立性。异或按位满足交换结合和自反抵消；掩码 DP 则是对子集大小做归纳。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。使用无符号整数能减少移位歧义。位数超过 31 时用 `long long` 或 `std::bitset`。表达式加括号，避免位运算优先级误读。本题还要特别注意：重复数出现多次、不能排序、不能改数组、长度为 n 加一。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：重复数出现多次、不能排序、不能改数组、长度为 n 加一。快慢指针要求值都能作为合法下标。实验包含零、负数、最高位、重复数、全集掩码、空集合、只差一位的状态和资源耗尽后的不可达状态。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

第 6 章

二分查找和答案空间

二分查找的本质不是在数组里找数，而是在单调结构中找边界。只要能定义判断函数 `check(x)`，并且它在答案空间上单调，就可以二分。

基础二分在有序数组中查找目标值。写法看似简单，错误通常来自边界没有排除 `mid`、返回值不清楚或 `left + right` 溢出。

Listing 6.1 基础二分查找

```
1 int binary_search_index(const std::vector<int>& nums, int target)
2 {
3     int left = 0;
4     int right = static_cast<int>(nums.size()) - 1;
5     while (left <= right) {
6         const int mid = left + (right - left) / 2;
7         if (nums[mid] == target) {
8             return mid;
9         }
10        if (nums[mid] < target) {
11            left = mid + 1;
12        } else {
13            right = mid - 1;
14        }
15    }
16    return -1;
17 }
```

找左边界时，标准库 `lower_bound` 是最好的参照：它找第一个不小于目标的位置。很多题不是找一个等于目标的位置，而是找第一个满足条件的位置。

搜索插入位置就是左边界模型。题目要求如果目标存在，返回下标；如果不存在，返回应该插入的位置。这个位置其实就是第一个 `nums[i] >= target` 的下标。如果所有数都小于目标，答案就是 `nums.size()`。暴力方法从左到右扫描，时间 $O(n)$ ；有序数组给了单调性，所以可以二分。

Listing 6.2 搜索插入位置：lower_bound 写法

```
1 int search_insert_position(const std::vector<int>& nums, int target)
2 {
3     int left = 0;
4     int right = static_cast<int>(nums.size());
5
6     while (left < right) {
7         const int mid = left + (right - left) / 2;
8         if (nums[mid] < target) {
9             left = mid + 1;
10        } else {
11            right = mid;
12        }
13    }
14 }
```



```

15     return left;
16 }

```

这里使用左闭右开区间 `[left, right)`。循环不变量是答案一定在这个区间里；当 `nums[mid] < target`，`mid` 以及左侧都不可能是第一个不小于目标的位置，所以令 `left = mid + 1`；否则 `mid` 可能就是答案，不能排除，只能令 `right = mid`。这个“保留 `mid`”是左边界二分的核心。

答案二分把“位置”换成“答案值”。爱吃香蕉的珂珂中，答案是速度。速度越大越容易在规定时间内吃完，所以 `check(speed)` 从 `false` 变成 `true`。我们要找第一个 `true`。分割数组最大值、运输包裹能力、制作花束最少天数也都是这个模型。

Listing 6.3 爱吃香蕉的珂珂：答案二分

```

1 int min_eating_speed(const std::vector<int>& piles, int hours)
2 {
3     int left = 1;
4     int right = *std::max_element(piles.begin(), piles.end());
5
6     auto can_finish = [&](int speed) {
7         long long used_hours = 0;
8         for (const int pile : piles) {
9             used_hours += (pile + speed - 1LL) / speed;
10        }
11        return used_hours <= hours;
12    };
13
14    while (left < right) {
15        const int mid = left + (right - left) / 2;
16        if (can_finish(mid)) {
17            right = mid;
18        } else {
19            left = mid + 1;
20        }
21    }
22
23    return left;
24 }

```

暴力方法可以从速度 1 一直试到最大堆大小，每次计算总小时数，复杂度 $O(mn)$ ，其中 m 是最大速度范围。二分优化来自单调性：速度越大，吃完所需小时数越少；如果某个速度可以完成，那么更大的速度也一定可以完成。我们要找最小可行速度，所以是“第一个 `true`”。计算向上取整时用 `(pile + speed - 1LL) / speed`，其中 `1LL` 避免整数溢出。

搜索旋转排序数组是另一类二分：单调性不是全局存在，而是每一步至少有一半区间有序。暴力扫描是 $O(n)$ 。二分时先判断 `[left, mid]` 是否有序；如果有序，再判断目标是否落在这一半里，落在里面就收缩到左半，否则去右半。若左半无序，右半一定有序，用同样逻辑判断。这个题不要死背条件，应该画出旋转数组的两段有序结构。

Listing 6.4 搜索旋转排序数组

```

1 int search_rotated_array(const std::vector<int>& nums, int target)
2 {
3     int left = 0;
4     int right = static_cast<int>(nums.size()) - 1;
5
6     while (left <= right) {

```

```

7     const int mid = left + (right - left) / 2;
8     if (nums[mid] == target) {
9         return mid;
10    }
11
12    if (nums[left] <= nums[mid]) {
13        if (nums[left] <= target && target < nums[mid]) {
14            right = mid - 1;
15        } else {
16            left = mid + 1;
17        }
18    } else {
19        if (nums[mid] < target && target <= nums[right]) {
20            left = mid + 1;
21        } else {
22            right = mid - 1;
23        }
24    }
25 }
26
27 return -1;
28 }

```

这题默认数组没有重复值。如果允许重复，例如 `[1, 0, 1, 1, 1]`，`nums[left] == nums[mid]` 时无法判断哪边有序，可能需要移动边界退化处理，最坏复杂度会变成 $O(n)$ 。面试时主动说明这个前提，会显得你不是机械套模板。

二分最重要的习惯是固定一种区间语义。左闭右闭 `[left, right]` 和左闭右开 `[left, right)` 都可以，但不能混用。本书在找边界时优先使用左闭右开：`left` 是可能答案，`right` 是不包含的上界，循环条件是 `left < right`。当 `check(mid)` 为真且 `mid` 可能是答案时，令 `right = mid`；当 `mid` 不可能是答案时，令 `left = mid + 1`。这个模板天然适合“找第一个满足条件的位置”。

查找目标的第一个和最后一个位置，可以拆成两个边界问题。第一个位置是 `lower_bound(target)`；最后一个位置可以用 `lower_bound(target + 1) - 1`，但 `target + 1` 可能溢出，也不适用于非整数类型。更通用的写法是写两个函数：第一个大于等于 `target` 的位置，以及第一个大于 `target` 的位置。若左边界越界或值不等于 `target`，说明目标不存在。

Listing 6.5 查找目标范围：两个边界

```

1 int lower_bound_index(const std::vector<int>& nums, int target)
2 {
3     int left = 0;
4     int right = static_cast<int>(nums.size());
5     while (left < right) {
6         const int mid = left + (right - left) / 2;
7         if (nums[mid] < target) {
8             left = mid + 1;
9         } else {
10            right = mid;
11        }
12    }
13    return left;
14 }
15

```

```

16 int upper_bound_index(const std::vector<int>& nums, int target)
17 {
18     int left = 0;
19     int right = static_cast<int>(nums.size());
20     while (left < right) {
21         const int mid = left + (right - left) / 2;
22         if (nums[mid] <= target) {
23             left = mid + 1;
24         } else {
25             right = mid;
26         }
27     }
28     return left;
29 }
30
31 std::vector<int> search_range(const std::vector<int>& nums, int target)
32 {
33     const int first = lower_bound_index(nums, target);
34     if (first == static_cast<int>(nums.size()) || nums[first] != target) {
35         return {-1, -1};
36     }
37     const int last = upper_bound_index(nums, target) - 1;
38     return {first, last};
39 }

```

这段代码比“找到一个 target 后向左右扩展”更稳定，因为后者在全数组都是 target 时会退化到 $O(n)$ 。边界二分始终是 $O(\log n)$ 。注意 `upper_bound_index` 的条件是 `nums[mid] <= target`，因为小于等于目标的位置都不可能是第一个大于目标的位置。

寻找峰值是“根据局部趋势排除一半”的二分。题目给出相邻元素不相等，峰值是大于左右邻居的位置，数组边界外可以视为负无穷。若 `nums[mid] < nums[mid + 1]`，说明从 `mid` 到 `mid + 1` 是上升趋势，右侧一定存在峰值；否则左侧包含 `mid` 一定存在峰值。这个证明来自连续上升最终会在边界或下降处形成峰。

Listing 6.6 寻找峰值：沿上坡方向保留峰值

```

1 int find_peak_element(const std::vector<int>& nums)
2 {
3     int left = 0;
4     int right = static_cast<int>(nums.size()) - 1;
5     while (left < right) {
6         const int mid = left + (right - left) / 2;
7         if (nums[mid] < nums[mid + 1]) {
8             left = mid + 1;
9         } else {
10             right = mid;
11         }
12     }
13     return left;
14 }

```

这里 `mid + 1` 不会越界，因为循环条件 `left < right` 保证 `mid < right`。这个细节说明二分边界和访问表达式要配套设计。若你用左闭右闭写法，也必须确保访问 `mid + 1` 安全。峰值题不要求找全局最大，只要任意峰值；如果题目要求最左峰值或最大峰值，模型就要改。

答案二分的检查函数往往比二分外壳更重要。分割数组最大值要求把数组分成 k 个非空连续子数组，使最大子数组和最小。暴力枚举切分点是组合爆炸，DP 可以做但复杂度较高。二分答案把最大段和上限设为 `limit`，检查能否在不超过 k 段内完成。贪心检查从左到右尽量往当前段放，放不下才开新段。这个贪心正确，因为提前开新段不会减少段数，只会让当前段剩余容量浪费。

Listing 6.7 分割数组最大值：二分最大段和

```
1 int split_array_largest_sum(const std::vector<int>& nums, int k)
2 {
3     int left = *std::max_element(nums.begin(), nums.end());
4     int right = std::accumulate(nums.begin(), nums.end(), 0);
5
6     auto can_split = [&](int limit) {
7         int parts = 1;
8         int current = 0;
9         for (const int x : nums) {
10             if (current + x > limit) {
11                 ++parts;
12                 current = 0;
13             }
14             current += x;
15         }
16         return parts <= k;
17     };
18
19     while (left < right) {
20         const int mid = left + (right - left) / 2;
21         if (can_split(mid)) {
22             right = mid;
23         } else {
24             left = mid + 1;
25         }
26     }
27     return left;
28 }
```

这题的答案下界是最大元素，因为任何段都必须容纳它；上界是总和，因为把整个数组作为一段一定可行。若某个 `limit` 可行，更大的 `limit` 也可行，所以可行性单调。我们要找最小可行上限，所以是第一个 true。船运包裹、画家分板、磁带分组等题都可以用同一模型。

制作花束最少天数也是答案二分。给定每朵花开花日期，要求制作 m 束花，每束需要 k 朵相邻开放的花。天数越大，可用花越多，能做出的花束数不会减少。检查函数扫描数组，连续开放花计数达到 k 就做一束并清零；遇到未开花则连续计数归零。若总花数小于 $m*k$ ，直接不可能。

Listing 6.8 制作花束：二分天数，扫描相邻开放段

```
1 int min_days_make_bouquets(const std::vector<int>& bloom_day, int m, int k)
2 {
3     if (static_cast<long long>(m) * k > static_cast<long long>(bloom_day.size())) {
4         return -1;
5     }
6
7     int left = *std::min_element(bloom_day.begin(), bloom_day.end());
8     int right = *std::max_element(bloom_day.begin(), bloom_day.end());
9 }
```



```

10     auto can_make = [&](int day) {
11         int bouquets = 0;
12         int consecutive = 0;
13         for (const int bloom : bloom_day) {
14             if (bloom <= day) {
15                 ++consecutive;
16                 if (consecutive == k) {
17                     ++bouquets;
18                     consecutive = 0;
19                 }
20             } else {
21                 consecutive = 0;
22             }
23         }
24         return bouquets >= m;
25     };
26
27     while (left < right) {
28         const int mid = left + (right - left) / 2;
29         if (can_make(mid)) {
30             right = mid;
31         } else {
32             left = mid + 1;
33         }
34     }
35     return left;
36 }

```

这题容易错在“相邻”。不能只统计开放花总数，因为一束花要求连续位置。检查函数中的 `consecutive` 就是在维护当前连续开放段长度。每做出一束后清零，是因为同一朵花不能重复使用。这个例子说明答案二分不等于检查函数随便写，检查函数本身也必须严格贴合题意。

二分还可以处理浮点答案，例如求平方根或最大平均值，但终止条件不同。整数二分可以用 `left < right`，因为答案空间离散；浮点二分通常迭代固定次数或直到区间长度小于精度。浮点比较存在误差，不要写需要精确相等的条件。刷题中如果要求保留 `1e-5`，迭代 60 次通常足够覆盖 `double` 精度。

Listing 6.9 浮点二分：固定迭代次数

```

1 double sqrt_by_binary_search(double value)
2 {
3     double left = 0.0;
4     double right = std::max(1.0, value);
5     constexpr int ITERATIONS = 80;
6
7     for (int i = 0; i < ITERATIONS; ++i) {
8         const double mid = left + (right - left) / 2.0;
9         if (mid * mid <= value) {
10             left = mid;
11         } else {
12             right = mid;
13         }
14     }
15     return left;

```

16 }

浮点二分的思想和整数二分相同：判断某个值是否偏小或偏大，然后保留包含答案的一侧。区别在于没有“mid 加一”这种离散排除，区间靠不断缩小逼近答案。若 `value` 可能非常大，`mid * mid` 也可能溢出为无穷，需要按题目范围决定是否改写判断。

二分的反例同样重要。如果 `check(x)` 不是单调的，二分会得到毫无意义的答案。比如数组中某个值是否等于 `x`，在 `x` 的数值范围上不是从 `false` 到 `true` 再保持 `true` 的单调函数；再比如某个速度下恰好用完所有时间，这个“恰好”也不单调。二分要找的是边界，而不是猜测某个离散点。

常见误区

二分常见错误有五个。第一，`mid = (left + right) / 2` 溢出，应该写 `left + (right - left) / 2`。第二，`right = mid - 1` 和 `right = mid` 混用，导致漏掉答案。第三，答案范围不包含真实答案。第四，检查函数不是单调的。第五，返回 `left` 后不验证目标是否存在，尤其是普通查找和范围查找。

二分实验：第一，把搜索插入位置用左闭右闭和左闭右开各写一版，给空数组、单元素、目标小于所有值、目标大于所有值做测试。第二，把查找范围改成找到一个目标后向两边扩展，用全重复数组比较复杂度。第三，为分割数组最大值打印不同 `limit` 下的段数，观察可行性单调。第四，故意构造一个非单调检查函数，用二分跑出错误结果，理解为什么单调性是前提。

答案二分三步

第一，写出答案范围。第二，写出 `check(x)`。第三，证明单调性，并说明要找最小可行值还是最大可行值。没有单调性，不要二分。

本章力扣主线：704 二分查找、35 搜索插入位置、34 查找第一个和最后一个位置、33 搜索旋转排序数组、162 寻找峰值、875 爱吃香蕉的珂珂、1011 在 D 天内送达包裹的能力、410 分割数组的最大值。

二分查找、边界和答案单调性

二分查找最难的不是中点公式，而是把问题改写成边界问题。搜索插入位置找的是第一个大于等于目标的位置，查找首尾区间找的是两个边界，旋转数组找的是有序半边，答案二分找的是可行和不可行的分界。只要边界语义清楚，循环条件、左右更新和返回值就会自然统一；如果边界语义模糊，就会陷入各种加一减一的补丁。

推荐固定左闭右开区间 `[left, right)`。初始 `right` 等于候选数量，循环条件是 `left` 小于 `right`，中点落在合法候选中。当谓词在 `mid` 处为假，说明答案在 `mid` 右侧，于是 `left` 等于 `mid` 加一；当谓词为真，说明 `mid` 可能就是答案，于是 `right` 等于 `mid`。循环结束时 `left` 等于 `right`，是第一个满足谓词的位置。这个模板能覆盖 `lower_bound`、答案最小可行值和许多边界题。

答案二分的关键是谓词单调。爱吃香蕉中，速度越快越容易完成；船运包裹中，容量越大需要天数越少；分割数组中，允许的最大段和越大，所需段数越少；最小体力路径中，体力阈值越大，可走边越多。这些问题的原数组未必有序，但答案空间有序。若检查函数内部使用了不正确的贪心，外层二分再漂亮也没有意义。

写检查函数时要先说明贪心为什么正确。船运包裹按顺序装包，当前天能放就放，放不下再开新天；因为包裹顺序不能改变，提前开新天只会让剩余天数更紧，不会更优。分割数组最大值也是同理：在段和不超过上限的前提下尽量延长当前段，可以最小化段数。检查函数本身常常就是一个贪心证明题。

二分的溢出问题不能忽略。中点用 `left` 加 $(right - left) / 2$ ，平方比较用 `mid <= x / mid`，求总耗时或总容量时用 `long long`。很多题的输入范围故意接近 `int` 上限，代码逻辑正确但乘法溢出会得到完全错误的判断。复杂度分析也要写成 $O(n \log M)$ ，其中 `M` 是答案范围，不要笼统写 $O(\log n)$ 。

旋转数组和峰值题展示了另一类二分：不是直接找单调谓词，而是利用局部比较保证某一侧存在答案。寻找峰值中，如果 `nums[mid]` 小于 `nums[mid + 1]`，右侧一定有峰；否则左侧含 `mid` 一定有

峰。这个证明依赖边界可视为负无穷和相邻不相等的条件。若题目条件改变，例如允许大量重复，就要重新证明是否还能排除一半。

实验建议：把同一道 `lower_bound` 用左闭右闭、左闭右开两种写法实现，再用长度从零到五的所有数组暴力枚举测试。答案二分实验中，先写一个很慢但显然正确的枚举版本，再和二分版本对比。对于每个失败用例，记录 `left`、`mid`、`right` 和谓词值，直到能指出哪条不变量被破坏。

本节复盘

阅读本节后，请回到相邻章节的例题，把暴力瓶颈、优化依据、状态不变量、C++ 容器成本和边界测试重新写一遍。能把同一思想迁移到变体题，才算真正掌握。

第 7 章

树、图、BFS 和 DFS

树和图题的第一难点是建模。题目未必直接给你图，但只要有状态和状态之间的一步转移，就可以看成图。网格题的格子是节点，相邻方向是边；课程表的课程是节点，先修关系是有向边；单词接龙的单词是节点，一次变化是边。

树是没有环的特殊图。二叉树节点通常只有左右孩子，父子关系天然给出层次；普通图没有固定根，可能有环，必须显式维护访问状态。二者的共同点是：算法不会凭空发生，所有遍历都在回答“当前节点能把什么信息交给相邻节点”。BFS 按层推进，适合无权最短步数、层序遍历和多源扩散；DFS 沿路径推进，适合连通块、路径搜索、状态检测和回溯枚举。本书会讲递归思想，但 C++ 实现优先使用显式队列、栈和循环，因为在线评测里极端深树和大图可能让递归调用栈失控。

为了让代码统一，本章二叉树示例使用下面的节点字段名。力扣原题多使用 `val`，实际提交时把 `value` 改成 `val` 即可，核心逻辑不变。

Listing 7.1 二叉树节点示意

```
1 struct TreeNode {
2     int value = 0;
3     TreeNode* left = nullptr;
4     TreeNode* right = nullptr;
5 };
```

二叉树层序遍历是 BFS 的入门题，但它不是“把队列模板背下来”这么简单。题目要求按层返回节点值，因此队列里保存的是下一批待访问节点，而外层循环开始时，队列必须恰好包含当前层的所有节点。暴力做法可以先求树高，再对每一层从根节点重新扫描一次，把深度等于目标层的节点收集出来。这个思路正确，但一棵高度为 h 的树会被重复扫描很多次，最坏退化到 $O(nh)$ 。

优化来自队列的先进先出顺序。根节点先入队，处理当前层时，把它的孩子按从左到右的顺序放到队尾。当前层节点全部弹出后，队列中剩下的正好是下一层节点。这里的关键变量是 `level_size`：它把“当前层的边界”固定下来，避免新入队的下一层节点被提前处理。

Listing 7.2 二叉树层序遍历：固定当前层边界

```
1 std::vector<std::vector<int>> level_order(TreeNode* root)
2 {
3     std::vector<std::vector<int>> levels;
4     if (root == nullptr) {
5         return levels;
6     }
7
8     std::queue<TreeNode*> queue;
9     queue.push(root);
10
11     while (!queue.empty()) {
12         const int level_size = static_cast<int>(queue.size());
13         std::vector<int> level;
14         level.reserve(level_size);
15
16         for (int i = 0; i < level_size; ++i) {
17             TreeNode* node = queue.front();
18             queue.pop();
```



```

19         level.push_back(node->value);
20
21         if (node->left != nullptr) {
22             queue.push(node->left);
23         }
24         if (node->right != nullptr) {
25             queue.push(node->right);
26         }
27     }
28
29     levels.push_back(std::move(level));
30 }
31
32 return levels;
33 }

```

这题的复杂度是 $O(n)$ ，每个节点入队一次、出队一次。空间复杂度是 $O(w)$ ， w 是树的最大宽度；如果把输出也计入空间，则是 $O(n)$ 。常见错误有三个：空树忘记返回空数组；没有记录 `level_size`，导致层边界混在一起；把右孩子放到左孩子前面，破坏题目要求的从左到右顺序。面试时要把“不变量”说出来：每次进入外层循环，队列中保存的是同一层、尚未输出的全部节点。

最大深度看起来更像递归题，数学定义也很自然：空树深度为 0，非空树深度是左右子树最大深度加 1。递归写法短，但深度很大的链式树会把调用栈压得很深。用 BFS 写法时，最大深度就是层序遍历处理过的层数；用显式栈写法时，栈里保存节点和它所在深度。两种实现都能避免把深度交给语言调用栈管理。

Listing 7.3 最大深度：显式栈保存节点深度

```

1 int max_depth(TreeNode* root)
2 {
3     if (root == nullptr) {
4         return 0;
5     }
6
7     int answer = 0;
8     std::vector<std::pair<TreeNode*, int>> stack{{root, 1}};
9
10    while (!stack.empty()) {
11        const auto [node, depth] = stack.back();
12        stack.pop_back();
13        answer = std::max(answer, depth);
14
15        if (node->left != nullptr) {
16            stack.emplace_back(node->left, depth + 1);
17        }
18        if (node->right != nullptr) {
19            stack.emplace_back(node->right, depth + 1);
20        }
21    }
22
23    return answer;
24 }

```

这段代码的状态非常明确：栈里的每个元素都是一个尚未展开的节点，以及从根到它经过的节

点数。暴力和优化的差别不在复杂度，递归和显式栈都是 $O(n)$ ；差别在工程边界。显式栈把潜在的系统调用栈风险变成可控的堆上容器，也更容易扩展成“最大深度路径”“带权深度”“同时记录父节点”等变体。易错点是深度定义：力扣 104 问节点数层数，根节点深度应是 1；如果题目问边数高度，根节点高度才可能记作 0。

验证二叉搜索树经常暴露“只看局部”的错误。很多人会写成：当前节点大于左孩子、小于右孩子，就继续检查左右子树。这个判断不够，因为 BST 的约束是全局的：左子树所有节点都必须小于当前节点，右子树所有节点都必须大于当前节点。暴力补救方法是对每个节点扫描整棵左子树最大值和右子树最小值，但会重复访问，最坏 $O(n^2)$ 。

更稳定的优化是利用 BST 的中序遍历性质。对合法 BST 做中序遍历，访问值必须严格递增。于是全局范围约束被转换成相邻访问值比较：只要当前值不大于前一个中序值，就一定非法。这里要强调“严格递增”，因为力扣 98 不允许重复值出现在左右子树。

Listing 7.4 验证二叉搜索树：显式栈中序遍历

```
1 bool is_valid_bst(TreeNode* root)
2 {
3     std::vector<TreeNode*> stack;
4     TreeNode* current = root;
5     std::optional<long long> previous;
6
7     while (current != nullptr || !stack.empty()) {
8         while (current != nullptr) {
9             stack.push_back(current);
10            current = current->left;
11        }
12
13        current = stack.back();
14        stack.pop_back();
15
16        if (previous.has_value() && current->value <= previous.value()) {
17            return false;
18        }
19        previous = current->value;
20        current = current->right;
21    }
22
23    return true;
24 }
```

这题的时间复杂度是 $O(n)$ ，空间复杂度是 $O(h)$ ， h 是树高。代码使用 `long long` 保存前一个值，是为了避免节点值接近 `int` 边界时用哨兵值出错；用 `std::optional` 比写一个“极小初值”更严谨。面试表达可以这样组织：先指出局部比较错误，再给出全局约束，最后说明中序遍历把全局约束转成单调序列检查。

最近公共祖先的题意是：给定二叉树中两个节点 `p` 和 `q`，找出最深的那个节点，使它同时是 `p` 和 `q` 的祖先。暴力想法是对每个候选节点分别判断它的子树里是否包含 `p` 和 `q`，这会产生大量重复搜索。经典递归解法会让子树向父节点汇报“是否找到了目标”，但同样存在深树调用栈问题。本书给出显式父指针版本：先遍历树，记录每个节点的父节点；再把 `p` 的所有祖先放入集合；最后从 `q` 不断向上走，第一次遇到的已访问祖先就是答案。

Listing 7.5 最近公共祖先：父指针和祖先集合

```
1 TreeNode* lowest_common_ancestor(TreeNode* root, TreeNode* p, TreeNode* q)
2 {
```

```

3   if (root == nullptr || p == nullptr || q == nullptr) {
4       return nullptr;
5   }
6
7   std::unordered_map<TreeNode*, TreeNode*> parent;
8   std::vector<TreeNode*> stack{root};
9   parent.emplace(root, nullptr);
10
11  while (!stack.empty() && (!parent.contains(p) || !parent.contains(q))) {
12      TreeNode* node = stack.back();
13      stack.pop_back();
14
15      if (node->left != nullptr) {
16          parent.emplace(node->left, node);
17          stack.push_back(node->left);
18      }
19      if (node->right != nullptr) {
20          parent.emplace(node->right, node);
21          stack.push_back(node->right);
22      }
23  }
24
25  std::unordered_set<TreeNode*> ancestors;
26  TreeNode* current = p;
27  while (current != nullptr) {
28      ancestors.insert(current);
29      current = parent[current];
30  }
31
32  current = q;
33  while (current != nullptr) {
34      if (ancestors.contains(current)) {
35          return current;
36      }
37      current = parent[current];
38  }
39
40  return nullptr;
41 }

```

为什么从 q 向上遇到的第一个公共祖先就是“最近”的？因为向上走的顺序是从深到浅，越早遇到越靠近 q ，同时它已经在 p 的祖先集合中。这个算法时间 $O(n)$ ，空间 $O(n)$ 。如果题目保证 p 和 q 一定在树中，前面的遍历可以在二者父指针都找到后提前停止；如果不保证，就需要完整遍历并检查二者是否出现。常见错误是把“节点值相同”当成“节点相同”，但力扣 236 给的是节点指针，应该比较节点地址。

岛屿数量把网格转成无向图。节点是值为 '1' 的格子，边是上下左右相邻的陆地。题目要求岛屿个数，就是要求连通块个数。暴力思路可能会对每个陆地格子都搜索一次，看它属于哪个岛，这会反复走过同一片陆地。正确优化是：扫描网格时，只要遇到一块尚未访问的陆地，就说明发现了一个新连通块；从它出发把整个连通块标记掉，答案加一。

Listing 7.6 岛屿数量：显式栈标记连通块

```

1 int num_islands(std::vector<std::vector<char>>& grid)

```

```

2 {
3     if (grid.empty() || grid[0].empty()) {
4         return 0;
5     }
6
7     constexpr char LAND = '1';
8     constexpr char WATER = '0';
9     constexpr std::array<std::pair<int, int>, 4> DIRECTIONS{
10         std::pair{1, 0}, std::pair{-1, 0},
11         std::pair{0, 1}, std::pair{0, -1}
12     };
13
14     const int rows = static_cast<int>(grid.size());
15     const int cols = static_cast<int>(grid[0].size());
16     int islands = 0;
17
18     for (int row = 0; row < rows; ++row) {
19         for (int col = 0; col < cols; ++col) {
20             if (grid[row][col] != LAND) {
21                 continue;
22             }
23
24             ++islands;
25             std::vector<std::pair<int, int>> stack{{row, col}};
26             grid[row][col] = WATER;
27
28             while (!stack.empty()) {
29                 const auto [r, c] = stack.back();
30                 stack.pop_back();
31
32                 for (const auto [dr, dc] : DIRECTIONS) {
33                     const int next_row = r + dr;
34                     const int next_col = c + dc;
35                     const bool out_of_bounds =
36                         next_row < 0 || next_row >= rows ||
37                         next_col < 0 || next_col >= cols;
38                     if (out_of_bounds || grid[next_row][next_col] != LAND) {
39                         continue;
40                     }
41
42                     grid[next_row][next_col] = WATER;
43                     stack.emplace_back(next_row, next_col);
44                 }
45             }
46         }
47     }
48
49     return islands;
50 }

```

这个实现把访问过的陆地直接改成水，因此不需要额外 **visited** 数组。如果题目不允许修改输入，就用同样大小的布尔矩阵保存访问状态。不变量是：扫描到任意位置时，所有已经发现过的岛都被完全标记为水，尚未被扫描到的陆地如果仍为 **'1'**，它一定属于一个尚未计数的新岛。时间复

杂度 $O(mn)$ ，空间复杂度最坏 $O(mn)$ ，来自显式栈。易错点是访问标记时机：应该在入栈或入队时立刻标记，而不是弹出后再标记，否则同一格子可能被多个邻居重复加入。

腐烂橘子是多源 BFS。每个新鲜橘子变烂的时间，等于它到最近初始腐烂橘子的最短四联通距离。暴力想法是每一分钟扫描整张网格，看哪些新鲜橘子旁边有腐烂橘子。这种写法虽然直观，却会反复扫描空格、已经腐烂的格子 and 暂时不会变化的格子。优化方法是把所有初始腐烂橘子同时入队，只从当前腐烂边界向外扩散。

Listing 7.7 腐烂橘子：多源 BFS 计算最短腐烂时间

```

1 int oranges_rotting(std::vector<std::vector<int>>& grid)
2 {
3     if (grid.empty() || grid[0].empty()) {
4         return 0;
5     }
6
7     constexpr int EMPTY = 0;
8     constexpr int FRESH = 1;
9     constexpr int ROTTEN = 2;
10    constexpr std::array<std::pair<int, int>, 4> DIRECTIONS{
11        std::pair{1, 0}, std::pair{-1, 0},
12        std::pair{0, 1}, std::pair{0, -1}
13    };
14
15    const int rows = static_cast<int>(grid.size());
16    const int cols = static_cast<int>(grid[0].size());
17    std::queue<std::pair<int, int>> queue;
18    int fresh_count = 0;
19
20    for (int row = 0; row < rows; ++row) {
21        for (int col = 0; col < cols; ++col) {
22            if (grid[row][col] == ROTTEN) {
23                queue.emplace(row, col);
24            } else if (grid[row][col] == FRESH) {
25                ++fresh_count;
26            }
27        }
28    }
29
30    int minutes = 0;
31    while (!queue.empty() && fresh_count > 0) {
32        const int level_size = static_cast<int>(queue.size());
33        for (int step = 0; step < level_size; ++step) {
34            const auto [row, col] = queue.front();
35            queue.pop();
36
37            for (const auto [dr, dc] : DIRECTIONS) {
38                const int next_row = row + dr;
39                const int next_col = col + dc;
40                const bool out_of_bounds =
41                    next_row < 0 || next_row >= rows ||
42                    next_col < 0 || next_col >= cols;
43                if (out_of_bounds || grid[next_row][next_col] != FRESH) {
44                    continue;
45                }

```

```

46
47         grid[next_row][next_col] = ROTTEN;
48         --fresh_count;
49         queue.emplace(next_row, next_col);
50     }
51 }
52 ++minutes;
53 }
54
55 return fresh_count == 0 ? minutes : -1;
56 }

```

这里 `level_size` 表示同一分钟内会继续扩散的腐烂橘子数量。处理完这一层后，才让 `minutes` 加一。初始没有新鲜橘子时，答案是 0；队列空了但仍有新鲜橘子时，说明有橘子被空格隔开，永远不会腐烂，答案是 -1。代码中 `EMPTY` 没有参与判断，但保留常量能让输入语义更清楚。复杂度仍是 $O(mn)$ ，因为每个格子最多入队一次。

课程表是有向图判环。每门课是节点，先修关系 `[course, prerequisite]` 表示必须先学 `prerequisite`，再学 `course`，所以边的方向应是 `prerequisite -> course`。暴力想法是反复扫描所有课程，找当前没有未完成先修课的课程；每学一门课又重新检查所有依赖，容易写成 $O(nm)$ 。拓扑排序的优化点是入度数组：`indegree[x]` 表示课程 `x` 还有多少前置依赖没有被满足。

Listing 7.8 课程表 II：拓扑排序输出可行学习顺序

```

1 std::vector<int> find_order(
2     int num_courses,
3     const std::vector<std::vector<int>>& prerequisites)
4 {
5     std::vector<std::vector<int>> graph(num_courses);
6     std::vector<int> indegree(num_courses, 0);
7
8     for (const auto& edge : prerequisites) {
9         const int course = edge[0];
10        const int prerequisite = edge[1];
11        graph[prerequisite].push_back(course);
12        ++indegree[course];
13    }
14
15    std::queue<int> queue;
16    for (int course = 0; course < num_courses; ++course) {
17        if (indegree[course] == 0) {
18            queue.push(course);
19        }
20    }
21
22    std::vector<int> order;
23    order.reserve(num_courses);
24
25    while (!queue.empty()) {
26        const int course = queue.front();
27        queue.pop();
28        order.push_back(course);
29
30        for (const int next : graph[course]) {

```

```

31         --indegree[next];
32         if (indegree[next] == 0) {
33             queue.push(next);
34         }
35     }
36 }
37
38 if (static_cast<int>(order.size()) != num_courses) {
39     return {};
40 }
41 return order;
42 }

```

这段代码同时覆盖力扣 207 和 210。207 只问能否完成所有课程，可以返回 `order.size() == num_courses`；210 要返回一个具体顺序，若有环则返回空数组。正确性的核心是：入度为 0 的节点当前没有任何未满足依赖，可以安全学习；学习它只会减少它指向的后继课程入度，不会影响其他边。每个节点最多入队一次，每条边处理一次，复杂度 $O(n + m)$ 。最常见错误是把边方向建反。边建反时，有时仍然能判断是否有环，但输出顺序会变成反向，面试中必须主动说清方向。

单词接龙是无权图最短路。节点是单词，两个单词只差一个字符就连一条边；答案是起点词到终点词的最短转换长度。直接建图的暴力方法是枚举所有单词对，逐字符比较是否只差一位，复杂度约为 $O(n^2L)$ ，其中 L 是单词长度。BFS 本身没有问题，慢在找邻居。

优化思路是用通配模式把邻居查询变成哈希查询。比如 `hot` 可以生成 `*ot`、`h*t`、`ho*` 三个模式；所有能匹配同一模式的单词，彼此之间只差一个字符。预处理时把模式映射到单词列表，BFS 时对当前单词生成模式，从桶里取候选邻居。这样避免了每一步都扫描全词表。

Listing 7.9 单词接龙：通配模式加 BFS

```

1 int ladder_length(const std::string& begin_word,
2                  const std::string& end_word,
3                  const std::vector<std::string>& word_list)
4 {
5     constexpr char WILDCARD = '*';
6     const int word_length = static_cast<int>(begin_word.size());
7
8     std::unordered_set<std::string> dictionary(
9         word_list.begin(), word_list.end());
10    if (!dictionary.contains(end_word)) {
11        return 0;
12    }
13
14    std::unordered_map<std::string, std::vector<std::string>> buckets;
15    for (const std::string& word : word_list) {
16        for (int i = 0; i < word_length; ++i) {
17            std::string pattern = word;
18            pattern[i] = WILDCARD;
19            buckets[pattern].push_back(word);
20        }
21    }
22
23    std::queue<std::pair<std::string, int>> queue;
24    std::unordered_set<std::string> visited;
25    queue.emplace(begin_word, 1);
26    visited.insert(begin_word);

```

```

27
28 while (!queue.empty()) {
29     const auto [word, distance] = queue.front();
30     queue.pop();
31     if (word == end_word) {
32         return distance;
33     }
34
35     for (int i = 0; i < word_length; ++i) {
36         std::string pattern = word;
37         pattern[i] = WILDCARD;
38         auto found = buckets.find(pattern);
39         if (found == buckets.end()) {
40             continue;
41         }
42
43         for (const std::string& next : found->second) {
44             if (visited.contains(next)) {
45                 continue;
46             }
47             visited.insert(next);
48             queue.emplace(next, distance + 1);
49         }
50         found->second.clear();
51     }
52 }
53
54 return 0;
55 }

```

这题的 BFS 不变量是：队列中每个状态的 `distance` 都是从起点到该单词的最短转换长度；第一次到达 `endWord` 时，答案已经最短。为什么可以在访问后清空模式桶？因为一个模式桶只需要被展开一次。之后再遇到同一模式，桶里的所有邻居要么已经访问，要么已经入队；重复扫描只会增加时间。预处理复杂度约为 $O(nL^2)$ ，因为每次生成模式会复制长度为 `L` 的字符串；BFS 展开在清桶优化后也是同量级。若要继续优化，可以把单词编号化，模式只保存编号，但面试中这个版本已经足够清晰。

树图题复盘模板

每道树图题都先写四句话：节点是什么，边是什么，状态里保存什么，访问标记何时设置。二叉树题再补一句返回值语义或队列不变量；图题再补一句为什么 BFS 第一次到达就是最短，或者为什么 DFS 标记完整连通块。写完代码后，用空输入、单节点、链式树、完全树、孤立节点、有环依赖、不可达终点各测一遍。能把这些边界说清楚，才算真正掌握，而不是只背了遍历模板。

tree-graph 深度题卡

二叉树与树形状态

下面的题卡都按同一条链路展开：从问题出发，先写暴力，再定位重复工作，然后选择数据结构或状态不变量，最后落到 C++ 实现、边界和扩展。格式统一是为了训练面试表达，而每张卡的关键观察和失效条件都要单独复盘。

图示：二叉树与树形状态推导链

暴力基线	枚举候选、完整模拟或逐个检查，先保证覆盖全部可能答案。
瓶颈定位	慢点在于同一棵子树被重复求高度、最大值、是否包含目标等信息。若每个节点能一次性向父节点汇报足够信息，就不应重复下钻。
结构选择	树形 DP 把每个节点看成子问题根；BFS 把层次边界显式放进队列；BST 题利用中序有序或上下界约束。工程实现优先考虑显式栈避免深树调用栈风险。
不变量	返回值或栈元素必须有语义：当前子树高度、当前路径和、当前节点合法上下界、父指针、层号。不要只说遍历。
代码落地	示例中可以用指针节点，但比较节点身份时比较指针。极端深树用 <code>std::vector<TreeNode*></code> 显式栈或队列。最大路径类题要用足够小的初值，不能默认零。

94 二叉树中序遍历

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。中序访问顺序是左子树、根、右子树，BST 中会得到有序序列。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“二叉树与树形状态”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力做法对每个节点反复扫描子树，或者只检查父子局部关系。树题真正考的是状态如何从父到子、从子到父或按层传播。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。慢点在于同一棵子树被重复求高度、最大值、是否包含目标等信息。若每个节点能一次性向父节点汇报足够信息，就不应重复下钻。放到本题里，要重点观察“用显式栈模拟一路向左，再回退访问根并转向右子树。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。树形 DP 把每个节点看成子问题根；BFS 把层次边界显式放进队列；BST 题利用中序有序或上下界约束。工程实现优先考虑显式栈避免深树调用栈风险。本题的优化应当围绕“用显式栈模拟一路向左，再回退访问根并转向右子树。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。返回值或栈元素必须有语义：当前子树高度、当前路径和、当前节点合法上下界、父指针、层号。不要只说遍历。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。正确性通常是对子树归纳。左右子树已经给出稳定答案，父节点只需按题目约束合并这两个答案。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。示例中可以用指针节点，但比较节点身份时比较指针。极端深树用 `std::vector<TreeNode*>` 显式栈或队列。最大路径类题要用足够小的初值，不能默认零。本题还要特别注意：空树、单节点、链式左树、链式右树。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重

点是：空树、单节点、链式左树、链式右树。Morris 遍历可做到常数额外空间，但会临时改树结构。实验包含空树、单节点、链式树、完全树、含重复值、全负值、目标节点不存在和极端不平衡树。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

98 验证二叉搜索树

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。BST 约束是全局上下界，不是只比较父子节点。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“二叉树与树形状态”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力做法对每个节点反复扫描子树，或者只检查父子局部关系。树题真正考的是状态如何从父到子、从子到父或按层传播。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。慢点在于同一棵树被重复求高度、最大值、是否包含目标等信息。若每个节点能一次性向父节点汇报足够信息，就不应重复下钻。放到本题里，要重点观察“中序严格递增或显式传递上下界都可以；中序版本要保存前一个访问值。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。树形 DP 把每个节点看成子问题根；BFS 把层次边界显式放进队列；BST 题利用中序有序或上下界约束。工程实现优先考虑显式栈避免深树调用栈风险。本题的优化应当围绕“中序严格递增或显式传递上下界都可以；中序版本要保存前一个访问值。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。返回值或栈元素必须有语义：当前子树高度、当前路径和、当前节点合法上下界、父指针、层号。不要只说遍历。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。正确性通常是对子树归纳。左右子树已经给出稳定答案，父节点只需按题目约束合并这两个答案。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。示例中可以用指针节点，但比较节点身份时比较指针。极端深树用 `std::vector<TreeNode*>` 显式栈或队列。最大路径类题要用足够小的初值，不能默认零。本题还要特别注意：重复值、int 最小最大值、局部合法但全局非法。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：重复值、int 最小最大值、局部合法但全局非法。若允许重复值，需要明确重复放左还是放右。实验包含空树、单节点、链式树、完全树、含重复值、全负值、目标节点不存在和极端不平衡树。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

102 二叉树层序遍历

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。队列在每轮开始时保存当前层所有节点。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“二叉树与树形状态”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再

对每个候选重新检查是否合法。暴力做法对每个节点反复扫描子树，或者只检查父子局部关系。树题真正考的是状态如何从父到子、从子到父或按层传播。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。慢点在于同一棵子树被重复求高度、最大值、是否包含目标等信息。若每个节点能一次性向父节点汇报足够信息，就不应重复下钻。放到本题里，要重点观察“记录 level size 固定层边界，避免下一层节点混入当前层。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。树形 DP 把每个节点看成子问题根；BFS 把层次边界显式放进队列；BST 题利用中序有序或上下界约束。工程实现优先考虑显式栈避免深树调用栈风险。本题的优化应当围绕“记录 level size 固定层边界，避免下一层节点混入当前层。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。返回值或栈元素必须有语义：当前子树高度、当前路径和、当前节点合法上下界、父指针、层号。不要只说遍历。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。正确性通常是对子树归纳。左右子树已经给出稳定答案，父节点只需按题目约束合并这两个答案。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。示例中可以用指针节点，但比较节点身份时比较指针。极端深树用 `std::vector<TreeNode*>` 显式栈或队列。最大路径类题要用足够小的初值，不能默认零。本题还要特别注意：空树、只有根、最后一层不满、左右顺序。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：空树、只有根、最后一层不满、左右顺序。锯齿形层序只是在每层输出顺序上增加方向控制。实验包含空树、单节点、链式树、完全树、含重复值、全负值、目标节点不存在和极端不平衡树。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

103 二叉树锯齿形层序遍历

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。BFS 层边界不变，变化的是当前层写入结果的方向。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“二叉树与树形状态”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力做法对每个节点反复扫描子树，或者只检查父子局部关系。树题真正考的是状态如何从父到子、从子到父或按层传播。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。慢点在于同一棵子树被重复求高度、最大值、是否包含目标等信息。若每个节点能一次性向父节点汇报足够信息，就不应重复下钻。放到本题里，要重点观察“可以层内反转，也可以预分配数组按方向写入目标下标。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。树形 DP 把每个节点看成子问题

根；BFS 把层次边界显式放进队列；BST 题利用中序有序或上下界约束。工程实现优先考虑显式栈避免深树调用栈风险。本题的优化应当围绕“可以层内反转，也可以预分配数组按方向写入目标下标。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。返回值或栈元素必须有语义：当前子树高度、当前路径和、当前节点合法上下界、父指针、层号。不要只说遍历。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。正确性通常是对子树归纳。左右子树已经给出稳定答案，父节点只需按题目约束合并这两个答案。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。示例中可以用指针节点，但比较节点身份时比较指针。极端深树用 `std::vector<TreeNode*>` 显式栈或队列。最大路径类题要用足够小的初值，不能默认零。本题还要特别注意：单层、偶数层、奇数层、空树。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：单层、偶数层、奇数层、空树。若要求从底向上输出，收集后反转层列表即可。实验包含空树、单节点、链式树、完全树、含重复值、全负值、目标节点不存在和极端不平衡树。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

104 二叉树最大深度

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。深度可以看成层数，也可以作为栈中携带的状态。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“二叉树与树形状态”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力做法对每个节点反复扫描子树，或者只检查父子局部关系。树题真正考的是状态如何从父到子、从子到父或按层传播。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。慢点在于同一棵子树被重复求高度、最大值、是否包含目标等信息。若每个节点能一次性向父节点汇报足够信息，就不应重复下钻。放到本题里，要重点观察“显式栈保存节点和深度，避免极端链式树递归栈过深。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。树形 DP 把每个节点看成子问题根；BFS 把层次边界显式放进队列；BST 题利用中序有序或上下界约束。工程实现优先考虑显式栈避免深树调用栈风险。本题的优化应当围绕“显式栈保存节点和深度，避免极端链式树递归栈过深。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。返回值或栈元素必须有语义：当前子树高度、当前路径和、当前节点合法上下界、父指针、层号。不要只说遍历。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。正确性通常是对子树归纳。左右子树已经给出稳定答案，父节点只需按题目约束合并这两个答案。证明不需

要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。示例中可以用指针节点，但比较节点身份时比较指针。极端深树用 `std::vector<TreeNode*>` 显式栈或队列。最大路径类题要用足够小的初值，不能默认零。本题还要特别注意：空树返回零、根深度是一、链式树。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：空树返回零、根深度是一、链式树。最小深度不能简单取左右最小，因为空子树不构成叶子路径。实验包含空树、单节点、链式树、完全树、含重复值、全负值、目标节点不存在和极端不平衡树。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

105 前序中序构造二叉树

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。前序给根，中序把左右子树切开。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“二叉树与树形状态”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力做法对每个节点反复扫描子树，或者只检查父子局部关系。树题真正考的是状态如何从父到子、从子到父或按层传播。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。慢点在于同一棵子树被重复求高度、最大值、是否包含目标等信息。若每个节点能一次性向父节点汇报足够信息，就不应重复下钻。放到本题里，要重点观察“用哈希表记录中序下标，避免每次线性寻找根位置。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。树形 DP 把每个节点看成子问题根；BFS 把层次边界显式放进队列；BST 题利用中序有序或上下界约束。工程实现优先考虑显式栈避免深树调用栈风险。本题的优化应当围绕“用哈希表记录中序下标，避免每次线性寻找根位置。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。返回值或栈元素必须有语义：当前子树高度、当前路径和、当前节点合法上下界、父指针、层号。不要只说遍历。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。正确性通常是对子树归纳。左右子树已经给出稳定答案，父节点只需按题目约束合并这两个答案。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。示例中可以用指针节点，但比较节点身份时比较指针。极端深树用 `std::vector<TreeNode*>` 显式栈或队列。最大路径类题要用足够小的初值，不能默认零。本题还要特别注意：节点值唯一、空子树、左右子树长度计算、输入不合法。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重

点是：节点值唯一、空子树、左右子树长度计算、输入不合法。后序中序构造类似，只是根来自后序末尾。实验包含空树、单节点、链式树、完全树、含重复值、全负值、目标节点不存在和极端不平衡树。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

106 中序后序构造二叉树

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。后序末尾是根，中序位置确定左右子树规模。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“二叉树与树形状态”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力做法对每个节点反复扫描子树，或者只检查父子局部关系。树题真正考的是状态如何从父到子、从子到父或按层传播。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。慢点在于同一棵树被重复求高度、最大值、是否包含目标等信息。若每个节点能一次性向父节点汇报足够信息，就不应重复下钻。放到本题里，要重点观察“构造时要小心后序左右区间边界，避免切分错位。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。树形 DP 把每个节点看成子问题根；BFS 把层次边界显式放进队列；BST 题利用中序有序或上下界约束。工程实现优先考虑显式栈避免深树调用栈风险。本题的优化应当围绕“构造时要小心后序左右区间边界，避免切分错位。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。返回值或栈元素必须有语义：当前子树高度、当前路径和、当前节点合法上下界、父指针、层号。不要只说遍历。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。正确性通常是对子树归纳。左右子树已经给出稳定答案，父节点只需按题目约束合并这两个答案。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。示例中可以用指针节点，但比较节点身份时比较指针。极端深树用 `std::vector<TreeNode*>` 显式栈或队列。最大路径类题要用足够小的初值，不能默认零。本题还要特别注意：空树、单节点、全左链、全右链。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：空树、单节点、全左链、全右链。若给前序和后序，二叉树通常不能唯一确定，除非额外限制。实验包含空树、单节点、链式树、完全树、含重复值、全负值、目标节点不存在和极端不平衡树。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

108 有序数组转二叉搜索树

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。有序数组中点作为根能让左右规模接近。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“二叉树与树形状态”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再

对每个候选重新检查是否合法。暴力做法对每个节点反复扫描子树，或者只检查父子局部关系。树题真正考的是状态如何从父到子、从子到父或按层传播。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。慢点在于同一棵子树被重复求高度、最大值、是否包含目标等信息。若每个节点能一次性向父节点汇报足够信息，就不应重复下钻。放到本题里，要重点观察“每次选择区间中点，左右子区间构造左右子树。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。树形 DP 把每个节点看成子问题根；BFS 把层次边界显式放进队列；BST 题利用中序有序或上下界约束。工程实现优先考虑显式栈避免深树调用栈风险。本题的优化应当围绕“每次选择区间中点，左右子区间构造左右子树。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。返回值或栈元素必须有语义：当前子树高度、当前路径和、当前节点合法上下界、父指针、层号。不要只说遍历。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。正确性通常是对子树归纳。左右子树已经给出稳定答案，父节点只需按题目约束合并这两个答案。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。示例中可以用指针节点，但比较节点身份时比较指针。极端深树用 `std::vector<TreeNode*>` 显式栈或队列。最大路径类题要用足够小的初值，不能默认零。本题还要特别注意：空区间、偶数长度选左中还是右中、高度平衡定义。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：空区间、偶数长度选左中还是右中、高度平衡定义。工程上递归可读，若严格避免递归可用队列保存区间和父指针。实验包含空树、单节点、链式树、完全树、含重复值、全负值、目标节点不存在和极端不平衡树。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

110 平衡二叉树

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。每个节点需要知道子树高度以及是否已经失衡。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“二叉树与树形状态”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力做法对每个节点反复扫描子树，或者只检查父子局部关系。树题真正考的是状态如何从父到子、从子到父或按层传播。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。慢点在于同一棵子树被重复求高度、最大值、是否包含目标等信息。若每个节点能一次性向父节点汇报足够信息，就不应重复下钻。放到本题里，要重点观察“后序汇总时一旦子树失衡就向上返回失败标记，避免重复算高度。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。树形 DP 把每个节点看成子问题

根；BFS 把层次边界显式放进队列；BST 题利用中序有序或上下界约束。工程实现优先考虑显式栈避免深树调用栈风险。本题的优化应当围绕“后序汇总时一旦子树失衡就向上返回失败标记，避免重复算高度。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。返回值或栈元素必须有语义：当前子树高度、当前路径和、当前节点合法上下界、父指针、层号。不要只说遍历。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。正确性通常是对子树归纳。左右子树已经给出稳定答案，父节点只需按题目约束合并这两个答案。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。示例中可以用指针节点，但比较节点身份时比较指针。极端深树用 `std::vector<TreeNode*>` 显式栈或队列。最大路径类题要用足够小的初值，不能默认零。本题还要特别注意：空树、单节点、左右高度差正好一、链式树。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：空树、单节点、左右高度差正好一、链式树。可以把返回值设计成 optional 高度，空表示不平衡。实验包含空树、单节点、链式树、完全树、含重复值、全负值、目标节点不存在和极端不平衡树。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

111 二叉树最小深度

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。最小深度是到最近叶子的节点数，不是左右深度简单取最小。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“二叉树与树形状态”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力做法对每个节点反复扫描子树，或者只检查父子局部关系。树题真正考的是状态如何从父到子、从子到父或按层传播。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。慢点在于同一棵子树被重复求高度、最大值、是否包含目标等信息。若每个节点能一次性向父节点汇报足够信息，就不应重复下钻。放到本题里，要重点观察“BFS 第一次遇到叶子就是最小深度，因为按层扩展。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。树形 DP 把每个节点看成子问题根；BFS 把层次边界显式放进队列；BST 题利用中序有序或上下界约束。工程实现优先考虑显式栈避免深树调用栈风险。本题的优化应当围绕“BFS 第一次遇到叶子就是最小深度，因为按层扩展。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。返回值或栈元素必须有语义：当前子树高度、当前路径和、当前节点合法上下界、父指针、层号。不要只说遍历。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。正确性通常是对子树归纳。左右子树已经给出稳定答案，父节点只需按题目约束合并这两个答案。证明不需

要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。示例中可以用指针节点，但比较节点身份时比较指针。极端深树用 `std::vector<TreeNode*>` 显式栈或队列。最大路径类题要用足够小的初值，不能默认零。本题还要特别注意：只有左子树、只有右子树、根为空、根是叶子。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：只有左子树、只有右子树、根为空、根是叶子。DFS 写法必须特殊处理空子树不能作为最小路径。实验包含空树、单节点、链式树、完全树、含重复值、全负值、目标节点不存在和极端不平衡树。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

112 路径总和

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。路径必须从根到叶子，状态是剩余目标值。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“二叉树与树形状态”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力做法对每个节点反复扫描子树，或者只检查父子局部关系。树题真正考的是状态如何从父到子、从子到父或按层传播。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。慢点在于同一棵子树被重复求高度、最大值、是否包含目标等信息。若每个节点能一次性向父节点汇报足够信息，就不应重复下钻。放到本题里，要重点观察“沿路径减去当前节点值，到叶子时检查剩余是否等于叶子值。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。树形 DP 把每个节点看成子问题根；BFS 把层次边界显式放进队列；BST 题利用中序有序或上下界约束。工程实现优先考虑显式栈避免深树调用栈风险。本题的优化应当围绕“沿路径减去当前节点值，到叶子时检查剩余是否等于叶子值。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。返回值或栈元素必须有语义：当前子树高度、当前路径和、当前节点合法上下界、父指针、层号。不要只说遍历。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。正确性通常是对子树归纳。左右子树已经给出稳定答案，父节点只需按题目约束合并这两个答案。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。示例中可以用指针节点，但比较节点身份时比较指针。极端深树用 `std::vector<TreeNode*>` 显式栈或队列。最大路径类题要用足够小的初值，不能默认零。本题还要特别注意：负数节点、目标为零、非叶子提前达到目标、空树。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点

是：负数节点、目标为零、非叶子提前达到目标、空树。若要求列出所有路径，需要保存路径数组并在回退时弹出。实验包含空树、单节点、链式树、完全树、含重复值、全负值、目标节点不存在和极端不平衡树。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

113 路径总和 II

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。相比判定题，状态还要保存当前路径。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“二叉树与树形状态”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力做法对每个节点反复扫描子树，或者只检查父子局部关系。树题真正考的是状态如何从父到子、从子到父或按层传播。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。慢点在于同一棵子树被重复求高度、最大值、是否包含目标等信息。若每个节点能一次性向父节点汇报足够信息，就不应重复下钻。放到本题里，要重点观察“到叶子且和满足时复制路径；回溯时恢复路径。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。树形 DP 把每个节点看成子问题根；BFS 把层次边界显式放进队列；BST 题利用中序有序或上下界约束。工程实现优先考虑显式栈避免深树调用栈风险。本题的优化应当围绕“到叶子且和满足时复制路径；回溯时恢复路径。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。返回值或栈元素必须有语义：当前子树高度、当前路径和、当前节点合法上下界、父指针、层号。不要只说遍历。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。正确性通常是对子树归纳。左右子树已经给出稳定答案，父节点只需按题目约束合并这两个答案。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。示例中可以用指针节点，但比较节点身份时比较指针。极端深树用 `std::vector<TreeNode*>` 显式栈或队列。最大路径类题要用足够小的初值，不能默认零。本题还要特别注意：多个答案、负数、空树、路径数组复制时机。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：多个答案、负数、空树、路径数组复制时机。若路径不要求从根到叶，模型会变成前缀和加哈希。实验包含空树、单节点、链式树、完全树、含重复值、全负值、目标节点不存在和极端不平衡树。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

114 二叉树展开为链表

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。目标是按前序顺序把树原地改成右指针链。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“二叉树与树形状态”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再

对每个候选重新检查是否合法。暴力做法对每个节点反复扫描子树，或者只检查父子局部关系。树题真正考的是状态如何从父到子、从子到父或按层传播。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。慢点在于同一棵子树被重复求高度、最大值、是否包含目标等信息。若每个节点能一次性向父节点汇报足够信息，就不应重复下钻。放到本题里，要重点观察“显式栈按右后左先入顺序处理，逐个接到前驱右侧。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。树形 DP 把每个节点看成子问题根；BFS 把层次边界显式放进队列；BST 题利用中序有序或上下界约束。工程实现优先考虑显式栈避免深树调用栈风险。本题的优化应当围绕“显式栈按右后左先入顺序处理，逐个接到前驱右侧。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。返回值或栈元素必须有语义：当前子树高度、当前路径和、当前节点合法上下界、父指针、层号。不要只说遍历。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。正确性通常是对子树归纳。左右子树已经给出稳定答案，父节点只需按题目约束合并这两个答案。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。示例中可以用指针节点，但比较节点身份时比较指针。极端深树用 `std::vector<TreeNode*>` 显式栈或队列。最大路径类题要用足够小的初值，不能默认零。本题还要特别注意：空树、单节点、原有右子树不能丢、左指针置空。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：空树、单节点、原有右子树不能丢、左指针置空。也可用寻找左子树最右节点的原地方法。实验包含空树、单节点、链式树、完全树、含重复值、全负值、目标节点不存在和极端不平衡树。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

124 二叉树最大路径和

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。路径可以从任意节点到任意节点，但不能分叉向父节点贡献两边。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“二叉树与树形状态”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力做法对每个节点反复扫描子树，或者只检查父子局部关系。树题真正考的是状态如何从父到子、从子到父或按层传播。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。慢点在于同一棵子树被重复求高度、最大值、是否包含目标等信息。若每个节点能一次性向父节点汇报足够信息，就不应重复下钻。放到本题里，要重点观察“每个节点向父亲贡献单边最大增益，全局答案可用左右两边加当前节点更新。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。树形 DP 把每个节点看成子问题根；BFS 把层次边界显式放进队列；BST 题利用中序有序或上下界约束。工程实现优先考虑显式栈避免深树调用栈风险。本题的优化应当围绕“每个节点向父亲贡献单边最大增益，全局答案可用左右两边加当前节点更新。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。返回值或栈元素必须有语义：当前子树高度、当前路径和、当前节点合法上下界、父指针、层号。不要只说遍历。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。正确性通常是对子树归纳。左右子树已经给出稳定答案，父节点只需按题目约束合并这两个答案。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。示例中可以用指针节点，但比较节点身份时比较指针。极端深树用 `std::vector<TreeNode*>` 显式栈或队列。最大路径类题要用足够小的初值，不能默认零。本题还要特别注意：全负数、单节点、左右增益为负时取零、答案初始值。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：全负数、单节点、左右增益为负时取零、答案初始值。若路径必须从根到叶，状态和答案位置完全不同。实验包含空树、单节点、链式树、完全树、含重复值、全负值、目标节点不存在和极端不平衡树。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

199 二叉树右视图

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。每层从右侧能看到的的就是该层最后访问或最先访问的右侧节点。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“二叉树与树形状态”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力做法对每个节点反复扫描子树，或者只检查父子局部关系。树题真正考的是状态如何从父到子、从子到父或按层传播。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。慢点在于同一棵子树被重复求高度、最大值、是否包含目标等信息。若每个节点能一次性向父节点汇报足够信息，就不应重复下钻。放到本题里，要重点观察“BFS 固定层边界，取每层最后一个；或 DFS 按右优先首次到达深度。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。树形 DP 把每个节点看成子问题根；BFS 把层次边界显式放进队列；BST 题利用中序有序或上下界约束。工程实现优先考虑显式栈避免深树调用栈风险。本题的优化应当围绕“BFS 固定层边界，取每层最后一个；或 DFS 按右优先首次到达深度。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。返回值或栈元素必须有语义：当前子树高度、当前路径和、当前节点合法上下界、父指针、层号。不要只说遍历。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。正确性通

常是对子树归纳。左右子树已经给出稳定答案，父节点只需按题目约束合并这两个答案。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。示例中可以用指针节点，但比较节点身份时比较指针。极端深树用 `std::vector<TreeNode*>` 显式栈或队列。最大路径类题要用足够小的初值，不能默认零。本题还要特别注意：空树、左链、右链、左右交错。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：空树、左链、右链、左右交错。若求左视图，只改变层内取值方向。实验包含空树、单节点、链式树、完全树、含重复值、全负值、目标节点不存在和极端不平衡树。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

208 实现 Trie

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。前缀树把字符串公共前缀共享成路径。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“二叉树与树形状态”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力做法对每个节点反复扫描子树，或者只检查父子局部关系。树题真正考的是状态如何从父到子、从子到父或按层传播。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。慢点在于同一棵子树被重复求高度、最大值、是否包含目标等信息。若每个节点能一次性向父节点汇报足够信息，就不应重复下钻。放到本题里，要重点观察“插入和查询都逐字符沿边移动，不存在的边按需要创建。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。树形 DP 把每个节点看成子问题根；BFS 把层次边界显式放进队列；BST 题利用中序有序或上下界约束。工程实现优先考虑显式栈避免深树调用栈风险。本题的优化应当围绕“插入和查询都逐字符沿边移动，不存在的边按需要创建。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。返回值或栈元素必须有语义：当前子树高度、当前路径和、当前节点合法上下界、父指针、层号。不要只说遍历。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。正确性通常是对子树归纳。左右子树已经给出稳定答案，父节点只需按题目约束合并这两个答案。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。示例中可以用指针节点，但比较节点身份时比较指针。极端深树用 `std::vector<TreeNode*>` 显式栈或队列。最大路径类题要用足够小的初值，不能默认零。本题还要特别注意：空字符串约定、单词结束标记、前缀存在不等于单词存在。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点

是：空字符串约定、单词结束标记、前缀存在不等于单词存在。若字符集很大，用哈希子节点；小写字母用数组更快。实验包含空树、单节点、链式树、完全树、含重复值、全负值、目标节点不存在和极端不平衡树。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

211 添加与搜索单词

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。点号通配符让查询可能分叉，但前缀树仍能剪掉不匹配前缀。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“二叉树与树形状态”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力做法对每个节点反复扫描子树，或者只检查父子局部关系。树题真正考的是状态如何从父到子、从子到父或按层传播。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。慢点在于同一棵子树被重复求高度、最大值、是否包含目标等信息。若每个节点能一次性向父节点汇报足够信息，就不应重复下钻。放到本题里，要重点观察“普通字符沿唯一边走，点号枚举当前节点所有孩子。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。树形 DP 把每个节点看成子问题根；BFS 把层次边界显式放进队列；BST 题利用中序有序或上下界约束。工程实现优先考虑显式栈避免深树调用栈风险。本题的优化应当围绕“普通字符沿唯一边走，点号枚举当前节点所有孩子。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。返回值或栈元素必须有语义：当前子树高度、当前路径和、当前节点合法上下界、父指针、层号。不要只说遍历。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。正确性通常是对子树归纳。左右子树已经给出稳定答案，父节点只需按题目约束合并这两个答案。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。示例中可以用指针节点，但比较节点身份时比较指针。极端深树用 `std::vector<TreeNode*>` 显式栈或队列。最大路径类题要用足够小的初值，不能默认零。本题还要特别注意：连续通配符、空结果、单词结束标记、字符集大小。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：连续通配符、空结果、单词结束标记、字符集大小。大量通配符会接近遍历整棵 Trie，需要规模约束。实验包含空树、单节点、链式树、完全树、含重复值、全负值、目标节点不存在和极端不平衡树。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

226 翻转二叉树

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。每个节点交换左右孩子即可，遍历顺序不影响最终结构。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“二叉树与树形状态”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再

对每个候选重新检查是否合法。暴力做法对每个节点反复扫描子树，或者只检查父子局部关系。树题真正考的是状态如何从父到子、从子到父或按层传播。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。慢点在于同一棵子树被重复求高度、最大值、是否包含目标等信息。若每个节点能一次性向父节点汇报足够信息，就不应重复下钻。放到本题里，要重点观察“显式队列或栈逐个节点交换，避免递归深度。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。树形 DP 把每个节点看成子问题根；BFS 把层次边界显式放进队列；BST 题利用中序有序或上下界约束。工程实现优先考虑显式栈避免深树调用栈风险。本题的优化应当围绕“显式队列或栈逐个节点交换，避免递归深度。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。返回值或栈元素必须有语义：当前子树高度、当前路径和、当前节点合法上下界、父指针、层号。不要只说遍历。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。正确性通常是对子树归纳。左右子树已经给出稳定答案，父节点只需按题目约束合并这两个答案。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。示例中可以用指针节点，但比较节点身份时比较指针。极端深树用 `std::vector<TreeNode*>` 显式栈或队列。最大路径类题要用足够小的初值，不能默认零。本题还要特别注意：空树、单节点、只有一侧子树、重复值。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：空树、单节点、只有一侧子树、重复值。这题简单但适合训练树节点指针修改。实验包含空树、单节点、链式树、完全树、含重复值、全负值、目标节点不存在和极端不平衡树。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

230 二叉搜索树中第 K 小

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。BST 中序遍历按升序访问节点。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“二叉树与树形状态”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力做法对每个节点反复扫描子树，或者只检查父子局部关系。树题真正考的是状态如何从父到子、从子到父或按层传播。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。慢点在于同一棵子树被重复求高度、最大值、是否包含目标等信息。若每个节点能一次性向父节点汇报足够信息，就不应重复下钻。放到本题里，要重点观察“显式栈中序，访问第 k 个节点时返回。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。树形 DP 把每个节点看成子问题

根；BFS 把层次边界显式放进队列；BST 题利用中序有序或上下界约束。工程实现优先考虑显式栈避免深树调用栈风险。本题的优化应当围绕“显式栈中序，访问第 k 个节点时返回。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。返回值或栈元素必须有语义：当前子树高度、当前路径和、当前节点合法上下界、父指针、层号。不要只说遍历。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。正确性通常是对子树归纳。左右子树已经给出稳定答案，父节点只需按题目约束合并这两个答案。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。示例中可以用指针节点，但比较节点身份时比较指针。极端深树用 `std::vector<TreeNode*>` 显式栈或队列。最大路径类题要用足够小的初值，不能默认零。本题还要特别注意： k 为一、 k 为节点数、树不平衡、是否有重复值。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是： k 为一、 k 为节点数、树不平衡、是否有重复值。若频繁查询，可在节点维护子树大小成为顺序统计树。实验包含空树、单节点、链式树、完全树、含重复值、全负值、目标节点不存在和极端不平衡树。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

235 二叉搜索树最近公共祖先

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。BST 的值域关系能决定两个目标在当前节点的哪一侧。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“二叉树与树形状态”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力做法对每个节点反复扫描子树，或者只检查父子局部关系。树题真正考的是状态如何从父到子、从子到父或按层传播。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。慢点在于同一棵子树被重复求高度、最大值、是否包含目标等信息。若每个节点能一次性向父节点汇报足够信息，就不应重复下钻。放到本题里，要重点观察“两个值都小去左，都大去右，否则当前节点就是分叉点。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。树形 DP 把每个节点看成子问题根；BFS 把层次边界显式放进队列；BST 题利用中序有序或上下界约束。工程实现优先考虑显式栈避免深树调用栈风险。本题的优化应当围绕“两个值都小去左，都大去右，否则当前节点就是分叉点。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。返回值或栈元素必须有语义：当前子树高度、当前路径和、当前节点合法上下界、父指针、层号。不要只说遍历。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。正确性通常是对子树归纳。左右子树已经给出稳定答案，父节点只需按题目约束合并这两个答案。证明不需

要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。示例中可以用指针节点，但比较节点身份时比较指针。极端深树用 `std::vector<TreeNode*>` 显式栈或队列。最大路径类题要用足够小的初值，不能默认零。本题还要特别注意：一个节点是另一个祖先、目标顺序、重复值约定。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：一个节点是另一个祖先、目标顺序、重复值约定。普通二叉树不能用值域剪枝，需要父指针或后序状态。实验包含空树、单节点、链式树、完全树、含重复值、全负值、目标节点不存在和极端不平衡树。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

236 二叉树最近公共祖先

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。普通树没有有序性，只能依靠祖先关系。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“二叉树与树形状态”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力做法对每个节点反复扫描子树，或者只检查父子局部关系。树题真正考的是状态如何从父到子、从子到父或按层传播。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。慢点在于同一棵子树被重复求高度、最大值、是否包含目标等信息。若每个节点能一次性向父节点汇报足够信息，就不应重复下钻。放到本题里，要重点观察“父指针集合或后序返回目标命中状态都可行。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。树形 DP 把每个节点看成子问题根；BFS 把层次边界显式放进队列；BST 题利用中序有序或上下界约束。工程实现优先考虑显式栈避免深树调用栈风险。本题的优化应当围绕“父指针集合或后序返回目标命中状态都可行。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。返回值或栈元素必须有语义：当前子树高度、当前路径和、当前节点合法上下界、父指针、层号。不要只说遍历。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。正确性通常是对子树归纳。左右子树已经给出稳定答案，父节点只需按题目约束合并这两个答案。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。示例中可以用指针节点，但比较节点身份时比较指针。极端深树用 `std::vector<TreeNode*>` 显式栈或队列。最大路径类题要用足够小的初值，不能默认零。本题还要特别注意：目标不存在、 p 等于 q 、一个是另一个祖先、比较节点地址。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重

点是：目标不存在、 p 等于 q 、一个是另一个祖先、比较节点地址。多次 LCA 查询可预处理倍增祖先表。实验包含空树、单节点、链式树、完全树、含重复值、全负值、目标节点不存在和极端不平衡树。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

337 打家劫舍 III

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。树上相邻节点不能同时选，每个节点需要偷和不偷两个状态。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“二叉树与树形状态”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力做法对每个节点反复扫描子树，或者只检查父子局部关系。树题真正考的是状态如何从父到子、从子到父或按层传播。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。慢点在于同一棵子树被重复求高度、最大值、是否包含目标等信息。若每个节点能一次性向父节点汇报足够信息，就不应重复下钻。放到本题里，要重点观察“后序汇总：偷当前则孩子不能偷，不偷当前则孩子可偷可不偷。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。树形 DP 把每个节点看成子问题根；BFS 把层次边界显式放进队列；BST 题利用中序有序或上下界约束。工程实现优先考虑显式栈避免深树调用栈风险。本题的优化应当围绕“后序汇总：偷当前则孩子不能偷，不偷当前则孩子可偷可不偷。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。返回值或栈元素必须有语义：当前子树高度、当前路径和、当前节点合法上下界、父指针、层号。不要只说遍历。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。正确性通常是对子树归纳。左右子树已经给出稳定答案，父节点只需按题目约束合并这两个答案。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。示例中可以用指针节点，但比较节点身份时比较指针。极端深树用 `std::vector<TreeNode*>` 显式栈或队列。最大路径类题要用足够小的初值，不能默认零。本题还要特别注意：空树、负值是否可能、链式树、递归深度。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：空树、负值是否可能、链式树、递归深度。可用显式栈后序保存节点状态，避免调用栈。实验包含空树、单节点、链式树、完全树、含重复值、全负值、目标节点不存在和极端不平衡树。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

450 删除二叉搜索树中的节点

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。删除节点后仍要保持 BST 中序有序。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“二叉树与树形状态”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再

对每个候选重新检查是否合法。暴力做法对每个节点反复扫描子树，或者只检查父子局部关系。树题真正考的是状态如何从父到子、从子到父或按层传播。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。慢点在于同一棵子树被重复求高度、最大值、是否包含目标等信息。若每个节点能一次性向父节点汇报足够信息，就不应重复下钻。放到本题里，要重点观察“若有两个孩子，用后继或前驱替换当前值，再删除那个后继节点。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。树形 DP 把每个节点看成子问题根；BFS 把层次边界显式放进队列；BST 题利用中序有序或上下界约束。工程实现优先考虑显式栈避免深树调用栈风险。本题的优化应当围绕“若有两个孩子，用后继或前驱替换当前值，再删除那个后继节点。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。返回值或栈元素必须有语义：当前子树高度、当前路径和、当前节点合法上下界、父指针、层号。不要只说遍历。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。正确性通常是对子树归纳。左右子树已经给出稳定答案，父节点只需按题目约束合并这两个答案。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。示例中可以用指针节点，但比较节点身份时比较指针。极端深树用 `std::vector<TreeNode*>` 显式栈或队列。最大路径类题要用足够小的初值，不能默认零。本题还要特别注意：删除叶子、一个孩子、两个孩子、删除根节点。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：删除叶子、一个孩子、两个孩子、删除根节点。工程实现要清楚是移动值还是重连节点。实验包含空树、单节点、链式树、完全树、含重复值、全负值、目标节点不存在和极端不平衡树。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

543 二叉树直径

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。直径经过某节点时等于左高加右高，但向父亲只能贡献单边高度。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“二叉树与树形状态”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力做法对每个节点反复扫描子树，或者只检查父子局部关系。树题真正考的是状态如何从父到子、从子到父或按层传播。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。慢点在于同一棵子树被重复求高度、最大值、是否包含目标等信息。若每个节点能一次性向父节点汇报足够信息，就不应重复下钻。放到本题里，要重点观察“后序遍历同时更新全局直径并返回高度。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。树形 DP 把每个节点看成子问题

根；BFS 把层次边界显式放进队列；BST 题利用中序有序或上下界约束。工程实现优先考虑显式栈避免深树调用栈风险。本题的优化应当围绕“后序遍历同时更新全局直径并返回高度。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。返回值或栈元素必须有语义：当前子树高度、当前路径和、当前节点合法上下界、父指针、层号。不要只说遍历。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。正确性通常是对子树归纳。左右子树已经给出稳定答案，父节点只需按题目约束合并这两个答案。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。示例中可以用指针节点，但比较节点身份时比较指针。极端深树用 `std::vector<TreeNode*>` 显式栈或队列。最大路径类题要用足够小的初值，不能默认零。本题还要特别注意：空树、单节点、链式树、边数还是节点数定义。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：空树、单节点、链式树、边数还是节点数定义。最大路径和是同型题，但贡献值和全局更新有负数处理。实验包含空树、单节点、链式树、完全树、含重复值、全负值、目标节点不存在和极端不平衡树。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

图建模、BFS 与 DFS

下面的题卡都按同一条链路展开：从问题出发，先写暴力，再定位重复工作，然后选择数据结构或状态不变量，最后落到 C++ 实现、边界和扩展。格式统一是为了训练面试表达，而每张卡的关键观察和失效条件都要单独复盘。

图示：图建模、BFS 与 DFS 推导链

暴力基线	枚举候选、完整模拟或逐个检查，先保证覆盖全部可能答案。
瓶颈定位	慢点来自重复访问和邻居查询过慢。若节点很多但边很少，用邻接矩阵会浪费空间；若每次找邻居都扫描全体节点，BFS 本身再正确也会超时。
结构选择	无权最短步数用 BFS，连通块和状态检测用 DFS 或显式栈，有向无环依赖用拓扑排序。多源问题把所有源同时入队，状态带资源的问题不能只按位置去重。
不变量	访问标记通常在入队或入栈时设置，保证一个状态只被加入一次。BFS 队列按距离递增；拓扑排序队列保存当前入度为零的节点。
代码落地	图节点连续时用 <code>std::vector<std::vector<int>></code> ；节点是字符串时先编号或用哈希表。网格方向数组用 <code>constexpr std::array</code> ，边界判断集中写。

127 单词接龙

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。每个单词是节点，一次修改一个字符形成边。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“图建模、BFS 与 DFS”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力做法常从每个节点重新搜索，或者在图中不加访问标记地反复

扩展状态。图题第一步不是选 DFS 还是 BFS，而是明确节点和边。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。慢点来自重复访问和邻居查询过慢。若节点很多但边很少，用邻接矩阵会浪费空间；若每次找邻居都扫描全体节点，BFS 本身再正确也会超时。放到本题里，要重点观察“通配模式桶加速邻居查询，BFS 第一次到达终点就是最短转换长度。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。无权最短步数用 BFS，连通块和状态检测用 DFS 或显式栈，有向无环依赖用拓扑排序。多源问题把所有源同时入队，状态带资源的问题不能只按位置去重。本题的优化应当围绕“通配模式桶加速邻居查询，BFS 第一次到达终点就是最短转换长度。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。访问标记通常在入队或入栈时设置，保证一个状态只被加入一次。BFS 队列按距离递增；拓扑排序队列保存当前入度为零的节点。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。BFS 正确性来自层次扩展，第一次到达某状态就是最短距离。连通块算法正确性来自一次搜索会标记与起点可达的全部节点且不会越过非法边。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。图节点连续时用 `std::vector<std::vector<int>>`；节点是字符串时先编号或用哈希表。网格方向数组用 `constexpr std::array`，边界判断集中写。本题还要特别注意：终点不在词表、起点和终点相同、桶重复扫描、单词长度一致。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：终点不在词表、起点和终点相同、桶重复扫描、单词长度一致。双向 BFS 可以进一步降低搜索空间。实验包含孤立节点、自环、重边、不连通图、有向环、不可达目标、多源同层竞争、网格边界和障碍物。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

130 被围绕的区域

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。只有不连通到边界的 O 才会被翻转。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“图建模、BFS 与 DFS”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力做法常从每个节点重新搜索，或者在图中不加访问标记地反复扩展状态。图题第一步不是选 DFS 还是 BFS，而是明确节点和边。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。慢点来自重复访问和邻居查询过慢。若节点很多但边很少，用邻接矩阵会浪费空间；若每次找邻居都扫描全体节点，BFS 本身再正确也会超时。放到本题里，要重点观察“从边界 O 出发标记安全区域，再翻转剩余 O。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。无权最短步数用 BFS，连通块和

状态检测用 DFS 或显式栈，有向无环依赖用拓扑排序。多源问题把所有源同时入队，状态带资源的问题不能只按位置去重。本题的优化应当围绕“从边界 O 出发标记安全区域，再翻转剩余 O。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。访问标记通常在入队或入栈时设置，保证一个状态只被加入一次。BFS 队列按距离递增；拓扑排序队列保存当前入度为零的节点。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。BFS 正确性来自层次扩展，第一次到达某状态就是最短距离。连通块算法正确性来自一次搜索会标记与起点可达的全部节点且不会越过非法边。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。图节点连续时用 `std::vector<std::vector<int>>`；节点是字符串时先编号或用哈希表。网格方向数组用 `constexpr std::array`，边界判断集中写。本题还要特别注意：全边界、无 O、单行单列、内部岛屿。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：全边界、无 O、单行单列、内部岛屿。这是反向思维：找不能被翻转的区域比直接判断每块区域更简单。实验包含孤立节点、自环、重边、不连通图、有向环、不可达目标、多源同层竞争、网格边界和障碍物。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

133 克隆图

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。每个原节点需要唯一对应一个新节点，边按原图连接。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“图建模、BFS 与 DFS”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力做法常从每个节点重新搜索，或者在图中不加访问标记地反复扩展状态。图题第一步不是选 DFS 还是 BFS，而是明确节点和边。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。慢点来自重复访问和邻居查询过慢。若节点很多但边很少，用邻接矩阵会浪费空间；若每次找邻居都扫描全体节点，BFS 本身再正确也会超时。放到本题里，要重点观察“哈希表保存原节点到克隆节点的映射，BFS 或 DFS 遍历图。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。无权最短步数用 BFS，连通块和状态检测用 DFS 或显式栈，有向无环依赖用拓扑排序。多源问题把所有源同时入队，状态带资源的问题不能只按位置去重。本题的优化应当围绕“哈希表保存原节点到克隆节点的映射，BFS 或 DFS 遍历图。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。访问标记通常在入队或入栈时设置，保证一个状态只被加入一次。BFS 队列按距离递增；拓扑排序队列保存当前入度为零的节点。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法

答案。BFS 正确性来自层次扩展，第一次到达某状态就是最短距离。连通块算法正确性来自一次搜索会标记与起点可达的全部节点且不会越过非法边。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。图节点连续时用 `std::vector<std::vector<int>>`；节点是字符串时先编号或用哈希表。网格方向数组用 `constexpr std::array`，边界判断集中写。本题还要特别注意：空图、自环、重边、环结构、不能按节点值唯一假设。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：空图、自环、重边、环结构、不能按节点值唯一假设。若图节点值不唯一，必须用地址作为键。实验包含孤立节点、自环、重边、不连通图、有向环、不可达目标、多源同层竞争、网格边界和障碍物。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

200 岛屿数量

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。陆地格子是节点，四方向相邻是边，岛屿是连通块。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“图建模、BFS 与 DFS”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力做法常从每个节点重新搜索，或者在图中不加访问标记地反复扩展状态。图题第一步不是选 DFS 还是 BFS，而是明确节点和边。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。慢点来自重复访问和邻居查询过慢。若节点很多但边很少，用邻接矩阵会浪费空间；若每次找邻居都扫描全体节点，BFS 本身再正确也会超时。放到本题里，要重点观察“扫描遇到未访问陆地就启动搜索并标记整块。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。无权最短步数用 BFS，连通块和状态检测用 DFS 或显式栈，有向无环依赖用拓扑排序。多源问题把所有源同时入队，状态带资源的问题不能只按位置去重。本题的优化应当围绕“扫描遇到未访问陆地就启动搜索并标记整块。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。访问标记通常在入队或入栈时设置，保证一个状态只被加入一次。BFS 队列按距离递增；拓扑排序队列保存当前入度为零的节点。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。BFS 正确性来自层次扩展，第一次到达某状态就是最短距离。连通块算法正确性来自一次搜索会标记与起点可达的全部节点且不会越过非法边。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。图节点连续时用 `std::vector<std::vector<int>>`；节点是字符串时先编号或用哈希表。网格方向数组用 `constexpr std::array`，边界判断集中写。本题还要特别注意：空网格、全水、全陆、斜对角不连通、是否允许修改输入。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答

案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：空网格、全水、全陆、斜对角不连通、是否允许修改输入。也可用并查集合并相邻陆地，适合动态岛屿扩展。实验包含孤立节点、自环、重边、不连通图、有向环、不可达目标、多源同层竞争、网格边界和障碍物。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

207 课程表

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。课程是节点，先修关系是有向边，问题是判断是否有环。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“图建模、BFS 与 DFS”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力做法常从每个节点重新搜索，或者在图中不加访问标记地反复扩展状态。图题第一步不是选 DFS 还是 BFS，而是明确节点和边。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。慢点来自重复访问和邻居查询过慢。若节点很多但边很少，用邻接矩阵会浪费空间；若每次找邻居都扫描全体节点，BFS 本身再正确也会超时。放到本题里，要重点观察“入度拓扑排序不断学习当前无依赖课程。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。无权最短步数用 BFS，连通块和状态检测用 DFS 或显式栈，有向无环依赖用拓扑排序。多源问题把所有源同时入队，状态带资源的问题不能只按位置去重。本题的优化应当围绕“入度拓扑排序不断学习当前无依赖课程。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。访问标记通常在入队或入栈时设置，保证一个状态只被加入一次。BFS 队列按距离递增；拓扑排序队列保存当前入度为零的节点。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。BFS 正确性来自层次扩展，第一次到达某状态就是最短距离。连通块算法正确性来自一次搜索会标记与起点可达的全部节点且不会越过非法边。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。图节点连续时用 `std::vector<std::vector<int>>`；节点是字符串时先编号或用哈希表。网格方向数组用 `constexpr std::array`，边界判断集中写。本题还要特别注意：边方向、孤立课程、环、自依赖、重复边。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：边方向、孤立课程、环、自依赖、重复边。DFS 三色标记也能判环，但要明确访问状态。实验包含孤立节点、自环、重边、不连通图、有向环、不可达目标、多源同层竞争、网格边界和障碍物。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

210 课程表 II

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。在判环基础上还要输出一个可行拓扑序。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“图建模、BFS 与 DFS”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力做法常从每个节点重新搜索，或者在图中不加访问标记地反复扩展状态。图题第一步不是选 DFS 还是 BFS，而是明确节点和边。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。慢点来自重复访问和邻居查询过慢。若节点很多但边很少，用邻接矩阵会浪费空间；若每次找邻居都扫描全体节点，BFS 本身再正确也会超时。放到本题里，要重点观察“每弹出一个入度为零课程，就把它加入顺序并删除出边。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。无权最短步数用 BFS，连通块和状态检测用 DFS 或显式栈，有向无环依赖用拓扑排序。多源问题把所有源同时入队，状态带资源的问题不能只按位置去重。本题的优化应当围绕“每弹出一个入度为零课程，就把它加入顺序并删除出边。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。访问标记通常在入队或入栈时设置，保证一个状态只被加入一次。BFS 队列按距离递增；拓扑排序队列保存当前入度为零的节点。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。BFS 正确性来自层次扩展，第一次到达某状态就是最短距离。连通块算法正确性来自一次搜索会标记与起点可达的全部节点且不会越过非法边。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。图节点连续时用 `std::vector<std::vector<int>>`；节点是字符串时先编号或用哈希表。网格方向数组用 `constexpr std::array`，边界判断集中写。本题还要特别注意：有环返回空数组、多个可行顺序、边方向。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：有环返回空数组、多个可行顺序、边方向。若需要字典序最小拓扑序，把队列换成小顶堆。实验包含孤立节点、自环、重边、不连通图、有向环、不可达目标、多源同层竞争、网格边界和障碍物。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

417 太平洋大西洋水流问题

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。从每个格子向海搜索很慢，反向从海边沿非递减高度搜索。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“图建模、BFS 与 DFS”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力做法常从每个节点重新搜索，或者在图中不加访问标记地反复扩展状态。图题第一步不是选 DFS 还是 BFS，而是明确节点和边。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。慢点来自重复访问和邻居查询过慢。若节点很多但边很少，用邻接矩阵会浪费空间；若每次找邻居都扫描全体节点，BFS 本身再正确也会超时。放到本题里，要重点观察“分别标记能到太平洋和大西洋的格子，交集就是答案。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。无权最短步数用 BFS，连通块和状态检测用 DFS 或显式栈，有向无环依赖用拓扑排序。多源问题把所有源同时入队，状态带资源的问题不能只按位置去重。本题的优化应当围绕“分别标记能到太平洋和大西洋的格子，交集就是答案。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。访问标记通常在入队或入栈时设置，保证一个状态只被加入一次。BFS 队列按距离递增；拓扑排序队列保存当前入度为零的节点。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。BFS 正确性来自层次扩展，第一次到达某状态就是最短距离。连通块算法正确性来自一次搜索会标记与起点可达的全部节点且不会越过非法边。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。图节点连续时用 `std::vector<std::vector<int>>`；节点是字符串时先编号或用哈希表。网格方向数组用 `constexpr std::array`，边界判断集中写。本题还要特别注意：单行单列、平台高度、边界重复入队、方向条件反向。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：单行单列、平台高度、边界重复入队、方向条件反向。反向搜索是很多可达性题的重要技巧。实验包含孤立节点、自环、重边、不连通图、有向环、不可达目标、多源同层竞争、网格边界和障碍物。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

695 岛屿的最大面积

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。面积就是一个连通块中陆地格子的数量。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“图建模、BFS 与 DFS”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力做法常从每个节点重新搜索，或者在图中不加访问标记地反复扩展状态。图题第一步不是选 DFS 还是 BFS，而是明确节点和边。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。慢点来自重复访问和邻居查询过慢。若节点很多但边很少，用邻接矩阵会浪费空间；若每次找邻居都扫描全体节点，BFS 本身再正确也会超时。放到本题里，要重点观察“每发现新陆地就搜索完整连通块并计数，取最大值。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。无权最短步数用 BFS，连通块和状态检测用 DFS 或显式栈，有向无环依赖用拓扑排序。多源问题把所有源同时入队，状态带资源的问题不能只按位置去重。本题的优化应当围绕“每发现新陆地就搜索完整连通块并计数，取最大值。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。访问标记通常在入队或入栈时设置，保证一个状态只被加入一次。BFS 队列按距离递增；拓扑排序队列保存当前入度为零的节点。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法

答案。BFS 正确性来自层次扩展，第一次到达某状态就是最短距离。连通块算法正确性来自一次搜索会标记与起点可达的全部节点且不会越过非法边。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。图节点连续时用 `std::vector<std::vector<int>>`；节点是字符串时先编号或用哈希表。网格方向数组用 `constexpr std::array`，边界判断集中写。本题还要特别注意：全水、全陆、细长岛、是否允许修改输入。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：全水、全陆、细长岛、是否允许修改输入。与岛屿数量相比，只是连通块聚合值不同。实验包含孤立节点、自环、重边、不连通图、有向环、不可达目标、多源同层竞争、网格边界和障碍物。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

752 打开转盘锁

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。四位密码是状态，每次转动一位形成边。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“图建模、BFS 与 DFS”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力做法常从每个节点重新搜索，或者在图中不加访问标记地反复扩展状态。图题第一步不是选 DFS 还是 BFS，而是明确节点和边。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。慢点来自重复访问和邻居查询过慢。若节点很多但边很少，用邻接矩阵会浪费空间；若每次找邻居都扫描全体节点，BFS 本身再正确也会超时。放到本题里，要重点观察“BFS 从起点扩展，死亡状态不能入队，访问状态避免重复。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。无权最短步数用 BFS，连通块和状态检测用 DFS 或显式栈，有向无环依赖用拓扑排序。多源问题把所有源同时入队，状态带资源的问题不能只按位置去重。本题的优化应当围绕“BFS 从起点扩展，死亡状态不能入队，访问状态避免重复。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。访问标记通常在入队或入栈时设置，保证一个状态只被加入一次。BFS 队列按距离递增；拓扑排序队列保存当前入度为零的节点。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。BFS 正确性来自层次扩展，第一次到达某状态就是最短距离。连通块算法正确性来自一次搜索会标记与起点可达的全部节点且不会越过非法边。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。图节点连续时用 `std::vector<std::vector<int>>`；节点是字符串时先编号或用哈希表。网格方向数组用 `constexpr std::array`，边界判断集中写。本题还要特别注意：起点死亡、目标就是起点、数字环绕、死亡集合很大。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答

案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：起点死亡、目标就是起点、数字环绕、死亡集合很大。双向 BFS 能明显减少状态展开。实验包含孤立节点、自环、重边、不连通图、有向环、不可达目标、多源同层竞争、网格边界和障碍物。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

994 腐烂的橘子

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。所有初始腐烂橘子同时作为 BFS 源点。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“图建模、BFS 与 DFS”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力做法常从每个节点重新搜索，或者在图中不加访问标记地反复扩展状态。图题第一步不是选 DFS 还是 BFS，而是明确节点和边。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。慢点来自重复访问和邻居查询过慢。若节点很多但边很少，用邻接矩阵会浪费空间；若每次找邻居都扫描全体节点，BFS 本身再正确也会超时。放到本题里，要重点观察“按层扩散，分钟数等于最后一个新鲜橘子被首次访问的层数。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。无权最短步数用 BFS，连通块和状态检测用 DFS 或显式栈，有向无环依赖用拓扑排序。多源问题把所有源同时入队，状态带资源的问题不能只按位置去重。本题的优化应当围绕“按层扩散，分钟数等于最后一个新鲜橘子被首次访问的层数。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。访问标记通常在入队或入栈时设置，保证一个状态只被加入一次。BFS 队列按距离递增；拓扑排序队列保存当前入度为零的节点。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。BFS 正确性来自层次扩展，第一次到达某状态就是最短距离。连通块算法正确性来自一次搜索会标记与起点可达的全部节点且不会越过非法边。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。图节点连续时用 `std::vector<std::vector<int>>`；节点是字符串时先编号或用哈希表。网格方向数组用 `constexpr std::array`，边界判断集中写。本题还要特别注意：没有新鲜橘子、没有腐烂源、隔离新鲜橘子、空格阻断。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：没有新鲜橘子、没有腐烂源、隔离新鲜橘子、空格阻断。多源 BFS 也适用于最近零距离、地图扩散等题。实验包含孤立节点、自环、重边、不连通图、有向环、不可达目标、多源同层竞争、网格边界和障碍物。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

最短路与带权图

下面的题卡都按同一条链路展开：从问题出发，先写暴力，再定位重复工作，然后选择数据结构或状态不变量，最后落到 C++ 实现、边界和扩展。格式统一是为了训练面试表达，而每张卡的关键观察和失效条件都要单独复盘。

图示：最短路与带权图推导链

暴力基线	枚举候选、完整模拟或逐个检查，先保证覆盖全部可能答案。
瓶颈定位	路径数量可能指数级。最短路算法的核心是用当前已知最小代价组织扩展顺序，而不是盲目枚举路径。
结构选择	非负权图用 Dijkstra，小顶堆保存当前距离最小的候选；边权为一用 BFS；有负边要考虑 Bellman-Ford 或重新建模。路径代价若是最大边权，也可用 Dijkstra 或二分答案。
不变量	Dijkstra 的不变量是：每次弹出的未过期节点，其距离已经是最终最短距离。过期堆元素必须跳过，否则会重复扩展旧状态。
代码落地	由于 <code>priority_queue</code> 没有 <code>decrease-key</code> ，更新距离时压入新状态，弹出时比较距离数组。距离用 <code>long long</code> 更稳，图用邻接表保存 (<code>to, weight</code>)。

743 网络延迟时间

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。有向正权图从起点到所有节点的最短路最大值。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“最短路与带权图”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力做法可能枚举所有路径，或者把带权边错误当成普通 BFS。只要边权不全相等，按步数最少就不等于总代价最小。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。路径数量可能指数级。最短路算法的核心是用当前已知最小代价组织扩展顺序，而不是盲目枚举路径。放到本题里，要重点观察“Dijkstra 求每个节点最短距离，最后取最大，若有无穷则不可达。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。非负权图用 Dijkstra，小顶堆保存当前距离最小的候选；边权为一用 BFS；有负边要考虑 Bellman-Ford 或重新建模。路径代价若是最大边权，也可用 Dijkstra 或二分答案。本题的优化应当围绕“Dijkstra 求每个节点最短距离，最后取最大，若有无穷则不可达。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。Dijkstra 的不变量是：每次弹出的未过期节点，其距离已经是最终最短距离。过期堆元素必须跳过，否则会重复扩展旧状态。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。非负边保证从当前最小距离节点再走出去不会得到更短路径回头改写它。二分可达性则要证明阈值越大，可用边只会增加。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。由于 `priority_queue` 没有 `decrease-key`，更新距离时压入新状态，弹出时比较距离数组。距离用 `long long` 更稳，图用邻接表保存 (`to, weight`)。本题还要特别注意：节点编号从一开始、不可达、重边、过期堆元素。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：节点编号从一开始、不可达、重边、过期堆元素。边权相同才可用普通 BFS。实验包含不可

达节点、零权边、多条等长路径、旧堆状态、起点到自己、网格代价不是求和而是取最大值等情形。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

1631 最小体力消耗路径

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。路径代价是沿途最大高度差，不是边权和。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“最短路与带权图”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力做法可能枚举所有路径，或者把带权边错误当成普通 BFS。只要边权不全相等，按步数最少就不等于总代价最小。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。路径数量可能指数级。最短路算法的核心是用当前已知最小代价组织扩展顺序，而不是盲目枚举路径。放到本题里，要重点观察“Dijkstra 的松弛改成取当前代价和新边代价的最大值。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。非负权图用 Dijkstra，小顶堆保存当前距离最小的候选；边权为一用 BFS；有负边要考虑 Bellman-Ford 或重新建模。路径代价若是最大边权，也可用 Dijkstra 或二分答案。本题的优化应当围绕“Dijkstra 的松弛改成取当前代价和新边代价的最大值。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。Dijkstra 的不变量是：每次弹出的未过期节点，其距离已经是最终最短距离。过期堆元素必须跳过，否则会重复扩展旧状态。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。非负边保证从当前最小距离节点再走出去不会得到更短路径回头改写它。二分可达性则要证明阈值越大，可用边只会增加。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。由于 `priority_queue` 没有 `decrease-key`，更新距离时压入新状态，弹出时比较距离数组。距离用 `long long` 更稳，图用邻接表保存 `(to, weight)`。本题还要特别注意：单格、平坦网格、必须绕路、多个相同代价。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：单格、平坦网格、必须绕路、多个相同代价。也可二分体力上限后 BFS 检查可达。实验包含不可达节点、零权边、多条等长路径、旧堆状态、起点到自己、网格代价不是求和而是取最大值等情形。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

并查集与动态连通性

下面的题卡都按同一条链路展开：从问题出发，先写暴力，再定位重复工作，然后选择数据结构或状态不变量，最后落到 C++ 实现、边界和扩展。格式统一是为了训练面试表达，而每张卡的关键观察和失效条件都要单独复盘。

图示：并查集与动态连通性推导链

暴力基线	枚举候选、完整模拟或逐个检查，先保证覆盖全部可能答案。
瓶颈定位	瓶颈是连通性查询和合并交替出现。若只需要知道是否同组，不需要路径本身，并查集正好匹配。
结构选择	路径压缩把查找路径上的节点直接挂到根，按大小或按秩合并控制树高。邮箱、等式、边、网格都可以映射成元素集合。
不变量	每个集合有一个代表元， <code>parent[root] == root</code> 。合并只改变一个根的父亲，非根节点的父亲可能不是最终根，但 <code>find</code> 会返回正确代表。
代码落地	类成员访问使用 <code>this-></code> 。迭代版 <code>find</code> 可以避免递归。若需要维护集合大小、边数、权值差或奇偶关系，要扩展额外数组并在合并时同步更新。

547 省份数量

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。城市连接关系形成无向图，省份就是连通分量。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“并查集与动态连通性”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力做法每加一条边就重新 DFS 判断连通，或每次查询都扫描整个分量。这在动态合并场景下会重复处理大量已经稳定的连接关系。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。瓶颈是连通性查询和合并交替出现。若只需要知道是否同组，不需要路径本身，并查集正好匹配。放到本题里，要重点观察“并查集合并所有直接连接的城市，最后统计根数量。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。路径压缩把查找路径上的节点直接挂到根，按大小或按秩合并控制树高。邮箱、等式、边、网格都可以映射成元素集合。本题的优化应当围绕“并查集合并所有直接连接的城市，最后统计根数量。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。每个集合有一个代表元，`parent[root] == root`。合并只改变一个根的父亲，非根节点的父亲可能不是最终根，但 `find` 会返回正确代表。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。正确性来自等价关系维护：自反、对称、传递由集合代表保证。合并两个代表后，两个集合中任意元素的代表都会归到同一根。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。类成员访问使用 `this->`。迭代版 `find` 可以避免递归。若需要维护集合大小、边数、权值差或奇偶关系，要扩展额外数组并在合并时同步更新。本题还要特别注意：邻接矩阵对称、自连接、孤立城市、只扫描上三角。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：邻接矩阵对称、自连接、孤立城市、只扫描上三角。BFS 也可做；并查集更突出动态合并。实验包括重复合并、已经连通的边、孤立节点、字符串编号映射、带权比例冲突、按大小合并前后树

高变化。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

684 冗余连接

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。树加一条边会形成唯一环。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“并查集与动态连通性”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力做法每加一条边就重新 DFS 判断连通，或每次查询都扫描整个分量。这在动态合并场景下会重复处理大量已经稳定的连接关系。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。瓶颈是连通性查询和合并交替出现。若只需要知道是否同组，不需要路径本身，并查集正好匹配。放到本题里，要重点观察“按顺序加边，若一条边两端已连通，则它就是冗余边。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。路径压缩把查找路径上的节点直接挂到根，按大小或按秩合并控制树高。邮箱、等式、边、网格都可以映射成元素集合。本题的优化应当围绕“按顺序加边，若一条边两端已连通，则它就是冗余边。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。每个集合有一个代表元，`parent[root] == root`。合并只改变一个根的父亲，非根节点的父亲可能不是最终根，但 `find` 会返回正确代表。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。正确性来自等价关系维护：自反、对称、传递由集合代表保证。合并两个代表后，两个集合中任意元素的代表都会归到同一根。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。类成员访问使用 `this->`。迭代版 `find` 可以避免递归。若需要维护集合大小、边数、权值差或奇偶关系，要扩展额外数组并在合并时同步更新。本题还要特别注意：节点编号、最后一条成环边、重复边、并查集初始化大小。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：节点编号、最后一条成环边、重复边、并查集初始化大小。有向冗余连接需要同时处理入度为二和环，复杂度更高。实验包括重复合并、已经连通的边、孤立节点、字符串编号映射、带权比例冲突、按大小合并前后树高变化。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

图搜索、最短路和状态建模

图题不是看到网格就写 DFS，也不是看到最短就写 BFS。第一步永远是建模：节点是什么，边是什么，边权是什么，状态是否还包含钥匙、剩余障碍次数、访问集合或时间。岛屿数量的节点是陆地格子，边是四方向相邻；课程表的节点是课程，边是先修依赖；开锁问题的节点是四位密码，边是转动一位；最小体力路径的节点是格子，边权是高度差，但路径代价是最大边权。

BFS 的正确性来自层次扩展。无权图中，从起点出发，队列先处理距离为零的状态，再处理距离为一的状态，然后是距离为二。只要访问标记在入队时设置，一个状态第一次入队时的距离就是最短距离。若等到出队才标记，同一状态可能被同层多个前驱重复加入，虽然有时答案仍对，复杂

度却会膨胀。多源 BFS 只是把所有源点同时放进距离为零的层。

DFS 更适合连通块、路径搜索和状态检测。连通块题中，一次 DFS 或显式栈搜索会标记从起点可达的全部节点；拓扑判环中，三色标记能区分未访问、当前路径中、已完成；回溯搜索中，DFS 路径状态可以在进入和退出时维护。递归写法短，但工程上要认识深度风险，尤其是网格很大或树退化成链时。

拓扑排序是有向图依赖关系的核心工具。入度为零的节点表示当前没有未满足依赖，可以安全输出；输出它以后，只会减少它指向的后继入度。若最终输出节点数少于总数，剩余节点一定在环中或被环依赖。边方向必须讲清楚：课程表中，先修课指向后续课程，还是课程指向先修课，决定输出顺序是否正确。

Dijkstra 适用于非负边权。它维护当前已知最短距离，每次从堆中取出距离最小的未过期状态。非负边保证从这个状态再走出去不会产生一条更短路径回头修改它。C++ 的 `priority_queue` 不支持 `decrease-key`，所以常用做法是更新距离时压入新状态，弹出时若距离不是当前 `distance` 数组中的值，就说明它过期，直接跳过。

有些题看起来不像普通最短路，但仍满足 Dijkstra 的单调扩展。最小体力路径中，路径代价不是边权和，而是路径上最大高度差；扩展一条边后，新代价是当前代价和边代价的最大值。这个值不会随着路径延长而降低，因此仍可用堆按当前最小代价扩展。另一种解法是二分体力阈值，再用 BFS 检查是否可达，因为阈值越大，可走边越多。

状态压缩 BFS 要特别小心 `visited` 的维度。位置相同但钥匙集合不同，未来能力不同；位置相同但剩余障碍消除次数不同，未来能力也不同。此时 `visited` 不能只按位置记录，而要按照位置加资源状态记录。很多看似超时或错误的 BFS，根本原因是把不同状态错误合并，导致要么漏解，要么重复搜索。

实验建议：把腐烂橘子写成每分钟全图扫描和多源 BFS 两版，比较重复扫描次数。把网络延迟时间写成普通 BFS 和 Dijkstra 两版，用不同权重反例证明 BFS 错误。再把带钥匙迷宫的 `visited` 从二维改成三维，观察二维 `visited` 如何错误剪掉合法路径。

本节复盘

阅读本节后，请回到相邻章节的例题，把暴力瓶颈、优化依据、状态不变量、C++ 容器成本和边界测试重新写一遍。能把同一思想迁移到变体题，才算真正掌握。

回溯、枚举和搜索空间剪枝

回溯题表面上是在“试所有可能”，本质上是在一棵决策树上搜索。每一层做一个选择，每条从根到叶子的路径代表一个候选答案。真正要学会的不是某个递归模板，而是四个问题：当前层决定什么，已经选择了什么，下一步还能选什么，什么时候可以停止或剪枝。只要这四件事说不清，代码再短也只是碰巧通过样例。

暴力枚举和回溯的边界并不绝对。最朴素的暴力会把所有组合全部生成出来，再逐个检查是否合法；回溯则把“合法性判断”提前到生成过程中。一旦发现当前前缀已经不可能形成答案，就不再向下扩展。这个动作叫剪枝。剪枝不是为了让代码看起来高级，而是减少决策树里根本不需要访问的分支。

C++面试题常用递归表达回溯，因为调用栈天然保存了“走到第几层、下一步尝试哪个分支、回退后恢复什么状态”。但递归深度不受控时会带来工程风险。本章讲递归心智模型，代码尽量把栈帧显式写出来，让搜索过程、状态恢复和边界条件更可见。这样写会比递归略长，但它能训练你看清楚回溯真正维护的状态。

全排列是回溯的第一道标准题。题目给定互不相同的数字，要求返回所有排列。暴力想法是枚举长度为 n 的所有序列，再检查是否每个数字恰好使用一次。若把每个位置都看成有 n 种选择，总分支数是 n^n ，绝大多数序列会因为重复使用元素而被丢弃。优化方法是在生成过程中维护 `used` 数组，保证同一个下标不会被选两次。这样搜索树只保留真正的排列分支，叶子数变成 $n!$ 。

下面的实现用显式栈模拟递归。每个栈帧保存当前层接下来要尝试的候选下标 `next_index`。进入下一层时，把选择加入 `path` 并标记 `used`；一层候选全部尝试完时，弹出栈帧，并撤销上一层做出的选择。

Listing 8.1 全排列：显式栈模拟回溯

```

1 std::vector<std::vector<int>> permute(const std::vector<int>& nums)
2 {
3     struct Frame {
4         int next_index = 0;
5     };
6
7     const int n = static_cast<int>(nums.size());
8     std::vector<std::vector<int>> answer;
9     std::vector<int> path;
10    std::vector<char> used(n, false);
11    std::vector<Frame> stack{{0}};
12
13    while (!stack.empty()) {
14        if (static_cast<int>(path.size()) == n) {
15            answer.push_back(path);
16            stack.pop_back();
17            if (!path.empty()) {
18                const int last_value = path.back();
19                path.pop_back();
20                const auto found = std::find(nums.begin(), nums.end(), last_value);
21                used[static_cast<int>(found - nums.begin())] = false;
22            }
23            continue;
24        }

```

```

25
26     Frame& frame = stack.back();
27     while (frame.next_index < n && used[frame.next_index]) {
28         ++frame.next_index;
29     }
30     if (frame.next_index == n) {
31         stack.pop_back();
32         if (!path.empty()) {
33             const int last_value = path.back();
34             path.pop_back();
35             const auto found = std::find(nums.begin(), nums.end(), last_value);
36             used[static_cast<int>(found - nums.begin())] = false;
37         }
38         continue;
39     }
40
41     const int chosen_index = frame.next_index;
42     ++frame.next_index;
43     used[chosen_index] = true;
44     path.push_back(nums[chosen_index]);
45     stack.push_back(Frame{0});
46 }
47
48 return answer;
49 }

```

这段代码为了突出栈帧，撤销选择时用值反查下标；在生产代码里可以把“本层选择的下标”也放进下一层栈帧，避免反查。它的时间复杂度必须按输出规模理解：一共有 $n!$ 个排列，每个排列复制长度为 n 的路径，所以输出成本是 $O(n \cdot n!)$ 。额外空间是路径、标记和栈，都是 $O(n)$ ，不计答案。复盘这题时要能说清楚：`used` 是路径合法性的约束，`path.size() == n` 是叶子条件，回退必须撤销最后一次选择。

如果输入含重复数字，全排列 II 的难点就变成去重。很多人会先生成所有排列，再放进 `set` 去重；这当然能做，但浪费大量重复分支。更好的方法是先排序，然后在同一层跳过“相同数值且前一个相同值还没有被本路径使用”的候选。这个条件的含义是：同一层里，相同值只允许最左边那个代表这一类选择，避免生成形如交换两个相同值位置的重复排列。

Listing 8.2 全排列 II：排序后按层去重

```

1 std::vector<std::vector<int>> permute_unique(std::vector<int> nums)
2 {
3     std::sort(nums.begin(), nums.end());
4
5     const int n = static_cast<int>(nums.size());
6     std::vector<std::vector<int>> answer;
7     std::vector<int> path;
8     std::vector<char> used(n, false);
9
10    struct State {
11        std::vector<int> path;
12        std::vector<char> used;
13    };
14
15    std::vector<State> stack{{path, used}};

```



```

16 while (!stack.empty()) {
17     State state = std::move(stack.back());
18     stack.pop_back();
19
20     if (static_cast<int>(state.path.size()) == n) {
21         answer.push_back(std::move(state.path));
22         continue;
23     }
24
25     for (int i = n - 1; i >= 0; --i) {
26         if (state.used[i]) {
27             continue;
28         }
29         if (i > 0 && nums[i] == nums[i - 1] && !state.used[i - 1]) {
30             continue;
31         }
32
33         State next = state;
34         next.used[i] = true;
35         next.path.push_back(nums[i]);
36         stack.push_back(std::move(next));
37     }
38 }
39
40 return answer;
41 }

```

这个版本为了让去重条件清楚，直接把状态复制进栈，适合教材解释；高性能版本会用一份路径和标记做原地回退。去重条件最容易写反。排序后，相同数字挨在一起，`!state.used[i - 1]`表示前一个相同数字在当前层还没有被拿来作为代表，此时选择后一个会制造重复分支，所以跳过。时间复杂度仍然与唯一排列数量成正比，最坏不超过 $O(n \cdot n!)$ 。

子集题的决策树更简单。每个元素只有两种选择：选或者不选。暴力枚举可以用二进制掩码，从 0 到 $2^n - 1$ ，第 i 位表示是否选择第 i 个元素。回溯写法则是沿着数组位置向后决定。两种方法本质相同，都是遍历大小为 2^n 的搜索空间。

Listing 8.3 子集：迭代扩展已有答案

```

1 std::vector<std::vector<int>> subsets(const std::vector<int>& nums)
2 {
3     std::vector<std::vector<int>> answer{{}};
4
5     for (const int x : nums) {
6         const int old_size = static_cast<int>(answer.size());
7         for (int i = 0; i < old_size; ++i) {
8             std::vector<int> next = answer[i];
9             next.push_back(x);
10            answer.push_back(std::move(next));
11        }
12    }
13
14    return answer;
15 }

```

这段代码的不变量是：处理完前 i 个元素后，`answer` 包含这些元素的全部子集。加入新元素 x 时，旧子集保持“不选 x ”的版本；每个旧子集复制一份并加上 x ，得到“选 x ”的版本。这个算法没有剪枝，因为所有子集都是合法答案。时间复杂度 $O(n2^n)$ ，空间复杂度同样由输出规模主导。

组合总和比子集多了一个目标约束。题目给定候选数组和目标值，要求找出所有和为目标值的组合，同一个数字可以重复使用。暴力思路是不断选数字，直到长度达到某个上限再检查和；问题是上限不好定，而且大量分支会在总和已经超过目标后继续扩展。优化来自两个事实：候选数字都是正数；排序后，如果当前候选已经大于剩余目标，后面的更大候选也不可能使用。

Listing 8.4 组合总和：显式状态保存起点和剩余目标

```

1 std::vector<std::vector<int>> combination_sum(std::vector<int> candidates,
2                                             int target)
3 {
4     std::sort(candidates.begin(), candidates.end());
5
6     struct State {
7         int start = 0;
8         int remaining = 0;
9         std::vector<int> path;
10    };
11
12    std::vector<std::vector<int>> answer;
13    std::vector<State> stack{{0, target, {}}};
14
15    while (!stack.empty()) {
16        State state = std::move(stack.back());
17        stack.pop_back();
18
19        if (state.remaining == 0) {
20            answer.push_back(std::move(state.path));
21            continue;
22        }
23
24        for (int i = static_cast<int>(candidates.size()) - 1;
25             i >= state.start;
26             --i) {
27            if (candidates[i] > state.remaining) {
28                continue;
29            }
30            State next = state;
31            next.path.push_back(candidates[i]);
32            next.remaining -= candidates[i];
33            next.start = i;
34            stack.push_back(std::move(next));
35        }
36    }
37
38    return answer;
39 }

```

`start` 的语义很重要：组合不关心顺序，`[2,3,2]` 和 `[2,2,3]` 是同一个答案，因此后续只能从当前下标及其右侧继续选，避免同一组合以不同顺序重复出现。因为允许重复使用当前数字，下一层的 `start` 仍然是 `i`；如果题目变成每个数字只能用一次，就应设为 `i + 1`。这就是组合类题最

核心的变体开关。复杂度取决于目标值和候选分布，不能简单写成多项式；面试里应说明它是指数级搜索，但剪枝能显著减少无效分支。

括号生成的关键不是枚举所有长度为 $2n$ 的括号串再检查。那样有 2^{2n} 个候选，大多数不合法。合法前缀必须满足两个计数约束：左括号数量不能超过 n ；任何前缀中右括号数量不能超过左括号数量。把这两个约束提前放进生成过程，就不会产生明显非法的前缀。

Listing 8.5 括号生成：前缀合法性剪枝

```

1 std::vector<std::string> generate_parenthesis(int n)
2 {
3     struct State {
4         std::string text;
5         int left = 0;
6         int right = 0;
7     };
8
9     std::vector<std::string> answer;
10    std::vector<State> stack{{"", 0, 0}};
11
12    while (!stack.empty()) {
13        State state = std::move(stack.back());
14        stack.pop_back();
15
16        if (static_cast<int>(state.text.size()) == 2 * n) {
17            answer.push_back(std::move(state.text));
18            continue;
19        }
20
21        if (state.right < state.left) {
22            State next = state;
23            next.text.push_back(')');
24            ++next.right;
25            stack.push_back(std::move(next));
26        }
27        if (state.left < n) {
28            State next = state;
29            next.text.push_back('(');
30            ++next.left;
31            stack.push_back(std::move(next));
32        }
33    }
34
35    return answer;
36 }

```

这题的正确性可以用前缀不变量解释：任何时刻 $\text{left} \leq n$ 且 $\text{right} \leq \text{left}$ 。如果 $\text{right} > \text{left}$ ，说明某个右括号没有左括号匹配，后面再加字符也无法修复这个非法前缀；如果 $\text{left} > n$ ，也已经超过题目允许。输出数量是第 n 个卡特兰数，算法复杂度至少与答案数量成正比。易错点是终止条件：必须长度达到 $2n$ 才能加入答案，只看 $\text{left} == \text{right}$ 会把短前缀误加进去。

单词搜索把回溯放进网格图。节点是格子，边是上下左右相邻，路径不能重复使用同一个格子。暴力想法是从每个格子出发尝试所有方向，最后检查路径字符串；优化点是边走边比较目标单词的当前位置，一旦字符不匹配立即停止。访问标记也必须属于当前路径，而不是全局永久标记，因为同一个格子可以出现在不同起点或不同路径中。

Listing 8.6 单词搜索：显式栈保存路径访问状态

```

1  bool exist(std::vector<std::vector<char>>& board, const std::string& word)
2  {
3      if (word.empty()) {
4          return true;
5      }
6      if (board.empty() || board[0].empty()) {
7          return false;
8      }
9
10     constexpr std::array<std::pair<int, int>, 4> DIRECTIONS{
11         std::pair{1, 0}, std::pair{-1, 0},
12         std::pair{0, 1}, std::pair{0, -1}
13     };
14
15     const int rows = static_cast<int>(board.size());
16     const int cols = static_cast<int>(board[0].size());
17
18     struct State {
19         int row = 0;
20         int col = 0;
21         int index = 0;
22         std::vector<std::vector<char>> visited;
23     };
24
25     for (int row = 0; row < rows; ++row) {
26         for (int col = 0; col < cols; ++col) {
27             if (board[row][col] != word[0]) {
28                 continue;
29             }
30
31             std::vector<std::vector<char>> visited(rows, std::vector<char>(cols, ↵
↵ false));
32             visited[row][col] = true;
33             std::vector<State> stack{{row, col, 0, std::move(visited)}};
34
35             while (!stack.empty()) {
36                 State state = std::move(stack.back());
37                 stack.pop_back();
38                 if (state.index + 1 == static_cast<int>(word.size())) {
39                     return true;
40                 }
41
42                 for (const auto [dr, dc] : DIRECTIONS) {
43                     const int next_row = state.row + dr;
44                     const int next_col = state.col + dc;
45                     const int next_index = state.index + 1;
46                     const bool out_of_bounds =
47                         next_row < 0 || next_row >= rows ||
48                         next_col < 0 || next_col >= cols;
49                     if (out_of_bounds || state.visited[next_row][next_col] ||
50                         board[next_row][next_col] != word[next_index]) {
51                         continue;

```



```

52         }
53
54         State next = state;
55         next.row = next_row;
56         next.col = next_col;
57         next.index = next_index;
58         next.visited[next_row][next_col] = true;
59         stack.push_back(std::move(next));
60     }
61 }
62 }
63 }
64
65 return false;
66 }

```

这个教材版把 `visited` 放进状态里，语义最清楚，但会复制较多布尔矩阵；原地回退版性能更好。本题最坏复杂度接近 $O(mn \cdot 3^L)$ ：第一步最多从每个格子出发，之后每一步通常不能回到上一个格子，所以分支近似 3， L 是单词长度。常见错误是把访问标记设成全局，导致某个格子被一条失败路径占用后，其他合法路径无法使用它。

N 皇后是回溯剪枝的代表。暴力方法在 $n \times n$ 棋盘上选 n 个格子，再检查行、列、对角线是否冲突，搜索空间巨大。优化方式是按行放皇后：第 `row` 层只决定第 `row` 行皇后放在哪一列。这样行冲突天然消失，只需要维护列、主对角线和副对角线是否被占用。

Listing 8.7 N 皇后：按行放置并维护三类占用

```

1 std::vector<std::vector<std::string>> solve_n_queens(int n)
2 {
3     struct State {
4         int row = 0;
5         std::vector<int> queens;
6         std::vector<char> cols;
7         std::vector<char> diag1;
8         std::vector<char> diag2;
9     };
10
11     std::vector<std::vector<std::string>> answer;
12     std::vector<State> stack{{0,
13         {},
14         std::vector<char>(n, false),
15         std::vector<char>(2 * n - 1, false),
16         std::vector<char>(2 * n - 1, false)}};
17
18     while (!stack.empty()) {
19         State state = std::move(stack.back());
20         stack.pop_back();
21
22         if (state.row == n) {
23             std::vector<std::string> board(n, std::string(n, '.'));
24             for (int row = 0; row < n; ++row) {
25                 board[row][state.queens[row]] = 'Q';
26             }
27             answer.push_back(std::move(board));

```

```

28         continue;
29     }
30
31     for (int col = n - 1; col >= 0; --col) {
32         const int main_diag = state.row - col + n - 1;
33         const int anti_diag = state.row + col;
34         if (state.cols[col] || state.diag1[main_diag] ||
35             state.diag2[anti_diag]) {
36             continue;
37         }
38
39         State next = state;
40         next.queens.push_back(col);
41         next.cols[col] = true;
42         next.diag1[main_diag] = true;
43         next.diag2[anti_diag] = true;
44         ++next.row;
45         stack.push_back(std::move(next));
46     }
47 }
48
49 return answer;
50 }

```

主对角线上的格子满足 $\text{row} - \text{col}$ 相同；为了把负数下标变成数组下标，代码加上 $n - 1$ 。副对角线满足 $\text{row} + \text{col}$ 相同，范围天然是从 0 到 $2n - 2$ 。这题的面试重点不是把棋盘画出来，而是把冲突检测从扫描整张棋盘优化成三个数组的常数查询。复杂度仍然是指数级，但剪枝把不可行分支尽早挡在下一层之外。

回溯去重可以分成两类：路径内去重和同层去重。路径内去重解决“同一个元素不能被重复使用”，典型工具是 `used` 数组；同层去重解决“相同值作为同一层选择会生成重复答案”，典型工具是排序后跳过相邻重复值。全排列 II 同时需要这两类去重，组合总和 II 主要需要同层去重。只要把这两个概念混在一起，条件就很容易写反。

组合总和 II 中，每个候选数字只能使用一次，输入可能有重复值，答案组合不能重复。与组合总和 I 相比，下一层起点必须是 $i + 1$ ，因为当前下标不能再次使用；与全排列 II 相比，我们不需要 `used` 数组，因为 `start` 已经保证每个下标最多被使用一次。同层去重条件是：在同一个循环层里，如果当前值和前一个值相同，并且前一个值已经作为本层候选被跳过或处理过，那么当前值也不能再作为新的分支开头。

Listing 8.8 组合总和 II：每个下标最多使用一次，同层跳过重复值

```

1 std::vector<std::vector<int>> combination_sum2(std::vector<int> candidates,
2                                              int target)
3 {
4     std::sort(candidates.begin(), candidates.end());
5
6     struct State {
7         int start = 0;
8         int remaining = 0;
9         std::vector<int> path;
10    };
11
12    std::vector<std::vector<int>> answer;
13    std::vector<State> stack{{0, target, {}}};

```

```

14
15 while (!stack.empty()) {
16     State state = std::move(stack.back());
17     stack.pop_back();
18
19     if (state.remaining == 0) {
20         answer.push_back(std::move(state.path));
21         continue;
22     }
23
24     for (int i = static_cast<int>(candidates.size()) - 1;
25          i >= state.start;
26          --i) {
27         if (i + 1 < static_cast<int>(candidates.size()) &&
28             candidates[i] == candidates[i + 1] &&
29             i + 1 >= state.start) {
30             continue;
31         }
32         if (candidates[i] > state.remaining) {
33             continue;
34         }
35
36         State next = state;
37         next.path.push_back(candidates[i]);
38         next.remaining -= candidates[i];
39         next.start = i + 1;
40         stack.push_back(std::move(next));
41     }
42 }
43
44 return answer;
45 }

```

这段显式栈代码为了保持输出方向，用倒序枚举候选；倒序时同层去重要和相邻的后一个元素比较。递归版本通常正序枚举，条件写成 `i > start && candidates[i] == candidates[i - 1]`。两个条件本质相同：同一个递归层里，相同值只能由一个下标代表。面试中如果不确定倒序条件，先写递归式正序伪代码说明去重，再实现自己更熟悉的版本。最重要的是讲清楚：同层重复会制造相同组合，路径重复则是同一个下标重复使用，这两个问题不能用同一个条件糊过去。

字符串切分题也属于回溯。分割回文串要求把字符串切成若干段，每一段都是回文。暴力做法是枚举所有切分方式，再逐段检查是否回文；如果每次检查都扫描子串，会产生大量重复判断。优化方法是先用 DP 预处理 `is_palindrome[left][right]`，表示闭区间子串是否回文。之后回溯只需要常数时间判断某一段是否合法。

Listing 8.9 分割回文串：预处理回文表后搜索切分点

```

1 std::vector<std::vector<std::string>> partition_palindrome(const std::string& s)
2 {
3     const int n = static_cast<int>(s.size());
4     std::vector<std::vector<char>> is_palindrome(n, std::vector<char>(n, false));
5     for (int length = 1; length <= n; ++length) {
6         for (int left = 0; left + length - 1 < n; ++left) {
7             const int right = left + length - 1;
8             if (s[left] != s[right]) {

```

```

9         continue;
10    }
11    is_palindrome[left][right] =
12        length <= 2 || is_palindrome[left + 1][right - 1];
13    }
14    }
15
16    struct State {
17        int start = 0;
18        std::vector<std::string> parts;
19    };
20
21    std::vector<std::vector<std::string>> answer;
22    std::vector<State> stack{{0, {}}};
23    while (!stack.empty()) {
24        State state = std::move(stack.back());
25        stack.pop_back();
26        if (state.start == n) {
27            answer.push_back(std::move(state.parts));
28            continue;
29        }
30
31        for (int end = n - 1; end >= state.start; --end) {
32            if (!is_palindrome[state.start][end]) {
33                continue;
34            }
35            State next = state;
36            next.parts.push_back(s.substr(state.start, end - state.start + 1));
37            next.start = end + 1;
38            stack.push_back(std::move(next));
39        }
40    }
41    return answer;
42 }

```

这题的决策树层数不是固定的 n ，而是取决于切了多少段。每个状态的 `start` 表示下一个待切分位置，选择是从 `start` 开始切到哪个 `end`。预处理回文表的转移也很经典：两端字符相等，且内部子串是回文，整个子串才是回文；长度为 1 或 2 时内部为空或单字符，可以直接成立。搜索复杂度仍然可能指数级，因为合法切分数量本身可能很多，但预处理避免了在搜索树中反复扫描同一段字符串。

复原 IP 地址是切分题的约束版。字符串要切成四段，每段长度 1 到 3，数值 0 到 255，且不能有前导零。暴力枚举三个切分点就能做，因为段数固定；也可以用回溯统一表达。剪枝点很明确：剩余字符太少或太多时停止；当前段长度超过 3 停止；以 0 开头的多位段非法；数值超过 255 非法。

Listing 8.10 复原 IP 地址：段数和剩余长度剪枝

```

1 std::vector<std::string> restore_ip_addresses(const std::string& s)
2 {
3     constexpr int SEGMENT_COUNT = 4;
4     constexpr int MAX_SEGMENT_VALUE = 255;
5     std::vector<std::string> answer;
6
7     struct State {

```



```

8      int index = 0;
9      std::vector<std::string> segments;
10     };
11
12     std::vector<State> stack{{0, {}}};
13     while (!stack.empty()) {
14         State state = std::move(stack.back());
15         stack.pop_back();
16
17         const int used_segments = static_cast<int>(state.segments.size());
18         const int remaining_segments = SEGMENT_COUNT - used_segments;
19         const int remaining_chars = static_cast<int>(s.size()) - state.index;
20         if (remaining_chars < remaining_segments ||
21             remaining_chars > remaining_segments * 3) {
22             continue;
23         }
24
25         if (used_segments == SEGMENT_COUNT) {
26             if (state.index == static_cast<int>(s.size())) {
27                 answer.push_back(state.segments[0] + "." + state.segments[1] +
28                                 "." + state.segments[2] + "." +
29                                 state.segments[3]);
30             }
31             continue;
32         }
33
34         int value = 0;
35         for (int length = 1; length <= 3; ++length) {
36             if (state.index + length > static_cast<int>(s.size())) {
37                 break;
38             }
39             if (length > 1 && s[state.index] == '0') {
40                 break;
41             }
42             value = value * 10 + (s[state.index + length - 1] - '0');
43             if (value > MAX_SEGMENT_VALUE) {
44                 break;
45             }
46
47             State next = state;
48             next.segments.push_back(s.substr(state.index, length));
49             next.index += length;
50             stack.push_back(std::move(next));
51         }
52     }
53
54     return answer;
55 }

```

这题适合训练“剩余量剪枝”。如果剩余字符数少于剩余段数，每段至少一个字符都不够；如果剩余字符数大于剩余段数乘 3，每段最多三个字符也放不下。这种剪枝不依赖数据运气，是严格必要条件，因此不会漏解。很多组合题都可以做类似估算：剩余元素数量是否足够，剩余目标是否可能达到，剩余步数是否还能完成。

回溯题也可以用位掩码表示集合。子集枚举、访问状态、N 皇后列占用都可以压成整数位。位掩码的优势是状态复制便宜、哈希和比较方便；缺点是可读性下降，并且 n 不能太大。对于 $n \leq 20$ 的集合问题，位掩码常常是从回溯过渡到状态压缩 DP 的桥梁。

Listing 8.11 位掩码枚举子集：每一位表示选或不选

```
1 std::vector<std::vector<int>> subsets_by_mask(const std::vector<int>& nums)
2 {
3     const int n = static_cast<int>(nums.size());
4     const int state_count = 1 << n;
5     std::vector<std::vector<int>> answer;
6     answer.reserve(state_count);
7
8     for (int mask = 0; mask < state_count; ++mask) {
9         std::vector<int> subset;
10        for (int i = 0; i < n; ++i) {
11            if ((mask & (1 << i)) != 0) {
12                subset.push_back(nums[i]);
13            }
14        }
15        answer.push_back(std::move(subset));
16    }
17
18    return answer;
19 }
```

位掩码写法的复杂度是 $O(n2^n)$ ，因为每个集合要检查 n 位。它没有递归栈，也不需要回退，适合子集这种所有状态都合法的题；如果有复杂剪枝，例如组合总和或单词搜索，回溯更自然。选择表达方式时要看题目结构，不要为了使用位运算牺牲清晰性。

剪枝必须证明不会漏解。常见剪枝有四类。第一，合法性剪枝：括号前缀右括号不能超过左括号。第二，容量剪枝：组合总和中当前和超过目标立即停止。第三，剩余量剪枝：IP 地址剩余字符数量必须落在可行范围。第四，排序剪枝：候选已排序时，当前值超过剩余目标，后续更大值也不可能使用。没有证明的剪枝很危险，比如“看起来这个分支不太可能有答案”不能作为正确算法的一部分。

回溯实验：第一，把组合总和 II 的同层去重条件删除，观察重复答案如何出现。第二，把复原 IP 地址的剩余量剪枝删除，统计访问状态数量变化。第三，把分割回文串的回文预处理去掉，改成每次扫描判断，比较长字符串上的耗时。第四，用位掩码和迭代扩展两种方式生成子集，比较输出顺序和代码复杂度。

回溯题复盘模板

回溯题不要先背代码。先画出决策树的一层，写清本层选择、路径状态、可选集合、终止条件和剪枝条件。组合题重点看 **start** 是否允许重复使用当前元素；排列题重点看 **used** 和同层去重；网格题重点看访问标记是路径级还是全局级；括号和棋盘题重点把非法前缀提前剪掉。最后再问一句：答案数量本身有多大？如果答案就是指数级，复杂度不可能被写成线性或多项式。

第 9 章

动态规划：状态、转移和重复子问题

动态规划不是“看到最值就开数组”。它解决的是有重叠子问题、且大问题能由小问题稳定推出的问题。暴力搜索之所以慢，是因为同一个子问题被不同路径反复计算；动态规划的优化，就是给子问题命名、保存结果，并按照依赖顺序把结果推出来。学 DP 最重要的五件事是：状态表示什么，答案在哪里，初始状态是什么，转移从哪里来，遍历顺序为什么满足依赖。

爬楼梯是最小的 DP 模型。题目问到达第 n 阶有多少种方法，每次可以爬 1 或 2 阶。暴力递归会写成 $\text{ways}(n) = \text{ways}(n - 1) + \text{ways}(n - 2)$ ，但 $\text{ways}(n - 2)$ 、 $\text{ways}(n - 3)$ 等子问题会被反复计算，形成指数级时间。优化方法是从小到大保存每个阶数的答案。

Listing 9.1 爬楼梯：滚动变量保存最近两个状态

```
1 int climb_stairs(int n)
2 {
3     if (n <= 2) {
4         return n;
5     }
6
7     int previous_two = 1;
8     int previous_one = 2;
9     for (int step = 3; step <= n; ++step) {
10         const int current = previous_one + previous_two;
11         previous_two = previous_one;
12         previous_one = current;
13     }
14
15     return previous_one;
16 }
```

这里状态可以写成 $\text{dp}[i]$ ：到达第 i 阶的方法数。最后一步要么从 $i - 1$ 走 1 阶，要么从 $i - 2$ 走 2 阶，因此转移是 $\text{dp}[i] = \text{dp}[i - 1] + \text{dp}[i - 2]$ 。由于每个状态只依赖前两个状态，可以把数组压成两个变量。时间复杂度 $O(n)$ ，空间复杂度 $O(1)$ 。这题的价值不在难度，而在于展示 DP 的完整骨架。

打家劫舍把“相邻不能同时选”的约束放进状态。暴力方法是对每间房决定偷或不偷，枚举 2^n 个子集，再过滤相邻冲突。更好的建模是让 $\text{dp}[i]$ 表示考虑前 i 间房能偷到的最大金额。第 i 间房只有两种结局：不偷它，答案是 $\text{dp}[i - 1]$ ；偷它，就不能偷第 $i - 1$ 间，答案是 $\text{dp}[i - 2] + \text{nums}[i - 1]$ 。

Listing 9.2 打家劫舍：选与不选的状态转移

```
1 int rob(const std::vector<int>& nums)
2 {
3     int previous_two = 0;
4     int previous_one = 0;
5
6     for (const int money : nums) {
7         const int current = std::max(previous_one, previous_two + money);
8         previous_two = previous_one;
9         previous_one = current;
10     }
```

```

10     }
11
12     return previous_one;
13 }

```

这段代码把 `dp[i - 2]` 和 `dp[i - 1]` 压成两个变量。遍历到当前房屋时, `previous_one` 是不偷当前房屋时可继承的最优值, `previous_two + money` 是偷当前房屋时的最优值。正确性来自最优子结构: 当前决策之后, 剩余可用的历史范围只与前一间或前两间有关, 不需要知道更早的具体选择。易错点是下标: 如果用数组, 建议定义 `dp[0] = 0`, `dp[1] = nums[0]`, 这样第 `i` 个状态表示前 `i` 间房, 避免把房屋下标和状态下标混在一起。

零钱兑换展示“最少数量”型 DP。题目给硬币面额和目标金额, 要求凑成目标金额所需的最少硬币数。暴力搜索会尝试所有硬币序列, 许多不同顺序会到达同一个剩余金额, 例如先选 1 再选 2, 与先选 2 再选 1, 都可能进入同一个子问题。状态应定义为 `dp[amount]`: 凑出金额 `amount` 的最少硬币数。

Listing 9.3 零钱兑换: 从可达金额推出新金额

```

1 int coin_change(const std::vector<int>& coins, int amount)
2 {
3     constexpr int INF = 1'000'000'000;
4     std::vector<int> dp(amount + 1, INF);
5     dp[0] = 0;
6
7     for (int value = 1; value <= amount; ++value) {
8         for (const int coin : coins) {
9             if (coin > value || dp[value - coin] == INF) {
10                 continue;
11             }
12             dp[value] = std::min(dp[value], dp[value - coin] + 1);
13         }
14     }
15
16     return dp[amount] == INF ? -1 : dp[amount];
17 }

```

转移的含义是: 如果最后一枚硬币是 `coin`, 那么前面必须先凑出 `value - coin`, 所以候选答案是 `dp[value - coin] + 1`。这里用一个足够大的 `INF` 表示不可达, 而不是用 `-1` 参与最小值计算, 因为 `-1 + 1` 会污染转移。时间复杂度 $O(\text{amount} \cdot c)$, `c` 是硬币种类数。完全背包类题还要区分“组合数”和“排列数”: 本题只问最少硬币数, 内外循环顺序不会影响最优值; 如果问方案数量, 循环顺序就会改变计数语义。

最长递增子序列有两种经典解法。第一种 DP 定义 `dp[i]` 为以 `nums[i]` 结尾的最长递增子序列长度。为了求它, 需要看所有 `j < i` 且 `nums[j] < nums[i]` 的位置, 转移为 `dp[i] = max(dp[j] + 1)`。这比枚举所有子序列的 2^n 好很多, 但仍是 $O(n^2)$ 。

Listing 9.4 最长递增子序列: 二次 DP

```

1 int length_of_lis_dp(const std::vector<int>& nums)
2 {
3     if (nums.empty()) {
4         return 0;
5     }
6
7     std::vector<int> dp(nums.size(), 1);

```



```

8     int answer = 1;
9
10    for (int i = 0; i < static_cast<int>(nums.size()); ++i) {
11        for (int j = 0; j < i; ++j) {
12            if (nums[j] < nums[i]) {
13                dp[i] = std::max(dp[i], dp[j] + 1);
14            }
15        }
16        answer = std::max(answer, dp[i]);
17    }
18
19    return answer;
20 }

```

第二种优化来自“只关心长度对应的最小结尾”。维护数组 `tails`，其中 `tails[len - 1]` 表示长度为 `len` 的递增子序列中，最小可能结尾值。结尾越小，后面越容易接上更大的数。遍历新数 `x` 时，用二分找到第一个大于等于 `x` 的位置并替换；如果找不到，说明 `x` 可以扩展出更长序列。

Listing 9.5 最长递增子序列：贪心尾值加二分

```

1 int length_of_lis_binary_search(const std::vector<int>& nums)
2 {
3     std::vector<int> tails;
4
5     for (const int x : nums) {
6         auto position = std::lower_bound(tails.begin(), tails.end(), x);
7         if (position == tails.end()) {
8             tails.push_back(x);
9         } else {
10            *position = x;
11        }
12    }
13
14    return static_cast<int>(tails.size());
15 }

```

这个算法经常被误解成 `tails` 本身就是最终的子序列。它不一定是。它保存的是每个长度下最有潜力的最小结尾。替换操作不会减少已经找到的长度，只会让同长度的结尾更小，从而给未来元素更多机会。时间复杂度 $O(n \log n)$ ，空间复杂度 $O(n)$ 。如果面试官追问为什么能替换，要回答：相同长度下，更小结尾不会比更大结尾更差，因为它能接纳的后续元素集合只会更多。

单词拆分把字符串前缀变成状态。题目给字符串和词典，问字符串能否被切成若干词典单词。暴力搜索会尝试每个切分点，遇到相同后缀时重复判断。状态 `dp[i]` 表示前 `i` 个字符是否可以被合法拆分。若存在 `j < i`，使得 `dp[j]` 为真且子串 `s[j..i)` 在词典中，则 `dp[i]` 为真。

Listing 9.6 单词拆分：前缀可达状态

```

1 bool word_break(const std::string& s, const std::vector<std::string>& word_dict)
2 {
3     std::unordered_set<std::string> words(word_dict.begin(), word_dict.end());
4     std::vector<char> dp(s.size() + 1, false);
5     dp[0] = true;
6
7     for (std::size_t right = 1; right <= s.size(); ++right) {
8         for (std::size_t left = 0; left < right; ++left) {

```

```

9         if (!dp[left]) {
10             continue;
11         }
12         if (words.contains(s.substr(left, right - left))) {
13             dp[right] = true;
14             break;
15         }
16     }
17 }
18
19 return dp[s.size()];
20 }

```

这里 `dp[0] = true` 代表空前缀可拆分，是所有从 0 开始的单词能够成立的基础。代码直接使用 `substr`，写法清晰，但会复制子串；如果输入规模更大，可以限制最大单词长度，或者用字符串视图和自定义哈希、字典树来减少复制。复杂度通常写成 $O(n^3)$ 的保守形式，因为有两层切分枚举，子串构造和哈希也与长度有关；若把子串查询视作均摊常数，则是 $O(n^2)$ 。面试中最好主动说明这个实现的复制成本。

最长公共子序列是二维 DP 的代表。子序列可以删除字符但不能改变相对顺序。暴力方法会枚举两个字符串的所有子序列再比较，数量指数级。状态 `dp[i][j]` 表示 `text1` 的前 `i` 个字符和 `text2` 的前 `j` 个字符的最长公共子序列长度。

Listing 9.7 最长公共子序列：二维前缀 DP

```

1 int longest_common_subsequence(const std::string& text1,
2                               const std::string& text2)
3 {
4     const int rows = static_cast<int>(text1.size());
5     const int cols = static_cast<int>(text2.size());
6     std::vector<std::vector<int>> dp(rows + 1, std::vector<int>(cols + 1, 0));
7
8     for (int i = 1; i <= rows; ++i) {
9         for (int j = 1; j <= cols; ++j) {
10             if (text1[i - 1] == text2[j - 1]) {
11                 dp[i][j] = dp[i - 1][j - 1] + 1;
12             } else {
13                 dp[i][j] = std::max(dp[i - 1][j], dp[i][j - 1]);
14             }
15         }
16     }
17
18     return dp[rows][cols];
19 }

```

为什么相等时来自左上角？因为当前两个字符可以配成公共子序列的最后一个字符，剩下问题变成两个前缀各去掉这个字符。为什么不等时取上方和左方最大值？因为两个末尾字符不能同时作为同一个结尾，至少要丢掉其中一个。多出来的第 0 行和第 0 列代表空串，避免了边界分支。时间复杂度 $O(mn)$ ，空间复杂度 $O(mn)$ ；如果只要长度，可以滚动数组压到 $O(n)$ 。

编辑距离比 LCS 更综合。题目允许插入、删除、替换，把一个单词变成另一个单词，求最少操作数。状态 `dp[i][j]` 表示把 `word1` 的前 `i` 个字符变成 `word2` 的前 `j` 个字符的最少操作数。这个定义必须精确，因为三种操作都围绕“前缀末尾”展开。

Listing 9.8 编辑距离：插入、删除、替换三种转移

```

1 int min_distance(const std::string& word1, const std::string& word2)
2 {
3     const int rows = static_cast<int>(word1.size());
4     const int cols = static_cast<int>(word2.size());
5     std::vector<std::vector<int>> dp(rows + 1, std::vector<int>(cols + 1, 0));
6
7     for (int i = 0; i <= rows; ++i) {
8         dp[i][0] = i;
9     }
10    for (int j = 0; j <= cols; ++j) {
11        dp[0][j] = j;
12    }
13
14    for (int i = 1; i <= rows; ++i) {
15        for (int j = 1; j <= cols; ++j) {
16            if (word1[i - 1] == word2[j - 1]) {
17                dp[i][j] = dp[i - 1][j - 1];
18                continue;
19            }
20
21            const int delete_cost = dp[i - 1][j] + 1;
22            const int insert_cost = dp[i][j - 1] + 1;
23            const int replace_cost = dp[i - 1][j - 1] + 1;
24            dp[i][j] = std::min({delete_cost, insert_cost, replace_cost});
25        }
26    }
27
28    return dp[rows][cols];
29 }

```

初始化含义是：把长度为 i 的前缀变成空串，需要删除 i 次；把空串变成长度为 j 的前缀，需要插入 j 次。若末尾字符相同，不需要新增操作，直接继承左上角；若不同，删除对应上方，插入对应左方，替换对应左上角。时间和空间都是 $O(mn)$ 。易错点是从哪个字符串视角解释插入和删除：只要状态定义固定， $dp[i - 1][j] + 1$ 就是删掉 `word1` 的末尾， $dp[i][j - 1] + 1$ 就是在 `word1` 末尾插入 `word2[j - 1]`。

动态规划题经常可以做空间优化，但不要为了炫技先写滚动数组。二维表能展示依赖关系，滚动数组则要求你非常清楚哪些旧值还要用、哪些可以覆盖。面试时，先给出清晰状态和二维实现，再说明可压缩空间，通常比一上来写难读的一维代码更稳。

从暴力搜索到 DP，通常要经历三步。第一步写出递归含义，例如 `solve(i, cap)` 表示处理到第 i 个物品、剩余容量为 `cap` 时的最优值。第二步观察重复子问题：不同选择路径可能来到同一个 (i, cap) 。第三步把这个状态保存起来，或者按依赖顺序填表。动态规划不是凭空发明公式，而是把搜索树压缩成状态图。

0-1 背包是最重要的状态压缩训练。每个物品只能选一次，状态可以定义为 $dp[i][cap]$ ：只考虑前 i 个物品、容量为 `cap` 时的最大价值。第 i 个物品要么不选，继承 $dp[i-1][cap]$ ；要么选，在容量足够时得到 $dp[i-1][cap-weight] + value$ 。二维表最清楚，一维表最高频。

Listing 9.9 0-1 背包：二维状态到一维滚动

```

1 int knapsack_01(const std::vector<int>& weights,
2               const std::vector<int>& values,
3               int capacity)
4 {

```

```

5     std::vector<int> dp(capacity + 1, 0);
6
7     for (int item = 0; item < static_cast<int>(weights.size()); ++item) {
8         for (int cap = capacity; cap >= weights[item]; --cap) {
9             dp[cap] = std::max(dp[cap],
10                                dp[cap - weights[item]] + values[item]);
11         }
12     }
13
14     return dp[capacity];
15 }

```

一维数组必须倒序遍历容量。原因是 `dp[cap - weights[item]]` 应该来自上一轮物品集合，而不是当前物品已经更新后的状态。如果正序遍历，当前物品会在同一轮被重复使用，语义就变成完全背包。这个遍历方向是背包题的核心，不是实现细节。分割等和子集、目标和都可以看成 0-1 背包变体。

完全背包中，每个物品可以使用无限次。零钱兑换 II 问组合数，状态 `dp[value]` 表示凑出金额 `value` 的组合数量。外层遍历硬币，内层正序遍历金额。正序是为了允许当前硬币重复使用；硬币在外层是为了每种组合只按一种硬币顺序被统计，避免把排列当组合。

Listing 9.10 零钱兑换 II：完全背包组合数

```

1 int change(int amount, const std::vector<int>& coins)
2 {
3     std::vector<int> dp(amount + 1, 0);
4     dp[0] = 1;
5
6     for (const int coin : coins) {
7         for (int value = coin; value <= amount; ++value) {
8             dp[value] += dp[value - coin];
9         }
10    }
11
12    return dp[amount];
13 }

```

如果题目问排列数，例如“有多少种有序序列凑成目标”，循环顺序要反过来：外层金额，内层硬币。这样 `1+2` 和 `2+1` 会在不同转移路径中被分别统计。背包题最容易错的不是公式，而是没有分清“每个物品能用几次”和“答案是否区分顺序”。这两个问题决定循环方向。

目标和展示代数转换。给每个数加正号或负号，使结果为 `target`。设正号集合和为 `P`，负号集合和为 `N`，总和为 `S`，则 $P - N = \text{target}$ ， $P + N = S$ ，推出 $P = (S + \text{target}) / 2$ 。问题变成从数组中选若干数，使和为 `P` 的方案数。每个数只能选一次，所以是 0-1 背包计数。

Listing 9.11 目标和：0-1 背包计数

```

1 int find_target_sum_ways(const std::vector<int>& nums, int target)
2 {
3     const int total = std::accumulate(nums.begin(), nums.end(), 0);
4     if (std::abs(target) > total || (total + target) % 2 != 0) {
5         return 0;
6     }
7
8     const int positive = (total + target) / 2;
9     std::vector<int> dp(positive + 1, 0);

```



```

10     dp[0] = 1;
11
12     for (const int x : nums) {
13         for (int sum = positive; sum >= x; --sum) {
14             dp[sum] += dp[sum - x];
15         }
16     }
17
18     return dp[positive];
19 }

```

如果数组中有 0，这段代码仍然正确。当 $x == 0$ 时， $dp[sum] += dp[sum]$ ，所有已有方案翻倍，正好对应给 0 加正号或负号都不改变总和但算作不同表达式。很多手写特判会在这里出错。计数 DP 还要关注结果范围；力扣题通常保证答案放进 `int`，工程中应考虑 `long long`。

股票问题是状态机 DP。只允许一次交易时，可以维护历史最低价；但加入冷冻期、手续费、交易次数限制后，最低价模型不够用。状态机方法把“每天结束时处于什么状态”作为 DP 状态。以含手续费的无限次交易为例，状态只有两个：`hold` 表示持有股票的最大收益，`cash` 表示不持有股票的最大收益。买入让 `cash` 变成 `hold`，卖出让 `hold` 变成 `cash`。

Listing 9.12 股票含手续费：两状态 DP

```

1 int max_profit_with_fee(const std::vector<int>& prices, int fee)
2 {
3     if (prices.empty()) {
4         return 0;
5     }
6
7     int hold = -prices[0];
8     int cash = 0;
9
10    for (int i = 1; i < static_cast<int>(prices.size()); ++i) {
11        const int previous_hold = hold;
12        const int previous_cash = cash;
13        hold = std::max(previous_hold, previous_cash - prices[i]);
14        cash = std::max(previous_cash, previous_hold + prices[i] - fee);
15    }
16
17    return cash;
18 }

```

必须先保存昨天状态，否则同一天买入后又立刻卖出会污染转移。手续费可以在买入时扣，也可以在卖出时扣，只要保持一致。冷冻期则需要额外状态或引用更早的 `cash`。至多 K 次交易需要把交易次数加入状态维度。股票题统一复盘方式是：列出状态，列出动作，说明动作受哪些规则限制。

区间 DP 的思维与线性 DP 不同。线性 DP 通常按位置从左到右推进；区间 DP 处理一个连续区间，转移往往枚举最后一个操作点或切分点。戳气球的关键是反过来想“最后戳哪个气球”。如果先戳某个气球，它的邻居会变化，状态不稳定；如果最后戳 `mid`，那么它的左右邻居一定是区间边界，收益可确定。

Listing 9.13 戳气球：区间 DP 枚举最后操作点

```

1 int max_coins(const std::vector<int>& nums)
2 {
3     std::vector<int> values;

```

```

4   values.reserve(nums.size() + 2);
5   values.push_back(1);
6   values.insert(values.end(), nums.begin(), nums.end());
7   values.push_back(1);
8
9   const int n = static_cast<int>(values.size());
10  std::vector<std::vector<int>> dp(n, std::vector<int>(n, 0));
11
12  for (int length = 2; length < n; ++length) {
13      for (int left = 0; left + length < n; ++left) {
14          const int right = left + length;
15          for (int mid = left + 1; mid < right; ++mid) {
16              const int score = dp[left][mid] + dp[mid][right] +
17                              values[left] * values[mid] * values[right];
18              dp[left][right] = std::max(dp[left][right], score);
19          }
20      }
21  }
22
23  return dp[0][n - 1];
24 }

```

`dp[left][right]` 表示开区间 $(left, right)$ 内所有气球被戳掉的最大收益。按区间长度从小到大遍历，是因为大区间依赖小区间。复杂度 $O(n^3)$ ，空间 $O(n^2)$ 。区间 DP 不是所有题都能优化，除非存在决策单调性、四边形不等式等额外性质；面试中不要随便承诺能降复杂度。

状态压缩 DP 把集合表示成二进制位，适合 n 很小但状态与“已选集合”有关的问题。访问所有节点的最短路径可以看成 BFS，也可以看成状态压缩：状态是 $(node, mask)$ ，表示当前在 `node`，已经访问集合为 `mask`。同一个节点配不同访问集合，是不同状态，因为未来能力不同。

Listing 9.14 访问所有节点：状态是当前位置加访问集合

```

1  int shortest_path_length(const std::vector<std::vector<int>>& graph)
2  {
3      const int n = static_cast<int>(graph.size());
4      const int full = (1 << n) - 1;
5      std::queue<std::pair<int, int>> queue;
6      std::vector<std::vector<int>> distance(n, std::vector<int>(1 << n, -1));
7
8      for (int node = 0; node < n; ++node) {
9          const int mask = 1 << node;
10         distance[node][mask] = 0;
11         queue.emplace(node, mask);
12     }
13
14     while (!queue.empty()) {
15         const auto [node, mask] = queue.front();
16         queue.pop();
17         if (mask == full) {
18             return distance[node][mask];
19         }
20
21         for (const int next : graph[node]) {
22             const int next_mask = mask | (1 << next);

```

```

23         if (distance[next][next_mask] != -1) {
24             continue;
25         }
26         distance[next][next_mask] = distance[node][mask] + 1;
27         queue.emplace(next, next_mask);
28     }
29 }
30
31 return 0;
32 }

```

状态数量是 $O(n2^n)$ ，边扩展约为 $O(m2^n)$ 。这类算法只适合 n 较小的题。状态压缩的核心不是位运算技巧，而是意识到“访问集合”是状态的一部分，不能只用当前位置去重。很多网格题带钥匙、障碍消除、剩余资源，也有类似思想：位置相同但资源不同，未来能力不同，就不能合并成一个状态。

DP 和图最短路之间可以互相转化。若状态转移形成有向无环图，可以按拓扑顺序填 DP；若转移有环且边权为 1，就可能需要 BFS；若边权非负，就可能需要 Dijkstra；若有负边，就要考虑 Bellman-Ford。所谓状态，就是图上的节点；所谓转移，就是边。这个视角能帮助你判断一道题到底该填表、该 BFS，还是该用堆。

常见误区

DP 常见错误有四个。第一，状态定义含糊，例如只写 `dp[i]` 是“最优值”，却不说考虑哪些元素、必须不必须选第 i 个。第二，初始化不符合状态含义。第三，遍历顺序让当前阶段读到了已经被污染的状态。第四，空间优化后无法解释旧值来自哪里。解决办法是先写自然语言状态，再写转移，再写代码。

DP 实验：第一，把 0-1 背包容量循环改成正序，用一个物品被重复使用的用例验证错误。第二，把零钱兑换 II 的循环顺序反过来，观察组合数变成排列数。第三，给股票含手续费打印每天的 `hold` 和 `cash`，理解状态机转移。第四，用小数组手算戳气球区间 DP 表，观察为什么要按区间长度遍历。

动态规划题复盘模板

每道 DP 题按五步复盘：第一，状态是什么，必须能用自然语言解释；第二，答案在 `dp[n]`、`dp[m][n]` 还是某个最大值里；第三，空状态和最小状态如何初始化；第四，转移枚举的是最后一步、最后一个选择，还是最后一段区间；第五，遍历顺序是否保证依赖状态已经算完。若你只能背出公式，却说不出公式对应的最后一步选择，这题就还没有真正掌握。

dp-greedy 深度题卡

动态规划与状态定义

下面的题卡都按同一条链路展开：从问题出发，先写暴力，再定位重复工作，然后选择数据结构或状态不变量，最后落到 C++ 实现、边界和扩展。格式统一是为了训练面试表达，而每张卡的关键观察和失效条件都要单独复盘。

图示：动态规划与状态定义推导链

暴力基线	枚举候选、完整模拟或逐个检查，先保证覆盖全部可能答案。
瓶颈定位	慢点不是递归写法，而是相同状态被反复计算。若一个状态的将来只取决于少量参数，就应保存结果并按依赖顺序计算。
结构选择	状态定义要包含足够信息但不能过度膨胀。线性 DP 看最后一步选择，二维 DP 看两个前缀或网格位置，背包看物品阶段和容量，区间 DP 看最后合并点。
不变量	填表不变量是：计算当前状态时，所有依赖状态已经算完且语义一致。滚动数组必须解释当前格读到的是上一轮还是本轮。
代码落地	数组维度和初始化要服务于状态含义。不可达用大正数或负数哨兵时要避免溢出。方案数可能超过 int 时改用 <code>long long</code> 或按题目取模。

53 最大子数组和

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。状态是必须以当前位置结尾的最大连续和。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“动态规划与状态定义”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力搜索枚举选择序列、切分点或路径，很多不同路径会到达同一个子问题。DP 的第一步是给这些重复子问题命名。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。慢点不是递归写法，而是相同状态被反复计算。若一个状态的将来只取决于少量参数，就应保存结果并按依赖顺序计算。放到本题里，要重点观察“若前一个后缀为负，接上它只会拖累当前元素，因此可以重新开始。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。状态定义要包含足够信息但不能过度膨胀。线性 DP 看最后一步选择，二维 DP 看两个前缀或网格位置，背包看物品阶段和容量，区间 DP 看最后合并点。本题的优化应当围绕“若前一个后缀为负，接上它只会拖累当前元素，因此可以重新开始。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。填表不变量是：计算当前状态时，所有依赖状态已经算完且语义一致。滚动数组必须解释当前格读到的是上一轮还是本轮。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。正确性通常从最优子结构证明：任意最优解的最后一步必然落入枚举集合；每个枚举候选又由已经正确的子问题构成。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。数组维度和初始化要服务于状态含义。不可达用大正数或负数哨兵时要避免溢出。方案数可能超过 int 时改用 `long long` 或按题目取模。本题还要特别注意：全负数组、单元素、和溢出、答案不能初始化为零。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：全负数组、单元素、和溢出、答案不能初始化为零。若要求返回区间下标，同步记录重新开始位

置和最优左右端。实验包含最小输入、全零、全负、无法到达、刚好到达、容量为零、字符串为空和空间压缩前后结果一致。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

62 不同路径

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。网格状态由上方和左方两种最后一步转移而来。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“动态规划与状态定义”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力搜索枚举选择序列、切分点或路径，很多不同路径会到达同一个子问题。DP 的第一步是给这些重复子问题命名。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。慢点不是递归写法，而是相同状态被反复计算。若一个状态的将来只取决于少量参数，就应保存结果并按依赖顺序计算。放到本题里，要重点观察“二维表可压缩为一维，因为当前行只依赖上一行同列和当前行左侧。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。状态定义要包含足够信息但不能过度膨胀。线性 DP 看最后一步选择，二维 DP 看两个前缀或网格位置，背包看物品阶段和容量，区间 DP 看最后合并点。本题的优化应当围绕“二维表可压缩为一维，因为当前行只依赖上一行同列和当前行左侧。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。填表不变量是：计算当前状态时，所有依赖状态已经算完且语义一致。滚动数组必须解释当前格读到的是上一轮还是本轮。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。正确性通常从最优子结构证明：任意最优解的最后一步必然落入枚举集合；每个枚举候选又由已经正确的子问题构成。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。数组维度和初始化要服务于状态含义。不可达用大正数或负数哨兵时要避免溢出。方案数可能超过 int 时改用 `long long` 或按题目取模。本题还要特别注意：一行、一列、较大网格结果溢出、障碍物版本边界。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：一行、一列、较大网格结果溢出、障碍物版本边界。带障碍物时遇到障碍状态清零，第一行第一列要按阻断处理。实验包含最小输入、全零、全负、无法到达、刚好到达、容量为零、字符串为空和空间压缩前后结果一致。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

63 不同路径 II

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。障碍物让某些格子的路径数变为零。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“动态规划与状态定义”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力搜索枚举选择序列、切分点或路径，很多不同路径会到达同一

个子问题。DP 的第一步是给这些重复子问题命名。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。慢点不是递归写法，而是相同状态被反复计算。若一个状态的将来只取决于少量参数，就应保存结果并按依赖顺序计算。放到本题里，要重点观察“滚动数组中遇到障碍直接把当前位置清零，否则累加左侧。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。状态定义要包含足够信息但不能过度膨胀。线性 DP 看最后一步选择，二维 DP 看两个前缀或网格位置，背包看物品阶段和容量，区间 DP 看最后合并点。本题的优化应当围绕“滚动数组中遇到障碍直接把当前位置清零，否则累加左侧。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。填表不变量是：计算当前状态时，所有依赖状态已经算完且语义一致。滚动数组必须解释当前格读到的是上一轮还是本轮。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。正确性通常从最优子结构证明：任意最优解的最后一步必然落入枚举集合；每个枚举候选又由已经正确的子问题构成。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。数组维度和初始化要服务于状态含义。不可达用大正数或负数哨兵时要避免溢出。方案数可能超过 int 时改用 `long long` 或按题目取模。本题还要特别注意：起点或终点是障碍、整行被截断、单行障碍。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：起点或终点是障碍、整行被截断、单行障碍。若允许向上或向左走，图会有环，普通填表不再成立。实验包含最小输入、全零、全负、无法到达、刚好到达、容量为零、字符串为空和空间压缩前后结果一致。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

64 最小路径和

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。状态是到达当前格子的最小代价，最后一步来自上方或左方。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“动态规划与状态定义”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力搜索枚举选择序列、切分点或路径，很多不同路径会到达同一个子问题。DP 的第一步是给这些重复子问题命名。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。慢点不是递归写法，而是相同状态被反复计算。若一个状态的将来只取决于少量参数，就应保存结果并按依赖顺序计算。放到本题里，要重点观察“先初始化第一行和第一列，再处理内部，主转移保持简单。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。状态定义要包含足够信息但不能过度膨胀。线性 DP 看最后一步选择，二维 DP 看两个前缀或网格位置，背包看物品阶段和容量，区

间 DP 看最后合并点。本题的优化应当围绕“先初始化第一行和第一列，再处理内部，主转移保持简单。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。填表不变量是：计算当前状态时，所有依赖状态已经算完且语义一致。滚动数组必须解释当前格读到的是上一轮还是本轮。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。正确性通常从最优子结构证明：任意最优解的最后一步必然落入枚举集合；每个枚举候选又由已经正确的子问题构成。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。数组维度和初始化要服务于状态含义。不可达用大正数或负数哨兵时要避免溢出。方案数可能超过 int 时改用 `long long` 或按题目取模。本题还要特别注意：单行、单列、权值为零、路径和溢出。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：单行、单列、权值为零、路径和溢出。若允许四方向移动，变成最短路而不是普通网格 DP。实验包含最小输入、全零、全负、无法到达、刚好到达、容量为零、字符串为空和空间压缩前后结果一致。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

70 爬楼梯

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。最后一步来自前一阶或前两阶，是最小的重叠子问题模型。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“动态规划与状态定义”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力搜索枚举选择序列、切分点或路径，很多不同路径会到达同一个子问题。DP 的第一步是给这些重复子问题命名。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。慢点不是递归写法，而是相同状态被反复计算。若一个状态的未来只取决于少量参数，就应保存结果并按依赖顺序计算。放到本题里，要重点观察“滚动变量保存最近两个状态即可，不需要完整数组。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。状态定义要包含足够信息但不能过度膨胀。线性 DP 看最后一步选择，二维 DP 看两个前缀或网格位置，背包看物品阶段和容量，区间 DP 看最后合并点。本题的优化应当围绕“滚动变量保存最近两个状态即可，不需要完整数组。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。填表不变量是：计算当前状态时，所有依赖状态已经算完且语义一致。滚动数组必须解释当前格读到的是上一轮还是本轮。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。正确性通常从最优子结构证明：任意最优解的最后一步必然落入枚举集合；每个枚举候选又由已经正确的子问题构成。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说

明你还没有把优化从暴力中推导出来。

代码落地。数组维度和初始化要服务于状态含义。不可达用大正数或负数哨兵时要避免溢出。方案数可能超过 `int` 时改用 `long long` 或按题目取模。本题还要特别注意： n 为一、 n 为二、题目是否允许零阶、结果范围。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是： n 为一、 n 为二、题目是否允许零阶、结果范围。若每次可走一到 k 阶，转移变成固定宽度窗口求和。实验包含最小输入、全零、全负、无法到达、刚好到达、容量为零、字符串为空和空间压缩前后结果一致。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

72 编辑距离

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。二维状态表示两个前缀之间的最少编辑操作。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“动态规划与状态定义”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力搜索枚举选择序列、切分点或路径，很多不同路径会到达同一个子问题。DP 的第一步是给这些重复子问题命名。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。慢点不是递归写法，而是相同状态被反复计算。若一个状态的未来只取决于少量参数，就应保存结果并按依赖顺序计算。放到本题里，要重点观察“末尾字符相同继承左上；不同则枚举插入、删除、替换。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。状态定义要包含足够信息但不能过度膨胀。线性 DP 看最后一步选择，二维 DP 看两个前缀或网格位置，背包看物品阶段和容量，区间 DP 看最后合并点。本题的优化应当围绕“末尾字符相同继承左上；不同则枚举插入、删除、替换。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。填表不变量是：计算当前状态时，所有依赖状态已经算完且语义一致。滚动数组必须解释当前格读到的是上一轮还是本轮。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。正确性通常从最优子结构证明：任意最优解的最后一步必然落入枚举集合；每个枚举候选又由已经正确的子问题构成。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。数组维度和初始化要服务于状态含义。不可达用大正数或负数哨兵时要避免溢出。方案数可能超过 `int` 时改用 `long long` 或按题目取模。本题还要特别注意：空串、完全相同、完全不同、操作代价不同。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：空串、完全相同、完全不同、操作代价不同。若只允许插入删除，问题退化到和最长公共子序列

相关。实验包含最小输入、全零、全负、无法到达、刚好到达、容量为零、字符串为空和空间压缩前后结果一致。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

121 买卖股票的最佳时机

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。卖出日固定时，最佳买入日是此前最低价格。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“动态规划与状态定义”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力搜索枚举选择序列、切分点或路径，很多不同路径会到达同一个子问题。DP 的第一步是给这些重复子问题命名。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。慢点不是递归写法，而是相同状态被反复计算。若一个状态的将来只取决于少量参数，就应保存结果并按依赖顺序计算。放到本题里，要重点观察“扫描维护历史最低价和最大利润，一次交易无需复杂状态机。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。状态定义要包含足够信息但不能过度膨胀。线性 DP 看最后一步选择，二维 DP 看两个前缀或网格位置，背包看物品阶段和容量，区间 DP 看最后合并点。本题的优化应当围绕“扫描维护历史最低价和最大利润，一次交易无需复杂状态机。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。填表不变量是：计算当前状态时，所有依赖状态已经算完且语义一致。滚动数组必须解释当前格读到的是上一轮还是本轮。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。正确性通常从最优子结构证明：任意最优解的最后一步必然落入枚举集合；每个枚举候选又由已经正确的子问题构成。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。数组维度和初始化要服务于状态含义。不可达用大正数或负数哨兵时要避免溢出。方案数可能超过 int 时改用 `long long` 或按题目取模。本题还要特别注意：单日价格、一直下跌、一直上涨、价格相同。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：单日价格、一直下跌、一直上涨、价格相同。多次交易、冷冻期和手续费版本都要扩展成状态机。实验包含最小输入、全零、全负、无法到达、刚好到达、容量为零、字符串为空和空间压缩前后结果一致。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

123 买卖股票 III

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。最多两次交易，状态需要包含交易次数和持有状态。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“动态规划与状态定义”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力搜索枚举选择序列、切分点或路径，很多不同路径会到达同一

个子问题。DP 的第一步是给这些重复子问题命名。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。慢点不是递归写法，而是相同状态被反复计算。若一个状态的未来只取决于少量参数，就应保存结果并按依赖顺序计算。放到本题里，要重点观察“维护 buy1、sell1、buy2、sell2 四个状态，按天更新。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。状态定义要包含足够信息但不能过度膨胀。线性 DP 看最后一步选择，二维 DP 看两个前缀或网格位置，背包看物品阶段和容量，区间 DP 看最后合并点。本题的优化应当围绕“维护 buy1、sell1、buy2、sell2 四个状态，按天更新。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。填表不变量是：计算当前状态时，所有依赖状态已经算完且语义一致。滚动数组必须解释当前格读到的是上一轮还是本轮。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。正确性通常从最优子结构证明：任意最优解的最后一步必然落入枚举集合；每个枚举候选又由已经正确的子问题构成。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。数组维度和初始化要服务于状态含义。不可达用大正数或负数哨兵时要避免溢出。方案数可能超过 int 时改用 long long 或按题目取模。本题还要特别注意：价格为空、只够一次交易、两次交易间不能重叠。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：价格为空、只够一次交易、两次交易间不能重叠。最多 k 次交易是它的泛化，k 很大时退化为无限次交易。实验包含最小输入、全零、全负、无法到达、刚好到达、容量为零、字符串为空和空间压缩前后结果一致。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

139 单词拆分

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。状态表示前缀能否由字典单词拼出。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“动态规划与状态定义”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力搜索枚举选择序列、切分点或路径，很多不同路径会到达同一个子问题。DP 的第一步是给这些重复子问题命名。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。慢点不是递归写法，而是相同状态被反复计算。若一个状态的未来只取决于少量参数，就应保存结果并按依赖顺序计算。放到本题里，要重点观察“枚举最后一个单词的起点，左侧前缀可达且子串在字典中则当前可达。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。状态定义要包含足够信息但不能过度膨胀。线性 DP 看最后一步选择，二维 DP 看两个前缀或网格位置，背包看物品阶段和容量，区

间 DP 看最后合并点。本题的优化应当围绕“枚举最后一个单词的起点，左侧前缀可达且子串在字典中则当前可达。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。填表不变量是：计算当前状态时，所有依赖状态已经算完且语义一致。滚动数组必须解释当前格读到的是上一轮还是本轮。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。正确性通常从最优子结构证明：任意最优解的最后一步必然落入枚举集合；每个枚举候选又由已经正确的子问题构成。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。数组维度和初始化要服务于状态含义。不可达用大正数或负数哨兵时要避免溢出。方案数可能超过 int 时改用 `long long` 或按题目取模。本题还要特别注意：空串、重复单词、长单词、substr 复制成本。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：空串、重复单词、长单词、substr 复制成本。若要输出所有句子，需要 DFS 加记忆化或前驱列表。实验包含最小输入、全零、全负、无法到达、刚好到达、容量为零、字符串为空和空间压缩前后结果一致。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

152 乘积最大子数组

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。负数会让最大乘积和最小乘积互换。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“动态规划与状态定义”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力搜索枚举选择序列、切分点或路径，很多不同路径会到达同一个子问题。DP 的第一步是给这些重复子问题命名。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。慢点不是递归写法，而是相同状态被反复计算。若一个状态的未来只取决于少量参数，就应保存结果并按依赖顺序计算。放到本题里，要重点观察“同时维护以当前位置结尾的最大和最小乘积。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。状态定义要包含足够信息但不能过度膨胀。线性 DP 看最后一步选择，二维 DP 看两个前缀或网格位置，背包看物品阶段和容量，区间 DP 看最后合并点。本题的优化应当围绕“同时维护以当前位置结尾的最大和最小乘积。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。填表不变量是：计算当前状态时，所有依赖状态已经算完且语义一致。滚动数组必须解释当前格读到的是上一轮还是本轮。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。正确性通常从最优子结构证明：任意最优解的最后一步必然落入枚举集合；每个枚举候选又由已经正确的子问题构成。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说

明你还没有把优化从暴力中推导出来。

代码落地。数组维度和初始化要服务于状态含义。不可达用大正数或负数哨兵时要避免溢出。方案数可能超过 `int` 时改用 `long long` 或按题目取模。本题还要特别注意：零、负数个数奇偶、单元元素、乘积溢出。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：零、负数个数奇偶、单元元素、乘积溢出。如果要求返回区间，同步记录状态来源。实验包含最小输入、全零、全负、无法到达、刚好到达、容量为零、字符串为空和空间压缩前后结果一致。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

198 打家劫舍

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。每间房只有偷和不偷两种结局，偷当前就不能偷前一间。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“动态规划与状态定义”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力搜索枚举选择序列、切分点或路径，很多不同路径会到达同一个子问题。DP 的第一步是给这些重复子问题命名。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。慢点不是递归写法，而是相同状态被反复计算。若一个状态的未来只取决于少量参数，就应保存结果并按依赖顺序计算。放到本题里，要重点观察“滚动保存前一状态和前两状态。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。状态定义要包含足够信息但不能过度膨胀。线性 DP 看最后一步选择，二维 DP 看两个前缀或网格位置，背包看物品阶段和容量，区间 DP 看最后合并点。本题的优化应当围绕“滚动保存前一状态和前两状态。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。填表不变量是：计算当前状态时，所有依赖状态已经算完且语义一致。滚动数组必须解释当前格读到的是上一轮还是本轮。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。正确性通常从最优子结构证明：任意最优解的最后一步必然落入枚举集合；每个枚举候选又由已经正确的子问题构成。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。数组维度和初始化要服务于状态含义。不可达用大正数或负数哨兵时要避免溢出。方案数可能超过 `int` 时改用 `long long` 或按题目取模。本题还要特别注意：空数组、单间、全零、金额很大。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：空数组、单间、全零、金额很大。环形房屋要拆成不含首和不含尾两个线性问题。实验包含最小输入、全零、全负、无法到达、刚好到达、容量为零、字符串为空和空间压缩前后结果一致。实验报

告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

221 最大正方形

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。以当前位置为右下角的最大正方形由上、左、左上三个状态限制。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“动态规划与状态定义”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力搜索枚举选择序列、切分点或路径，很多不同路径会到达同一个子问题。DP 的第一步是给这些重复子问题命名。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。慢点不是递归写法，而是相同状态被反复计算。若一个状态的未来只取决于少量参数，就应保存结果并按依赖顺序计算。放到本题里，要重点观察“当前位置为一时取三者最小加一，否则为零。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。状态定义要包含足够信息但不能过度膨胀。线性 DP 看最后一步选择，二维 DP 看两个前缀或网格位置，背包看物品阶段和容量，区间 DP 看最后合并点。本题的优化应当围绕“当前位置为一时取三者最小加一，否则为零。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。填表不变量是：计算当前状态时，所有依赖状态已经算完且语义一致。滚动数组必须解释当前格读到的是上一轮还是本轮。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。正确性通常从最优子结构证明：任意最优解的最后一步必然落入枚举集合；每个枚举候选又由已经正确的子问题构成。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。数组维度和初始化要服务于状态含义。不可达用大正数或负数哨兵时要避免溢出。方案数可能超过 `int` 时改用 `long long` 或按题目取模。本题还要特别注意：全零、全一、单行单列、字符一和数字一不要混淆。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：全零、全一、单行单列、字符一和数字一不要混淆。最大矩形则需要按行构造柱状图再用单调栈。实验包含最小输入、全零、全负、无法到达、刚好到达、容量为零、字符串为空和空间压缩前后结果一致。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

300 最长递增子序列

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。二次 DP 固定结尾，二分优化维护每个长度的最小结尾。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“动态规划与状态定义”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力搜索枚举选择序列、切分点或路径，很多不同路径会到达同一个子问题。DP 的第一步是给这些重复子问题命名。写暴力不是为了提交它，而是为了暴露状态空

间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。慢点不是递归写法，而是相同状态被反复计算。若一个状态的未来只取决于少量参数，就应保存结果并按依赖顺序计算。放到本题里，要重点观察“tails 不是实际答案序列，而是未来扩展潜力表。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。状态定义要包含足够信息但不能过度膨胀。线性 DP 看最后一步选择，二维 DP 看两个前缀或网格位置，背包看物品阶段和容量，区间 DP 看最后合并点。本题的优化应当围绕“tails 不是实际答案序列，而是未来扩展潜力表。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。填表不变量是：计算当前状态时，所有依赖状态已经算完且语义一致。滚动数组必须解释当前格读到的是上一轮还是本轮。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。正确性通常从最优子结构证明：任意最优解的最后一步必然落入枚举集合；每个枚举候选又由已经正确的子问题构成。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。数组维度和初始化要服务于状态含义。不可达用大正数或负数哨兵时要避免溢出。方案数可能超过 int 时改用 `long long` 或按题目取模。本题还要特别注意：重复元素、严格递增和非递减区别、空数组。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：重复元素、严格递增和非递减区别、空数组。若要还原路径，需要额外前驱数组，不能只看 tails。实验包含最小输入、全零、全负、无法到达、刚好到达、容量为零、字符串为空和空间压缩前后结果一致。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

309 最佳买卖股票含冷冻期

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。冷冻期让买入来源受昨天卖出状态限制。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“动态规划与状态定义”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力搜索枚举选择序列、切分点或路径，很多不同路径会到达同一个子问题。DP 的第一步是给这些重复子问题命名。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。慢点不是递归写法，而是相同状态被反复计算。若一个状态的未来只取决于少量参数，就应保存结果并按依赖顺序计算。放到本题里，要重点观察“状态机维护持有、刚卖出、休息三种日终状态。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。状态定义要包含足够信息但不能过度膨胀。线性 DP 看最后一步选择，二维 DP 看两个前缀或网格位置，背包看物品阶段和容量，区间 DP 看最后合并点。本题的优化应当围绕“状态机维护持有、刚卖出、休息三种日终状态。”建立

一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。填表不变量是：计算当前状态时，所有依赖状态已经算完且语义一致。滚动数组必须解释当前格读到的是上一轮还是本轮。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。正确性通常从最优子结构证明：任意最优解的最后一步必然落入枚举集合；每个枚举候选又由已经正确的子问题构成。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。数组维度和初始化要服务于状态含义。不可达用大正数或负数哨兵时要避免溢出。方案数可能超过 `int` 时改用 `long long` 或按题目取模。本题还要特别注意：空价格、一天、连续上涨、卖出后不能次日买入。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：空价格、一天、连续上涨、卖出后不能次日买入。手续费版本可以在买入或卖出转移中扣费。实验包含最小输入、全零、全负、无法到达、刚好到达、容量为零、字符串为空和空间压缩前后结果一致。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

714 买卖股票含手续费

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。手续费改变交易收益，仍可用持有和不持有状态。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“动态规划与状态定义”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力搜索枚举选择序列、切分点或路径，很多不同路径会到达同一个子问题。DP 的第一步是给这些重复子问题命名。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。慢点不是递归写法，而是相同状态被反复计算。若一个状态的未来只取决于少量参数，就应保存结果并按依赖顺序计算。放到本题里，要重点观察“卖出时扣费或买入时扣费都可以，但不要重复扣。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。状态定义要包含足够信息但不能过度膨胀。线性 DP 看最后一步选择，二维 DP 看两个前缀或网格位置，背包看物品阶段和容量，区间 DP 看最后合并点。本题的优化应当围绕“卖出时扣费或买入时扣费都可以，但不要重复扣。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。填表不变量是：计算当前状态时，所有依赖状态已经算完且语义一致。滚动数组必须解释当前格读到的是上一轮还是本轮。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。正确性通常从最优子结构证明：任意最优解的最后一步必然落入枚举集合；每个枚举候选又由已经正确的子问题构成。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。数组维度和初始化要服务于状态含义。不可达用大正数或负数哨兵时要避免溢出。方案数可能超过 `int` 时改用 `long long` 或按题目取模。本题还要特别注意：手续费为零、价格震荡、小利润不足手续费。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：手续费为零、价格震荡、小利润不足手续费。这是状态机比局部贪心更稳的例子。实验包含最小输入、全零、全负、无法到达、刚好到达、容量为零、字符串为空和空间压缩前后结果一致。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

1143 最长公共子序列

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。状态表示两个前缀的最长公共子序列长度。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“动态规划与状态定义”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力搜索枚举选择序列、切分点或路径，很多不同路径会到达同一个子问题。DP 的第一步是给这些重复子问题命名。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。慢点不是递归写法，而是相同状态被反复计算。若一个状态的未来只取决于少量参数，就应保存结果并按依赖顺序计算。放到本题里，要重点观察“末尾相同取左上加一，不同取上方和左方最大。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。状态定义要包含足够信息但不能过度膨胀。线性 DP 看最后一步选择，二维 DP 看两个前缀或网格位置，背包看物品阶段和容量，区间 DP 看最后合并点。本题的优化应当围绕“末尾相同取左上加一，不同取上方和左方最大。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。填表不变量是：计算当前状态时，所有依赖状态已经算完且语义一致。滚动数组必须解释当前格读到的是上一轮还是本轮。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。正确性通常从最优子结构证明：任意最优解的最后一步必然落入枚举集合；每个枚举候选又由已经正确的子问题构成。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。数组维度和初始化要服务于状态含义。不可达用大正数或负数哨兵时要避免溢出。方案数可能超过 `int` 时改用 `long long` 或按题目取模。本题还要特别注意：空串、完全相同、无公共字符、子序列不是子串。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：空串、完全相同、无公共字符、子序列不是子串。若要求还原序列，需要从表右下角反向追踪选择。实验包含最小输入、全零、全负、无法到达、刚好到达、容量为零、字符串为空和空间压缩前后

结果一致。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

背包模型

下面的题卡都按同一条链路展开：从问题出发，先写暴力，再定位重复工作，然后选择数据结构或状态不变量，最后落到 C++ 实现、边界和扩展。格式统一是为了训练面试表达，而每张卡的关键观察和失效条件都要单独复盘。

图示：背包模型推导链

暴力基线	枚举候选、完整模拟或逐个检查，先保证覆盖全部可能答案。
瓶颈定位	瓶颈是阶段相同、剩余容量相同的子问题反复出现。容量把未来选择压缩成一个数字，因此可以用表保存。
结构选择	0-1 背包每个物品只能用一次，一维压缩时容量倒序；完全背包物品可无限使用，容量正序。计数组合和计数排列的循环顺序不同。
不变量	处理到第 i 个物品后， $dp[cap]$ 表示只使用前 i 类物品能达到的最优值或方案数。倒序是为了不在同一轮重复使用当前物品。
代码落地	容量通常与数值大小有关，所以复杂度是伪多项式。实现时先判断总和奇偶、目标范围和负数是否存在。

279 完全平方数

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。完全平方数可以看成无限使用的物品，目标是凑出 n 的最少数目。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“背包模型”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力枚举每个物品选或不选，或者枚举每种硬币使用次数。物品数稍大时，搜索树会迅速膨胀。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。瓶颈是阶段相同、剩余容量相同的子问题反复出现。容量把未来选择压缩成一个数字，因此可以用表保存。放到本题里，要重点观察“完全背包最少物品数，或把数字看成图节点做 BFS。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。0-1 背包每个物品只能用一次，一维压缩时容量倒序；完全背包物品可无限使用，容量正序。计数组合和计数排列的循环顺序不同。本题的优化应当围绕“完全背包最少物品数，或把数字看成图节点做 BFS。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。处理到第 i 个物品后， $dp[cap]$ 表示只使用前 i 类物品能达到的最优值或方案数。倒序是为了不在同一轮重复使用当前物品。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。正确性来自选择当前物品的次数枚举。0-1 只有选与不选，完全背包允许从当前轮状态继续转移。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。容量通常与数值大小有关，所以复杂度是伪多项式。实现时先判断总和奇偶、目标范围和负数是否存在。本题还要特别注意： n 为零或一、完全平方数本身、较大 n 。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是： n 为零或一、完全平方数本身、较大 n 。数学四平方定理可给更强解法，但面试 DP 更通用。实验把容量循环方向故意写反，观察 0-1 物品被重复使用。再把硬币组合循环顺序反过来，观察组合数变成排列数。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

322 零钱兑换

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。金额是容量，硬币可无限使用，目标是最少硬币数。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“背包模型”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力枚举每个物品选或不选，或者枚举每种硬币使用次数。物品数稍大时，搜索树会迅速膨胀。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。瓶颈是阶段相同、剩余容量相同的子问题反复出现。容量把未来选择压缩成一个数字，因此可以用表保存。放到本题里，要重点观察“ $dp[value]$ 从 $value$ 减 $coin$ 的已知状态转移。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。0-1 背包每个物品只能用一次，一维压缩时容量倒序；完全背包物品可无限使用，容量正序。计数组合和计数排列的循环顺序不同。本题的优化应当围绕“ $dp[value]$ 从 $value$ 减 $coin$ 的已知状态转移。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。处理到第 i 个物品后， $dp[cap]$ 表示只使用前 i 类物品能达到的最优值或方案数。倒序是为了不在同一轮重复使用当前物品。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。正确性来自选择当前物品的次数枚举。0-1 只有选与不选，完全背包允许从当前轮状态继续转移。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。容量通常与数值大小有关，所以复杂度是伪多项式。实现时先判断总和奇偶、目标范围和负数是否存在。本题还要特别注意：无法凑出、amount 为零、硬币面额大于目标、哨兵溢出。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：无法凑出、amount 为零、硬币面额大于目标、哨兵溢出。零钱兑换 II 问组合数，循环语义不同。实验把容量循环方向故意写反，观察 0-1 物品被重复使用。再把硬币组合循环顺序反过来，观察组合数变成排列数。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

416 分割等和子集

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。总和一半是背包容量，每个数只能使用一次。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、

集合还是判定结果。把这些拆开以后，才进入“背包模型”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力枚举每个物品选或不选，或者枚举每种硬币使用次数。物品数稍大时，搜索树会迅速膨胀。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。瓶颈是阶段相同、剩余容量相同的子问题反复出现。容量把未来选择压缩成一个数字，因此可以用表保存。放到本题里，要重点观察“0-1 背包可达性，一维容量必须倒序。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。0-1 背包每个物品只能用一次，一维压缩时容量倒序；完全背包物品可无限使用，容量正序。计数组合和计数排列的循环顺序不同。本题的优化应当围绕“0-1 背包可达性，一维容量必须倒序。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。处理到第 i 个物品后， $dp[cap]$ 表示只使用前 i 类物品能达到的最优值或方案数。倒序是为了不在同一轮重复使用当前物品。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。正确性来自选择当前物品的次数枚举。0-1 只有选与不选，完全背包允许从当前轮状态继续转移。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。容量通常与数值大小有关，所以复杂度是伪多项式。实现时先判断总和奇偶、目标范围和负数是否存在。本题还要特别注意：总和为奇数、目标为零、数值较大、复杂度依赖 $target$ 。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：总和为奇数、目标为零、数值较大、复杂度依赖 $target$ 。负数会破坏普通容量模型，需要重新建模。实验把容量循环方向故意写反，观察 0-1 物品被重复使用。再把硬币组合循环顺序反过来，观察组合数变成排列数。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

494 目标和

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。正负号选择可代数转换为选若干数凑出特定正和。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“背包模型”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力枚举每个物品选或不选，或者枚举每种硬币使用次数。物品数稍大时，搜索树会迅速膨胀。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。瓶颈是阶段相同、剩余容量相同的子问题反复出现。容量把未来选择压缩成一个数字，因此可以用表保存。放到本题里，要重点观察“若 S 加 $target$ 为奇数或目标绝对值过大，直接无解。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。0-1 背包每个物品只能用一次，一维压缩时容量倒序；完全背包物品可无限使用，容量正序。计数组合和计数排列的循环顺序不同。本题的优化应当围绕“若 S 加 $target$ 为奇数或目标绝对值过大，直接无解。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。处理到第 i 个物品后， $dp[cap]$ 表示只使用前 i 类物品能达到的最优值或方案数。倒序是为了不在同一轮重复使用当前物品。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。正确性来自选择当前物品的次数枚举。0-1 只有选与不选，完全背包允许从当前轮状态继续转移。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。容量通常与数值大小有关，所以复杂度是伪多项式。实现时先判断总和奇偶、目标范围和负数是否存在。本题还要特别注意：零元素会让方案数翻倍、目标为负、容量为零。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：零元素会让方案数翻倍、目标为负、容量为零。也可用哈希 DP 保存所有可能和，但背包转换更高效。实验把容量循环方向故意写反，观察 0-1 物品被重复使用。再把硬币组合循环顺序反过来，观察组合数变成排列数。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

518 零钱兑换 II

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。硬币无限使用，问组合数而不是最少数量。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“背包模型”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力枚举每个物品选或不选，或者枚举每种硬币使用次数。物品数稍大时，搜索树会迅速膨胀。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。瓶颈是阶段相同、剩余容量相同的子问题反复出现。容量把未来选择压缩成一个数字，因此可以用表保存。放到本题里，要重点观察“外层遍历硬币、内层金额正序，保证组合按固定硬币顺序统计。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。0-1 背包每个物品只能用一次，一维压缩时容量倒序；完全背包物品可无限使用，容量正序。计数组合和计数排列的循环顺序不同。本题的优化应当围绕“外层遍历硬币、内层金额正序，保证组合按固定硬币顺序统计。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。处理到第 i 个物品后， $dp[cap]$ 表示只使用前 i 类物品能达到的最优值或方案数。倒序是为了不在同一轮重复使用当前物品。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。正确性来自选择当前物品的次数枚举。0-1 只有选与不选，完全背包允许从当前轮状态继续转移。证

明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。容量通常与数值大小有关，所以复杂度是伪多项式。实现时先判断总和奇偶、目标范围和负数是否存在。本题还要特别注意：amount 为零、无硬币、组合数较大、循环顺序反例。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：amount 为零、无硬币、组合数较大、循环顺序反例。若外层金额内层硬币，会统计排列数。实验把容量循环方向故意写反，观察 0-1 物品被重复使用。再把硬币组合循环顺序反过来，观察组合数变成排列数。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

贪心与交换证明

下面的题卡都按同一条链路展开：从问题出发，先写暴力，再定位重复工作，然后选择数据结构或状态不变量，最后落到 C++ 实现、边界和扩展。格式统一是为了训练面试表达，而每张卡的关键观察和失效条件都要单独复盘。

图示：贪心与交换证明推导链

暴力基线	枚举候选、完整模拟或逐个检查，先保证覆盖全部可能答案。
瓶颈定位	慢点是选择空间爆炸。若每一步都有很多候选，而某个排序规则或边界规则能安全丢掉大部分候选，就可能存在贪心。
结构选择	贪心需要找到可交换的局部最优：最早结束、最小右端、当前最远覆盖、当前最大收益或最小代价。排序只是手段，不是证明。
不变量	贪心不变量通常是当前方案在相同选择次数下不落后于任何其他方案，或者当前保留的资源最多、结束最早、右边界最小。
代码落地	实现时先把比较器写成明确的业务顺序。区问题注意起点相同时终点升序还是降序；覆盖题注意闭区间和半开区间的等号。

55 跳跃游戏

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。扫描过程中只关心当前能到达的最远位置。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“贪心与交换证明”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力做法枚举所有选择顺序或所有子集，找出最优方案。贪心题的难点不在写循环，而在证明局部选择不会破坏全局最优。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。慢点是选择空间爆炸。若每一步都有很多候选，而某个排序规则或边界规则能安全丢掉大部分候选，就可能存在贪心。放到本题里，要重点观察“如果下标超过最远可达，任何之前路径都不能到达它；否则用它扩展边界。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。贪心需要找到可交换的局部最优：最早结束、最小右端、当前最远覆盖、当前最大收益或最小代价。排序只是手段，不是证明。本题的优化应当围绕“如果下标超过最远可达，任何之前路径都不能到达它；否则用它扩展边界。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，

例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。贪心不变量通常是当前方案在相同选择次数下不落后于任何其他方案，或者当前保留的资源最多、结束最早、右边界最小。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。常见证明有交换论证和领先性论证。交换论证说明最优解可以被替换成贪心第一步而不变差；领先性论证说明贪心每一步都至少和任何方案一样好。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。实现时先把比较器写成明确的业务顺序。区问题注意起点相同时终点升序还是降序；覆盖题注意闭区间和半开区间的等号。本题还要特别注意：第一个元素为零、数组长度一、恰好到达最后、远超最后。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：第一个元素为零、数组长度一、恰好到达最后、远超最后。求最少跳数时把可达区间看成 BFS 层。实验要造反例：如果把排序关键字改成另一个直觉规则，是否失败。能给出反例，才说明真正理解了贪心条件。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

122 买卖股票 II

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。允许多次交易且不能同时持有，所有正收益差都可以收集。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“贪心与交换证明”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力做法枚举所有选择顺序或所有子集，找出最优方案。贪心题的难点不在写循环，而在证明局部选择不会破坏全局最优。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。慢点是选择空间爆炸。若每一步都有很多候选，而某个排序规则或边界规则能安全丢掉大部分候选，就可能存在贪心。放到本题里，要重点观察“把长上涨段拆成每天买卖收益相同，因此累加正差值最优。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。贪心需要找到可交换的局部最优：最早结束、最小右端、当前最远覆盖、当前最大收益或最小代价。排序只是手段，不是证明。本题的优化应当围绕“把长上涨段拆成每天买卖收益相同，因此累加正差值最优。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。贪心不变量通常是当前方案在相同选择次数下不落后于任何其他方案，或者当前保留的资源最多、结束最早、右边界最小。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。常见证明有交换论证和领先性论证。交换论证说明最优解可以被替换成贪心第一步而不变差；领先性论证说明贪心每一步都至少和任何方案一样好。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。实现时先把比较器写成明确的业务顺序。区问题注意起点相同时终点升序还是降序；覆盖题注意闭区间和半开区间的等号。本题还要特别注意：下降段、平台价格、只有一天、手续费不存在。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁

式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：下降段、平台价格、只有一天、手续费不存在。有手续费时不能简单累加正差，需要状态机或调整贪心阈值。实验要造反例：如果把排序关键字改成另一个直觉规则，是否失败。能给出反例，才说明真正理解了贪心条件。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

169 多数元素

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。超过一半的元素可以和其他元素两两抵消后仍然剩下。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“贪心与交换证明”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力做法枚举所有选择顺序或所有子集，找出最优方案。贪心题的难点不在写循环，而在证明局部选择不会破坏全局最优。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。慢点是选择空间爆炸。若每一步都有很多候选，而某个排序规则或边界规则能安全丢掉大部分候选，就可能存在贪心。放到本题里，要重点观察“Boyer-Moore 投票维护候选和计数。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。贪心需要找到可交换的局部最优：最早结束、最小右端、当前最远覆盖、当前最大收益或最小代价。排序只是手段，不是证明。本题的优化应当围绕“Boyer-Moore 投票维护候选和计数。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。贪心不变量通常是当前方案在相同选择次数下不落后于任何其他方案，或者当前保留的资源最多、结束最早、右边界最小。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。常见证明有交换论证和领先性论证。交换论证说明最优解可以被替换成贪心第一步而不变差；领先性论证说明贪心每一步都至少和任何方案一样好。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。实现时先把比较器写成明确的业务顺序。区问题注意起点相同时终点升序还是降序；覆盖题注意闭区间和半开区间的等号。本题还要特别注意：题目是否保证存在多数、计数归零、全相同。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：题目是否保证存在多数、计数归零、全相同。若找超过三分之一的元素，需要维护两个候选。实验要造反例：如果把排序关键字改成另一个直觉规则，是否失败。能给出反例，才说明真正理解了贪心条件。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

435 无重叠区间

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。保留最多不重叠区间等价于删除最少区间。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“贪心与交换证明”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力做法枚举所有选择顺序或所有子集，找出最优方案。贪心题的难点不在写循环，而在证明局部选择不会破坏全局最优。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。慢点是选择空间爆炸。若每一步都有很多候选，而某个排序规则或边界规则能安全丢掉大部分候选，就可能存在贪心。放到本题里，要重点观察“按终点升序选择，结束越早给后续留下空间越多。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。贪心需要找到可交换的局部最优：最早结束、最小右端、当前最远覆盖、当前最大收益或最小代价。排序只是手段，不是证明。本题的优化应当围绕“按终点升序选择，结束越早给后续留下空间越多。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。贪心不变量通常是当前方案在相同选择次数下不落后于任何其他方案，或者当前保留的资源最多、结束最早、右边界最小。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。常见证明有交换论证和领先性论证。交换论证说明最优解可以被替换成贪心第一步而不变差；领先性论证说明贪心每一步都至少和任何方案一样好。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。实现时先把比较器写成明确的业务顺序。区问题注意起点相同时终点升序还是降序；覆盖题注意闭区间和半开区间的等号。本题还要特别注意：端点相接是否重叠、完全覆盖、相同终点、空列表。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：端点相接是否重叠、完全覆盖、相同终点、空列表。用交换论证说明最早结束的选择不劣于其他第一选择。实验要造反例：如果把排序关键字改成另一个直觉规则，是否失败。能给出反例，才说明真正理解了贪心条件。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

452 用最少数量的箭引爆气球

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。一支箭的位置可以覆盖所有包含该位置的区间。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“贪心与交换证明”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力做法枚举所有选择顺序或所有子集，找出最优方案。贪心题的难点不在写循环，而在证明局部选择不会破坏全局最优。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新

取得。慢点是选择空间爆炸。若每一步都有很多候选，而某个排序规则或边界规则能安全丢掉大部分候选，就可能存在贪心。放到本题里，要重点观察“按右端点排序，当前箭放在最早结束气球的右端。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。贪心需要找到可交换的局部最优：最早结束、最小右端、当前最远覆盖、当前最大收益或最小代价。排序只是手段，不是证明。本题的优化应当围绕“按右端点排序，当前箭放在最早结束气球的右端。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。贪心不变量通常是当前方案在相同选择次数下不落后于任何其他方案，或者当前保留的资源最多、结束最早、右边界最小。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。常见证明有交换论证和领先性论证。交换论证说明最优解可以被替换成贪心第一步而不变差；领先性论证说明贪心每一步都至少和任何方案一样好。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。实现时先把比较器写成明确的业务顺序。区问题注意起点相同时终点升序还是降序；覆盖题注意闭区间和半开区间的等号。本题还要特别注意：端点相同、负坐标、区间包含、闭区间边界。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：端点相同、负坐标、区间包含、闭区间边界。它和无重叠区间是同一类最早结束贪心。实验要造反例：如果把排序关键字改成另一个直觉规则，是否失败。能给出反例，才说明真正理解了贪心条件。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

763 划分字母区间

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。每个片段必须覆盖其中所有字符的最后出现位置。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“贪心与交换证明”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力做法枚举所有选择顺序或所有子集，找出最优方案。贪心题的难点不在写循环，而在证明局部选择不会破坏全局最优。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。慢点是选择空间爆炸。若每一步都有很多候选，而某个排序规则或边界规则能安全丢掉大部分候选，就可能存在贪心。放到本题里，要重点观察“扫描维护当前片段最远右端，到达右端即可切分。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。贪心需要找到可交换的局部最优：最早结束、最小右端、当前最远覆盖、当前最大收益或最小代价。排序只是手段，不是证明。本题的优化应当围绕“扫描维护当前片段最远右端，到达右端即可切分。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。贪心不变量通常是当前方案在相同选择次数下不落后于任何其他方案，或者当

前保留的资源最多、结束最早、右边界最小。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。常见证明有交换论证和领先性论证。交换论证说明最优解可以被替换成贪心第一步而不变差；领先性论证说明贪心每一步都至少和任何方案一样好。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。实现时先把比较器写成明确的业务顺序。区间问题注意起点相同时终点升序还是降序；覆盖题注意闭区间和半开区间的等号。本题还要特别注意：单字符、所有字符相同、字符多次跨段。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：单字符、所有字符相同、字符多次跨段。这是最早合法切分贪心，能最大化片段数量。实验要造反例：如果把排序关键字改成另一个直觉规则，是否失败。能给出反例，才说明真正理解了贪心条件。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

区间、排序与合并

下面的题卡都按同一条链路展开：从问题出发，先写暴力，再定位重复工作，然后选择数据结构或状态不变量，最后落到 C++ 实现、边界和扩展。格式统一是为了训练面试表达，而每张卡的关键观察和失效条件都要单独复盘。

图示：区间、排序与合并推导链

暴力基线	枚举候选、完整模拟或逐个检查，先保证覆盖全部可能答案。
瓶颈定位	瓶颈是没有利用排序后相邻关系。区间按起点或终点排序后，很多远离的区间不可能再影响当前决策。
结构选择	合并按起点排序，调度按终点排序，覆盖题常用起点升序且终点降序。扫描时维护当前最远右端、结果尾区间或已选区间终点。
不变量	结果数组中的区间已经互不重叠且有序；当前扫描区间只可能影响结果最后一个区间或当前边界。
代码落地	比较器必须处理相同起点，否则覆盖和去重会错。区间端点可能溢出时用更大整数。闭区间和半开区间决定是否用小于等于。

56 合并区间

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。排序后可能与当前区间相交的只会是结果尾部。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“区间、排序与合并”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力做法对每个区间和其他所有区间比较相交、覆盖或冲突，复杂度通常是平方级。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。瓶颈是没有利用排序后相邻关系。区间按起点或终点排序后，很多远离的区间不可能再影响当前决策。放到本题里，要重点观察“按起点排序，扫描时维护结果最后区间的右端点。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。合并按起点排序，调度按终点排序，覆盖题常用起点升序且终点降序。扫描时维护当前最远右端、结果尾区间或已选区间终点。本

题的优化应当围绕“按起点排序，扫描时维护结果最后区间的右端点。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。结果数组中的区间已经互不重叠且有序；当前扫描区间只可能影响结果最后一个区间或当前边界。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。排序保证所有可能与当前区间冲突的候选已经聚集在附近。贪心按最早结束选择，可以用交换论证证明不减少可选数量。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。比较器必须处理相同起点，否则覆盖和去重会错。区间端点可能溢出时用更大整数。闭区间和半开区间决定是否用小于等于。本题还要特别注意：端点相接、完全覆盖、无重叠、空列表、输入未排序。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：端点相接、完全覆盖、无重叠、空列表、输入未排序。若区间是半开区间，端点相等不一定合并。实验包含完全覆盖、端点相接、同起点不同终点、负端点、单区间、空区间列表和排序比较器反例。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

57 插入区间

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。新区间只会影响与它相交的一段连续区间。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“区间、排序与合并”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力做法对每个区间和其他所有区间比较相交、覆盖或冲突，复杂度通常是平方级。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。瓶颈是没有利用排序后相邻关系。区间按起点或终点排序后，很多远离的区间不可能再影响当前决策。放到本题里，要重点观察“先追加左侧完全不相交区间，再合并重叠段，最后追加右侧。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。合并按起点排序，调度按终点排序，覆盖题常用起点升序且终点降序。扫描时维护当前最远右端、结果尾区间或已选区间终点。本题的优化应当围绕“先追加左侧完全不相交区间，再合并重叠段，最后追加右侧。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。结果数组中的区间已经互不重叠且有序；当前扫描区间只可能影响结果最后一个区间或当前边界。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。排序保证所有可能与当前区间冲突的候选已经聚集在附近。贪心按最早结束选择，可以用交换论证证明不减少可选数量。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。比较器必须处理相同起点，否则覆盖和去重会错。区间端点可能溢出时用更大整数。

闭区间和半开区间决定是否用小于等于。本题还要特别注意：新区间在最前、最后、覆盖全部、刚好相邻、原列表为空。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：新区间在最前、最后、覆盖全部、刚好相邻、原列表为空。若输入不是有序且不重叠，必须先排序或换模型。实验包含完全覆盖、端点相接、同起点不同终点、负端点、单区间、空区间列表和排序比较器反例。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

1288 删除被覆盖区间

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。被覆盖要求另一区间起点不晚且终点不早。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“区间、排序与合并”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力做法对每个区间和其他所有区间比较相交、覆盖或冲突，复杂度通常是平方级。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。瓶颈是没有利用排序后相邻关系。区间按起点或终点排序后，很多远离的区间不可能再影响当前决策。放到本题里，要重点观察“按起点升序、终点降序排序，扫描维护最大右端。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。合并按起点排序，调度按终点排序，覆盖题常用起点升序且终点降序。扫描时维护当前最远右端、结果尾区间或已选区间终点。本题的优化应当围绕“按起点升序、终点降序排序，扫描维护最大右端。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。结果数组中的区间已经互不重叠且有序；当前扫描区间只可能影响结果最后一个区间或当前边界。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。排序保证所有可能与当前区间冲突的候选已经聚集在附近。贪心按最早结束选择，可以用交换论证证明不减少可选数量。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。比较器必须处理相同起点，否则覆盖和去重会错。区间端点可能溢出时用更大整数。闭区间和半开区间决定是否用小于等于。本题还要特别注意：同起点区间、完全覆盖、无覆盖、端点相等。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：同起点区间、完全覆盖、无覆盖、端点相等。终点降序是关键，否则同起点小区间会先出现造成误判。实验包含完全覆盖、端点相接、同起点不同终点、负端点、单区间、空区间列表和排序比较器反例。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

回溯、枚举与剪枝

下面的题卡都按同一条链路展开：从问题出发，先写暴力，再定位重复工作，然后选择数据结构或状态不变量，最后落到 C++ 实现、边界和扩展。格式统一是为了训练面试表达，而每张卡的关键观察和失效条件都要单独复盘。

图示：回溯、枚举与剪枝推导链

暴力基线	枚举候选、完整模拟或逐个检查，先保证覆盖全部可能答案。
瓶颈定位	慢点在于大量分支在很早就已经不可能成功，仍然被继续展开。重复元素还会产生等价排列或组合。
结构选择	剪枝来自容量、剩余长度、排序后去重、合法性预检查和提前终止。组合题关注起点推进，排列题关注元素是否使用，分割题关注当前片段是否合法。
不变量	路径数组保存已经做出的选择；起点下标保证组合不回头；同层去重保证同一个位置层级不会选择相同值；已使用数组保证排列不重复使用同一元素。
代码落地	回溯代码虽然天然递归，但工程中要意识到深度。题目规模较小时递归可读性好；若要遵守严格工程栈限制，可以改成显式栈状态机。

78 子集

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。每个元素都有选或不选两种选择，答案是所有路径节点。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“回溯、枚举与剪枝”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力回溯会枚举整个搜索树。它不是错误，而是必须先写清楚选择列表、路径状态和终止条件，再讨论剪枝。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。慢点在于大量分支在很早就已经不可能成功，仍然被继续展开。重复元素还会产生等价排列或组合。放到本题里，要重点观察“回溯按起点向后枚举，避免不同顺序生成同一集合。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。剪枝来自容量、剩余长度、排序后去重、合法性预检查和提前终止。组合题关注起点推进，排列题关注元素是否使用，分割题关注当前片段是否合法。本题的优化应当围绕“回溯按起点向后枚举，避免不同顺序生成同一集合。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。路径数组保存已经做出的选择；起点下标保证组合不回头；同层去重保证同一个位置层级不会选择相同值；已使用数组保证排列不重复使用同一元素。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。正确性要说明搜索树覆盖所有合法答案且每个答案只生成一次。剪枝条件必须只删除不可能通向答案的分支。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。回溯代码虽然天然递归，但工程中要意识到深度。题目规模较小时递归可读性好；若要遵守严格工程栈限制，可以改成显式栈状态机。本题还要特别注意：空数组、单元素、输出数量为二的 n 次方、顺序不要求。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极

值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：空数组、单元素、输出数量为二的 n 次方、顺序不要求。也可用位掩码枚举所有子集，适合 n 较小的场景。实验包括空结果、重复元素、目标过小、目标刚好、字符串边界、棋盘第一行无解和剪枝前后节点数对比。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

79 单词搜索

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。网格路径不能重复使用同一格，状态包含位置、匹配下标和访问标记。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“回溯、枚举与剪枝”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力回溯会枚举整个搜索树。它不是错误，而是必须先写清楚选择列表、路径状态和终止条件，再讨论剪枝。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。慢点在于大量分支在很早就已经不可能成功，仍然被继续展开。重复元素还会产生等价排列或组合。放到本题里，要重点观察“剪枝包括首字符不匹配、剩余长度不足、字符频次不足和越界访问。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。剪枝来自容量、剩余长度、排序后去重、合法性预检查和提前终止。组合题关注起点推进，排列题关注元素是否使用，分割题关注当前片段是否合法。本题的优化应当围绕“剪枝包括首字符不匹配、剩余长度不足、字符频次不足和越界访问。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。路径数组保存已经做出的选择；起点下标保证组合不回头；同层去重保证同一个位置层级不会选择相同值；已使用数组保证排列不重复使用同一元素。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。正确性要说明搜索树覆盖所有合法答案且每个答案只生成一次。剪枝条件必须只删除不可能通向答案的分支。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。回溯代码虽然天然递归，但工程中要意识到深度。题目规模较小时递归可读性好；若要遵守严格工程栈限制，可以改成显式栈状态机。本题还要特别注意：单字符单词、路径需要回退、重复字母、单元格不能复用。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：单字符单词、路径需要回退、重复字母、单元格不能复用。若要查很多单词，应使用 Trie 合并搜索前缀。实验包括空结果、重复元素、目标过小、目标刚好、字符串边界、棋盘第一行无解和剪枝前后节点数对比。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

90 子集 II

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。重复元素会让不同下标路径生成相同集合。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“回溯、枚举与剪枝”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力回溯会枚举整个搜索树。它不是错误，而是必须先写清楚选择列表、路径状态和终止条件，再讨论剪枝。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。慢点在于大量分支在很早就已经不可能成功，仍然被继续展开。重复元素还会产生等价排列或组合。放到本题里，要重点观察“排序后在同一层跳过相同值，保留不同层使用重复值的可能。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。剪枝来自容量、剩余长度、排序后去重、合法性预检查和提前终止。组合题关注起点推进，排列题关注元素是否使用，分割题关注当前片段是否合法。本题的优化应当围绕“排序后在同一层跳过相同值，保留不同层使用重复值的可能。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。路径数组保存已经做出的选择；起点下标保证组合不回头；同层去重保证同一个位置层级不会选择相同值；已使用数组保证排列不重复使用同一元素。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。正确性要说明搜索树覆盖所有合法答案且每个答案只生成一次。剪枝条件必须只删除不可能通向答案的分支。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。回溯代码虽然天然递归，但工程中要意识到深度。题目规模较小时递归可读性好；若要遵守严格工程栈限制，可以改成显式栈状态机。本题还要特别注意：全重复、空集、重复值相邻、去重条件写成同层而不是全局。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：全重复、空集、重复值相邻、去重条件写成同层而不是全局。组合总和 II 使用同样的同层去重思想。实验包括空结果、重复元素、目标过小、目标刚好、字符串边界、棋盘第一行无解和剪枝前后节点数对比。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

131 分割回文串

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。每次选择一个从当前位置开始的回文前缀。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“回溯、枚举与剪枝”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力回溯会枚举整个搜索树。它不是错误，而是必须先写清楚选择列表、路径状态和终止条件，再讨论剪枝。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。慢点在于大量分支在很早就已经不可能成功，仍然被继续展开。重复元素还会产生等价排列或组合。放到本题里，要重点观察“预处理回文 DP 表，让回溯合法性检查为常数。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。剪枝来自容量、剩余长度、排序后去重、合法性预检查和提前终止。组合题关注起点推进，排列题关注元素是否使用，分割题关注当前片段是否合法。本题的优化应当围绕“预处理回文 DP 表，让回溯合法性检查为常数。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。路径数组保存已经做出的选择；起点下标保证组合不回头；同层去重保证同一个位置层级不会选择相同值；已使用数组保证排列不重复使用同一元素。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。正确性要说明搜索树覆盖所有合法答案且每个答案只生成一次。剪枝条件必须只删除不可能通向答案的分支。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。回溯代码虽然天然递归，但工程中要意识到深度。题目规模较小时递归可读性好；若要遵守严格工程栈限制，可以改成显式栈状态机。本题还要特别注意：空串、单字符、全相同字符、重复分割路径。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：空串、单字符、全相同字符、重复分割路径。若只求最少切割次数，可改成 DP 最短切分。实验包括空结果、重复元素、目标过小、目标刚好、字符串边界、棋盘第一行无解和剪枝前后节点数对比。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

216 组合总和 III

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。从一到九中选 k 个数和为 n ，每个数最多用一次。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“回溯、枚举与剪枝”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力回溯会枚举整个搜索树。它不是错误，而是必须先写清楚选择列表、路径状态和终止条件，再讨论剪枝。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。慢点在于大量分支在很早就已经不可能成功，仍然被继续展开。重复元素还会产生等价排列或组合。放到本题里，要重点观察“按递增起点枚举，利用剩余个数和剩余和剪枝。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。剪枝来自容量、剩余长度、排序后去重、合法性预检查和提前终止。组合题关注起点推进，排列题关注元素是否使用，分割题关注当前片段是否合法。本题的优化应当围绕“按递增起点枚举，利用剩余个数和剩余和剪枝。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，

例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。路径数组保存已经做出的选择；起点下标保证组合不回头；同层去重保证同一个位置层级不会选择相同值；已使用数组保证排列不重复使用同一元素。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。正确性要说明搜索树覆盖所有合法答案且每个答案只生成一次。剪枝条件必须只删除不可能通向答案的分支。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。回溯代码虽然天然递归，但工程中要意识到深度。题目规模较小时递归可读性好；若要遵守严格工程栈限制，可以改成显式栈状态机。本题还要特别注意： n 太小、 n 太大、 k 为零、路径长度已经超过 k 。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是： n 太小、 n 太大、 k 为零、路径长度已经超过 k 。可预估剩余最小和最大和进一步剪枝。实验包括空结果、重复元素、目标过小、目标刚好、字符串边界、棋盘第一行无解和剪枝前后节点数对比。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

堆与动态极值

下面的题卡都按同一条链路展开：从问题出发，先写暴力，再定位重复工作，然后选择数据结构或状态不变量，最后落到 C++ 实现、边界和扩展。格式统一是为了训练面试表达，而每张卡的关键观察和失效条件都要单独复盘。

图示：堆与动态极值推导链

暴力基线	枚举候选、完整模拟或逐个检查，先保证覆盖全部可能答案。
瓶颈定位	慢点在于动态极值查询。我们只需要当前最极端元素，不需要整个集合完全有序。
结构选择	堆把插入和弹出极值控制在对数复杂度。Top K 用固定大小堆维护边界，合并多路序列用堆保存每一路当前头部，数据流中位数用双堆维护两半。
不变量	堆不变量不是全局有序，而是堆顶代表当前最优候选。若存在过期元素，需要在弹出时懒删除，保证真正使用堆顶前它仍然有效。
代码落地	C++ 的 <code>std::priority_queue</code> 默认大顶堆。小顶堆用 <code>std::greater<T></code> ；结构体比较器要按谁优先级更低来写。堆中保存大对象时尽量保存下标或指针。

215 数组中的第 K 个最大元素

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。只关心第 K 大，不需要完整排序。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“堆与动态极值”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力做法每轮扫描所有候选找最大或最小，或者每次变化后重新排序。若候选集合动态变化，这会把大量旧排序工作丢掉。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。慢点在于动态极值查询。我们只需要当前最极端元素，不需要整个集合完全有序。放到本题

里，要重点观察“大小为 K 的小顶堆保存当前前 K 大边界；快速选择可做到平均线性。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。堆把插入和弹出极值控制在对数复杂度。Top K 用固定大小堆维护边界，合并多路序列用堆保存每一路当前头部，数据流中位数用双堆维护两半。本题的优化应当围绕“大小为 K 的小顶堆保存当前前 K 大边界；快速选择可做到平均线性。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。堆不变量不是全局有序，而是堆顶代表当前最优候选。若存在过期元素，需要在弹出时懒删除，保证真正使用堆顶前它仍然有效。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。正确性常用边界候选证明：被堆丢弃的元素不可能进入答案，仍在堆中的元素覆盖所有可能的下一步选择。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。C++ 的 `std::priority_queue` 默认大顶堆。小顶堆用 `std::greater<T>`；结构体比较器要按谁优先级更低来写。堆中保存大对象时尽量保存下标或指针。本题还要特别注意： K 为一、 K 为 n 、重复值、随机 pivot。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是： K 为一、 K 为 n 、重复值、随机 pivot。若数据流持续到来，堆方案比一次性快速选择更自然。实验要验证 K 为一、 K 等于元素数、频次相同、堆顶过期、多路序列某一路为空、偶数个数据求中位数等边界。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

295 数据流的中位数

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。数据流需要动态维护较小一半和较大一半。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“堆与动态极值”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力做法每轮扫描所有候选找最大或最小，或者每次变化后重新排序。若候选集合动态变化，这会把大量旧排序工作丢掉。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。慢点在于动态极值查询。我们只需要当前最极端元素，不需要整个集合完全有序。放到本题里，要重点观察“大顶堆保存较小一半，小顶堆保存较大一半，大小差不超过一。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。堆把插入和弹出极值控制在对数复杂度。Top K 用固定大小堆维护边界，合并多路序列用堆保存每一路当前头部，数据流中位数用双堆维护两半。本题的优化应当围绕“大顶堆保存较小一半，小顶堆保存较大一半，大小差不超过一。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。堆不变量不是全局有序，而是堆顶代表当前最优候选。若存在过期元素，需要在弹出时懒删除，保证真正使用堆顶前它仍然有效。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。正确性常用边界候选证明：被堆丢弃的元素不可能进入答案，仍在堆中的元素覆盖所有可能的下一步选择。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。C++ 的 `std::priority_queue` 默认大顶堆。小顶堆用 `std::greater<T>`；结构体比较器要按谁优先级更低来写。堆中保存大对象时尽量保存下标或指针。本题还要特别注意：偶数个元素、负数、重复值、平均值溢出。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：偶数个元素、负数、重复值、平均值溢出。若还要删除任意元素，需要延迟删除哈希表。实验要验证 K 为一、 K 等于元素数、频次相同、堆顶过期、多路序列某一路为空、偶数个数据求中位数等边界。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

347 前 K 个高频元素

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。先统计频次，再只维护前 K 个候选。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“堆与动态极值”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力做法每轮扫描所有候选找最大或最小，或者每次变化后重新排序。若候选集合动态变化，这会把大量旧排序工作丢掉。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。慢点在于动态极值查询。我们只需要当前最极端元素，不需要整个集合完全有序。放到本题里，要重点观察“小顶堆大小超过 K 就弹出最低频；桶排序利用频次上界。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。堆把插入和弹出极值控制在对数复杂度。Top K 用固定大小堆维护边界，合并多路序列用堆保存每一路当前头部，数据流中位数用双堆维护两半。本题的优化应当围绕“小顶堆大小超过 K 就弹出最低频；桶排序利用频次上界。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。堆不变量不是全局有序，而是堆顶代表当前最优候选。若存在过期元素，需要在弹出时懒删除，保证真正使用堆顶前它仍然有效。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。正确性常用边界候选证明：被堆丢弃的元素不可能进入答案，仍在堆中的元素覆盖所有可能的下一步选择。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。C++ 的 `std::priority_queue` 默认大顶堆。小顶堆用 `std::greater<T>`；结构体比较器要按谁优先级更低来写。堆中保存大对象时尽量保存下标或指针。本题还要特别注意： K 等于不同元素数、频次相同、结果顺序不要求。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就

别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：K 等于不同元素数、频次相同、结果顺序不要求。若要按频次和字典序排序，比较器要同时处理两个键。实验要验证 K 为一、K 等于元素数、频次相同、堆顶过期、多路序列某一路为空、偶数个数据求中位数等边界。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

621 任务调度器

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。相同任务之间有冷却间隔，核心受最高频任务骨架限制。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“堆与动态极值”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力做法每轮扫描所有候选找最大或最小，或者每次变化后重新排序。若候选集合动态变化，这会把大量旧排序工作丢掉。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。慢点在于动态极值查询。我们只需要当前最极端元素，不需要整个集合完全有序。放到本题里，要重点观察“可用大顶堆模拟，也可用最高频公式计算最短时间。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。堆把插入和弹出极值控制在对数复杂度。Top K 用固定大小堆维护边界，合并多路序列用堆保存每一路当前头部，数据流中位数用双堆维护两半。本题的优化应当围绕“可用大顶堆模拟，也可用最高频公式计算最短时间。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。堆不变量不是全局有序，而是堆顶代表当前最优候选。若存在过期元素，需要在弹出时懒删除，保证真正使用堆顶前它仍然有效。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。正确性常用边界候选证明：被堆丢弃的元素不可能进入答案，仍在堆中的元素覆盖所有可能的下一步选择。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。C++ 的 `std::priority_queue` 默认大顶堆。小顶堆用 `std::greater<T>`；结构体比较器要按谁优先级更低来写。堆中保存大对象时尽量保存下标或指针。本题还要特别注意：冷却为零、多个最高频、空闲被其他任务填满。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：冷却为零、多个最高频、空闲被其他任务填满。若任务有不同释放时间或执行时长，就变成更一般的调度问题。实验要验证 K 为一、K 等于元素数、频次相同、堆顶过期、多路序列某一路为空、偶数个数据求中位数等边界。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

973 最接近原点的 K 个点

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。距离只需比较平方，避免开方误差和成本。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“堆与动态极值”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力做法每轮扫描所有候选找最大或最小，或者每次变化后重新排序。若候选集合动态变化，这会把大量旧排序工作丢掉。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。慢点在于动态极值查询。我们只需要当前最极端元素，不需要整个集合完全有序。放到本题里，要重点观察“大小为 K 的大顶堆保存当前最近 K 个点中最远者。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。堆把插入和弹出极值控制在对数复杂度。Top K 用固定大小堆维护边界，合并多路序列用堆保存每一路当前头部，数据流中位数用双堆维护两半。本题的优化应当围绕“大小为 K 的大顶堆保存当前最近 K 个点中最远者。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。堆不变量不是全局有序，而是堆顶代表当前最优候选。若存在过期元素，需要在弹出时懒删除，保证真正使用堆顶前它仍然有效。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。正确性常用边界候选证明：被堆丢弃的元素不可能进入答案，仍在堆中的元素覆盖所有可能的下一步选择。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。C++ 的 `std::priority_queue` 默认大顶堆。小顶堆用 `std::greater<T>`；结构体比较器要按谁优先级更低来写。堆中保存大对象时尽量保存下标或指针。本题还要特别注意：K 为一、K 为全部、距离相同、坐标平方溢出。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：K 为一、K 为全部、距离相同、坐标平方溢出。快速选择可平均线性，但堆更适合数据流。实验要验证 K 为一、K 等于元素数、频次相同、堆顶过期、多路序列某一路为空、偶数个数据求中位数等边界。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

链表与指针重连

下面的题卡都按同一条链路展开：从问题出发，先写暴力，再定位重复工作，然后选择数据结构或状态不变量，最后落到 C++ 实现、边界和扩展。格式统一是为了训练面试表达，而每张卡的关键观察和失效条件都要单独复盘。

图示：链表与指针重连推导链

暴力基线	枚举候选、完整模拟或逐个检查，先保证覆盖全部可能答案。
瓶颈定位	链表慢点在于不能随机访问。要找倒数位置、分组边界或交点，只能通过指针间距离、快慢指针或哈希记录访问过的节点。
结构选择	优化来自哨兵节点、快慢指针、前驱指针和局部反转。链表题的核心不是值交换，而是保持断开和接回的顺序正确。
不变量	必须明确每个指针指向哪一段：已经处理好的前缀头尾、当前待处理节点、下一段头节点。每次改 <code>next</code> 前先保存后继。
代码落地	节点通常由评测平台管理，代码不要随意 <code>delete</code> 输入节点。若创建新节点，要说明所有权。比较交点时比较地址，不比较值。

2 两数相加

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。链表按低位到高位保存数字，本质是竖式加法而不是整数转换。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“链表与指针重连”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力做法常把链表拷到数组里处理，再写回或重建。这样虽然降低指针难度，却失去原地节点重连的训练价值。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。链表慢点在于不能随机访问。要找倒数位置、分组边界或交点，只能通过指针间距离、快慢指针或哈希记录访问过的节点。放到本题里，要重点观察“维护两个游标和进位，任一链未结束或进位非零都继续生成节点。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。优化来自哨兵节点、快慢指针、前驱指针和局部反转。链表题的核心不是值交换，而是保持断开和接回的顺序正确。本题的优化应当围绕“维护两个游标和进位，任一链未结束或进位非零都继续生成节点。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。必须明确每个指针指向哪一段：已经处理好的前缀头尾、当前待处理节点、下一段头节点。每次改 `next` 前先保存后继。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。正确性靠结构不变量：已经反转或合并的部分保持合法链，未处理部分仍可达，二者之间只有一个明确连接点。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。节点通常由评测平台管理，代码不要随意 `delete` 输入节点。若创建新节点，要说明所有权。比较交点时比较地址，不比较值。本题还要特别注意：两链长度不同、最后进位、输入为零、不能用内置整数承载超长数字。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：两链长度不同、最后进位、输入为零、不能用内置整数承载超长数字。若链表高位在前，可以用栈反向处理或先反转链表。实验覆盖空链表、单节点、删除头尾、长度不足一组、奇偶长度、两链不相交、有环和重复值但节点不同等情况。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

141 环形链表

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。快慢指针在有环时必然相遇，无环时快指针先到空。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“链表与指针重连”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力做法常把链表拷到数组里处理，再写回或重建。这样虽然降低指针难度，却失去原地节点重连的训练价值。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。链表慢点在于不能随机访问。要找倒数位置、分组边界或交点，只能通过指针间距离、快慢指针或哈希记录访问过的节点。放到本题里，要重点观察“不用哈希表也能常数空间检测环。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。优化来自哨兵节点、快慢指针、前驱指针和局部反转。链表题的核心不是值交换，而是保持断开和接回的顺序正确。本题的优化应当围绕“不用哈希表也能常数空间检测环。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。必须明确每个指针指向哪一段：已经处理好的前缀头尾、当前待处理节点、下一段头节点。每次改 `next` 前先保存后继。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。正确性靠结构不变量：已经反转或合并的部分保持合法链，未处理部分仍可达，二者之间只有一个明确连接点。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。节点通常由评测平台管理，代码不要随意 `delete` 输入节点。若创建新节点，要说明所有权。比较交点时比较地址，不比较值。本题还要特别注意：空链、单节点自环、两节点环、尾部无环。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：空链、单节点自环、两节点环、尾部无环。若要求环入口，用相遇后双指针同步走。实验覆盖空链表、单节点、删除头尾、长度不足一组、奇偶长度、两链不相交、有环和重复值但节点不同等情况。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

142 环形链表 II

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。相遇点到入口的距离关系来自快慢指针速度差。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“链表与指针重连”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力做法常把链表拷到数组里处理，再写回或重建。这样虽然降低指针难度，却失去原地节点重连的训练价值。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新

取得。链表慢点在于不能随机访问。要找倒数位置、分组边界或交点，只能通过指针间距离、快慢指针或哈希记录访问过的节点。放到本题里，要重点观察“相遇后一个指针回头，两个指针每次走一步，再次相遇就是入口。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。优化来自哨兵节点、快慢指针、前驱指针和局部反转。链表题的核心不是值交换，而是保持断开和接回的顺序正确。本题的优化应当围绕“相遇后一个指针回头，两个指针每次走一步，再次相遇就是入口。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。必须明确每个指针指向哪一段：已经处理好的前缀头尾、当前待处理节点、下一段头节点。每次改 `next` 前先保存后继。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。正确性靠结构不变量：已经反转或合并的部分保持合法链，未处理部分仍可达，二者之间只有一个明确连接点。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。节点通常由评测平台管理，代码不要随意 `delete` 输入节点。若创建新节点，要说明所有权。比较交点时比较地址，不比较值。本题还要特别注意：入口在头节点、长尾短环、无环、单节点环。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：入口在头节点、长尾短环、无环、单节点环。也可用哈希集合，但空间更高。实验覆盖空链表、单节点、删除头尾、长度不足一组、奇偶长度、两链不相交、有环和重复值但节点不同等情况。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

146 LRU 缓存

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。哈希表负责按 `key` 定位，双向链表负责维护新旧顺序。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“链表与指针重连”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力做法常把链表拷到数组里处理，再写回或重建。这样虽然降低指针难度，却失去原地节点重连的训练价值。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。链表慢点在于不能随机访问。要找倒数位置、分组边界或交点，只能通过指针间距离、快慢指针或哈希记录访问过的节点。放到本题里，要重点观察“访问和更新都把节点移动到头部，容量满时删除尾部最旧节点。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。优化来自哨兵节点、快慢指针、前驱指针和局部反转。链表题的核心不是值交换，而是保持断开和接回的顺序正确。本题的优化应当围绕“访问和更新都把节点移动到头部，容量满时删除尾部最旧节点。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。必须明确每个指针指向哪一段：已经处理好的前缀头尾、当前待处理节点、下

一段头节点。每次改 `next` 前先保存后继。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。正确性靠结构不变量：已经反转或合并的部分保持合法链，未处理部分仍可达，二者之间只有一个明确连接点。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。节点通常由评测平台管理，代码不要随意 `delete` 输入节点。若创建新节点，要说明所有权。比较交点时比较地址，不比较值。本题还要特别注意：容量为一、重复 `put`、`get` 不存在、更新已有值。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：容量为一、重复 `put`、`get` 不存在、更新已有值。C++ 可用 `std::list` 加迭代器，也可手写双向链表理解所有权。实验覆盖空链表、单节点、删除头尾、长度不足一组、奇偶长度、两链不相交、有环和重复值但节点不同等情况。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

148 排序链表

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。链表不能随机访问，适合归并排序而不是快速排序数组 `partition`。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“链表与指针重连”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力做法常把链表拷到数组里处理，再写回或重建。这样虽然降低指针难度，却失去原地节点重连的训练价值。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。链表慢点在于不能随机访问。要找倒数位置、分组边界或交点，只能通过指针间距离、快慢指针或哈希记录访问过的节点。放到本题里，要重点观察“自底向上迭代归并能避免递归栈，同时保持 $O(n \log n)$ 。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。优化来自哨兵节点、快慢指针、前驱指针和局部反转。链表题的核心不是值交换，而是保持断开和接回的顺序正确。本题的优化应当围绕“自底向上迭代归并能避免递归栈，同时保持 $O(n \log n)$ 。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。必须明确每个指针指向哪一段：已经处理好的前缀头尾、当前待处理节点、下一段头节点。每次改 `next` 前先保存后继。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。正确性靠结构不变量：已经反转或合并的部分保持合法链，未处理部分仍可达，二者之间只有一个明确连接点。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。节点通常由评测平台管理，代码不要随意 `delete` 输入节点。若创建新节点，要说明所有权。比较交点时比较地址，不比较值。本题还要特别注意：空链、单节点、重复值、奇数长度、切断子链。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域

大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：空链、单节点、重复值、奇数长度、切断子链。若把值复制到数组排序，会改变节点身份语义。实验覆盖空链表、单节点、删除头尾、长度不足一组、奇偶长度、两链不相交、有环和重复值但节点不同等情况。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

160 相交链表

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。相交是节点地址相同，不是节点值相同。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“链表与指针重连”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力做法常把链表拷到数组里处理，再写回或重建。这样虽然降低指针难度，却失去原地节点重连的训练价值。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。链表慢点在于不能随机访问。要找倒数位置、分组边界或交点，只能通过指针间距离、快慢指针或哈希记录访问过的节点。放到本题里，要重点观察“两个指针走完各自链表后切换到另一条链，走过总长度相同后相遇。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。优化来自哨兵节点、快慢指针、前驱指针和局部反转。链表题的核心不是值交换，而是保持断开和接回的顺序正确。本题的优化应当围绕“两个指针走完各自链表后切换到另一条链，走过总长度相同后相遇。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。必须明确每个指针指向哪一段：已经处理好的前缀头尾、当前待处理节点、下一段头节点。每次改 `next` 前先保存后继。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。正确性靠结构不变量：已经反转或合并的部分保持合法链，未处理部分仍可达，二者之间只有一个明确连接点。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。节点通常由评测平台管理，代码不要随意 `delete` 输入节点。若创建新节点，要说明所有权。比较交点时比较地址，不比较值。本题还要特别注意：不相交、头节点相交、尾节点相交、长度差很大。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：不相交、头节点相交、尾节点相交、长度差很大。哈希集合更直观但空间更高。实验覆盖空链表、单节点、删除头尾、长度不足一组、奇偶长度、两链不相交、有环和重复值但节点不同等情况。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

206 反转链表

从问题出发。先不要问“这题套什么模板”，而要把题面翻译成对象、关系、约束和输出目标。每次把当前节点的 `next` 指向已经反转好的前缀头。对象决定我们要保存数组下标、节点、区间、字符还是图状态；关系决定候选之间能不能比较、合并或淘汰；输出目标决定最后保存的是最值、计数、路径、集合还是判定结果。把这些拆开以后，才进入“链表与指针重连”这条主线。

先把暴力写清楚。按上面的模型，最直接的办法是完整枚举可能答案或完整模拟题目过程，再对每个候选重新检查是否合法。暴力做法常把链表拷到数组里处理，再写回或重建。这样虽然降低指针难度，却失去原地节点重连的训练价值。写暴力不是为了提交它，而是为了暴露状态空间：哪些候选一定会出现，哪些判断被反复做，哪些中间量其实只和少量历史状态有关。只有这一步说清楚，后面的优化才不是凭空跳出来的答案。

瓶颈定位。暴力版本跑慢，通常不是因为代码写得不够巧，而是因为同一类信息被一次次重新取得。链表慢点在于不能随机访问。要找倒数位置、分组边界或交点，只能通过指针间距离、快慢指针或哈希记录访问过的节点。放到本题里，要重点观察“先保存后继，再改指针，再推进 previous 和 current。”这条线索：它说明有一部分信息可以从已经处理过的输入中继承下来，而不需要回头重算。这里的关键观察不是结论，而是从暴力过程里提炼出来的重复工作。

结构选择和优化。既然瓶颈已经定位，下一步才是选择结构。优化来自哨兵节点、快慢指针、前驱指针和局部反转。链表题的核心不是值交换，而是保持断开和接回的顺序正确。本题的优化应当围绕“先保存后继，再改指针，再推进 previous 和 current。”建立一个可维护状态：新增输入只更新一次，过期候选及时删除，已经稳定的子答案被合并使用。优化后的循环不应重新审查全部输入，而应只处理这一步真正变化的那一小块状态。若题目条件发生变化，例如有序性、正数条件、字符集、边权或是否允许修改输入发生改变，就必须重新验证这个结构是否仍然成立。

不变量和证明。必须明确每个指针指向哪一段：已经处理好的前缀头尾、当前待处理节点、下一段头节点。每次改 next 前先保存后继。用这道题复盘时，把不变量写成三句话：已经处理的部分满足什么，尚未处理的部分还保留什么可能性，当前操作为什么不会丢掉最优答案或合法答案。正确性靠结构不变量：已经反转或合并的部分保持合法链，未处理部分仍可达，二者之间只有一个明确连接点。证明不需要很形式化，但必须能排除反例；如果只能说“这样写好像可以”，说明你还没有把优化从暴力中推导出来。

代码落地。节点通常由评测平台管理，代码不要随意 delete 输入节点。若创建新节点，要说明所有权。比较交点时比较地址，不比较值。本题还要特别注意：空链、单节点、两个节点、不要丢失剩余链。写代码前先把边界条件放进函数入口、循环条件或状态定义里，而不是写完主体后再用补丁式分支修样例。容器选择要和访问模式一致：需要顺序就别用无序表，需要极值就别完整排序，需要历史计数就别只保存布尔值。

复杂度、边界和实验。时间复杂度要按真实输入维度说明，不能把字符串长度、图边数、值域大小都混成一个 n 。空间复杂度也要区分辅助结构和输出本身。测试时至少覆盖：最小输入、没有答案、答案在边界、重复元素或重复状态、以及会让暴力解最慢的规模输入。对于这题，边界复盘重点是：空链、单节点、两个节点、不要丢失剩余链。K 组反转是在这段局部反转外增加分组边界。实验覆盖空链表、单节点、删除头尾、长度不足一组、奇偶长度、两链不相交、有环和重复值但节点不同等情况。实验报告建议记录三件事：暴力版本在什么规模开始变慢，优化版本减少了哪类重复工作，哪一个边界用例最容易打破错误实现。

动态规划证明、背包和区间 DP

动态规划不是数组技巧，而是对子问题图的压缩求解。暴力搜索中，一个节点表示当前处理到哪里、已经做了哪些选择、剩余资源是什么。若不同路径到达同一个状态后，未来可选集合完全相同，就出现重叠子问题。DP 把这个状态命名并保存结果，再按依赖顺序计算。状态定义越稳定，代码越短；状态缺失一项，转移就会偷偷依赖历史细节。

线性 DP 常按最后一步思考。最大子数组和问以当前位置结尾的最佳后缀，打家劫舍问考虑前 i 个房屋的最大收益，股票题问每天结束时持有或不持有的最大收益。最后一步方法的价值在于把全局最优拆成互斥候选：当前元素单独开始还是接在前面，当前房屋偷还是不偷，今天买、卖还是休息。每个候选都必须对应一个已经计算好的状态。

二维字符串 DP 需要精确定义前缀。最长公共子序列的 $dp[i][j]$ 表示两个前缀的答案，不是以某个字符结尾的答案。编辑距离的 $dp[i][j]$ 表示把 word1 前 i 个字符变成 word2 前 j 个字符的最少操作。多出第零行和第零列不是为了方便而已，它们代表空串，给插入和删除操作提供统一边界。

背包题最容易背错循环顺序。0-1 背包中每个物品只能用一次，一维压缩时容量必须倒序，确保读取的是上一轮物品阶段的状态。完全背包中物品可无限使用，容量正序允许当前物品在同一轮被再次使用。组合计数外层遍历物品，排列计数外层遍历容量。循环顺序不是模板差异，而是状态语义差异。

目标和、分割等和子集和零钱兑换 II 是背包三连。分割等和是可达性，目标容量为总和一半；目标和通过代数变换把正负号选择转成选若干数凑出正集合和；零钱兑换 II 是完全背包组合计数。三题放在一起能训练四个问题：物品是否只能用一次，状态保存布尔值还是方案数，容量遍历正序还是倒序，答案是否受零元素影响。

区间 DP 的关键是按长度枚举。戳气球中，如果最后戳破区间内某个气球，那么它左右两侧已经分别被处理完，边界气球仍在；矩阵链乘法中，最后一次乘法把区间切成左右两段；回文相关问题中，外层字符关系决定内部区间是否有效。区间 DP 的状态通常是 $dp[left][right]$ ，依赖更短区间，因此要先枚举区间长度。

空间压缩必须能解释旧值来自哪里。二维 LCS 压成一维时， $dp[j]$ 在更新前是上一行同列， $dp[j-1]$ 是当前行左侧，左上角需要额外变量保存。若解释不清这三个来源，空间压缩很容易写错。初学阶段先写二维表，确认状态和转移，再压缩空间。

实验建议：把 0-1 背包容量循环改成正序，用一个物品重量为一、价值为一、容量为二的用例观察它被重复使用。把零钱兑换 II 外层内层调换，观察组合数变成排列数。给 LCS 打印整个 DP 表，再手动从右下角回溯出一个公共子序列，理解表中每个值的来源。

本节复盘

阅读本节后，请回到相邻章节的例题，把暴力瓶颈、优化依据、状态不变量、C++ 容器成本和边界测试重新写一遍。能把同一思想迁移到变体题，才算真正掌握。

第零部分

面试专项：从题型到工程表达

第 10 章

线性结构专项：链表、栈、队列和单调结构

线性结构专项题看上去都很短，但它们是面试中最容易暴露基本功的一类。数组和字符串通常考察下标、窗口和局部状态；链表考察节点关系和指针顺序；栈考察最近未解决状态；队列考察层次推进；单调栈和单调队列考察候选淘汰。很多候选人的代码错在一个小边界，但真正的原因不是粗心，而是没有说清楚“当前结构里保存的对象到底代表什么”。

链表题首先要把“值”和“节点”分开。数组里交换两个元素，很多时候只要交换值；链表里更重要的是节点之间的连接关系。删除节点、反转链表、合并链表、K 个一组翻转，都不是在移动一段连续内存，而是在改变若干个 `next` 指针。写链表题之前，建议先在纸上画三段：已经处理好的部分、当前正在处理的部分、尚未处理的部分。每次改指针前都问一句：原来的后继是否已经保存？新的前驱是谁？最终头节点从哪里返回？

删除倒数第 N 个节点的暴力想法是先遍历一遍得到长度 `len`，再删除第 `len - n + 1` 个节点。这个做法完全正确，复杂度也是 $O(n)$ ，但需要两次遍历。面试更常希望你展示快慢指针：让快指针先走 `n` 步，然后快慢一起走；当快指针到达尾部时，慢指针正好停在待删除节点的前驱。优化的本质不是少了一行代码，而是把“倒数位置”转换成“两个指针之间保持固定距离”的不变量。

Listing 10.1 删除倒数第 N 个节点：虚拟头节点和快慢指针

```
1 struct ListNode {
2     int value = 0;
3     ListNode* next = nullptr;
4 };
5
6 ListNode* remove_nth_from_end(ListNode* head, int n)
7 {
8     ListNode dummy;
9     dummy.next = head;
10
11     ListNode* fast = &dummy;
12     ListNode* slow = &dummy;
13
14     for (int step = 0; step < n; ++step) {
15         fast = fast->next;
16     }
17     while (fast->next != nullptr) {
18         fast = fast->next;
19         slow = slow->next;
20     }
21
22     ListNode* removed = slow->next;
23     slow->next = removed->next;
24     return dummy.next;
25 }
```

虚拟头节点的价值在这里非常明显。如果删除的是原来的头节点，没有虚拟头，就要单独写一段分支；有了虚拟头，删除头节点和删除中间节点统一成“修改某个前驱节点的 `next`”。这类题的面试表达要明确：快慢指针之间相隔 `n` 个真实节点，循环结束时快指针停在最后一个真实节点，慢指针停在待删节点前驱。边界要测三个：只有一个节点，删除头节点，删除尾节点。

合并两个有序链表是链表和双指针结合的题。暴力方法可以把两个链表的值全部放进数组，排序后重新建链表；这样会丢掉链表题真正考察的节点重连能力，也多用了 $O(n)$ 额外空间。更自然的做法是维护一个结果链表尾指针，每次从两个链表头部取较小节点接到尾部。由于两个输入链表本来有序，每一步的局部最小节点就是全局下一个节点。

Listing 10.2 合并两个有序链表：尾指针接入节点

```

1 ListNode* merge_two_lists(ListNode* first, ListNode* second)
2 {
3     ListNode dummy;
4     ListNode* tail = &dummy;
5
6     while (first != nullptr && second != nullptr) {
7         if (first->value <= second->value) {
8             tail->next = first;
9             first = first->next;
10        } else {
11            tail->next = second;
12            second = second->next;
13        }
14        tail = tail->next;
15    }
16
17    tail->next = first != nullptr ? first : second;
18    return dummy.next;
19 }

```

这段代码没有创建新节点，只是复用原链表节点。因此如果题目要求不能修改输入，就要改成新建节点版本。稳定性也值得说明：当两个节点值相等时，代码优先取 **first**，这会保持第一条链表中相等元素的相对顺序。复杂度是 $O(m+n)$ ，空间是 $O(1)$ 。错误高发点是忘记移动 **tail**，或者循环结束后忘记把剩余链表整体接上。

K 个一组翻转链表比普通反转多了“分组边界”。暴力想法是把所有节点放进数组，按 K 个一组交换指针或值，再重新连接；这样虽然容易写，但牺牲了链表题的原地处理能力。原地做法要先检查当前组是否有 K 个节点，不足 K 个就保持原样；足够 K 个时，只翻转这一段，并把上一组的尾部接到新头，把新尾接到下一组开头。

Listing 10.3 K 个一组翻转链表：先找边界再局部反转

```

1 ListNode* reverse_k_group(ListNode* head, int k)
2 {
3     if (k <= 1 || head == nullptr) {
4         return head;
5     }
6
7     ListNode dummy;
8     dummy.next = head;
9     ListNode* group_previous = &dummy;
10
11    while (true) {
12        ListNode* kth = group_previous;
13        for (int step = 0; step < k && kth != nullptr; ++step) {
14            kth = kth->next;
15        }
16        if (kth == nullptr) {

```

```

17         break;
18     }
19
20     ListNode* group_next = kth->next;
21     ListNode* previous = group_next;
22     ListNode* current = group_previous->next;
23
24     while (current != group_next) {
25         ListNode* next = current->next;
26         current->next = previous;
27         previous = current;
28         current = next;
29     }
30
31     ListNode* new_group_tail = group_previous->next;
32     group_previous->next = kth;
33     group_previous = new_group_tail;
34 }
35
36 return dummy.next;
37 }

```

这题最重要的是命名。没有 `group_previous`、`kth`、`group_next` 这三个边界名，代码会变成一堆难以验证的指针操作。反转循环里让 `previous` 初始等于 `group_next`，是为了反转完成后原组头节点自然指向下一组开头。这个技巧能减少额外连接分支。面试时不要急着写代码，先画出一组翻转前后：上一组尾、当前组头、当前组尾、下一组头，四个位置说清楚后，代码才有依据。

栈题的核心是“最近未解决”。有效括号中，最近出现的左括号必须最先被匹配；路径简化中，最近进入的目录会被 `..` 回退；表达式求值中，最近的操作数和运算符构成局部归约。栈的优化不是为了降低所有问题的复杂度，而是把嵌套结构或最近依赖关系变成线性扫描。

最小栈是一个很好的设计题。只用一个普通栈保存元素，`top` 和 `pop` 都是常数时间，但 `getMin` 需要扫描所有元素。优化方法是额外维护一个同步最小值栈。每压入一个值，同时压入“截至当前栈顶的最小值”；弹出时两个栈一起弹。这样任意时刻最小值栈栈顶就是当前全部元素的最小值。

Listing 10.4 最小栈：数据栈和最小值栈同步

```

1 class MinStack {
2 public:
3     void push(int value)
4     {
5         this->values_.push_back(value);
6         if (this->minimums_.empty()) {
7             this->minimums_.push_back(value);
8         } else {
9             this->minimums_.push_back(std::min(value, this->minimums_.back()));
10        }
11    }
12
13    void pop()
14    {
15        this->values_.pop_back();
16        this->minimums_.pop_back();
17    }
18 }

```



```

19     int top() const
20     {
21         return this->values_.back();
22     }
23
24     int get_min() const
25     {
26         return this->minimums_.back();
27     }
28
29 private:
30     std::vector<int> values_;
31     std::vector<int> minimums_;
32 };

```

这段代码故意使用 `std::vector` 作为栈，因为 `vector` 支持连续存储、`back`、`push_back` 和 `pop_back`，足够表达栈语义。类成员访问使用 `this->`，避免成员和局部变量混淆。常见错误是只在新值更小时才压入最小值栈，但弹出重复最小值时就会错。例如依次压入 2, 2, 3，如果只记录一个 2，弹出一个 2 后最小值仍应是 2。同步栈写法用空间换清晰和可靠。

单调栈是“最近未解决”的强化版：栈里保存的不是所有历史元素，而是一组仍有可能成为答案的候选。每日温度中，栈保存还没找到更高温度的下标，且栈中温度从底到顶单调不增。新温度到来时，如果高于栈顶，就说明栈顶那一天的答案已经确定，因为当前天是它右侧第一个更高温度的位置。

柱状图中最大的矩形更能体现单调栈的威力。暴力做法枚举每根柱子作为矩形高度，再向左右扩展找到第一个更矮柱子，最坏 $O(n^2)$ 。优化的关键是：当我们知道某根柱子左右两侧第一个更矮的位置时，以它为最低高度的最大矩形宽度就确定了。单调递增栈可以在一次扫描中确定“右侧第一个更矮”。

Listing 10.5 柱状图最大矩形：单调递增栈

```

1  int largest_rectangle_area(const std::vector<int>& heights)
2  {
3      std::vector<int> stack;
4      stack.reserve(heights.size() + 1);
5      int answer = 0;
6
7      for (int i = 0; i <= static_cast<int>(heights.size()); ++i) {
8          const int current_height =
9              i == static_cast<int>(heights.size()) ? 0 : heights[i];
10
11          while (!stack.empty() && heights[stack.back()] > current_height) {
12              const int height = heights[stack.back()];
13              stack.pop_back();
14              const int left_less_index = stack.empty() ? -1 : stack.back();
15              const int width = i - left_less_index - 1;
16              answer = std::max(answer, height * width);
17          }
18          stack.push_back(i);
19      }
20
21      return answer;
22 }

```

这里末尾人为加入高度 0 的哨兵，目的是把栈里剩余柱子全部弹出结算。弹出某个下标时，当

前下标 i 是它右侧第一个更矮的位置；弹出后新的栈顶是它左侧第一个更矮的位置。因此宽度是两者之间的柱子数量。这个解释比“套单调栈模板”重要得多。复杂度是 $O(n)$ ，因为每个下标最多入栈一次、出栈一次。易错点是比较符号：使用 $>$ 或 $>=$ 都能做，但对重复高度的归属不同，必须和宽度计算保持一致。

队列和双端队列负责维护“按时间进入、按窗口过期”的候选。滑动窗口最大值的暴力解对每个窗口扫描一遍最大值，复杂度 $O(nk)$ 。堆也能做，但需要处理过期下标。单调队列更贴合题目：队头永远是当前窗口最大值下标，队尾负责删除不可能再成为最大值的候选。

Listing 10.6 滑动窗口最大值：队头保存当前最大值下标

```

1 std::vector<int> max_sliding_window(const std::vector<int>& nums, int k)
2 {
3     std::vector<int> answer;
4     if (nums.empty() || k <= 0) {
5         return answer;
6     }
7
8     std::deque<int> deque;
9     for (int right = 0; right < static_cast<int>(nums.size()); ++right) {
10         while (!deque.empty() && nums[deque.back()] <= nums[right]) {
11             deque.pop_back();
12         }
13         deque.push_back(right);
14
15         const int left = right - k + 1;
16         if (deque.front() < left) {
17             deque.pop_front();
18         }
19         if (left >= 0) {
20             answer.push_back(nums[deque.front()]);
21         }
22     }
23     return answer;
24 }

```

为什么队尾较小元素可以删除？设队尾下标为 j ，新下标为 $right$ ，如果 $nums[j] \leq nums[right]$ ，那么在未来任何同时包含 j 和 $right$ 的窗口里， j 都不可能比 $right$ 更优；而且 j 更早过期。因此删除 j 不会影响任何未来答案。这就是候选淘汰的证明。单调队列的复盘要说清两个动作：入队前从队尾删掉被新元素支配的候选；出答案前从队头删掉已过期的候选。

环形链表是快慢指针的另一条主线。暴力方法可以把走过的节点地址放进哈希集合，每到一个节点就检查是否见过；如果见过，说明有环；如果走到空指针，说明无环。这个方法直观，时间 $O(n)$ ，空间 $O(n)$ 。快慢指针把空间降到常数：慢指针每次走一步，快指针每次走两步。如果没有环，快指针会先到空；如果有环，快指针进入环后会不断追赶慢指针，最终相遇。

Listing 10.7 环形链表：快慢指针检测相遇

```

1 bool has_cycle(ListNode* head)
2 {
3     ListNode* slow = head;
4     ListNode* fast = head;
5
6     while (fast != nullptr && fast->next != nullptr) {
7         slow = slow->next;
8         fast = fast->next->next;

```

```

9         if (slow == fast) {
10             return true;
11         }
12     }
13
14     return false;
15 }

```

这题的正确性可以用相对速度解释。进入环之后，快指针相对慢指针每轮多走一步；环长度有限，因此相对距离会按环长度取模不断变化，必然出现 0，也就是相遇。不要把这题说成“快指针总会追上慢指针”就结束，最好补一句“因为在环内相对距离每次加一，对环长取模”。如果题目进一步要求找到入环节点，做法是在相遇后让一个指针回到头节点，两个指针都每次走一步，再次相遇的位置就是入口。这来自公式：头到入口距离加上入口到相遇点距离，与快慢指针走过路程差之间存在整环倍数关系。面试里不一定要完整推公式，但要知道这不是经验结论。

LRU 缓存是线性结构和哈希表组合的设计题。题目要求 `get` 和 `put` 都是 $O(1)$ ，并且容量满时淘汰最近最少使用的键。单独用数组或链表，可以维护使用顺序，但查找 `key` 是 $O(n)$ ；单独用哈希表可以常数查找，但无法常数时间移动“最近使用”的位置。正确组合是：双向链表维护从最近到最久的顺序，哈希表从 `key` 映射到链表节点。

Listing 10.8 LRU 缓存：哈希表定位节点，双向链表维护顺序

```

1 class LRUCache {
2 public:
3     explicit LRUCache(int capacity)
4         : capacity_(capacity)
5     {
6     }
7
8     int get(int key)
9     {
10         const auto found = this->items_.find(key);
11         if (found == this->items_.end()) {
12             return -1;
13         }
14         this->touch(found->second);
15         return found->second->second;
16     }
17
18     void put(int key, int value)
19     {
20         const auto found = this->items_.find(key);
21         if (found != this->items_.end()) {
22             found->second->second = value;
23             this->touch(found->second);
24             return;
25         }
26
27         this->entries_.emplace_front(key, value);
28         this->items_[key] = this->entries_.begin();
29
30         if (static_cast<int>(this->entries_.size()) > this->capacity_) {
31             const int removed_key = this->entries_.back().first;
32             this->items_.erase(removed_key);

```



```

33         this->entries_.pop_back();
34     }
35 }
36
37 private:
38     using Entry = std::pair<int, int>;
39     using Iterator = std::list<Entry>::iterator;
40
41     void touch(Iterator iterator)
42     {
43         this->entries_.splice(this->entries_.begin(), this->entries_, iterator);
44     }
45
46     int capacity_ = 0;
47     std::list<Entry> entries_;
48     std::unordered_map<int, Iterator> items_;
49 };

```

这段代码使用 `std::list`，不是因为链表在刷题中通常更快，而是因为 LRU 需要在已知节点迭代器的情况下常数时间移动节点到表头。`splice` 不复制节点，只改变链表连接关系；哈希表保存的迭代器仍指向同一节点。这里 `std::list` 的节点稳定性正好匹配需求。反过来，如果用 `vector` 保存顺序，移动到表头会导致大量元素移动，还可能让保存的下标或迭代器失效。LRU 的复盘重点是“为什么必须两个结构一起用”：哈希表负责定位，双向链表负责顺序，任何一个单独使用都达不到题目复杂度。

接雨水可以用多个思路解决。暴力方法枚举每个位置，向左找最高柱子，向右找最高柱子，该位置能接的水是左右最高的较小值减当前高度；每个位置都向两边扫描，复杂度 $O(n^2)$ 。前后缀数组可以把左右最高值预处理出来，时间降到 $O(n)$ ，空间 $O(n)$ 。双指针进一步把空间降到 $O(1)$ ：左右两侧各维护当前最高柱子，哪边较低，就结算哪边。

Listing 10.9 接雨水：双指针维护两侧最高柱

```

1 int trap(const std::vector<int>& height)
2 {
3     int left = 0;
4     int right = static_cast<int>(height.size()) - 1;
5     int left_max = 0;
6     int right_max = 0;
7     int water = 0;
8
9     while (left < right) {
10         if (height[left] < height[right]) {
11             left_max = std::max(left_max, height[left]);
12             water += left_max - height[left];
13             ++left;
14         } else {
15             right_max = std::max(right_max, height[right]);
16             water += right_max - height[right];
17             --right;
18         }
19     }
20
21     return water;
22 }

```

为什么可以结算较低的一侧？假设 `height[left] < height[right]`，右侧至少存在一个高度大于左侧当前位置的柱子，因此当前位置水位的右边界不会低于 `height[left]`；它能接多少水只取决于左侧最高柱 `left_max`。如果左侧最高还不够高，就接不到水；如果够高，就能立刻计算。单调栈也能做接雨水，它在遇到右边界时弹出低洼底部并结算横向宽度。双指针更节省空间，单调栈更接近“找左右边界”的模型，面试中可以比较两者。

表达式求值考察栈对局部归约的建模。最简单的四则表达式如果没有括号，但有乘除优先级，可以扫描字符串，用栈保存已经归约到加减层面的项。遇到数字时，根据前一个运算符决定：加号就压入正数，减号压入负数，乘除就和栈顶立即合并。这样乘除不会等到最后才处理，加减则通过最后求和完成。

Listing 10.10 基本计算器 II：栈处理乘除优先级

```

1 int calculate(const std::string& s)
2 {
3     std::vector<int> stack;
4     char operation = '+';
5     int number = 0;
6
7     for (int i = 0; i <= static_cast<int>(s.size()); ++i) {
8         const char ch = i == static_cast<int>(s.size()) ? '+' : s[i];
9         if (std::isdigit(static_cast<unsigned char>(ch))) {
10             number = number * 10 + (ch - '0');
11             continue;
12         }
13         if (ch == ' ') {
14             continue;
15         }
16
17         if (operation == '+') {
18             stack.push_back(number);
19         } else if (operation == '-') {
20             stack.push_back(-number);
21         } else if (operation == '*') {
22             stack.back() *= number;
23         } else if (operation == '/') {
24             stack.back() /= number;
25         }
26
27         operation = ch;
28         number = 0;
29     }
30
31     return std::accumulate(stack.begin(), stack.end(), 0);
32 }

```

这里在循环末尾人为补一个加号，是为了触发最后一个数字的归约。表达式题的易错点非常多：多位数要持续累积；空格要跳过；除法在 C++ 中对整数向 0 截断，和题目语义是否一致要确认；如果有括号，就需要另一个层次的栈或显式状态机。不要一开始就写复杂解析器，先按题目支持的语法范围选择最小可行模型。面试表达时可以说：栈里保存的是已经处理完乘除优先级、等待最终加总的项。

线性结构题的实验训练不应只跑样例。链表题要构造空链表、单节点、删除头、删除尾、K 不整除长度；栈题要构造重复最小值、空弹出是否题目保证、嵌套和交叉括号；单调结构要构造严格递增、严格递减、全相等、窗口为 1、窗口等于数组长度。面试中如果能主动说出这些测试，说明你

不仅知道算法，也知道边界来自哪里。

相交链表是链表“节点身份”意识的经典题。两个链表如果相交，意思不是某个节点的值相等，而是从某个节点开始共享同一批节点对象。暴力方法可以把第一条链表所有节点地址放入哈希集合，再扫描第二条链表，第一个出现在集合中的节点就是交点，复杂度 $O(m+n)$ ，空间 $O(m)$ 。双指针优化的思路更巧：指针 A 走完第一条链后转到第二条链，指针 B 走完第二条链后转到第一条链。若两链相交，它们会在交点相遇；若不相交，最终同时到达空指针。

Listing 10.11 相交链表：双指针走完两条路径

```

1 ListNode* get_intersection_node(ListNode* first, ListNode* second)
2 {
3     ListNode* a = first;
4     ListNode* b = second;
5
6     while (a != b) {
7         a = a == nullptr ? second : a->next;
8         b = b == nullptr ? first : b->next;
9     }
10
11     return a;
12 }

```

为什么这段代码成立？设第一条链表独有长度为 x ，第二条独有长度为 y ，共享尾部长度为 z 。指针 A 走过的路径是 $x + z + y$ 后到达交点，指针 B 走过的路径是 $y + z + x$ 后到达交点，总长度相同。若没有共享尾部，两个指针都会在走完 $m+n$ 后变成空指针。这里不能比较节点值，因为不同节点可以有相同值；也不能修改链表结构，除非题目允许。

回文链表可以用数组保存所有值后双指针判断，时间 $O(n)$ 、空间 $O(n)$ 。如果要求 $O(1)$ 额外空间，就要找到中点、反转后半段、逐个比较，最后最好把链表恢复。这个题综合考察快慢指针和链表反转。快指针每次走两步，慢指针每次走一步；当快指针到达尾部时，慢指针在中点附近。奇数长度时，中间节点不影响回文判断，可以让后半段从慢指针之后开始。

Listing 10.12 回文链表：反转后半段后比较

```

1 ListNode* reverse_list(ListNode* head)
2 {
3     ListNode* previous = nullptr;
4     ListNode* current = head;
5     while (current != nullptr) {
6         ListNode* next = current->next;
7         current->next = previous;
8         previous = current;
9         current = next;
10    }
11    return previous;
12 }
13
14 bool is_palindrome_list(ListNode* head)
15 {
16     if (head == nullptr || head->next == nullptr) {
17         return true;
18     }
19
20     ListNode* slow = head;
21     ListNode* fast = head;

```



```

22     while (fast->next != nullptr && fast->next->next != nullptr) {
23         slow = slow->next;
24         fast = fast->next->next;
25     }
26
27     ListNode* second_half = reverse_list(slow->next);
28     ListNode* restore_head = second_half;
29     ListNode* first_half = head;
30     bool ok = true;
31
32     while (second_half != nullptr) {
33         if (first_half->value != second_half->value) {
34             ok = false;
35             break;
36         }
37         first_half = first_half->next;
38         second_half = second_half->next;
39     }
40
41     slow->next = reverse_list(restore_head);
42     return ok;
43 }

```

这段代码刻意恢复链表，因为真实工程里“检查是否回文”不应该悄悄改变输入结构。力扣题可能不检查恢复，但教材要训练副作用意识。中点循环使用 `fast->next != nullptr && fast->next->next != nullptr`，能让 `slow` 停在前半段尾部。比较时只遍历后半段长度即可。易错点是反转后半段之前没有保存头节点，导致无法恢复。

链表排序可以用归并排序。数组排序可以随机访问中点，链表不行；链表找中点要用快慢指针，并从中点断开。暴力做法是把节点值放进数组排序再写回链表，这改变的是值，不是节点顺序；如果节点携带更多字段或外部引用，这种做法就不等价。链表归并排序的优势是稳定、时间 $O(n \log n)$ ，额外空间如果用递归会有栈成本；为了符合工程中避免深递归的习惯，可以使用自底向上的迭代归并。

Listing 10.13 链表排序：自底向上迭代归并

```

1  ListNode* split_list(ListNode* head, int size)
2  {
3      for (int i = 1; head != nullptr && i < size; ++i) {
4          head = head->next;
5      }
6      if (head == nullptr) {
7          return nullptr;
8      }
9      ListNode* second = head->next;
10     head->next = nullptr;
11     return second;
12 }
13
14 ListNode* merge_sorted_lists(ListNode* first, ListNode* second, ListNode** tail)
15 {
16     ListNode dummy;
17     ListNode* current = &dummy;
18     while (first != nullptr && second != nullptr) {

```

```

19     if (first->value <= second->value) {
20         current->next = first;
21         first = first->next;
22     } else {
23         current->next = second;
24         second = second->next;
25     }
26     current = current->next;
27 }
28 current->next = first != nullptr ? first : second;
29 while (current->next != nullptr) {
30     current = current->next;
31 }
32 *tail = current;
33 return dummy.next;
34 }
35
36 ListNode* sort_list(ListNode* head)
37 {
38     int length = 0;
39     for (ListNode* node = head; node != nullptr; node = node->next) {
40         ++length;
41     }
42
43     ListNode dummy;
44     dummy.next = head;
45     for (int size = 1; size < length; size *= 2) {
46         ListNode* previous = &dummy;
47         ListNode* current = dummy.next;
48         while (current != nullptr) {
49             ListNode* first = current;
50             ListNode* second = split_list(first, size);
51             current = split_list(second, size);
52             ListNode* merged_tail = nullptr;
53             previous->next = merge_sorted_lists(first, second, &merged_tail);
54             previous = merged_tail;
55         }
56     }
57     return dummy.next;
58 }

```

自底向上归并的思想是先合并长度为 1 的有序段，再合并长度为 2、4、8 的段，直到段长覆盖整个链表。每次 `split_list` 都把一段切断，避免合并时越界串到后续段。代码较长，但它把递归栈换成了显式循环。面试中如果时间有限，可以先讲递归归并，再说明生产环境可改成迭代版本。这里的质量点不是炫技，而是知道链表排序的节点重连语义和递归深度风险。

删除排序链表中的重复元素也有两个版本。简单版保留一个重复值，只要当前节点和下一个节点值相等，就跳过下一个节点。进阶版删除所有重复值，只保留原链表中没有重复出现的值；这时必须使用虚拟头节点和前驱指针，因为重复段可能从头节点开始。先识别一整段相等值，再决定是保留还是跳过整段。

Listing 10.14 删除所有重复节点：识别重复段后整体跳过

```

1 ListNode* delete_duplicates_all(ListNode* head)

```

```

2 {
3     ListNode dummy;
4     dummy.next = head;
5     ListNode* previous = &dummy;
6     ListNode* current = head;
7
8     while (current != nullptr) {
9         bool duplicated = false;
10        while (current->next != nullptr &&
11               current->value == current->next->value) {
12            duplicated = true;
13            current = current->next;
14        }
15        if (duplicated) {
16            previous->next = current->next;
17        } else {
18            previous = previous->next;
19        }
20        current = current->next;
21    }
22
23    return dummy.next;
24 }

```

这题的重点是 `previous` 的含义：它始终指向结果链表的尾部，也就是最后一个确认保留的节点。遇到重复段时，`previous` 不动，只修改它的后继；遇到非重复节点时，`previous` 才前进。边界包括全是重复值、头部重复、尾部重复、没有重复。链表题里，前驱指针比当前指针更重要，因为删除动作发生在前驱的 `next` 上。

路径简化是栈在字符串解析中的应用。Unix 路径中，多个斜杠等价于一个斜杠，`.` 表示当前目录，`..` 表示回到上一级目录。暴力按字符处理容易被连续斜杠和边界搞乱。更清晰的做法是把路径按斜杠切分成段，栈保存有效目录名：普通目录入栈，`..` 弹出一个目录，空段和 `.` 忽略。最后用栈中目录重建规范路径。

Listing 10.15 简化路径：栈保存有效目录段

```

1 std::string simplify_path(const std::string& path)
2 {
3     std::vector<std::string> stack;
4     int index = 0;
5     while (index < static_cast<int>(path.size())) {
6         while (index < static_cast<int>(path.size()) && path[index] == '/') {
7             ++index;
8         }
9         int start = index;
10        while (index < static_cast<int>(path.size()) && path[index] != '/') {
11            ++index;
12        }
13        const std::string part = path.substr(start, index - start);
14        if (part.empty() || part == ".") {
15            continue;
16        }
17        if (part == "..") {
18            if (!stack.empty()) {

```



```

19         stack.pop_back();
20     }
21     } else {
22         stack.push_back(part);
23     }
24 }
25
26 if (stack.empty()) {
27     return "/";
28 }
29 std::string result;
30 for (const std::string& part : stack) {
31     result += "/";
32     result += part;
33 }
34 return result;
35 }

```

这里的“栈”不是必须用 `std::stack`，用 `vector` 更方便遍历重建路径。容器选择要服务于操作：我们需要尾部压入弹出，也需要最后从头到尾遍历，所以 `vector` 很合适。若路径很长，频繁字符串拼接可以用 `ostringstream` 或先估算长度，但刷题中清晰优先。易错点是根目录下遇到 `..` 不能继续向上，只能保持根。

括号相关题还会扩展为“最长有效括号”。暴力枚举所有子串再判断是否合法，复杂度高。栈做法保存下标，而不是保存字符。栈底保存最近一个无法匹配的位置，作为当前有效段的左边界。遇到左括号入栈；遇到右括号先弹出一个左括号，若弹出后栈空，说明当前右括号无法匹配，它的下标作为新边界；否则当前有效长度是当前下标减栈顶下标。

Listing 10.16 最长有效括号：栈保存边界下标

```

1 int longest_valid_parentheses(const std::string& s)
2 {
3     std::vector<int> stack;
4     stack.push_back(-1);
5     int answer = 0;
6
7     for (int i = 0; i < static_cast<int>(s.size()); ++i) {
8         if (s[i] == '(') {
9             stack.push_back(i);
10            continue;
11        }
12
13        stack.pop_back();
14        if (stack.empty()) {
15            stack.push_back(i);
16        } else {
17            answer = std::max(answer, i - stack.back());
18        }
19    }
20
21    return answer;
22 }

```

这个题中 `-1` 是哨兵边界，表示在字符串开始之前有一个不可跨越位置。栈顶在计算长度时不

是当前有效段的第一个字符，而是当前有效段左侧的边界。这个语义非常重要。若只保存括号字符，就无法计算长度；若只保存左括号数量，就无法处理中间断裂。单调栈、括号栈、路径栈都在提醒同一件事：栈里保存的不是“看起来像栈的东西”，而是后续计算真正需要的历史状态。

队列题的进阶点是层次和边界。普通 BFS 中队列保存待扩展节点；若需要输出每层，就在每轮开始记录队列大小；若需要最短步数，就把层数和状态一起维护，或每处理一层步数加一。双端队列出现在两类场景：一类是单调队列维护窗口候选，另一类是 0-1 BFS 中根据边权把状态放队首或队尾。它们都不是“队列模板”，而是对候选优先级的精确表达。

线性结构实验：第一，用同一组链表节点分别测试删除倒数节点、回文判断、排序，检查函数是否意外破坏输入。第二，把最长有效括号的栈内容逐步打印出来，观察栈顶为何是边界下标。第三，分别用数组扫描、单调队列解决滑动窗口最大值，统计窗口大小增大时的运行差异。第四，给路径简化加入连续斜杠、根目录回退、隐藏文件名等测试，确认规则没有被字符串细节干扰。

线性结构题的统一问题

链表问“哪条边会断，断之前有没有保存”；栈问“哪个历史状态最近且必须先解决”；队列问“当前层或当前窗口的边界在哪里”；单调结构问“哪个候选被新元素支配，为什么永远不可能再成为答案”。只要这四个问题能回答，线性结构题就不再是零散模板。

第 11 章

排序、选择和区间问题

排序不是为了把数组排好看，而是为了制造结构。无序输入里，两个元素之间没有稳定关系；排序之后，大小关系变成单调关系，重复元素挨在一起，区间端点可以按起点或终点扫描，双指针和二分才有施展空间。面试中很多题的关键不是“调用 `std::sort`”，而是说明排序之后哪个约束变简单了，排序代价是否值得，以及排序是否会破坏题目要求的原始下标或稳定性。

C++ 的 `std::sort` 通常是内省排序，综合快速排序、堆排序和插入排序思想，平均和最坏复杂度都能控制在 $O(n \log n)$ 级别。刷题时可以放心使用它，但要理解比较器必须满足严格弱序。比较器不能写成 `a <= b`，也不能依赖会变化的外部状态，否则排序行为未定义。对结构体排序时，比较器应该只表达排序键，例如先按起点升序，再按终点升序。

Listing 11.1 区间排序比较器：严格弱序

```
1 struct Interval {
2     int start = 0;
3     int end = 0;
4 };
5
6 void sort_intervals(std::vector<Interval>& intervals)
7 {
8     std::sort(intervals.begin(), intervals.end(),
9               [](const Interval& lhs, const Interval& rhs) {
10                 if (lhs.start != rhs.start) {
11                     return lhs.start < rhs.start;
12                 }
13                 return lhs.end < rhs.end;
14             });
15 }
```

这个比较器在 `lhs.start == rhs.start && lhs.end == rhs.end` 时返回 `false`，满足严格弱序要求。很多线上题数据小，错误比较器也可能侥幸通过，但这是 C++ 基本功问题。排序后，如果题目仍需要原始下标，就不能只排序值；要把值和下标一起保存成 `pair` 或结构体。

颜色分类是荷兰国旗问题。题目只包含 0、1、2 三种值，要求原地排序。直接调用排序当然能过，复杂度 $O(n \log n)$ ；计数排序也能 $O(n)$ ，但需要两次扫描。更经典的一次扫描做法是维护三个区域：左侧全是 0，中间是已经确认的 1，右侧全是 2，未知区域在中间游动。指针 `low` 指向下一个 0 应放的位置，`mid` 扫描未知区域，`high` 指向下一个 2 应放的位置。

Listing 11.2 颜色分类：三指针维护四段区域

```
1 void sort_colors(std::vector<int>& nums)
2 {
3     constexpr int RED = 0;
4     constexpr int WHITE = 1;
5     constexpr int BLUE = 2;
6
7     int low = 0;
8     int mid = 0;
9     int high = static_cast<int>(nums.size()) - 1;
10 }
```



```

11 while (mid <= high) {
12     if (nums[mid] == RED) {
13         std::swap(nums[low], nums[mid]);
14         ++low;
15         ++mid;
16     } else if (nums[mid] == WHITE) {
17         ++mid;
18     } else if (nums[mid] == BLUE) {
19         std::swap(nums[mid], nums[high]);
20         --high;
21     }
22 }
23 }

```

这题最容易错在交换 2 之后是否移动 `mid`。答案是不能移动，因为从 `high` 换过来的元素还没有检查，可能是 0、1 或 2。交换 0 后可以移动 `mid`，因为 `low` 左边已经处理完，换到 `mid` 的元素来自已确认的 1 区或当前位置本身。这个差别正是不变量的体现：`[0, low)` 是 0，`[low, mid)` 是 1，`(high, n)` 是 2，`[mid, high]` 未知。

合并区间展示排序如何把二维关系变成线性扫描。暴力方法是反复找任意两个相交区间合并，直到不能合并为止；每次合并后都可能重新扫描全部区间，复杂度很高。按起点排序后，相交关系只需要和当前结果的最后一个区间比较。因为后续区间起点只会越来越大，如果它不能和最后区间相交，就不可能和更早已经结束的区间相交。

Listing 11.3 合并区间：排序后只看结果尾部

```

1 std::vector<std::vector<int>> merge_intervals(
2     std::vector<std::vector<int>> intervals)
3 {
4     if (intervals.empty()) {
5         return {};
6     }
7
8     std::sort(intervals.begin(), intervals.end(),
9         [](const std::vector<int>& lhs, const std::vector<int>& rhs) {
10             return lhs[0] < rhs[0];
11         });
12
13     std::vector<std::vector<int>> merged;
14     merged.push_back(intervals.front());
15
16     for (int i = 1; i < static_cast<int>(intervals.size()); ++i) {
17         std::vector<int>& last = merged.back();
18         const std::vector<int>& current = intervals[i];
19         if (current[0] <= last[1]) {
20             last[1] = std::max(last[1], current[1]);
21         } else {
22             merged.push_back(current);
23         }
24     }
25
26     return merged;
27 }

```

这里参数按值传入，是因为函数内部排序会改变区间顺序。如果调用者不需要保留原顺序，传引用也可以。复杂度是排序 $O(n \log n)$ 加扫描 $O(n)$ 。易错点是相交条件：如果区间是闭区间，`current[0] <= last[1]` 表示相交或接触；如果题目定义半开区间，就要重新审视等号是否成立。区间问题一定要先明确端点语义。

插入区间与合并区间类似，但输入已经有序且不重叠。暴力方法可以把新区间放进去重新排序再合并，复杂度仍可接受，但没有利用已有条件。线性做法分三段：先加入所有结束在新区间之前的区间；再把所有和新区间相交的区间合并进新区间；最后加入剩余区间。这种写法体现了区间扫描的分段思维。

Listing 11.4 插入区间：前段、中段、后段

```

1 std::vector<std::vector<int>> insert_interval(
2     const std::vector<std::vector<int>>& intervals,
3     std::vector<int> new_interval)
4 {
5     std::vector<std::vector<int>> answer;
6     int index = 0;
7     const int n = static_cast<int>(intervals.size());
8
9     while (index < n && intervals[index][1] < new_interval[0]) {
10         answer.push_back(intervals[index]);
11         ++index;
12     }
13
14     while (index < n && intervals[index][0] <= new_interval[1]) {
15         new_interval[0] = std::min(new_interval[0], intervals[index][0]);
16         new_interval[1] = std::max(new_interval[1], intervals[index][1]);
17         ++index;
18     }
19     answer.push_back(new_interval);
20
21     while (index < n) {
22         answer.push_back(intervals[index]);
23         ++index;
24     }
25
26     return answer;
27 }

```

这题的优化条件是“原区间已经按起点排序且互不重叠”。如果这个条件不存在，就必须先排序或换模型。第一段的判断是 `intervals[index][1] < new_interval[0]`，说明当前区间完全在新区间左边；第二段的判断是 `intervals[index][0] <= new_interval[1]`，说明存在重叠。两个等号同样取决于闭区间语义。面试中把三段说清楚，代码通常很自然。

无重叠区间是区间贪心题。题目要求删除最少区间，使剩余区间互不重叠。等价地，可以保留最多数量的不重叠区间。暴力方法是枚举所有区间子集，检查是否互不重叠，复杂度指数级。贪心优化依据是：每次选择结束最早的区间，会给后面留下最大的空间。按结束时间排序，能选就选，不能选就跳过。

Listing 11.5 无重叠区间：按结束时间贪心保留最多区间

```

1 int erase_overlap_intervals(std::vector<std::vector<int>> intervals)
2 {
3     if (intervals.empty()) {
4         return 0;

```

```

5     }
6
7     std::sort(intervals.begin(), intervals.end(),
8             [](const std::vector<int>& lhs, const std::vector<int>& rhs) {
9                 return lhs[1] < rhs[1];
10            });
11
12     int kept = 0;
13     int current_end = std::numeric_limits<int>::min();
14     for (const std::vector<int>& interval : intervals) {
15         if (interval[0] >= current_end) {
16             ++kept;
17             current_end = interval[1];
18         }
19     }
20
21     return static_cast<int>(intervals.size()) - kept;
22 }

```

这个贪心的交换论证是：假设某个最优解第一步选择了一个结束更晚的区间，而存在另一个兼容区间结束更早。把最优解里的第一个区间替换成结束更早的区间，不会减少后续可选空间，因此不会让答案变差。反复替换，就得到按结束时间贪心的解。区间贪心题不要只说“排序后贪心”，必须说明排序键为什么是结束时间而不是开始时间。

数组中的第 K 大元素可以排序解决，时间 $O(n \log n)$ 。如果只需要第 K 大，不需要完整有序，快速选择更合适。它借鉴快速排序的分区思想：选择一个枢轴，把大于枢轴的放左边，小于枢轴的放右边；如果枢轴最终位置正好是 $k - 1$ ，答案找到；如果位置偏左或偏右，只在一侧继续。平均复杂度 $O(n)$ 。

Listing 11.6 第 K 大元素：迭代快速选择

```

1 int find_kth_largest_quickselect(std::vector<int> nums, int k)
2 {
3     const int target = k - 1;
4     int left = 0;
5     int right = static_cast<int>(nums.size()) - 1;
6
7     while (left <= right) {
8         const int pivot = nums[right];
9         int store = left;
10        for (int i = left; i < right; ++i) {
11            if (nums[i] > pivot) {
12                std::swap(nums[store], nums[i]);
13                ++store;
14            }
15        }
16        std::swap(nums[store], nums[right]);
17
18        if (store == target) {
19            return nums[store];
20        }
21        if (store < target) {
22            left = store + 1;
23        } else {

```



```

24     right = store - 1;
25     }
26 }
27
28 return nums[target];
29 }

```

这里按值传参是刻意的，因为快速选择会重排数组。如果题目允许修改输入，可以传引用减少一次复制。代码使用最后一个元素作枢轴，最坏情况下会退化到 $O(n^2)$ ；工业实现通常会随机化枢轴或使用更稳健的选择算法。面试中要主动说出这个风险。快速选择的正确性来自分区后的位置含义：左侧元素都比枢轴大，因此枢轴排名已经确定，不需要管左右两侧内部是否有序。

排序和选择题经常和去重绑定。三数之和先排序，是为了让固定一个数后，剩余两数可以用双指针线性查找；合并区间先排序，是为了让所有可能相交的区间在扫描时相邻；第 K 大使用分区，是为了只确定目标排名而不完整排序。三者共同点是：排序或分区不是目的，而是让后续操作只看局部信息。

归并排序在刷题里经常不是为了排序本身，而是为了在“合并两个有序段”的过程中统计跨区间关系。逆序对就是典型例子。暴力方法枚举所有 $i < j$ ，检查 $nums[i] > nums[j]$ ，复杂度 $O(n^2)$ 。归并排序把数组分成左右两半，递归意义上左右内部逆序对已经统计完；合并时，只需要统计左半元素和右半元素形成的跨区间逆序对。由于左右两边已经有序，若 $left[i] > right[j]$ ，那么从 i 到左半末尾的所有元素都大于 $right[j]$ 。

Listing 11.7 逆序对：归并过程中统计跨区间关系

```

1 long long count_reverse_pairs(std::vector<int> nums)
2 {
3     std::vector<int> buffer(nums.size());
4     long long answer = 0;
5
6     for (int width = 1; width < static_cast<int>(nums.size()); width *= 2) {
7         for (int left = 0; left < static_cast<int>(nums.size()); left += 2 * width) {
8             const int mid = std::min(left + width, static_cast<int>(nums.size()));
9             const int right = std::min(left + 2 * width, static_cast<int>(nums.size())↵
↵ ));
10
11             int i = left;
12             int j = mid;
13             int out = left;
14             while (i < mid && j < right) {
15                 if (nums[i] <= nums[j]) {
16                     buffer[out++] = nums[i++];
17                 } else {
18                     answer += mid - i;
19                     buffer[out++] = nums[j++];
20                 }
21             }
22             while (i < mid) {
23                 buffer[out++] = nums[i++];
24             }
25             while (j < right) {
26                 buffer[out++] = nums[j++];
27             }
28             for (int k = left; k < right; ++k) {
29                 nums[k] = buffer[k];

```

```

30     }
31     }
32 }
33
34 return answer;
35 }

```

这里使用自底向上的迭代归并，避免递归。`width` 表示当前已经有序的段长度，每轮把相邻两段合并成更长有序段。统计逆序对的关键行是 `answer += mid - i`：当右侧当前元素更小，左侧从 `i` 到 `mid - 1` 的所有元素都会和它形成逆序对。这个推理依赖左半段已经有序。归并类统计题的复盘要问：跨区间关系能不能在两个有序段合并时一次性数出来？如果能，通常可以从平方复杂度降到 $O(n \log n)$ 。

会议室问题把区间排序和堆结合起来。会议室 I 问一个人能否参加所有会议，只要按开始时间排序，检查相邻会议是否重叠即可。会议室 II 问至少需要多少会议室，就不能只看相邻会议，因为多个会议可能同时进行。暴力方法对每个会议扫描当前所有房间，找能复用的房间；优化方法用小顶堆保存每个房间当前会议的结束时间。新会议开始时，如果最早结束的房间已经空出来，就复用它；否则需要新房间。

Listing 11.8 会议室 II：小顶堆保存房间结束时间

```

1 int min_meeting_rooms(std::vector<std::vector<int>> intervals)
2 {
3     if (intervals.empty()) {
4         return 0;
5     }
6
7     std::sort(intervals.begin(), intervals.end(),
8               [](const std::vector<int>& lhs, const std::vector<int>& rhs) {
9                 return lhs[0] < rhs[0];
10            });
11
12     std::priority_queue<int, std::vector<int>, std::greater<int>> endings;
13     for (const std::vector<int>& meeting : intervals) {
14         if (!endings.empty() && endings.top() <= meeting[0]) {
15             endings.pop();
16         }
17         endings.push(meeting[1]);
18     }
19
20     return static_cast<int>(endings.size());
21 }

```

堆顶表示当前所有占用房间中最早结束的会议。如果它都晚于新会议开始，那么其他房间结束更晚，也不能复用；如果它不晚于新会议开始，就弹出并把该房间的新结束时间压入。复杂度是排序 $O(n \log n)$ 加每个会议一次堆操作。注意这里一次只弹出一个结束时间就够了，因为每个新会议只需要一个房间；如果题目问某个时间点同时进行会议数量，可能会弹出所有已结束会议。

差分数组是区间更新题的另一种排序替代思路。如果题目反复对区间加值，再询问每个位置最终值，暴力对每次更新逐个位置加，复杂度 $O(qn)$ 。差分数组保存相邻元素变化量：对区间 `[left, right]` 加 `value`，只需要 `diff[left] += value`，`diff[right + 1] -= value`。最后做一次前缀和还原所有位置。

Listing 11.9 航班预订统计：差分数组处理区间加法

```

1 std::vector<int> corp_flight_bookings(const std::vector<std::vector<int>>& bookings,
2                                     int n)
3 {
4     std::vector<int> diff(n + 1, 0);
5     for (const std::vector<int>& booking : bookings) {
6         const int left = booking[0] - 1;
7         const int right = booking[1] - 1;
8         const int seats = booking[2];
9         diff[left] += seats;
10        if (right + 1 < n) {
11            diff[right + 1] -= seats;
12        }
13    }
14
15    std::vector<int> answer(n, 0);
16    int current = 0;
17    for (int i = 0; i < n; ++i) {
18        current += diff[i];
19        answer[i] = current;
20    }
21    return answer;
22 }

```

差分正确性来自“影响从 left 开始，在 right 之后结束”。它把区间内每个点都加的操作，变成两个边界事件。扫描线也有类似思想：把区间起点看成 +1 事件，终点看成 -1 事件，排序后扫描事件就能得到当前重叠数量。差分适合坐标连续且范围可开数组的场景；扫描线适合坐标很大或稀疏，需要排序事件的场景。

计数排序和桶排序利用值域信息。若数组长度很大但值域很小，比较排序的 $O(n \log n)$ 下界不再是必须承受的，因为我们不靠两两比较区分所有排列，而是统计每个值出现次数。颜色分类可以看成值域只有 3 的计数排序变体；前 K 高频元素也可以在统计频次后按频次分桶。桶排序的风险是空间由值域或频次范围决定，不能在值域巨大时盲目开数组。

Listing 11.10 前 K 高频元素：桶按频次收集

```

1 std::vector<int> top_k_frequent_bucket(const std::vector<int>& nums, int k)
2 {
3     std::unordered_map<int, int> frequency;
4     frequency.reserve(nums.size());
5     for (const int x : nums) {
6         ++frequency[x];
7     }
8
9     std::vector<std::vector<int>> buckets(nums.size() + 1);
10    for (const auto& [value, count] : frequency) {
11        buckets[count].push_back(value);
12    }
13
14    std::vector<int> answer;
15    for (int count = static_cast<int>(buckets.size()) - 1;
16         count >= 0 && static_cast<int>(answer.size()) < k;
17         --count) {
18        for (const int value : buckets[count]) {
19            answer.push_back(value);

```

```

20         if (static_cast<int>(answer.size()) == k) {
21             break;
22         }
23     }
24 }
25 return answer;
26 }

```

这个版本的时间复杂度是 $O(n)$ ，空间复杂度也是 $O(n)$ ，因为最大频次不超过数组长度。它比堆更快吗？理论上是线性的，但常数和内存占用更高，且输出顺序不固定。实际面试可以先讲堆，因为堆适用范围更广；再补桶排序作为值域或频次范围明确时的优化。高质量答案不是只追求最低复杂度，而是解释方案适用条件。

相对排序数组展示“值域有限时不一定要比较排序”。题目给两个数组，第二个数组给出部分元素的相对顺序，要求第一个数组中这些元素按第二个数组顺序排列，其余元素升序排列。暴力做法可以写自定义比较器：若两个元素都在第二个数组中，就比较它们的排名；若只有一个在第二个数组中，它排前面；否则按值升序。这个做法清晰，但每次比较都要查询排名，复杂度仍是排序的 $O(n \log n)$ 。如果值域较小，可以计数后按规则输出，复杂度接近线性。

Listing 11.11 相对排序：计数后按给定顺序输出

```

1  std::vector<int> relative_sort_array(const std::vector<int>& arr1,
2                                     const std::vector<int>& arr2)
3  {
4      constexpr int MAX_VALUE = 1000;
5      std::array<int, MAX_VALUE + 1> count{};
6      for (const int x : arr1) {
7          ++count[x];
8      }
9
10     std::vector<int> answer;
11     answer.reserve(arr1.size());
12     for (const int x : arr2) {
13         while (count[x] > 0) {
14             answer.push_back(x);
15             --count[x];
16         }
17     }
18     for (int value = 0; value <= MAX_VALUE; ++value) {
19         while (count[value] > 0) {
20             answer.push_back(value);
21             --count[value];
22         }
23     }
24     return answer;
25 }

```

这段代码的前提是题目给定值域不超过 1000，所以 `MAX_VALUE` 是题目约束，不是随便拍脑袋。若值域很大，就不能开这么大的数组，应改用哈希表统计，再对剩余元素排序。这个题训练的是“比较排序下界的适用范围”：当元素值域和输出顺序被额外约束时，我们可以不通过比较确定所有相对顺序。

按奇偶排序、按自定义规则排序、把负数放前面等题，看起来简单，其实都在考“稳定性”。如果题目要求保持同类元素的原始相对顺序，就需要稳定划分，可以新开数组，或者用 `stable_partition`。如果不要稳定，就可以用左右指针原地交换。原地不稳定划分的复杂度低、空间少，但会改变相

对顺序；稳定划分保留顺序，但通常需要额外空间或更复杂的移动。面试中主动问清稳定性，是比直接写代码更成熟的表现。

Listing 11.12 稳定划分：保持非零元素相对顺序

```

1 std::vector<int> stable_partition_by_sign(const std::vector<int>& nums)
2 {
3     std::vector<int> answer;
4     answer.reserve(nums.size());
5     for (const int x : nums) {
6         if (x < 0) {
7             answer.push_back(x);
8         }
9     }
10    for (const int x : nums) {
11        if (x >= 0) {
12            answer.push_back(x);
13        }
14    }
15    return answer;
16 }

```

这不是最省空间的写法，但语义非常清晰。若题目必须原地且稳定，问题会变难，因为把一个负数移动到前面可能要整体搬移中间元素。很多面试题默认不要求稳定，允许左右指针交换；但如果业务语义中原始顺序有意义，例如日志时间顺序、用户输入顺序，就不能随便破坏。排序和划分题一定要把“允许改变什么”说清楚。

数组中的逆序对除了归并计数，还可以用树状数组。归并排序从“跨左右两半的逆序对”出发；树状数组从“扫描到当前数时，之前有多少数比它大”出发。由于数值可能很大或为负，需要先离散化，把值映射到排名。扫描数组时，查询已经出现过的元素总数减去小于等于当前排名的数量，就是以当前元素为右端的逆序对数量；然后把当前排名出现次数加一。

Listing 11.13 逆序对：离散化加树状数组

```

1 class FenwickTree {
2 public:
3     explicit FenwickTree(int size) : tree_(size + 1, 0) {}
4
5     void add(int index, int delta)
6     {
7         for (int i = index; i < static_cast<int>(this->tree_.size());
8             i += i & -i) {
9             this->tree_[i] += delta;
10        }
11    }
12
13    int sum(int index) const
14    {
15        int result = 0;
16        for (int i = index; i > 0; i -= i & -i) {
17            result += this->tree_[i];
18        }
19        return result;
20    }
21 }

```

```

22 private:
23     std::vector<int> tree_;
24 };
25
26 long long reverse_pairs_by_fenwick(const std::vector<int>& nums)
27 {
28     std::vector<int> values = nums;
29     std::sort(values.begin(), values.end());
30     values.erase(std::unique(values.begin(), values.end()), values.end());
31
32     FenwickTree tree(static_cast<int>(values.size()));
33     long long answer = 0;
34     for (int i = 0; i < static_cast<int>(nums.size()); ++i) {
35         const int rank = static_cast<int>(
36             std::lower_bound(values.begin(), values.end(), nums[i]) -
37             values.begin()) + 1;
38         const int seen = i;
39         const int not_greater = tree.sum(rank);
40         answer += seen - not_greater;
41         tree.add(rank, 1);
42     }
43     return answer;
44 }

```

树状数组适合动态维护前缀计数。它的下标从 1 开始， i & $-i$ 表示当前节点覆盖区间的长度。这个结构比归并计数更抽象，但能迁移到很多“动态排名、区间频次、前缀和修改查询”问题。面试中如果只考逆序对，归并排序更容易解释；如果题目有多次更新或在线查询，树状数组更有优势。这里再次体现：排序不是唯一制造顺序的方法，离散化加索引树也能制造排名结构。

扫描线是区间问题的重要模型。区间重叠数量、天际线、会议室、航班预订，都可以看成一系列事件按坐标发生。区间开始是一个加事件，区间结束是一个减事件。排序事件后从左到右扫描，当前累计值就是当前位置的活跃区间数量。对于闭区间和半开区间，开始和结束在同一坐标时的处理顺序不同；例如会议室通常认为一个会议在时间 t 结束，另一个会议在时间 t 开始可以复用同一房间，所以结束事件应该先于开始事件处理。

Listing 11.14 会议室数量：扫描线事件排序

```

1 int min_meeting_rooms_by_events(const std::vector<std::vector<int>>& intervals)
2 {
3     std::vector<std::pair<int, int>> events;
4     events.reserve(intervals.size() * 2);
5     for (const std::vector<int>& interval : intervals) {
6         events.emplace_back(interval[0], 1);
7         events.emplace_back(interval[1], -1);
8     }
9
10    std::sort(events.begin(), events.end(),
11              [](const auto& lhs, const auto& rhs) {
12                  if (lhs.first != rhs.first) {
13                      return lhs.first < rhs.first;
14                  }
15                  return lhs.second < rhs.second;
16              });
17

```

```

18     int active = 0;
19     int answer = 0;
20     for (const auto [time, delta] : events) {
21         (void)time;
22         active += delta;
23         answer = std::max(answer, active);
24     }
25     return answer;
26 }

```

这里 `-1` 事件在同一时间排在 `+1` 前面，所以会议结束先释放房间。若题目是闭区间覆盖点，结束和开始的同点处理可能要反过来。扫描线的难点不是代码，而是事件语义：一个事件表示影响从哪里开始，在哪里结束，边界是否包含。差分数组是坐标连续且范围可开数组时的扫描线；事件排序是坐标稀疏或很大时的扫描线。

用最少数引爆气球是区间贪心。每个气球是一个水平区间，一支箭射在某个坐标可以引爆所有覆盖该坐标的气球。暴力想法是尝试每个可能射击位置，但坐标范围巨大。排序后贪心：按右端点升序排列，第一支箭射在第一个气球的右端点，这个选择最不容易影响后续，因为它尽可能靠右，同时仍能引爆当前气球。之后扫描气球，若当前气球起点大于当前箭位置，说明必须新射一箭。

Listing 11.15 最少箭引爆气球：按右端点贪心

```

1 int find_min_arrow_shots(std::vector<std::vector<int>> points)
2 {
3     if (points.empty()) {
4         return 0;
5     }
6     std::sort(points.begin(), points.end(),
7               [](const std::vector<int>& lhs, const std::vector<int>& rhs) {
8                 return lhs[1] < rhs[1];
9             });
10
11     int arrows = 1;
12     int arrow_position = points[0][1];
13     for (int i = 1; i < static_cast<int>(points.size()); ++i) {
14         if (points[i][0] <= arrow_position) {
15             continue;
16         }
17         ++arrows;
18         arrow_position = points[i][1];
19     }
20     return arrows;
21 }

```

正确性可以用交换论证说明：对于最早结束的气球，任何可行解都必须有一支箭射在它的区间内。把这支箭移动到该气球右端点，不会让它失去对已覆盖气球的覆盖能力，反而给后续区间留下更大机会。因此选择最早右端点是安全的。这个思路也适用于无重叠区间中“保留最多区间”：按结束时间选择最早结束的区间，给后面留下空间。

区间列表交集是双指针题。两个区间列表各自有序且互不重叠。暴力两两比较复杂度 $O(mn)$ 。双指针优化来自有序性：当前两个区间的交集左端是较大起点，右端是较小终点；若左端不大于右端，就有交集。然后谁的终点更早，谁就不可能再和对方后续区间相交，移动谁的指针。

Listing 11.16 区间列表交集：移动较早结束的一方

```

1 std::vector<std::vector<int>> interval_intersection(

```

```

2   const std::vector<std::vector<int>>& first,
3   const std::vector<std::vector<int>>& second)
4 {
5     std::vector<std::vector<int>> answer;
6     int i = 0;
7     int j = 0;
8     while (i < static_cast<int>(first.size()) &&
9            j < static_cast<int>(second.size())) {
10        const int left = std::max(first[i][0], second[j][0]);
11        const int right = std::min(first[i][1], second[j][1]);
12        if (left <= right) {
13            answer.push_back({left, right});
14        }
15
16        if (first[i][1] < second[j][1]) {
17            ++i;
18        } else {
19            ++j;
20        }
21    }
22    return answer;
23 }

```

这题的推理比代码重要：终点较早的区间在当前比较后已经没有机会和另一个当前区间或它后面的区间形成新的交集，因为后续区间起点只会更大。因此可以安全丢弃它。这个“丢弃谁”的逻辑和合并区间、插入区间、会议室复用都属于同一类端点单调推理。

日志重排和版本号比较这类题看起来不是传统算法，但同样考排序规则。日志重排要求字母日志排在数字日志前，字母日志按内容排序，内容相同按标识符排序，数字日志保持原始相对顺序。若直接写比较器处理数字日志之间返回 `false`，还需要使用稳定排序才能保持它们原顺序；或者把字母日志单独取出排序，再追加数字日志。这里的关键是理解 `std::sort` 不稳定，不能指望相等元素原顺序不变。

常见误区

排序比较器有一个底线：不能让两个元素互相都“小于”等关系成立，也不能让相等元素比较返回真。比较器依赖外部可变状态、随机数、递增计数器，都会破坏排序算法假设。若题目要求保持相对顺序，请使用稳定排序或显式分组，不要假设普通排序“碰巧没改顺序”。

区间和排序题的测试要覆盖边界语义。合并区间要测相邻是否合并、完全包含、乱序输入；会议室要测一个会议结束另一个会议开始；最少箭要测端点相同的大坐标；快速选择要测重复值、K 为 1、K 为数组长度；计数排序要测值域边界。很多排序题代码看起来一行 `sort` 很短，但真正的错误都藏在比较器和边界定义里。

排序区间实验：第一，把会议室问题分别用堆和扫描线写一遍，用同一组边界用例比较答案。第二，把无重叠区间、最少箭、会议安排放在一起复盘，写出它们为什么都偏好按右端点排序。第三，给相对排序构造值域很大但元素很少的用例，比较计数数组和哈希统计的空间差异。第四，故意写一个 `a <= b` 的错误比较器，观察小数据可能不报错但语义已经未定义。

排序题复盘模板

先写清楚排序键，再说明排序后获得了什么性质：相同值相邻、端点单调、排名确定、候选可淘汰，还是可以双指针。然后说明是否需要保留原下标，比较器是否满足严格弱序，

排序是否改变输入。最后比较完整排序、堆、快速选择、计数排序等方案的适用范围。能说清这些，排序题才不是“先 sort 一下试试”。

排序、选择和分治的系统讲解

排序题表面是在排列元素，实际是在制造顺序信息。没有顺序时，两数比较、区间合并、去重、前驱后继都可能需要线性寻找；排序以后，许多关系会变成相邻关系或单调关系。三数之和能够从三层枚举降到固定一数加双指针，是因为排序把剩余两数的和变成可单调调整的量。合并区间能够线性扫描，是因为按起点排序后，所有可能与当前结果尾部相交的区间都会连续出现。排序不是免费的，复杂度通常是 $O(n \log n)$ ，所以必须说明排序换来了什么查询能力。

比较排序的下界来自信息论： n 个不同元素有 $n!$ 种可能顺序，比较一次最多提供一位左右的信息，因此最坏至少需要数量级为 $n \log n$ 的比较。这个结论告诉我们，想突破 $n \log n$ ，必须利用额外条件，例如值域有限、元素是整数、只需要第 K 个而不是完整顺序、或者允许随机化平均复杂度。计数排序利用小值域，桶排序利用分布，基数排序利用数字位，快速选择利用只关心目标排名。这些算法不是比标准排序更高级，而是使用了更强前提。

快速排序的核心是 partition：选择一个枢轴，把小于它的元素放一边，大于它的元素放另一边。partition 完成后，枢轴位置已经确定，左右两边再分别排序。快速选择只进入目标排名所在的一边，因此平均时间为线性。这里的平均性依赖枢轴足够随机；若每次都选到极端枢轴，最坏会退化到平方。面试中写快速选择必须主动说明随机化或三数取中，不然复杂度承诺不完整。

归并排序和快速排序的思维不同。归并排序先分再合，合并两个有序段时保证稳定性，最坏复杂度稳定为 $O(n \log n)$ ，但需要额外空间。链表排序常用归并，因为链表合并不需要随机访问，节点重连成本低；数组排序常用 introsort 一类混合策略，在快速排序、堆排序、插入排序之间切换，兼顾平均性能和最坏情况。理解这些取舍后，才能解释为什么 C++ 的 `std::sort` 不承诺稳定，而 `std::stable_sort` 需要更多空间。

选择第 K 大时有三种主线。完整排序最简单，适合数据量不大或后续还需要有序结果；大小为 K 的堆适合 K 远小于 n 或数据流持续到来；快速选择适合一次性数组、需要平均线性。三种方法没有绝对优劣，差别在输出需求、数据是否在线、实现风险和最坏复杂度。高质量题解应该比较它们，而不是只写一种。

排序题的边界往往出在比较器。比较器必须满足严格弱序，不能写出自相矛盾的关系。区间覆盖题中同起点要按终点降序，合并区间按起点升序即可，会议室按开始时间或结束时间有不同含义。比较器写错时，代码看似能跑，但排序结果不满足算法不变量。每次写比较器都要用三组数据验证：相同主键、完全相同元素、主键相反顺序。

实验建议：实现完整排序、大小为 K 的堆、快速选择三种第 K 大算法，用随机数组、几乎有序数组、逆序数组和大量重复数组测试。记录比较次数或运行时间，观察 K 很小时堆的优势、需要完整有序时排序的优势、随机化快速选择的波动。再把快速选择枢轴固定为首元素，用有序数组制造最坏情况，理解为什么工程库不会使用裸快速排序。

本节复盘

阅读本节后，请回到相邻章节的例题，把暴力瓶颈、优化依据、状态不变量、C++ 容器成本和边界测试重新写一遍。能把同一思想迁移到变体题，才算真正掌握。

第 12 章

堆、Top K 和贪心证明

堆和贪心经常一起出现，但它们不是同一个概念。堆是一种数据结构，负责快速取出当前最大或最小候选；贪心是一种算法策略，声称每一步做局部最优选择不会破坏全局最优。堆题的关键是“当前最该处理的候选是谁”；贪心题的关键是“为什么现在做这个选择以后不会后悔”。面试中只写出 `priority_queue` 远远不够，还要讲清候选含义、过期规则、比较器方向和正确性理由。

C++ 的 `std::priority_queue` 默认是大顶堆。要写小顶堆，需要使用 `std::greater<T>`。如果元素是结构体，需要自定义比较器。比较器的方向经常让人困惑：`priority_queue` 会把“优先级最大”的元素放在 `top`，比较器返回 `true` 表示左侧优先级低于右侧。因此写自定义比较器时，最好用“谁应该排后面”来理解。

Listing 12.1 `priority_queue` 的比较器方向

```
1 struct Entry {
2     int value = 0;
3     int frequency = 0;
4 };
5
6 struct LowerFrequencyFirst {
7     bool operator()(const Entry& lhs, const Entry& rhs) const
8     {
9         return lhs.frequency > rhs.frequency;
10    }
11 };
12
13 using MinHeap = std::priority_queue<Entry,
14                                     std::vector<Entry>,
15                                     LowerFrequencyFirst>;
```

这段比较器让频次更小的元素优先出现在堆顶，因为当 `lhs.frequency > rhs.frequency` 时，`lhs` 被认为优先级更低。很多人会在这里写反，导致小顶堆变成大顶堆。简单类型能用 `std::greater<int>` 时就用标准库；复杂类型必须写清比较器语义，并用小样例验证堆顶是谁。

前 K 个高频元素是哈希表和堆的组合题。暴力做法是统计频次后把所有不同元素按频次排序，复杂度 $O(m \log m)$ ， m 是不同元素个数。如果只要前 K 个，不需要完整排序，可以维护一个大小为 K 的小顶堆。堆顶是当前前 K 高频候选里频次最低的元素；新元素频次如果不超过堆顶，就不可能进入前 K；如果超过，就替换堆顶。

Listing 12.2 前 K 个高频元素：频次表加大小为 K 的小顶堆

```
1 std::vector<int> top_k_frequent(const std::vector<int>& nums, int k)
2 {
3     std::unordered_map<int, int> frequency;
4     frequency.reserve(nums.size());
5     for (const int x : nums) {
6         ++frequency[x];
7     }
8
9     using Entry = std::pair<int, int>; // frequency, value
10    std::priority_queue<Entry, std::vector<Entry>, std::greater<Entry>> heap;
```

```

11
12     for (const auto& [value, count] : frequency) {
13         heap.emplace(count, value);
14         if (static_cast<int>(heap.size()) > k) {
15             heap.pop();
16         }
17     }
18
19     std::vector<int> answer;
20     answer.reserve(k);
21     while (!heap.empty()) {
22         answer.push_back(heap.top().second);
23         heap.pop();
24     }
25     return answer;
26 }

```

这里用 `pair<count, value>` 是为了让 `std::greater` 按频次小顶堆工作；如果频次相同，会按值比较，但题目通常不要求固定顺序。复杂度是统计 $O(n)$ ，堆维护 $O(m \log k)$ 。如果 K 接近 m ，完整排序可能更简单；如果值域有限，也可以用桶排序按频次分桶，达到线性复杂度。面试时要比较这三种方案，而不是只给一种。

数据流中位数展示双堆模型。一个大顶堆保存较小一半，一个小顶堆保存较大一半。两个堆的大小差不超过 1，并且大顶堆堆顶不大于小顶堆堆顶。这样总数为奇数时，中位数是元素更多那一侧的堆顶；总数为偶数时，中位数是两个堆顶平均值。暴力方法每次插入后排序或线性插入，查询中位数容易但插入代价高。

Listing 12.3 数据流中位数：双堆维护两半

```

1 class MedianFinder {
2 public:
3     void add_num(int num)
4     {
5         if (this->lower_.empty() || num <= this->lower_.top()) {
6             this->lower_.push(num);
7         } else {
8             this->upper_.push(num);
9         }
10        this->rebalance();
11    }
12
13    double find_median() const
14    {
15        if (this->lower_.size() == this->upper_.size()) {
16            return (static_cast<double>(this->lower_.top()) +
17                    static_cast<double>(this->upper_.top())) / 2.0;
18        }
19        return this->lower_.size() > this->upper_.size()
20               ? static_cast<double>(this->lower_.top())
21               : static_cast<double>(this->upper_.top());
22    }
23
24 private:
25     void rebalance()

```

```

26     {
27         if (this->lower_.size() > this->upper_.size() + 1) {
28             this->upper_.push(this->lower_.top());
29             this->lower_.pop();
30         } else if (this->upper_.size() > this->lower_.size() + 1) {
31             this->lower_.push(this->upper_.top());
32             this->upper_.pop();
33         }
34     }
35
36     std::priority_queue<int> lower_;
37     std::priority_queue<int, std::vector<int>, std::greater<int>> upper_;
38 };

```

这段代码把重平衡逻辑拆成私有函数，避免 `add_num` 过长。成员访问使用 `this->`，符合 C++ 代码规范。复杂度是插入 $O(\log n)$ ，查询 $O(1)$ 。易错点有两个：偶数个元素时要转成 `double` 避免整数溢出和整数除法；重平衡只保证大小差，不要忘记插入时先按堆顶决定放入哪一侧，否则大小平衡但顺序不平衡。

合并 K 个升序链表是堆在链表上的应用。暴力方法可以每次扫描 K 个链表头，找最小节点接入结果，若总节点数为 N ，复杂度 $O(NK)$ 。堆优化是把每条链表当前头节点放入小顶堆，每次弹出最小节点，再把它后继放入堆。堆里最多 K 个节点，所以复杂度 $O(N \log K)$ 。

Listing 12.4 合并 K 个升序链表：堆保存每条链当前头节点

```

1 ListNode* merge_k_lists(std::vector<ListNode*> lists)
2 {
3     struct CompareNode {
4         bool operator()(const ListNode* lhs, const ListNode* rhs) const
5         {
6             return lhs->value > rhs->value;
7         }
8     };
9
10    std::priority_queue<ListNode*, std::vector<ListNode*>, CompareNode> heap;
11    for (ListNode* node : lists) {
12        if (node != nullptr) {
13            heap.push(node);
14        }
15    }
16
17    ListNode dummy;
18    ListNode* tail = &dummy;
19    while (!heap.empty()) {
20        ListNode* node = heap.top();
21        heap.pop();
22        if (node->next != nullptr) {
23            heap.push(node->next);
24        }
25        tail->next = node;
26        tail = tail->next;
27    }
28    tail->next = nullptr;
29    return dummy.next;

```



```
30 }
```

这里比较器把值更大的节点放到后面，因此堆顶是值最小节点。弹出节点后先把后继放入堆，再接入结果，两个顺序都可以，但最后要确保 `tail->next = nullptr`，避免结果尾部还连着旧链表的残余路径产生误解。这个算法没有创建新节点，复用输入节点；如果不允许修改输入，需要新建节点。面试表达要强调堆里保存的是“每条链表尚未合并部分的最小候选”。

跳跃游戏是贪心的入门题。题目问能否从下标 0 跳到最后。暴力搜索会从每个位置尝试所有可跳长度，存在大量重复状态。DP 可以记录每个位置是否可达，复杂度 $O(n^2)$ 。贪心更简单：扫描过程中维护当前能到达的最远下标 `farthest`。如果扫描到的位置已经超过 `farthest`，说明这个位置不可达；否则用它继续更新最远范围。

Listing 12.5 跳跃游戏：维护当前可达最远位置

```
1 bool can_jump(const std::vector<int>& nums)
2 {
3     int farthest = 0;
4     for (int i = 0; i < static_cast<int>(nums.size()); ++i) {
5         if (i > farthest) {
6             return false;
7         }
8         farthest = std::max(farthest, i + nums[i]);
9         if (farthest >= static_cast<int>(nums.size()) - 1) {
10             return true;
11         }
12     }
13     return true;
14 }
```

这个贪心的正确性来自覆盖区间：只要当前位置 `i` 可达，那么从它出发能扩展的所有位置都被 `farthest` 覆盖；我们不关心具体从哪条路径到达，只关心可达区域的右边界。每次扫描到边界内的新位置，都可能把边界向右推。若某个位置在边界之外，任何之前位置都无法到达它，后面的位置更不可能被访问。

跳跃游戏 II 要求最少跳跃次数。暴力 BFS 可以把每个下标当节点，每条可跳边连接到后续位置，求无权最短路，但边数最坏 $O(n^2)$ 。贪心做法把一跳能到达的范围看成 BFS 的一层：`current_end` 是当前跳数能覆盖的最右位置，扫描这一层内所有点，计算下一跳能到达的最远位置 `farthest`；当扫描到 `current_end`，说明这一层结束，必须增加一次跳跃。

Listing 12.6 跳跃游戏 II：按层扩展可达区间

```
1 int jump(const std::vector<int>& nums)
2 {
3     if (nums.size() <= 1) {
4         return 0;
5     }
6
7     int jumps = 0;
8     int current_end = 0;
9     int farthest = 0;
10
11     for (int i = 0; i < static_cast<int>(nums.size()) - 1; ++i) {
12         farthest = std::max(farthest, i + nums[i]);
13         if (i == current_end) {
14             ++jumps;
15             current_end = farthest;
16         }
17     }
```

```

16     }
17 }
18
19 return jumps;
20 }

```

这段代码本质上是压缩版 BFS。每次增加 `jumps`，代表进入下一层；`current_end` 以内的位置都属于当前层。为什么循环只到倒数第二个位置？因为一旦已经能到达最后，就不需要从最后一个位置再跳。常见错误是在每个位置都加跳数，或者没有区分当前层边界和下一层最远边界。

贪心证明通常有两种表达：交换论证和领先性论证。区间调度用交换论证：把最优解中结束更晚的选择换成结束更早的选择，不会变差。跳跃游戏用领先性论证：贪心维护的可达最远边界始终不落落后于任何其他方案在相同跳数下能到达的边界。面试中给出哪种证明不重要，重要的是不要把“看起来对”当成证明。

任务调度器是堆、计数和构造思维的交叉题。题目给一组任务和冷却时间 `n`，相同任务之间必须至少间隔 `n` 个单位时间，问完成所有任务的最短时间。暴力模拟可以每个时间点扫描所有任务，选择当前可执行且剩余次数最多的任务；如果任务种类多，扫描会重复。堆优化是把可执行任务按剩余次数放进大顶堆，把冷却中的任务放进队列，时间推进时把冷却结束的任务放回堆。

Listing 12.7 任务调度器：堆处理可执行任务，队列处理冷却

```

1 int least_interval(const std::vector<char>& tasks, int n)
2 {
3     std::array<int, 26> count{};
4     for (const char task : tasks) {
5         ++count[task - 'A'];
6     }
7
8     std::priority_queue<int> available;
9     for (const int c : count) {
10         if (c > 0) {
11             available.push(c);
12         }
13     }
14
15     std::queue<std::pair<int, int>> cooling; // ready_time, remaining
16     int time = 0;
17     while (!available.empty() || !cooling.empty()) {
18         ++time;
19         if (!available.empty()) {
20             int remaining = available.top();
21             available.pop();
22             --remaining;
23             if (remaining > 0) {
24                 cooling.emplace(time + n, remaining);
25             }
26         }
27
28         if (!cooling.empty() && cooling.front().first == time) {
29             available.push(cooling.front().second);
30             cooling.pop();
31         }
32     }
33 }

```

```

34     return time;
35 }

```

这个模拟版本容易解释，但还有数学构造解。设最高频任务出现 `max_count` 次，有 `max_kinds` 种任务都达到最高频。最短时间至少是 $(\text{max_count} - 1) * (n + 1) + \text{max_kinds}$ ，因为最高频任务形成 `max_count - 1` 个完整间隔块，最后一组放所有最高频任务。答案还不能小于任务总数，所以取二者最大。模拟适理解过程，公式适合直接求最短长度。面试中给出一种后，最好能说明另一种的直觉。

加油站是贪心的区间累积题。题目给每个站的油量和到下一站的消耗，问是否存在起点能绕一圈。暴力方法从每个站出发模拟一圈，复杂度 $O(n^2)$ 。贪心依据是：如果从 `start` 出发到 `i` 之间某处油量变成负数，那么 `start` 到 `i` 之间任何站都不可能作为合法起点。因为从这些中间站出发，只会少掉 `start` 到该站之前积累的油量，不会更好。

Listing 12.8 加油站：失败区间整体跳过

```

1 int can_complete_circuit(const std::vector<int>& gas,
2                           const std::vector<int>& cost)
3 {
4     int total = 0;
5     int tank = 0;
6     int start = 0;
7
8     for (int i = 0; i < static_cast<int>(gas.size()); ++i) {
9         const int delta = gas[i] - cost[i];
10        total += delta;
11        tank += delta;
12        if (tank < 0) {
13            start = i + 1;
14            tank = 0;
15        }
16    }
17
18    return total >= 0 ? start : -1;
19 }

```

`total < 0` 表示全局油量不足，任何起点都不可能成功。若全局油量足够，局部失败时把起点跳到 `i + 1` 是安全的。这个跳跃是贪心正确性的核心，不是经验优化。加油站题的复盘要把“失败区间整体排除”说清楚，否则代码看起来像玄学。

划分字母区间展示“最后出现位置”如何制造贪心边界。题目要求把字符串划分成尽可能多的片段，使每个字母最多出现在一个片段中。暴力方法枚举所有切分并检查字符集合是否交叉，复杂度高。优化思路是先记录每个字符最后出现的位置。扫描时，当前片段右边界必须至少覆盖已经遇到的所有字符的最后位置；当扫描下标到达这个右边界时，当前片段可以安全切开。

Listing 12.9 划分字母区间：扫描当前片段最远边界

```

1 std::vector<int> partition_labels(const std::string& s)
2 {
3     std::array<int, 26> last{};
4     for (int i = 0; i < static_cast<int>(s.size()); ++i) {
5         last[s[i] - 'a'] = i;
6     }
7
8     std::vector<int> answer;
9     int start = 0;

```

```

10     int end = 0;
11     for (int i = 0; i < static_cast<int>(s.size()); ++i) {
12         end = std::max(end, last[s[i] - 'a']);
13         if (i == end) {
14             answer.push_back(end - start + 1);
15             start = i + 1;
16         }
17     }
18     return answer;
19 }

```

这个贪心的边界语义非常清楚：当前片段中出现过的所有字符，它们的最后出现位置都不超过 `end`。当 `i == end`，说明这些字符不会在后面再次出现，因此切开不会违反题意；如果提前切开，至少有一个字符会跨片段。类似思想也出现在字符串覆盖、最小区间和扫描线问题中：先知道每个对象的最后影响范围，再扫描形成最小合法边界。

IPO 项目选择题把贪心和双堆结合起来。给定初始资本、最多做 k 个项目、每个项目需要资本和带来利润，问最大资本。暴力方法每轮扫描所有项目，找当前资本能做且利润最高的项目，复杂度 $O(kn)$ 。优化方法是先按资本要求排序，用指针把当前资本可负担的项目加入利润大顶堆，每轮从堆顶选择利润最高项目。

Listing 12.10 IPO：按资本解锁项目，用利润堆选择

```

1  int find_maximized_capital(int k,
2                             int capital,
3                             std::vector<int> profits,
4                             std::vector<int> required)
5  {
6      std::vector<std::pair<int, int>> projects;
7      projects.reserve(profits.size());
8      for (int i = 0; i < static_cast<int>(profits.size()); ++i) {
9          projects.emplace_back(required[i], profits[i]);
10     }
11     std::sort(projects.begin(), projects.end());
12
13     std::priority_queue<int> available_profits;
14     int index = 0;
15     for (int round = 0; round < k; ++round) {
16         while (index < static_cast<int>(projects.size()) &&
17             projects[index].first <= capital) {
18             available_profits.push(projects[index].second);
19             ++index;
20         }
21         if (available_profits.empty()) {
22             break;
23         }
24         capital += available_profits.top();
25         available_profits.pop();
26     }
27
28     return capital;
29 }

```

这个问题的局部最优选择是：在当前能做的项目中，选择利润最高的。为什么不会后悔？因为

做项目只会增加资本，不会消耗资本；利润越高，下一轮可选项目集合只会更大，不会更小。这个性质非常关键。如果项目会消耗资源，或者项目之间有依赖，简单贪心就可能失败。堆这里只是让“当前可选项目中利润最高”查询更快。

贪心失败的例子同样重要。零钱兑换若面额是 1, 3, 4，目标 6，贪心先选 4，再选 1、1，共 3 枚；最优是 3、3，共 2 枚。这个反例说明“每次选最大面额”不一定全局最优。判断贪心是否成立，不能只看样例，要找交换论证、单调边界、领先性或 matroid 这类结构。没有证明时，优先退回 DP 或搜索。

重构字符串是堆处理“当前最紧急约束”的例子。题目要求重新排列字符串，使相邻字符不同。暴力回溯可以尝试所有排列，但重复字符多时搜索空间巨大。贪心想法是每次优先使用剩余次数最多的字符，因为它最难安排；但不能连续使用同一个字符，所以每轮从大顶堆取出两个频次最高的不同字符，依次放入答案，再把剩余频次放回堆。若某个字符出现次数超过 $(n+1)/2$ ，必然无法重构。

Listing 12.11 重构字符串：每次取两个最高频字符

```

1 std::string reorganize_string(const std::string& s)
2 {
3     std::array<int, 26> count{};
4     for (const char ch : s) {
5         ++count[ch - 'a'];
6     }
7
8     using Entry = std::pair<int, char>;
9     std::priority_queue<Entry> heap;
10    for (int i = 0; i < static_cast<int>(count.size()); ++i) {
11        if (count[i] > 0) {
12            heap.emplace(count[i], static_cast<char>('a' + i));
13        }
14    }
15
16    std::string answer;
17    answer.reserve(s.size());
18    while (heap.size() >= 2) {
19        auto [first_count, first_char] = heap.top();
20        heap.pop();
21        auto [second_count, second_char] = heap.top();
22        heap.pop();
23
24        answer.push_back(first_char);
25        answer.push_back(second_char);
26        if (--first_count > 0) {
27            heap.emplace(first_count, first_char);
28        }
29        if (--second_count > 0) {
30            heap.emplace(second_count, second_char);
31        }
32    }
33
34    if (!heap.empty()) {
35        auto [remaining_count, remaining_char] = heap.top();
36        if (remaining_count > 1) {
37            return "";
38        }
    }

```

```

39     if (!answer.empty() && answer.back() == remaining_char) {
40         return "";
41     }
42     answer.push_back(remaining_char);
43 }
44
45 return answer;
46 }

```

这个贪心的直觉是“高频字符最危险，越晚处理越可能无处可放”。每次取两个不同字符，可以保证刚放入的两个字符相邻不同，也避免同一个最高频字符连续出现。更简洁的写法是每次取一个字符，并把上一轮用过但还剩次数的字符延迟一轮再放回堆；两种写法本质相同。正确性还可以用必要条件解释：最高频字符需要插入到其他字符形成的空隙里，空隙数量最多是其他字符数加一，因此最高频不能超过 $(n+1)/2$ 。

任务调度器和重构字符串类似，但它要求相同任务之间至少间隔 n 个单位时间，可以插入空闲。模拟做法是每轮取最多 $n+1$ 个不同任务执行，剩余任务放回堆；如果任务还没做完而本轮不足 $n+1$ ，就需要补空闲。数学做法更快：设最高频为 maxFreq ，拥有最高频的任务种类数为 maxCount ，最短时间至少是 $(\text{maxFreq} - 1) * (n + 1) + \text{maxCount}$ ，再和任务总数取最大。

Listing 12.12 任务调度器：按最高频任务计算骨架长度

```

1 int least_interval(const std::vector<char>& tasks, int cooldown)
2 {
3     std::array<int, 26> count{};
4     for (const char task : tasks) {
5         ++count[task - 'A'];
6     }
7
8     int max_frequency = 0;
9     int max_count = 0;
10    for (const int frequency : count) {
11        if (frequency > max_frequency) {
12            max_frequency = frequency;
13            max_count = 1;
14        } else if (frequency == max_frequency) {
15            ++max_count;
16        }
17    }
18
19    const int skeleton =
20        (max_frequency - 1) * (cooldown + 1) + max_count;
21    return std::max(static_cast<int>(tasks.size()), skeleton);
22 }

```

公式的含义是：最高频任务形成 $\text{max_frequency} - 1$ 个完整间隔块，每块长度至少 $\text{cooldown} + 1$ ，最后再放所有最高频任务的尾部。若其他任务足够多，它们可以填满空隙，答案就是任务总数；若不够，就必须空闲，答案由骨架决定。这个题很好地区分了“堆模拟”和“数学贪心”：堆模拟更通用，公式更简洁但依赖对结构的深理解。

课程表 III 是“按截止时间排序 + 最大堆反悔”的经典题。每门课有持续时间和截止时间，要求最多能上多少门。暴力搜索选课集合是指数级。贪心先按截止时间升序处理课程，尽量把当前课程加入计划；如果加入后总时间超过当前截止时间，就从已选课程中删除耗时最长的一门。为什么删最长？因为课程数量减少一门时，删除耗时最长能让总时间降得最多，给后续课程留下最大空间。

Listing 12.13 课程表 III: 超过截止时删除最长课程

```

1 int schedule_course(std::vector<std::vector<int>> courses)
2 {
3     std::sort(courses.begin(), courses.end(),
4               [](const std::vector<int>& lhs, const std::vector<int>& rhs) {
5                 return lhs[1] < rhs[1];
6             });
7
8     std::priority_queue<int> selected_durations;
9     int total_time = 0;
10    for (const std::vector<int>& course : courses) {
11        const int duration = course[0];
12        const int deadline = course[1];
13        selected_durations.push(duration);
14        total_time += duration;
15
16        if (total_time > deadline) {
17            total_time -= selected_durations.top();
18            selected_durations.pop();
19        }
20    }
21
22    return static_cast<int>(selected_durations.size());
23 }

```

这个算法的堆不是为了取下一门课，而是为了在当前已选集合中执行“反悔”。按截止时间排序保证当处理当前课程时，只需要满足当前及更早截止时间；如果超时，必须删掉一门已选课程。删最长课程不会减少已选课程数量以外的可行性，且让总耗时最小。这个证明属于交换论证：如果某个可行方案删的是较短课程而保留较长课程，交换为保留较短、删除较长，课程数量不变，总时间不增。

无重叠区间也能从“反悔”角度理解。按起点排序扫描时，若当前区间和上一个保留区间重叠，就必须删除其中一个。为了给后续留下更多空间，应保留结束更早的那个，删除结束更晚的那个。按终点排序的版本更直接：每次选择结束最早且与已选区间不重叠的区间，最大化保留数量；答案也可以用总数减去最多保留数量。

Listing 12.14 无重叠区间：保留结束更早的区间

```

1 int erase_overlap_intervals(std::vector<std::vector<int>> intervals)
2 {
3     if (intervals.empty()) {
4         return 0;
5     }
6     std::sort(intervals.begin(), intervals.end(),
7               [](const std::vector<int>& lhs, const std::vector<int>& rhs) {
8                 return lhs[0] < rhs[0];
9             });
10
11    int removed = 0;
12    int current_end = intervals[0][1];
13    for (int i = 1; i < static_cast<int>(intervals.size()); ++i) {
14        if (intervals[i][0] >= current_end) {
15            current_end = intervals[i][1];
16            continue;
17        }
18        removed++;
19    }
20    return intervals.size() - removed;
21 }

```

```

17     }
18     ++removed;
19     current_end = std::min(current_end, intervals[i][1]);
20 }
21 return removed;
22 }

```

当发生重叠时, `current_end = min(current_end, intervals[i][1])` 表示保留结束更早的区间。这个写法不显式记录保留哪个区间, 但保留了对后续唯一重要的信息: 当前已保留区间的最小结束时间。贪心题经常不需要保留完整历史, 只保留能决定未来可行性的摘要状态。若题目要求输出具体删除哪些区间, 就需要额外记录选择。

根据身高重建队列是排序和贪心的组合。每个人由 `height` 和 `k` 表示, `k` 是前面身高大于等于自己的人数。若从矮到高放, 后面更高的人插入会影响矮个子的 `k`; 若从高到矮放, 已经放入的人都不矮于当前人, 当前人的 `k` 就等于插入位置。排序规则是身高降序, 身高相同则 `k` 升序, 然后按 `k` 插入结果数组。

Listing 12.15 根据身高重建队列: 高个先放

```

1 std::vector<std::vector<int>> reconstruct_queue(
2     std::vector<std::vector<int>> people)
3 {
4     std::sort(people.begin(), people.end(),
5         [](const std::vector<int>& lhs, const std::vector<int>& rhs) {
6             if (lhs[0] != rhs[0]) {
7                 return lhs[0] > rhs[0];
8             }
9             return lhs[1] < rhs[1];
10        });
11
12     std::vector<std::vector<int>> queue;
13     queue.reserve(people.size());
14     for (const std::vector<int>& person : people) {
15         queue.insert(queue.begin() + person[1], person);
16     }
17     return queue;
18 }

```

这段代码的插入是 $O(n)$, 整体 $O(n^2)$, 但力扣规模通常可接受。若规模很大, 可以用树状数组或线段树找第 `k` 个空位。正确性来自处理顺序: 当插入当前人时, 队列中已有的人都比他高或等高, 因此插入下标正好决定了他前面符合条件的人数; 后续更矮的人插入不会影响他的 `k`。同高的人按 `k` 升序处理, 避免同高较大 `k` 先插入造成位置不稳定。

合并石头、戳气球、最优二叉搜索树这类题看上去也有“每次选局部最小或最大”的诱惑, 但通常不能简单贪心。以合并石头为例, 如果每次合并当前最小的相邻两堆, 不一定得到全局最优, 因为相邻约束会改变未来可合并结构。没有相邻约束的哈夫曼合并可以每次取最小两项, 这是因为任意两项都可合并, 且最优前缀树有明确交换性质; 有相邻约束后, 问题变成区间 DP。这个对比非常重要: 同样是“合并代价”, 约束不同, 算法类型就不同。

柠檬水找零是简单贪心, 但很适合训练“优先保留灵活资源”。顾客支付 5、10、20, 饮料价格 5。收到 10 需要找一个 5; 收到 20 时, 优先找 10+5, 而不是三个 5, 因为 10 只能在找 20 时使用, 5 更灵活, 后续找 10 和 20 都需要。这个证明不是复杂数学, 而是资源灵活性: 越通用的资源越应该保留。若无法找零则返回 `false`。

Listing 12.16 柠檬水找零: 优先使用较不灵活的 10 元


```

1 bool lemonade_change(const std::vector<int>& bills)
2 {
3     int five = 0;
4     int ten = 0;
5     for (const int bill : bills) {
6         if (bill == 5) {
7             ++five;
8         } else if (bill == 10) {
9             if (five == 0) {
10                return false;
11            }
12            --five;
13            ++ten;
14        } else {
15            if (ten > 0 && five > 0) {
16                --ten;
17                --five;
18            } else if (five >= 3) {
19                five -= 3;
20            } else {
21                return false;
22            }
23        }
24    }
25    return true;
26 }

```

这个题也能说明贪心证明可以很朴素。不是所有贪心都需要长篇交换论证，有时只要明确资源的支配关系：一张 5 元可以参与两类找零，一张 10 元只能参与 20 元找零；在能完成当前找零的前提下，消耗更不灵活的资源不会让未来更差。若题目币值变化，这个策略可能失效，因此不能把它泛化成“永远优先用大面额”。

最小加油次数是堆和贪心的又一个“反悔”模型。汽车沿路经过加油站，每个站有位置和油量，目标是用最少加油次数到达终点。暴力搜索在哪些站加油是指数级。贪心扫描所有可达加油站，把它们的油量放入大顶堆；当当前油量无法到达下一个站或终点时，就从已经路过且可选择的站中取油量最大的一个加油。虽然看起来像回头加油，但等价于在之前经过时选择了那个站。

Listing 12.17 最小加油次数：不可达时从已过站点中选最大油量

```

1 int min_refuel_stops(int target,
2                     int start_fuel,
3                     std::vector<std::vector<int>> stations)
4 {
5     stations.push_back({target, 0});
6     std::priority_queue<int> fuels;
7     int fuel = start_fuel;
8     int previous_position = 0;
9     int stops = 0;
10
11     for (const std::vector<int>& station : stations) {
12         const int position = station[0];
13         const int available_fuel = station[1];
14         fuel -= position - previous_position;
15         while (fuel < 0 && !fuels.empty()) {

```

```

16         fuel += fuels.top();
17         fuels.pop();
18         ++stops;
19     }
20     if (fuel < 0) {
21         return -1;
22     }
23     fuels.push(available_fuel);
24     previous_position = position;
25 }
26
27 return stops;
28 }

```

这个算法的正确性来自延迟决策：在还没被迫加油之前，不决定具体在哪个已过站点加；一旦必须加油，为了让未来最容易可达，就选择已过站点中油量最大的。选择更小油量不会减少加油次数，反而可能更早再次陷入不可达。堆中保存的是已经经过但尚未使用的加油机会。这个模型和 IPO、课程表 III 都有相似结构：先收集可用候选，必要时选最有利的一个。

贪心题的讲解应该包含失败测试。跳跃游戏 II 如果每一步只跳到当前能跳到的最远下标，可能不是最少跳数；正确做法是按当前层覆盖范围推进，选择下一层最远覆盖。课程表 III 如果只按持续时间短排序，会错过截止时间约束；如果只按截止时间排序但不反悔，也会被长课程拖垮。重构字符串如果每次只取最高频字符但不延迟上一字符，就可能产生相邻重复。把这些失败原因讲出来，答案才可信。

堆与贪心实验：第一，为零钱兑换构造 **1,3,4** 和目标 6，写出贪心失败过程，再用 DP 修正。第二，给课程表 III 构造一门很长但截止早的课程，观察最大堆如何反悔。第三，用任务调度器分别实现堆模拟和公式法，比较输出一致性。第四，把最小加油次数中的大顶堆改成小顶堆，找一个用例说明为什么会增加加油次数或失败。

堆和贪心题复盘模板

堆题先说明堆里保存什么候选，堆顶为什么是下一步最该处理的对象，候选何时入堆、何时出堆、何时过期。贪心题先给暴力或 DP，再说明局部选择为什么不会影响全局最优，最好用交换论证或领先性论证。遇到 Top K，主动比较排序、大小为 K 的堆、快速选择和桶排序；遇到区间覆盖，主动说明排序键和边界含义。

第 13 章

图论进阶：并查集、最短路和状态图

图论进阶题的难点通常不在 BFS 或 DFS 的基本写法，而在选择正确的图模型。无向连通性适合并查集；无权最短路适合 BFS；边权非负的最短路适合 Dijkstra；存在状态压缩或多维状态时，要先定义状态节点，再决定边和代价。很多题目表面不是图，例如密码锁、单词变化、网格体力消耗、课程安排、账户合并，只要存在“状态”和“一步转移”，就可以用图论语言重写。

并查集解决的是动态连通性。它不回答路径是什么，也不回答最短距离是多少；它只维护“两个元素当前是否属于同一集合”。暴力做法在每次合并后用 DFS 或 BFS 检查连通性，会重复扫描大量边。并查集把每个集合表示成一棵树，用根节点作为代表。路径压缩让查询时把沿途节点直接挂到根上，按大小合并让小树挂到大树下，两者结合后，均摊复杂度接近常数。

Listing 13.1 并查集：迭代 find、路径压缩和按大小合并

```
1 class DisjointSetUnion {
2 public:
3     explicit DisjointSetUnion(int n)
4         : parent_(n), size_(n, 1)
5     {
6         std::iota(this->parent_.begin(), this->parent_.end(), 0);
7     }
8
9     int find(int x)
10    {
11        int root = x;
12        while (this->parent_[root] != root) {
13            root = this->parent_[root];
14        }
15        while (this->parent_[x] != x) {
16            const int next = this->parent_[x];
17            this->parent_[x] = root;
18            x = next;
19        }
20        return root;
21    }
22
23    bool unite(int a, int b)
24    {
25        int root_a = this->find(a);
26        int root_b = this->find(b);
27        if (root_a == root_b) {
28            return false;
29        }
30        if (this->size_[root_a] < this->size_[root_b]) {
31            std::swap(root_a, root_b);
32        }
33        this->parent_[root_b] = root_a;
34        this->size_[root_a] += this->size_[root_b];
35        return true;
36    }
```

```

37
38 private:
39     std::vector<int> parent_;
40     std::vector<int> size_;
41 };

```

这里的 `find` 是迭代写法，避免递归深度问题。第一次循环找根，第二次循环压缩路径。`unite` 返回 `false` 表示两个元素本来已经连通，合并会形成环。这个返回值在很多题里正好就是答案判断条件。成员访问使用 `this->`，让读者清楚哪些状态属于并查集对象。

冗余连接是并查集的标准例题。题目给一棵树额外加一条边，要求找出导致环的边。暴力方法是每加入一条边，就在已有图中检查这两个端点是否已经连通；如果已经连通，再加边就会成环。并查集正好维护这个关系：扫描边时，如果 `unite(u, v)` 返回 `false`，说明这条边连接了同一集合内的两个点，它就是冗余边。

Listing 13.2 冗余连接：合并失败的边就是成环边

```

1 std::vector<int> find_redundant_connection(
2     const std::vector<std::vector<int>>& edges)
3 {
4     DisjointSetUnion dsu(static_cast<int>(edges.size()) + 1);
5     for (const std::vector<int>& edge : edges) {
6         const int u = edge[0];
7         const int v = edge[1];
8         if (!dsu.unite(u, v)) {
9             return edge;
10        }
11    }
12    return {};
13 }

```

这题要注意节点编号通常从 1 开始，所以并查集大小开到 `edges.size() + 1`。如果题目节点编号不连续，就需要哈希映射到连续编号。复杂度近似 $O(n)$ 。面试表达中要把树的性质说出来：一棵 n 个节点的树有 $n-1$ 条边且无环；额外加入一条边后，第一次发现两个端点已经连通的边，就是让无环结构变成有环结构的边。

省份数量也是连通块问题。输入是邻接矩阵，`isConnected[i][j] == 1` 表示城市 i 和 j 直接连接。DFS/BFS 可以做，并查集也可以做。暴力思路可能是从每个城市出发搜索所有可达城市并统计，但会重复访问。并查集做法扫描矩阵上三角，把直接连接的城市合并，最后统计根节点个数。

Listing 13.3 省份数量：邻接矩阵合并连通城市

```

1 int find_circle_num(const std::vector<std::vector<int>>& is_connected)
2 {
3     const int n = static_cast<int>(is_connected.size());
4     DisjointSetUnion dsu(n);
5
6     for (int i = 0; i < n; ++i) {
7         for (int j = i + 1; j < n; ++j) {
8             if (is_connected[i][j] == 1) {
9                 dsu.unite(i, j);
10            }
11        }
12    }
13
14    int provinces = 0;

```



```

15     for (int city = 0; city < n; ++city) {
16         if (dsu.find(city) == city) {
17             ++provinces;
18         }
19     }
20     return provinces;
21 }

```

邻接矩阵是对称的，因此只扫描上三角即可。这里并查集的优势不是绝对性能，而是模型清晰：省份就是连通分量，直接连接就是合并操作。若题目还要求列出每个省份包含哪些城市，则并查集统计完根后还要按根分组；若题目要求最短路径，并查集就不够，因为它丢掉了路径长度信息。

Dijkstra 处理非负边权最短路。无权图 BFS 成立，是因为每条边代价相同，按层访问保证第一次到达最短；带权图中，一条边可能代价很大，普通 BFS 不再可靠。Dijkstra 的贪心点是：每次从尚未确定的节点中取当前距离最小者，它的最短距离已经确定。这个结论依赖所有边权非负，因为后续绕路不可能通过负边把距离降得更低。

网络延迟时间是 Dijkstra 的典型题。节点是服务器，有向边带传播时间。题目问从起点发信号，到所有节点都收到信号需要多久，也就是起点到所有节点最短路的最大值。暴力方法可以反复松弛所有边，类似 Bellman-Ford，复杂度 $O(nm)$ 。由于边权非负，用堆优化 Dijkstra 更合适。

Listing 13.4 网络延迟时间：堆优化 Dijkstra

```

1  int network_delay_time(const std::vector<std::vector<int>>& times,
2                          int n,
3                          int source)
4  {
5      using Edge = std::pair<int, int>; // neighbor, weight
6      std::vector<std::vector<Edge>> graph(n + 1);
7      for (const std::vector<int>& edge : times) {
8          graph[edge[0]].emplace_back(edge[1], edge[2]);
9      }
10
11     constexpr int INF = 1'000'000'000;
12     std::vector<int> distance(n + 1, INF);
13     distance[source] = 0;
14
15     using State = std::pair<int, int>; // distance, node
16     std::priority_queue<State, std::vector<State>, std::greater<State>> heap;
17     heap.emplace(0, source);
18
19     while (!heap.empty()) {
20         const auto [dist, node] = heap.top();
21         heap.pop();
22         if (dist != distance[node]) {
23             continue;
24         }
25
26         for (const auto [next, weight] : graph[node]) {
27             if (dist + weight < distance[next]) {
28                 distance[next] = dist + weight;
29                 heap.emplace(distance[next], next);
30             }
31         }
32     }

```

```

33
34     int answer = 0;
35     for (int node = 1; node <= n; ++node) {
36         if (distance[node] == INF) {
37             return -1;
38         }
39         answer = std::max(answer, distance[node]);
40     }
41     return answer;
42 }

```

这段代码没有使用显式 `visited`，而是用 `dist != distance[node]` 跳过堆中的过期状态。因为 C++ 标准堆不支持 `decrease-key`，更新距离时直接把新状态再压入堆，旧状态留在堆里，弹出时识别并跳过。复杂度是 $O((n+m)\log m)$ 或常写为 $O(m\log n)$ 。如果存在负边，Dijkstra 不适用，应考虑 Bellman-Ford 或 SPFA 的风险分析。

最小体力消耗路径看似网格题，实质是最短路变体。路径代价不是边权之和，而是路径上最大高度差的最小可能值。状态是格子，边是上下左右相邻格子，走一条边的代价是高度差。到达某个格子的最优体力定义为：从起点到它的路径中，最大边权尽可能小。转移时，新代价是 **max(当前路径体力, 新边高度差)**。这个代价仍满足 Dijkstra 的非负和单调扩展性质。

Listing 13.5 最小体力路径：以最大边权作为路径代价

```

1  int minimum_effort_path(const std::vector<std::vector<int>>& heights)
2  {
3      constexpr std::array<std::pair<int, int>, 4> DIRECTIONS{
4          std::pair{1, 0}, std::pair{-1, 0},
5          std::pair{0, 1}, std::pair{0, -1}
6      };
7      const int rows = static_cast<int>(heights.size());
8      const int cols = static_cast<int>(heights[0].size());
9      constexpr int INF = 1'000'000'000;
10
11     std::vector<std::vector<int>> effort(rows, std::vector<int>(cols, INF));
12     effort[0][0] = 0;
13
14     using State = std::tuple<int, int, int>; // effort, row, col
15     std::priority_queue<State, std::vector<State>, std::greater<State>> heap;
16     heap.emplace(0, 0, 0);
17
18     while (!heap.empty()) {
19         const auto [cost, row, col] = heap.top();
20         heap.pop();
21         if (row == rows - 1 && col == cols - 1) {
22             return cost;
23         }
24         if (cost != effort[row][col]) {
25             continue;
26         }
27
28         for (const auto [dr, dc] : DIRECTIONS) {
29             const int next_row = row + dr;
30             const int next_col = col + dc;
31             const bool out_of_bounds =

```

```

32         next_row < 0 || next_row >= rows ||
33         next_col < 0 || next_col >= cols;
34     if (out_of_bounds) {
35         continue;
36     }
37
38     const int edge_cost =
39         std::abs(heights[row][col] - heights[next_row][next_col]);
40     const int next_cost = std::max(cost, edge_cost);
41     if (next_cost < effort[next_row][next_col]) {
42         effort[next_row][next_col] = next_cost;
43         heap.emplace(next_cost, next_row, next_col);
44     }
45 }
46 }
47
48 return 0;
49 }

```

这题也可以二分答案加 BFS：猜一个最大允许体力，看能否只走高度差不超过该值的边到终点。二分答案复杂度是 $O(mn \log H)$ ，Dijkstra 是 $O(mn \log(mn))$ 。两种方法都合理，面试时可以先给更统一的 Dijkstra，再说明二分可行的原因是“体力上限越大，可达性越强”，存在单调性。

状态图题还包括打开转盘锁、单词接龙、蛇梯棋等。共同点是输入没有直接给边，需要你根据规则生成邻居。生成邻居时要注意两件事：第一，状态编码要能高效去重；第二，访问标记要在入队时设置，避免同一状态被多个父状态重复加入。无权最短路使用 BFS，第一次到达目标即为最短；带权状态图则要换 Dijkstra 或 A*，不能强行 BFS。

真正写图论题之前，还要先决定图如何存储。邻接矩阵适合节点数量较小、需要频繁判断任意两点是否有边的场景，空间是 $O(n^2)$ ；邻接表适合稀疏图，空间是 $O(n + m)$ ；边数组适合 Kruskal、Bellman-Ford 这类按边扫描的算法；网格图通常不需要显式建图，用坐标和方向数组现场生成邻居即可。很多低质量答案只说“建图”，却没有说明为什么这样建。面试中应该先看输入规模：如果 n 可以到十万，邻接矩阵基本不可能；如果 m 远小于 n^2 ，邻接表更自然；如果题目要求按边权排序，边数组会比邻接表更直接。

邻接表也有工程差异。`vector<vector<pair<int,int>>>` 写法直观，但每个内层 `vector` 都是独立对象，极端情况下会产生很多小分配；竞赛中常见的前向星把所有边放进几个连续数组，通过 `head` 和 `next` 串起来，局部性更好，但代码可读性低。刷题面试通常以正确性和表达清晰为第一优先，选 `vector` 邻接表即可；当题目数据极大或需要讲性能时，可以补充连续边数组的思路。这个判断体现的是工程能力：知道什么时候写简单代码，什么时候需要控制内存布局。

Listing 13.6 边数组转邻接表：保留输入语义并便于最短路

```

1 struct WeightedEdge {
2     int from;
3     int to;
4     int weight;
5 };
6
7 std::vector<std::vector<std::pair<int, int>>> build_directed_graph(
8     int node_count,
9     const std::vector<WeightedEdge>& edges)
10 {
11     std::vector<std::vector<std::pair<int, int>>> graph(node_count);
12     for (const WeightedEdge& edge : edges) {
13         graph[edge.from].emplace_back(edge.to, edge.weight);

```

```

14     }
15     return graph;
16 }

```

这段代码看似简单，但它体现了一个重要习惯：输入边和算法图结构分开。`WeightedEdge` 保留题目给出的边语义，邻接表服务于后续算法。若后面同时要跑 Kruskal 和 Dijkstra，就不必从邻接表反推出边数组。很多工程 bug 来自过早把所有信息塞进一种结构，后面算法一换就需要反复转换。

BFS 的本质是按距离层扩展。普通队列能保证这一点，因为所有边权都是 1。访问标记必须在入队时设置，而不是出队时设置；如果出队时才标记，一个节点可能被同一层的多个父节点重复入队，轻则浪费时间，重则在记录父节点或计数时产生错误。多源 BFS 只是把多个起点同时放入队列，把它们的初始距离都设为 0。腐烂的橘子、地图分析、离所有建筑最近的空地等题，都可以用这个模型理解。

Listing 13.7 多源 BFS：所有起点同时入队

```

1  int shortest_distance_from_sources(
2      const std::vector<std::vector<int>>& grid,
3      const std::vector<std::pair<int, int>>& sources)
4  {
5      constexpr std::array<std::pair<int, int>, 4> DIRECTIONS{
6          std::pair{1, 0}, std::pair{-1, 0},
7          std::pair{0, 1}, std::pair{0, -1}
8      };
9      const int rows = static_cast<int>(grid.size());
10     const int cols = static_cast<int>(grid[0].size());
11
12     std::vector<std::vector<int>> distance(rows, std::vector<int>(cols, -1));
13     std::queue<std::pair<int, int>> queue;
14     for (const auto [row, col] : sources) {
15         distance[row][col] = 0;
16         queue.emplace(row, col);
17     }
18
19     int farthest = 0;
20     while (!queue.empty()) {
21         const auto [row, col] = queue.front();
22         queue.pop();
23         farthest = std::max(farthest, distance[row][col]);
24
25         for (const auto [dr, dc] : DIRECTIONS) {
26             const int next_row = row + dr;
27             const int next_col = col + dc;
28             const bool out_of_bounds =
29                 next_row < 0 || next_row >= rows ||
30                 next_col < 0 || next_col >= cols;
31             if (out_of_bounds || grid[next_row][next_col] == 1 ||
32                 distance[next_row][next_col] != -1) {
33                 continue;
34             }
35             distance[next_row][next_col] = distance[row][col] + 1;
36             queue.emplace(next_row, next_col);
37         }

```



```

38     }
39
40     return farthest;
41 }

```

这段模板里，`distance == -1` 同时承担“尚未访问”和“距离未知”的含义。若题目需要允许障碍、空地、起点不同类型，就不要直接修改原数组，单独开 `distance` 会更安全。复杂度是 $O(mn)$ ，因为每个格子最多入队一次，每条网格边最多检查常数次数。面试时要强调：多源 BFS 不是从每个源分别 BFS，而是把所有源放在第 0 层同时扩散；这样第一次到达某个点的源，就是离它最近的一批源之一。

0-1 BFS 是普通 BFS 和 Dijkstra 之间的桥梁。当边权只可能是 0 或 1 时，使用优先队列当然正确，但 `deque` 可以把复杂度降到线性。权重为 0 的转移放到队首，权重为 1 的转移放到队尾。这样队列中的状态仍然按照非降距离处理，只是用双端队列代替了堆。最典型的题是网格中每个格子给出推荐方向，顺着方向走代价为 0，改方向代价为 1，求从左上到右下的最小修改次数。

Listing 13.8 0-1 BFS：零代价边入队首，一代价边入队尾

```

1 int min_cost_valid_path(const std::vector<std::vector<int>>& grid)
2 {
3     constexpr std::array<std::pair<int, int>, 4> DIRECTIONS{
4         std::pair{0, 1}, std::pair{0, -1},
5         std::pair{1, 0}, std::pair{-1, 0}
6     };
7     constexpr int INF = 1'000'000'000;
8     const int rows = static_cast<int>(grid.size());
9     const int cols = static_cast<int>(grid[0].size());
10
11     std::vector<std::vector<int>> distance(rows, std::vector<int>(cols, INF));
12     std::deque<std::pair<int, int>> deque;
13     distance[0][0] = 0;
14     deque.emplace_front(0, 0);
15
16     while (!deque.empty()) {
17         const auto [row, col] = deque.front();
18         deque.pop_front();
19
20         for (int dir = 0; dir < static_cast<int>(DIRECTIONS.size()); ++dir) {
21             const auto [dr, dc] = DIRECTIONS[dir];
22             const int next_row = row + dr;
23             const int next_col = col + dc;
24             const bool out_of_bounds =
25                 next_row < 0 || next_row >= rows ||
26                 next_col < 0 || next_col >= cols;
27             if (out_of_bounds) {
28                 continue;
29             }
30
31             const int extra_cost = (grid[row][col] == dir + 1) ? 0 : 1;
32             const int next_distance = distance[row][col] + extra_cost;
33             if (next_distance >= distance[next_row][next_col]) {
34                 continue;
35             }
36

```

```

37         distance[next_row][next_col] = next_distance;
38         if (extra_cost == 0) {
39             deque.emplace_front(next_row, next_col);
40         } else {
41             deque.emplace_back(next_row, next_col);
42         }
43     }
44 }
45
46 return distance[rows - 1][cols - 1];
47 }

```

0-1 BFS 的正确性可以这样解释：普通 BFS 每次扩展当前最小层；Dijkstra 每次从堆里取最小距离；当新增边代价只有 0 或 1 时，当前状态的后继要么仍在当前距离层，要么进入下一距离层。放队首和队尾正好维护了这个层次。易错点有三个：第一，不能用普通 `queue`，因为 0 代价边应该优先处理；第二，不能只用 `visited`，因为某个状态可能先以较大代价到达，后来被较小代价改进；第三，方向编号要和题目约定对齐，示例里题目方向从 1 开始，所以比较时使用 `dir + 1`。

拓扑排序处理有向无环图。它回答的不是最短路，而是依赖顺序：如果一件事必须在另一件事之前完成，就可以画一条有向边。课程表就是标准题。暴力想法是不断找没有前置依赖的课程，删除它，再更新其他课程；拓扑排序把这个过程用入度数组和队列高效实现。若最后处理课程数小于总课程数，说明存在环，环内课程互相等待，无法完成。

Listing 13.9 课程表：Kahn 拓扑排序判断是否有环

```

1 bool can_finish(int course_count,
2                 const std::vector<std::vector<int>>& prerequisites)
3 {
4     std::vector<std::vector<int>> graph(course_count);
5     std::vector<int> indegree(course_count, 0);
6
7     for (const std::vector<int>& relation : prerequisites) {
8         const int course = relation[0];
9         const int dependency = relation[1];
10        graph[dependency].push_back(course);
11        ++indegree[course];
12    }
13
14    std::queue<int> ready;
15    for (int course = 0; course < course_count; ++course) {
16        if (indegree[course] == 0) {
17            ready.push(course);
18        }
19    }
20
21    int processed = 0;
22    while (!ready.empty()) {
23        const int course = ready.front();
24        ready.pop();
25        ++processed;
26
27        for (const int next : graph[course]) {
28            --indegree[next];
29            if (indegree[next] == 0) {

```

```

30         ready.push(next);
31     }
32 }
33 }
34
35 return processed == course_count;
36 }

```

这题最容易写反边。题目常写成 $[a, b]$ 表示想学 a 必须先学 b ，所以边应该是 $b \rightarrow a$ 。入度表示还有多少依赖没有满足。队列中放的是当前已经没有前置依赖的课程。正确性来自不变量：队列中的每门课入度为 0，已经可以安排；处理它等价于完成这门课，因此它指向的后继少一个未满足依赖。若图中有环，环上每个点至少依赖环内另一个点，永远不会全部入队。

拓扑排序不只判断能不能完成，还能在 DAG 上做动态规划。只要图无环，就可以按拓扑顺序保证前驱状态已经计算完，再更新后继。比如有向无环图中求从起点到每个点的最长路径，不能用 Dijkstra，因为目标是最长而不是最短；也不能在有环图上直接做，因为正权环会让最长路径没有上界。DAG 的拓扑顺序让“阶段”变得明确。

Listing 13.10 DAG 最长路径：拓扑序上的动态规划

```

1 std::vector<int> longest_path_in_dag(
2     int node_count,
3     const std::vector<std::tuple<int, int, int>>& edges,
4     int source)
5 {
6     std::vector<std::vector<std::pair<int, int>>> graph(node_count);
7     std::vector<int> indegree(node_count, 0);
8     for (const auto [from, to, weight] : edges) {
9         graph[from].emplace_back(to, weight);
10        ++indegree[to];
11    }
12
13    std::queue<int> ready;
14    for (int node = 0; node < node_count; ++node) {
15        if (indegree[node] == 0) {
16            ready.push(node);
17        }
18    }
19
20    constexpr int NEG_INF = -1'000'000'000;
21    std::vector<int> distance(node_count, NEG_INF);
22    distance[source] = 0;
23
24    while (!ready.empty()) {
25        const int node = ready.front();
26        ready.pop();
27        if (distance[node] != NEG_INF) {
28            for (const auto [next, weight] : graph[node]) {
29                distance[next] = std::max(distance[next], distance[node] + weight);
30            }
31        }
32        for (const auto [next, weight] : graph[node]) {
33            (void)weight;
34            --indegree[next];

```

```

35         if (indegree[next] == 0) {
36             ready.push(next);
37         }
38     }
39 }
40
41 return distance;
42 }

```

上面代码故意把“松弛后继”和“减少入度”分开写，便于读者看清两个动作。实际工程中可以合并循环，但教材里先强调概念更重要。DAG DP 的状态是 `distance[v]`，含义是从 `source` 到 `v` 的最大路径和；转移来自所有前驱。拓扑序保证当某个节点出队时，它的所有前驱已经处理完，状态不再被未来节点回头修改。这个结构和一维 DP 很像：所谓“阶段顺序”，在图上就是拓扑顺序。

如果图中存在负边，Dijkstra 的贪心基础会崩塌。因为一个当前距离较大的节点，未来可能通过负边把另一个已经确定的节点拉得更小。Bellman-Ford 的想法更朴素：一条最短路径如果最多包含 k 条边，那么做 k 轮按边松弛就能得到它。经典最短路最多需要 $n - 1$ 条边，因为无负环的最短简单路径不会重复经过节点。力扣中“最多经过 K 站中转”的机票题，本质是限制边数的 Bellman-Ford。

Listing 13.11 K 站中转内最便宜航班：按边数分层松弛

```

1 int find_cheapest_price(int n,
2                          const std::vector<std::vector<int>>& flights,
3                          int source,
4                          int target,
5                          int max_stops)
6 {
7     constexpr int INF = 1'000'000'000;
8     std::vector<int> previous(n, INF);
9     previous[source] = 0;
10
11     for (int step = 0; step <= max_stops; ++step) {
12         std::vector<int> current = previous;
13         for (const std::vector<int>& flight : flights) {
14             const int from = flight[0];
15             const int to = flight[1];
16             const int price = flight[2];
17             if (previous[from] == INF) {
18                 continue;
19             }
20             current[to] = std::min(current[to], previous[from] + price);
21         }
22         previous = std::move(current);
23     }
24
25     return previous[target] == INF ? -1 : previous[target];
26 }

```

这里必须使用 `previous` 和 `current` 两层数组，不能在同一个数组里原地更新。原因是第 `step` 轮只允许使用最多 `step + 1` 条边，如果原地更新，一轮内刚更新出来的状态又被后面的边继续使用，就会偷偷使用更多边。这个问题和 0-1 背包的倒序遍历本质类似：转移要读取上一阶段，不能让当前阶段污染自己。复杂度是 $O((K + 1)m)$ ，空间 $O(n)$ 。

最小生成树回答的是“把所有点连起来的最低总成本”，它不是最短路。最短路关心从某个起点到其他点的路径长度；最小生成树关心选出一组边，让全图连通且总权重最小。Kruskal 的策略是按

边权从小到大扫描，能连接两个不同连通块就选，否则跳过。并查集用来判断一条边是否会成环。这个贪心正确性来自割性质：任意一个割上，权重最小的跨割边一定存在某棵最小生成树包含它。

Listing 13.12 Kruskal：按边权连接不同连通块

```

1 struct UndirectedEdge {
2     int u;
3     int v;
4     int weight;
5 };
6
7 int minimum_spanning_tree_cost(int node_count,
8                                 std::vector<UndirectedEdge> edges)
9 {
10     std::sort(edges.begin(), edges.end(),
11               [](const UndirectedEdge& lhs, const UndirectedEdge& rhs) {
12                 return lhs.weight < rhs.weight;
13             });
14
15     DisjointSetUnion dsu(node_count);
16     int total_cost = 0;
17     int used_edges = 0;
18
19     for (const UndirectedEdge& edge : edges) {
20         if (!dsu.unite(edge.u, edge.v)) {
21             continue;
22         }
23         total_cost += edge.weight;
24         ++used_edges;
25         if (used_edges == node_count - 1) {
26             return total_cost;
27         }
28     }
29
30     return -1;
31 }

```

面试中区分 Kruskal 和 Prim 的方式很简单：Kruskal 站在“边”的视角，排序后用并查集选边；Prim 站在“点”的视角，从一个连通块开始，每次选连接外部点的最小边。边比较少、已经容易生成边数组时，Kruskal 很顺；完全图且边很多时，Prim 可以避免显式存储所有边。连接所有点的最小费用中，点与点之间距离是曼哈顿距离，朴素建完全图有 $O(n^2)$ 条边， n 不大时可以接受；如果规模上去，就要研究几何图优化，而不是盲目套模板。

并查集还能和二分答案结合。最小体力路径除了 Dijkstra，也可以二分一个体力上限 `limit`，只把高度差不超过 `limit` 的相邻格子合并，最后看起点和终点是否连通。若某个 `limit` 可行，那么更大的上限一定可行；若不可行，更小的上限一定不可行。这就是二分的单调性。这个方法的优点是解释直观，缺点是每次检查都要重新扫描网格。

Listing 13.13 最小体力路径：二分答案加并查集

```

1 bool can_reach_with_effort(const std::vector<std::vector<int>>& heights,
2                             int limit)
3 {
4     const int rows = static_cast<int>(heights.size());
5     const int cols = static_cast<int>(heights[0].size());

```

```

6   DisjointSetUnion dsu(rows * cols);
7
8   auto id = [cols](int row, int col) {
9       return row * cols + col;
10  };
11
12  for (int row = 0; row < rows; ++row) {
13      for (int col = 0; col < cols; ++col) {
14          if (row + 1 < rows &&
15              std::abs(heights[row][col] - heights[row + 1][col]) <= limit) {
16              dsu.unite(id(row, col), id(row + 1, col));
17          }
18          if (col + 1 < cols &&
19              std::abs(heights[row][col] - heights[row][col + 1]) <= limit) {
20              dsu.unite(id(row, col), id(row, col + 1));
21          }
22      }
23  }
24
25  return dsu.find(0) == dsu.find(rows * cols - 1);
26 }
27
28 int minimum_effort_path_by_binary_search(
29     const std::vector<std::vector<int>>& heights)
30 {
31     int low = 0;
32     int high = 1'000'000;
33     while (low < high) {
34         const int mid = low + (high - low) / 2;
35         if (can_reach_with_effort(heights, mid)) {
36             high = mid;
37         } else {
38             low = mid + 1;
39         }
40     }
41     return low;
42 }

```

这段代码只合并向下和向右的边，是因为无向网格中每条相邻边检查一次即可。复杂度是 $O(mn \log H \cdot \alpha(mn))$ ，其中 H 是高度差范围。和 Dijkstra 相比，二分并查集依赖答案单调性；Dijkstra 依赖路径代价的单调扩展性质。两种方法各有适用场景。面试时能同时说出这两种建模，说明你不是在背题，而是在识别“最小化最大值”的结构。

账户合并是并查集的另一个高频场景。题目给出若干账户，每个账户包含姓名和邮箱；只要两个账户共享邮箱，就属于同一个人。暴力方法是两两比较账户邮箱集合，复杂度可能达到 $O(n^2s)$ ，其中 s 是邮箱数量。更好的模型是：邮箱是节点，同一个账户里的所有邮箱属于同一集合。扫描账户时，把第一个邮箱和后续邮箱合并；最后按根收集邮箱，再排序输出。

Listing 13.14 账户合并：把邮箱映射为并查集节点

```

1  std::vector<std::vector<std::string>> accounts_merge(
2      const std::vector<std::vector<std::string>>& accounts)
3  {
4      std::unordered_map<std::string, int> email_id;

```

```

5     std::unordered_map<std::string, std::string> email_name;
6
7     for (const std::vector<std::string>& account : accounts) {
8         const std::string& name = account[0];
9         for (int i = 1; i < static_cast<int>(account.size()); ++i) {
10            const std::string& email = account[i];
11            if (!email_id.contains(email)) {
12                const int id = static_cast<int>(email_id.size());
13                email_id.emplace(email, id);
14            }
15            email_name[email] = name;
16        }
17    }
18
19    DisjointSetUnion dsu(static_cast<int>(email_id.size()));
20    for (const std::vector<std::string>& account : accounts) {
21        const int first_id = email_id[account[1]];
22        for (int i = 2; i < static_cast<int>(account.size()); ++i) {
23            dsu.unite(first_id, email_id[account[i]]);
24        }
25    }
26
27    std::unordered_map<int, std::vector<std::string>> groups;
28    for (const auto& [email, id] : email_id) {
29        groups[dsu.find(id)].push_back(email);
30    }
31
32    std::vector<std::vector<std::string>> answer;
33    answer.reserve(groups.size());
34    for (auto& [root, emails] : groups) {
35        (void)root;
36        std::sort(emails.begin(), emails.end());
37        std::vector<std::string> merged;
38        merged.reserve(emails.size() + 1);
39        merged.push_back(email_name[emails.front()]);
40        merged.insert(merged.end(), emails.begin(), emails.end());
41        answer.push_back(std::move(merged));
42    }
43    return answer;
44 }

```

这里的关键不是并查集代码，而是节点选择。若把账户编号作为节点，也能合并共享邮箱的账户，但需要用邮箱反查已经出现过的账户编号；若把邮箱作为节点，则最后天然得到邮箱分组。由于输出要求邮箱有序，每个组还要排序。姓名通常不用于判断合并，只用于输出；如果真实业务中同一邮箱对应多个姓名，就要定义冲突规则，不能默认题目条件成立。刷题题解可以借题目保证，工程代码必须把数据一致性说清楚。

单词接龙是状态图建模的代表。每个单词是节点，若两个单词只差一个字符，就有一条无权边。暴力建图需要两两比较单词，复杂度高。更好的做法是用通配模式建立中间索引，例如 **hot** 可以生成 ***ot**、**h*t**、**ho***。所有拥有同一通配模式的单词互为相邻候选。搜索时从当前单词生成模式，再找到候选单词。由于边权相同，用 BFS，第一次到达目标就是最短转换长度。

Listing 13.15 单词接龙：通配模式建索引后 BFS

```

1 int ladder_length(const std::string& begin_word,
2                  const std::string& end_word,
3                  const std::vector<std::string>& word_list)
4 {
5     std::unordered_set<std::string> dictionary(word_list.begin(), word_list.end());
6     if (!dictionary.contains(end_word)) {
7         return 0;
8     }
9     dictionary.insert(begin_word);
10
11     std::unordered_map<std::string, std::vector<std::string>> buckets;
12     for (const std::string& word : dictionary) {
13         for (int i = 0; i < static_cast<int>(word.size()); ++i) {
14             std::string pattern = word;
15             pattern[i] = '*';
16             buckets[pattern].push_back(word);
17         }
18     }
19
20     std::queue<std::string> queue;
21     std::unordered_map<std::string, int> distance;
22     queue.push(begin_word);
23     distance[begin_word] = 1;
24
25     while (!queue.empty()) {
26         const std::string word = queue.front();
27         queue.pop();
28         if (word == end_word) {
29             return distance[word];
30         }
31
32         for (int i = 0; i < static_cast<int>(word.size()); ++i) {
33             std::string pattern = word;
34             pattern[i] = '*';
35             for (const std::string& next : buckets[pattern]) {
36                 if (distance.contains(next)) {
37                     continue;
38                 }
39                 distance[next] = distance[word] + 1;
40                 queue.push(next);
41             }
42         }
43     }
44
45     return 0;
46 }

```

这段代码为了表达清楚保留了字符串拷贝。面试里可以进一步优化：每个模式桶被访问后清空，避免不同路径反复扫描同一桶；也可以使用双向 BFS，从起点集合和终点集合中较小的一侧扩展，降低搜索宽度。暴力为什么慢？因为单词数量为 n 、长度为 L 时，两两比较是 $O(n^2L)$ ；通配索引构建约为 $O(nL^2)$ ，搜索时主要消耗在模式生成和桶扫描。若使用 `string_view` 或预编码单词，工程上还能减少临时字符串分配，但面试中要先保证模型清楚。

双向 BFS 的优化依据是搜索树宽度。单向 BFS 深度为 d 、平均分支因子为 b 时，可能访问约

b^d 个状态；双向 BFS 从两端各扩展约 $d/2$ 层，理想情况下访问量接近 $2b^{d/2}$ 。这不是改变最坏复杂度的魔法，而是利用了许多状态图的指数扩展特征。写双向 BFS 时，永远扩展当前较小的一侧，并在生成新状态时检查它是否已经出现在另一侧。如果相遇，当前步数就是答案。

Listing 13.16 打开转盘锁：双向 BFS 的状态集合扩展

```

1 int open_lock(const std::vector<std::string>& deadends,
2               const std::string& target)
3 {
4     const std::unordered_set<std::string> dead(deadends.begin(), deadends.end());
5     const std::string start = "0000";
6     if (dead.contains(start)) {
7         return -1;
8     }
9     if (target == start) {
10        return 0;
11    }
12
13    auto neighbors = [](const std::string& state) {
14        std::vector<std::string> result;
15        result.reserve(8);
16        for (int i = 0; i < static_cast<int>(state.size()); ++i) {
17            for (const int delta : {-1, 1}) {
18                std::string next = state;
19                const int digit = state[i] - '0';
20                next[i] = static_cast<char>('0' + (digit + delta + 10) % 10);
21                result.push_back(std::move(next));
22            }
23        }
24        return result;
25    };
26
27    std::unordered_set<std::string> front{start};
28    std::unordered_set<std::string> back{target};
29    std::unordered_set<std::string> visited{start, target};
30    int steps = 0;
31
32    while (!front.empty() && !back.empty()) {
33        if (front.size() > back.size()) {
34            std::swap(front, back);
35        }
36        std::unordered_set<std::string> next_front;
37        ++steps;
38
39        for (const std::string& state : front) {
40            for (const std::string& next : neighbors(state)) {
41                if (dead.contains(next) || visited.contains(next)) {
42                    continue;
43                }
44                if (back.contains(next)) {
45                    return steps;
46                }
47                visited.insert(next);
48                next_front.insert(next);

```

```

49     }
50     }
51     front = std::move(next_front);
52 }
53
54 return -1;
55 }

```

转盘锁的状态空间最多一万个，因此普通 BFS 也足够；这里展示双向 BFS，是为了训练状态集合扩展的写法。注意 `visited` 同时包含两端访问过的状态，避免重复回头。若先检查 `visited` 再检查 `back`，可能会错过相遇状态，所以上面代码对新状态的处理顺序是：死点跳过，已访问跳过，相遇返回，再加入下一层。也可以把相遇检查放在已访问之前，并单独维护两侧访问集；关键是语义一致。

图题中还有一类“边不是原题里的动作，而是你定义出来的关系”。比如岛屿数量把相邻陆地视为同一连通块；方程除法把变量视为节点、比例视为带权有向边；除法求值时， $a / b = 2$ 可以建立 $a \rightarrow b$ 权重 2 和 $b \rightarrow a$ 权重 $1/2$ ，查询 x / y 就是从 x 到 y 的路径乘积。这个模型不适合普通最短路，因为路径权重不是相加，而是相乘；但 DFS/BFS 沿路径累计乘积即可。若查询很多，还可以用带权并查集维护每个节点到根的比例。

带权并查集比普通并查集难，因为集合关系不仅是“是否连通”，还要维护相对权值。以方程除法为例，`weight[x]` 可以表示 $x / \text{parent}[x]$ 。路径压缩时，需要同时更新 x / root 。合并 x 和 y 且已知 $x / y = \text{value}$ 时，若令 `root_x` 挂到 `root_y`，就要计算 $\text{root}_x / \text{root}_y$ 的值。这个推导容易错，因此面试中如果时间紧，先给 BFS 图解法更稳；只有查询量大或题目明确动态合并时，再写带权并查集。

网格图的另一个高频变化是“状态不只包含位置”。最短路径消除障碍物中，状态应为 `(row, col, remaining)`，表示走到某个格子时还剩多少次消除机会。若只用 `visited[row][col]`，会错误地剪掉更优状态：同样到达一个格子，剩余消除次数越多，后续能力越强。正确的访问标记可以是三维布尔数组，也可以记录到达每个格子时见过的最大剩余次数；当新状态剩余次数不超过历史最好时，才可以剪掉。

Listing 13.17 障碍消除最短路：位置加资源才是完整状态

```

1 int shortest_path_with_obstacle_elimination(
2     const std::vector<std::vector<int>>& grid,
3     int eliminations)
4 {
5     constexpr std::array<std::pair<int, int>, 4> DIRECTIONS{
6         std::pair{1, 0}, std::pair{-1, 0},
7         std::pair{0, 1}, std::pair{0, -1}
8     };
9     const int rows = static_cast<int>(grid.size());
10    const int cols = static_cast<int>(grid[0].size());
11
12    struct State {
13        int row;
14        int col;
15        int remaining;
16        int steps;
17    };
18
19    std::vector<std::vector<int>> best_remaining(
20        rows, std::vector<int>(cols, -1));
21    std::queue<State> queue;
22    queue.push(State{0, 0, eliminations, 0});

```

```

23     best_remaining[0][0] = eliminations;
24
25     while (!queue.empty()) {
26         const State state = queue.front();
27         queue.pop();
28         if (state.row == rows - 1 && state.col == cols - 1) {
29             return state.steps;
30         }
31
32         for (const auto [dr, dc] : DIRECTIONS) {
33             const int next_row = state.row + dr;
34             const int next_col = state.col + dc;
35             const bool out_of_bounds =
36                 next_row < 0 || next_row >= rows ||
37                 next_col < 0 || next_col >= cols;
38             if (out_of_bounds) {
39                 continue;
40             }
41             const int next_remaining =
42                 state.remaining - grid[next_row][next_col];
43             if (next_remaining < 0 ||
44                 next_remaining <= best_remaining[next_row][next_col]) {
45                 continue;
46             }
47             best_remaining[next_row][next_col] = next_remaining;
48             queue.push(State{next_row, next_col, next_remaining, state.steps + 1});
49         }
50     }
51
52     return -1;
53 }

```

这个题非常适合训练“状态支配”思想。到达同一格子时，步数由 BFS 层保证是非降的；如果某个新状态剩余消除次数更少或相等，那么它在未来不会比历史状态更好，可以剪掉。若新状态剩余次数更多，即使步数不更短，也可能穿过更多障碍，不能剪。状态支配比简单 `visited` 更细，是很多高阶 BFS 的核心。

深入理解

从源码和工程角度看，图算法的性能经常被内存访问模式决定。邻接表遍历时，外层 `vector` 的每个桶指向独立存储；节点度数分布很不均匀时，热点节点的邻接边会形成大块连续扫描，而大量低度节点会带来分散访问。前向星或 CSR 风格结构把所有边压进连续数组，遍历时更接近顺序读。Linux 内核大量使用侵入式链表、红黑树、基数树或 XArray 这类结构，是因为内核对象本身已经存在，数据结构常常不拥有对象，只负责把对象链接进某种索引。刷题不需要照搬内核写法，但要理解背后的原则：结构选择服务于访问模式、生命周期和更新频率。

实验建议：选三道题做横向对比。第一道用 Dijkstra 和二分并查集分别解决最小体力路径，记录两种建模的状态、转移和复杂度。第二道用单向 BFS 与双向 BFS 解决打开转盘锁，统计访问状态数量。第三道把课程表从“能否完成”改成“给出一种学习顺序”，再改成“每学期最多学 k 门课时最少需要几学期”，观察拓扑排序、BFS 和状态压缩之间的边界。实验报告不要求长，但必须写清楚为什么一种模型能替代另一种模型，什么时候不能替代。

图论进阶选型

只问是否连通或连通块数量，优先考虑并查集或 BFS/DFS；问无权最短步数，优先 BFS；问非负权最短距离，优先 Dijkstra；问能否在某个阈值下连通，可以考虑二分答案加 BFS/并查集；问所有边反复松弛且可能有负边，才考虑 Bellman-Ford。先选模型，再写代码。

第 14 章

动态规划进阶、容器原理和源码阅读

刷题到中后期，很多错误不再来自不会写循环，而来自状态定义不稳定、容器成本估计错误、以及对标准库行为理解不够。动态规划进阶要把问题拆成“选择、容量、阶段、状态”；C++ 容器原理要知道数据如何存储、什么时候扩容、迭代器何时失效；源码阅读要能从接口行为反推实现约束。面试中如果能同时讲算法和容器成本，答案会明显更可靠。

0-1 背包是 DP 进阶的基础模型。给定若干物品，每个物品只能选一次，有重量和价值，背包容量有限，求最大价值。暴力方法是枚举每个物品选或不选，复杂度 $O(2^n)$ 。状态可以定义为 `dp[i][cap]`：只考虑前 `i` 个物品、容量为 `cap` 时的最大价值。第 `i` 个物品要么不选，要么在容量足够时选择。

Listing 14.1 0-1 背包：一维滚动数组

```
1 int knapsack_01(const std::vector<int>& weights,
2               const std::vector<int>& values,
3               int capacity)
4 {
5     std::vector<int> dp(capacity + 1, 0);
6
7     for (int i = 0; i < static_cast<int>(weights.size()); ++i) {
8         for (int cap = capacity; cap >= weights[i]; --cap) {
9             dp[cap] = std::max(dp[cap], dp[cap - weights[i]] + values[i]);
10        }
11    }
12
13    return dp[capacity];
14 }
```

一维滚动数组必须倒序遍历容量，因为每个物品只能使用一次。若正序遍历，`dp[cap - weights[i]]` 可能已经包含当前物品，等价于重复选择。这个遍历方向就是 0-1 背包和完全背包最重要的区别。面试中不要只背公式，要说清楚倒序是为了让当前物品的转移读取上一轮状态。

分割等和子集是 0-1 背包的判定版本。题目问数组能否分成两个和相等的子集。总和若为奇数，直接不可能；否则问题变成能否从数组中选一些数，使和为 `sum / 2`。每个数只能选一次，所以是 0-1 背包。状态 `dp[cap]` 表示是否能凑出容量 `cap`。

Listing 14.2 分割等和子集：0-1 背包可达性

```
1 bool can_partition(const std::vector<int>& nums)
2 {
3     const int total = std::accumulate(nums.begin(), nums.end(), 0);
4     if (total % 2 != 0) {
5         return false;
6     }
7
8     const int target = total / 2;
9     std::vector<bool> dp(target + 1, false);
10    dp[0] = true;
11
12    for (const int x : nums) {
13        for (int cap = target; cap >= x; --cap) {
```

```

14         dp[cap] = dp[cap] || dp[cap - x];
15     }
16 }
17
18 return dp[target];
19 }

```

这题的关键不是代码，而是把“分成两个相等子集”转成“选择若干元素凑出总和一半”。如果数组元素都是正数，背包容量有限，DP 很自然；如果存在负数，容量模型就不直接适用，需要偏移或换思路。复杂度是 $O(n \cdot \text{target})$ ，其中 **target** 与数值大小有关，所以这是伪多项式复杂度，不是严格输入长度多项式。

目标和可以转成背包计数。给每个数加正号或负号，使表达式等于 target。设正数集合和为 P，负数集合和为 N，总和 S。则 $P - N = \text{target}$ ， $P + N = S$ ，推出 $P = (S + \text{target}) / 2$ 。问题变成从数组中选若干数，使和为 P 的方案数。暴力枚举符号是 $O(2^n)$ ；DP 保存每个和的方案数。

Listing 14.3 目标和：转换为 0-1 背包计数

```

1 int find_target_sum_ways(const std::vector<int>& nums, int target)
2 {
3     const int total = std::accumulate(nums.begin(), nums.end(), 0);
4     if (std::abs(target) > total || (total + target) % 2 != 0) {
5         return 0;
6     }
7
8     const int positive_sum = (total + target) / 2;
9     std::vector<int> dp(positive_sum + 1, 0);
10    dp[0] = 1;
11
12    for (const int x : nums) {
13        for (int sum = positive_sum; sum >= x; --sum) {
14            dp[sum] += dp[sum - x];
15        }
16    }
17
18    return dp[positive_sum];
19 }

```

这题要特别注意 0。若数组里有 0，给它加正号或负号都不改变和，但方案数应翻倍。上述 DP 能自然处理：当 $x == 0$ 时， $\text{dp}[\text{sum}] += \text{dp}[\text{sum}]$ ，所有已有方案翻倍。很多手写特殊逻辑反而容易错。计数型 DP 要关注整型范围，力扣原题结果通常能放进 int，但工程中应考虑 **long long**。

零钱兑换 II 是完全背包计数。硬币可以无限使用，问凑成金额的组合数。注意是组合数，不是排列数。若外层遍历金额、内层遍历硬币，会把不同顺序当作不同方案；若外层遍历硬币、内层正序遍历金额，每种硬币按固定顺序加入，就只统计组合。

Listing 14.4 零钱兑换 II：完全背包组合计数

```

1 int change(int amount, const std::vector<int>& coins)
2 {
3     std::vector<int> dp(amount + 1, 0);
4     dp[0] = 1;
5
6     for (const int coin : coins) {
7         for (int value = coin; value <= amount; ++value) {
8             dp[value] += dp[value - coin];

```

```

9     }
10 }
11
12 return dp[amount];
13 }

```

为什么容量正序？因为当前硬币可以重复使用，`dp[value - coin]` 应该允许已经使用当前硬币。为什么硬币在外层？因为组合不关心顺序，我们规定按硬币种类逐步开放选择。这个题非常适合区分 0-1 背包、完全背包、组合计数和排列计数。只要循环顺序解释不清，背包题就还没有真正掌握。

股票题是状态机 DP。买卖股票的最佳时机 I 只允许一次交易，最简单做法是扫描时维护历史最低买入价和当前最大利润。暴力枚举买入日和卖出日是 $O(n^2)$ ；优化来自“卖出日固定时，最优买入日一定是它之前价格最低的一天”。

Listing 14.5 买卖股票 I：维护历史最低价格

```

1 int max_profit_one_transaction(const std::vector<int>& prices)
2 {
3     int min_price = std::numeric_limits<int>::max();
4     int answer = 0;
5
6     for (const int price : prices) {
7         min_price = std::min(min_price, price);
8         answer = std::max(answer, price - min_price);
9     }
10
11     return answer;
12 }

```

含冷冻期的股票题不能只维护最低价，因为今天能不能买取决于前一天是否卖出。状态机更清楚：`hold` 表示今天结束时持有股票的最大收益；`sold` 表示今天刚卖出；`rest` 表示今天结束时不持有且不在刚卖出的状态。转移时，买入只能从 `rest` 来，卖出只能从 `hold` 来，休息可以从昨天的 `rest` 或 `sold` 来。

Listing 14.6 含冷冻期股票：三状态 DP

```

1 int max_profit_with_cooldown(const std::vector<int>& prices)
2 {
3     if (prices.empty()) {
4         return 0;
5     }
6
7     int hold = -prices[0];
8     int sold = 0;
9     int rest = 0;
10
11     for (int i = 1; i < static_cast<int>(prices.size()); ++i) {
12         const int previous_hold = hold;
13         const int previous_sold = sold;
14         const int previous_rest = rest;
15
16         hold = std::max(previous_hold, previous_rest - prices[i]);
17         sold = previous_hold + prices[i];
18         rest = std::max(previous_rest, previous_sold);
19     }
20 }

```

```

19     }
20
21     return std::max(sold, rest);
22 }

```

这段代码的关键是先保存昨天状态，避免同一天内状态互相污染。返回值不能是 `hold`，因为结束时仍持有股票不代表已经实现收益。股票 DP 的复盘模板是：每天结束时有哪些状态，状态之间允许哪些动作，动作是否受冷冻期、手续费、交易次数限制。把限制放进状态机，公式就会自然出现。

动态规划最容易被学成公式库，但公式不是起点。起点永远是暴力搜索：有哪些选择，搜索树的节点表示什么，哪些节点会重复出现。以最长递增子序列为例，暴力做法是对每个元素选择“选或不选”，同时维护上一个被选元素，复杂度指数级。重复子问题来自“处理到某个位置、上一个选择约束相同”时，后续选择空间完全一样。第一种 DP 定义可以是 `dp[i]`：以 `nums[i]` 结尾的最长递增子序列长度。这样每个状态都问一个局部稳定的问题：最后一个元素固定为 `i`，前一个元素只能从 `0..i-1` 中找，并且值要更小。

Listing 14.7 最长递增子序列：二次 DP 和路径还原

```

1 std::vector<int> reconstruct_lis(const std::vector<int>& nums)
2 {
3     const int n = static_cast<int>(nums.size());
4     if (n == 0) {
5         return {};
6     }
7
8     std::vector<int> dp(n, 1);
9     std::vector<int> parent(n, -1);
10    int best_index = 0;
11
12    for (int i = 0; i < n; ++i) {
13        for (int j = 0; j < i; ++j) {
14            if (nums[j] >= nums[i] || dp[j] + 1 <= dp[i]) {
15                continue;
16            }
17            dp[i] = dp[j] + 1;
18            parent[i] = j;
19        }
20        if (dp[i] > dp[best_index]) {
21            best_index = i;
22        }
23    }
24
25    std::vector<int> sequence;
26    for (int index = best_index; index != -1; index = parent[index]) {
27        sequence.push_back(nums[index]);
28    }
29    std::reverse(sequence.begin(), sequence.end());
30    return sequence;
31 }

```

这段代码不是最优复杂度，但它最适合讲清楚状态。`parent[i]` 记录让 `dp[i]` 变大的前驱下标，所以最后可以从最优结尾倒着还原路径。正确性来自一个简单不变量：外层处理到 `i` 时，所有 `j < i` 的 `dp[j]` 已经是以前 `j` 结尾的最优答案；若 `nums[j] < nums[i]`，把 `i` 接到以前 `j` 结尾的最

优序列后面，一定得到一个合法递增序列。枚举所有可能前驱后取最大值，就不会漏掉以 i 结尾的最优解。复杂度 $O(n^2)$ ，空间 $O(n)$ 。

最长递增子序列还可以优化到 $O(n \log n)$ 。优化依据不是简单二分，而是维护一个更抽象的数组 **tails**：长度为 $\text{len} + 1$ 的递增子序列，结尾最小值是多少。结尾越小，未来接上新元素的机会越大。扫描每个数 x ，用二分找到第一个大于等于 x 的位置并替换它；如果找不到，就把 x 追加到末尾。**tails** 不一定是一条真实子序列，但它的长度等于 LIS 长度。这个区别必须讲清楚，否则读者会误以为 **tails** 本身就是答案路径。

Listing 14.8 最长递增子序列：tails 数组和二分优化

```
1 int length_of_lis(const std::vector<int>& nums)
2 {
3     std::vector<int> tails;
4     tails.reserve(nums.size());
5
6     for (const int x : nums) {
7         auto it = std::lower_bound(tails.begin(), tails.end(), x);
8         if (it == tails.end()) {
9             tails.push_back(x);
10        } else {
11            *it = x;
12        }
13    }
14
15    return static_cast<int>(tails.size());
16 }
```

为什么替换不会破坏答案？因为替换位置 **pos** 的含义是：存在长度为 $\text{pos} + 1$ 的递增子序列以 x 结尾，并且 x 不大于原来的结尾。更小的结尾不会让未来变差，只会让未来更容易扩展。若题目要求严格递增，使用 **lower_bound**；若要求非递减，使用 **upper_bound**。这是面试里常见追问：二分函数的选择取决于相等元素是否允许接在后面。

网格路径 DP 是二维状态的入门。不同路径间从左上到右下只能向右或向下走有多少条路径。暴力递归会从每个格子继续向右和向下，重复计算大量子矩形。状态 $\text{dp}[\text{row}][\text{col}]$ 表示到达当前格子的路径数，它只依赖上方和左方。由于每个状态只依赖上一行和当前行左侧，可以压缩成一维数组。这里的遍历顺序不是随便的：从左到右更新时， $\text{dp}[\text{col}]$ 仍表示上一行同列， $\text{dp}[\text{col} - 1]$ 已经表示当前行左侧。

Listing 14.9 不同路径：一维滚动数组

```
1 int unique_paths(int rows, int cols)
2 {
3     std::vector<int> dp(cols, 1);
4     for (int row = 1; row < rows; ++row) {
5         for (int col = 1; col < cols; ++col) {
6             dp[col] += dp[col - 1];
7         }
8     }
9     return dp[cols - 1];
10 }
```

最小路径和与不同路径结构相同，只是“方案数相加”变成“代价取最小”。状态 $\text{dp}[\text{col}]$ 表示处理到当前行时，到达该列的最小路径和。第一行只能从左边来，第一列只能从上边来，其余位置取上方和左方较小者。边界处理是这类题的主要错误来源。一个稳妥写法是先初始化第一行，再

逐行处理第一列和内部格子；不要在双循环里塞很多特殊判断，让主转移被边界噪声淹没。

Listing 14.10 最小路径和：边界初始化后主转移

```
1 int min_path_sum(const std::vector<std::vector<int>>& grid)
2 {
3     const int rows = static_cast<int>(grid.size());
4     const int cols = static_cast<int>(grid[0].size());
5     std::vector<int> dp(cols, 0);
6
7     dp[0] = grid[0][0];
8     for (int col = 1; col < cols; ++col) {
9         dp[col] = dp[col - 1] + grid[0][col];
10    }
11
12    for (int row = 1; row < rows; ++row) {
13        dp[0] += grid[row][0];
14        for (int col = 1; col < cols; ++col) {
15            dp[col] = std::min(dp[col], dp[col - 1]) + grid[row][col];
16        }
17    }
18
19    return dp[cols - 1];
20 }
```

编辑距离是字符串 DP 的核心题。给定两个单词，允许插入、删除、替换，求把一个单词变成另一个单词的最少操作数。暴力递归从两个字符串末尾比较：若字符相同，两个指针都左移；若不同，可以删除一个字符、插入一个字符、替换一个字符。暴力慢是因为同一对前缀长度会被多条路径反复访问。状态 $dp[i][j]$ 表示 `word1` 的前 i 个字符变成 `word2` 的前 j 个字符的最少操作数。空串边界很重要： $dp[i][0] = i$, $dp[0][j] = j$ 。

Listing 14.11 编辑距离：两行滚动数组

```
1 int min_distance(const std::string& word1, const std::string& word2)
2 {
3     const int n = static_cast<int>(word1.size());
4     const int m = static_cast<int>(word2.size());
5
6     std::vector<int> previous(m + 1, 0);
7     std::vector<int> current(m + 1, 0);
8     for (int j = 0; j <= m; ++j) {
9         previous[j] = j;
10    }
11
12    for (int i = 1; i <= n; ++i) {
13        current[0] = i;
14        for (int j = 1; j <= m; ++j) {
15            if (word1[i - 1] == word2[j - 1]) {
16                current[j] = previous[j - 1];
17                continue;
18            }
19            const int remove_cost = previous[j] + 1;
20            const int insert_cost = current[j - 1] + 1;
21            const int replace_cost = previous[j - 1] + 1;
```

```

22         current[j] = std::min({remove_cost, insert_cost, replace_cost});
23     }
24     std::swap(previous, current);
25 }
26
27 return previous[m];
28 }

```

三个操作的含义要能对上表格位置。删除 `word1[i-1]` 后，问题变成 `word1` 前 `i-1` 个字符到 `word2` 前 `j` 个字符，所以来自 `previous[j]`；插入 `word2[j-1]` 后，等价于先把 `word1` 前 `i` 个字符变成 `word2` 前 `j-1` 个字符，所以来自 `current[j-1]`；替换则来自 `previous[j-1]`。两行滚动数组的空间是 $O(m)$ ，时间是 $O(nm)$ 。如果题目要求还原操作序列，就不能只保留两行，需要记录决策或重新回溯。

最长公共子序列和编辑距离很接近。状态 `dp[i][j]` 表示两个前缀的 LCS 长度。若末尾字符相同，就从 `dp[i-1][j-1] + 1` 转移；否则取删掉任一末尾后的最大值。它和最长公共子串不同：子序列允许不连续，子串必须连续。这个概念差异决定了转移差异。最长公共子串在字符不同时必须归零，LCS 在字符不同时取上方或左方最大值。面试里一旦题目出现“可以删除若干字符但保持相对顺序”，就要想到子序列。

Listing 14.12 最长公共子序列：一维数组和左上角缓存

```

1 int longest_common_subsequence(const std::string& text1,
2                               const std::string& text2)
3 {
4     const int n = static_cast<int>(text1.size());
5     const int m = static_cast<int>(text2.size());
6     std::vector<int> dp(m + 1, 0);
7
8     for (int i = 1; i <= n; ++i) {
9         int diagonal = 0;
10        for (int j = 1; j <= m; ++j) {
11            const int old_up = dp[j];
12            if (text1[i - 1] == text2[j - 1]) {
13                dp[j] = diagonal + 1;
14            } else {
15                dp[j] = std::max(dp[j], dp[j - 1]);
16            }
17            diagonal = old_up;
18        }
19    }
20
21    return dp[m];
22 }

```

一维 LCS 的难点在于保存左上角。更新前的 `dp[j]` 是上一行同列，`dp[j-1]` 是当前行左侧，`diagonal` 是上一行左上角。每次循环结束前把旧的 `dp[j]` 放进 `diagonal`，供下一列使用。若你觉得这个写法难以保证正确，可以先写二维表；面试时清楚解释二维转移比写一个容易错的一维优化更重要。空间优化应该建立在状态依赖已经完全理解之后。

单词拆分是“前缀可达性”DP。题目问字符串能否被字典中的单词拼接出来。暴力方法枚举第一段单词，然后递归处理剩余后缀；同一个起点会被反复递归。状态 `dp[i]` 表示前 `i` 个字符能否被拆分。转移枚举最后一个单词的起点 `j`：若 `dp[j]` 为真，且 `s[j..i)` 在字典中，则 `dp[i]` 为真。这里要注意 `substr` 的成本，它会创建新字符串。数据量不大时可接受；数据量大时，应限制最大单词长度，或用字典树、字符串视图、滚动哈希减少复制。

Listing 14.13 单词拆分：限制最大单词长度降低无效枚举

```

1 bool word_break(const std::string& s,
2                 const std::vector<std::string>& word_dict)
3 {
4     std::unordered_set<std::string> words(word_dict.begin(), word_dict.end());
5     int max_word_length = 0;
6     for (const std::string& word : word_dict) {
7         max_word_length = std::max(max_word_length,
8                                     static_cast<int>(word.size()));
9     }
10
11     const int n = static_cast<int>(s.size());
12     std::vector<char> dp(n + 1, false);
13     dp[0] = true;
14
15     for (int end = 1; end <= n; ++end) {
16         const int begin_limit = std::max(0, end - max_word_length);
17         for (int begin = end - 1; begin >= begin_limit; --begin) {
18             if (!dp[begin]) {
19                 continue;
20             }
21             if (words.contains(s.substr(begin, end - begin))) {
22                 dp[end] = true;
23                 break;
24             }
25         }
26     }
27
28     return dp[n];
29 }

```

这个优化不是改变问题模型，而是减少不可能成为单词的长度。若字典中最长单词长度为 L ，枚举起点时只需要看最近 L 个位置。复杂度粗略为 $O(nL)$ 次查询，但每次 `substr` 仍有拷贝成本。面试表达可以分层：先给清楚的 DP，再说明字符串切片成本，最后提出可选优化。不要一上来写复杂字典树，把主线埋掉。

区间 DP 的状态通常是“处理一个连续区间的最优值”。它与普通线性 DP 的区别是：最后一步可能把区间切成两半，或者选择区间内某个点作为最后处理对象。戳气球是代表题。直接思考“先戳哪个气球”很混乱，因为戳掉一个气球会改变邻居；换成“最后戳哪个气球”就稳定了。设 `dp[left][right]` 表示开区间 $(left, right)$ 内的气球全部戳掉能获得的最大硬币数。若最后戳 `mid`，此时它左右邻居一定是 `left` 和 `right`，收益为 `nums[left] * nums[mid] * nums[right]`，左右两边互不影响。

Listing 14.14 戳气球：选择最后一个被戳的气球

```

1 int max_coins(const std::vector<int>& nums)
2 {
3     std::vector<int> values;
4     values.reserve(nums.size() + 2);
5     values.push_back(1);
6     values.insert(values.end(), nums.begin(), nums.end());
7     values.push_back(1);
8
9     const int n = static_cast<int>(values.size());

```



```

10     std::vector<std::vector<int>> dp(n, std::vector<int>(n, 0));
11
12     for (int length = 2; length < n; ++length) {
13         for (int left = 0; left + length < n; ++left) {
14             const int right = left + length;
15             for (int mid = left + 1; mid < right; ++mid) {
16                 const int score = dp[left][mid] + dp[mid][right] +
17                     values[left] * values[mid] * values[right];
18                 dp[left][right] = std::max(dp[left][right], score);
19             }
20         }
21     }
22
23     return dp[0][n - 1];
24 }

```

区间 DP 的遍历顺序按区间长度从小到大，因为大区间依赖小区间。这里的边界 1 是虚拟气球，不会被戳，只是让左右边界收益统一。正确性要抓住“最后一步”：如果 `mid` 是最后被戳的气球，那么 `left..mid` 和 `mid..right` 两个子问题已经在它之前完成，且互相独立；枚举所有可能的最后气球，取最大值。复杂度是 $O(n^3)$ ，空间 $O(n^2)$ 。若面试官追问能否优化，要先说明一般区间 DP 不一定有四边形不等式或决策单调性，不能乱套优化。

状态压缩 DP 把集合放进整数二进制位。适用条件通常是元素数量不大，例如 $n \leq 20$ 。每一位表示某个任务、节点或物品是否已经被选择。经典问题是旅行商或访问所有节点的最短路径。若图无权，访问所有节点的最短路径可以用 BFS，把状态定义为 `(node, mask)`：当前在 `node`，已经访问的节点集合是 `mask`。这其实是状态压缩和图 BFS 的结合，而不是传统表格 DP。

Listing 14.15 访问所有节点的最短路径：节点加集合状态 BFS

```

1 int shortest_path_length(const std::vector<std::vector<int>>& graph)
2 {
3     const int n = static_cast<int>(graph.size());
4     const int full_mask = (1 << n) - 1;
5
6     std::queue<std::pair<int, int>> queue;
7     std::vector<std::vector<int>> distance(n, std::vector<int>(1 << n, -1));
8
9     for (int node = 0; node < n; ++node) {
10         const int mask = 1 << node;
11         distance[node][mask] = 0;
12         queue.emplace(node, mask);
13     }
14
15     while (!queue.empty()) {
16         const auto [node, mask] = queue.front();
17         queue.pop();
18         if (mask == full_mask) {
19             return distance[node][mask];
20         }
21
22         for (const int next : graph[node]) {
23             const int next_mask = mask | (1 << next);
24             if (distance[next][next_mask] != -1) {
25                 continue;

```

```

26         }
27         distance[next][next_mask] = distance[node][mask] + 1;
28         queue.emplace(next, next_mask);
29     }
30 }
31
32 return 0;
33 }

```

这题如果只把节点作为访问标记，会出错。到达同一个节点时，已访问集合不同，未来能力不同。**(node, mask)** 才是完整状态。多源初始化也很关键：路径可以从任意节点开始，所以所有单节点状态都在第 0 层。复杂度是 $O(n2^n + m2^n)$ ，只适合小 n 。状态压缩题要先估算 2^n 是否可承受，再决定是否使用；看到集合并不意味着一定能压缩，如果 n 是一百，二进制集合就不现实。

另一类状态压缩是子集枚举。比如把数组分成若干组、每组和相等。可以用 **dp[mask]** 表示选择集合 **mask** 后，当前桶中已经放入的对目标桶容量取模是多少；若 **dp[mask] == -1** 表示不可达。每次尝试加入一个未选元素，只要当前桶不超容量，就更新新集合。这个写法比回溯更像 DP，因为每个集合状态只计算一次。

Listing 14.16 划分为 K 个等和子集：集合状态 DP

```

1 bool can_partition_k_subsets(std::vector<int> nums, int k)
2 {
3     const int total = std::accumulate(nums.begin(), nums.end(), 0);
4     if (total % k != 0) {
5         return false;
6     }
7     const int target = total / k;
8     std::sort(nums.begin(), nums.end(), std::greater<int>());
9     if (!nums.empty() && nums.front() > target) {
10         return false;
11     }
12
13     const int n = static_cast<int>(nums.size());
14     const int state_count = 1 << n;
15     std::vector<int> dp(state_count, -1);
16     dp[0] = 0;
17
18     for (int mask = 0; mask < state_count; ++mask) {
19         if (dp[mask] == -1) {
20             continue;
21         }
22         for (int i = 0; i < n; ++i) {
23             if ((mask & (1 << i)) != 0) {
24                 continue;
25             }
26             const int next_sum = dp[mask] + nums[i];
27             if (next_sum > target) {
28                 continue;
29             }
30             const int next_mask = mask | (1 << i);
31             dp[next_mask] = next_sum % target;
32         }
33     }

```

```

34
35     return dp[state_count - 1] == 0;
36 }

```

这个 DP 的状态值不是总和，而是当前桶的剩余进度。每当一个桶刚好装满，对 `target` 取模后的进度回到 0，表示开始装下一个桶。排序不是正确性必需，但大的数先处理能更早剪掉不可能状态。复杂度是 $O(n^2)$ 。与回溯相比，状态 DP 的优势是避免不同选择顺序到达同一集合时重复搜索；缺点是内存随 2^n 增长。

单调队列优化 DP 是从“转移枚举范围”中挤出复杂度。以带限制的最大子序列和为例，题目要求选择一个非空子序列，相邻被选元素下标差最多为 k ，求最大和。朴素 DP 定义 $dp[i]$ 为必须选择 `nums[i]` 且以它结尾的最大和，则

$$dp[i] = nums[i] + \max(0, dp[j]) \quad i - k \leq j < i.$$

如果每个 i 都扫描前 k 个位置，复杂度 $O(nk)$ 。优化来自窗口最大值：我们只需要最近 k 个 $dp[j]$ 的最大值，用单调队列维护即可。

Listing 14.17 受限子序列和：单调队列优化转移最大值

```

1 int constrained_subset_sum(const std::vector<int>& nums, int k)
2 {
3     std::deque<int> deque;
4     std::vector<int> dp(nums.size(), 0);
5     int answer = std::numeric_limits<int>::min();
6
7     for (int i = 0; i < static_cast<int>(nums.size()); ++i) {
8         while (!deque.empty() && deque.front() < i - k) {
9             deque.pop_front();
10        }
11
12        dp[i] = nums[i];
13        if (!deque.empty()) {
14            dp[i] = std::max(dp[i], nums[i] + dp[deque.front()]);
15        }
16        answer = std::max(answer, dp[i]);
17
18        while (!deque.empty() && dp[deque.back()] <= dp[i]) {
19            deque.pop_back();
20        }
21        if (dp[i] > 0) {
22            deque.push_back(i);
23        }
24    }
25
26    return answer;
27 }

```

队列中保存下标，而不是保存值，因为要判断是否过期。队首永远是当前窗口内 dp 最大的位置。只把正的 $dp[i]$ 放入队列，是因为负贡献不如从当前元素重新开始。正确性来自两个不变量：队列下标从前到后递增，保证能判断窗口；对应 dp 值从前到后递减，保证队首最大。每个下标最多进队一次、出队一次，所以总复杂度 $O(n)$ 。

动态规划和最短路之间有很深的联系。许多 DP 可以看成在有向无环图上找路径：状态是节点，转移是边，遍历顺序是拓扑序。若状态图有环但边权非负，可能变成 Dijkstra；若边权全为 1，可能变成 BFS；若存在负边但没有负环，可能变成 Bellman-Ford。这样理解后，题型边界会更清楚：不

是所有“求最优”都叫 DP，也不是所有“状态转移”都要填表。你要先问状态图是否有自然阶段顺序，是否允许回头，转移代价是否单调。

常见误区

DP 常见误区有四个。第一，只背公式，不说明状态含义，导致换一个题面就失效。第二，滚动数组时忘记转移读的是上一阶段还是当前阶段。第三，把组合数和排列数混在一起，循环顺序写反。第四，看到最值就写 DP，却没有检查状态数量是否可承受。每次写完 DP，都要用一句话解释 **dp** 的含义，用一句话解释遍历顺序，用一句话解释为什么不会漏解或重复计数。

C++ 容器原理是刷题代码质量的底座。**std::vector** 是连续数组，支持随机访问，扩容时会搬迁元素，导致指针、引用和迭代器失效。**std::deque** 分段连续，支持两端高效插入删除，但随机访问局部性通常不如 **vector**。**std::list** 节点稳定，但缓存局部性差，刷题中很少是首选。**std::unordered_map** 平均常数查询，但 rehash 会让迭代器失效，且不维护顺序。**std::map** 有序但每次操作是 $O(\log n)$ 。

阅读标准库源码时，不需要一开始钻进所有模板细节。先看三层：公开接口、核心数据成员、关键操作路径。以 **vector** 为例，大多数实现内部会维护三个指针或等价状态：开始位置、当前结束位置、容量结束位置。**size()** 是当前结束减开始，**capacity()** 是容量结束减开始，**push_back** 在容量足够时原地构造，在容量不足时申请更大内存并移动旧元素。理解这三根指针，就能解释很多迭代器失效规则。

Listing 14.18 vector 使用习惯：reserve、resize 和引用失效

```
1 void collect_values(const std::vector<int>& input)
2 {
3     std::vector<int> values;
4     values.reserve(input.size());
5
6     for (const int x : input) {
7         values.push_back(x * 2);
8     }
9
10    const int* data = values.data();
11    values.push_back(42);
12    // 如果 push_back 触发扩容，data 已经失效，不能继续解引用。
13 }
```

reserve 和 **resize** 的区别必须非常熟。**reserve** 只保证容量，不改变大小；**resize** 改变大小并构造元素。DP 数组需要可下标访问，应使用 **vector<int> dp(n + 1)** 或 **resize**；收集答案时知道最多数量的，可以先 **reserve** 再 **push_back**。把二者混用，是 C++ 刷题里非常常见的越界来源。

unordered_map 的源码阅读重点是桶数组、哈希函数、负载因子和 rehash。插入元素时，容器通过哈希值定位桶，再在桶内查找等价 key。平均常数复杂度依赖哈希分布；当元素数量相对桶数过多，会触发 rehash，重新分配桶并把元素放到新桶中。刷题时使用 **reserve** 可以减少 rehash 次数，尤其是两数之和、前缀和计数、频次统计这类一次性插入较多 key 的题。

Listing 14.19 unordered_map 查询：contains 与 find 的取舍

```
1 int count_frequency(const std::vector<int>& nums, int target)
2 {
3     std::unordered_map<int, int> frequency;
4     frequency.reserve(nums.size());
5
6     for (const int x : nums) {
```



```

7     ++frequency[x];
8 }
9
10    const auto found = frequency.find(target);
11    if (found == frequency.end()) {
12        return 0;
13    }
14    return found->second;
15 }

```

如果只判断存在，C++20 的 `contains` 很清楚；如果判断后还要取值，用 `find` 避免查两次。不要用 `operator[]` 做纯查询，因为 key 不存在时会插入默认值，改变容器状态。这个细节在前缀和计数里尤其重要：有些查询应该只读历史状态，不能悄悄插入一个 0。

源码阅读也能帮助理解算法复杂度的真实成本。字符串 `substr` 会创建新字符串，放在双层循环里可能把 $O(n^2)$ 变成接近 $O(n^3)$ 的实际成本；`vector<vector<int>>` 的每一行是独立分配，矩阵遍历仍然可用，但内存不一定完全连续；`priority_queue` 没有删除任意元素和 `decrease-key`，所以 Dijkstra 常用“压入新状态，弹出时跳过过期状态”的写法。理解容器能力边界，才能解释为什么代码这样写。

`std::deque` 的名字容易误导初学者。它不是双向链表，而是分段连续数组。典型实现会维护一个“map”，其中每个槽指向一段固定大小的缓冲区；首尾插入删除大多只移动段内指针，必要时分配新段。它支持随机访问，但随机访问需要先定位段再定位段内偏移，局部性通常不如 `vector`。这解释了为什么滑动窗口最大值适合用 `deque` 存下标，而普通顺序 DP 数组不应该为了“可能两端操作”随手换成 `deque`。

Listing 14.20 deque 在单调队列中的正确用法

```

1 std::vector<int> max_sliding_window(const std::vector<int>& nums, int k)
2 {
3     std::deque<int> deque;
4     std::vector<int> answer;
5     answer.reserve(nums.size());
6
7     for (int i = 0; i < static_cast<int>(nums.size()); ++i) {
8         while (!deque.empty() && deque.front() <= i - k) {
9             deque.pop_front();
10        }
11        while (!deque.empty() && nums[deque.back()] <= nums[i]) {
12            deque.pop_back();
13        }
14        deque.push_back(i);
15        if (i + 1 >= k) {
16            answer.push_back(nums[deque.front()]);
17        }
18    }
19
20    return answer;
21 }

```

这里 `deque` 保存下标而不是值，原因和单调队列 DP 一样：需要判断过期位置。下标还能处理重复值，若只存值，窗口移出时很难知道队首值是否对应离开的元素。源码层面看，`pop_front` 和 `push_back` 都是 `deque` 的强项；若用 `vector` 在头部删除，就会移动大量元素；若用 `list`，虽然删除快，但无法高效访问队尾并且缓存局部性差。容器选择和算法不变量应该匹配。

`std::string` 在现代实现中通常具有小字符串优化。短字符串可以直接存放在对象内部，不必

单独堆分配；长字符串才指向堆内存。这个事实能解释为什么偶尔创建短字符串不一定灾难，但不能因此在热循环里无限 `substr`。标准并不要求具体的小字符串容量，不同标准库实现也不同，因此工程代码不能依赖“小于多少字符一定不分配”。源码阅读要区分“标准保证的语义”和“某个实现的优化”。

Listing 14.21 字符串扫描：用下标边界避免大量 `substr`

```
1 bool has_word_at(const std::string& text,
2                 int begin,
3                 const std::string& word)
4 {
5     if (begin + static_cast<int>(word.size()) >
6         static_cast<int>(text.size())) {
7         return false;
8     }
9     for (int i = 0; i < static_cast<int>(word.size()); ++i) {
10        if (text[begin + i] != word[i]) {
11            return false;
12        }
13    }
14    return true;
15 }
```

这段代码展示的是思想：当你只需要比较一段文本时，不一定要构造新字符串。C++20 可以使用 `std::string_view` 表示非拥有切片，但要注意生命周期；视图不能比原字符串活得更久。刷题中 `substr` 更短、更不易错，数据量允许时可以使用；工程中如果它处在双层或三层循环内，就应该估算分配和拷贝成本。

`std::priority_queue` 是容器适配器，默认底层容器通常是 `vector`，并用堆算法维护最大堆。它只暴露 `top`、`push`、`pop`，不支持遍历、不支持删除任意元素、不支持 `decrease-key`。这些限制不是缺陷，而是接口选择：标准库给的是简单、稳定、低成本的堆抽象。Dijkstra 中距离变小后重新入堆，是顺应这个接口；双堆中位数中延迟删除，也是因为堆不能直接删中间元素。

Listing 14.22 双堆延迟删除：滑动窗口中位数的核心结构

```
1 class DualHeap {
2 public:
3     explicit DualHeap(int window_size) : window_size_(window_size) {}
4
5     void insert(int value)
6     {
7         if (this->small_.empty() || value <= this->small_.top()) {
8             this->small_.push(value);
9             ++this->small_size_;
10        } else {
11            this->large_.push(value);
12            ++this->large_size_;
13        }
14        this->rebalance();
15    }
16
17     void erase(int value)
18     {
19         ++this->delayed_[value];
20         if (value <= this->small_.top()) {
```

```

21         --this->small_size_;
22         if (value == this->small_.top()) {
23             this->prune_small();
24         }
25     } else {
26         --this->large_size_;
27         if (!this->large_.empty() && value == this->large_.top()) {
28             this->prune_large();
29         }
30     }
31     this->rebalance();
32 }
33
34 double median() const
35 {
36     if (this->window_size_ % 2 == 1) {
37         return this->small_.top();
38     }
39     return (static_cast<double>(this->small_.top()) +
40             static_cast<double>(this->large_.top())) / 2.0;
41 }
42
43 private:
44 void rebalance()
45 {
46     if (this->small_size_ > this->large_size_ + 1) {
47         this->large_.push(this->small_.top());
48         this->small_.pop();
49         --this->small_size_;
50         ++this->large_size_;
51         this->prune_small();
52     } else if (this->small_size_ < this->large_size_) {
53         this->small_.push(this->large_.top());
54         this->large_.pop();
55         ++this->small_size_;
56         --this->large_size_;
57         this->prune_large();
58     }
59 }
60
61 void prune_small()
62 {
63     while (!this->small_.empty()) {
64         const int value = this->small_.top();
65         auto found = this->delayed_.find(value);
66         if (found == this->delayed_.end()) {
67             return;
68         }
69         if (--found->second == 0) {
70             this->delayed_.erase(found);
71         }
72         this->small_.pop();
73     }

```

```

74     }
75
76     void prune_large()
77     {
78         while (!this->large_.empty()) {
79             const int value = this->large_.top();
80             auto found = this->delayed_.find(value);
81             if (found == this->delayed_.end()) {
82                 return;
83             }
84             if (--found->second == 0) {
85                 this->delayed_.erase(found);
86             }
87             this->large_.pop();
88         }
89     }
90
91     int window_size_;
92     std::priority_queue<int> small_;
93     std::priority_queue<int, std::vector<int>, std::greater<int>> large_;
94     std::unordered_map<int, int> delayed_;
95     int small_size_ = 0;
96     int large_size_ = 0;
97 };

```

这段结构比普通数据流中位数复杂，因为窗口滑动时需要删除离开的元素。既然堆不能删除任意位置，就把待删除值记在 `delayed_` 中；只有当它出现在堆顶时才真正弹出。`small_size_` 和 `large_size_` 记录的是有效元素数量，不是堆内部实际元素数量。成员访问使用 `this->` 是为了明确哪些变量属于对象状态。复杂度均摊 $O(\log k)$ 。如果用 `multiset`，删除任意元素更直接，但常数和迭代器维护方式不同；两种方案都要能解释取舍。

`std::map` 和 `std::set` 通常由红黑树实现，保证有序遍历和 $O(\log n)$ 插入、删除、查找。红黑树是一种近似平衡二叉搜索树，插入删除后通过旋转和染色维持高度对数级。面试刷题里，使用 `map` 的信号通常是需有序键、前驱后继、区间边界或按顺序输出。若只需要存在性或计数，`unordered_map` 平均更快；若哈希可能被攻击、需要稳定顺序或需要 `lower_bound`，就选有序树。

Listing 14.23 有序映射：用 `upper_bound` 找右侧区间

```

1 int count_covered_points(const std::map<int, int>& intervals, int point)
2 {
3     const auto right = intervals.upper_bound(point);
4     if (right == intervals.begin()) {
5         return 0;
6     }
7     const auto candidate = std::prev(right);
8     return candidate->second >= point ? 1 : 0;
9 }

```

`upper_bound(point)` 返回第一个起点大于 `point` 的区间，前一个区间才可能覆盖 `point`。这类写法是有序容器的典型优势。注意 `std::prev(right)` 之前必须判断 `right != begin()`，否则会越界。源码阅读中看到树迭代器时，要记住它不是连续内存指针，自增自减会沿树结构寻找后继前驱，复杂度通常是均摊常数或对数级细节由实现保证，但缓存局部性不如数组。

`unordered_map` 的平均常数复杂度不是无条件的。它依赖哈希函数把键分散到桶里，也依赖

负载因子保持在合理范围。若大量键落入同一桶，查找会退化。C++ 标准没有要求具体冲突处理方式必须是链表还是开放寻址，常见实现多使用桶加节点。节点式实现的好处是 rehash 时可以重挂节点，坏处是每个元素单独分配，局部性不如连续表。刷题时使用 `reserve`，不只是为了速度，也是为了减少 rehash 导致的迭代器失效风险。

Listing 14.24 前缀和计数：查询和更新顺序不能反

```
1 int subarray_sum(const std::vector<int>& nums, int target)
2 {
3     std::unordered_map<int, int> count;
4     count.reserve(nums.size() * 2);
5     count[0] = 1;
6
7     int prefix = 0;
8     int answer = 0;
9     for (const int x : nums) {
10         prefix += x;
11         const auto found = count.find(prefix - target);
12         if (found != count.end()) {
13             answer += found->second;
14         }
15         ++count[prefix];
16     }
17     return answer;
18 }
```

这里必须先查询历史前缀，再加入当前前缀。若先更新，`target == 0` 时会把空子数组错误计入。这个例子说明容器 API 和算法不变量是绑在一起的：`count` 的语义是“当前下标之前出现过的 前缀和次数”，不是所有前缀和次数。`find` 用于查询，`operator[]` 用于确认要插入或累加的位置；二者角色不同。

源码阅读可以按固定路线进行。第一步读标准语义：这个容器承诺什么复杂度，哪些操作会让迭代器失效。第二步看实现的核心成员：`vector` 的三个指针，`string` 的大小容量和小字符串缓冲，`deque` 的段表，`unordered_map` 的桶数组和节点，`map` 的树根和节点颜色。第三步跟一条热路径：`push_back` 如何判断容量，`rehash` 如何搬迁，`erase` 如何维护链接，`lower_bound` 如何沿树下降。第四步回到题目，解释为什么某个 API 成本可接受或不可接受。

深入理解

阅读 `libstdc++` 或 `libc++` 源码时，不要被模板细节吓住。可以先从头文件里的 `public` 函数名定位到内部辅助函数，再沿着分配、构造、移动、销毁四类操作看。比如 `vector::push_back` 最核心的问题只有两个：容量够不够，元素如何构造；容量不够时，才进入重新分配和移动旧元素的路径。再比如 `unordered_map::operator[]` 的关键语义是“没有 key 就插入默认值”，源码里会表现为查找失败后构造节点。把这些行为和题目中的不变量联系起来，源码阅读才会变成能力，而不是机械翻文件。

实验建议：第一，自己写一个小程序，创建 `vector<int>`，连续 `push_back` 并打印 `size`、`capacity` 和 `data()` 地址，观察扩容时机和地址变化。第二，写前缀和计数题，把“先查询后更新”故意改反，用 `target = 0` 的用例验证错误。第三，比较 `substr` 和下标比较在长字符串 DP 中的运行时间。第四，阅读本机标准库中 `vector`、`deque`、`unordered_map` 的头文件，只记录核心数据成员和迭代器失效规则，不要求理解每个模板分支。

DP 和容器源码阅读复盘

DP 先问状态是否稳定，转移是否只依赖已计算状态，遍历顺序是否正确；容器先问数据是否连续、插入删除位置在哪里、查询是否需要顺序、迭代器何时失效。源码阅读不追求背实现细节，而是用实现模型解释复杂度、失效规则和 API 选择。

代表性 C++20 代码骨架

下面这些代码骨架不是为了替代题解，而是给读者提供可复用的实现基准。每段代码都尽量把状态命名清楚，把边界放在显式位置，把容器成本控制题目需要的范围内。真正刷题时不要机械复制，而要先说清楚题目的不变量，再决定是否使用这些骨架。

Listing 14.25 并查集：迭代 find、路径压缩和按大小合并

```

1 class DisjointSetUnion {
2 public:
3     explicit DisjointSetUnion(int n)
4         : parent_(n), size_(n, 1)
5     {
6         std::iota(this->parent_.begin(), this->parent_.end(), 0);
7     }
8
9     int find(int x)
10    {
11        int root = x;
12        while (this->parent_[root] != root) {
13            root = this->parent_[root];
14        }
15        while (this->parent_[x] != x) {
16            const int next = this->parent_[x];
17            this->parent_[x] = root;
18            x = next;
19        }
20        return root;
21    }
22
23    bool unite(int lhs, int rhs)
24    {
25        int root_lhs = this->find(lhs);
26        int root_rhs = this->find(rhs);
27        if (root_lhs == root_rhs) {
28            return false;
29        }
30        if (this->size_[root_lhs] < this->size_[root_rhs]) {
31            std::swap(root_lhs, root_rhs);
32        }
33        this->parent_[root_rhs] = root_lhs;
34        this->size_[root_lhs] += this->size_[root_rhs];
35        return true;
36    }
37
38 private:
39     std::vector<int> parent_;
40     std::vector<int> size_;
41 };

```

Listing 14.26 Dijkstra: 跳过 priority queue 中的过期状态

```

1 std::vector<long long> shortest_paths(
2     int n,
3     const std::vector<std::vector<std::pair<int, int>>>& graph,
4     int source)
5 {
6     constexpr long long INF = 4'000'000'000'000'000'000LL;
7     using State = std::pair<long long, int>;
8
9     std::vector<long long> distance(n, INF);
10    std::priority_queue<State, std::vector<State>, std::greater<State>> heap;
11    distance[source] = 0;
12    heap.emplace(0, source);
13
14    while (!heap.empty()) {
15        const auto [current_distance, node] = heap.top();
16        heap.pop();
17        if (current_distance != distance[node]) {
18            continue;
19        }
20
21        for (const auto& [next, weight] : graph[node]) {
22            const long long candidate = current_distance + weight;
23            if (candidate >= distance[next]) {
24                continue;
25            }
26            distance[next] = candidate;
27            heap.emplace(candidate, next);
28        }
29    }
30
31    return distance;
32 }

```

Listing 14.27 滑动窗口最大值: 单调队列保存下标

```

1 std::vector<int> max_sliding_window(const std::vector<int>& nums, int k)
2 {
3     std::deque<int> candidates;
4     std::vector<int> answer;
5     answer.reserve(nums.size());
6
7     for (int right = 0; right < static_cast<int>(nums.size()); ++right) {
8         while (!candidates.empty() && nums[candidates.back()] <= nums[right]) {
9             candidates.pop_back();
10        }
11        candidates.push_back(right);
12
13        const int expired = right - k;
14        if (!candidates.empty() && candidates.front() <= expired) {

```

```

15         candidates.pop_front();
16     }
17     if (right + 1 >= k) {
18         answer.push_back(nums[candidates.front()]);
19     }
20 }
21
22 return answer;
23 }

```

Listing 14.28 前缀和计数：历史状态先查询后更新

```

1 int subarray_sum_equals_k(const std::vector<int>& nums, int target)
2 {
3     std::unordered_map<int, int> prefix_count;
4     prefix_count.reserve(nums.size() * 2 + 1);
5     prefix_count[0] = 1;
6
7     int prefix = 0;
8     int answer = 0;
9     for (const int x : nums) {
10         prefix += x;
11         const auto found = prefix_count.find(prefix - target);
12         if (found != prefix_count.end()) {
13             answer += found->second;
14         }
15         ++prefix_count[prefix];
16     }
17     return answer;
18 }

```

Listing 14.29 迭代二叉树中序遍历：验证 BST 的基础骨架

```

1 bool is_valid_bst(TreeNode* root)
2 {
3     std::vector<TreeNode*> stack;
4     TreeNode* current = root;
5     std::optional<long long> previous;
6
7     while (current != nullptr || !stack.empty()) {
8         while (current != nullptr) {
9             stack.push_back(current);
10            current = current->left;
11        }
12
13        current = stack.back();
14        stack.pop_back();
15        if (previous.has_value() && current->value <= previous.value()) {
16            return false;
17        }
18        previous = current->value;
19        current = current->right;
20    }
21    return true;
22 }

```



```

20     }
21
22     return true;
23 }

```

实验和 Linux 源码阅读路线

算法书需要实验，是因为很多概念只有跑起来才会稳定。第一组实验是复杂度实验：分别写平方枚举、前缀和、哈希前缀计数，用不同 n 观察增长曲线。第二组实验是容器实验：比较 `vector` 顺序扫描、`list` 顺序扫描、`unordered_map` 查询和 `map` 查询，在数据量变大后观察缓存局部性和常数差异。第三组实验是状态实验：把 BFS 的访问标记从入队改到出队，观察重复入队数量如何膨胀。

Linux 源码阅读可以围绕数据结构而不是模块盲逛。先看链表宏如何通过成员地址反推出外层结构体，再看红黑树如何把比较逻辑留给调用者，再看 `XArray` 或基数树如何服务于页缓存和 ID 分配。阅读目标不是背字段名，因为字段会随版本变化；目标是理解为什么内核倾向侵入式结构、为什么要避免频繁小对象分配、为什么并发读写需要锁或 RCU 这类同步机制。

把刷题题型和内核结构对应起来会很有帮助。LRU 缓存对应哈希表加双向链表；定时器或调度候选对应按时间或虚拟运行时间排序的有序结构；文件描述符和页缓存索引对应整数到对象的映射；网络包队列对应队列和优先队列；内存伙伴系统对应分层空闲链表。对应关系不是说内核用的就是力扣模板，而是说明数据结构选择永远由访问模式决定。

源码实验要小步进行。可以先写一个最小侵入式链表：业务对象中放 `prev` 和 `next`，通过容器 `_of` 思想从链表节点拿回业务对象。再写一个普通 `std::list` 版本，对比所有权和迭代器稳定性。然后写一个 LRU，分别用 `std::list` 加 `unordered_map` 和手写双向链表实现。这个实验能把链表、哈希、对象生命周期和异常安全放在同一张图里。

性能实验要避免伪结论。先固定编译优化级别，预热数据，避免把输出打印放进计时代码；再多次运行取中位数；最后解释结果。`vector` 往往因为连续内存比 `list` 更快，即使两者同为线性扫描。`unordered_map` 平均常数很快，但如果 `key` 构造很重，整体成本仍可能高。算法复杂度给上界，工程测量暴露常数、缓存和分配成本。

每章实验都应产出一页报告：问题是什么，暴力基线是什么，优化版本改变了哪一步，输入规模如何设置，结果是否符合复杂度预期，出现偏差的原因是什么。报告不追求华丽，追求可复现。以后复习时，能用自己的实验解释一个算法，比只背题解更可靠。

专题复盘要求

读完本节后，不要只记结论。请把每个结论还原成一个可验证的问题：它解决了哪一步重复工作，依赖哪个题目条件，使用哪个容器或状态，边界条件如何破坏错误写法。

C++ 面试代码质量和容器工程细节

算法面试中的 C++ 代码不只是能 AC，还要让状态、所有权和复杂度可读。函数参数只读大对象应使用 `const` 引用，返回结果时依赖移动优化即可；循环中遍历复杂对象使用 `const auto&`，不要无意复制字符串、向量或节点记录。局部常量要有名字，尤其是方向数组、无穷大、字符状态、取模常数和哨兵值。

整数类型选择要主动。数组下标和 `size` 返回 `size_t`，但很多题需要与 `-1` 哨兵或差值比较，直接混用无符号可能产生隐蔽 bug。常见做法是在函数开始把 `size` 转成 `int`，并保证输入规模在 `int` 范围内；涉及总和、乘积、面积、距离时使用 `long long`。二分平方、堆距离、图最短路都要优先防溢出。

容器 API 语义要讲清。`unordered_map` 的 `operator[]` 在 `key` 不存在时会插入默认值，适合计数累加，不适合只检查存在。`find` 返回迭代器，适合查询后读取值。`contains` 只判断存在，如果随后还要取值会导致二次查找。`vector` 的 `reserve` 只改容量，`resize` 才改大小。`priority_queue` 的 `top` 只保证极值，不能遍历出有序序列。

迭代器失效是工程题常见追问。`vector` 扩容后所有指针和迭代器失效；`erase` 中间元素会让后续位置移动；`unordered_map` `rehash` 会让迭代器失效；`list` 的 `splice` 能移动节点且保持其他迭代器稳定。LRU 缓存为什么用 `list` 加哈希表，核心就是哈希定位和链表节点稳定的组合。

代码组织要让不变量有位置。二分答案把检查函数单独写出来，图最短路把松弛逻辑集中，回

溯把 push、搜索、pop 成对出现，DP 把初始化和主转移分开。这样做能让读者看到状态生命周期。把所有逻辑塞进一个巨大循环，短期可能省行数，长期会让边界和证明都变模糊。

面试时可以主动说明替代方案。第 K 大可以排序、堆、快速选择；岛屿数量可以 DFS、BFS、并查集；最小体力路径可以 Dijkstra、二分加 BFS、并查集排序边；单词接龙可以普通 BFS、通配桶、双向 BFS。比较方案不是炫耀，而是说明你知道当前选择依赖什么约束。

Linux 源码阅读可以作为 C++ 容器理解的反面教材。内核不使用 STL 容器，而是大量使用侵入式链表、红黑树、哈希表和 XArray，因为它需要明确控制内存分配、锁、对象生命周期和布局。理解这一点后，回看 STL 会更清楚：STL 更强调泛型接口和异常安全，内核结构更强调嵌入式节点和可控开销。两者都围绕访问模式服务。

实验建议：写一个前缀和计数题，分别用 contains 加 operator[]、find、直接 operator[] 三种方式实现，观察语义差异。写一个 vector 扩容实验，保存元素地址后继续 push_back，验证地址失效。写一个 LRU，先用 vector 保存顺序，再用 list 加 unordered_map，比较移动到头部的成本。

本节复盘

阅读本节后，请回到相邻章节的例题，把暴力瓶颈、优化依据、状态不变量、C++ 容器成本和边界测试重新写一遍。能把同一思想迁移到变体题，才算真正掌握。

力扣刷题手册

刷题不要按编号顺序推进，要按题型推进。每道题都记录题型、输入规模、暴力解、优化来源、不变量、使用的数据结构、复杂度、错因和一周后是否能独立重写。

数组和字符串主线：1 两数之和、26 删除有序数组重复项、27 移除元素、88 合并两个有序数组、53 最大子数组和、283 移动零、344 反转字符串。

双指针和窗口主线：3 无重复字符的最长子串、11 盛最多水的容器、15 三数之和、42 接雨水、76 最小覆盖子串、209 长度最小的子数组、438 找到字符串中所有字母异位词。

哈希和前缀和主线：49 字母异位词分组、128 最长连续序列、202 快乐数、560 和为 K 的子数组、523 连续的子数组和、525 连续数组。

栈和堆主线：20 有效括号、155 最小栈、739 每日温度、84 柱状图中最大的矩形、215 数组中的第 K 个最大元素、347 前 K 个高频元素、295 数据流的中位数。

二分主线：35 搜索插入位置、34 在排序数组中查找元素的第一个和最后一个位置、33 搜索旋转排序数组、162 寻找峰值、875 爱吃香蕉的珂珂、410 分割数组的最大值。

树和图主线：102 二叉树层序遍历、104 最大深度、98 验证二叉搜索树、236 最近公共祖先、200 岛屿数量、994 腐烂的橘子、207 课程表、210 课程表 II、127 单词接龙。

回溯主线：46 全排列、47 全排列 II、78 子集、39 组合总和、22 括号生成、79 单词搜索、51 N 皇后。

动态规划主线：70 爬楼梯、198 打家劫舍、322 零钱兑换、300 最长递增子序列、139 单词拆分、1143 最长公共子序列、72 编辑距离。

面试专项主线：19 删除链表倒数第 N 个节点、21 合并两个有序链表、25 K 个一组翻转链表、84 柱状图中最大的矩形、239 滑动窗口最大值、56 合并区间、75 颜色分类、215 数组中的第 K 个最大元素、347 前 K 个高频元素、295 数据流的中位数、55 跳跃游戏、45 跳跃游戏 II、547 省份数量、684 冗余连接、743 网络延迟时间、1631 最小体力消耗路径、416 分割等和子集、494 目标和、518 零钱兑换 II、121 买卖股票的最佳时机、309 最佳买卖股票时机含冷冻期。

下面的题目卡片不是为了替代正文，而是为了复习时快速恢复推理链。每一题都要先自己说出暴力解，再说优化依据；如果只能背出最终代码，就把它当作没有掌握。

1 两数之和。题目要求在数组中找到两个数，使它们的和等于目标值。暴力方法是双层循环枚举两个下标，复杂度 $O(n^2)$ 。优化来自“当前数需要的另一个数是确定的”：扫描数组时，用哈希表保存已经见过的值到下标的映射，处理 x 时查询 $target - x$ 是否出现过。必须先查再插入当前数，避免同一个元素被使用两次。复杂度 $O(n)$ ，空间 $O(n)$ 。若数组已排序，可以用双指针，但原题通常要求返回原始下标，哈希更直接。

26 删除有序数组重复项。有序是核心条件。暴力可以新建数组保存不重复元素，再复制回原数组；原地优化用慢指针表示下一个唯一元素应该写入的位置，快指针扫描所有元素。因为重复元素相邻，只要当前值和前一个不同，就写到慢指针位置。正确性不变量是： $[0, slow)$ 始终保存已经处理前缀中的唯一元素且顺序不变。复杂度 $O(n)$ ，额外空间 $O(1)$ 。易错点是空数组和返回的是新长度，不是数组本身。

27 移除元素。题目要求原地移除所有等于给定值的元素，元素相对顺序通常不重要或不强制。保持顺序的做法是快慢指针：快指针扫描，遇到不等于目标的值就写到慢指针位置。若不要求顺序，还可以用左右指针，把等于目标的左侧元素和右端未处理元素交换，从而减少写入次数。面试中要先确认题目是否要求稳定顺序。复杂度 $O(n)$ ，空间 $O(1)$ 。

88 合并两个有序数组。暴力可以把两个数组合并后排序，但没有利用有序条件。由于第一个数组尾部有空位，最优写法从后往前填充：比较两个数组当前最大值，把较大者放到最终位置。这样不会覆盖第一个数组中尚未比较的元素。若从前往后写，会破坏还没读取的值。循环结束后，如果第二个数组还有剩余，需要继续复制；第一个数组剩余则天然已经在正确位置。复杂度 $O(m+n)$ 。

53 最大子数组和。暴力枚举所有子数组并求和，直接做是 $O(n^3)$ ，用前缀和可降到 $O(n^2)$ 。进一步优化是 Kadane 算法：令 $current$ 表示必须以当前位置结尾的最大子数组和，那么它要么接

在前一个最大后缀后面，要么从当前元素重新开始。转移是 $\text{current} = \max(x, \text{current} + x)$ 。答案是所有 current 的最大值。易错点是全负数组，初始答案不能设成 0。

283 移动零。题目要求把零移动到末尾，同时保持非零元素相对顺序。暴力可以不断删除零再追加零，但移动成本高。双指针做法先把所有非零元素按原顺序写到前面，再把剩余位置填 0。也可以扫描时遇到非零就和慢指针交换，这样一次扫描完成。正确性不变量是：慢指针左侧都是已经稳定放好的非零元素。复杂度 $O(n)$ ，空间 $O(1)$ 。

344 反转字符串。这是双指针基本题。左右指针分别从两端向中间移动，每次交换字符。暴力新建字符串再倒序复制也正确，但题目通常要求原地修改。正确性来自每次交换后，两端各有一个位置已经放到最终位置。复杂度 $O(n)$ ，空间 $O(1)$ 。它看似简单，但能训练边界：空串、单字符、偶数长度、奇数长度都要自然通过。

3 无重复字符的最长子串。暴力枚举所有子串并检查是否重复，复杂度高。滑动窗口优化来自单调性：当右端扩展导致重复时，左端只能右移，不能左移。用哈希表或数组记录字符上次出现位置，扫描右端时把左边界更新到重复字符上次位置之后。窗口始终保持无重复。复杂度 $O(n)$ 。易错点是左边界只能取更大值，不能被窗口外的旧位置拉回去。

11 盛最多水的容器。暴力枚举两条线，复杂度 $O(n^2)$ 。双指针优化的依据是面积由较短边决定。左右指针形成当前宽度，若移动较高边，宽度变小且短板不变或更差，不可能得到更大面积；因此每次移动较短边，才有机会提高短板。这个排除论证是本题核心。复杂度 $O(n)$ 。不要把它和接雨水混淆：本题只选两条边，接雨水计算每个位置贡献。

15 三数之和。暴力三层循环是 $O(n^3)$ 。排序后固定第一个数，剩下两个数用双指针找和为相反数的组合。排序制造了单调性：和太小就左指针右移，和太大就右指针左移。去重要分两层：固定数相同要跳过；找到一组解后，左右指针也要跳过相同值。复杂度 $O(n^2)$ 。易错点是不能用哈希随便拼结果而不处理重复。

42 接雨水。暴力对每个位置分别向左向右找最高柱，复杂度 $O(n^2)$ 。前后缀最大数组能把每个位置的左右最高值预处理出来，时间 $O(n)$ 、空间 $O(n)$ 。双指针进一步维护 left_max 和 right_max ，每次结算较低的一侧，因为较低侧的水位已经由本侧最大值决定。也可用单调栈在遇到右边界时结算凹槽。面试中要说明每种解法的状态含义。

76 最小覆盖子串。暴力枚举所有子串再检查是否覆盖目标字符，复杂度很高。滑动窗口成立的原因是：右端扩展只会让覆盖条件更容易满足，左端收缩只会让窗口更短但可能破坏覆盖。用两个计数表维护需求和窗口内字符数量，并用 formed 记录已经满足的字符种类数。每当窗口满足覆盖，就尽量左移缩短。易错点是字符重复需求，例如目标串有两个 A 时，窗口中一个 A 不够。

209 长度最小的子数组。数组元素为正时是滑动窗口的关键。暴力枚举起点终点，复杂度 $O(n^2)$ 。因为元素为正，右端扩展会让和变大，左端收缩会让和变小，窗口具有单调性。扫描右端累加，当和达到目标时，不断左移并更新最短长度。若数组存在负数，这个方法会失效，需要前缀和加单调队列等更复杂方法。复杂度 $O(n)$ 。

438 找到字符串中所有字母异位词。暴力枚举每个长度为 p 的子串并排序比较，复杂度高。固定长度窗口可以维护字符频次，右进左出。若字符集是小写字母，用长度 26 的数组比哈希表更快。窗口长度达到 $p.\text{size}()$ 后检查频次数组是否相等。更优化的写法维护差异计数，避免每次比较 26 个位置，但面试中先写清楚版本更稳。复杂度 $O(n)$ 或 $O(26n)$ 。

49 字母异位词分组。暴力两两判断是否异位，复杂度高。优化是为每个字符串构造规范 key：排序后的字符串，或者 26 个字符计数组成的 key。异位词会得到相同 key，放入同一哈希桶。排序 key 的复杂度是每个词 $O(L \log L)$ ，计数 key 是 $O(L)$ 但实现稍繁。易错点是 key 必须无歧义，例如计数拼接要加分隔符，避免 1,11 和 11,1 混淆。

128 最长连续序列。排序后扫描连续段是 $O(n \log n)$ 。哈希优化是把所有数放入集合，只从“没有前驱”的数开始扩展连续段。若 $x - 1$ 存在，说明 x 不是序列起点，跳过它，避免重复扫描。每个数最多作为某个连续段的一部分被访问常数次数，因此平均 $O(n)$ 。易错点是重复元素，哈希集合天然去重。

202 快乐数。反复把数字替换为各位平方和，若最终到 1 则快乐。关键是检测循环。可以用哈希集合记录出现过的数字；如果再次出现同一个数字，就会无限循环。也可以用快慢指针，把转换函数看成链表的 next ，若存在环且环入口不是 1，就不是快乐数。暴力一直算不加检测会死循环。由于数字平方和会很快下降到有限范围，复杂度很小。

560 和为 K 的子数组。暴力枚举所有子数组并求和，用前缀和可到 $O(n^2)$ 。进一步优化是扫描

前缀和 `prefix`，如果之前出现过 `prefix - k`，那么这些位置到当前下标之间的子数组和为 `k`。哈希表保存历史前缀和出现次数。必须先查询再更新当前前缀，避免把空区间算进去。复杂度 $O(n)$ ，可以处理负数，这是它和滑动窗口的重要区别。

523 连续的子数组和。题目要求长度至少为 2 的子数组和是 `k` 的倍数。根据同余性质，若两个前缀和对 `k` 取模相同，它们之间的区间和能被 `k` 整除。哈希表保存某个余数第一次出现的位置，尽量保留最早位置以满足长度条件。初始化余数 0 的位置为 -1，表示从开头开始的区间。易错点是 `k` 的取值和长度至少为 2。

525 连续数组。把 0 看成 -1，1 看成 +1，问题变成找和为 0 的最长子数组。扫描前缀和，若同一个前缀和值再次出现，两次出现之间的区间和为 0，说明 0 和 1 数量相等。哈希表保存每个前缀和第一次出现的位置，因为要求最长区间。复杂度 $O(n)$ 。这题训练的是“把计数相等转换成前缀和相等”。

20 有效括号。栈保存尚未匹配的左括号。遇到左括号入栈，遇到右括号时检查栈顶是否是对应左括号。若栈空或类型不匹配，立即失败；扫描结束后栈必须为空。暴力反复删除相邻匹配括号也能做，但复杂度差且不自然。栈模型对应“最近打开的括号必须最先关闭”。易错点是只检查匹配过程却忘记最后栈非空。

155 最小栈。普通栈无法常数时间返回最小值，因为最小值可能埋在中间。同步最小栈保存每个深度下的历史最小值，压入元素时同时压入当前最小，弹出时同步弹出。也可以只在新值不大于当前最小时压入辅助栈，但弹出时要处理重复最小值。核心不变量是：辅助结构栈顶永远对应当前数据栈的最小值。所有操作 $O(1)$ 。

739 每日温度。暴力对每一天向后找第一个更高温度，复杂度 $O(n^2)$ 。单调栈保存还没找到更高温度的下标，且对应温度从栈底到栈顶单调递减。扫描到新温度时，它会解决所有比它低的栈顶日子，答案就是下标差。每个下标最多入栈出栈一次，复杂度 $O(n)$ 。易错点是栈里存下标，不是只存温度。

84 柱状图中最大的矩形。暴力枚举每根柱子作为最矮柱向左右扩展，复杂度 $O(n^2)$ 。单调栈维护递增高度下标；当遇到更矮柱子时，栈顶柱子的右边界确定，左边界是弹出后新的栈顶。为统一边界，可在左右加入高度 0 的哨兵。每根柱子弹出时计算一次以它为最矮高度的最大矩形。复杂度 $O(n)$ 。关键是解释宽度，不要只背代码。

215 数组中的第 K 个最大元素。完整排序是 $O(n \log n)$ 。维护大小为 `K` 的小顶堆是 $O(n \log k)$ ，堆顶就是当前第 `K` 大。快速选择平均 $O(n)$ ，通过 partition 每次只递归或迭代进入目标所在一侧。面试中最稳的是先讲堆，再补快速选择。若写快速选择，要随机化 pivot 或说明最坏复杂度 $O(n^2)$ 的风险。

347 前 K 个高频元素。先用哈希表统计频次。方案一是按频次完整排序，复杂度 $O(m \log m)$ 。方案二是大小为 `K` 的小顶堆，复杂度 $O(m \log k)$ 。方案三是桶排序，频次最大不超过 `n`，把元素按频次放桶后从高频桶向低频桶收集，复杂度 $O(n)$ 但空间更高。要根据 `K` 和数据规模比较方案。

295 数据流的中位数。如果每次插入后排序，插入成本高。双堆把数据分成较小一半和较大一半：大顶堆保存较小一半，小顶堆保存较大一半。保持两个堆大小差不超过 1，且左堆堆顶不大于右堆堆顶。中位数由堆顶得到。插入复杂度 $O(\log n)$ ，查询 $O(1)$ 。易错点是偶数个元素时求平均要防溢出。

35 搜索插入位置。本质是找第一个大于等于目标值的位置，也就是 `lower_bound`。二分不变量可以定义为答案在 `[left, right]` 或 `[left, right)` 中，选择一种写法坚持到底。若中点值小于目标，答案一定在右侧；否则答案在当前位置或左侧。复杂度 $O(\log n)$ 。不要在等于时提前返回后又忘记插入位置语义，使用 `lower_bound` 模型最统一。

34 在排序数组中查找元素的第一个和最后一个位置。分别找第一个大于等于 `target` 的位置和第一个大于 `target` 的位置，后者减一就是最后位置。这样比在找到一个 `target` 后向两边扩展更稳定，最坏仍为 $O(\log n)$ 。若第一个位置越界或值不是 `target`，说明不存在。关键是把“范围查找”拆成两个边界查找。

33 搜索旋转排序数组。数组由有序数组旋转而来，每次二分至少有一半是有序的。比较 `nums[left]` 和 `nums[mid]` 判断左半是否有序；若目标落在有序半边范围内，就收缩到那边，否则去另一边。暴力线性扫描是 $O(n)$ 。二分复杂度 $O(\log n)$ 。若存在重复元素，判断有序半边会变复杂，可能退化，需要额外处理。

162 寻找峰值。峰值是比相邻元素大的位置。二分依据是斜率：如果 `nums[mid] < nums[mid`

+ 1], 右侧一定存在峰值; 否则左侧含 `mid` 一定存在峰值。这个结论来自数组边界可视作负无穷, 以及上升段最终要么继续到边界形成峰, 要么下降形成峰。复杂度 $O(\log n)$ 。不要试图找全局最大, 题目只要任意峰值。

875 爱吃香蕉的珂珂。速度越快, 吃完所需时间越少, 存在单调性。二分速度 `speed`, 检查以该速度能否在 `h` 小时内吃完。检查函数中每堆耗时是向上取整, 可以写成 $(pile + speed - 1) / speed$ 。最小可行速度用左闭右闭或左闭右开二分都可以。复杂度 $O(n \log M)$ 。本题训练“答案空间二分”, 不是在数组下标上二分。

410 分割数组的最大值。给数组分成 `k` 段, 最小化各段和的最大值。暴力枚举切分位置复杂度高, 也可用 DP。更常见是二分答案: 猜最大段和上限 `limit`, 贪心扫描数组, 尽量往当前段放, 超过上限就开新段, 统计所需段数。如果段数不超过 `k`, 说明上限可行, 可以尝试更小。下界是最大元素, 上界是总和。复杂度 $O(n \log S)$ 。

102 二叉树层序遍历。队列保存当前层待处理节点。每轮先记录队列大小, 这个大小就是当前层节点数; 弹出这些节点并把子节点加入队列。暴力递归按深度收集也可以, 但队列更直接体现层次。复杂度 $O(n)$ 。易错点是如果不记录层大小, 容易把下一层节点混进当前层结果。

104 二叉树最大深度。递归定义很自然: 树深度等于左右子树最大深度加一。但为了避免深树递归栈问题, 也可以用层序遍历迭代计算层数。面试中如果题目环境不限制树深, 递归可读性高; 工程中要意识到递归深度风险。复杂度 $O(n)$ 。这题核心是深度定义和空树返回 0。

98 验证二叉搜索树。不能只检查每个节点大于左孩子、小于右孩子, 因为子树中更深的节点也受祖先边界约束。方法一是中序遍历应严格递增; 方法二是迭代栈遍历时维护上一个访问值; 方法三是给每个节点传入合法上下界。若允许重复值, 比较条件要根据题目定义调整。复杂度 $O(n)$ 。

236 最近公共祖先。若是普通二叉树, 可以记录每个节点父指针, 再把一个节点的祖先放入集合, 向上查另一个节点; 也可以递归后序判断左右子树是否包含目标。若是二叉搜索树, 可利用值的大小关系从根向下走: 两个目标都小于当前节点就去左边, 都大于就去右边, 否则当前节点就是分叉点。复习时要区分普通树和搜索树条件。

200 岛屿数量。网格中的陆地相邻即连通块。暴力从每个陆地反复搜索会重复计数。正确做法是扫描网格, 遇到未访问陆地就启动 BFS/DFS, 把整座岛标记访问, 岛屿数加一。也可用并查集合并相邻陆地。复杂度 $O(mn)$ 。易错点是访问标记必须在入队时设置, 避免同一格子重复入队。

994 腐烂的橘子。多源 BFS。所有初始腐烂橘子同时作为第 0 分钟入队, 按层扩散到相邻新鲜橘子。最后若还有新鲜橘子未腐烂, 返回 -1; 否则返回最大层数。不要从每个腐烂橘子分别 BFS, 那会重复扩散且难以处理最短时间。复杂度 $O(mn)$ 。核心是“同时扩散”。

207 课程表。有向图判环。Kahn 拓扑排序用入度数组找当前没有前置依赖的课程, 处理后减少后继入度。若最终处理数量等于课程数, 则无环; 否则存在环。边方向要特别注意: `[a,b]` 表示学 `a` 前要学 `b`, 因此边是 `b` 指向 `a`。复杂度 $O(n + m)$ 。

210 课程表 II。在课程表判环基础上, 把出队顺序记录下来就是一种可行学习顺序。若最后记录长度小于课程数, 返回空数组。它和 207 的区别只是要输出拓扑序。若有多个入度为 0 的课程, 选择任意一个都可; 若题目要求字典序最小, 就把队列换成小顶堆。

127 单词接龙。单词是状态, 一次改变一个字符形成边, 边权相同, 所以 BFS 求最短转换长度。暴力建图两两比较单词复杂度高; 通配模式如 `h*t` 可以把相邻候选聚到同一桶。若数据较大, 双向 BFS 从起点和终点同时扩展能显著减少访问状态数。易错点是起始单词不一定在字典中, 结束单词必须在字典中。

46 全排列。回溯构造路径, 每一层选择一个尚未使用的元素。状态包括当前路径和 `used` 数组。暴力本质就是枚举所有排列, 复杂度 $O(n!)$, 无法从渐进意义上优化, 只能通过剪枝或生成方式减少常数。回溯不变量是: 路径中元素互不重复, 深度等于已选择数量。到深度 `n` 时收集答案。

47 全排列 II。输入有重复元素, 需要去重。先排序, 让相同元素相邻。每层枚举候选时, 如果当前元素和前一个相同, 且前一个相同元素在本层还没有被使用, 则跳过当前元素, 避免同一层选择相同值造成重复排列。去重条件要和 `used` 数组配合, 不能简单跳过所有相等值, 否则会漏解。

78 子集。每个元素都有选或不选两种选择, 答案数是 $O(2^n)$ 。可以用回溯: 每进入一个状态就收集当前路径, 然后从当前位置之后继续选择元素。也可以用位掩码枚举所有集合。回溯写法更直观, 位掩码适合讲状态压缩。子集题不要求固定长度, 因此每个节点都是一个合法答案。

39 组合总和。元素可以重复使用, 组合不区分顺序。回溯时传入起始下标 `start`, 下一层仍可从当前下标开始, 因为可以重复选; 但不能回到更小下标, 否则会产生排列重复。剪枝条件是当前

和超过目标。若候选数组排序，还可以在超过目标时提前 break。核心是“允许重复使用，但保持非降选择顺序”。

22 括号生成。状态是已经放入的左括号数和右括号数。任何前缀中右括号数不能超过左括号数，左括号数不能超过 n 。暴力生成所有长度为 $2n$ 的括号串再检查会浪费大量非法前缀；回溯在构造过程中剪掉非法前缀。正确性来自所有合法括号串都满足这两个约束，所有满足约束且长度为 $2n$ 的串都合法。

79 单词搜索。网格回溯。状态包括当前位置、匹配到单词的下标和访问标记。每一步只能走上下左右，且同一格不能重复使用。暴力从每个格子作为起点尝试匹配；剪枝包括首字符不匹配直接跳过，越界、已访问、字符不等立即返回。复杂度最坏指数级，但实际由剪枝控制。易错点是回溯后恢复访问标记。

51 N 皇后。每行放一个皇后，递归或迭代枚举行。合法性由列、主对角线、副对角线三个集合判断。主对角线可用 $row - col$ 标识，副对角线用 $row + col$ 标识。暴力在棋盘任意格放皇后组合巨大；按行放置把每行一个皇后的约束提前固定。答案数本来可能很多，重点是剪枝和状态恢复。

70 爬楼梯。暴力递归会重复计算同一层数的方案数。状态 $dp[i]$ 表示到达第 i 阶的方法数，最后一步来自 $i-1$ 或 $i-2$ ，所以是斐波那契型转移。可以用两个变量滚动。复杂度 $O(n)$ ，空间 $O(1)$ 。这是理解“重叠子问题”的入门题。

198 打家劫舍。每间房可偷或不偷，相邻不能同时偷。状态 $dp[i]$ 表示考虑前 i 间房的最大收益。第 i 间不偷，收益是 $dp[i-1]$ ；偷，则收益是 $dp[i-2] + nums[i]$ 。取最大。可用两个变量滚动。若题目变成环形房屋，第一间和最后一间互斥，需要拆成不含首、不含尾两个线性问题。

322 零钱兑换。求最少硬币数，是完全背包最值问题。状态 $dp[amount]$ 表示凑出金额 $amount$ 的最少硬币数，初始化为无穷， $dp[0]=0$ 。对每个金额，尝试每种硬币，从 $dp[value - coin] + 1$ 转移。和零钱兑换 II 不同，这里求最少数量，不是组合数。若无法凑出，返回 -1。复杂度 $O(amount \cdot coins)$ 。

300 最长递增子序列。二次 DP 定义 $dp[i]$ 为以 i 结尾的 LIS 长度，枚举所有前驱，复杂度 $O(n^2)$ 。二分优化维护 $tails[len]$ ，表示长度为 $len+1$ 的递增子序列的最小可能结尾。扫描每个数，用 $lower_bound$ 替换第一个大于等于它的位置。tails 长度是答案，但 tails 本身不一定是一条真实子序列。

139 单词拆分。 $dp[i]$ 表示前 i 个字符能否被字典拼出。枚举最后一个单词起点 j ，若 $dp[j]$ 为真且 $s[j..i]$ 在字典中，则 $dp[i]$ 为真。暴力递归会反复处理同一个后缀。优化时可以限制最大单词长度，减少枚举起点；也要注意 `substr` 的拷贝成本。复杂度取决于字符串长度和字典查询成本。

1143 最长公共子序列。状态 $dp[i][j]$ 表示两个字符串前缀的 LCS 长度。若末尾字符相同，从左上角加一；否则取上方和左方最大值。它和最长公共子串不同，子序列允许不连续，因此不匹配时不是归零。复杂度 $O(nm)$ 。一维优化需要保存左上角旧值，初学阶段先写二维更可靠。

72 编辑距离。状态 $dp[i][j]$ 表示第一个单词前 i 个字符变成第二个单词前 j 个字符的最少操作。字符相同则继承左上角；不同则在删除、插入、替换三种操作中取最小。空串边界分别是插入或删除所有字符。暴力递归会重复求同一对前缀。复杂度 $O(nm)$ ，可用两行滚动数组优化空间。

19 删除链表倒数第 N 个节点。两遍扫描先求长度再删除是完全正确的基础解。快慢指针优化让快指针先走 n 步，再同步前进，保持距离不变量。虚拟头节点统一删除头节点和中间节点。循环结束时慢指针在待删节点前驱，修改它的 `next` 即可。测试要覆盖删除头、删除尾、单节点。

21 合并两个有序链表。维护结果链表尾指针，每次接入两个链表头部较小的节点。由于输入已经有序，局部较小者就是全局下一个节点。剩余链表可以整体接上。复杂度 $O(m+n)$ ，额外空间 $O(1)$ 。若题目要求不修改原链表，则需要创建新节点。

25 K 个一组翻转链表。难点是分组边界。每轮先从上一组尾部向后找第 K 个节点，不足 K 个则结束。保存下一组头节点后，在当前闭区间内反转指针，再把上一组尾接到新头，把新尾接到下一组。不要边数不足时也反转。画出上一组尾、当前组头、当前组尾、下一组头四个位置，代码就清楚。

239 滑动窗口最大值。暴力每个窗口扫描最大值，复杂度 $O(nk)$ 。单调队列保存可能成为最大值的下标，队列中的值递减。右端新元素进入时，弹出所有不大于它的尾部候选；队首若过期则弹出。窗口形成后队首就是最大值。每个下标进出队一次，复杂度 $O(n)$ 。必须存下标才能判断过期。

56 合并区间。按起点排序后线性扫描。当前区间若起点不超过结果最后区间的终点，则合并

并更新终点；否则追加为新区间。排序让所有可能与当前结果尾部相交的区间连续出现。复杂度 $O(n \log n)$ 。先明确闭区间还是半开区间，决定等号是否成立。

75 颜色分类。三指针维护四段：已放好的 0、已放好的 1、未知区、已放好的 2。遇到 0 和低端交换并移动低端和扫描指针；遇到 1 只移动扫描指针；遇到 2 和高端交换并只移动高端，因为换回来的元素还没检查。复杂度 $O(n)$ ，空间 $O(1)$ 。

55 跳跃游戏。贪心维护当前能到达的最远位置。扫描到位置 i 时，如果 i 已经超过最远可达，说明无法到达；否则用 $i + \text{nums}[i]$ 更新最远位置。若最远位置覆盖最后下标，返回真。暴力从每个位置尝试所有跳法会指数爆炸。贪心正确性来自：只关心可达前缀内能扩展到的最远边界，不需要记录具体路径。

45 跳跃游戏 II。求最少跳数。扫描当前跳数能覆盖的区间，同时维护下一跳能到达的最远位置。当扫描到当前区间右端时，必须跳一次，并把区间扩展到下一跳最远位置。它相当于在隐式图上按层 BFS，但不显式建队列。复杂度 $O(n)$ 。易错点是不要在最后一个位置再额外增加跳数。

547 省份数量。城市直接连接关系形成无向图，省份就是连通分量。可以 BFS/DFS，也可以并查集。邻接矩阵只需扫描上三角，遇到连接就合并。最后统计根节点数量。复杂度 $O(n^2)$ 。并查集适合强调动态合并，BFS 适合直接遍历图，两者都合理。

684 冗余连接。树加一条边会形成唯一环。按输入顺序扫描边，用并查集维护已加入边形成的连通关系。如果一条边的两个端点已经连通，再加入它就会成环，这条边就是答案。暴力每加一条边就 DFS 检查连通性会重复扫描。并查集复杂度近似 $O(n)$ 。

743 网络延迟时间。有向正权图，从起点到所有点的最短路最大值。边权非负，适合 Dijkstra。邻接表建图，小顶堆保存当前距离最小的待处理状态；堆中旧状态用距离不等于当前最短距离来跳过。若最终有节点距离仍为无穷，返回 -1。复杂度 $O(m \log n)$ 。不能用普通 BFS，因为边权不相同。

1631 最小体力消耗路径。路径代价是经过边的最大高度差，而不是边权和。Dijkstra 仍可用，因为扩展路径时新代价是当前代价和新边代价的最大值，具有单调性。也可以二分体力上限，再 BFS 或并查集检查可达性，依据是上限越大越容易到达。面试中能说出两种模型会很加分。

416 分割等和子集。总和为奇数直接失败；否则转成能否选若干数凑出 $\text{sum}/2$ 。每个数只能用一次，是 0-1 背包可达性。滚动数组要倒序遍历容量，避免同一个数在一轮内被重复使用。复杂度 $O(n \cdot \text{target})$ ，这是伪多项式复杂度，和数值大小有关。

494 目标和。设正号集合和为 P ，负号集合和为 N ，总和 S ，有 $P - N = \text{target}$ 且 $P + N = S$ ，推出 $P = (S + \text{target})/2$ 。问题转成从数组中选若干数凑出 P 的方案数。若 $S + \text{target}$ 为奇数或目标绝对值超过总和，答案为 0。0 元素会让方案数翻倍，标准背包计数能自然处理。

518 零钱兑换 II。硬币无限使用，问组合数。外层遍历硬币，内层正序遍历金额，这样每种组合只会按固定硬币顺序被统计一次。若外层遍历金额、内层遍历硬币，会把不同顺序算作不同排列。状态 $\text{dp}[\text{value}]$ 表示凑出金额 value 的组合数， $\text{dp}[0] = 1$ 。复杂度 $O(\text{amount} \cdot \text{coins})$ 。

121 买卖股票的最佳时机。只允许一次买卖。暴力枚举买卖日是 $O(n^2)$ 。优化依据是：卖出日固定时，最佳买入日一定是此前最低价格。扫描价格，维护历史最低价和最大利润。复杂度 $O(n)$ 。若价格一直下降，最大利润应为 0，表示不交易。

309 最佳买卖股票时机含冷冻期。冷冻期让“只维护最低价”失效，因为今天能否买取决于昨天是否卖出。状态机更稳：**hold** 表示持有，**sold** 表示今天刚卖，**rest** 表示不持有且不处于刚卖状态。买入只能从 **rest** 来，卖出只能从 **hold** 来，休息可从 **rest** 或 **sold** 来。转移时先保存昨天状态，避免同日污染。

2 两数相加。链表按低位到高位保存数字，要求返回两数之和。暴力可以把链表转成整数再相加，但数字长度可能超过内置整数范围。正确模型是模拟竖式加法：两个指针分别扫描两条链表，维护进位，每一位创建一个新节点。循环条件是任一链表未结束或进位不为 0。易错点是最后进位、两链长度不同、不要使用可能溢出的整数。这个题训练的是链表遍历和状态变量，而不是数学大数库。

5 最长回文子串。暴力枚举所有子串并检查回文，复杂度可达 $O(n^3)$ 。中心扩展利用回文关于中心对称的性质：每个位置可以作为奇数长度中心，每两个相邻位置之间可以作为偶数长度中心，向两侧扩展并更新最长区间。复杂度 $O(n^2)$ ，空间 $O(1)$ 。DP 也可做，状态表示子串是否回文，但中心扩展更容易写。易错点是偶数长度回文，例如 **abba**。

31 下一个排列。题目要求原地得到字典序中的下一个排列。若整个序列从右到左非递减，即从左到右非递增，说明已经是最大排列，直接反转成最小排列。否则从右向左找到第一个 $\text{nums}[i] < \text{nums}[i+1]$ 的位置，再从右侧找到第一个大于它的元素交换，最后反转右侧后缀。优化依据是右侧

后缀本来是降序，交换后反转能得到最小的更大后缀。它不是排序题，而是字典序结构题。

54 螺旋矩阵。直接模拟四个边界：上、下、左、右。每一轮依次走上边、右边、下边、左边，走完收缩对应边界。暴力用访问数组也能做，但四边界更简洁。易错点是单行、单列、最后一圈只剩一行或一列时不要重复加入元素。循环条件可以写成 `top <= bottom && left <= right`，每走完一条边都检查边界是否仍合法。

69 x 的平方根。求不超过平方根的整数。二分答案，范围从 0 到 x 。判断 `mid * mid <= x` 时要避免整数溢出，可以改成 `mid <= x / mid`。若条件成立，记录 `mid` 并向右找更大值；否则向左。复杂度 $O(\log x)$ 。这个题训练的是二分答案和溢出意识，不是浮点计算。

136 只出现一次的数字。所有数字除一个外都出现两次，使用异或。异或满足交换律、结合律，且 $x \text{ xor } x = 0$ 、 $x \text{ xor } 0 = x$ ，所以把所有数异或起来，成对数字抵消，剩下的就是答案。暴力哈希计数当然可做，但空间 $O(n)$ 。位运算解法空间 $O(1)$ 。如果题目改成其他数字出现三次，就要按位计数取模，不能继续简单异或。

146 LRU 缓存。需要 `get` 和 `put` 都是常数时间，并在容量满时淘汰最久未使用项。哈希表能常数定位 `key`，但不能维护新旧顺序；双向链表能常数移动节点和删除尾部，但不能常数按 `key` 查找。组合起来：哈希表保存 `key` 到链表节点迭代器，链表头表示最新使用，尾表示最久未使用。每次访问或更新都把节点移动到头部。核心是两个结构职责互补。

150 逆波兰表达式求值。后缀表达式中，操作符出现在操作数之后。扫描 `token`，遇到数字入栈，遇到运算符就弹出两个操作数计算后再压回。注意弹出顺序：先弹出的是右操作数，后弹出的是左操作数，减法和除法不能写反。复杂度 $O(n)$ 。它和中缀表达式求值的区别是没有括号和优先级解析压力，因为表达式顺序已经编码了计算依赖。

160 相交链表。交点是节点地址相同，不是值相同。哈希集合能做，双指针更省空间。两个指针分别走自己的链，走到空后切换到另一条链。若有交点，两个指针走过相同总长度后相遇；若无交点，同时为空。这个题要明确不能修改链表，否则可以临时连尾成环再检测，但那会引入副作用和恢复问题。

169 多数元素。如果某个元素出现次数超过一半，可以用 Boyer-Moore 投票。维护候选值和计数，遇到相同值计数加一，不同值计数减一，计数归零时更换候选。直觉是多数元素可以和其他元素两两抵消，最后剩下的候选一定是多数元素。若题目不保证多数元素存在，最后还要再扫描验证候选出现次数。复杂度 $O(n)$ ，空间 $O(1)$ 。

189 轮转数组。暴力每次右移一位，重复 k 次，复杂度 $O(nk)$ 。优化有三种：使用额外数组把元素放到 $(i+k) \bmod n$ ；使用环状替换；使用三次反转。三次反转最常见：先整体反转，再反转前 k 个，再反转后 $n-k$ 个。注意 k 需要对 n 取模。这个题训练原地数组操作和下标边界。

206 反转链表。迭代写法维护 `previous`、`current`、`next`。每次先保存 `current->next`，再把当前节点指向前驱，然后两个指针前进。暴力用数组保存节点再重连也可以，但不如原地反转体现链表基本功。正确性不变量是：`previous` 指向已经反转好的前缀头，`current` 指向尚未处理的第一个节点。

217 存在重复元素。哈希集合扫描，若插入前已经存在当前值，返回 `true`。排序后检查相邻也可以，时间 $O(n \log n)$ ，空间可更低。哈希平均 $O(n)$ ，但需要额外空间。面试中要根据是否允许修改输入来选择：排序会改变顺序，若输入顺序后续有意义，哈希更稳。

238 除自身以外数组的乘积。暴力对每个位置乘其他所有元素，复杂度 $O(n^2)$ ；用总乘积再除以当前数会被 0 破坏，也可能不允许除法。前缀积和后缀积解决：答案先保存左侧所有元素乘积，再从右向左乘上右侧乘积。空间可以做到输出数组以外 $O(1)$ 。易错点是多个 0 和一个 0 的情况，前后缀写法自然处理。

240 搜索二维矩阵 II。每行升序、每列升序。从右上角开始，若当前值大于目标，当前列下面更大，列可以左移；若当前值小于目标，当前行左侧更小，行可以下移。每一步排除一行或一列，复杂度 $O(m+n)$ 。从左上角无法决定排除方向，因为右边和下边都更大。这个题训练利用二维单调性选择起点。

287 寻找重复数。数组长度 $n+1$ ，数值在 1 到 n ，存在重复。不能修改数组且要求常数空间时，可以把数组看成函数图：下标指向 `nums[index]`，重复数就是环入口。使用快慢指针先相遇，再从起点和相遇点同步走找到入口。也可以二分值域，统计小于等于 `mid` 的数量，根据抽屉原理判断重复数在哪一侧。两种方法都不依赖排序。

394 字符串解码。形如 `3[a2[c]]` 的嵌套结构适合栈。扫描字符串，数字累积成重复次数；遇

到左括号，把当前字符串和次数入栈，重置当前字符串；遇到右括号，弹出上一层，把当前字符串重复指定次数后接回上一层。暴力递归也自然，但迭代栈更显式。易错点是多位数字和嵌套层次。

496 下一个更大元素 I。对第二个数组使用单调递减栈，扫描时如果当前元素大于栈顶，就说明当前元素是栈顶元素的下一个更大值，弹出并记录映射。最后第一个数组按映射查询答案。暴力对每个元素向右找更大值是 $O(n^2)$ 。单调栈把每个元素最多压入弹出一次，复杂度 $O(n)$ 。

503 下一个更大元素 II。环形数组使右侧可以绕回开头。做法是扫描两遍下标 $0..2n-1$ ，真实位置用 $i \bmod n$ 。栈仍保存还没找到下一个更大值的下标。为了避免重复入栈，通常只在第一遍把下标入栈，第二遍只负责帮助第一遍元素找到答案。复杂度 $O(n)$ 。

581 最短无序连续子数组。排序后比较原数组和排序数组，找到左右第一个不同位置是一种清晰解法，复杂度 $O(n \log n)$ 。线性做法从左到右维护历史最大值，若当前值小于历史最大值，说明它必须进入待排序区间，更新右边界；从右到左维护历史最小值，若当前值大于历史最小值，更新左边界。核心是找出破坏全局有序性的最远位置。

621 任务调度器。相同任务之间需要冷却时间。可以用大顶堆模拟每个时间片可执行的最高频任务，也可以用最高频公式计算最短长度。公式的骨架由最高频任务决定：前 $\text{maxFreq} - 1$ 轮每轮至少长度 $n+1$ ，最后放所有最高频任务的尾部。答案是任务总数和骨架长度的较大值。若其他任务足够填满空隙，就不需要空闲。

704 二分查找。最基础但最容易暴露边界混乱。推荐固定一种区间语义，例如左闭右开 $[\text{left}, \text{right})$ ：初始 $\text{left}=0, \text{right}=n$ ，循环条件 $\text{left}<\text{right}$ ，当 $\text{nums}[\text{mid}] < \text{target}$ 时 $\text{left}=\text{mid}+1$ ，否则 $\text{right}=\text{mid}$ 。最后检查 left 是否越界和值是否等于目标。统一语义比背多个模板更可靠。

763 划分字母区间。先记录每个字符最后出现位置。扫描字符串时，当前片段右边界是已见字符最后位置的最大值。当扫描下标到达右边界，说明当前片段中所有字符都不会出现在后面，可以切分。贪心正确性来自最早合法切分：一旦能切就切，可以最大化片段数量。复杂度 $O(n)$ 。

853 车队。每辆车到终点所需时间是 $(\text{target} - \text{position}) / \text{speed}$ 。按位置从靠近终点到远离终点排序扫描。后车如果到达时间小于等于前方车队时间，会追上并成为同一车队；否则形成新车队。这个题的关键是从右往左看，因为前车决定后车能否被阻挡。不要按速度排序，速度本身不能决定车队。

973 最接近原点的 K 个点。距离比较可用平方距离，避免开方。方案一完整排序，复杂度 $O(n \log n)$ ；方案二维护大小为 K 的大顶堆，堆顶是当前 K 个候选里最远的点，新点更近则替换，复杂度 $O(n \log k)$ ；方案三快速选择平均 $O(n)$ 。如果 K 很小，堆很实用；如果要原地平均线性，快速选择更强但实现风险更高。

981 基于时间的键值存储。每个 key 对应按时间递增的列表。插入时追加 $(\text{timestamp}, \text{value})$ ；查询某个时间戳时，在该 key 的列表中二分找最后一个时间不大于查询时间的记录。暴力线性向前找会超时。这个题本质是“每个 key 内部的有序数组 + upper_bound”。易错点是不存在 key 或所有时间都大于查询时间时返回空。

1011 在 D 天内送达包裹的能力。船容量越大，需要天数越少，具有单调性。二分容量，下界是最大包裹重量，上界是总重量。检查函数按顺序装包，当前天放不下就开新一天，统计需要天数。若天数不超过 D，容量可行，尝试更小。它和分割数组最大值是同一个模型：最小化最大段和。

1046 最后一块石头的重量。每次取两块最重石头粉碎，适合大顶堆。暴力每轮排序复杂度 $O(n^2)$ ；堆能把每轮取最大和插入差值降到 $O(\log n)$ 。这个题不需要证明复杂贪心，因为规则本身指定每次取最重两块，堆只是高效实现规则。注意当两块相等时不需要放回 0。

1288 删除被覆盖区间。区间 $[a, b]$ 被 $[c, d]$ 覆盖，要求 $c \leq a$ 且 $b \leq d$ 。排序时按起点升序，起点相同按终点降序，这样大区间会在小区间前面。扫描维护当前最大右端点，若当前右端点不超过最大右端点，说明被覆盖；否则保留并更新最大右端点。起点相同按终点降序是关键，否则小区间先出现会误判。

148 排序链表。若要求 $O(n \log n)$ 时间和较低空间，链表归并排序是主线。快慢指针找中点并断开，再合并两个有序链表。递归版本清晰，但有栈深度；迭代自底向上版本更工程化。不要把值复制到数组排序再写回视为完全等价，因为真实节点可能携带更多数据或被外部引用。

152 乘积最大子数组。和最大子数组不同，负数会让最大值和最小值互换。扫描时同时维护以当前位置结尾的最大乘积和最小乘积；遇到负数时，旧最小乘以负数可能变成新最大。若当前元素单独开始更好，也要允许重新开始。复杂度 $O(n)$ 。易错点是 0 会截断乘积，状态转移自然处理它。

221 最大正方形。二维 DP。状态 `dp[i][j]` 表示以当前位置为右下角的最大正方形边长。如果当前位置是 1，那么它取决于上方、左方、左上方三个状态的最小值加一；如果当前位置是 0，则为 0。为什么取最小？因为一个更大的正方形需要三个方向都能支持。答案是最大边长平方。复杂度 $O(mn)$ 。

279 完全平方数。求和为 n 的最少完全平方数个数。可以看作完全背包最少物品数：物品是 $1, 4, 9, \dots$ ，容量是 n ，每种平方数可无限使用。也可以看成图最短路：每个数是节点，从 x 到 $x + \text{square}$ 有一条边，BFS 到 n 。DP 写法更直接，初始化无穷，`dp[0]=0`，对每个金额尝试平方数。复杂度约 $O(n\sqrt{n})$ 。

337 打家劫舍 III。树形 DP。每个节点有两种状态：偷当前节点时，左右孩子不能偷；不偷当前节点时，左右孩子可偷可不偷，取各自最大。若用递归，函数返回二元状态最清晰。工程上深树可能有递归风险，可用显式栈后序遍历实现。这个题的关键是不要只按层奇偶偷，因为树结构不一定平衡，局部最优需要节点状态。

416、494、518 背包三连复盘。416 是 0-1 背包可达性，容量倒序；494 是 0-1 背包方案数，容量也倒序，并且 0 会让方案翻倍；518 是完全背包组合数，硬币外层、金额正序。三题放在一起看，最重要的不是公式，而是“物品能用几次”和“统计组合还是排列”。循环顺序就是由这两个问题决定的。

股票题复盘。121 只允许一次交易，维护历史最低价；122 允许无限次交易，可以累加所有正收益差，也可以写持有和不持有状态；123 至多两次交易，需要交易次数维度；188 至多 k 次交易，是 123 的泛化；309 有冷冻期，买入来源受限制；714 有手续费，卖出或买入时扣费。统一方法是状态机：每天结束时持有什么状态，动作如何改变状态。

单调栈复盘。每日温度、下一个更大元素、柱状图最大矩形、接雨水都能用单调栈，但栈含义不同。每日温度和下一个更大元素中，栈保存等待被更大值解决的下标；柱状图中，栈保存递增高度，遇到更矮柱子时结算被弹出柱子的最大宽度；接雨水中，弹出的是低洼底部，左右边界由新栈顶和当前柱子提供。不要只说“用单调栈”，要说弹出时解决了什么问题。

单调队列复盘。滑动窗口最大值和受限子序列和都维护窗口内最优候选。队列保存下标，前者按数组值递减，后者按 DP 值递减。队首过期就弹出，队尾被新候选支配就弹出。每个下标最多进出一次，所以线性复杂度。单调队列适合“转移只需要最近 k 个状态的最大或最小值”的问题。

并查集复盘。并查集只回答连通性，不回答路径长度。冗余连接用合并失败找环；省份数量统计根节点；账户合并把邮箱作为节点；最小体力路径可以二分阈值后用并查集检查连通。并查集的两个优化是路径压缩和按大小或按秩合并。若题目需要维护比例、距离或奇偶性，就要扩展为带权并查集，不能直接套普通版本。

Dijkstra 复盘。使用条件是边权非负，目标是最短路径。堆中保存 `(distance, node)`，因为 C++ 的 `priority_queue` 不支持 `decrease-key`，所以更新距离时压入新状态，弹出时跳过过期状态。网络延迟时间是普通最短路；最小体力路径把路径代价定义为最大边权，也满足单调扩展；若有负边，应该考虑 Bellman-Ford 或重新建模。

二分答案复盘。二分答案不是看数组是否有序，而是看答案是否具有单调可行性。爱吃香蕉、分割数组最大值、船运包裹、最小体力路径都属于这一类。套路是：定义答案范围，写检查函数，证明若某个答案可行，则更宽松答案也可行。最常见错误是检查函数里使用了不满足单调性的贪心，导致二分外壳正确但内部判断错误。

C++ 容器选择复盘。需要随机访问和连续内存，优先 `vector`；需要两端进出和窗口下标，考虑 `deque`；需要按 key 平均常数查询，使用 `unordered_map`，并注意 `reserve`；需要有序遍历、前驱后继、区间边界，使用 `map` 或 `set`；需要不断取最大最小候选，使用 `priority_queue`；需要节点稳定和常数移动，少数场景使用 `list`。容器不是背名字，而是匹配访问模式。

源码阅读复盘。看标准库源码时先问接口语义，再看核心数据成员，最后跟关键操作路径。`vector` 看大小、容量、扩容和迭代器失效；`deque` 看分段数组；`unordered_map` 看桶、负载因子和 `rehash`；`map` 看树结构和有序迭代；`priority_queue` 看底层容器和堆算法。刷题时能把 API 成本说清楚，代码选择就更可信。

Linux 源码阅读关联。Linux 内核常用侵入式链表、红黑树、基数树和 XArray，这些结构和 STL 容器的最大差异是所有权模型。STL 容器通常拥有元素或管理元素存储，内核结构常把链接节点嵌入业务对象中，数据结构只负责把已有对象挂到索引里。刷题不需要写侵入式结构，但理解这种设计能帮助你解释 LRU、调度队列、内存页缓存这类系统问题：结构服务于访问模式、生命周期

和并发约束。

面试表达复盘。一道题的完整回答应包含六句话：第一，暴力方法是什么；第二，暴力慢在哪里；第三，优化利用了哪个性质；第四，数据结构里保存什么不变量；第五，复杂度是多少；第六，边界用例有哪些。代码只是其中一部分。如果只能写代码但讲不清为什么，换一个题面就会失效。

如果一道题三次都需要看题解，不要继续刷新题，先回到它所属章节，重新写暴力到优化推理链。

进阶综合题卡

4 寻找两个正序数组的中位数。暴力可以合并两个数组再取中位数，复杂度 $O(m+n)$ 。优化做法是二分较短数组的切分位置，使左半部分元素数量等于右半部分，且左半最大值不大于右半最小值。难点在边界哨兵和奇偶长度。这个题训练的是二分在答案结构上的应用，不是普通下标查找。

10 正则表达式匹配。暴力递归尝试星号匹配零次或多次，会重复处理同一对下标。DP 状态 $dp[i][j]$ 表示字符串前 i 个字符能否匹配模式前 j 个字符。星号转移最容易错：它修饰前一个字符，可以表示出现零次，也可以在当前字符匹配时消耗一个字符。边界初始化要处理 a 星号 b 星号这种能匹配空串的模式。

23 合并 K 个升序链表。暴力每轮扫描 K 个链表头，总代价 $O(NK)$ 。小顶堆保存每条链表当前头节点，每次弹出最小节点并压入它的后继，复杂度 $O(N \log K)$ 。分治两两合并也是 $O(N \log K)$ ，常数和空间不同。面试中应说明堆里最多 K 个节点，且复用节点时要处理尾指针。

25 K 个一组翻转链表。关键不是反转，而是找到每组边界。每轮从上一组尾部向后找第 K 个节点，不足则停止；保存下一组头，反转当前组，再把上一组尾接到新头、旧头接到下一组。必须先保存 $next$ ，否则反转中会丢失后续链。虚拟头节点可以统一处理第一组。

32 最长有效括号。栈保存尚未匹配的左括号下标和最后一个非法右括号位置。遇到右括号匹配成功后，当前有效长度由当前下标减去栈顶或非法边界得到。DP 也可做，状态表示以当前位置结尾的最长有效长度。这个题训练的是括号结构和连续区间长度的结合。

41 缺失的第一个正数。排序是 $O(n \log n)$ ，哈希是 $O(n)$ 空间。原地做法利用数组本身作为哈希表，把值 x 放到下标 $x-1$ 的位置。扫描时不断交换到正确位置，最后第一个 $nums[i]$ 不等于 $i+1$ 的位置就是答案。边界是只处理一到 n 的正数，重复值不能无限交换。

44 通配符匹配。问号匹配单个字符，星号匹配任意长度。DP 状态与正则匹配类似，但星号语义不同：它可以匹配空串，也可以消耗当前字符后继续保持星号。贪心双指针也能做，记录最近星号位置和当时字符串位置，失配时让星号多吞一个字符。两种解法都要讲清星号回退机制。

51 N 皇后。回溯逐行放皇后，列、主对角线、副对角线三个集合判断冲突。暴力在棋盘每格尝试会产生大量无效状态；逐行保证每行一个皇后，把选择空间降为每行选列。对角线可用 $row-col$ 和 $row+col$ 编码。剪枝正确性来自皇后攻击规则完全由这三个集合覆盖。

72 编辑距离。这个题必须能从操作反推转移：删除 $word1$ 末尾，对应 $dp[i-1][j]$ 加一；插入 $word2$ 末尾，对应 $dp[i][j-1]$ 加一；替换末尾，对应 $dp[i-1][j-1]$ 加一。字符相同则不需要操作。空串边界是连续插入或删除。它是二维 DP 状态定义最经典的训练题。

85 最大矩形。把每一行当作柱状图底部，连续一的高度逐行累积，然后对每行运行柱状图最大矩形单调栈算法。暴力枚举矩形会很慢，优化来自把二维问题转成多次一维单调栈。边界是字符零会把高度清零，哨兵零帮助结算所有柱子。

97 交错字符串。状态 $dp[i][j]$ 表示 $s1$ 前 i 个字符和 $s2$ 前 j 个字符能否交错组成 $s3$ 前 $i+j$ 个字符。转移来自最后一个字符取自 $s1$ 或 $s2$ 。暴力递归会在相同 i, j 上重复。边界先判断长度和，再初始化第一行第一列。这个题训练两个来源序列的前缀状态。

115 不同的子序列。状态 $dp[i][j]$ 表示 s 前 i 个字符中有多少个子序列等于 t 前 j 个字符。若当前字符相等，可以用它匹配 t 的末尾，也可以不用它；若不等，只能不用它。计数可能很大，工程上要考虑 long long。它和 LCS 都是双字符串 DP，但一个求长度，一个求数量。

126 单词接龙 II。相比单词接龙 I，不仅要最短长度，还要输出所有最短路径。做法是 BFS 建立最短层级和前驱关系，不在 BFS 中立即深搜所有路径；BFS 完成后从终点按前驱回溯。关键是同一层的多个前驱都要保留，不能因为某个单词已经被本层访问就丢掉另一条同长路径。

135 分发糖果。每个孩子至少一颗，评分更高者比相邻低评分者多。左到右保证比左邻高的约束，右到左保证比右邻高的约束，最终取两次约束最大值。暴力反复调整会来回传播。这个题体现局部相邻约束可以分两次单向扫描合并。

140 单词拆分 II。先用 DP 判断后缀是否可拆，避免在无解分支上回溯；再 DFS 枚举句子。若直接暴力切分，会在大量不可达后缀上浪费时间。输出规模可能指数级，所以复杂度必须包含输出。实现时要注意路径拼接成本，可以保存单词列表最后 join。

188 买卖股票 IV。状态包含交易次数和持有状态。若 k 大于等于 $n/2$ ，退化为无限次交易；否则维护 buy[t] 和 sell[t]。更新顺序要避免同一天状态污染。这个题说明状态维度来自约束，交易次数不是后处理能补上的信息。

212 单词搜索 II。对每个单词单独 DFS 会重复遍历棋盘。Trie 把所有单词前缀合并，网格 DFS 时沿 Trie 前进，不存在前缀立即剪枝。找到单词后可标记避免重复加入答案，也可删除叶子做进一步剪枝。边界是访问标记回退和 Trie 节点生命周期。

224 基本计算器。表达式有括号和加减号。可以用栈保存进入括号前的结果和符号，也可以写递归下降解析。扫描时数字要累积多位，遇到符号结算前一个数字。这个题不是数学题，而是状态机和栈处理嵌套上下文。

227 基本计算器 II。乘除优先级高于加减。常见做法是扫描数字和运算符，遇到乘除立即和栈顶结合，遇到加减把带符号数字入栈，最后求和。除法要按题目要求向零截断。它训练的是延迟低优先级、立即处理高优先级。

295 数据流中位数。双堆的顺序不变量和大小不变量缺一不可。大顶堆所有元素不大于小顶堆所有元素，两个堆大小差不超过一。插入时先按堆顶分配，再重平衡。若只维护大小，不维护左右顺序，中位数会错。若要支持删除，需要懒删除机制。

312 戳气球。若按第一个戳的气球思考，左右邻居会随操作变化，状态不稳定。改成最后戳某个气球，左右边界在最后一刻固定，区间就能独立。dp[left][right] 表示开区间内气球全部戳完的最大收益，枚举最后戳的 mid。这个题是区间 DP 状态设计的代表。

329 矩阵中的最长递增路径。从每个格子 DFS 找递增路径会重复计算后续格子。记忆化搜索把每个格子的最长路径保存下来；也可以把格子按值排序做 DAG DP。由于路径必须严格递增，状态图无环。若允许不下降，就可能出现环，需要重新处理。

394 字符串解码。栈保存上一层字符串和重复次数。遇到左括号入栈，遇到右括号弹出并展开。多位数字、嵌套括号和当前字符串重置是主要边界。这个题适合训练解析类问题的上下文保存，而不是简单字符串拼接。

460 LFU 缓存。LFU 不只要按频次淘汰，还要在相同频次下按最久未使用淘汰。常见结构是 key 到节点信息的哈希表、频次到双向链表的哈希表、当前最小频次。访问时节点频次加一，从旧频次链表移到新频次链表。它比 LRU 多了频次维度，能训练多个索引结构协同维护。

560 和为 K 的子数组。它在很多变体中出现：若问数量，哈希表保存前缀和频次；若问最长，保存最早下标；若问最短且有负数和至少 K，用前缀和加单调队列；若元素全正，可以滑动窗口。复盘时要根据目标选择历史状态，不能只记一个模板。

632 最小区间覆盖 K 个列表。每个列表当前取一个元素，区间由当前最小和最大决定。小顶堆保存各列表当前元素，同时维护当前最大值；弹出最小元素并推进它所在列表。若某个列表耗尽，就无法再覆盖所有列表。这个题是多路归并和滑动覆盖思想的结合。

678 有效的括号字符串。星号可以当左括号、右括号或空。贪心维护可能未匹配左括号数量的最小值和最大值区间。遇到左括号区间加一，右括号区间减一，星号让下界减一、上界加一，并把下界截到零。若上界为负则不可能。这个题展示状态集合可被区间压缩。

721 账户合并。邮箱是连接账户的证据。可以把邮箱映射到编号并查集合并，也可以把账户编号合并。最后按根收集邮箱并排序。姓名通常只用于输出，不能作为合并依据。这个题训练字符串哈希、并查集和输出排序的组合。

815 公交路线。站点很多，路线也很多。若把站点作为 BFS 节点，每次找经过该站的路线；更高效的建模是路线作为节点，两个路线若共享站点则可换乘。实际实现常用站点到路线列表的哈希表，从起点站可乘路线开始 BFS。访问路线比访问站点更能控制重复。

862 和至少为 K 的最短子数组。数组含负数，普通滑动窗口失效。前缀和数组上维护单调递增队列：队首若和当前前缀差至少 K，就更新答案并弹出；队尾若前缀和不小于当前前缀，就被当前下标支配。这个题是理解单调队列支配关系的关键。

895 最大频率栈。需要弹出频率最高且最近入栈的元素。用 value 到 frequency 的哈希表记录频次，用 frequency 到栈的哈希表保存达到该频次的元素顺序，再维护当前最大频次。push 时频次加一并入对应栈，pop 时从最大频次栈弹出。它是哈希表和栈按维度组合的例子。

981 基于时间的键值存储。每个 key 对应按时间递增的值列表，查询某个时间戳不超过目标的最新值。哈希表定位 key，列表中二分 upper_bound 找第一个大于目标的位置再退一。它训练按 key 分组后在组内二分，不要把所有记录放进一个全局结构里。

1091 二进制矩阵中的最短路径。八方向无权网格最短路，BFS 即可。起点或终点为阻塞直接返回 -1。访问标记在入队时设置，距离可以放在队列状态里，也可以直接把矩阵改成距离。它和岛屿题的区别是要求最短路径，所以 DFS 不合适。

1235 规划兼职工作。按结束时间排序，每个工作选或不选。选择当前工作时，需要找到上一个结束时间不超过当前开始时间的工作，用二分定位。dp[i] 表示考虑前 i 个工作最大收益。它是排序、二分和 DP 的组合题，常见错误是只按收益贪心。

1248 统计优美子数组。恰好 K 个奇数可以转成前缀奇数计数差为 K。哈希表保存某个奇数计数出现次数；扫描到当前计数 c 时，历史 c-K 的次数就是新增答案。也可以用至多 K 减至多 K-1 的滑动窗口。两个解法对应前缀计数和窗口计数两种思维。

1438 绝对差不超过限制的最长连续子数组。窗口合法性取决于最大值和最小值差。用两个单调队列分别维护窗口最大和最小，右端扩展后若差超过限制就移动左端并清理过期下标。这个题说明窗口状态不一定是和或频次，也可以是动态极值。

1462 课程表 IV。如果查询很多，不能每次为一对课程单独搜索。可以用 Floyd 传递闭包、每个点 BFS/DFS 可达集合，或者拓扑 DP 合并先修集合。选择取决于课程数和查询数。它训练预处理和多查询之间的权衡。

1514 概率最大的路径。路径权重是概率乘积，目标是最大乘积。可以把 Dijkstra 的距离改成最大概率，用大顶堆每次扩展当前概率最高节点；概率乘以边概率不会变大，满足单调性。也可以取负对数变成最短路。这个题说明最短路框架可以迁移到其他单调代价。

1675 数组的最小偏移量。每个奇数可以先乘二变偶数，之后所有偶数都可以不断除二。把所有数变成可减少的最大形态，用大顶堆维护当前最大值，同时维护当前最小值；每次尝试降低最大偶数并更新答案。若最大值变奇数就无法继续。这个题是堆和状态空间规范化的结合。

本节复盘

阅读本节后，请回到相邻章节的例题，把暴力瓶颈、优化依据、状态不变量、C++ 容器成本和边界测试重新写一遍。能把同一思想迁移到变体题，才算真正掌握。

二刷深度复盘卡

这一组卡片用于第二遍和第三遍复习。第一遍题卡帮助你建立解法，二刷卡片要求你把证明、追问、调试和工程成本讲清楚。每道题都要准备一个反例、一个边界用例和一个能说明优化价值的规模用例。

1 两数之和。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：当前元素只需要寻找唯一补数，历史值到下标的映射就是最小状态。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：先查再插入，避免同一个下标被用两次；若数组有序才考虑双指针。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：重复数字、目标为偶数且补数等于当前值、没有答案时返回空结果。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避开空话。正确性来自代数等价。若两个前缀满足差值、同余或计数差要求，则它们之间的区间正好满足题意；反过来任意合法区间也对应这样一对前缀。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。实现时用 find 判断是否存在，用 operator[] 只在确认要插入或累加时使用。总和可能溢出时使用 long long；模运算要处理负数余数归一化。此外要说

明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。若改成返回所有不重复数对，要先排序或用频次表处理重复。追问通常会问先查询还是先更新、哈希表保存次数还是最早下标、为什么负数不影响前缀和。请准备说明从任意合法区间如何反推到两个前缀状态。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

2 两数相加。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：链表按低位到高位保存数字，本质是竖式加法而不是整数转换。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：维护两个游标和进位，任一链未结束或进位非零都继续生成节点。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：两链长度不同、最后进位、输入为零、不能用内置整数承载超长数字。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。正确性靠结构不变量：已经反转或合并的部分保持合法链，未处理部分仍可达，二者之间只有一个明确连接点。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。节点通常由评测平台管理，代码不要随意 `delete` 输入节点。若创建新节点，要说明所有权。比较交点时比较地址，不比较值。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。若链表高位在前，可以用栈反向处理或先反转链表。追问通常会让你画指针。请准备说明上一段尾、当前段头、当前节点、后继节点分别是谁，以及哪一步保存 `next` 防止断链。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

3 无重复字符的最长子串。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：窗口保存当前无重复子串，右端每次加入一个字符。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：用上次出现位置直接跳过无效左端，左边界只能向右不能回退。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：空串、全相同字符、重复字符出现在窗口左侧之外、扩展字符集。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。证明重点是排除。某个左端被移过之后，所有以它为左端的更长窗口都不可能比当前答案更优，或者已经被当前状态完整统计。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。C++ 中建议把窗口写成左闭右开或左闭右闭中的一种，并在全题坚持。字符计数用 `std::array<int, 128>` 或 26 长度数组时要说明字符集假设；泛化字符集再换哈希表。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。若要求恰好包含 k 种字符，窗口条件从无重复变成种类计数。追问通

常会问窗口为什么不会漏掉左端、为什么有负数会失效、固定窗口和可变窗口有什么区别。请准备一个反例证明错误窗口条件会漏解。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

5 最长回文子串。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：回文围绕中心对称，中心可以是字符也可以是两个字符之间。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：中心扩展避免枚举子串后再逐个检查；每次扩展只比较新增左右边界。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：偶数长度回文、单字符、全相同字符、多个同长答案。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。正确性通常用区间不变量证明。每次循环只扩大已经正确的区域，未知区严格缩小，且被移出的元素已经放到符合语义的位置。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。C++ 实现优先使用 `std::vector<int>` 和明确的整型下标。涉及乘积、面积、总和时要主动判断是否需要 `long long`。如果题目要求原地修改，要说明返回值表示新长度还是数组本身，避免把输出语义和容器容量混在一起。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。若要求回文子序列，应转成区间 DP，不能用中心扩展。追问通常会落在原地修改、稳定顺序、输出规模和整数溢出上。请准备说明为什么保存下标比保存指针更稳，为什么某个位置一旦写入就不会再被未来元素破坏。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

11 盛最多水的容器。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：左右指针代表当前选取的两条边，面积由短板和宽度共同决定。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：每次移动短板，因为移动长板只会缩小宽度且不会提高短板。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：两端高度相等、数组长度为二、面积乘法可能溢出。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。证明重点是排除。某个左端被移过之后，所有以它为左端的更长窗口都不可能比当前答案更优，或者已经被当前状态完整统计。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。C++ 中建议把窗口写成左闭右开或左闭右闭中的一种，并在全题坚持。字符计数用 `std::array<int, 128>` 或 26 长度数组时要说明字符集假设；泛化字符集再换哈希表。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。接雨水计算每个位置贡献，不能把本题双指针证明直接搬过去。追问通常会问窗口为什么不会漏掉左端、为什么有负数会失效、固定窗口和可变窗口有什么区别。请准备一个反例证明错误窗口条件会漏解。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

15 三数之和。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：排序后固定第一个数，剩余两数转成有序双指针。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：和太小移动左指针，和太大移动右指针；固定值和左右值都要跳过重复。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：全零、重复元素很多、没有答案、结果顺序不要求但组合不能重复。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。证明重点是排除。某个左端被移过之后，所有以它为左端的更长窗口都不可能比当前答案更优，或者已经被当前状态完整统计。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。C++ 中建议把窗口写成左闭右开或左闭右闭中的一种，并在全题坚持。字符计数用 `std::array<int, 128>` 或 26 长度数组时要说明字符集假设；泛化字符集再换哈希表。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。四数之和再外层固定一个数，并用 long long 防止目标和溢出。追问通常会问窗口为什么不会漏掉左端、为什么有负数会失效、固定窗口和可变窗口有什么区别。请准备一个反例证明错误窗口条件会漏解。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

16 最接近的三数之和。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：排序后固定一数，双指针寻找最接近目标的两数和。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：每次根据当前和与目标的大小移动指针，同时更新最小绝对差。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：差值相等、负数、目标很大、三数和溢出。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。证明重点是排除。某个左端被移过之后，所有以它为左端的更长窗口都不可能比当前答案更优，或者已经被当前状态完整统计。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。C++ 中建议把窗口写成左闭右开或左闭右闭中的一种，并在全题坚持。字符计数用 `std::array<int, 128>` 或 26 长度数组时要说明字符集假设；泛化字符集再换哈希表。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。若要求返回所有最接近组合，需要记录同差值并去重。追问通常会问窗口为什么不会漏掉左端、为什么有负数会失效、固定窗口和可变窗口有什么区别。请准备一个反例证明错误窗口条件会漏解。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

18 四数之和。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：两层固定加一层双指针，关键是剪枝和去重。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：排序提供单调性，外层可用最小可能和和最大可能和提前跳过。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：重复元素、目标超出 `int`、答案很多导致输出空间主导复杂度。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。证明重点是排除。某个左端被移过之后，所有以它为左端的更长窗口都不可能比当前答案更优，或者已经被当前状态完整统计。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。C++ 中建议把窗口写成左闭右开或左闭右闭中的一种，并在全题坚持。字符计数用 `std::array<int, 128>` 或 26 长度数组时要说明字符集假设；泛化字符集再换哈希表。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。k 数之和可以递归降维，但工程实现要控制重复和边界。追问通常会问窗口为什么不会漏掉左端、为什么有负数会失效、固定窗口和可变窗口有什么区别。请准备一个反例证明错误窗口条件会漏解。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

26 删除有序数组重复项。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：有序数组让重复元素相邻，慢指针表示下一个唯一值写入位置。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：快指针扫描，只有发现新值才写入慢指针并推进。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：空数组、全重复、无重复、返回长度不等于物理容量。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。正确性通常用区间不变量证明。每次循环只扩大已经正确的区域，未知区严格缩小，且被移出的元素已经放到符合语义的位置。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。C++ 实现优先使用 `std::vector<int>` 和明确的整型下标。涉及乘积、面积、总和时要主动判断是否需要 `long long`。如果题目要求原地修改，要说明返回值表示新长度还是数组本身，避免把输出语义和容器容量混在一起。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。若允许每个元素最多保留两次，只需改变写入条件为和前两个比较。追问通常会落在原地修改、稳定顺序、输出规模和整数溢出上。请准备说明为什么保存下标比保存指针更稳，为什么某个位置一旦写入就不会再被未来元素破坏。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

27 移除元素。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：原地过滤，慢指针左侧始终是不等于目标值的稳定前缀。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：保持顺序用快慢指针；不保持顺序可用左右交换减少写入。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录

前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：所有元素都被移除、没有元素被移除、目标值在尾部密集。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。正确性通常用区间不变量证明。每次循环只扩大已经正确的区域，未知区严格缩小，且被移出的元素已经放到符合语义的位置。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。C++ 实现优先使用 `std::vector<int>` 和明确的整型下标。涉及乘积、面积、总和时要主动判断是否需要 `long long`。如果题目要求原地修改，要说明返回值表示新长度还是数组本身，避免把输出语义和容器容量混在一起。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。工程里要先确认是否要求稳定顺序，不同要求对应不同写法。追问通常会落在原地修改、稳定顺序、输出规模和整数溢出上。请准备说明为什么保存下标比保存指针更稳，为什么某个位置一旦写入就不会再被未来元素破坏。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

31 下一个排列。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：字典序结构由最长非递增后缀决定，枢轴是后缀前一个位置。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：从右找第一个上升点，再从右找刚好更大的元素交换，最后反转后缀。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：完全降序、重复数字、只有一个元素、交换后后缀必须最小化。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。正确性通常用区间不变量证明。每次循环只扩大已经正确的区域，未知区严格缩小，且被移出的元素已经放到符合语义的位置。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。C++ 实现优先使用 `std::vector<int>` 和明确的整型下标。涉及乘积、面积、总和时要主动判断是否需要 `long long`。如果题目要求原地修改，要说明返回值表示新长度还是数组本身，避免把输出语义和容器容量混在一起。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。若要求上一个排列，比较方向和后缀处理对称变化。追问通常会落在原地修改、稳定顺序、输出规模和整数溢出上。请准备说明为什么保存下标比保存指针更稳，为什么某个位置一旦写入就不会再被未来元素破坏。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

33 搜索旋转排序数组。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：旋转数组每次二分至少有一半保持有序。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：判断哪一半有序，再看目标是否落在有序半边范围内。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：未旋转、长度一、目标不存在、边界等号、存在重复元素时退化。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。证明二分不是证明循环会停，而是证明被排除的一半确实不含答案。答案空间二分还要证明可行性谓词单调。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。中点写成 $\text{left} + (\text{right} - \text{left}) / 2$ 。判断平方、乘法、总量时避免溢出。用 `std::lower_bound` 能表达清楚时优先用它，但面试要能手写同等语义。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。若重复元素很多，需要在无法判断有序半边时收缩边界。追问通常会要求你写出单调谓词和答案边界。请准备说明 left、right 的语义，为什么 mid 被排除或保留，以及返回值是否还需要二次验证。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

34 排序数组中查找首尾位置。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：首尾位置可以拆成两个边界：第一个大于等于和第一个大于。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：不要找到一个目标后向两边线性扩展，否则重复元素很多时退化。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：目标不存在、目标在开头或结尾、数组为空、全是目标。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。证明二分不是证明循环会停，而是证明被排除的一半确实不含答案。答案空间二分还要证明可行性谓词单调。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。中点写成 $\text{left} + (\text{right} - \text{left}) / 2$ 。判断平方、乘法、总量时避免溢出。用 `std::lower_bound` 能表达清楚时优先用它，但面试要能手写同等语义。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。这个模型能迁移到计数某值出现次数和插入区间边界。追问通常会要求你写出单调谓词和答案边界。请准备说明 left、right 的语义，为什么 mid 被排除或保留，以及返回值是否还需要二次验证。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

35 搜索插入位置。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：题目等价于寻找第一个大于等于目标的位置。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：统一用 lower bound 语义，不在相等时提前返回也能保持边界一致。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：目标小于所有元素、目标大于所有元素、重复值、空数组。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。证明二分不是证明循环会停，而是证明被排除的一半确实不含答案。答案空间二分还要证明可行性谓词单调。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。中点写成 $\text{left} + (\text{right} - \text{left}) / 2$ 。判断平方、乘法、总量时避免溢出。用 `std::lower_bound` 能表达清楚时优先用它，但面试要能手写同等语义。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。手写二分时用左闭右开可直接返回 `left`。追问通常会要求你写出单调谓词和答案边界。请准备说明 `left`、`right` 的语义，为什么 `mid` 被排除或保留，以及返回值是否还需要二次验证。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

42 接雨水。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：每个位置的水由左侧最高和右侧最高的较小者决定。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：前后缀、双指针、单调栈都是不同状态维护方式；要能说明各自结算对象。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：单调递增、单调递减、平台高度、多个凹槽、数组长度不足三。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。每个元素最多入栈一次、出栈一次，所以时间线性。正确性来自弹出时机：一旦遇到破坏单调性的元素，栈顶元素的答案已经确定。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。栈里通常存下标而不是值，因为要计算距离、宽度和过期。可以用 `std::vector<int>` 当栈，便于查看顶部和减少适配器限制。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。若柱子变成二维网格，需要最小堆从边界向内扩展。追问通常会问栈里为什么存下标、相等元素如何处理、弹出时到底结算了谁。请准备用一个严格递增和一个严格递减样例解释入栈出栈次数。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

49 字母异位词分组。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：异位词共享同一个规范表示，可以是排序字符串或字符计数。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：哈希表按规范键分桶，避免两两比较字符串。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：空字符串、重复单词、字符集不止小写字母、计数键必须无歧义。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。正确性来自代数等价。若两个前缀满足差值、同余或计数差要求，则它们之间的区间正好满足题意；反过来任意合法区间也对应这样一对前缀。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。实现时用 `find` 判断是否存在，用 `operator[]` 只在确认要插入或累加时使用。总和可能溢出时使用 `long long`；模运算要处理负数余数归一化。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。若字符串很长且字符集固定，计数键通常比排序键更稳定。追问通常会问先查询还是先更新、哈希表保存次数还是最早下标、为什么负数不影响前缀和。请准备说明从任意合法区间如何反推到两个前缀状态。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

53 最大子数组和。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：状态是必须以当前位置结尾的最大连续和。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：若前一个后缀为负，接上它只会拖累当前元素，因此可以重新开始。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：全负数组、单元素、和溢出、答案不能初始化为零。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。正确性通常从最优子结构证明：任意最优解的最后一步必然落入枚举集合；每个枚举候选又由已经正确的子问题构成。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。数组维度和初始化要服务于状态含义。不可达用大正数或负数哨兵时要避免溢出。方案数可能超过 int 时改用 `long long` 或按题目取模。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。若要求返回区间下标，同步记录重新开始位置和最优左右端。追问通常会问状态是否足够、初始化为什么这样、空间压缩读到的是旧值还是新值。请准备从暴力搜索树指出重复子问题。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

54 螺旋矩阵。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：四个边界收缩模拟一圈一圈访问。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：每走完一条边就收缩对应边界，并在走反方向前检查是否仍合法。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：单行、单列、空矩阵、最后一圈只剩一个元素。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。正确性通常用区间不变量证明。每次循环只扩大已经正确的区域，未知区严格缩小，且被移出的元素已经放到符合语义的位置。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。C++ 实现优先使用 `std::vector<int>` 和明确的整型下标。涉及乘积、面积、总和时要主动判断是否需要 `long long`。如果题目要求原地修改，要说明返回值表示新长度还是数组本身，避免把输出语义和容器容量混在一起。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。若改成生成矩阵，边界逻辑相同，只是读变成写。追问通常会落在原地修改、稳定顺序、输出规模和整数溢出上。请准备说明为什么保存下标比保存指针更稳，为什么某个位置一旦写入就不会再被未来元素破坏。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

55 跳跃游戏。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：扫描过程中只关心当前能到达的最远位置。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：如果下标超过最远可达，任何之前路径都不能到达它；否则用它扩展边界。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的

部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：第一个元素为零、数组长度一、恰好到达最后、远超最后。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。常见证明有交换论证和领先性论证。交换论证说明最优解可以被替换成贪心第一步而不变差；领先性论证说明贪心每一步都至少和任何方案一样好。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。实现时先把比较器写成明确的业务顺序。区间题注意起点相同时终点升序还是降序；覆盖题注意闭区间和半开区间的等号。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。求最少跳数时把可达区间看成 BFS 层。追问通常会要求证明。请准备交换论证或领先性论证，并给出一个看似合理但错误的贪心规则反例。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

56 合并区间。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：排序后可能与当前区间相交的只会是结果尾部。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：按起点排序，扫描时维护结果最后区间的右端点。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：端点相接、完全覆盖、无重叠、空列表、输入未排序。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。排序保证所有可能与当前区间冲突的候选已经聚集在附近。贪心按最早结束选择，可以用交换论证证明不减少可选数量。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。比较器必须处理相同起点，否则覆盖和去重会错。区间端点可能溢出时用更大整数。闭区间和半开区间决定是否用小于等于。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。若区间是半开区间，端点相等不一定合并。追问通常会问排序键为什么这样、端点相等算不算重叠、比较器是否满足严格弱序。请准备说明排序后哪些候选可以被安全丢弃。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

57 插入区间。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：新区间只会影响与它相交的一段连续区间。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：先追加左侧完全不相交区间，再合并重叠段，最后追加右侧。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：新区间在最前、最后、覆盖全部、刚好相邻、原列表为空。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。排序保证所有可能与当前区间冲突的候选已经聚集在附近。贪心按最早结束选择，可以用交换论证证明不减少可选数量。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可

能重复进入。

C++ 追问重点是实现边界和成本。比较器必须处理相同起点，否则覆盖和去重会错。区间端点可能溢出时用更大整数。闭区间和半开区间决定是否用小于等于。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。若输入不是有序且不重叠，必须先排序或换模型。追问通常会问排序键为什么这样、端点相等算不算重叠、比较器是否满足严格弱序。请准备说明排序后哪些候选可以被安全丢弃。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

62 不同路径。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：网格状态由上方和左方两种最后一步转移而来。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：二维表可压缩为一维，因为当前行只依赖上一行同列和当前行左侧。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：一行、一列、较大网格结果溢出、障碍物版本边界。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。正确性通常从最优子结构证明：任意最优解的最后一步必然落入枚举集合；每个枚举候选又由已经正确的子问题构成。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。数组维度和初始化要服务于状态含义。不可达用大正数或负数哨兵时要避免溢出。方案数可能超过 int 时改用 `long long` 或按题目取模。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。带障碍物时遇到障碍状态清零，第一行第一列要按阻断处理。追问通常会问状态是否足够、初始化为什么这样、空间压缩读到的是旧值还是新值。请准备从暴力搜索树指出重复子问题。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

63 不同路径 II。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：障碍物让某些格子的路径数变为零。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：滚动数组中遇到障碍直接把当前位置清零，否则累加左侧。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：起点或终点是障碍、整行被截断、单行障碍。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。正确性通常从最优子结构证明：任意最优解的最后一步必然落入枚举集合；每个枚举候选又由已经正确的子问题构成。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。数组维度和初始化要服务于状态含义。不可达用大正数或负数哨兵时要避免溢出。方案数可能超过 int 时改用 `long long` 或按题目取模。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。若允许向上或向左走，图会有环，普通填表不再成立。追问通常会问状态是否足够、初始化为为什么这样、空间压缩读到的是旧值还是新值。请准备从暴力搜索树指出重

复子问题。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

64 最小路径和。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：状态是到达当前格子的最小代价，最后一步来自上方或左方。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：先初始化第一行和第一列，再处理内部，主转移保持简单。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：单行、单列、权值为零、路径和溢出。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。正确性通常从最优子结构证明：任意最优解的最后一步必然落入枚举集合；每个枚举候选又由已经正确的子问题构成。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。数组维度和初始化要服务于状态含义。不可达用大正数或负数哨兵时要避免溢出。方案数可能超过 int 时改用 `long long` 或按题目取模。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。若允许四方向移动，变成最短路而不是普通网格 DP。追问通常会问状态是否足够、初始化为什么这样、空间压缩读到的是旧值还是新值。请准备从暴力搜索树指出重复子问题。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

70 爬楼梯。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：最后一步来自前一阶或前两阶，是最小的重叠子问题模型。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：滚动变量保存最近两个状态即可，不需要完整数组。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：n 为一、n 为二、题目是否允许零阶、结果范围。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。正确性通常从最优子结构证明：任意最优解的最后一步必然落入枚举集合；每个枚举候选又由已经正确的子问题构成。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。数组维度和初始化要服务于状态含义。不可达用大正数或负数哨兵时要避免溢出。方案数可能超过 int 时改用 `long long` 或按题目取模。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。若每次可走一到 k 阶，转移变成固定宽度窗口求和。追问通常会问状态是否足够、初始化为什么这样、空间压缩读到的是旧值还是新值。请准备从暴力搜索树指出重复子问题。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

72 编辑距离。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：二维状态表示两个前缀之间的最少编辑操作。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：末尾字符相同继承左上；不同则枚举插入、删除、替换。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪

费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：空串、完全相同、完全不同、操作代价不同。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。正确性通常从最优子结构证明：任意最优解的最后一步必然落入枚举集合；每个枚举候选又由已经正确的子问题构成。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。数组维度和初始化要服务于状态含义。不可达用大正数或负数哨兵时要避免溢出。方案数可能超过 int 时改用 `long long` 或按题目取模。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。若只允许插入删除，问题退化到和最长公共子序列相关。追问通常会问状态是否足够、初始化为为什么这样、空间压缩读到的是旧值还是新值。请准备从暴力搜索树指出重复子问题。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

75 颜色分类。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：三指针维护零区、一号区、未知区和二区。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：遇到二时只移动右边界，不移动扫描指针，因为换回来的元素未知。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：全零、全二、已经有序、逆序、交换后漏检查。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。正确性通常用区间不变量证明。每次循环只扩大已经正确的区域，未知区严格缩小，且被移出的元素已经放到符合语义的位置。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。C++ 实现优先使用 `std::vector<int>` 和明确的整型下标。涉及乘积、面积、总和时要主动判断是否需要 `long long`。如果题目要求原地修改，要说明返回值表示新长度还是数组本身，避免把输出语义和容器容量混在一起。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。这是荷兰国旗问题，能推广到三类分区的原地维护。追问通常会落在原地修改、稳定顺序、输出规模和整数溢出上。请准备说明为什么保存下标比保存指针更稳，为什么某个位置一旦写入就不会再被未来元素破坏。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

76 最小覆盖子串。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：窗口内字符计数必须覆盖目标计数，满足后尽量收缩左端。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：用 `formed` 记录满足需求的字符种类，避免每次全量比较。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：目标有重复字符、源串缺字符、最小窗口在开头或结尾、大小写。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。证明重点是排除。某个左端被移过之后，所有以它为左端的更长窗口都

不可能比当前答案更优，或者已经被当前状态完整统计。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。C++ 中建议把窗口写成左闭右开或左闭右闭中的一种，并在全题坚持。字符计数用 `std::array<int, 128>` 或 26 长度数组时要说明字符集假设；泛化字符集再换哈希表。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。若字符集固定，数组计数更快；若是 Unicode，哈希表更通用。追问通常会问窗口为什么不会漏掉左端、为什么有负数会失效、固定窗口和可变窗口有什么区别。请准备一个反例证明错误窗口条件会漏解。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

78 子集。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：每个元素都有选或不选两种选择，答案是所有路径节点。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：回溯按起点向后枚举，避免不同顺序生成同一集合。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：空数组、单元素、输出数量为二的 n 次方、顺序不要求。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。正确性要说明搜索树覆盖所有合法答案且每个答案只生成一次。剪枝条件必须只删除不可能通向答案的分支。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。回溯代码虽然天然递归，但工程中要意识到深度。题目规模较小时递归可读性好；若要遵守严格工程栈限制，可以改成显式栈状态机。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。也可用位掩码枚举所有子集，适合 n 较小的场景。追问通常会问去重发生在同层还是路径、剪枝是否会删掉合法解、输出规模是多少。请准备画出前三层搜索树。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

79 单词搜索。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：网格路径不能重复使用同一格，状态包含位置、匹配下标和访问标记。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：剪枝包括首字符不匹配、剩余长度不足、字符频次不足和越界访问。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：单字符单词、路径需要回退、重复字母、单元格不能复用。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。正确性要说明搜索树覆盖所有合法答案且每个答案只生成一次。剪枝条件必须只删除不可能通向答案的分支。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。回溯代码虽然天然递归，但工程中要意识到深度。题目规模较小时递归可读性好；若要遵守严格工程栈限制，可以改成显式栈状态机。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值

的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。若要查很多单词，应使用 Trie 合并搜索前缀。追问通常会问去重发生在同层还是路径、剪枝是否会删掉合法解、输出规模是多少。请准备画出前三层搜索树。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

84 柱状图中最大的矩形。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：每根柱子作为最矮高度时，需要知道左右第一个更矮位置。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：单调递增栈在遇到更矮柱时结算被弹出柱子的最大宽度。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：递增、递减、相等高度、哨兵零、宽度计算。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。每个元素最多入栈一次、出栈一次，所以时间线性。正确性来自弹出时机：一旦遇到破坏单调性的元素，栈顶元素的答案已经确定。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。栈里通常存下标而不是值，因为要计算距离、宽度和过期。可以用 `std::vector<int>` 当栈，便于查看顶部和减少适配器限制。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。最大矩形可以作为二维矩阵最大矩形的行压缩子问题。追问通常会问栈里为什么存下标、相等元素如何处理、弹出时到底结算了谁。请准备用一个严格递增和一个严格递减样例解释入栈出栈次数。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

88 合并两个有序数组。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：第一个数组尾部有空位，适合从后往前写入最终最大值。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：逆向填充避免覆盖尚未读取的 `nums1` 元素。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：第二个数组为空、第一个有效元素为空、重复值、尾部剩余。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。正确性通常用区间不变量证明。每次循环只扩大已经正确的区域，未知区严格缩小，且被移出的元素已经放到符合语义的位置。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。C++ 实现优先使用 `std::vector<int>` 和明确的整型下标。涉及乘积、面积、总和时要主动判断是否需要 `long long`。如果题目要求原地修改，要说明返回值表示新长度还是数组本身，避免把输出语义和容器容量混在一起。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。若没有额外空位，只能新开数组或使用更复杂原地归并。追问通常会落在原地修改、稳定顺序、输出规模和整数溢出上。请准备说明为什么保存下标比保存指针更稳，为什么某个位置一旦写入就不会再被未来元素破坏。最后给自己留一个实验：把正确实现中的关键条

件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

90 子集 II。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：重复元素会让不同下标路径生成相同集合。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：排序后在同一层跳过相同值，保留不同层使用重复值的可能。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：全重复、空集、重复值相邻、去重条件写成同层而不是全局。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。正确性要说明搜索树覆盖所有合法答案且每个答案只生成一次。剪枝条件必须只删除不可能通向答案的分支。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。回溯代码虽然天然递归，但工程中要意识到深度。题目规模较小时递归可读性好；若要遵守严格工程栈限制，可以改成显式栈状态机。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。组合总和 II 使用同样的同层去重思想。追问通常会问去重发生在同层还是路径、剪枝是否会删掉合法解、输出规模是多少。请准备画出前三层搜索树。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

94 二叉树中序遍历。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：中序访问顺序是左子树、根、右子树，BST 中会得到有序序列。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：用显式栈模拟一路向左，再回退访问根并转向右子树。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：空树、单节点、链式左树、链式右树。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。正确性通常是对子树归纳。左右子树已经给出稳定答案，父节点只需按题目约束合并这两个答案。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。示例中可以用指针节点，但比较节点身份时比较指针。极端深树用 `std::vector<TreeNode*>` 显式栈或队列。最大路径类题要用足够小的初值，不能默认零。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。Morris 遍历可做到常数额外空间，但会临时改树结构。追问通常会问返回值语义、空节点初值、全负数和极端深树。请准备把递归思想改写成显式栈或队列的口头方案。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

98 验证二叉搜索树。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：BST 约束是全局上下界，不是只比较父子节点。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：中序严格递增或显式传递上下界都可以；中序版本要保存前一个访问值。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪

费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：重复值、int 最小最大值、局部合法但全局非法。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。正确性通常是对子树归纳。左右子树已经给出稳定答案，父节点只需按题目约束合并这两个答案。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。示例中可以用指针节点，但比较节点身份时比较指针。极端深树用 `std::vector<TreeNode*>` 显式栈或队列。最大路径类题要用足够小的初值，不能默认零。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。若允许重复值，需要明确重复放左还是放右。追问通常会问返回值语义、空节点初值、全负数和极端深树。请准备把递归思想改写成显式栈或队列的口头方案。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

102 二叉树层序遍历。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：队列在每轮开始时保存当前层所有节点。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：记录 level size 固定层边界，避免下一层节点混入当前层。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：空树、只有根、最后一层不满、左右顺序。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。正确性通常是对子树归纳。左右子树已经给出稳定答案，父节点只需按题目约束合并这两个答案。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。示例中可以用指针节点，但比较节点身份时比较指针。极端深树用 `std::vector<TreeNode*>` 显式栈或队列。最大路径类题要用足够小的初值，不能默认零。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。锯齿形层序只是在每层输出顺序上增加方向控制。追问通常会问返回值语义、空节点初值、全负数和极端深树。请准备把递归思想改写成显式栈或队列的口头方案。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

103 二叉树锯齿形层序遍历。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：BFS 层边界不变，变化的是当前层写入结果的方向。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：可以层内反转，也可以预先配数组按方向写入目标下标。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：单层、偶数层、奇数层、空树。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。正确性通常是对子树归纳。左右子树已经给出稳定答案，父节点只需按题目约束合并这两个答案。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。示例中可以用指针节点，但比较节点身份时比较指针。极端深树用 `std::vector<TreeNode*>` 显式栈或队列。最大路径类题要用足够小的初值，不能默认零。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。若要求从底向上输出，收集后反转层列表即可。追问通常会问返回值语义、空节点初值、全负数和极端深树。请准备把递归思想改写成显式栈或队列的口头方案。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

104 二叉树最大深度。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：深度可以看成层数，也可以作为栈中携带的状态。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：显式栈保存节点和深度，避免极端链式树递归栈过深。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：空树返回零、根深度是一、链式树。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。正确性通常是对子树归纳。左右子树已经给出稳定答案，父节点只需按题目约束合并这两个答案。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。示例中可以用指针节点，但比较节点身份时比较指针。极端深树用 `std::vector<TreeNode*>` 显式栈或队列。最大路径类题要用足够小的初值，不能默认零。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。最小深度不能简单取左右最小，因为空子树不构成叶子路径。追问通常会问返回值语义、空节点初值、全负数和极端深树。请准备把递归思想改写成显式栈或队列的口头方案。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

105 前序中序构造二叉树。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：前序给根，中序把左右子树切开。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：用哈希表记录中序下标，避免每次线性寻找根位置。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：节点值唯一、空子树、左右子树长度计算、输入不合法。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。正确性通常是对子树归纳。左右子树已经给出稳定答案，父节点只需按题目约束合并这两个答案。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。示例中可以用指针节点，但比较节点身份时比较指针。极端深树用 `std::vector<TreeNode*>` 显式栈或队列。最大路径类题要用足够小的初值，不能默认零。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。后序中序构造类似，只是根来自后序末尾。追问通常会问返回值语义、空节点初值、全负数和极端深树。请准备把递归思想改写成显式栈或队列的口头方案。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

106 中序后序构造二叉树。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整

推理。第一句话从问题出发：后序末尾是根，中序位置确定左右子树规模。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：构造时要小心后序左右区间边界，避免切分错位。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：空树、单节点、全左链、全右链。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。正确性通常是对子树归纳。左右子树已经给出稳定答案，父节点只需按题目约束合并这两个答案。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。示例中可以用指针节点，但比较节点身份时比较指针。极端深树用 `std::vector<TreeNode*>` 显式栈或队列。最大路径类题要用足够小的初值，不能默认零。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。若给前序和后序，二叉树通常不能唯一确定，除非额外限制。追问通常会问返回值语义、空节点初值、全负数和极端深树。请准备把递归思想改写成显式栈或队列的口头方案。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

108 有序数组转二叉搜索树。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：有序数组中点作为根能让左右规模接近。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：每次选择区间中点，左右子区间构造左右子树。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：空区间、偶数长度选左中还是右中、高度平衡定义。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。正确性通常是对子树归纳。左右子树已经给出稳定答案，父节点只需按题目约束合并这两个答案。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。示例中可以用指针节点，但比较节点身份时比较指针。极端深树用 `std::vector<TreeNode*>` 显式栈或队列。最大路径类题要用足够小的初值，不能默认零。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。工程上递归可读，若严格避免递归可用队列保存区间和父指针。追问通常会问返回值语义、空节点初值、全负数和极端深树。请准备把递归思想改写成显式栈或队列的口头方案。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

110 平衡二叉树。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：每个节点需要知道子树高度以及是否已经失衡。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：后序汇总时一旦子树失衡就向上返回失败标记，避免重复算高度。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：空树、单节点、左右高度差正好一、链式树。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。正确性通常是对子树归纳。左右子树已经给出稳定答案，父节点只需按

题目约束合并这两个答案。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。示例中可以用指针节点，但比较节点身份时比较指针。极端深树用 `std::vector<TreeNode*>` 显式栈或队列。最大路径类题要用足够小的初值，不能默认零。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。可以把返回值设计成 optional 高度，空表示不平衡。追问通常会问返回值语义、空节点初值、全负数和极端深树。请准备把递归思想改写成显式栈或队列的口头方案。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

111 二叉树最小深度。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：最小深度是到最近叶子的节点数，不是左右深度简单取最小。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：BFS 第一次遇到叶子就是最小深度，因为按层扩展。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：只有左子树、只有右子树、根为空、根是叶子。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。正确性通常是对子树归纳。左右子树已经给出稳定答案，父节点只需按题目约束合并这两个答案。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。示例中可以用指针节点，但比较节点身份时比较指针。极端深树用 `std::vector<TreeNode*>` 显式栈或队列。最大路径类题要用足够小的初值，不能默认零。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。DFS 写法必须特殊处理空子树不能作为最小路径。追问通常会问返回值语义、空节点初值、全负数和极端深树。请准备把递归思想改写成显式栈或队列的口头方案。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

112 路径总和。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：路径必须从根到叶子，状态是剩余目标值。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：沿路径减去当前节点值，到叶子时检查剩余是否等于叶子值。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：负数节点、目标为零、非叶子提前达到目标、空树。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。正确性通常是对子树归纳。左右子树已经给出稳定答案，父节点只需按题目约束合并这两个答案。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。示例中可以用指针节点，但比较节点身份时比较指针。极端深树用 `std::vector<TreeNode*>` 显式栈或队列。最大路径类题要用足够小的初值，不能默认零。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。若要求列出所有路径，需要保存路径数组并在回退时弹出。追问通常会问返回值语义、空节点初值、全负数和极端深树。请准备把递归思想改写成显式栈或队列的口头

方案。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

113 路径总和 II。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：相比判定题，状态还要保存当前路径。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：到叶子且和满足时复制路径；回溯时恢复路径。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：多个答案、负数、空树、路径数组复制时机。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。正确性通常是对子树归纳。左右子树已经给出稳定答案，父节点只需按题目约束合并这两个答案。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。示例中可以用指针节点，但比较节点身份时比较指针。极端深树用 `std::vector<TreeNode*>` 显式栈或队列。最大路径类题要用足够小的初值，不能默认零。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。若路径不要求从根到叶，模型会变成前缀和加哈希。追问通常会问返回值语义、空节点初值、全负数和极端深树。请准备把递归思想改写成显式栈或队列的口头方案。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

114 二叉树展开为链表。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：目标是按前序顺序把树原地改成右指针链。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：显式栈按右后左先入顺序处理，逐个接到前驱右侧。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：空树、单节点、原有右子树不能丢、左指针置空。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。正确性通常是对子树归纳。左右子树已经给出稳定答案，父节点只需按题目约束合并这两个答案。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。示例中可以用指针节点，但比较节点身份时比较指针。极端深树用 `std::vector<TreeNode*>` 显式栈或队列。最大路径类题要用足够小的初值，不能默认零。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。也可用寻找左子树最右节点的原地方法。追问通常会问返回值语义、空节点初值、全负数和极端深树。请准备把递归思想改写成显式栈或队列的口头方案。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

121 买卖股票的最佳时机。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：卖出日固定时，最佳买入日是此前最低价格。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：扫描维护历史最低价和最大利润，一次交易无需复杂状态机。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录

前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：单日价格、一直下跌、一直上涨、价格相同。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。正确性通常从最优子结构证明：任意最优解的最后一步必然落入枚举集合；每个枚举候选又由已经正确的子问题构成。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。数组维度和初始化要服务于状态含义。不可达用大正数或负数哨兵时要避免溢出。方案数可能超过 int 时改用 `long long` 或按题目取模。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。多次交易、冷冻期和手续费版本都要扩展成状态机。追问通常会问状态是否足够、初始化为什么这样、空间压缩读到的是旧值还是新值。请准备从暴力搜索树指出重复子问题。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

122 买卖股票 II。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：允许多次交易且不能同时持有，所有正收益差都可以收集。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：把长上涨段拆成每天买卖收益相同，因此累加正差值最优。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：下降段、平台价格、只有一天、手续费不存在。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。常见证明有交换论证和领先性论证。交换论证说明最优解可以被替换成贪心第一步而不变差；领先性论证说明贪心每一步都至少和任何方案一样好。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。实现时先把比较器写成明确的业务顺序。区间题注意起点相同时终点升序还是降序；覆盖题注意闭区间和半开区间的等号。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。有手续费时不能简单累加正差，需要状态机或调整贪心阈值。追问通常会要求证明。请准备交换论证或领先性论证，并给出一个看似合理但错误的贪心规则反例。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

123 买卖股票 III。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：最多两次交易，状态需要包含交易次数和持有状态。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：维护 buy1、sell1、buy2、sell2 四个状态，按天更新。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：价格为空、只够一次交易、两次交易间不能重叠。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。正确性通常从最优子结构证明：任意最优解的最后一步必然落入枚举集合；每个枚举候选又由已经正确的子问题构成。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重

复进入。

C++ 追问重点是实现边界和成本。数组维度和初始化要服务于状态含义。不可达用大正数或负数哨兵时要避免溢出。方案数可能超过 `int` 时改用 `long long` 或按题目取模。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。最多 k 次交易是它的泛化， k 很大时退化为无限次交易。追问通常会问状态是否足够、初始化为什么这样、空间压缩读到的是旧值还是新值。请准备从暴力搜索树指出重复子问题。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

124 二叉树最大路径和。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：路径可以从任意节点到任意节点，但不能分叉向父节点贡献两边。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：每个节点向父亲贡献单边最大增益，全局答案可用左右两边加当前节点更新。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：全负数、单节点、左右增益为负时取零、答案初始值。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。正确性通常是对子树归纳。左右子树已经给出稳定答案，父节点只需按题目约束合并这两个答案。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。示例中可以用指针节点，但比较节点身份时比较指针。极端深树用 `std::vector<TreeNode*>` 显式栈或队列。最大路径类题要用足够小的初值，不能默认零。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。若路径必须从根到叶，状态和答案位置完全不同。追问通常会问返回值语义、空节点初值、全负数和极端深树。请准备把递归思想改写成显式栈或队列的口头方案。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

127 单词接龙。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：每个单词是节点，一次修改一个字符形成边。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：通配模式桶加速邻居查询，BFS 第一次到达终点就是最短转换长度。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：终点不在词表、起点和终点相同、桶重复扫描、单词长度一致。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。BFS 正确性来自层次扩展，第一次到达某状态就是最短距离。连通块算法正确性来自一次搜索会标记与起点可达的全部节点且不会越过非法边。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。图节点连续时用 `std::vector<std::vector<int>>`；节点是字符串时先编号或用哈希表。网格方向数组用 `constexpr std::array`，边界判断集中写。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。双向 BFS 可以进一步降低搜索空间。追问通常会问节点和边的建模、

访问标记时机、BFS 为什么最短。请准备说明状态是否还包含资源，为什么不能只按位置去重。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

128 最长连续序列。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：连续段只需要从没有前驱的数开始扩展。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：哈希集合判断 x 减一是否存在，跳过非起点避免重复扫描。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：重复元素、负数、空数组、整数边界。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。正确性来自代数等价。若两个前缀满足差值、同余或计数差要求，则它们之间的区间正好满足题意；反过来任意合法区间也对应这样一对前缀。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。实现时用 `find` 判断是否存在，用 `operator[]` 只在确认要插入或累加时使用。总和可能溢出时使用 `long long`；模运算要处理负数余数归一化。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。若数据流动态插入，可以用并查集或区间合并维护长度。追问通常会问先查询还是先更新、哈希表保存次数还是最早下标、为什么负数不影响前缀和。请准备说明从任意合法区间如何反推到两个前缀状态。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

130 被围绕的区域。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：只有不连通到边界的 O 才会被翻转。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：从边界 O 出发标记安全区域，再翻转剩余 O。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：全边界、无 O、单行单列、内部岛屿。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。BFS 正确性来自层次扩展，第一次到达某状态就是最短距离。连通块算法正确性来自一次搜索会标记与起点可达的全部节点且不会越过非法边。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。图节点连续时用 `std::vector<std::vector<int>>`；节点是字符串时先编号或用哈希表。网格方向数组用 `constexpr std::array`，边界判断集中写。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。这是反向思维：找不能被翻转的区域比直接判断每块区域更简单。追问通常会问节点和边的建模、访问标记时机、BFS 为什么最短。请准备说明状态是否还包含资源，为什么不能只按位置去重。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

131 分割回文串。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：每次选择一个从当前位置开始的回文前缀。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：预处理回文 DP 表，让回溯合法性检查为常数。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：空串、单字符、全相同字符、重复分割路径。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。正确性要说明搜索树覆盖所有合法答案且每个答案只生成一次。剪枝条件必须只删除不可能通向答案的分支。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。回溯代码虽然天然递归，但工程中要意识到深度。题目规模较小时递归可读性好；若要遵守严格工程栈限制，可以改成显式栈状态机。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。若只求最少切割次数，可改成 DP 最短切分。追问通常会问去重发生在同层还是路径、剪枝是否会删掉合法解、输出规模是多少。请准备画出前三层搜索树。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

133 克隆图。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：每个原节点需要唯一对应一个新节点，边按原图连接。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：哈希表保存原节点到克隆节点的映射，BFS 或 DFS 遍历图。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：空图、自环、重边、环结构、不能按节点值唯一假设。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。BFS 正确性来自层次扩展，第一次到达某状态就是最短距离。连通块算法正确性来自一次搜索会标记与起点可达的全部节点且不会越过非法边。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。图节点连续时用 `std::vector<std::vector<int>>`；节点是字符串时先编号或用哈希表。网格方向数组用 `constexpr std::array`，边界判断集中写。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。若图节点值不唯一，必须用地址作为键。追问通常会问节点和边的建模、访问标记时机、BFS 为什么最短。请准备说明状态是否还包含资源，为什么不能只按位置去重。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

136 只出现一次的数字。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：成对元素可以用异或抵消，剩下单个元素。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：异或满足交换结合，扫描顺序不影响答案。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：零、负数、唯一值在开头或结尾、所有其他值恰好两次。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。正确性来自二进制位独立性。异或按位满足交换结合和自反抵消；掩码 DP 则是对子集大小做归纳。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还

要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。使用无符号整数能减少移位歧义。位数超过 31 时用 `long long` 或 `std::bitset`。表达式加括号，避免位运算优先级误读。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。若其他值出现三次，要改为按位计数取模。追问通常会问每一位代表什么、负数和移位是否安全、状态相同但资源不同能否合并。请准备说明位运算只是编码，真正关键仍是状态语义。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

139 单词拆分。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：状态表示前缀能否由字典单词拼出。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：枚举最后一个单词的起点，左侧前缀可达且子串在字典中则当前可达。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：空串、重复单词、长单词、substr 复制成本。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。正确性通常从最优子结构证明：任意最优解的最后一步必然落入枚举集合；每个枚举候选又由已经正确的子问题构成。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。数组维度和初始化要服务于状态含义。不可达用大正数或负数哨兵时要避免溢出。方案数可能超过 int 时改用 `long long` 或按题目取模。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。若要输出所有句子，需要 DFS 加记忆化或前驱列表。追问通常会问状态是否足够、初始化为为什么这样、空间压缩读到的是旧值还是新值。请准备从暴力搜索树指出重复子问题。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

141 环形链表。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：快慢指针在有环时必然相遇，无环时快指针先到空。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：不用哈希表也能常数空间检测环。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：空链、单节点自环、两节点环、尾部无环。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。正确性靠结构不变量：已经反转或合并的部分保持合法链，未处理部分仍可达，二者之间只有一个明确连接点。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。节点通常由评测平台管理，代码不要随意 `delete` 输入节点。若创建新节点，要说明所有权。比较交点时比较地址，不比较值。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。若要求环入口，用相遇后双指针同步走。追问通常会让你画指针。请准备说明上一段尾、当前段头、当前节点、后继节点分别是谁，以及哪一步保存 next 防止断链。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失

败原因，才说明你真正掌握了这道题。

142 环形链表 II。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：相遇点到入口的距离关系来自快慢指针速度差。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：相遇后一个指针回头，两个指针每次走一步，再次相遇就是入口。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：入口在头节点、长尾短环、无环、单节点环。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。正确性靠结构不变量：已经反转或合并的部分保持合法链，未处理部分仍可达，二者之间只有一个明确连接点。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或DP表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。节点通常由评测平台管理，代码不要随意 `delete` 输入节点。若创建新节点，要说明所有权。比较交点时比较地址，不比较值。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。也可用哈希集合，但空间更高。追问通常会让你画指针。请准备说明上一段尾、当前段头、当前节点、后继节点分别是谁，以及哪一步保存 `next` 防止断链。最后给自己留一个实验：把正确实现中的关键条件反过来或删掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

146 LRU 缓存。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：哈希表负责按 `key` 定位，双向链表负责维护新旧顺序。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：访问和更新都把节点移动到头部，容量满时删除尾部最旧节点。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：容量为一、重复 `put`、`get` 不存在、更新已有值。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。正确性靠结构不变量：已经反转或合并的部分保持合法链，未处理部分仍可达，二者之间只有一个明确连接点。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或DP表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。节点通常由评测平台管理，代码不要随意 `delete` 输入节点。若创建新节点，要说明所有权。比较交点时比较地址，不比较值。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。C++ 可用 `std::list` 加迭代器，也可手写双向链表理解所有权。追问通常会让你画指针。请准备说明上一段尾、当前段头、当前节点、后继节点分别是谁，以及哪一步保存 `next` 防止断链。最后给自己留一个实验：把正确实现中的关键条件反过来或删掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

148 排序链表。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：链表不能随机访问，适合归并排序而不是快速排序数组 `partition`。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：自底向上迭代归并能避免递归栈，同时保持 $O(n \log n)$ 。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记

录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：空链、单节点、重复值、奇数长度、切断子链。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。正确性靠结构不变量：已经反转或合并的部分保持合法链，未处理部分仍可达，二者之间只有一个明确连接点。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。节点通常由评测平台管理，代码不要随意 `delete` 输入节点。若创建新节点，要说明所有权。比较交点时比较地址，不比较值。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。若把值复制到数组排序，会改变节点身份语义。追问通常会让你画指针。请准备说明上一段尾、当前段头、当前节点、后继节点分别是谁，以及哪一步保存 `next` 防止断链。最后给自己留一个实验：把正确实现中的关键条件反过来或删除，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

150 逆波兰表达式求值。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：后缀表达式把优先级和括号都编码进 `token` 顺序。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：遇到数字入栈，遇到运算符弹出右操作数和左操作数计算。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：负数 `token`、多位数、除法向零截断、减法顺序。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。每个元素最多入栈一次、出栈一次，所以时间线性。正确性来自弹出时机：一旦遇到破坏单调性的元素，栈顶元素的答案已经确定。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。栈里通常存下标而不是值，因为要计算距离、宽度和过期。可以用 `std::vector<int>` 当栈，便于查看顶部和减少适配器限制。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。中缀表达式求值需要另一个运算符栈处理优先级。追问通常会问栈里为什么存下标、相等元素如何处理、弹出时到底结算了谁。请准备用一个严格递增和一个严格递减样例解释入栈出栈次数。最后给自己留一个实验：把正确实现中的关键条件反过来或删除，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

152 乘积最大子数组。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：负数会让最大乘积和最小乘积互换。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：同时维护以当前位置结尾的最大和最小乘积。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：零、负数个数奇偶、单元素、乘积溢出。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。正确性通常从最优子结构证明：任意最优解的最后一步必然落入枚举集合；每个枚举候选又由已经正确的子问题构成。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。数组维度和初始化要服务于状态含义。不可达用大正数或负

数哨兵时要避免溢出。方案数可能超过 `int` 时改用 `long long` 或按题目取模。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。如果要求返回区间，同步记录状态来源。追问通常会问状态是否足够、初始化为什么这样、空间压缩读到的是旧值还是新值。请准备从暴力搜索树指出重复子问题。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

153 寻找旋转排序数组中的最小值。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：最小值是旋转断点，右端点可帮助判断中点在哪一段。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：若 `mid` 大于 `right`，最小值在右侧；否则在左侧含 `mid`。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：未旋转、长度一、旋转一位、无重复条件。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。证明二分不是证明循环会停，而是证明被排除的一半确实不含答案。答案空间二分还要证明可行性谓词单调。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。中点写成 `left + (right - left) / 2`。判断平方、乘法、总量时避免溢出。用 `std::lower_bound` 能表达清楚时优先用它，但面试要能手写同等语义。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。有重复时遇到相等只能收缩 `right`，最坏退化。追问通常会要求你写出单调谓词和答案边界。请准备说明 `left`、`right` 的语义，为什么 `mid` 被排除或保留，以及返回值是否还需要二次验证。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

154 寻找旋转排序数组中的最小值 II。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：重复元素会让 `mid` 和 `right` 相等时无法判断最小值方向。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：相等时只能安全地丢掉一个 `right`，因为它不是唯一必要候选。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：全相等、重复值跨断点、最小值出现多次。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。证明二分不是证明循环会停，而是证明被排除的一半确实不含答案。答案空间二分还要证明可行性谓词单调。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。中点写成 `left + (right - left) / 2`。判断平方、乘法、总量时避免溢出。用 `std::lower_bound` 能表达清楚时优先用它，但面试要能手写同等语义。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。这题说明二分依赖的信息不足时会退化，不是所有旋转题都严格对数。追问通常会要求你写出单调谓词和答案边界。请准备说明 `left`、`right` 的语义，为什么 `mid` 被排除或保留，以及返回值是否还需要二次验证。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

155 最小栈。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：普通栈只能看到栈顶，不能常数时间知道历史最小值。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：辅助栈同步保存每个深度对应的最小值。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：重复最小值、弹出最小值、空栈操作约定。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。每个元素最多入栈一次、出栈一次，所以时间线性。正确性来自弹出时机：一旦遇到破坏单调性的元素，栈顶元素的答案已经确定。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。栈里通常存下标而不是值，因为要计算距离、宽度和过期。可以用 `std::vector<int>` 当栈，便于查看顶部和减少适配器限制。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。也可保存差值压缩空间，但可读性和溢出处理更复杂。追问通常会问栈里为什么存下标、相等元素如何处理、弹出时到底结算了谁。请准备用一个严格递增和一个严格递减样例解释入栈出栈次数。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

160 相交链表。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：相交是节点地址相同，不是节点值相同。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：两个指针走完各自链表后切换到另一条链，走过总长度相同后相遇。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：不相交、头节点相交、尾节点相交、长度差很大。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。正确性靠结构不变量：已经反转或合并的部分保持合法链，未处理部分仍可达，二者之间只有一个明确连接点。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。节点通常由评测平台管理，代码不要随意 `delete` 输入节点。若创建新节点，要说明所有权。比较交点时比较地址，不比较值。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。哈希集合更直观但空间更高。追问通常会让你画指针。请准备说明上一段尾、当前段头、当前节点、后继节点分别是谁，以及哪一步保存 `next` 防止断链。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

162 寻找峰值。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：比较 `mid` 和 `mid` 加一的斜率能判断某侧一定存在峰值。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：上升则右侧有峰，下降则左侧含 `mid` 有峰。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：长度一、峰在边界、多个峰、相邻

不相等假设。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。证明二分不是证明循环会停，而是证明被排除的一半确实不含答案。答案空间二分还要证明可行性谓词单调。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。中点写成 `left + (right - left) / 2`。判断平方、乘法、总量时避免溢出。用 `std::lower_bound` 能表达清楚时优先用它，但面试要能手写同等语义。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。二维峰值需要按行列最大值进行更复杂二分。追问通常会要求你写出单调谓词和答案边界。请准备说明 `left`、`right` 的语义，为什么 `mid` 被排除或保留，以及返回值是否还需要二次验证。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

169 多数元素。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：超过一半的元素可以和其他元素两两抵消后仍然剩下。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：Boyer-Moore 投票维护候选和计数。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：题目是否保证存在多数、计数归零、全相同。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。常见证明有交换论证和领先性论证。交换论证说明最优解可以被替换成贪心第一步而不变差；领先性论证说明贪心每一步都至少和任何方案一样好。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。实现时先把比较器写成明确的业务顺序。区间题注意起点相同时终点升序还是降序；覆盖题注意闭区间和半开区间的等号。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。若找超过三分之一的元素，需要维护两个候选。追问通常会要求证明。请准备交换论证或领先性论证，并给出一个看似合理但错误的贪心规则反例。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

189 轮转数组。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：右移 k 位等价于把数组切成两段后换序。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：三次反转能原地完成：整体、前 k 、后 $n - k$ 。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在： k 为零、 k 大于 n 、空数组、单元素。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。正确性通常用区间不变量证明。每次循环只扩大已经正确的区域，未知区严格缩小，且被移出的元素已经放到符合语义的位置。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。C++ 实现优先使用 `std::vector<int>` 和明确的整型下标。涉及乘积、面积、总和时要主动判断是否需要 `long long`。如果题目要求原地修改，要说明返

返回值表示新长度还是数组本身，避免把输出语义和容器容量混在一起。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。环状替换也可做，但要处理最大公约数个循环。追问通常会落在原地修改、稳定顺序、输出规模和整数溢出上。请准备说明为什么保存下标比保存指针更稳，为什么某个位置一旦写入就不会再被未来元素破坏。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

198 打家劫舍。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：每间房只有偷和不偷两种结局，偷当前就不能偷前一间。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：滚动保存前一状态和前两状态。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：空数组、单间、全零、金额很大。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。正确性通常从最优子结构证明：任意最优解的最后一步必然落入枚举集合；每个枚举候选又由已经正确的子问题构成。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。数组维度和初始化要服务于状态含义。不可达用大正数或负数哨兵时要避免溢出。方案数可能超过 int 时改用 `long long` 或按题目取模。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。环形房屋要拆成不含首和不含尾两个线性问题。追问通常会问状态是否足够、初始化为什么这样、空间压缩读到的是旧值还是新值。请准备从暴力搜索树指出重复子问题。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

199 二叉树右视图。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：每层从右侧能看到的就是该层最后访问或最先访问的右侧节点。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：BFS 固定层边界，取每层最后一个；或 DFS 按右优先首次到达深度。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：空树、左链、右链、左右交错。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。正确性通常是对子树归纳。左右子树已经给出稳定答案，父节点只需按题目约束合并这两个答案。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。示例中可以用指针节点，但比较节点身份时比较指针。极端深树用 `std::vector<TreeNode*>` 显式栈或队列。最大路径类题要用足够小的初值，不能默认零。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。若求左视图，只改变层内取值方向。追问通常会问返回值语义、空节点初值、全负数和极端深树。请准备把递归思想改写成显式栈或队列的口头方案。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

200 岛屿数量。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：陆地格子是节点，四方向相邻是边，岛屿是连通块。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：扫描遇到未访问陆地就启动搜索并标记整块。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：空网格、全水、全陆、斜对角不连通、是否允许修改输入。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。BFS 正确性来自层次扩展，第一次到达某状态就是最短距离。连通块算法正确性来自一次搜索会标记与起点可达的全部节点且不会越过非法边。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。图节点连续时用 `std::vector<std::vector<int>>`；节点是字符串时先编号或用哈希表。网格方向数组用 `constexpr std::array`，边界判断集中写。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。也可用并查集和相邻陆地，适合动态岛屿扩展。追问通常会问节点和边的建模、访问标记时机、BFS 为什么最短。请准备说明状态是否还包含资源，为什么不能只按位置去重。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

202 快乐数。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：反复变换数字会进入一或循环，状态空间最终有限。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：用集合检测重复状态，或用快慢指针看成函数图。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：输入为一、进入循环、位平方和函数、负数不在题意内。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。正确性来自代数等价。若两个前缀满足差值、同余或计数差要求，则它们之间的区间正好满足题意；反过来任意合法区间也对应这样一对前缀。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。实现时用 `find` 判断是否存在，用 `operator[]` 只在确认要插入或累加时使用。总和可能溢出时使用 `long long`；模运算要处理负数余数归一化。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。快慢指针版本能训练把数值迭代看成链表。追问通常会问先查询还是先更新、哈希表保存次数还是最早下标、为什么负数不影响前缀和。请准备说明从任意合法区间如何反推到两个前缀状态。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

206 反转链表。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：每次把当前节点的 `next` 指向已经反转好的前缀头。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：先保存后继，再改指针，再推进 `previous` 和 `current`。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录

前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：空链、单节点、两个节点、不要丢失剩余链。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。正确性靠结构不变量：已经反转或合并的部分保持合法链，未处理部分仍可达，二者之间只有一个明确连接点。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。节点通常由评测平台管理，代码不要随意 `delete` 输入节点。若创建新节点，要说明所有权。比较交点时比较地址，不比较值。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。K 组反转是在这段局部反转外增加分组边界。追问通常会让你画指针。请准备说明上一段尾、当前段头、当前节点、后继节点分别是谁，以及哪一步保存 `next` 防止断链。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

207 课程表。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：课程是节点，先修关系是有向边，问题是判断是否有环。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：入度拓扑排序不断学习当前无依赖课程。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：边方向、孤立课程、环、自依赖、重复边。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。BFS 正确性来自层次扩展，第一次到达某状态就是最短距离。连通块算法正确性来自一次搜索会标记与起点可达的全部节点且不会越过非法边。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。图节点连续时用 `std::vector<std::vector<int>>`；节点是字符串时先编号或用哈希表。网格方向数组用 `constexpr std::array`，边界判断集中写。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。DFS 三色标记也能判环，但要明确访问状态。追问通常会问节点和边的建模、访问标记时机、BFS 为什么最短。请准备说明状态是否还包含资源，为什么不能只按位置去重。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

208 实现 Trie。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：前缀树把字符串公共前缀共享成路径。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：插入和查询都逐字符沿边移动，不存在的边按需要创建。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：空字符串约定、单词结束标记、前缀存在不等于单词存在。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。正确性通常是对子树归纳。左右子树已经给出稳定答案，父节点只需按题目约束合并这两个答案。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。示例中可以用指针节点，但比较节点身份时比较指针。极端深树用 `std::vector<TreeNode*>` 显式栈或队列。最大路径类题要用足够小的初值，不能默认零。

此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。若字符集很大，用哈希子节点；小写字母用数组更快。追问通常会问返回值语义、空节点初值、全负数和极端深树。请准备把递归思想改写成显式栈或队列的口头方案。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

209 长度最小的子数组。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：正数数组让窗口和随右端增加、随左端减少。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：右端扩展到满足目标后，不断左移缩短并更新答案。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：不存在答案、单元素达到目标、全正是要条件。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。证明重点是排除。某个左端被移过之后，所有以它为左端的更长窗口都不可能比当前答案更优，或者已经被当前状态完整统计。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。C++ 中建议把窗口写成左闭右开或左闭右闭中的一种，并在全题坚持。字符计数用 `std::array<int, 128>` 或 26 长度数组时要说明字符集假设；泛化字符集再换哈希表。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。若含负数，需要前缀和加单调队列。追问通常会问窗口为什么不会漏掉左端、为什么有负数会失效、固定窗口和可变窗口有什么区别。请准备一个反例证明错误窗口条件会漏解。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

210 课程表 II。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：在判环基础上还要输出一个可行拓扑序。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：每弹出一个入度为零课程，就把它加入顺序并删除出边。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：有环返回空数组、多个可行顺序、边方向。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。BFS 正确性来自层次扩展，第一次到达某状态就是最短距离。连通块算法正确性来自一次搜索会标记与起点可达的全部节点且不会越过非法边。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。图节点连续时用 `std::vector<std::vector<int>>`；节点是字符串时先编号或用哈希表。网格方向数组用 `constexpr std::array`，边界判断集中写。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。若需要字典序最小拓扑序，把队列换成小顶堆。追问通常会问节点和边的建模、访问标记时机、BFS 为什么最短。请准备说明状态是否还包含资源，为什么不能只按位置去重。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

211 添加与搜索单词。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：点号通配符让查询可能分叉，但前缀树仍能剪掉不匹配前缀。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：普通字符沿唯一边走，点号枚举当前节点所有孩子。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：连续通配符、空结果、单词结束标记、字符集大小。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。正确性通常是对子树归纳。左右子树已经给出稳定答案，父节点只需按题目约束合并这两个答案。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。示例中可以用指针节点，但比较节点身份时比较指针。极端深树用 `std::vector<TreeNode*>` 显式栈或队列。最大路径类题要用足够小的初值，不能默认零。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。大量通配符会接近遍历整棵 Trie，需要规模约束。追问通常会问返回值语义、空节点初值、全负数和极端深树。请准备把递归思想改写成显式栈或队列的口头方案。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

215 数组中的第 K 个最大元素。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：只关心第 K 大，不需要完整排序。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：大小为 K 的小顶堆保存当前前 K 大边界；快速选择可做到平均线性。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：K 为一、K 为 n、重复值、随机 pivot。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。正确性常用边界候选证明：被堆丢弃的元素不可能进入答案，仍在堆中的元素覆盖所有可能的下一步选择。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。C++ 的 `std::priority_queue` 默认大顶堆。小顶堆用 `std::greater<T>`；结构体比较器要按谁优先级更低来写。堆中保存大对象时尽量保存下标或指针。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。若数据流持续到来，堆方案比一次性快速选择更自然。追问通常会问为什么不完整排序、堆顶是否可能过期、比较器方向是否写反。请准备说明堆里元素的业务含义，而不是只说 `priority_queue`。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

216 组合总和 III。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：从一到九中选 k 个数和为 n，每个数最多用一次。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：按递增起点枚举，利用剩余个数和剩余和剪枝。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：n 太小、n 太大、k 为零、路径长

度已经超过 k 。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。正确性要说明搜索树覆盖所有合法答案且每个答案只生成一次。剪枝条件必须只删除不可能通向答案的分支。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。回溯代码虽然天然递归，但工程中要意识到深度。题目规模较小时递归可读性好；若要遵守严格工程栈限制，可以改成显式栈状态机。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。可预估剩余最小和最大和进一步剪枝。追问通常会问去重发生在同层还是路径、剪枝是否会删掉合法解、输出规模是多少。请准备画出前三层搜索树。最后给自己留一个实验：把正确实现中的关键条件反过来或删掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

217 存在重复元素。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：判断是否出现过是哈希集合最直接的职责。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：扫描时若插入前已经存在则立即返回真。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：空数组、全唯一、重复在末尾、值域很小。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。正确性来自代数等价。若两个前缀满足差值、同余或计数差要求，则它们之间的区间正好满足题意；反过来任意合法区间也对应这样一对前缀。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。实现时用 `find` 判断是否存在，用 `operator[]` 只在确认要插入或累加时使用。总和可能溢出时使用 `long long`；模运算要处理负数余数归一化。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。若允许修改输入，排序后看相邻可以降低额外空间。追问通常会问先查询还是先更新、哈希表保存次数还是最早下标、为什么负数不影响前缀和。请准备说明从任意合法区间如何反推到两个前缀状态。最后给自己留一个实验：把正确实现中的关键条件反过来或删掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

221 最大正方形。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：以当前位置为右下角的最大正方形由上、左、左上三个状态限制。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：当前位置为一时取三者最小加一，否则为零。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：全零、全一、单行单列、字符一和数字一不要混淆。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。正确性通常从最优子结构证明：任意最优解的最后一步必然落入枚举集合；每个枚举候选又由已经正确的子问题构成。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。数组维度和初始化要服务于状态含义。不可达用大正数或负数哨兵时要避免溢出。方案数可能超过 `int` 时改用 `long long` 或按题目取模。此外要说明是否会修

改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。最大矩形则需要按行构造柱状图再用单调栈。追问通常会问状态是否足够、初始化为什么这样、空间压缩读到的是旧值还是新值。请准备从暴力搜索树指出重复子问题。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

226 翻转二叉树。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：每个节点交换左右孩子即可，遍历顺序不影响最终结构。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：显式队列或栈逐个节点交换，避免递归深度。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：空树、单节点、只有一侧子树、重复值。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。正确性通常是对子树归纳。左右子树已经给出稳定答案，父节点只需按题目约束合并这两个答案。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。示例中可以用指针节点，但比较节点身份时比较指针。极端深树用 `std::vector<TreeNode*>` 显式栈或队列。最大路径类题要用足够小的初值，不能默认零。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。这题简单但适合训练树节点指针修改。追问通常会问返回值语义、空节点初值、全负数和极端深树。请准备把递归思想改写成显式栈或队列的口头方案。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

230 二叉搜索树中第 K 小。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：BST 中序遍历按升序访问节点。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：显式栈中序，访问第 k 个节点时返回。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：k 为一、k 为节点数、树不平衡、是否有重复值。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。正确性通常是对子树归纳。左右子树已经给出稳定答案，父节点只需按题目约束合并这两个答案。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。示例中可以用指针节点，但比较节点身份时比较指针。极端深树用 `std::vector<TreeNode*>` 显式栈或队列。最大路径类题要用足够小的初值，不能默认零。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。若频繁查询，可在节点维护子树大小成为顺序统计树。追问通常会问返回值语义、空节点初值、全负数和极端深树。请准备把递归思想改写成显式栈或队列的口头方案。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

232 用栈实现队列。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：两个栈分别负责输入顺序和输出顺序。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：只有输出栈为空时，才把输入栈整体倒过去，保

证均摊常数。如果这三句话说不出来,说明你记住的是答案形状,不是题目结构。

暴力基线要讲具体:枚举哪些对象,检查函数做了什么,最坏输入如何把它拖慢。接着定位浪费:是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态,还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责:保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题,边界风险集中在:连续 peek、空队列、交替 push pop。请至少准备一个能打破错误实现的用例,并解释错误发生在哪个不变量上。

正确性说明要避开空话。每个元素最多入栈一次、出栈一次,所以时间线性。正确性来自弹出时机:一旦遇到破坏单调性的元素,栈顶元素的答案已经确定。二刷时可以把证明压成两段:第一段证明所有合法答案都会被覆盖,第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表,还要补一句容器中每个元素代表什么,什么时候进入,什么时候离开,是否可能重复进入。

C++ 追问重点是实现边界和成本。栈里通常存下标而不是值,因为要计算距离、宽度和过期。可以用 `std::vector<int>` 当栈,便于查看顶部和减少适配器限制。此外要说明是否会修改输入,是否需要保留原下标,是否存在整数溢出,是否有迭代器失效,是否使用了可能插入默认值的 API。面试中能主动说出这些成本,通常比只写短代码更有说服力。

变体迁移是二刷的核心。用队列实现栈则要通过轮转保持最近元素在队首。追问通常会问栈里为什么存下标、相等元素如何处理、弹出时到底结算了谁。请准备用一个严格递增和一个严格递减样例解释入栈出栈次数。最后给自己留一个实验:把正确实现中的关键条件反过来或删除掉,观察哪个测试首先失败。能解释失败原因,才说明你真正掌握了这道题。

235 二叉搜索树最近公共祖先。二刷这题时,不要满足于能写出代码,而要能在三分钟内讲出完整推理。第一句话从问题出发:BST 的值域关系能决定两个目标在当前节点的哪一侧。第二句话交代暴力基线:按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索:两个值都小去左,都大去右,否则当前节点就是分叉点。如果这三句话说不出来,说明你记住的是答案形状,不是题目结构。

暴力基线要讲具体:枚举哪些对象,检查函数做了什么,最坏输入如何把它拖慢。接着定位浪费:是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态,还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责:保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题,边界风险集中在:一个节点是另一个祖先、目标顺序、重复值约定。请至少准备一个能打破错误实现的用例,并解释错误发生在哪个不变量上。

正确性说明要避开空话。正确性通常是对子树归纳。左右子树已经给出稳定答案,父节点只需按题目约束合并这两个答案。二刷时可以把证明压成两段:第一段证明所有合法答案都会被覆盖,第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表,还要补一句容器中每个元素代表什么,什么时候进入,什么时候离开,是否可能重复进入。

C++ 追问重点是实现边界和成本。示例中可以用指针节点,但比较节点身份时比较指针。极端深树用 `std::vector<TreeNode*>` 显式栈或队列。最大路径类题要用足够小的初值,不能默认零。此外要说明是否会修改输入,是否需要保留原下标,是否存在整数溢出,是否有迭代器失效,是否使用了可能插入默认值的 API。面试中能主动说出这些成本,通常比只写短代码更有说服力。

变体迁移是二刷的核心。普通二叉树不能用值域剪枝,需要父指针或后序状态。追问通常会问返回值语义、空节点初值、全负数和极端深树。请准备把递归思想改写成显式栈或队列的口头方案。最后给自己留一个实验:把正确实现中的关键条件反过来或删除掉,观察哪个测试首先失败。能解释失败原因,才说明你真正掌握了这道题。

236 二叉树最近公共祖先。二刷这题时,不要满足于能写出代码,而要能在三分钟内讲出完整推理。第一句话从问题出发:普通树没有有序性,只能依靠祖先关系。第二句话交代暴力基线:按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索:父指针集合或后序返回目标命中状态都可行。如果这三句话说不出来,说明你记住的是答案形状,不是题目结构。

暴力基线要讲具体:枚举哪些对象,检查函数做了什么,最坏输入如何把它拖慢。接着定位浪费:是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态,还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责:保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题,边界风险集中在:目标不存在、p 等于 q、一个是另一个祖先、比较节点地址。请至少准备一个能打破错误实现的用例,并解释错误发生在哪个不变量上。

正确性说明要避免空话。正确性通常是对子树归纳。左右子树已经给出稳定答案，父节点只需按题目约束合并这两个答案。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。示例中可以用指针节点，但比较节点身份时比较指针。极端深树用 `std::vector<TreeNode*>` 显式栈或队列。最大路径类题要用足够小的初值，不能默认零。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。多次 LCA 查询可预处理倍增祖先表。追问通常会问返回值语义、空节点初值、全负数和极端深树。请准备把递归思想改写成显式栈或队列的口头方案。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

238 除自身以外数组的乘积。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：不能用除法时，答案由左侧乘积和右侧乘积组成。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：先写入左侧前缀积，再从右向左乘右侧后缀积。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：一个零、多个零、负数、乘积溢出。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。正确性通常用区间不变量证明。每次循环只扩大已经正确的区域，未知区严格缩小，且被移出的元素已经放到符合语义的位置。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。C++ 实现优先使用 `std::vector<int>` 和明确的整型下标。涉及乘积、面积、总和时要主动判断是否需要 `long long`。如果题目要求原地修改，要说明返回值表示新长度还是数组本身，避免把输出语义和容器容量混在一起。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。这个题展示输出数组可作为状态存储的一部分。追问通常会落在原地修改、稳定顺序、输出规模和整数溢出上。请准备说明为什么保存下标比保存指针更稳，为什么某个位置一旦写入就不会再被未来元素破坏。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

239 滑动窗口最大值。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：每个窗口最大值来自仍未过期且没有被新元素支配的候选。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：单调队列保存下标，队尾小于等于新值的候选被弹出。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：k 为一、k 等于数组长度、重复最大值、队首过期。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。每个元素最多入栈一次、出栈一次，所以时间线性。正确性来自弹出时机：一旦遇到破坏单调性的元素，栈顶元素的答案已经确定。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。栈里通常存下标而不是值，因为要计算距离、宽度和过期。可以用 `std::vector<int>` 当栈，便于查看顶部和减少适配器限制。此外要说明是否会修改输入，

是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。受限子序列和使用同样队列维护最近 k 个 DP 最大值。追问通常会问栈里为什么存下标、相等元素如何处理、弹出时到底结算了谁。请准备用一个严格递增和一个严格递减样例解释入栈出栈次数。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

240 搜索二维矩阵 II。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：右上角同时具有向左变小、向下变大的排除能力。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：当前值大于目标就左移，小于目标就下移。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：空矩阵、单行、单列、目标在角落、重复值。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。证明二分不是证明循环会停，而是证明被排除的一半确实不含答案。答案空间二分还要证明可行性谓词单调。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。中点写成 $\text{left} + (\text{right} - \text{left}) / 2$ 。判断平方、乘法、总量时避免溢出。用 `std::lower_bound` 能表达清楚时优先用它，但面试要能手写同等语义。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。从左上角无法一次排除一行或一列。追问通常会要求你写出单调谓词和答案边界。请准备说明 left、right 的语义，为什么 mid 被排除或保留，以及返回值是否还需要二次验证。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

279 完全平方数。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：完全平方数可以看成无限使用的物品，目标是凑出 n 的最少数目。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：完全背包最少物品数，或把数字看成图节点做 BFS。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在： n 为零或一、完全平方数本身、较大 n 。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。正确性来自选择当前物品的次数枚举。0-1 只有选与不选，完全背包允许从当前轮状态继续转移。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。容量通常与数值大小有关，所以复杂度是伪多项式。实现时先判断总和奇偶、目标范围和负数是否存在。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。数学四平方定理可给更强解法，但面试 DP 更通用。追问通常会问倒序和正序的区别、组合和排列的区别、为什么复杂度是伪多项式。请准备一个循环方向写反的最小反例。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

283 移动零。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：稳定保留非零元素顺序，把零集中到尾部。第二句话交代暴力基线：按题意完整枚

举或模拟会覆盖哪些候选。第三句话才讲优化线索：慢指针写入下一个非零位置，最后补零或用交换。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：全零、无零、零在开头结尾、稳定顺序要求。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。正确性通常用区间不变量证明。每次循环只扩大已经正确的区域，未知区严格缩小，且被移出的元素已经放到符合语义的位置。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。C++ 实现优先使用 `std::vector<int>` 和明确的整型下标。涉及乘积、面积、总和时要主动判断是否需要 `long long`。如果题目要求原地修改，要说明返回值表示新长度还是数组本身，避免把输出语义和容器容量混在一起。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。若目标是移动指定值，模型与移除元素相同。追问通常会落在原地修改、稳定顺序、输出规模和整数溢出上。请准备说明为什么保存下标比保存指针更稳，为什么某个位置一旦写入就不会再被未来元素破坏。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

287 寻找重复数。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：数组值域使下标到值形成函数图，重复数是环入口。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：快慢指针不修改数组且常数空间；值域二分用抽屉原理。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：重复数出现多次、不能排序、不能改数组、长度为 n 加一。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。正确性来自二进制位独立性。异或按位满足交换结合和自反抵消；掩码 DP 则是对子集大小做归纳。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。使用无符号整数能减少移位歧义。位数超过 31 时用 `long long` 或 `std::bitset`。表达式加括号，避免位运算优先级误读。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。快慢指针要求值都能作为合法下标。追问通常会问每一位代表什么、负数和移位是否安全、状态相同但资源不同能否合并。请准备说明位运算只是编码，真正关键仍是状态语义。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

295 数据流的中位数。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：数据流需要动态维护较小一半和较大一半。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：大顶堆保存较小一半，小顶堆保存较大一半，大小差不超过一。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：偶数个元素、负数、重复值、平均值溢出。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。正确性常用边界候选证明：被堆丢弃的元素不可能进入答案，仍在堆中的元素覆盖所有可能的下一步选择。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。C++ 的 `std::priority_queue` 默认大顶堆。小顶堆用 `std::greater<T>`；结构体比较器要按谁优先级更低来写。堆中保存大对象时尽量保存下标或指针。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。若还要删除任意元素，需要延迟删除哈希表。追问通常会问为什么不完整排序、堆顶是否可能过期、比较器方向是否写反。请准备说明堆里元素的业务含义，而不是只说 `priority_queue`。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

300 最长递增子序列。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：二次 DP 固定结尾，二分优化维护每个长度的最小结尾。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：tails 不是实际答案序列，而是未来扩展潜力表。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：重复元素、严格递增和非递减区别、空数组。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。正确性通常从最优子结构证明：任意最优解的最后一步必然落入枚举集合；每个枚举候选又由已经正确的子问题构成。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。数组维度和初始化要服务于状态含义。不可达用大正数或负数哨兵时要避免溢出。方案数可能超过 int 时改用 `long long` 或按题目取模。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。若要还原路径，需要额外前驱数组，不能只看 tails。追问通常会问状态是否足够、初始化为什么这样、空间压缩读到的是旧值还是新值。请准备从暴力搜索树指出重复子问题。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

309 最佳买卖股票含冷冻期。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：冷冻期让买入来源受昨天卖出状态限制。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：状态机维护持有、刚卖出、休息三种日终状态。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：空价格、一天、连续上涨、卖出后不能次日买入。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。正确性通常从最优子结构证明：任意最优解的最后一步必然落入枚举集合；每个枚举候选又由已经正确的子问题构成。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。数组维度和初始化要服务于状态含义。不可达用大正数或负数哨兵时要避免溢出。方案数可能超过 int 时改用 `long long` 或按题目取模。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认

值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。手续费版本可以在买入或卖出转移中扣费。追问通常会问状态是否足够、初始化为什么这样、空间压缩读到的是旧值还是新值。请准备从暴力搜索树指出重复子问题。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

322 零钱兑换。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：金额是容量，硬币可无限使用，目标是最少硬币数。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：dp[value] 从 value 减 coin 的已知状态转移。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：无法凑出、amount 为零、硬币面额大于目标、哨兵溢出。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。正确性来自选择当前物品的次数枚举。0-1 只有选与不选，完全背包允许从当前轮状态继续转移。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。容量通常与数值大小有关，所以复杂度是伪多项式。实现时先判断总和奇偶、目标范围和负数是否存在。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。零钱兑换 II 问组合数，循环语义不同。追问通常会问倒序和正序的区别、组合和排列的区别、为什么复杂度是伪多项式。请准备一个循环方向写反的最小反例。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

337 打家劫舍 III。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：树上相邻节点不能同时选，每个节点需要偷和不偷两个状态。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：后序汇总：偷当前则孩子不能偷，不偷当前则孩子可偷可不偷。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：空树、负值是否可能、链式树、递归深度。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。正确性通常是对子树归纳。左右子树已经给出稳定答案，父节点只需按题目约束合并这两个答案。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。示例中可以用指针节点，但比较节点身份时比较指针。极端深树用 `std::vector<TreeNode*>` 显式栈或队列。最大路径类题要用足够小的初值，不能默认零。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。可用显式栈后序保存节点状态，避免调用栈。追问通常会问返回值语义、空节点初值、全负数和极端深树。请准备把递归思想改写成显式栈或队列的口头方案。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

347 前 K 个高频元素。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：先统计频次，再只维护前 K 个候选。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：小顶堆大小超过 K 就弹出最低频；桶排序

利用频次上界。如果这三句话说不出来,说明你记住的是答案形状,不是题目结构。

暴力基线要讲具体:枚举哪些对象,检查函数做了什么,最坏输入如何把它拖慢。接着定位浪费:是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态,还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责:保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题,边界风险集中在:K 等于不同元素数、频次相同、结果顺序不要求。请至少准备一个能打破错误实现的用例,并解释错误发生在哪个不变量上。

正确性说明要避免空话。正确性常用边界候选证明:被堆丢弃的元素不可能进入答案,仍在堆中的元素覆盖所有可能的下一步选择。二刷时可以把证明压成两段:第一段证明所有合法答案都会被覆盖,第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表,还要补一句容器中每个元素代表什么,什么时候进入,什么时候离开,是否可能重复进入。

C++ 追问重点是实现边界和成本。C++ 的 `std::priority_queue` 默认大顶堆。小顶堆用 `std::greater<T>`; 结构体比较器要按优先级更低来写。堆中保存大对象时尽量保存下标或指针。此外要说明是否会修改输入,是否需要保留原下标,是否存在整数溢出,是否有迭代器失效,是否使用了可能插入默认值的 API。面试中能主动说出这些成本,通常比只写短代码更有说服力。

变体迁移是二刷的核心。若要按频次和字典序排序,比较器要同时处理两个键。追问通常会问为什么不完整排序、堆顶是否可能过期、比较器方向是否写反。请准备说明堆里元素的业务含义,而不是只说 `priority_queue`。最后给自己留一个实验:把正确实现中的关键条件反过来或删除掉,观察哪个测试首先失败。能解释失败原因,才说明你真正掌握了这道题。

394 字符串解码。二刷这题时,不要满足于能写出代码,而要能在三分钟内讲出完整推理。第一句话从问题出发:嵌套结构需要保存上一层字符串和重复次数。第二句话交代暴力基线:按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索:遇到左括号入栈当前状态,右括号弹出并拼接展开。如果这三句话说不出来,说明你记住的是答案形状,不是题目结构。

暴力基线要讲具体:枚举哪些对象,检查函数做了什么,最坏输入如何把它拖慢。接着定位浪费:是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态,还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责:保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题,边界风险集中在:多位数字、嵌套、空片段、数字后必须跟括号。请至少准备一个能打破错误实现的用例,并解释错误发生在哪个不变量上。

正确性说明要避免空话。每个元素最多入栈一次、出栈一次,所以时间线性。正确性来自弹出时机:一旦遇到破坏单调性的元素,栈顶元素的答案已经确定。二刷时可以把证明压成两段:第一段证明所有合法答案都会被覆盖,第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表,还要补一句容器中每个元素代表什么,什么时候进入,什么时候离开,是否可能重复进入。

C++ 追问重点是实现边界和成本。栈里通常存下标而不是值,因为要计算距离、宽度和过期。可以用 `std::vector<int>` 当栈,便于查看顶部和减少适配器限制。此外要说明是否会修改输入,是否需要保留原下标,是否存在整数溢出,是否有迭代器失效,是否使用了可能插入默认值的 API。面试中能主动说出这些成本,通常比只写短代码更有说服力。

变体迁移是二刷的核心。递归下降解析更通用,但迭代栈更符合工程栈控制。追问通常会问栈里为什么存下标、相等元素如何处理、弹出时到底结算了谁。请准备用一个严格递增和一个严格递减样例解释入栈出栈次数。最后给自己留一个实验:把正确实现中的关键条件反过来或删除掉,观察哪个测试首先失败。能解释失败原因,才说明你真正掌握了这道题。

416 分割等和子集。二刷这题时,不要满足于能写出代码,而要能在三分钟内讲出完整推理。第一句话从问题出发:总和一半是背包容量,每个数只能使用一次。第二句话交代暴力基线:按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索:0-1 背包可达性,一维容量必须倒序。如果这三句话说不出来,说明你记住的是答案形状,不是题目结构。

暴力基线要讲具体:枚举哪些对象,检查函数做了什么,最坏输入如何把它拖慢。接着定位浪费:是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态,还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责:保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题,边界风险集中在:总和为奇数、目标为零、数值较大、复杂度依赖 `target`。请至少准备一个能打破错误实现的用例,并解释错误发生在哪个不变量上。

正确性说明要避免空话。正确性来自选择当前物品的次数枚举。0-1 只有选与不选,完全背包允许从当前轮状态继续转移。二刷时可以把证明压成两段:第一段证明所有合法答案都会被覆盖,第

二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。容量通常与数值大小有关，所以复杂度是伪多项式。实现时先判断总和奇偶、目标范围和负数是否存在。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。负数会破坏普通容量模型，需要重新建模。追问通常会问倒序和正序的区别、组合和排列的区别、为什么复杂度是伪多项式。请准备一个循环方向写反的最小反例。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

417 太平洋大西洋水流问题。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：从每个格子向海搜索很慢，反向从海边沿非递减高度搜索。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：分别标记能到太平洋和大西洋的格子，交集就是答案。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：单行单列、平台高度、边界重复入队、方向条件反向。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。BFS 正确性来自层次扩展，第一次到达某状态就是最短距离。连通块算法正确性来自一次搜索会标记与起点可达的全部节点且不会越过非法边。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。图节点连续时用 `std::vector<std::vector<int>>`；节点是字符串时先编号或用哈希表。网格方向数组用 `constexpr std::array`，边界判断集中写。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。反向搜索是很多可达性题的重要技巧。追问通常会问节点和边的建模、访问标记时机、BFS 为什么最短。请准备说明状态是否还包含资源，为什么不能只按位置去重。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

435 无重叠区间。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：保留最多不重叠区间等价于删除最少区间。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：按终点升序选择，结束越早给后续留下空间越多。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：端点相接是否重叠、完全覆盖、相同终点、空列表。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。常见证明有交换论证和领先性论证。交换论证说明最优解可以被替换成贪心第一步而不变差；领先性论证说明贪心每一步都至少和任何方案一样好。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。实现时先把比较器写成明确的业务顺序。区问题注意起点相同时终点升序还是降序；覆盖题注意闭区间和半开区间的等号。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。用交换论证说明最早结束的选择劣于其他第一选择。追问通常会要求证明。请准备交换论证或领先性论证，并给出一个看似合理但错误的贪心规则反例。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

438 找到字符串中所有字母异位词。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：固定长度窗口内字符频次等于目标频次即为答案。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：右进左出维护计数，窗口长度达到目标长度后检查差异。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：目标比源串长、重复字符、字符集固定、答案重叠。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。证明重点是排除。某个左端被移过之后，所有以它为左端的更长窗口都不可能比当前答案更优，或者已经被当前状态完整统计。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。C++ 中建议把窗口写成左闭右开或左闭右闭中的一种，并在全题坚持。字符计数用 `std::array<int, 128>` 或 26 长度数组时要说明字符集假设；泛化字符集再换哈希表。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。维护差异计数可以避免每次比较整个数组。追问通常会问窗口为什么不会漏掉左端、为什么有负数会失效、固定窗口和可变窗口有什么区别。请准备一个反例证明错误窗口条件会漏解。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

450 删除二叉搜索树中的节点。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：删除节点后仍要保持 BST 中序有序。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：若有两个孩子，用后继或前驱替换当前值，再删除那个后继节点。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：删除叶子、一个孩子、两个孩子、删除根节点。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。正确性通常是对子树归纳。左右子树已经给出稳定答案，父节点只需按题目约束合并这两个答案。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。示例中可以用指针节点，但比较节点身份时比较指针。极端深树用 `std::vector<TreeNode*>` 显式栈或队列。最大路径类题要用足够小的初值，不能默认零。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。工程实现要清楚是移动值还是重连节点。追问通常会问返回值语义、空节点初值、全负数和极端深树。请准备把递归思想改写成显式栈或队列的口头方案。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

452 用最少数量的箭引爆气球。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：一支箭的位置可以覆盖所有包含该位置的区间。第二句话交代暴

力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：按右端点排序，当前箭放在最早结束气球的右端。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：端点相同、负坐标、区间包含、闭区间边界。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避开空话。常见证明有交换论证和领先性论证。交换论证说明最优解可以被替换成贪心第一步而不变差；领先性论证说明贪心每一步都至少和任何方案一样好。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。实现时先把比较器写成明确的业务顺序。区问题注意起点相同时终点升序还是降序；覆盖题注意闭区间和半开区间的等号。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。它和无重叠区间是同一类最早结束贪心。追问通常会要求证明。请准备交换论证或领先性论证，并给出一个看似合理但错误的贪心规则反例。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

494 目标和。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：正负号选择可代数转换为选若干数凑出特定正和。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：若 S 加 target 为奇数或目标绝对值过大，直接无解。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：零元素会让方案数翻倍、目标为负、容量为零。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避开空话。正确性来自选择当前物品的次数枚举。0-1 只有选与不选，完全背包允许从当前轮状态继续转移。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。容量通常与数值大小有关，所以复杂度是伪多项式。实现时先判断总和奇偶、目标范围和负数是否存在。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。也可用哈希 DP 保存所有可能和，但背包转换更高效。追问通常会问倒序和正序的区别、组合和排列的区别、为什么复杂度是伪多项式。请准备一个循环方向写反的最小反例。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

496 下一个更大元素 I。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：第二个数组提供每个值右侧第一个更大值的全局关系。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：单调栈预处理值到下一个更大值映射，再回答第一个数组查询。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：不存在更大值、递减数组、值唯一假设。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。每个元素最多入栈一次、出栈一次，所以时间线性。正确性来自弹出时机：一旦遇到破坏单调性的元素，栈顶元素的答案已经确定。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。栈里通常存下标而不是值，因为要计算距离、宽度和过期。可以用 `std::vector<int>` 当栈，便于查看顶部和减少适配器限制。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。若值不唯一，应按下标而不是值建映射。追问通常会问栈里为什么存下标、相等元素如何处理、弹出时到底结算了谁。请准备用一个严格递增和一个严格递减样例解释入栈出栈次数。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

503 下一个更大元素 II。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：环形数组可以通过遍历两倍下标模拟绕回。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：第一遍入栈，第二遍只帮助未解决下标找到答案。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：全递减、全相等、长度一、取模下标。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。每个元素最多入栈一次、出栈一次，所以时间线性。正确性来自弹出时机：一旦遇到破坏单调性的元素，栈顶元素的答案已经确定。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。栈里通常存下标而不是值，因为要计算距离、宽度和过期。可以用 `std::vector<int>` 当栈，便于查看顶部和减少适配器限制。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。不要在第二遍重复入栈，否则可能死循环或重复计算。追问通常会问栈里为什么存下标、相等元素如何处理、弹出时到底结算了谁。请准备用一个严格递增和一个严格递减样例解释入栈出栈次数。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

518 零钱兑换 II。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：硬币无限使用，问组合数而不是最少数目。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：外层遍历硬币、内层金额正序，保证组合按固定硬币顺序统计。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：amount 为零、无硬币、组合数较大、循环顺序反例。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。正确性来自选择当前物品的次数枚举。0-1 只有选与不选，完全背包允许从当前轮状态继续转移。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。容量通常与数值大小有关，所以复杂度是伪多项式。实现时先判断总和奇偶、目标范围和负数是否存在。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些

成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。若外层金额内层硬币，会统计排列数。追问通常会问倒序和正序的区别、组合和排列的区别、为什么复杂度是伪多项式。请准备一个循环方向写反的最小反例。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

543 二叉树直径。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：直径经过某节点时等于左高加右高，但向父亲只能贡献单边高度。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：后序遍历同时更新全局直径并返回高度。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：空树、单节点、链式树、边数还是节点数定义。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。正确性通常是对子树归纳。左右子树已经给出稳定答案，父节点只需按题目约束合并这两个答案。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。示例中可以用指针节点，但比较节点身份时比较指针。极端深树用 `std::vector<TreeNode*>` 显式栈或队列。最大路径类题要用足够小的初值，不能默认零。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。最大路径和是同型题，但贡献值和全局更新有负数处理。追问通常会问返回值语义、空节点初值、全负数和极端深树。请准备把递归思想改写成显式栈或队列的口头方案。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

547 省份数量。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：城市连接关系形成无向图，省份就是连通分量。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：并查集合并所有直接连接的城市，最后统计根数量。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：邻接矩阵对称、自连接、孤立城市、只扫描上三角。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。正确性来自等价关系维护：自反、对称、传递由集合代表保证。合并两个代表后，两个集合中任意元素的代表都会归到同一根。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。类成员访问使用 `this->`。迭代版 `find` 可以避免递归。若需要维护集合大小、边数、权值差或奇偶关系，要扩展额外数组并在合并时同步更新。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。BFS 也可做；并查集更突出动态合并。追问通常会问并查集能否回答路径长度、路径压缩是否改变集合语义、字符串如何编号。请准备说明合并失败为什么意味着环。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

560 和为 K 的子数组。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：当前前缀和减去目标值，就是需要匹配的历史前缀。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：哈希表保存前缀和出现次

数，先查询再更新。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：负数、目标为零、从下标零开始的区间、重复前缀。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。正确性来自代数等价。若两个前缀满足差值、同余或计数差要求，则它们之间的区间正好满足题意；反过来任意合法区间也对应这样一对前缀。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。实现时用 `find` 判断是否存在，用 `operator[]` 只在确认要插入或累加时使用。总和可能溢出时使用 `long long`；模运算要处理负数余数归一化。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。若要求最长区间，哈希表应保存最早位置而不是次数。追问通常会问先查询还是先更新、哈希表保存次数还是最早下标、为什么负数不影响前缀和。请准备说明从任意合法区间如何反推到两个前缀状态。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

581 最短无序连续子数组。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：要找破坏全局有序性的最左和最右位置。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：从左维护历史最大值确定右边界，从右维护历史最小值确定左边界。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：已经有序、逆序、重复值、局部乱序。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。正确性通常用区间不变量证明。每次循环只扩大已经正确的区域，未知区严格缩小，且被移出的元素已经放到符合语义的位置。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。C++ 实现优先使用 `std::vector<int>` 和明确的整型下标。涉及乘积、面积、总和时要主动判断是否需要 `long long`。如果题目要求原地修改，要说明返回值表示新长度还是数组本身，避免把输出语义和容器容量混在一起。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。排序比较法更直观但复杂度更高。追问通常会落在原地修改、稳定顺序、输出规模和整数溢出上。请准备说明为什么保存下标比保存指针更稳，为什么某个位置一旦写入就不会再被未来元素破坏。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

621 任务调度器。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：相同任务之间有冷却间隔，核心受最高频任务骨架限制。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：可用大顶堆模拟，也可用最高频公式计算最短时间。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：冷却为零、多个最高频、空闲被其他任务填满。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。正确性常用边界候选证明：被堆丢弃的元素不可能进入答案，仍在堆中的元素覆盖所有可能的下一步选择。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。C++ 的 `std::priority_queue` 默认大顶堆。小顶堆用 `std::greater<T>`；结构体比较器要按谁优先级更低来写。堆中保存大对象时尽量保存下标或指针。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。若任务有不同释放时间或执行时长，就变成更一般的调度问题。追问通常会问为什么不完整排序、堆顶是否可能过期、比较器方向是否写反。请准备说明堆里元素的业务含义，而不是只说 `priority_queue`。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

647 回文子串。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：每个回文都有唯一中心或中心间隙。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：对每个中心向两边扩展，扩展成功一次就计数一次。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：空串、全相同字符、偶数回文、奇数回文。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。正确性通常用区间不变量证明。每次循环只扩大已经正确的区域，未知区严格缩小，且被移出的元素已经放到符合语义的位置。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。C++ 实现优先使用 `std::vector<int>` 和明确的整型下标。涉及乘积、面积、总和时要主动判断是否需要 `long long`。如果题目要求原地修改，要说明返回值表示新长度还是数组本身，避免把输出语义和容器容量混在一起。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。Manacher 算法可线性解决，但中心扩展更适合面试基础。追问通常会落在原地修改、稳定顺序、输出规模和整数溢出上。请准备说明为什么保存下标比保存指针更稳，为什么某个位置一旦写入就不会再被未来元素破坏。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

684 冗余连接。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：树加一条边会形成唯一环。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：按顺序加边，若一条边两端已连通，则它就是冗余边。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：节点编号、最后一条成环边、重复边、并查集初始化大小。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。正确性来自等价关系维护：自反、对称、传递由集合代表保证。合并两个代表后，两个集合中任意元素的代表都会归到同一根。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。类成员访问使用 `this->`。迭代版 `find` 可以避免递归。若需要维护集合大小、边数、权值差或奇偶关系，要扩展额外数组并在合并时同步更新。此外要说明

是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。有向冗余连接需要同时处理入度为二和环，复杂度更高。追问通常会问并查集能否回答路径长度、路径压缩是否改变集合语义、字符串如何编号。请准备说明合并失败为什么意味着环。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

695 岛屿的最大面积。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：面积就是一个连通块中陆地格子的数量。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：每发现新陆地就搜索完整连通块并计数，取最大值。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：全水、全陆、细长岛、是否允许修改输入。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。BFS 正确性来自层次扩展，第一次到达某状态就是最短距离。连通块算法正确性来自一次搜索会标记与起点可达的全部节点且不会越过非法边。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。图节点连续时用 `std::vector<std::vector<int>>`；节点是字符串时先编号或用哈希表。网格方向数组用 `constexpr std::array`，边界判断集中写。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。与岛屿数量相比，只是连通块聚合值不同。追问通常会问节点和边的建模、访问标记时机、BFS 为什么最短。请准备说明状态是否还包含资源，为什么不能只按位置去重。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

704 二分查找。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：基础题的重点是固定区间语义。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：左闭右开写法能统一 lower bound 和存在性检查。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：空数组、长度一、目标在边界、目标不存在。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。证明二分不是证明循环会停，而是证明被排除的一半确实不含答案。答案空间二分还要证明可行性谓词单调。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。中点写成 `left + (right - left) / 2`。判断平方、乘法、总量时避免溢出。用 `std::lower_bound` 能表达清楚时优先用它，但面试要能手写同等语义。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。所有复杂二分都应该能退回到这道题的边界不变量。追问通常会要求你写出单调谓词和答案边界。请准备说明 left、right 的语义，为什么 mid 被排除或保留，以及返回值是否还需要二次验证。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

714 买卖股票含手续费。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：手续费改变交易收益，仍可用持有和不持有状态。第二句话交代暴力基

线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：卖出时扣费或买入时扣费都可以，但不要重复扣。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：手续费为零、价格震荡、小利润不足手续费。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。正确性通常从最优子结构证明：任意最优解的最后一步必然落入枚举集合；每个枚举候选又由已经正确的子问题构成。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。数组维度和初始化要服务于状态含义。不可达用大正数或负数哨兵时要避免溢出。方案数可能超过 `int` 时改用 `long long` 或按题目取模。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。这是状态机比局部贪心更稳的例子。追问通常会问状态是否足够、初始化为什么这样、空间压缩读到的是旧值还是新值。请准备从暴力搜索树指出重复子问题。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

739 每日温度。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：每一天等待右侧第一个更高温度。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：单调递减栈保存尚未找到答案的下标。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：没有更高温度、相等温度、递增、递减。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。每个元素最多入栈一次、出栈一次，所以时间线性。正确性来自弹出时机：一旦遇到破坏单调性的元素，栈顶元素的答案已经确定。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。栈里通常存下标而不是值，因为要计算距离、宽度和过期。可以用 `std::vector<int>` 当栈，便于查看顶部和减少适配器限制。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。下一个更大元素与它同型，只是输出值或距离不同。追问通常会问栈里为什么存下标、相等元素如何处理、弹出时到底结算了谁。请准备用一个严格递增和一个严格递减样例解释入栈出栈次数。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

743 网络延迟时间。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：有向正权图从起点到所有节点的最短路最大值。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：Dijkstra 求每个节点最短距离，最后取最大，若有无穷则不可达。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：节点编号从一开始、不可达、重边、过期堆元素。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。非负边保证从当前最小距离节点再走出去不会得到更短路径回头改写它。二分可达性则要证明阈值越大,可用边只会增加。二刷时可以把证明压成两段:第一段证明所有合法答案都会被覆盖,第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表,还要补一句容器中每个元素代表什么,什么时候进入,什么时候离开,是否可能重复进入。

C++ 追问重点是实现边界和成本。由于 `priority_queue` 没有 `decrease-key`,更新距离时压入新状态,弹出时比较距离数组。距离用 `long long` 更稳,图用邻接表保存 (`to, weight`)。此外要说明是否会修改输入,是否需要保留原下标,是否存在整数溢出,是否有迭代器失效,是否使用了可能插入默认值的 API。面试中能主动说出这些成本,通常比只写短代码更有说服力。

变体迁移是二刷的核心。边权相同才可用普通 BFS。追问通常会问为什么普通 BFS 不行、Dijkstra 的非负边条件、堆中旧状态如何处理。请准备一个不同边权反例。最后给自己留一个实验:把正确实现中的关键条件反过来或删除掉,观察哪个测试首先失败。能解释失败原因,才说明你真正掌握了这道题。

752 打开转盘锁。二刷这题时,不要满足于能写出代码,而要能在三分钟内讲出完整推理。第一句话从问题出发:四位密码是状态,每次转动一位形成边。第二句话交代暴力基线:按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索:BFS 从起点扩展,死亡状态不能入队,访问状态避免重复。如果这三句话说不出来,说明你记住的是答案形状,不是题目结构。

暴力基线要讲具体:枚举哪些对象,检查函数做了什么,最坏输入如何把它拖慢。接着定位浪费:是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态,还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责:保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题,边界风险集中在:起点死亡、目标就是起点、数字环绕、死亡集合很大。请至少准备一个能打破错误实现的用例,并解释错误发生在哪个不变量上。

正确性说明要避免空话。BFS 正确性来自层次扩展,第一次到达某状态就是最短距离。连通块算法正确性来自一次搜索会标记与起点可达的全部节点且不会越过非法边。二刷时可以把证明压成两段:第一段证明所有合法答案都会被覆盖,第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表,还要补一句容器中每个元素代表什么,什么时候进入,什么时候离开,是否可能重复进入。

C++ 追问重点是实现边界和成本。图节点连续时用 `std::vector<std::vector<int>>`;节点是字符串时先编号或用哈希表。网格方向数组用 `constexpr std::array`,边界判断集中写。此外要说明是否会修改输入,是否需要保留原下标,是否存在整数溢出,是否有迭代器失效,是否使用了可能插入默认值的 API。面试中能主动说出这些成本,通常比只写短代码更有说服力。

变体迁移是二刷的核心。双向 BFS 能明显减少状态展开。追问通常会问节点和边的建模、访问标记时机、BFS 为什么最短。请准备说明状态是否还包含资源,为什么不能只按位置去重。最后给自己留一个实验:把正确实现中的关键条件反过来或删除掉,观察哪个测试首先失败。能解释失败原因,才说明你真正掌握了这道题。

763 划分字母区间。二刷这题时,不要满足于能写出代码,而要能在三分钟内讲出完整推理。第一句话从问题出发:每个片段必须覆盖其中所有字符的最后出现位置。第二句话交代暴力基线:按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索:扫描维护当前片段最远右端,到达右端即可切分。如果这三句话说不出来,说明你记住的是答案形状,不是题目结构。

暴力基线要讲具体:枚举哪些对象,检查函数做了什么,最坏输入如何把它拖慢。接着定位浪费:是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态,还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责:保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题,边界风险集中在:单字符、所有字符相同、字符多次跨段。请至少准备一个能打破错误实现的用例,并解释错误发生在哪个不变量上。

正确性说明要避免空话。常见证明有交换论证和领先性论证。交换论证说明最优解可以被替换成贪心第一步而不变差;领先性论证说明贪心每一步都至少和任何方案一样好。二刷时可以把证明压成两段:第一段证明所有合法答案都会被覆盖,第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表,还要补一句容器中每个元素代表什么,什么时候进入,什么时候离开,是否可能重复进入。

C++ 追问重点是实现边界和成本。实现时先把比较器写成明确的业务顺序。区间题注意起点相同时终点升序还是降序;覆盖题注意闭区间和半开区间的等号。此外要说明是否会修改输入,是否

需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。这是最早合法切分贪心，能最大化片段数量。追问通常会要求证明。请准备交换论证或领先性论证，并给出一个看似合理但错误的贪心规则反例。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

875 爱吃香蕉的珂珂。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：速度越大，总耗时越小，答案具有单调可行性。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：二分速度，检查函数用向上取整累加小时数。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：速度下界、堆很大、 h 等于堆数、乘法加法溢出。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。证明二分不是证明循环会停，而是证明被排除的一半确实不含答案。答案空间二分还要证明可行性谓词单调。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。中点写成 `left + (right - left) / 2`。判断平方、乘法、总量时避免溢出。用 `std::lower_bound` 能表达清楚时优先用它，但面试要能手写同等语义。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。船运包裹和分割数组都是同类最小可行上限。追问通常会要求你写出单调谓词和答案边界。请准备说明 `left`、`right` 的语义，为什么 `mid` 被排除或保留，以及返回值是否还需要二次验证。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

973 最接近原点的 K 个点。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：距离只需比较平方，避免开方误差和成本。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：大小为 K 的大顶堆保存当前最近 K 个点中最远者。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在： K 为一、 K 为全部、距离相同、坐标平方溢出。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。正确性常用边界候选证明：被堆丢弃的元素不可能进入答案，仍在堆中的元素覆盖所有可能的下一步选择。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。C++ 的 `std::priority_queue` 默认大顶堆。小顶堆用 `std::greater<T>`；结构体比较器要按谁优先级更低来写。堆中保存大对象时尽量保存下标或指针。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。快速选择可平均线性，但堆更适合数据流。追问通常会问为什么不完整排序、堆顶是否可能过期、比较器方向是否写反。请准备说明堆里元素的业务含义，而不是只说 `priority_queue`。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

994 腐烂的橘子。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：所有初始腐烂橘子同时作为 BFS 源点。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：按层扩散，分钟数等于最后一个新鲜橘子被首

次访问的层数。如果这三句话说不出来,说明你记住的是答案形状,不是题目结构。

暴力基线要讲具体:枚举哪些对象,检查函数做了什么,最坏输入如何把它拖慢。接着定位浪费:是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态,还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责:保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题,边界风险集中在:没有新鲜橘子、没有腐烂源、隔离新鲜橘子、空格阻断。请至少准备一个能打破错误实现的用例,并解释错误发生在哪个不变量上。

正确性说明要避免空话。BFS 正确性来自层次扩展,第一次到达某状态就是最短距离。连通块算法正确性来自一次搜索会标记与起点可达的全部节点且不会越过非法边。二刷时可以把证明压成两段:第一段证明所有合法答案都会被覆盖,第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表,还要补一句容器中每个元素代表什么,什么时候进入,什么时候离开,是否可能重复进入。

C++ 追问重点是实现边界和成本。图节点连续时用 `std::vector<std::vector<int>>`; 节点是字符串时先编号或用哈希表。网格方向数组用 `constexpr std::array`, 边界判断集中写。此外要说明是否会修改输入,是否需要保留原下标,是否存在整数溢出,是否有迭代器失效,是否使用了可能插入默认值的 API。面试中能主动说出这些成本,通常比只写短代码更有说服力。

变体迁移是二刷的核心。多源 BFS 也适用于最近零距离、地图扩散等题。追问通常会问节点和边的建模、访问标记时机、BFS 为什么最短。请准备说明状态是否还包含资源,为什么不能只按位置去重。最后给自己留一个实验:把正确实现中的关键条件反过来或删除掉,观察哪个测试首先失败。能解释失败原因,才说明你真正掌握了这道题。

1011 在 D 天内送达包裹。二刷这题时,不要满足于能写出代码,而要能在三分钟内讲出完整推理。第一句话从问题出发:容量越大,需要天数越少,存在最小可行容量。第二句话交代暴力基线:按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索:下界是最大包裹,上界是总重量,检查函数按顺序贪心装船。如果这三句话说不出来,说明你记住的是答案形状,不是题目结构。

暴力基线要讲具体:枚举哪些对象,检查函数做了什么,最坏输入如何把它拖慢。接着定位浪费:是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态,还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责:保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题,边界风险集中在: D 为一、D 等于包裹数、单个包裹超过猜测容量。请至少准备一个能打破错误实现的用例,并解释错误发生在哪个不变量上。

正确性说明要避免空话。证明二分不是证明循环会停,而是证明被排除的一半确实不含答案。答案空间二分还要证明可行性谓词单调。二刷时可以把证明压成两段:第一段证明所有合法答案都会被覆盖,第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表,还要补一句容器中每个元素代表什么,什么时候进入,什么时候离开,是否可能重复进入。

C++ 追问重点是实现边界和成本。中点写成 `left + (right - left) / 2`。判断平方、乘法、总量时避免溢出。用 `std::lower_bound` 能表达清楚时优先用它,但面试要能手写同等语义。此外要说明是否会修改输入,是否需要保留原下标,是否存在整数溢出,是否有迭代器失效,是否使用了可能插入默认值的 API。面试中能主动说出这些成本,通常比只写短代码更有说服力。

变体迁移是二刷的核心。它和分割数组最大值都是连续分段最大和模型。追问通常会要求你写出单调谓词和答案边界。请准备说明 left、right 的语义,为什么 mid 被排除或保留,以及返回值是否还需要二次验证。最后给自己留一个实验:把正确实现中的关键条件反过来或删除掉,观察哪个测试首先失败。能解释失败原因,才说明你真正掌握了这道题。

1143 最长公共子序列。二刷这题时,不要满足于能写出代码,而要能在三分钟内讲出完整推理。第一句话从问题出发:状态表示两个前缀的最长公共子序列长度。第二句话交代暴力基线:按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索:末尾相同取左上加一,不同取上方和左方最大。如果这三句话说不出来,说明你记住的是答案形状,不是题目结构。

暴力基线要讲具体:枚举哪些对象,检查函数做了什么,最坏输入如何把它拖慢。接着定位浪费:是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态,还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责:保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题,边界风险集中在:空串、完全相同、无公共字符、子序列不是子串。请至少准备一个能打破错误实现的用例,并解释错误发生在哪个不变量上。

正确性说明要避免空话。正确性通常从最优子结构证明:任意最优解的最后一步必然落入枚举集合;每个枚举候选又由已经正确的子问题构成。二刷时可以把证明压成两段:第一段证明所有合

法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。数组维度和初始化要服务于状态含义。不可达用大正数或负数哨兵时要避免溢出。方案数可能超过 `int` 时改用 `long long` 或按题目取模。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。若要求还原序列，需要从表右下角反向追踪选择。追问通常会问状态是否足够、初始化为什么这样、空间压缩读到的是旧值还是新值。请准备从暴力搜索树指出重复子问题。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

1288 删除被覆盖区间。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：被覆盖要求另一区间起点不晚且终点不早。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：按起点升序、终点降序排序，扫描维护最大右端。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：同起点区间、完全覆盖、无覆盖、端点相等。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。排序保证所有可能与当前区间冲突的候选已经聚集在附近。贪心按最早结束选择，可以用交换论证证明不减少可选数量。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。比较器必须处理相同起点，否则覆盖和去重会错。区间端点可能溢出时用更大整数。闭区间和半开区间决定是否用小于等于。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。终点降序是关键，否则同起点小区间会先出现造成误判。追问通常会问排序键为什么这样、端点相等算不算重叠、比较器是否满足严格弱序。请准备说明排序后哪些候选可以被安全丢弃。最后给自己留一个实验：把正确实现中的关键条件反过来或删除掉，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

1631 最小体力消耗路径。二刷这题时，不要满足于能写出代码，而要能在三分钟内讲出完整推理。第一句话从问题出发：路径代价是沿途最大高度差，不是边权和。第二句话交代暴力基线：按题意完整枚举或模拟会覆盖哪些候选。第三句话才讲优化线索：Dijkstra 的松弛改成取当前代价和新边代价的最大值。如果这三句话说不出来，说明你记住的是答案形状，不是题目结构。

暴力基线要讲具体：枚举哪些对象，检查函数做了什么，最坏输入如何把它拖慢。接着定位浪费：是重复查询历史、重复扫描区间、重复比较候选、重复搜索同一状态，还是重复维护已经稳定的部分。然后把优化解释成一个数据结构职责：保存历史、维护边界、弹出过期候选、合并集合、记录前驱、缓存子问题或维护有序关系。对于本题，边界风险集中在：单格、平坦网格、必须绕路、多个相同代价。请至少准备一个能打破错误实现的用例，并解释错误发生在哪个不变量上。

正确性说明要避免空话。非负边保证从当前最小距离节点再走出去不会得到更短路径回头改写它。二分可达性则要证明阈值越大，可用边只会增加。二刷时可以把证明压成两段：第一段证明所有合法答案都会被覆盖，第二段证明被剪掉或丢弃的候选不可能成为更优答案。若使用排序、堆、哈希、队列或 DP 表，还要补一句容器中每个元素代表什么，什么时候进入，什么时候离开，是否可能重复进入。

C++ 追问重点是实现边界和成本。由于 `priority_queue` 没有 `decrease-key`，更新距离时压入新状态，弹出时比较距离数组。距离用 `long long` 更稳，图用邻接表保存 (`to, weight`)。此外要说明是否会修改输入，是否需要保留原下标，是否存在整数溢出，是否有迭代器失效，是否使用了可能插入默认值的 API。面试中能主动说出这些成本，通常比只写短代码更有说服力。

变体迁移是二刷的核心。也可二分体力上限后 BFS 检查可达。追问通常会问为什么普通 BFS 不行、Dijkstra 的非负边条件、堆中旧状态如何处理。请准备一个不同边权反例。最后给自己留一个实验：把正确实现中的关键条件反过来或删除，观察哪个测试首先失败。能解释失败原因，才说明你真正掌握了这道题。

二刷标准

二刷不是重新看一遍答案，而是把题目变成可讲、可证、可调试、可迁移的知识块。每道题至少回答：暴力瓶颈、优化依据、不变量、复杂度、边界、C++ 成本和一个变体。

术语表

暴力法 直接枚举解空间的方法，是理解题目和建立正确性的起点。

不变量 算法执行过程中始终成立的性质，用来证明优化解正确。

滑动窗口 用左右边界维护连续区间，并增量更新窗口状态的方法。

前缀和 把区间和查询转化为两个前缀差值的数据表示。

哈希表 通过哈希函数把 key 映射到桶，平均支持常数时间查询的数据结构。

BFS 广度优先搜索，适合无权最短路和层次扩展。

DFS 深度优先搜索，适合连通块、路径搜索和回溯。

拓扑排序 对有向无环图按依赖顺序排列节点的方法。

索引

algorithm, ii

data structure, ii

数据结构, ii

算法, ii